2027 EDITION

Subject: Machine Learning · Interview Prep

ML Interview Master Notebook

From first principles to Staff-level reasoning — fundamentals, math, classical ML, deep learning, coding, debugging, production, system design, and the modern GenAI stack: transformers, LLMs, alignment, RAG, agents, evaluation and efficient serving. Every idea explained with a real-life analogy.

Name: _____

derive · reason · debug · **design** · defend
— never just memorize.

50 PARTS + EXAM

How to use this notebook

This notebook is built around one idea: **interviewers do not test whether you have seen an answer; they test whether you can rebuild it under pressure.** So almost every page follows the same loop — *intuition → precise definition → math → picture → example → failure → the question an interviewer asks → the follow-up that exposes shallow understanding.*

The 15 questions you should be able to answer about any ML concept

1. What is it?
2. Why does it exist — what problem does it solve?
3. How does it work (mechanism, not slogan)?
4. What is the math?
5. What does it assume?
6. What happens when those assumptions fail?
7. What does it cost (time, memory, data)?
8. When should I use it?
9. When should I **not** use it?
10. What are the alternatives?
11. What happens at 100× scale?
12. How does it behave in production?
13. How would I debug it?
14. How would I explain it in 2 minutes?
15. Which follow-up would expose a shallow answer?

Legend — what each notebook object means

Object	Meaning
YELLOW STICKY	The one idea to remember. If you forget everything else on the page, keep this.
PINK STICKY	A misconception or an interview trap — something that sounds right but is wrong.
ORANGE CARD	A real-world failure case: how this concept breaks systems in production.
MINT STICKY	A worked example with numbers.
INDEX CARD	An interview question with a strong answer.
RED PEN	What the interviewer asks next — the follow-up and the deeper follow-up.
WHITEBOARD	A derivation you should be able to reproduce standing up.
PURPLE CARD	A "why" question answered down to the mechanism.
TEAL CARD	A scenario: you are given a situation and must reason through it.
BLACK CARD	Staff-level reasoning: trade-offs, systems, business context.

Object	Meaning
DASHED RED BOX	Active recall. Answer from memory before checking the Answer Key at the back.
BULB SLIP	**Real-life analogy** — the idea as an everyday picture, the key formula read aloud in plain words, and where the analogy stops being true.

Things written in highlighter are the phrases worth saying out loud in an interview. **Blue-ink bold** marks technical terms. In the question cards, the line **Analogy:** gives a one-sentence everyday picture of the answer.

What's new in the 2027 edition

Ten new Parts (Unit IX — Modern ML & GenAI).

- Part 41 — Transformers & Attention (MHA, GQA/MQA, RoPE, FlashAttention, MoE)
- Part 42 — Language Models (tokenization, scaling laws, decoding, reasoning)
- Part 43 — Adapting & Aligning Models (SFT, LoRA/QLoRA, RLHF, DPO, GRPO, distillation)
- Part 44 — Retrieval, RAG & Agents (BM25, hybrid search, ANN indexes, agent reliability, prompt injection)
- Part 45 — Evaluating LLM Systems & Safety (pass@k, paired tests, LLM-as-judge, hallucination, red-teaming)
- Part 46 — Efficient Training & Inference (6ND, 16 bytes/param, KV cache, roofline, quantization, speculative decoding)
- Part 47 — Recommender Systems & Bandits (ALS, BPR, two-tower + logQ, off-policy evaluation, UCB/Thompson/LinUCB)
- Part 48 — Responsible AI (fairness impossibility, SHAP/IG, differential privacy, EU AI Act timeline)
- Part 49 — Other Frontiers (forecasting, GNNs, VAE/GAN/diffusion, contrastive learning, causal ML, RL)
- Part 50 — The 2027 Interview Landscape (loop structure, GenAI rubric, project deep-dive, 8-week plan)

Easier to read.

- A **real-life analogy** box in almost every concept section — with the formula **read aloud** in plain words and a line on **where the analogy breaks**.
- A one-line **Analogy** in all 200 question cards (including 50 new GenAI cards).
- A **symbol decoder** ("How to read the maths", just before the Contents) so no formula is ever unreadable.

Deeper everywhere.

- New sections in Parts 1–26: PAC sample complexity, double descent, SVD by hand, power iteration and conditioning, multivariate Gaussians, exponential families, Fisher information, MCMC and importance sampling, delta method, Bayesian A/B tests, causal inference toolkit, GLMs and IRLS, worked tree and Naive Bayes examples, SVR, GMM/EM, spectral clustering, and more.
- More tested from-scratch code (every block in this book was run and checked).
- GenAI system designs, a GenAI question bank, new follow-up chains, derivations, cheat sheets, final-exam sections and answer keys for Recall Sets 16–22.

Where the new Parts sit

Parts 41–50 are printed as **Unit IX, right after Part 26** — so you move from classical to modern ML before the interview layer (Parts 27–40), which now covers both. The original numbering of Parts 1–40 is unchanged, so every cross-reference ("see Part 17") still points to the same place.

How to study with it

The rule

Read a section once. Close the notebook. Explain it out loud as if to an interviewer, including the math. Only then check. The gap between what you *read* and what you can *say* is exactly what an interview measures.

An 8-week plan (about 1.5–2 hours a day). Part 50 has a detailed version with outputs for each week.

Week	Read	Practice
1	Parts 1–4 (foundations, linear algebra, probability, statistics)	Recall Sets 1–4; derive MLE for a Gaussian and a Bernoulli on paper
2	Parts 5–13 (regression, trees, ensembles, SVM, KNN, NB, clustering, PCA)	Recall Sets 5–8; code linear + logistic regression and k-means from scratch
3	Parts 14–19 (optimization, losses, evaluation, validation, imbalance, features)	Recall Sets 9–11; the fraud-metric and CTR A/B scenarios out loud
4	Parts 20–23 (deep learning, coding, debugging, production)	Recall Sets 12–14; backprop for a 2-layer net from a blank page
5	Parts 24–26 (system design, data-centric ML, model selection)	Recall Set 15; two designs per day (classic + GenAI), timed at 35–45 minutes
6	Parts 41–43 (transformers, LLMs, alignment)	Recall Sets 16–17; attention and LoRA from scratch; DPO derivation
7	Parts 44–47 (RAG, LLM evaluation, efficiency, recommenders)	Recall Sets 18–21; KV-cache, 6ND and decode-speed arithmetic drills
8	Parts 48–49, then 27–40 (question bank, chains, derivations, cheat sheets)	Recall Set 22; the Final Exam under time; Interviewer Mode on weak areas

Interviewer Mode and the final exam are live. A PDF cannot grade you, so Part 33 and the Final Exam are designed to be used with Claude: come back to the chat and say *"Interview me"* (optionally with a topic), or paste your exam answers. You will be asked one question at a time, graded on correctness, depth, clarity, math, practical judgment and communication, then pushed with follow-ups until the edge of your understanding is found.

How to read the maths

Never skip a formula — read it aloud

Every formula in this book is a sentence written in shorthand. Read it left to right, replacing each symbol with the words below, and it becomes an ordinary sentence. Try it on one formula now: $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$ reads "for every example i from 1 to n, take (true answer − predicted answer), square it, add them all up, and divide by n" — that is, the **average squared mistake**.

Real-life analogy: maths is a recipe card

Think of it like a recipe written by a busy chef: "2 tbsp oil, heat, add 1 tsp jeera, 30 s" is shorthand for a long sentence. Formulas are the same — dense, but each symbol is just an ingredient or an action. Once you know the abbreviations (the table below), any formula can be read like a recipe.

Where it breaks: recipes are forgiving about order; formulas are not — a missing minus sign or a swapped subscript changes the meaning.

Symbol decoder — the building blocks

Symbol	Read it as	Tiny example
x, y	a single number (input, answer)	$x = 3$
$\mathbf{x}$ (bold)	a **vector**: a list of numbers, e.g. one example's features	$\mathbf{x} = (\text{age}, \text{income}) = (30,\ 50)$
X (capital)	a **matrix**: a table; rows = examples, columns = features	1,000 customers × 5 features
x_i / x_j	the i-th example / the j-th feature (subscripts are "which one")	x_3 = the third item
$\hat{y}$ ("y-hat")	a **prediction** or estimate (the hat means "our guess")	$\hat{y} = 0.8$
$\bar{x}$ ("x-bar")	the **average**	$\bar{x}$ of 2, 4, 6 is 4
n, d, k	number of examples, number of features, a count you choose	$n = 1000$ rows
$\sum_{i=1}^{n}$	"**add up** for i from 1 to n"	$\sum_{i=1}^{3} i = 1+2+3 = 6$
$\prod_{i=1}^{n}$	"**multiply** together for i from 1 to n"	$\prod_{i=1}^{3} i = 6$
$\mathbf{x}^\top\mathbf{w}$	**dot product**: multiply matching entries and add	$(1,2)\cdot(3,4) = 3+8 = 11$
$X^\top$	**transpose**: flip the table so rows become columns	a 5×2 table becomes 2×5
A^{-1}	**inverse**: the matrix that "undoes" A (like dividing)	$A^{-1}A = I$
$\lvert x\rvert$ / $\lVert\mathbf{x}\rVert$	size of a number / **length** of a vector	$\lVert(3,4)\rVert = 5$
$\log x$, e^x	natural log / exponential (they undo each other)	$\log e^2 = 2$
$\max(0, z)$	the bigger of 0 and z ("never negative")	$\max(0,-3) = 0$
$\arg\min_w f(w)$	"the **value of** w that makes f smallest" (not the smallest value itself)	$\arg\min_w (w-2)^2 = 2$
$\approx$, $\propto$	approximately equal / proportional to ("grows in step with")	$\pi \approx 3.14$
$\in$, $\mathbb{R}^d$	"belongs to" / all lists of d real numbers	$\mathbf{x} \in \mathbb{R}^2$
$O(n^2)$	"grows like n^2" — cost doubles → roughly 4×	comparing all pairs
$\odot$	multiply **element by element**	$(1,2)\odot(3,4) = (3,8)$

Symbol	Read it as	Tiny example
$\mathbb{1}[\cdot]$	**indicator**: 1 if the condition is true, else 0	$\mathbb{1}[5 > 3] = 1$

Symbol decoder — calculus and probability

Symbol	Read it as	Tiny example
$\frac{df}{dx}$	**slope** of f: how fast f changes when x changes a little	$f = x^2 \Rightarrow \frac{df}{dx} = 2x$
$\frac{\partial L}{\partial w}$	**partial derivative**: slope of L when only w moves	"turn one knob"
∇L ("grad L")	the list of all partial derivatives — points **uphill**	$\nabla(w_1^2 + w_2^2) = (2w_1, 2w_2)$
$\int_a^b f(x)\,dx$	area under the curve — a continuous "add up"	total distance from speed
$P(A)$	probability that A happens (0 to 1)	$P(\text{heads}) = 0.5$
$P(A \mid B)$	probability of A **given that** B happened	P(rain given clouds)
$\mathbb{E}[X]$	**expected value**: the long-run average	dice: $\mathbb{E}[X] = 3.5$
$\mathrm{Var}(X)$, σ	spread (variance) and standard deviation $\sigma = \sqrt{\mathrm{Var}}$	heights vary by ~7 cm
$X \sim \mathcal{N}(\mu, \sigma^2)$	"X is drawn from a bell curve centred at μ with spread σ"	heights $\sim \mathcal{N}(165, 7^2)$
$\mathrm{KL}(P\|Q)$	how different distribution Q is from P (0 if equal)	extra cost of a wrong map
$\mathcal{L}$, L, J, ℓ	the **loss**: a score of how wrong the model is	smaller is better
$R(f)$, $\hat{R}_n(f)$	true error on all future data / error measured on our n samples	exam vs. mock test

Greek letters and what they usually mean in this book

Letter	Name	Usual meaning here
α	alpha	significance level; a smoothing or mixing weight
β	beta	regression coefficients; momentum; KL strength in RLHF/DPO
γ	gamma	discount factor (RL); focal-loss power; a small constant
δ, Δ	delta	a small failure probability; a change ("delta")
ε, ϵ	epsilon	noise; a small tolerance; privacy budget; exploration rate
η	eta	**learning rate** (step size)
θ	theta	the model's **parameters** (all the knobs)
λ	lambda	**regularization strength**; an eigenvalue
μ	mu	a **mean** (average)
π	pi	3.14159...; a **policy** in RL; mixture weights
ρ	rho	a correlation
σ	sigma	standard deviation — *or* the sigmoid function $\sigma(z)$ (context decides)
Σ	capital sigma	a covariance matrix (when it isn't the "add up" sign)
τ	tau	temperature; a threshold; a treatment effect
ϕ, ψ	phi, psi	a feature map or activation function

Reading a formula in four steps — worked example

Formula: the ridge regression objective $\min_{\mathbf{w}} \sum_{i=1}^{n}(y_i - \mathbf{x}_i^\top \mathbf{w})^2 + \lambda\|\mathbf{w}\|^2$.

1. **Find the verb.** $\min_{\mathbf{w}}$: "choose the weights $\mathbf{w}$ that make the following as small as possible".
2. **Read each piece.** $\mathbf{x}_i^\top \mathbf{w}$ = the prediction for example i; $(y_i - \mathbf{x}_i^\top \mathbf{w})^2$ = its squared mistake; $\sum$ = add over all examples; $\|\mathbf{w}\|^2$ = how large the weights are.
3. **Read the connectors.** "+ λ ×" means: add a penalty for big weights, with λ setting how much we care.
4. **Say it as a sentence.** "Pick weights that make the total squared mistake small, while keeping the weights small; λ decides the balance." That sentence is what an interviewer wants to hear before any algebra.

Contents

How to use this notebook . . . 2

How to read the maths . . . 5

Foundations of Learning . . . 15

PART 1 **ML Foundations** . . . 16

1.1 What machine learning actually optimizes . . . 16
1.2 Expected risk . . . 18
1.3 Empirical risk minimization (ERM) . . . 19
1.4 Generalization . . . 21
1.5 Hypothesis spaces . . . 23
1.6 Inductive bias . . . 24
1.7 Training, validation and test sets . . . 26
1.8 The IID assumption . . . 27
1.9 Distribution shift . . . 29
1.10 No Free Lunch (intuition) . . . 30
1.11 Bias–variance tradeoff . . . 32
1.12 Approximation vs. estimation error (and optimization error) . . . 34
1.13 Model capacity . . . 36
1.14 Underfitting . . . 37
1.15 Overfitting . . . 39
1.16 Regularization . . . 40
1.17 Data leakage . . . 41
1.18 PAC learning and sample complexity (with numbers) . . . 43
1.19 Double descent and benign overfitting . . . 45

Mathematics for ML Interviews . . . 48

PART 2 **Linear Algebra** . . . 49

2.1 Scalars, vectors, matrices, tensors . . . 49
2.2 Dot product . . . 50
2.3 Matrix multiplication . . . 50
2.4 Transpose . . . 51
2.5 Inverse . . . 51
2.6 Rank and linear independence . . . 52
2.7 Basis and orthogonality . . . 53
2.8 Norms . . . 54
2.9 Eigenvalues and eigenvectors . . . 54
2.10 Eigendecomposition . . . 55
2.11 Singular value decomposition (SVD) . . . 56
2.12 Positive definite matrices . . . 57
2.13 Covariance matrices . . . 58
2.14 Projection . . . 58
2.15 PCA geometry (preview) . . . 59
2.16 Matrix calculus you must be able to do by hand 60
2.17 SVD by hand: a 2×2 worked example . . . 60
2.18 Power iteration: how eigenvectors are actually computed . . . 61
2.19 Conditioning: why solvers avoid the normal equations . . . 62

PART 3 **Probability** . . . 64

3.1 Random variables and distributions . . . 64
3.2 Joint, marginal and conditional probability . . . 65
3.3 Independence . . . 66
3.4 Bayes' theorem . . . 66
3.5 Expectation, variance, covariance, correlation . . . 67
3.6 Law of total probability and total expectation . . . 68
3.7 Law of large numbers and central limit theorem 68
3.8 Maximum likelihood estimation (MLE) . . . 69
3.9 Why optimize the log-likelihood instead of the likelihood? . . . 70
3.10 Maximum a posteriori (MAP) . . . 71
3.11 Bayesian inference: prior, likelihood, posterior 72
3.12 Entropy . . . 72
3.13 Cross-entropy . . . 73
3.14 KL divergence . . . 73
3.15 Quick probability problems interviewers like . . . 75
3.16 The multivariate Gaussian . . . 75
3.17 Exponential families and GLMs . . . 76
3.18 Fisher information and the Cramér–Rao bound 77
3.19 Sampling methods: generating and estimating with randomness . . . 78
3.20 More probability puzzles . . . 79

PART 4 **Statistics** . . . 81

4.1 Population vs. sample . . . 81
4.2 Sampling . . . 82
4.3 Mean, median, variance, standard deviation . . . 82
4.4 Covariance and correlation (statistical view) . . . 83
4.5 Confidence intervals . . . 84
4.6 Hypothesis testing . . . 84
4.7 The standard tests . . . 85
4.8 Bootstrapping . . . 86
4.9 A/B testing . . . 86
4.10 Multiple hypothesis testing . . . 88
4.11 Selection bias, survivorship bias, confounding 88
4.12 Simpson's paradox . . . 89
4.13 Causation vs. correlation . . . 90
4.14 Ratio metrics and the delta method . . . 91
4.15 Bayesian A/B testing . . . 91

4.16 The causal inference toolkit, one level deeper 92

Classical Machine Learning 95

PART 5 Linear Regression 96
5.1 Model formulation 96
5.2 Loss function: MSE, and why squared 97
5.3 Ordinary least squares and the normal equation 97
5.4 Gradient descent solution 98
5.5 Assumptions — and what breaks when each fails 99
5.6 Residuals and diagnostics 99
5.7 R^2 and adjusted R^2 100
5.8 Multicollinearity 100
5.9 Heteroscedasticity 101
5.10 Outliers 101
5.11 Regularization: Ridge → Lasso → Elastic Net 102
5.12 Beyond Gaussian noise: Poisson, Gamma and Tweedie regression 105
5.13 Quantile regression and GAMs 106
5.14 Implementation from scratch 106
5.15 Interview questions on linear regression 108

PART 6 Logistic Regression 109
6.1 Why linear regression is inappropriate for classification 109
6.2 Sigmoid and log-odds 110
6.3 Decision boundary 110
6.4 Likelihood and binary cross-entropy 111
6.5 The gradient 111
6.6 Regularization 112
6.7 Threshold selection 113
6.8 Calibration 113
6.9 Multiclass logistic regression and softmax 115
6.10 Newton's method for logistic regression (IRLS), in code 116
6.11 Class weights, sample weights and prior shift 116
6.12 Interview questions on logistic regression 117

PART 7 Decision Trees 120
7.1 Tree construction 120
7.2 Splitting criteria 121
7.3 Regression trees 123
7.4 A split computed by hand 123
7.5 Trees for regression: a worked split 124
7.6 Continuous features and categorical features 124
7.7 Missing values 125
7.8 Tree depth, overfitting and pruning 125
7.9 Feature importance 126
7.10 Computational complexity 127
7.11 Strengths and weaknesses — with the mechanism 127

PART 8 Ensemble Methods 129
8.1 Bagging 129
8.2 Random forest 130
8.3 Boosting 131
8.4 Comparisons 135
8.5 Why boosting reduces bias; why random forests reduce variance 136
8.6 Stacking and blending 137
8.7 Practical tuning recipe for gradient boosting 137
8.8 Monotonic and interaction constraints 138
8.9 Explaining tree ensembles with SHAP 139

PART 9 Support Vector Machines 141
9.1 The maximum-margin classifier (geometry first) 141
9.2 Hard margin 142
9.3 Soft margin, slack and the C parameter 143
9.4 Hinge loss — the unconstrained view 143
9.5 Dual formulation (intuition you need) 144
9.6 The kernel trick 145
9.7 When SVMs perform well — and why they've faded for large-scale work 146
9.8 Support vector regression and one-class SVM 147

PART 10 K-Nearest Neighbors 148
10.1 Algorithm 148
10.2 Distance metrics 149
10.3 Choice of k 149
10.4 Normalization 149
10.5 The curse of dimensionality 150
10.6 Complexity, memory, and when KNN becomes impractical 151

PART 11 Naive Bayes 153
11.1 Bayes' theorem and the independence assumption 153
11.2 Variants 153
11.3 Text classification 154
11.4 A Naive Bayes spam filter by hand 155
11.5 Generative vs. discriminative 156

PART 12 Unsupervised Learning 158
12.1 K-means 158
12.2 Hierarchical clustering 159
12.3 DBSCAN 160
12.4 Comparison 160
12.5 Gaussian mixture models and the EM algorithm 161
12.6 Spectral clustering and HDBSCAN 163
12.7 When clustering becomes meaningless in high dimensions 164

PART 13 Dimensionality Reduction 166
13.1 PCA — derived conceptually 166

13.2 PCA — derived mathematically . . . 168
13.3 Why t-SNE should not be read as preserving global distances . . . 169
13.4 UMAP . . . 170
13.5 Kernel PCA, autoencoders and randomized SVD . . . 170
13.6 Choosing a method . . . 171

Training, Evaluation & Data . . . 174

PART 14 Optimization . . . 175
14.1 Objective functions . . . 175
14.2 Convex vs. non-convex optimization . . . 176
14.3 Gradients, partial derivatives and the chain rule . . . 176
14.4 Gradient descent . . . 177
14.5 SGD and mini-batch SGD . . . 178
14.6 Momentum . . . 179
14.7 Nesterov momentum . . . 179
14.8 AdaGrad, RMSProp, Adam . . . 180
14.9 AdamW and weight decay . . . 181
14.10 Learning rate — the most important hyperparameter . . . 182
14.11 Learning-rate schedules . . . 183
14.12 Gradient clipping . . . 184
14.13 Saddle points and local minima . . . 184
14.14 Vanishing and exploding gradients . . . 185
14.15 Second-order methods (what to say about them) . . . 185
14.16 Why might Adam converge faster but SGD generalize better? . . . 186
14.17 Batch size, learning-rate scaling and warmup . . . 187
14.18 Optimizers for the large-model era . . . 188
14.19 Sharpness-aware minimization (SAM) . . . 188

PART 15 Loss Functions . . . 190
15.1 Regression losses . . . 190
15.2 Classification losses . . . 192
15.3 Metric-learning losses . . . 194
15.4 Ranking losses (one paragraph you'll need for search/recsys) . . . 196
15.5 Proper scoring rules: log loss vs. Brier score . . . 196
15.6 Multi-label, ordinal and cost-sensitive losses . . . 197
15.7 Summary table . . . 197

PART 16 Model Evaluation . . . 200
16.1 Classification metrics . . . 200
16.2 Regression metrics . . . 203
16.3 Ranking metrics . . . 204
16.4 Choosing metrics from business costs . . . 204
16.5 Error bars on metrics: bootstrap and calibration error in code . . . 206
16.6 Slice-based evaluation . . . 207

PART 17 Data Splitting and Validation . . . 208
17.1 The strategies . . . 208
17.2 Leakage and train/test contamination (split-level) . . . 209
17.3 When random splitting gives misleading results . . . 210
17.4 Adversarial validation and purged cross-validation . . . 210

PART 18 Class Imbalance . . . 212
18.1 Techniques . . . 212
18.2 Why oversampling before splitting causes leakage . . . 213
18.3 A practical order of operations . . . 214

PART 19 Feature Engineering . . . 216
19.1 Numeric features . . . 216
19.2 Categorical features . . . 217
19.3 High-cardinality categories . . . 218
19.4 Missing data . . . 219
19.5 Feature interactions . . . 219
19.6 Log transformations (and the retransformation trap) . . . 220
19.7 Time features . . . 220
19.8 Text features . . . 221
19.9 Embeddings as features . . . 222
19.10 Leakage in features . . . 222
19.11 When feature engineering matters more or less . . . 223
19.12 Feature selection . . . 224

Deep Learning & ML Coding . . . 226

PART 20 Deep Learning Foundations . . . 227
20.1 The perceptron . . . 227
20.2 Multilayer perceptron (MLP) . . . 228
20.3 Activation functions . . . 228
20.4 Forward propagation, loss, and backpropagation . . . 229
20.5 Weight initialization . . . 231
20.6 Batch normalization . . . 231
20.7 Layer normalization . . . 232
20.8 Dropout . . . 233
20.9 Residual connections . . . 234
20.10 Why can deeper networks be harder to optimize? . . . 235
20.11 Convolutional networks: the arithmetic interviewers ask for . . . 235
20.12 Recurrent networks and LSTMs: why the gates matter . . . 236
20.13 Regularizing deep networks in practice . . . 237
20.14 A map of architectures (for orientation) . . . 238

PART 21 ML Coding Interview . . . 241
21.1 Linear regression . . . 241
21.2 Logistic regression . . . 242

21.3 Gradient descent (generic) 243
21.4 K-nearest neighbors 243
21.5 K-means (with k-means++) 244
21.6 Decision tree basics (Gini, classification) ... 245
21.7 PCA 246
21.8 Softmax and cross-entropy 247
21.9 Train/test split 247
21.10 Standard scaler 248
21.11 Confusion matrix, precision, recall, F1 (and AUC) 248
21.12 Mini-batch gradient descent 249
21.13 Simple neural network (2-layer MLP, softmax output) 250
21.14 Round 2: the implementations that show up in 2026–27 loops 251
21.15 Coding-round checklist 253

Debugging, Production + System Design 255

PART 22 ML Debugging 256
22.1 How to isolate problems scientifically 257
22.2 Scenario 1 — Training loss decreases, validation loss increases 257
22.3 Scenario 2 — Both training and validation accuracy are poor 258
22.4 Scenario 3 — Offline AUC improves but production metrics fall 259
22.5 Scenario 4 — The model suddenly predicts mostly one class 260
22.6 Scenario 5 — Training becomes NaN 261
22.7 Scenario 6 — Good overall, bad for one user segment 262
22.8 Scenario 7 — Performance degraded after deployment 263
22.9 Two more classics 263
22.10 Scenario 8 — An LLM feature got worse after a "minor" change 264
22.11 Scenario 9 — Fine-tuning an LLM: loss spikes, forgetting and "it learned the format, not the task" 264
22.12 Scenario 10 — The LLM bill tripled and latency doubled 265

PART 23 Production ML 267
23.1 Pipelines 267
23.2 Feature stores 268
23.3 Batch vs. online inference 268
23.4 Model serving 269
23.5 Model registry, data versioning, experiment tracking, reproducibility 269
23.6 Monitoring 270
23.7 Drift taxonomy 271
23.8 Retraining 271
23.9 Deployment strategies 272
23.10 LLMOps: what production changes for LLM systems 273

PART 24 ML System Design 275
24.1 The reusable framework 275
24.2 Design 1 — Spam detection (email) 276
24.3 Design 2 — Fraud detection (card payments) 277
24.4 Design 3 — Recommendation system (home feed) 278
24.5 Design 4 — Search ranking 280
24.6 Design 5 — Ads click-through-rate (CTR) prediction 281
24.7 Design 6 — Credit risk (loan default) 282
24.8 Design 7 — Churn prediction 283
24.9 Design 8 — Content moderation 284
24.10 Design 9 — ETA prediction (ride-hailing / delivery) 285
24.11 Design 10 — Demand forecasting (retail) . 285
24.12 Design 11 — Duplicate detection (listings, questions, images) 286
24.13 Design 12 — Anomaly detection (metrics or transactions, few labels) 287
24.14 A universal failure-mode checklist for any design 288
24.15 Design 13 — Enterprise knowledge assistant (RAG with permissions) 289
24.16 Design 14 — Customer-support agent that takes actions 290
24.17 Design 15 — LLM-enhanced product search 291
24.18 Design 16 — Code assistant (inline completion + chat) 292
24.19 Design 17 — Content moderation with LLMs at scale 293
24.20 Design 18 — A multi-tenant LLM inference platform 294

PART 25 Data-Centric ML 295
25.1 Why improving data can beat changing the model 295
25.2 Label noise and incorrect labels 296
25.3 Missing labels 297
25.4 Sampling bias 297
25.5 Data imbalance and the long tail 297
25.6 Duplicates 298
25.7 Outliers 298
25.8 Dataset shift and leakage (data-centric view) 299
25.9 Annotation quality and human disagreement 299
25.10 Data-centric work for LLMs: curation, synthetic data and contamination 300

PART 26 Practical Model Selection 302
26.1 The trade-off dimensions 302
26.2 Scenarios — which model and why? 303

26.3 Two models, identical accuracy — which to deploy? . . . 305
26.4 Prompt, RAG, fine-tune or build? Choosing an LLM approach (with cost math) . . . 305

Modern ML + GenAI . . . 308

PART 41 Transformers & Attention . . . 309

41.1 Why attention replaced recurrence . . . 309
41.2 Scaled dot-product attention . . . 310
41.3 Multi-head attention . . . 311
41.4 The transformer block and the residual stream . . . 311
41.5 Encoder, decoder, encoder–decoder . . . 312
41.6 Positional information . . . 313
41.7 Making attention cheaper . . . 313
41.8 Mixture of Experts (MoE) . . . 315
41.9 Transformers beyond text . . . 315
41.10 Code: attention, multi-head attention with GQA, KV cache, RoPE . . . 316
41.11 Interview questions on transformers . . . 317

PART 42 Language Models: Tokens, Pretraining, Scaling & Decoding . . . 319

42.1 The objective: next-token prediction . . . 319
42.2 Tokenization . . . 320
42.3 Pretraining: data, compute and dynamics . . . 321
42.4 Scaling laws . . . 322
42.5 Decoding . . . 323
42.6 In-context learning, prompting and reasoning . . . 324
42.7 Embedding models . . . 325

PART 43 Adapting & Aligning Models . . . 328

43.1 Choosing how to adapt . . . 328
43.2 Supervised fine-tuning (SFT) . . . 329
43.3 Parameter-efficient fine-tuning (PEFT) . . . 329
43.4 RLHF: learning from human preferences . . . 331
43.5 DPO: preference optimization without a reward model . . . 332
43.6 RL with verifiable rewards and GRPO . . . 333
43.7 Distillation . . . 333
43.8 AI feedback and constitutions . . . 334

PART 44 Retrieval, RAG & Agents . . . 337

44.1 Why RAG . . . 337
44.2 The pipeline, stage by stage . . . 338
44.3 Dense, sparse and hybrid retrieval . . . 338
44.4 Vector indexes (ANN) — what to know . . . 340
44.5 Evaluating RAG — separately at each stage . . . 340
44.6 Failure modes and fixes . . . 341
44.7 Agents and tool use . . . 341
44.8 Security: prompt injection and data exfiltration . . . 342

PART 45 Evaluating LLM Systems & Safety . . . 345

45.1 Why evaluating LLMs is hard . . . 345
45.2 The evaluation stack . . . 346
45.3 Reference-based metrics and their limits . . . 346
45.4 Execution-based evaluation and pass@k . . . 347
45.5 Benchmarks and contamination . . . 348
45.6 Statistical rigor: error bars and paired comparisons . . . 348
45.7 Human evaluation and pairwise preferences . . . 349
45.8 LLM-as-judge . . . 350
45.9 Building an eval harness for an LLM product . . . 351
45.10 Hallucination . . . 351
45.11 Safety, red-teaming and guardrails . . . 352

PART 46 Efficient Training & Inference . . . 355

46.1 Compute accounting: 6ND and 2N . . . 355
46.2 Memory accounting in training . . . 356
46.3 Number formats . . . 357
46.4 Parallelism . . . 357
46.5 Inference: prefill vs. decode . . . 358
46.6 The KV cache and its management . . . 359
46.7 Quantization . . . 360
46.8 Speculative decoding . . . 361
46.9 Cost engineering at the application layer . . . 362

PART 47 Recommender Systems & Bandits . . . 365

47.1 The funnel (a deeper look at Design 3) . . . 365
47.2 Feedback: what the data really is . . . 366
47.3 Collaborative filtering and matrix factorization . . . 366
47.4 Two-tower retrieval . . . 367
47.5 Ranking models . . . 368
47.6 Sequential and generative recommendation . . . 369
47.7 Offline evaluation . . . 369
47.8 Bias, feedback loops and counterfactual evaluation . . . 370
47.9 Diversity and re-ranking . . . 371
47.10 Cold start . . . 372
47.11 Bandits: learning while serving . . . 372

PART 48 Responsible AI: Fairness, Explainability, Privacy & Robustness . . . 376

48.1 Fairness: where unfairness comes from . . . 376
48.2 Fairness definitions (and why you can't have them all) . . . 377
48.3 Explainability . . . 378
48.4 Privacy . . . 379
48.5 Robustness and security . . . 381
48.6 Documentation, governance and regulation . . . 381

PART 49 Other Frontiers: Forecasting, Graphs, Generative & Causal ML, RL 384
49.1 Time series and forecasting 384
49.2 Graph machine learning 385
49.3 Generative models: VAEs, GANs, diffusion 386
49.4 Self-supervised and contrastive learning 388
49.5 Causal ML: uplift and heterogeneous treatment effects 389
49.6 Reinforcement learning essentials 390

PART 50 The 2027 Interview Landscape 393
50.1 What changed by 2026–27 393
50.2 The typical loop 393
50.3 Role archetypes and what to prioritize 394
50.4 The GenAI design rubric 395
50.5 The project deep-dive 395
50.6 Behavioral themes that recur 396
50.7 An 8-week plan for the 2027 edition 396
50.8 The last-week checklist 396

THE INTERVIEW LAYER 398

PART 27 200 High-Frequency Interview Questions 399
27.1 Easy 399
27.2 Medium 412
27.3 Hard 427
27.4 Staff-level reasoning 439
27.5 GenAI & modern ML (2027 edition) 447

PART 28 Why Questions 466
28.1 Why does regularization reduce overfitting? 466
28.2 Why does L1 produce sparse weights? 467
28.3 Why does feature scaling matter for some algorithms but not for trees? 467
28.4 Why does bagging reduce variance? 468
28.5 Why does boosting reduce bias? 469
28.6 Why does cross-entropy work for classification? 469
28.7 Why does gradient descent work? 470
28.8 Why can accuracy be misleading? 471
28.9 Why does high dimensionality hurt KNN? 471
28.10 Why do neural networks require nonlinear activations? 472
28.11 Why can deeper networks be harder to optimize? 472
28.12 Why does batch normalization help? 473
28.13 Why do residual connections help? 473
28.14 Why can offline metrics disagree with online performance? 474
28.15 Four more "why"s interviewers like 474

PART 29 Compare Questions 476
29.1 Models 476
29.2 Metrics and losses 477
29.3 Optimization and deep learning 477
29.4 Systems 477

PART 30 Trick and Follow-up Chains 478

PART 31 Derivations You Must Know 488
31.1 Linear regression loss (from maximum likelihood) 488
31.2 The normal equation 489
31.3 Gradient of MSE 489
31.4 Logistic regression likelihood 490
31.5 Binary cross-entropy and its gradient 490
31.6 Softmax 491
31.7 Cross-entropy (multiclass) and its gradient 492
31.8 Entropy 492
31.9 Information gain 493
31.10 Ridge regression 493
31.11 Lasso (intuition and the 1-D solution) 494
31.12 PCA 495
31.13 Backpropagation 495
31.14 Gradient descent update 496
31.15 Bias–variance decomposition 496
31.16 Maximum likelihood estimation 497
31.17 Attention scaling by √d (new in 2027) 498
31.18 The DPO loss from the RLHF objective (new in 2027) 498
31.19 KV-cache memory and training compute (new in 2027) 499
31.20 The delta method for ratio metrics (new in 2027) 500

PART 32 Interview Mental Models 501
32.1 The bias/variance lens 501
32.2 Data → Objective → Model → Evaluation → Production 501
32.3 The proxy ladder 502
32.4 The information-availability lens ("what does the model know at prediction time?") 502
32.5 The distribution lens 502
32.6 The signal-to-noise lens 502
32.7 The decision-and-cost lens 502
32.8 The inductive-bias-matching lens 502
32.9 The 100× lens 503
32.10 The feedback-loop lens 503
32.11 The simplest-thing ladder 503
32.12 The error-decomposition lens 503
32.13 The token-budget lens (new in 2027) 503
32.14 The eval-first lens (new in 2027) 503

PART 33 Interviewer Mode 505
33.1 Commands 505
33.2 The protocol you'll get 505
33.3 The rubric 505
33.4 What a turn looks like (illustration) 506

PART 34 Active Recall 507

PART 35 Interview Answer Frameworks . 508
35.1 Conceptual questions 508
35.2 System design questions 509
35.3 GenAI system design questions (new in 2027) 509
35.4 Debugging questions 509
35.5 Verbal techniques that signal seniority 510

PART 36 Staff-Level Questions 511
36.1 "You improved AUC from 0.82 to 0.86 but business revenue did not change. Why?" 511
36.2 "Model quality falls only for new users. How would you debug it?" 512
36.3 "Your dataset contains 500 million examples. Would you use cross-validation?" 512
36.4 "Production feature values differ from training features." 513
36.5 "Your ranking model improves NDCG offline but decreases CTR online." 513
36.6 "Your fraud model blocks too many legitimate users." 514
36.7 "The distribution changes every weekend." . 515
36.8 "Training takes four days. How would you reduce experimentation time?" 515
36.9 "Two models have identical accuracy. How would you decide which to deploy?" 516
36.10 "Leadership wants to replace our ranking model with an LLM." (new in 2027) 516
36.11 "Great demos, flat usage four weeks after launch. What happened?" (new in 2027) 516
36.12 "Twenty teams are building LLM features. How do you keep quality under control?" (new in 2027) . . 517

PART 37 Common Interview Traps 518

REVISION + READINESS 520

PART 38 Master Cheat Sheets 521
38.1 ML algorithms 521
38.2 Loss functions 522
38.3 Metrics 522
38.4 Probability 522
38.5 Statistics 523
38.6 Optimization 523
38.7 Regularization 523
38.8 Validation 524
38.9 Feature engineering 524
38.10 Model debugging 524
38.11 ML system design 525
38.12 Production ML 525
38.13 Mathematical formulas 525
38.14 Complexity of common algorithms 526
38.15 Transformers & language models (2027) . 526
38.16 Adapting & aligning models (2027) 527
38.17 Inference & training math (2027) 527
38.18 RAG, agents & LLM evaluation (2027) . . . 527
38.19 Recommender systems & bandits (2027) . 528
38.20 Responsible AI & frontiers (2027) 528

PART 39 The 130 Must-Know Questions . 530
39.1 Core (35) 530
39.2 Important (30) 531
39.3 Advanced (20) 532
39.4 Staff-level (15) 533
39.5 GenAI & modern ML (30, new in 2027) 533

PART 40 The Knowledge Graph 535
40.1 The main chains 535
40.2 Concept dependency table 536
40.3 One connected story (say this to yourself before an interview) 537

FINAL EXAM MAANG-Style ML Readiness Examination 538
E.1 Section A — Conceptual (20) 538
E.2 Section B — Mathematical (10) 539
E.3 Section C — ML coding (10) 540
E.4 Section D — Debugging scenarios (10) 540
E.5 Section E — Model selection scenarios (10) . . 541
E.6 Section F — Statistics and probability (10) . . 541
E.7 Section G — Production ML (10) 542
E.8 Section H — ML system design (10) 542
E.9 Section I — Extremely difficult follow-ups (10) 543
E.10 Section J — GenAI & LLM systems (15, new in 2027) 543
E.11 Section K — Recommenders, responsible AI & frontiers (10, new in 2027) 544

ANSWERS Answer Key — Active Recall . 546
A.1 Set 1 — Foundations 546
A.2 Set 2 — Linear algebra 547
A.3 Set 3 — Probability 548
A.4 Set 4 — Statistics 549
A.5 Set 5 — Linear & logistic regression 550
A.6 Set 6 — Trees & ensembles 550
A.7 Set 7 — SVM, KNN, Naive Bayes 551
A.8 Set 8 — Unsupervised learning & dimensionality reduction 552
A.9 Set 9 — Optimization & losses 553

A.10 Set 10 — Evaluation, validation, imbalance . 554
A.11 Set 11 — Feature engineering 555
A.12 Set 12 — Deep learning 556
A.13 Set 13 — ML coding 557
A.14 Set 14 — Debugging & production 557
A.15 Set 15 — System design, data-centric ML, model selection 558
A.16 Set 16 — Transformers & language models 559
A.17 Set 17 — Adapting & aligning models 560
A.18 Set 18 — Retrieval, RAG & agents 561
A.19 Set 19 — Evaluating LLM systems & safety 562
A.20 Set 20 — Efficient training & inference 563
A.21 Set 21 — Recommender systems & bandits 564
A.22 Set 22 — Responsible AI & other frontiers . 565

Unit I

Foundations of Learning

Part I — What learning is, why it generalizes, and every way it fails to.

Every later chapter is a special case of this one: a hypothesis space, a loss, a sample, and a hope that the sample looks like the future.

PART 1

ML Foundations

The one-sentence version of machine learning

We want a function that performs well on data **we have not seen**. We can only measure performance on data **we have seen**. Everything in this part — risk, generalization, bias, variance, regularization, validation, leakage — is about managing the gap between those two sentences.

The whole Part as one picture

Think of it like preparing for board exams using only past papers. The past papers are your training data; the real exam is the future data you never see in advance. Studying well means learning the *ideas* behind the questions, not memorising the answer key (overfitting), and not studying so little that you fail even the past papers (underfitting). Mock tests are your validation set; the board exam itself is the test set, taken once. A leaked question paper is data leakage, and a surprise change of syllabus is distribution shift.

Where it breaks: a student can ask the teacher what the exam will cover; a model only ever gets the sample, and must lean on built-in assumptions (its inductive bias) to guess beyond it.

1.1 What machine learning actually optimizes

① **Intuition.** Nobody actually wants "low cross-entropy". The business wants fewer fraud losses, more relevant search results, fewer churned users. ML turns that into something a computer can minimize — and every translation step loses something.

② **Definition.** ML optimizes a **surrogate loss** averaged over a **finite training sample**, inside a restricted **hypothesis space**, using an **approximate optimizer**, as a proxy for **expected performance on the future data distribution** of a **business metric**.

③ **Math.** The chain of proxies:

$$\underbrace{\text{business value}}_{\text{what we want}} \rightarrow \underbrace{\text{metric } M}_{\text{e.g. precision@k}} \rightarrow \underbrace{\mathbb{E}_{(x,y)\sim P}[\ell(f(x),y)]}_{\text{expected risk}} \rightarrow \underbrace{\frac{1}{n}\sum_i \ell(f(x_i),y_i)}_{\text{empirical risk}} \rightarrow \underbrace{\text{what SGD actually reaches}}_{\text{optimization}}$$

Real-life analogy: a restaurant's chain of stand-ins

Think of it like running a restaurant. You want a loved, busy restaurant for years (business value), so you track the star rating on a delivery app (the metric). The chef can't taste "star rating", so she tastes for salt and spice (the surrogate loss), on today's few dishes (the sample), with ten minutes to adjust (the optimizer). Each step stands in for the one above. You can ace today's tasting and still lose your regulars.

Formula, read aloud: the chain says: what we want → what we measure → average mistake on *all* future data → average mistake on the data we have → what training actually reaches. Each arrow means "is a stand-in for", and each one leaks a little.

Where it breaks: the owner sees customers daily; in ML the top rungs (revenue, trust) often show up only months later, through experiments.

④ **Picture.** A ladder where each rung is a proxy for the one above. You only ever touch the bottom rung.

```
business value      (revenue, safety, retention)          ← what matters
      |  proxy gap #1: metric misses things (long-term effects, fairness)
offline metric      (AUC, NDCG, RMSE)
      |  proxy gap #2: metric is not differentiable → use a surrogate loss
expected loss       E[ l(f(x), y) ] over the TRUE distribution
      |  proxy gap #3: we only have n samples
empirical loss      (1/n) sum l(f(x_i), y_i)
      |  proxy gap #4: optimizer is approximate, non-convex
what training finds                                       ← what you touch
```

⑤ Example

Ads CTR model. Business: ad revenue and user satisfaction. Metric: log loss / AUC on clicks. Loss: binary cross-entropy. Sample: last 30 days of impressions. Optimizer: SGD/Adam on a deep model. A model can improve log loss by predicting clicks on clickbait better — which raises CTR and *hurts* long-term satisfaction. Every rung was optimized; the top rung got worse.

⑥ Real-world failure

Recommendation systems optimized for watch time learned to promote outrage and autoplay-bait: the loss (predicted engagement) was minimized beautifully while the business objective (healthy long-term usage, advertiser trust) degraded. The fix was not a better model; it was a better **objective** (satisfaction surveys, long-term retention, "regret" signals added to the target).

⑦ Misconception

"The model's job is to minimize the loss." No — the loss is a tool. The model's job is to perform on **future data** on **the metric that matters**. A model with a lower training loss can be worse on every rung above it.

⑧ Interview question: "What does a machine learning model actually optimize?"

⑨ **Strong answer.** "Directly, it minimizes an empirical average of a differentiable surrogate loss over the training set, usually plus a regularizer, using an approximate optimizer. What we *want* is low expected loss on the future data distribution, and above that, a business metric. So there are several gaps: the metric-vs-business gap, the surrogate-loss gap — like cross-entropy standing in for accuracy — the sample-

vs-population gap, which is generalization, and the optimization gap. Good ML engineering is mostly about keeping each of those gaps small and measuring them separately."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Why not optimize accuracy directly?" — Sketch: the 0-1 loss is piecewise constant, so its gradient is zero almost everywhere and undefined at the boundary; minimizing it exactly is NP-hard for linear classifiers. Convex surrogates (log loss, hinge) upper-bound it and are* ***classification-calibrated****: minimizing them in the limit yields the Bayes-optimal classifier.*

Deeper: *"When does a lower loss give a worse metric?" — Sketch: whenever the loss and metric weight errors differently — e.g. log loss rewards calibration across all examples, while precision@10 only cares about the ranking at the very top; or MSE punishes outliers the business doesn't care about. Remedy: choose losses aligned with the metric (ranking losses, weighted losses) and always report the metric, not just the loss.*

⑫ **When the assumption breaks.** The whole chain assumes the future looks like the sample (IID) and that the metric reflects value. When users react to the model (feedback loops), when the population shifts, or when the metric can be gamed (Goodhart's law: *when a measure becomes a target, it ceases to be a good measure*), minimizing the bottom rung stops improving the top.

1.2 Expected risk

① **Intuition.** The average loss your model would suffer if you could deploy it forever on the real world's data. This is the number you actually care about and can never compute.

② **Definition.** For a data distribution $P(X, Y)$ and loss ℓ, the **expected risk** (true risk, population risk) of a predictor f is its expected loss on a fresh draw from P. The **Bayes predictor** f^* minimizes it over *all* functions; its risk R^* is the **Bayes risk**, the irreducible error.

③ **Math.**

$$R(f) = \mathbb{E}_{(x,y)\sim P}\big[\ell(f(x), y)\big] = \int \ell(f(x), y)\, dP(x, y), \qquad R^* = \inf_f R(f)$$

For squared loss the Bayes predictor is the conditional mean, $f^*(x) = \mathbb{E}[Y \mid X = x]$. For 0-1 loss it is the most probable class, $f^*(x) = \arg\max_k P(Y = k \mid X = x)$ — the **Bayes classifier**.



Real-life analogy: a forecaster's lifetime score

Think of it like judging a Chennai weather forecaster by her average error over *every day that will ever come*, not just the days you checked. Common kinds of days count more; a wrong call about snow costs almost nothing, because Chennai never gets snow. Even a perfect forecaster with the same instruments will sometimes miss, because weather is partly random: that floor is the Bayes risk.

Formula, read aloud: $R(f) = \mathbb{E}_{(x,y)\sim P}[\ell(f(x), y)]$ says: the true risk of f = its penalty, averaged over every situation the world can produce, with common situations weighted more. $R^* = \inf_f R(f)$ is the best score *any* predictor could get.

Where it breaks: a forecaster can be watched for decades; $R(f)$ averages over an idealized, fixed world that you never fully observe.

④ **Picture.** Imagine the joint density $P(x, y)$ as a landscape; $R(f)$ weights your loss at each point by how much probability mass sits there. Errors in regions where no data ever occurs cost nothing.

⑤ Example

Predict whether a coin flip lands heads from the coin's serial number (irrelevant feature). Every predictor has 0-1 risk 0.5, so $R^* = 0.5$. No model, however deep, can beat this: the **Bayes error is a property of the data**, not of the model.

⑥ Real-world failure

A team spent months pushing a medical-image model from 91% to 92% agreement with radiologists. Later analysis showed inter-radiologist agreement was ~90%: the labels' own noise put the Bayes error near where they already were. The remaining "errors" were label noise, and the gains were overfitting to one annotator's habits.

⑦ Misconception

"With enough data and a big enough model, error goes to zero." Only if $R^* = 0$, i.e. Y is a deterministic function of X. With overlapping classes or noisy labels, the floor is $R^* > 0$.

⑧ Interview question: "What is the lowest error any model could achieve on this problem?"

⑨ **Strong answer.** "The Bayes error — the expected loss of the Bayes-optimal predictor, which for classification picks the most likely class given the features. It's nonzero whenever the same features can come with different labels, from genuine randomness, missing features, or label noise. You can estimate a floor in practice through human inter-annotator agreement, or by checking duplicated inputs with conflicting labels. It matters because it tells you when to stop tuning the model and start improving the features or the labels."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"How would you reduce the Bayes error?" — Sketch: you can't with a better model; it's fixed for a given feature set. You change the problem: add informative features (new information about Y), clean the labels, or redefine the target.*

Deeper: *"Can adding a feature ever increase the Bayes error?" — Sketch: no. Conditioning on more information can't make the optimal predictor worse — at worst it ignores the feature. But it can increase the error of a learned model with finite data (more variance, curse of dimensionality). That's exactly the gap between R^* and what we achieve.*

⑫ **When the assumption breaks.** "The" distribution P assumes a fixed world. If P changes over time, "expected risk" must be defined over the deployment distribution, not the training one — and the two can differ a lot.

1.3 Empirical risk minimization (ERM)

① **Intuition.** We can't average over the real world, so we average over the sample we have and pick the function that does best there.

② **Definition.** Given IID samples $\{(x_i, y_i)\}_{i=1}^n$ and a hypothesis space $\mathcal{H}$, ERM chooses the $f \in \mathcal{H}$ that minimizes the **empirical risk** — the average training loss. **Regularized ERM** (structural risk minimization) adds a complexity penalty.

③ **Math.**

$$\hat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i), \qquad \hat{f} = \arg\min_{f \in \mathcal{H}} \hat{R}_n(f) + \lambda\,\Omega(f)$$

By the law of large numbers, $\hat{R}_n(f) \to R(f)$ for any **fixed** f. The catch: $\hat{f}$ is not fixed — it was *chosen* using the same data, which is why $\hat{R}_n(\hat{f})$ is optimistically biased.

Real-life analogy: hiring a singer from one audition

Think of it like hiring the best of 50 singers from one song each. One audition (the training sample) stands in for years of real singing (the true risk). The winner's score is flattering: part of it was a lucky day. Audition 500 singers and the winner's luck grows even bigger. That is why training error is optimistic for the model you picked.

Formula, read aloud: $\hat{f} = \arg\min_{f\in\mathcal{H}} \hat{R}_n(f) + \lambda\Omega(f)$ says: from the allowed menu $\mathcal{H}$, pick the function with the smallest average training mistake, plus a fine (λ times Ω) for being too complicated.

Where it breaks: you can hold a fresh audition; in ML that is held-out data, and reusing it to choose makes it flatter the winner too.

④ **Picture.** Draw the true risk curve $R(f)$ over hypothesis space and a wiggly, noisy estimate $\hat{R}_n(f)$ around it. ERM picks the lowest point of the *noisy* curve. The more hypotheses you search, the more likely you find a point where noise dipped low — that dip is overfitting.

⑤ Example

Linear regression is ERM with squared loss over linear functions. Logistic regression is ERM with log loss over linear log-odds functions. A neural net trained with Adam is (approximate) regularized ERM over a huge $\mathcal{H}$. Almost every supervised method you know is ERM with a particular ($\mathcal{H}$, ℓ, Ω).

⑥ Real-world failure

A team tried 4,000 feature subsets and picked the one with the best validation score. The "winning" subset gave +3% on validation and −1% in production. They had done ERM over *feature subsets* on the validation set — the selection itself overfit the validation noise.

⑦ Misconception

"Training loss is an estimate of test loss." It's a *biased* estimate for the chosen model, because the model was selected to make it small. Only data untouched by any selection gives an unbiased estimate.

⑧ Interview question: "Why does minimizing training error not guarantee low test error?"

⑨ **Strong answer.** "Because training error is evaluated on the same data used to choose the model. For any single fixed model the training average converges to the true risk, but ERM *searches* over many models and picks the one whose sample error happens to be lowest — partly due to real quality, partly due to noise fitting. The richer the hypothesis space relative to the sample size, the larger that optimistic bias. That's why we need held-out data, regularization, and capacity control — generalization bounds make this precise: the gap grows with hypothesis-space complexity and shrinks like one over root n."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: "Give me a bound." — Sketch: for a finite $\mathcal{H}$ and a loss in $[0,1]$, Hoeffding plus a union bound give, with probability $\geq 1-\delta$, simultaneously for all $f \in \mathcal{H}$: $R(f) \leq \hat{R}_n(f) + \sqrt{\frac{\ln|\mathcal{H}| + \ln(1/\delta)}{2n}}$. Bigger $\mathcal{H}$, bigger gap; more data, smaller gap.

Deeper: "Neural nets have effectively infinite $\mathcal{H}$ and can fit random labels. Why do they generalize?" — Sketch: worst-case capacity bounds are vacuous for them. Generalization comes from **what the optimizer actually finds** within $\mathcal{H}$ — SGD's implicit bias toward simple/flat/low-norm solutions — plus architecture priors (convolution, attention) and data structure. Norm-, margin- and compression-based bounds do better than parameter counts. Honest answer: this is still an open research area.

⑫ **When the assumption breaks.** ERM's justification is the LLN over **IID** samples from the **test** distribution. With heavy-tailed losses, the empirical average converges slowly (a few huge losses dominate); with dependent samples (time series, users with many rows) the effective sample size is far smaller than n; with shift, you minimize the wrong average — then reweighting (importance weighting) or robust objectives (e.g. distributionally robust optimization, worst-group loss) are used instead.

1.4 Generalization

① **Intuition.** A model generalizes when what it learned about the training data is also true of new data — it learned the *pattern*, not the *sample*.

② **Definition.** The **generalization gap** is the difference between expected risk and empirical risk of the learned model. A learning algorithm generalizes if this gap goes to zero as n grows.

③ **Math.**

$$\text{gap}(\hat{f}) = R(\hat{f}) - \hat{R}_n(\hat{f}), \qquad R(\hat{f}) = \underbrace{\hat{R}_n(\hat{f})}_{\text{fit}} + \underbrace{\text{gap}(\hat{f})}_{\text{generalization}}$$

Test risk is small only when **both** terms are small. Training error measures the first term; a held-out set estimates their sum.



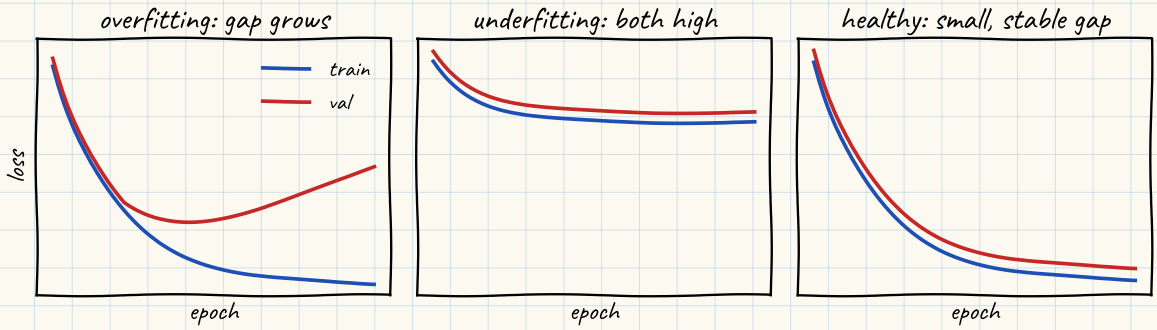
Real-life analogy: learning the route vs learning to drive

Think of it like two learner drivers who both pass the test on the same route. One memorised it ("brake at the blue gate, turn left at the chai shop"). The other learned how roads work: signals, lanes, mirrors. On the test route both look perfect (low training error). On a new road only the second copes: the first had a huge gap between the practice route and real driving.

Formula, read aloud: $R(\hat{f}) = \hat{R}_n(\hat{f}) + \text{gap}(\hat{f})$ says: how you'll do on new roads = how you did on the practice route + the penalty for things that were only true of that route.

Where it breaks: generalization is only promised for new data from the *same* distribution; driving in another country (distribution shift) is a different problem.

④ **Picture.** Two curves over training epochs: training loss falls monotonically; validation loss falls, flattens, then may rise. The vertical distance between them is the (estimated) gap.



Learning curves: the gap between train and validation tells you which failure you have

⑤ Example

A spam classifier memorizes that emails containing a specific campaign ID are spam: perfect training accuracy, and it catches that one campaign. Next week spammers change IDs; accuracy collapses. A model relying on generic cues (link-to-text ratio, sender reputation) has a slightly higher training error but a far smaller gap.

⑥ Real-world failure

Pneumonia classifiers trained on chest X-rays from one hospital generalized poorly to others. They had learned hospital-specific markers (scanner tokens, portable-scanner artifacts correlating with sicker inpatients) — a "shortcut" that is predictive within one distribution and meaningless across distributions.

⑦ Misconception

"A small train–validation gap means the model generalizes." It means it generalizes **to the validation distribution**. If validation was a random split of the same biased source, both can be equally wrong about production.

⑧ Interview question: "What does it mean for a model to generalize, and how do you measure it?"

⑨ **Strong answer.** "Generalization means low expected loss on new data from the deployment distribution, not just on training data. Formally, test risk equals training risk plus the generalization gap, so you need both a good fit and a small gap. You estimate it with data the model and every modelling decision never touched — a test set drawn to match deployment: time-based for temporal problems, group-held-out when the same users or patients repeat. Then I'd look at slices, because an average can hide a segment where the model doesn't generalize at all."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Your validation loss rises while training loss falls. What's happening mechanistically?" — Sketch: the model has begun fitting sample-specific structure (noise, rare coincidences) that reduces training loss but doesn't hold elsewhere. Options: early stopping, regularization, more data, augmentation, less capacity.*

Deeper: *"Can a model with zero training error generalize well?" — Sketch: yes — "benign overfitting". Over-parameterized models can interpolate noisy labels and still have low test error when the extra capacity spreads the noise-fitting across many directions that barely affect predictions. This is part of the* ***double descent*** *phenomenon: test error can fall again past the interpolation threshold.*

⑫ **When the assumption breaks.** Generalization guarantees are about the *same* distribution. Out-of-distribution generalization (new hospitals, new countries, new product surfaces) needs extra structure: invariant features, domain adaptation, or explicit data from the new domain.

1.5 Hypothesis spaces

① **Intuition.** The menu of functions the algorithm is allowed to pick from. Choose the menu before you choose the dish.

② **Definition.** A **hypothesis space** $\mathcal{H}$ is the set of candidate functions a learning algorithm can output — e.g. all linear functions $\{x \mapsto \mathbf{w}^\top \mathbf{x} + b\}$, all trees of depth ≤ 5, all networks of a given architecture.

③ **Math.** Size and "richness" of $\mathcal{H}$ are measured by cardinality $|\mathcal{H}|$ (finite case), **VC dimension** (the largest set of points $\mathcal{H}$ can label in every possible way — linear classifiers in $\mathbb{R}^d$ have VC dimension $d+1$), or Rademacher complexity (how well $\mathcal{H}$ can correlate with random noise):

$$\mathfrak{R}_n(\mathcal{H}) = \mathbb{E}_{\sigma,S}\Big[\sup_{f\in\mathcal{H}} \frac{1}{n}\sum_{i=1}^{n} \sigma_i f(x_i)\Big], \quad \sigma_i \in \{\pm 1\} \text{ uniformly at random}$$

Real-life analogy: a tailor's pattern book

Think of it like the shapes a tailor is able to cut. A shop selling only S, M and L (a small space) may fit nobody exactly: approximation error. A tailor who can cut *any* shape from one fitting (a huge space) will copy the odd way you slouched that day: estimation error. A good pattern book fits real bodies, not every wrinkle.

Formula, read aloud: $\mathfrak{R}_n(\mathcal{H})$ says: give each data point a random coin-flip label $\sigma_i = \pm 1$, let the best function in $\mathcal{H}$ try to agree with those fake labels, and average how well it manages. A space that can chase pure coin flips is dangerously flexible.

Where it breaks: a tailor's options are easy to see; a model's real flexibility is often hidden, and two models with the same number of parameters can differ hugely.

④ **Picture.** Nested sets: linear ⊂ quadratic ⊂ cubic ⊂ … The true function f^* may lie inside or outside. A small set may not contain anything near f^* (approximation error); a huge set contains many functions that fit the sample by accident (estimation error).

⑤ Example

Polynomial regression of degree k in 1-D: $\mathcal{H}_k$ has $k+1$ parameters. With 10 points, degree 9 interpolates perfectly — the space is rich enough to fit **any** labels, including pure noise. That's the red flag: if $\mathcal{H}$ can fit random labels, a perfect training fit tells you nothing.

⑥ Real-world failure

Forecasting sales with a linear model in time when demand is multiplicative and seasonal: no member of $\mathcal{H}$ can express "×1.8 in December", so residuals show clear yearly structure. No amount of data fixes it — the hypothesis space is wrong. Fix: log-transform the target (multiplicative → additive) or add seasonal features.

⑦ Misconception

"Number of parameters = capacity." Capacity depends on how parameters are used. $\sin(\omega x)$ has one parameter and infinite VC dimension; a deep net with weight sharing can have fewer effective degrees of freedom than its parameter count suggests.

⑧ Interview question: "How do you choose the hypothesis space for a problem?"

⑨ **Strong answer.** "From what I know about the structure of the problem and how much data I have. If the relationship is roughly additive and data is small, linear models with good features. If there are interactions and mixed feature types in tabular data, tree ensembles. If the input has spatial or sequential structure — images, text, audio — architectures that build that in, like convolutions or attention. Then I control capacity within the space by regularization and validation. The key trade-off is approximation error — can the space express the truth — versus estimation error — can I pick the right member with this much data."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"What's the VC dimension of a linear classifier in 2-D, and why?" — Sketch: 3. Any 3 non-collinear points can be labeled in all 2^3 ways by a line, but no 4 points can (XOR configuration fails for 4 points in general position — one labeling always requires a non-linear boundary).*

Deeper: *"If kernels map to infinite dimensions, why don't SVMs overfit catastrophically?" — Sketch: the effective capacity is controlled by the* ***margin****, not the dimension: margin-based bounds depend on R^2/γ^2 (data radius over margin squared), independent of the feature dimension. Maximizing the margin is capacity control.*

⑫ **When the assumption breaks.** If the true function lies far outside $\mathcal{H}$, even infinite data gives a biased model — this is **misspecification**. Symptoms: structured residuals, systematic errors on identifiable slices, learning curves that plateau at a high error.

1.6 Inductive bias

① **Intuition.** The assumptions a learner uses to guess beyond the data. Without them, the training set says nothing about unseen points.

② **Definition. Inductive bias** is the set of assumptions — encoded in the hypothesis space, the loss, the regularizer, the optimizer and the architecture — that makes a learner prefer some generalizations over others consistent with the same data.

③ **Math.** In Bayesian terms, inductive bias is the **prior** $p(f)$: the posterior $p(f \mid D) \propto p(D \mid f)\, p(f)$ ranks functions that fit the data equally well by their prior plausibility. Regularizers are negative log priors: an L2 penalty $\lambda\|\mathbf{w}\|^2 \Leftrightarrow$ Gaussian prior $\mathbf{w} \sim \mathcal{N}(0, \frac{1}{2\lambda} I)$ (up to noise scaling).

Real-life analogy: a detective's hunches

Think of it like a detective with a few clues that fit many possible stories. The clues alone can't choose; her experience does ("the simplest story is usually right", "people act for a motive"). Those built-in hunches are the inductive bias. A detective with no hunches could never name a suspect, and one with the wrong hunches confidently picks the wrong one.

Formula, read aloud: $p(f \mid D) \propto p(D \mid f)\, p(f)$ says: how much I believe a story after the clues is proportional to (how well it explains the clues) × (how plausible it was before any clues). The prior $p(f)$ is the hunch.

Where it breaks: a detective can revise her hunches mid-case; a learner's bias is fixed before training, and fails wherever the world breaks it (a CNN meeting rotated photos).

④ **Picture.** Four training points; infinitely many curves pass through them. The inductive bias is the rule that picks one: "smoothest", "straightest", "fewest pieces", "nearest neighbor's label", "translation invariant".

⑤ Example

Model	Inductive bias
Linear regression	output changes linearly and additively in each feature
k-NN	nearby points (in the chosen metric) have similar labels
Decision tree	axis-aligned, piecewise-constant structure; few relevant features
CNN	locality + translation equivariance + hierarchy of features
RNN / Transformer	sequential structure / content-based pairwise interactions
Naive Bayes	features independent given the class
L2 / weight decay	small weights ⇒ smooth functions

⑥ Real-world failure

A CNN trained to classify defects was rotated 90° in production because a camera was remounted. CNNs are translation-equivariant but **not** rotation-invariant: the inductive bias didn't cover the change, and accuracy fell sharply. Fix: rotation augmentation (teaches the invariance through data) or a rotation-equivariant architecture.

⑦ Misconception

"The best model has no assumptions." A learner with no inductive bias cannot generalize at all — it has no reason to prefer any label on unseen points (this is the core of No Free Lunch). The goal is **the right** assumptions, not none.

⑧ Interview question: "Why do CNNs work better than MLPs on images?"

⑨ **Strong answer.** "Inductive bias. Convolutions assume locality — nearby pixels are related — and weight sharing assumes a pattern useful in one place is useful everywhere, giving translation equivariance. That cuts parameters by orders of magnitude and shrinks the hypothesis space to functions that respect image structure, so we need much less data to find a good one. An MLP could represent the same functions but has to learn locality and translation from data, which needs far more samples. With huge

data, weaker-bias models like Vision Transformers catch up or win — the bias helps most when data is limited."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Then why did Vision Transformers win?" — Sketch: at very large data scale, a strong bias becomes a constraint (approximation error) while the data can teach the useful structure; ViTs have weaker bias, so they underperform CNNs on small data and match or beat them with large-scale pretraining or heavy augmentation.*

Deeper: *"Is data augmentation an inductive bias?" — Sketch: yes — it encodes invariances ("a flipped cat is still a cat") through the data rather than the architecture. It's equivalent in spirit to a prior that the function is invariant to that transformation.*

⑫ **When the assumption breaks.** When the data violates the bias — non-local dependencies for CNNs, strong feature interactions for linear models, dependent features for Naive Bayes — the model either needs much more data or fails systematically. Diagnose by asking: *what would this model have to assume to get these errors?*

1.7 Training, validation and test sets

① **Intuition.** Three jobs need three datasets: *learn* the parameters, *choose* among models and hyperparameters, and *report* how well the final choice will do. Any data used for a job is spent for the jobs after it.

② **Definition. Training set:** fits parameters. **Validation set** (dev set): used for model selection, hyperparameter tuning, early stopping, threshold choice. **Test set:** used once, at the end, for an unbiased estimate of the final model's performance.

③ **Math.** If you evaluate K candidate models on the same validation set and pick the best, the expected best score is optimistically biased: with independent estimation noise of standard deviation s, the expected maximum of K noisy scores exceeds the true best by roughly $s\sqrt{2\ln K}$. The more you look, the more you overfit the validation set.



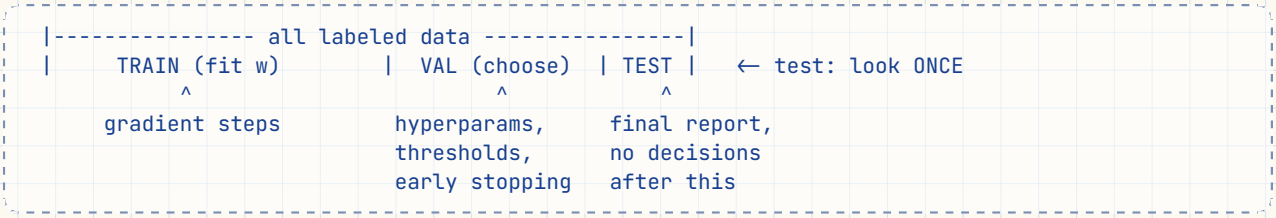
Real-life analogy: practice, family tasting, the judges

Think of it like preparing for a cooking competition. You practise at home to learn the dish (training). Your family tastes versions and you keep the one they like best (validation). The judges taste once, on the day (test). Try 50 versions on your family and part of their verdict is just their mood that evening, so the winner looks better than it is. Keep asking the judges and tweaking, and they quietly become your family.

Formula, read aloud: after picking the best of K models, the winning score overstates the truth by about $s\sqrt{2\ln K}$: (the noise in one score) × (a factor that grows slowly with how many you tried).

Where it breaks: a test set never wears out physically; it loses its honesty only because your decisions start to depend on it.

④ **Picture.**



⑤ Example

Typical splits: 80/10/10 for medium data; with 10M+ examples, 98/1/1 is fine — 100K test examples give a standard error on accuracy of about $\sqrt{0.9 \cdot 0.1/10^5} \approx 0.001$. With little data, use cross-validation for selection and keep a small test set, or nested CV.

⑥ Real-world failure

A Kaggle-style team submitted to a public leaderboard 300 times, each time keeping changes that raised the score. Final private-leaderboard rank dropped by hundreds of places — the public test set had become a validation set, and they had overfit it. The same thing happens inside companies with a "test set" everyone checks weekly.

⑦ Misconception

"The validation score is my expected production performance." After tuning on it, it's optimistic. Only the untouched test set (or a live experiment) gives an honest number — and only if its distribution matches production.

⑧ Interview question: "Why do we need a separate test set if we already have a validation set?"

⑨ **Strong answer.** "Because the validation set has been used to make decisions — hyperparameters, architecture, early stopping, threshold — so the chosen model is partly fitted to its noise, and its validation score is biased upward. The test set is used once for the final estimate, so it's unbiased. If I keep checking the test set and reacting, it silently becomes a second validation set. At scale, I'd also make sure the test set mirrors production: same time period structure, same user mix, deduplicated against training."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"How big should the test set be?" — Sketch: big enough that its standard error is well below the differences you care about. For a proportion p, SE $= \sqrt{p(1-p)/n}$. To detect a 0.5-point accuracy difference around 90% accuracy with SE $\approx$ 0.1 points, you need $n \approx 0.09/0.001^2 = 90{,}000$.*

Deeper: *"You have 2,000 labeled examples. How do you tune and report honestly?" — Sketch: nested cross-validation: an outer loop for performance estimation and an inner loop for tuning inside each outer training fold. Report the outer mean ± spread; then refit on all data with the selected procedure.*

⑫ **When the assumption breaks.** Random splitting assumes rows are exchangeable. With time dependence, repeated entities (users, patients), or near-duplicates, random splits leak information and inflate every score — see Part 17 for time-based and grouped splits.

1.8 The IID assumption

① **Intuition.** Every training example is a fresh, independent draw from the same process that will generate future examples.

② **Definition. Independent and identically distributed:** samples (x_i, y_i) are mutually independent and each follows the same joint distribution P — and, for deployment, test data follows that same P.

③ **Math.**

$$P\big((x_1, y_1), \ldots, (x_n, y_n)\big) = \prod_{i=1}^{n} P(x_i, y_i)$$

This is what makes the likelihood factor into a product, the loss into a sum, and the LLN / CLT apply to empirical averages.

Real-life analogy: a survey at a railway station

Think of it like surveying passengers at a railway station to plan tomorrow's trains. IID means each person you ask is picked at random, on their own, from the same crowd that will be there tomorrow. It breaks if you interview one family ten times (not independent), survey during the Diwali rush but plan for an ordinary Tuesday (not identically distributed), or if your questions change who stops to talk (feedback).

Formula, read aloud: $P\big((x_1, y_1), \ldots, (x_n, y_n)\big) = \prod_i P(x_i, y_i)$ says: the chance of the whole dataset = each example's chance multiplied together, as if each were a fresh draw with no memory of the others. That product is why the loss becomes a sum.

Where it breaks: a survey can be made random by design; logged ML data usually can't, because you inherit whatever process produced it.

④ **Picture.** Balls drawn with replacement from one urn — the same urn used tomorrow. Violations: the urn changes (non-identical), or drawing one ball changes the next (non-independent).

⑤ Example

Click logs violate IID in three ways at once: many rows per user (dependent), rows ordered in time with trends and seasonality (non-identical over time), and the model's own recommendations determine what gets logged (feedback — the data depends on the previous model).

⑥ Real-world failure

A churn model's cross-validated AUC was 0.91 using random row splits. Each customer had ~50 monthly rows, so the same customers appeared in train and validation; the model learned customer identity through stable features. With a customer-grouped split, AUC was 0.74. The IID assumption had silently multiplied the apparent sample size by 50.

⑦ Misconception

"IID is only a theory assumption; it doesn't matter in practice." It determines whether your validation numbers mean anything. Most "offline great, online bad" stories are IID violations.

⑧ Interview question: "Where does the IID assumption show up in ML, and what do you do when it's violated?"

⑨ **Strong answer.** "It shows up in three places: the likelihood factorizing into a product so the loss is a sum over examples; empirical averages converging to expectations, which justifies ERM and validation; and the test set representing deployment. It's violated by temporal dependence, grouped data, feedback loops and distribution shift. Remedies depend on which part breaks: time-based or grouped splits for evaluation, features that capture temporal context, sequence models, reweighting for shift, and for feedback loops, exploration or randomized logging so the data isn't only what the previous model chose."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: "What is the effective sample size with repeated measures?" — Sketch: with m rows per group and intra-group correlation ρ, the design effect is $1 + (m-1)\rho$, so $n_{\text{eff}} = n/(1+(m-1)\rho)$. With $m = 50$, $\rho = 0.5$: $n_{\text{eff}} \approx n/25$.

Deeper: "Is SGD's mini-batch sampling IID?" — Sketch: sampling without replacement (epochs of shuffled data) is not strictly IID, and non-shuffled or sorted data (e.g. by time or class) creates correlated batches that bias gradients and can destabilize training — which is why we shuffle and why BatchNorm behaves badly with non-random batches.

⑫ **When the assumption breaks.** Time series → forecasting models and walk-forward validation. Groups → mixed-effects models, group splits. Feedback loops → counterfactual evaluation with propensities, exploration. Shift → next section.

1.9 Distribution shift

① **Intuition.** The world the model is deployed into is not the world it was trained on.

② **Definition.** The deployment distribution $P_{\text{test}}(x, y)$ differs from $P_{\text{train}}(x, y)$. Decomposing $P(x, y) = P(x)P(y \mid x) = P(y)P(x \mid y)$ gives the standard types:

Type	What changes	What stays	Example
Covariate shift	$P(x)$	$P(y \mid x)$	model trained on young users, deployed to older ones
Label (prior) shift	$P(y)$	$P(x \mid y)$	disease prevalence rises during an outbreak
Concept drift	$P(y \mid x)$	—	fraudsters change tactics; the same behavior now means something else

③ **Math.** Under covariate shift, importance weighting recovers the test risk from training data:

$$R_{\text{test}}(f) = \mathbb{E}_{P_{\text{test}}}[\ell] = \mathbb{E}_{P_{\text{train}}}\left[\frac{P_{\text{test}}(x)}{P_{\text{train}}(x)}\,\ell(f(x), y)\right]$$

The density ratio can be estimated by training a classifier to distinguish train from test inputs: $\frac{P_{\text{test}}(x)}{P_{\text{train}}(x)} = \frac{P(\text{test}|x)}{P(\text{train}|x)} \cdot \frac{n_{\text{train}}}{n_{\text{test}}}$.



Real-life analogy: a kirana store when the season changes

Think of it like a kirana shopkeeper who learned what sells during summer. Then things change. *Covariate shift:* a hostel opens nearby, so more students walk in, but each student buys what students always bought. *Label shift:* flu season brings more flu customers, yet a flu customer looks the same as before. *Concept drift:* the same behaviour now means something new, like a big rice order that once meant a wedding and now means panic buying.

Formula, read aloud: $R_{\text{test}}(f) = \mathbb{E}_{P_{\text{train}}}\left[\frac{P_{\text{test}}(x)}{P_{\text{train}}(x)}\ell\right]$ says: to estimate the loss in the new world, reuse old examples but weight each by how much more (or less) common that kind of customer is now.

Where it breaks: the shopkeeper sees new customers at once; a model often gets true labels weeks later, so concept drift stays hidden until they arrive.

④ **Picture.** Two overlapping clouds of x: training mass concentrated on the left, deployment mass on the right. Where they don't overlap, the model is extrapolating and nothing in the training loss constrained its behavior there.

⑤ Example

Adversarial validation: label training rows 0 and recent production rows 1, train a classifier. AUC ≈ 0.5 → no detectable shift. AUC 0.85 → strong shift; the classifier's top features tell you which inputs moved (e.g. a new app version changed a logging field).

⑥ Real-world failure

COVID-19 (2020) broke many demand-forecasting, credit and fraud models simultaneously: purchase patterns (covariate shift), default rates (label shift) and the meaning of signals — e.g. sudden online purchases by older customers stopped being suspicious (concept drift). Teams with drift monitoring and fast retraining recovered in weeks; others in months.

⑦ Misconception

"If input distributions haven't changed, the model is fine." Concept drift changes $P(y \mid x)$ with **no change in** $P(x)$. Only labels (possibly delayed) or proxy outcome metrics reveal it.

⑧ Interview question: "Your model's accuracy dropped after deployment. How do you tell which kind of shift happened?"

⑨ **Strong answer.** "Compare distributions piece by piece. First inputs: per-feature drift statistics like PSI or KS tests, and a train-vs-production domain classifier — that detects covariate shift. Then the prediction distribution and, once labels arrive, the label rate — a changed base rate with stable class-conditional features suggests label shift, which I can correct by re-estimating priors and adjusting the output. If inputs look stable but error rises on fresh labels, $P(y \mid x)$ changed — concept drift — which needs retraining on recent data. Also rule out the boring cause first: a pipeline bug that changed a feature's meaning looks exactly like drift."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"How do you correct label shift without new labels?" — Sketch: if $P(x \mid y)$ is stable, adjust posteriors: $p_{\text{new}}(y \mid x) \propto p_{\text{old}}(y \mid x)\, \frac{\pi_{\text{new}}(y)}{\pi_{\text{old}}(y)}$. Estimate π_{new} with EM on unlabeled data (Saerens et al.) or with confusion-matrix methods (black-box shift estimation).*

Deeper: *"When does importance weighting fail?" — Sketch: when supports don't overlap (ratios infinite/undefined) or overlap weakly, so a few examples get huge weights and the variance explodes. Effective sample size is $(\sum w_i)^2 / \sum w_i^2$. Clip or normalize weights, or accept you need data from the new region.*

⑫ **When the assumption breaks.** All the fixes above assume you know *which* factorization is stable. Real shifts are mixtures, and the model's own actions can cause them (performative prediction: a credit model changes who applies). Monitoring plus regular retraining on recent data is the robust default.

1.10 No Free Lunch (intuition)

① **Intuition.** No learning algorithm is best for every problem. An algorithm that wins on some problems must lose on others.

② **Definition.** Wolpert's No Free Lunch theorem for supervised learning: averaged uniformly over **all possible target functions**, every learning algorithm has the same expected **off-training-set** error.

③ **Math.** For binary labels on a finite input space, if all $2^{|X|}$ labelings are equally likely, then on any point not in the training set each label is equally likely whatever you observed — so every algorithm's expected off-training-set accuracy is exactly $1/2$.

Real-life analogy: no shoe for every ground

Think of it like shoes. Spikes win on a running track, boots win in mud, sandals win on the beach. Average over *every conceivable* ground, including ice, lava and nails, and every shoe comes out equally bad. But real life is mostly roads, fields and floors, so choosing the right shoe for the ground you actually walk on matters a lot.

Formula, read aloud: if all $2^{|X|}$ ways of labelling the inputs are equally likely, then on any unseen point each label is a coin flip, so every algorithm's expected accuracy there is exactly $1/2$.

Where it breaks: you usually know which ground you'll walk on; in ML you often don't know the problem's structure up front, which is why validation on realistic data makes the final call.

④ **Picture.** A seesaw: whatever an algorithm gains on problems matching its assumptions it loses on problems that violate them. The world is not uniform over problems — real data has smoothness, sparsity, locality — so good inductive biases win *on the problems we actually face*.

⑤ Example

Gradient-boosted trees dominate on typical tabular business data; CNNs dominate on natural images; linear models win on very high-dimensional sparse text with limited data. Each is best where its assumptions hold — exactly what NFL predicts.

⑥ Real-world failure

An org mandated "deep learning for everything". On a 20K-row tabular risk problem, the neural model lost to logistic regression with engineered features and to gradient boosting, while costing 10× more to maintain. The mandate assumed a universally best algorithm.

⑦ Misconception

"NFL means all algorithms are equally good in practice." NFL averages over **all** possible problems, most of which are pure noise. Real problems are highly structured, so algorithm choice matters enormously.

⑧ Interview question: "What does the No Free Lunch theorem tell a practitioner?"

⑨ **Strong answer.** "That there is no universally best model — performance comes from matching the algorithm's assumptions to the structure of the data. Practically: start from a strong simple baseline, pick model families whose inductive biases fit the data type, and let validation on data resembling production decide. It's a caution against 'model X always wins' claims, not a claim that choice doesn't matter."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Then why do gradient-boosted trees win so often on tabular data?" — Sketch: their biases fit typical tabular data: heterogeneous feature scales, irregular non-smooth target functions, many uninformative features (trees ignore them), interactions, and missing values. Neural nets' bias toward smooth functions is a poorer match there.*

Deeper: *"Does NFL apply to hyperparameter tuning and AutoML?" — Sketch: yes, in principle no search strategy is best on all objective functions; in practice smooth-ish, low-effective-dimension hyperparameter landscapes make random search and Bayesian optimization reliably better than grid search.*

⑫ **When the assumption breaks.** NFL's uniform prior over problems is the "assumption"; the real world violates it, which is precisely why learning is possible.

1.11 Bias–variance tradeoff

① **Intuition.** Error has two learnable sources. **Bias:** the model is systematically wrong because it's too rigid. **Variance:** the model's answer swings wildly depending on which training sample you happened to get. Reducing one tends to increase the other.

② **Definition.** For a point x, imagine retraining on many independent training sets D. **Bias** is how far the *average* prediction is from the truth; **variance** is how much predictions spread around their own average; **noise** is the irreducible variability of y given x.

③ **Math.** Let $y = f(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\mathrm{Var}(\varepsilon) = \sigma^2$, and $\bar{f}(x) = \mathbb{E}_D[\hat{f}_D(x)]$. For squared loss:

$$\mathbb{E}_{D,\varepsilon}\left[(y - \hat{f}_D(x))^2\right] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{\left(f(x) - \bar{f}(x)\right)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_D\left[(\hat{f}_D(x) - \bar{f}(x))^2\right]}_{\text{variance}}$$

(Full derivation on the whiteboard in Part 31.) The cross terms vanish because ε is independent of D with mean zero, and $\mathbb{E}_D[\hat{f}_D - \bar{f}] = 0$.



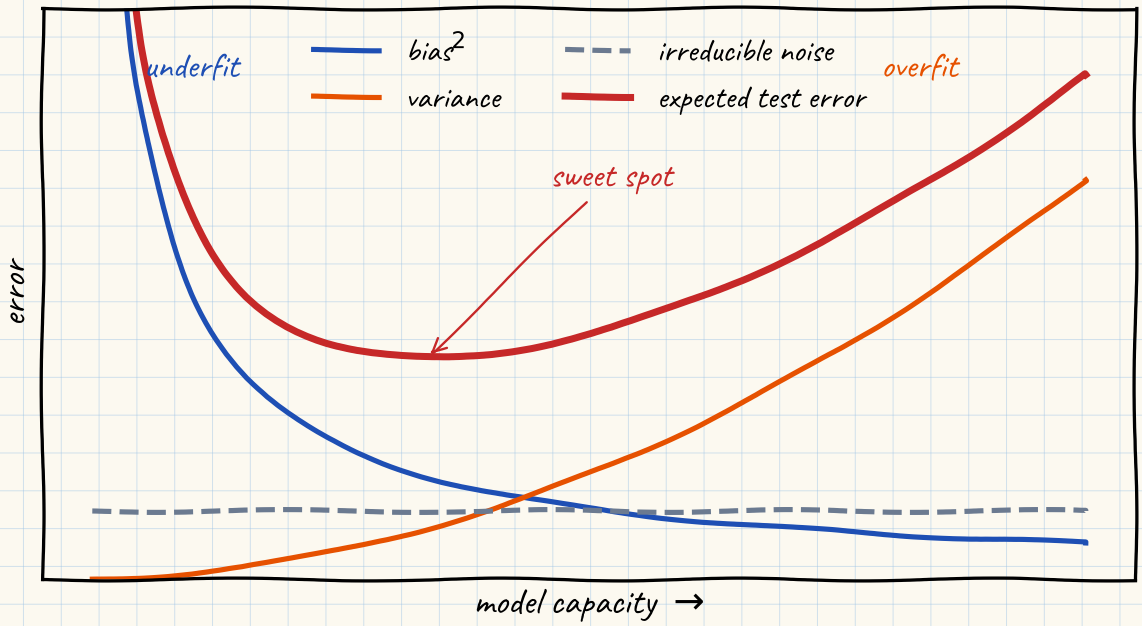
Real-life analogy: two kinds of bad bowlers

Think of it like watching bowlers aim at the stumps over many overs (many training sets). A *biased* bowler lands every ball in a tight group, but always outside off stump. A *high-variance* bowler is on target on average, but sprays the ball everywhere. And cracks in the pitch deflect even a perfect ball: that's the noise nobody can remove.

Formula, read aloud: expected squared error = σ^2 (the pitch cracks) + bias^2 (how far the *average* landing spot is from the stumps, squared) + variance (how scattered the balls are around their own average).

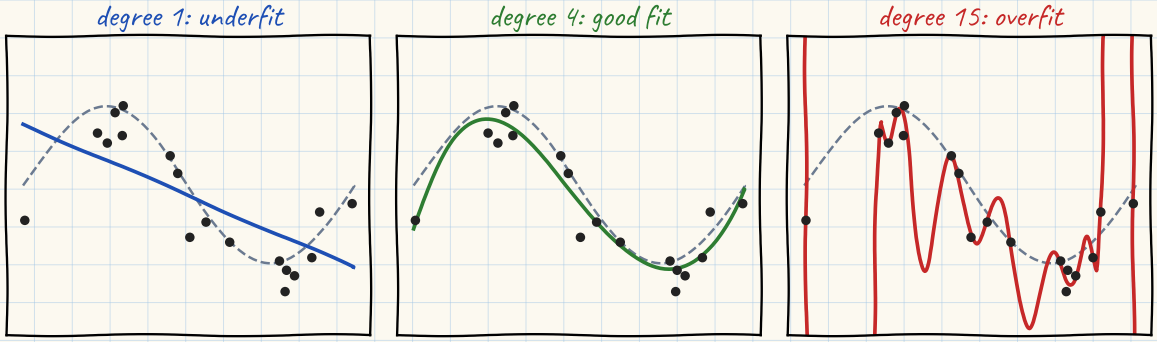
Where it breaks: a bowler bowls hundreds of balls you can watch; you usually train one model on one dataset, so you never see bias and variance directly and must infer them from training vs validation error.

④ **Picture.**



Classic U-curve: bias falls and variance rises with capacity

Dartboard version: high bias = tight cluster, off-center; high variance = centered on average but scattered; the goal is tight *and* centered.



Underfit, good fit and overfit polynomial fits to the same noisy sine data

⑤ Example

k-NN regression: with $k = 1$, the prediction is one neighbor's noisy label — low bias, high variance (variance $\approx \sigma^2$). With k neighbors, variance of the average noise drops to σ^2/k, but neighbors get farther away, so bias grows. With $k = n$, you predict the global mean: zero variance from the sample choice (almost), maximal bias.

⑥ Real-world failure

A team kept adding features and trees to a fraud model because each addition raised training AUC. Production precision fell: they were buying variance. The diagnostic they skipped: the train-validation gap had widened from 0.01 to 0.08 AUC.

⑦ Misconception

"Bias–variance is always a strict tradeoff along one knob." Some changes reduce both — **more data** reduces variance at no bias cost; **better features** reduce bias without adding variance; **ensembling** (bagging) reduces variance at little bias cost. And in overparameterized regimes, test error can fall again as capacity grows (double descent).

⑧ Interview question: "Explain the bias–variance tradeoff."

⑨ **Strong answer.** "Expected squared error at a point decomposes into noise, bias squared and variance. Bias is the error of the *average* model over training sets — from wrong assumptions, like fitting a line to a curve. Variance is how much the learned model changes when the training set changes — from too much flexibility relative to data. Increasing capacity typically lowers bias and raises variance, giving a U-shaped test error. In practice I diagnose with learning curves: high training error means bias — more capacity or features; low training but high validation error means variance — more data, regularization, bagging, early stopping. It's a decomposition of squared loss specifically; for 0-1 loss the decomposition isn't additive in the same way, but the intuition carries over."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Where does regularization strength sit on this?" — Sketch: increasing λ restricts the effective hypothesis space: bias ↑, variance ↓. Ridge shrinks along low-variance directions of the data most (Part 5), which is exactly where estimates are noisiest.*

Deeper: *"Deep nets have enormous capacity. Why isn't their variance enormous?" — Sketch: the classic U-curve assumes variance grows with parameter count. Past the interpolation threshold, among the many solutions that fit the data, SGD tends to find low-norm/flat ones, and variance from the initialization/sampling can decrease with width (averaging over many features). That's double descent; ensembling or wider models reduce the variance peak near the interpolation threshold.*

⑫ **When the assumption breaks.** The clean decomposition needs squared loss and a fixed test point; for other losses there are generalized decompositions but no universal additive form. The *diagnostic use* — train vs. validation error — works regardless.

1.12 Approximation vs. estimation error (and optimization error)

① **Intuition.** Three reasons your model is worse than perfect: the family you chose can't express the truth; you picked the wrong member because data was limited; your optimizer didn't find the best member.

② **Definition.** Let f^* be the Bayes predictor, $f_{\mathcal{H}} = \arg\min_{f \in \mathcal{H}} R(f)$ the best in the class, $\hat{f}_n$ the empirical risk minimizer, and $\tilde{f}_n$ what training actually returned.

③ **Math.** (Bottou & Bousquet's decomposition)

$$R(\tilde{f}_n) - R^* = \underbrace{R(f_{\mathcal{H}}) - R^*}_{\text{approximation}} + \underbrace{R(\hat{f}_n) - R(f_{\mathcal{H}})}_{\text{estimation}} + \underbrace{R(\tilde{f}_n) - R(\hat{f}_n)}_{\text{optimization}}$$

Approximation error falls as $\mathcal{H}$ grows. Estimation error rises with $\mathcal{H}$'s complexity and falls with n. Optimization error falls with compute.

Real-life analogy: finding a flat in a new city

Think of it like house-hunting. *Approximation error:* you only search in one neighbourhood (your model family), and the perfect flat is in another area — no amount of searching fixes that. *Estimation error:* you visited only a few flats (limited data), so you picked one that looked best on the day. *Optimization error:* you got tired and stopped searching before checking all the listings you had (training stopped early).

Formula, read aloud: total shortfall = (best flat in your area vs. best flat in the city) + (the flat you chose vs. best flat in your area) + (the flat you actually signed for vs. the one you meant to choose).

Where it breaks: in ML, widening the search area (a bigger model family) also makes it harder to judge flats from few visits — the two errors pull against each other.

④ **Picture.** A target (the truth) outside a circle (the hypothesis class). Approximation error: distance from the target to the circle's nearest point. Estimation error: distance from that nearest point to where noisy data led you. Optimization error: distance from there to where you stopped.

⑤ Example

Large-scale learning insight: with a fixed **compute budget** and effectively unlimited data, it's often better to run a cheap optimizer (SGD, a few passes) on **more data** than to optimize precisely on less — accepting optimization error to reduce estimation error. That's why one epoch of SGD over billions of ad impressions beats L-BFGS to convergence on a subsample.

⑥ Real-world failure

A team's transformer underperformed a small baseline. They added layers (targeting approximation error). The real problem was optimization: the learning rate was too high without warmup, and the loss plateaued. A learning-rate sweep fixed it — the model had enough capacity all along.

⑦ Misconception

"Bias = approximation error, variance = estimation error." Related but not identical: bias/variance is a decomposition of a *prediction's* squared error over training sets at a point; approximation/estimation decomposes *excess risk* across the whole distribution. The intuitions line up; the definitions differ.

⑧ Interview question: "Your model's test error is too high. How do you decide whether to get more data or a bigger model?"

⑨ **Strong answer.** "Separate the error sources. If training error itself is high, more data won't help much — that's approximation or optimization error: try more capacity, better features, or fix optimization; checking whether a bigger model can even overfit a small subset tells me which. If training error is low but validation is high, it's estimation error — more data, regularization or simpler models help. A learning curve — validation error versus training-set size — makes the call: if it's still falling steeply at the current size, more data pays; if it's flat, it won't."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: "How do you detect optimization error specifically?" — Sketch: the model can't even fit the training set despite enough capacity (can't overfit a batch of 100 examples); loss plateaus early; results vary a lot with learning rate or seed; a different optimizer or learning-rate schedule changes the final training loss.

Deeper: "What do neural scaling laws say about this decomposition?" — Sketch: empirically, loss falls as a power law in parameters, data and compute; for a fixed compute budget there's an optimal split between model size and training tokens (the "Chinchilla" finding was roughly proportional scaling — about 20 tokens per parameter for that setup). Under-trained large models waste compute on approximation capacity they can't estimate well.

⑫ **When the assumption breaks.** The decomposition assumes a fixed distribution; under shift, a fourth term appears — the difference between train-distribution and deployment-distribution risk — and no amount of data from the old distribution reduces it.

1.13 Model capacity

① **Intuition.** How complicated a pattern a model family can fit — how many different labelings it can "explain".

② **Definition. Capacity** (complexity) is the richness of the hypothesis space: formally VC dimension, Rademacher complexity, or number of effective parameters; practically, how easily the model can fit random labels.

③ **Math.** VC generalization bound (binary classification, with probability ≥ $1-\delta$):

$$R(f) \le \hat{R}_n(f) + O\left(\sqrt{\frac{d_{VC}\,\log(n/d_{VC}) + \log(1/\delta)}{n}}\right)$$

For ridge regression, **effective degrees of freedom** $= \sum_j \frac{s_j^2}{s_j^2+\lambda}$ where s_j are singular values of X — between 0 ($\lambda \to \infty$) and d ($\lambda = 0$). Capacity is continuous, not just a parameter count.



Real-life analogy: a tailor's measuring tape

Think of it like tailors with different skills. A tailor who only sells S/M/L shirts (low capacity) fits nobody perfectly but is never wildly wrong. A master tailor who can stitch any shape (high capacity) can match every bump of your body — including the wallet in your pocket on the day you were measured. Capacity is how many different "bodies" a model can fit exactly.

Formula, read aloud: true error ≤ training error + a penalty that grows with capacity (d_{VC}) and shrinks as the number of examples n grows — roughly $\sqrt{\text{capacity}/n}$.

Where it breaks: a tailor's skill is easy to see; a model's real capacity depends on how it is trained (regularization, early stopping), not just its number of parameters.

④ **Picture.** A knob. Turn it up: training error goes monotonically down; test error forms a U (classical regime), sometimes followed by a second descent (modern overparameterized regime).

⑤ Example

Knobs that change capacity: polynomial degree; tree depth and number of leaves; number of trees ×learning rate in boosting; hidden units and layers; k in k-NN (larger k = *lower* capacity); regularization strength (larger = lower); training time (early stopping limits effective capacity); feature count.

⑥ Real-world failure

Zhang et al. (2017) showed standard image CNNs reach ~100% training accuracy on **randomly shuffled** CIFAR-10 labels. Any argument of the form "it fits the training data, so it learned something real" is invalid for such models — capacity alone can explain the fit.

⑦ Misconception

"More parameters → more overfitting, always." Only in the classical regime. With implicit regularization and massive overparameterization, larger models often generalize *better*, and the biggest overfitting risk is near the interpolation threshold (parameters ≈ data points).

⑧ Interview question: "How would you measure the capacity of a model in practice?"

⑨ **Strong answer.** "Theoretically via VC dimension or Rademacher complexity, but in practice I measure what matters: how the train–validation gap behaves as I turn capacity knobs, and a randomization test — train on shuffled labels; if the model fits them easily, training accuracy is meaningless as evidence and I must rely on validation. For linear smoothers like ridge I can compute effective degrees of freedom exactly. The practical goal is not the number itself but choosing capacity where validation error bottoms out."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Does early stopping reduce capacity?" — Sketch: yes. For linear least squares with gradient descent from zero, stopping at step t behaves like ridge with $\lambda \approx 1/(\eta t)$: directions with small singular values haven't been fitted yet. Effective capacity grows with training time.*

Deeper: *"Explain double descent." — Sketch: test error vs. capacity rises toward the interpolation threshold (the model barely fits the data, only one very contorted solution exists, high variance), then falls again: with more capacity, many interpolating solutions exist and the minimum-norm one (which gradient methods find) is smoother. It appears in model size, training epochs and even dataset size (more data can temporarily hurt near the threshold).*

⑫ **When the assumption breaks.** Capacity measures assume the whole hypothesis space is reachable; in deep learning, the optimizer only explores a small, biased part of it — so "effective" capacity depends on the training procedure, not just the architecture.

1.14 Underfitting

① **Intuition.** The model is too simple, too constrained, or too under-trained to capture the signal — it's wrong even on data it has seen.

② **Definition.** High training error, with validation error close to it. Caused by high bias (approximation error), excessive regularization, missing features, or optimization failure.

③ **Math.** $\hat{R}_n(\hat{f}) \gg R^*$ and $R(\hat{f}) \approx \hat{R}_n(\hat{f})$: small generalization gap, large fit error.

Real-life analogy: a one-rule weather forecast

Think of it like a weather forecaster who says "it's 30°C" every single day in Delhi. He's wrong in January and wrong in May — even about days he has already seen. The rule is too simple to capture seasons. Giving him more past years of data won't help; he needs a richer rule (month, humidity, winds).

Where it breaks: a forecaster knows he is oversimplifying; a model can also underfit for hidden reasons — too much regularization, a bug, or an optimizer that stopped too early — so check those before blaming model size.

④ **Picture.** A straight line through a sine wave: residuals show clear structure (positive, then negative, then positive).

⑤ Example

Predicting house prices with only square footage using linear regression: training $R^2 = 0.45$, validation $R^2 = 0.44$. Adding location features and a log-transform of price: both jump to ~0.8. The model wasn't overfitting — it didn't have the information or flexibility.

⑥ Real-world failure

A regularization hyperparameter set once for a 1M-row dataset was reused after the dataset grew to 100M rows. With far more data, the optimal regularization is weaker; the old value now over-constrained the model, leaving accuracy on the table for months.

⑦ Misconception

"If validation error ≈ training error, the model is good." It's *stable*, not necessarily *good*. Both could be bad. Compare against the target performance or a Bayes-error estimate (human performance).

⑧ Interview question: "Training and validation accuracy are both poor. What do you do?"

⑨ **Strong answer.** "That's underfitting or a data problem, so I work in order. First, sanity checks: are labels correct and aligned with features, is there actually signal — can any model, even a big one, beat a trivial baseline? Can the model overfit a tiny subset? If not, it's a bug or optimization issue — learning rate, initialization, a broken feature pipeline. If yes, it's bias: add capacity, add informative features or interactions, reduce regularization, train longer. And I'd check label noise: if labels are very noisy, 'poor' might be close to the Bayes limit."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Why is 'overfit a small batch' such a good test?" — Sketch: a correctly wired model with sufficient capacity should drive loss near zero on ~10–100 examples. Failure isolates bugs in the loss, labels, gradient flow, or optimizer — before you waste a full training run.*

Deeper: *"You made the model 10× bigger and training error didn't move. What now?" — Sketch: capacity was never the bottleneck: suspect the features don't contain the information (Bayes-limited), label noise, or an optimization pathology (dead ReLUs, vanishing gradients, bad LR). Check gradient norms per layer and loss on a tiny subset.*

⑫ **When the assumption breaks.** "Poor training error" presumes the labels define the right target. If labels are noisy or inconsistent, the floor is high and "fixing underfitting" means improving labels, not the model.

1.15 Overfitting

① **Intuition.** The model learns the training sample's quirks — noise, coincidences, leaked identifiers — as if they were real patterns.

② **Definition.** Low training error but substantially higher validation/test error: a large generalization gap. Caused by high variance: too much capacity relative to data, too little regularization, too many modelling choices tuned on the same data, or leakage-like shortcuts.

③ **Math.** $R(\hat{f}) - \hat{R}_n(\hat{f})$ is large. For least squares with d features and n samples under a well-specified linear model, the expected test excess error is $\approx \sigma^2 \frac{d}{n-d-1}$ — it blows up as $d \to n$.

Real-life analogy: mugging up last year's paper

Think of it like a student who memorizes the answers to last year's question paper word for word. He scores 100% on that paper, but when the board changes the questions, he's lost. He learned the *paper*, not the *subject*. A student who understood the concepts scores a bit lower on the old paper but far better on the new one.

Where it breaks: students know when they are memorizing; models don't. And in large neural nets, fitting the training set perfectly isn't always bad (benign overfitting) — the real test is always new data.

④ **Picture.** A degree-15 polynomial threading every noisy point and oscillating wildly between them (see the figure above).

⑤ Example

A tree with no depth limit on 1,000 rows reaches 100% training accuracy — each leaf has one sample. Validation accuracy: 71%. Limiting `min_samples_leaf=20` gives training 84%, validation 80%. The 16 points of training accuracy it gave up were noise.

⑥ Real-world failure

Google Flu Trends (2008–2013) fit flu incidence from search-query frequencies, choosing among ~50 million queries using a few hundred weekly data points. It included queries correlated with winter (seasonal coincidences) rather than flu, and badly overestimated the 2012–13 peak. Massive feature search over tiny data is overfitting by construction.

⑦ Misconception

"Overfitting only happens when training accuracy is near 100%." Overfitting is about the *gap*, and it also happens at the **selection** level: tuning hundreds of hyperparameters or features against one validation set overfits it even if each model is simple.

⑧ Interview question: "How do you detect and prevent overfitting?"

⑨ **Strong answer.** "Detect with a held-out set that matches deployment: a training error much lower than validation error, a gap that widens with epochs or capacity, or performance that's very sensitive to

the random seed or data subsample. Prevent by attacking variance: more data or augmentation; lower capacity or stronger regularization — L1/L2, dropout, weight decay, tree depth limits, minimum leaf size; early stopping; bagging and ensembles; fewer, better features. And I protect against selection-level overfitting by limiting how many decisions touch the validation set, using cross-validation, and keeping a truly untouched test set."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Validation accuracy is still improving but the gap is growing. Stop training?" — Sketch: no — what matters is validation performance, not the gap. A growing gap with improving validation just means the model fits training data even faster. Stop when validation stops improving.*

Deeper: *"Is it possible to overfit with a model that has fewer parameters than data points?" — Sketch: yes — overfitting depends on capacity relative to the **signal-to-noise ratio** and on effective sample size. 100 parameters on 1,000 rows from 10 users (effective $n \approx 10$) overfits badly. And feature selection over many candidates overfits even with a tiny final model.*

⑫ **When the assumption breaks.** "More data fixes overfitting" assumes the new data is from the same distribution and labeled correctly. Adding biased or duplicated data can *increase* confidence in wrong patterns.

1.16 Regularization

① **Intuition.** Any technique that makes the learner prefer simpler solutions, trading a bit of training fit for a smaller generalization gap.

② **Definition. Explicit** regularization adds a penalty $\Omega(f)$ to the objective (L1, L2, elastic net, tree complexity penalties). **Implicit** regularization comes from the procedure: early stopping, SGD noise, dropout, data augmentation, ensembling, architecture constraints, label smoothing.

③ **Math.**

$$\hat{f} = \arg\min_f \frac{1}{n}\sum_i \ell(f(x_i), y_i) + \lambda\,\Omega(f) \quad \Longleftrightarrow \quad \arg\min_f \hat{R}_n(f) \ \text{ s.t. } \ \Omega(f) \le t$$

(Lagrangian duality: each λ corresponds to some budget t.) Bayesian view: $\lambda\Omega(f) = -\log p(f)$ + const, so regularized ERM = MAP estimation.



Real-life analogy: a strict monthly budget

Think of it like planning a wedding with a budget cap. Without a cap, you'd add every decoration that seems to help — some useful, most wasted. A budget (λ) forces you to keep only the things that clearly matter. A tighter budget = simpler plan; too tight and the wedding is bare (underfitting).

Formula, read aloud: best model = the one that makes (average training error) + λ × (complexity cost) smallest. Equivalently: fit the data as well as you can *while spending at most t on complexity*.

Where it breaks: a budget is in rupees you can count; a model's "complexity cost" is a choice (L1, L2, depth, dropout), and different choices simplify in different ways.

④ **Picture.** L2 shrinks the weight vector toward the origin along a sphere; L1 along a diamond whose corners lie on the axes, so solutions land on corners — sparse. (Full geometry in Part 5.)

⑤ Example

Ridge on 50 correlated features, $n = 200$: OLS coefficients swing between +40 and −38 on two nearly collinear features (they cancel). With $\lambda = 10$, both become ≈ +1: slightly worse training fit, far lower variance, better test error.

⑥ Real-world failure

A team applied L2 regularization without standardizing features. One feature was in dollars (values ~ 10^5), another a ratio (~10^{-1}). The penalty barely constrained the dollars feature (tiny coefficient) and crushed the ratio feature (large coefficient needed). Regularization silently encoded "features in small units matter less". Always standardize before penalized linear models.

⑦ Misconception

"Regularization always improves test performance." Only when variance is the problem. On an underfitting model it makes things worse. And it's not free: too much λ biases coefficients toward zero, which hurts calibration and effect-size interpretation.

⑧ Interview question: "Why does regularization reduce overfitting?"

⑨ **Strong answer.** "It shrinks the effective hypothesis space the optimizer can reach, which reduces variance — the model can't contort itself to fit noise because contortions, like large weights, are penalized. Mechanistically for L2: the noise-fitting directions are the ones with little data support — small singular values of X — and ridge shrinks exactly those directions the most, by factors $s^2/(s^2+\lambda)$. You accept a bit of bias for a larger variance reduction. Equivalently it's a prior: L2 says 'weights are probably small', L1 says 'most weights are probably zero'. The strength is chosen by validation because the right amount depends on the signal-to-noise ratio and n."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Why does large weight magnitude correspond to overfitting?" — Sketch: for smooth models, the output's sensitivity to input changes scales with weight norms (a Lipschitz constant bound). Fitting noise requires sharp changes between nearby points, i.e. large weights. Small weights ⇒ smooth functions.*

Deeper: *"Is weight decay the same as L2 regularization?" — Sketch: for vanilla SGD, yes (up to a rescaling of λ by the learning rate). For adaptive optimizers like Adam, no: L2 added to the loss gets divided by the adaptive denominator, so heavily-updated weights are regularized less. AdamW **decouples** decay from the gradient step, restoring uniform shrinkage — and generalizes better (Part 14).*

⑫ **When the assumption breaks.** Penalties assume "small/sparse is simpler" in the chosen parametrization. If true effects are large, or if features are on different scales, or if the truly relevant features are correlated (L1 then arbitrarily picks one), the penalty imposes the wrong prior.

1.17 Data leakage

① **Intuition.** The model gets access, during training or validation, to information it won't have at prediction time. It looks brilliant offline and fails in production.

② **Definition. Leakage** is any flow of information from the target or from the evaluation data into the features or the training process that won't exist at inference. Main forms: **target leakage** (features derived from or

caused by the label), **train–test contamination** (preprocessing fitted on all data, duplicates across splits), **temporal leakage** (using future information), **group leakage** (same entity in train and test).

③ **Math.** The model effectively learns $P(y \mid x, z)$ where z is leaked information, but at serving time only x is available. The offline metric estimates $R(f; P_{x,z})$ while production realizes $R(f; P_x)$ — the gap can be arbitrarily large.



Real-life analogy: seeing the answer key before the exam

Think of it like a student who accidentally sees the answer key while "practising". His mock-test scores are brilliant — but on exam day there is no answer key, and he collapses. Leakage is any way the answer (or future information) sneaks into training or validation that won't be available when the model is actually used.

Where it breaks: a peeked answer key is obvious; ML leakage is usually subtle — a feature created *after* the event, a scaler fitted on all data, or the same customer in both train and test.

④ **Picture.**

```
time ------------------------------------------------------------->
[ features known at prediction time ] | PREDICT | [ label happens ]
                                        ^
   leaked feature: computed AFTER this point (e.g. "refund_issued",
   "account_closed_date", "days_until_churn") → perfect offline, useless live
```

⑤ Example

Churn model with a feature `num_support_calls_last_30d`, computed over a window that extends past the prediction date. Churners call support to cancel, so the feature nearly encodes the label. Offline AUC 0.97; production 0.70. Fix: compute every feature with **point-in-time correctness** — only data with timestamps before the prediction timestamp.

⑥ Real-world failure

In the KDD Cup 2008 breast-cancer task, patient ID numbers turned out to be predictive of malignancy because of how data from different institutions had been assembled; models could score well by learning the ID pattern rather than the pathology. In another classic, a hospital pneumonia dataset showed asthma patients had *lower* mortality — because they were routinely sent to the ICU. A model deployed to triage would have sent asthma patients home. The signal was real but caused by the treatment policy, not biology.

⑦ Misconception

"Leakage means using the test set for training." That's only one form. Fitting a scaler, an imputer, target encoding, SMOTE or feature selection on the full dataset before splitting also leaks — subtly — and so does any feature computed with future data.

⑧ Interview question: "Your model's validation AUC is 0.99 on a problem where 0.8 would be excellent. What do you do?"

⑨ **Strong answer.** "Assume leakage until proven otherwise. I'd look at feature importance: a single dominant feature is the usual culprit. For each top feature, ask 'would I know this value at the exact moment of prediction?' — check how and when it's computed. Then check the split: duplicates or the same user across train and validation, random splits on temporal data, preprocessing or target encoding fitted be-

fore splitting. A quick test: retrain without the suspicious feature, or with a strict time-based and grouped split, and see if performance collapses to a plausible level. Only once it survives that do I celebrate."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"How do you prevent leakage systematically, not case by case?" — Sketch: put all preprocessing inside a pipeline fitted only on training folds; use a feature store with point-in-time joins (features as of the event timestamp); split by time and by entity; deduplicate across splits (including near-duplicates); keep a "feature availability" timestamp for every feature; review any feature with suspiciously high importance.*

Deeper: *"Is target encoding with K-fold out-of-fold means leak-free?" — Sketch: much safer, but not magically: with small categories, encodings are noisy and the model can still overfit them; with temporal data, out-of-fold encodings must also respect time order (use only past data). Smoothing toward the global mean and adding noise help.*

⑫ **When the assumption breaks.** Leakage prevention assumes you know when each piece of information becomes available. In complex systems (asynchronous logs, backfilled tables, late-arriving labels) this is hard — which is why training/serving skew monitoring (compare the features the model saw at serving time with the ones in training data) is a production must.

1.18 PAC learning and sample complexity (with numbers)

① **Intuition.** "Probably approximately correct": with enough data, a learner will *probably* (with probability $1-\delta$) output a hypothesis that is *approximately* right (error within ε). The theory tells you how "enough data" scales with the size of the hypothesis space, the accuracy you want and the confidence you demand.

② **Definition.** A class $\mathcal{H}$ is PAC-learnable if there is an algorithm and a sample size $n(\varepsilon, \delta)$, polynomial in $1/\varepsilon$ and $1/\delta$, such that for every data distribution, with probability at least $1-\delta$ the learned $\hat{h}$ has $R(\hat{h}) \le \min_{h \in \mathcal{H}} R(h) + \varepsilon$.

③ **Math.** For a **finite** class, Hoeffding's inequality plus a union bound over the $|\mathcal{H}|$ hypotheses gives:

$$n \ge \frac{\ln|\mathcal{H}| + \ln(1/\delta)}{2\varepsilon^2} \quad \text{(agnostic)}, \qquad n \ge \frac{\ln|\mathcal{H}| + \ln(1/\delta)}{\varepsilon} \quad \text{(realizable: some } h \text{ is perfect)}$$

For infinite classes, $\ln|\mathcal{H}|$ is replaced by a capacity measure such as the VC dimension (up to log factors).



Real-life analogy: how many questions make a fair exam?

Think of it like a board setting an exam to judge a student. With 5 questions, a lucky guesser can look brilliant; with 100, luck averages out. PAC learning asks the same: how many examples do we need before a rule that *looks* good on them is *probably* good in general? More candidate rules to choose from (a bigger $|\mathcal{H}vert$) means more chances that one fits by luck — so more questions are needed, but only in proportion to the *log* of the number of rules.

Formula, read aloud: $n \ge \frac{\ln|\mathcal{H}|+\ln(1/\delta)}{2\varepsilon^2}$ says: examples needed = (log of how many rules you're choosing among + log of how sure you want to be) ÷ (twice the squared error tolerance).

Where it breaks: real exams test one student; PAC bounds must hold for every rule and every possible data distribution at once, which makes them very cautious (loose) for real models.

④ Worked numbers

$|\mathcal{H}| = 10^6$ candidate rules, confidence $\delta = 0.05$, tolerance $\varepsilon = 0.05$:

$\ln 10^6 \approx 13.8$, $\ln 20 \approx 3.0$ → agnostic: $n \geq 16.8/(2 \times 0.0025) \approx$ **3,360**; realizable: $n \geq 16.8/0.05 \approx$ **336**.

Halving ε quadruples the agnostic requirement ($1/\varepsilon^2$). Multiplying the class size by 1,000 adds only $\ln 1000 \approx 6.9$ to the numerator — **sample complexity grows with the log of the number of hypotheses**, which is why searching a huge but finite space of models is not hopeless.

⑤ Example

Choosing among 50 hyperparameter configurations on a 2,000-example validation set: treat the configurations as $|\mathcal{H}| = 50$. With $\delta = 0.05$ the uniform deviation is at most $\sqrt{(\ln 50 + \ln 20)/(2 \cdot 2000)} \approx 0.042$ — the winner's validation accuracy may be optimistic by up to ~4 points. That's the quantitative reason to keep an untouched test set (Part 17).

⑥ Real-world failure

A team screened 10,000 engineered features against a 1,500-row dataset and kept the 30 with the strongest validation correlation. The bound (and common sense) says selection over that many candidates on so little data guarantees spurious winners; the model's lift vanished on the next quarter's data.

⑦ Misconception

"The bounds predict how much data I need." For modern models they are **loose by orders of magnitude** — they hold for every distribution, including adversarial ones. Their value is in the **scaling laws they reveal** ($\log|\mathcal{H}|$, $1/\varepsilon^2$, the cost of selection), not in their constants.

⑧ Interview question: "What does PAC learning tell you in practice?"

⑨ **Strong answer.** "It formalizes that generalization depends on sample size relative to the richness of what you search over. For a finite class, the needed sample grows with the log of the class size, with $\log(1/\delta)$ for confidence, and with $1/\varepsilon^2$ for accuracy in the agnostic case. The practical lessons: every model or feature you *select* on a validation set consumes some of its reliability; halving the error tolerance needs roughly four times the data; and realizable problems — where a perfect rule exists — need far less data than noisy ones. I wouldn't use the constants to size a dataset; I'd use learning curves for that."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Why is the realizable case $1/\varepsilon$ rather than $1/\varepsilon^2$?" — Sketch: a bad hypothesis (error $> \varepsilon$) survives n examples with probability $(1-\varepsilon)^n \leq e^{-\varepsilon n}$; you only need to rule out bad hypotheses, not estimate every error rate precisely.*

Deeper: *"Deep nets have far more parameters than examples, yet generalize. Do the bounds fail?" — Sketch: parameter-count bounds are vacuous there. Better explanations use **norm-based or margin-based** capacity, **compression** (the trained net can be described with few bits), PAC-Bayes bounds around flat minima, and the implicit bias of SGD toward simple solutions — see double descent next.*

⑫ **When the assumption breaks.** PAC guarantees assume IID train and test data from the same distribution. Under distribution shift (Part 1, distribution shift) or adaptive reuse of the test set, they say nothing.

1.19 Double descent and benign overfitting

① **Intuition.** Classical theory says: past the sweet spot, more capacity means more overfitting. Modern practice shows a second regime: as models become large enough to fit the training data *exactly*, test error peaks — then **falls again** as capacity keeps growing.

② **Definition. Double descent:** test error as a function of capacity (or data size, or training time) rises toward the **interpolation threshold** — where the model can just barely fit the training set — then decreases beyond it. **Benign overfitting:** a model fits noisy training labels perfectly yet still predicts well.

③ **Math.** For minimum-norm least squares on p features and n points, the solution is $\hat{\beta} = F^{+}y$ (pseudo-inverse). Near $p = n$ the feature matrix is square and badly conditioned — its smallest singular values approach zero — so $\|\hat{\beta}\|$ and the variance explode. With $p \gg n$, many interpolating solutions exist, and the minimum-norm one spreads the fit over many directions, damping noise; gradient descent from zero initialization converges to exactly this minimum-norm solution.

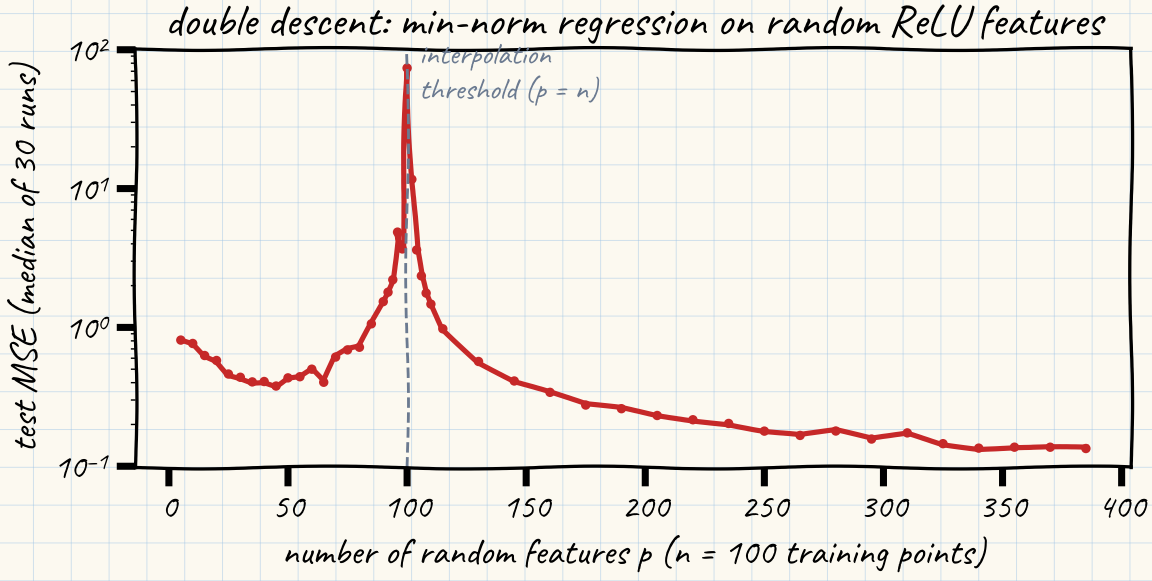
Real-life analogy: a road forced through every checkpoint

Think of it like an engineer who must build a road passing *exactly* through 100 checkpoints. With a design that has just barely enough bends to hit all 100, the road swings wildly between checkpoints — that's the peak. With far more freedom, the engineer can hit every checkpoint *and* choose the smoothest, gentlest road among all that do — that's the second descent.

Formula, read aloud: $\hat{\beta} = F^{+}y$ is "among all weight settings that fit the data exactly, pick the one with the smallest size" — the gentlest fit.

Where it breaks: an engineer chooses smoothness deliberately; a model gets it only because of how it is trained (minimum-norm solutions, gradient descent from small starting weights), and noisy labels still leave some bulges.

④ **Picture.**



Test error of minimum-norm regression on random ReLU features: the peak sits at p = n

⑤ Example

The figure: 100 training points, a linear target observed with noise, fitted by minimum-norm least squares on p random ReLU features. Test error first falls, blows up by orders of magnitude at $p = 100$, and then declines steadily for $p \gg 100$ — without any explicit regularization.

⑥ Real-world failure

A team sized a model "in the middle" — just big enough to reach zero training error on their data — and saw erratic results across seeds. They were sitting on the interpolation peak. Either a smaller model with regularization or a substantially larger one (plus early stopping) was more stable.

⑦ Misconception

"Double descent means bigger is always better, so regularization is obsolete." The peak can be removed by **optimal regularization** (tuned ridge, early stopping), and "bigger is better" holds only with enough compute, suitable inductive biases, and data that isn't dominated by label noise.

⑧ Interview question: "How can a model with more parameters than training examples generalize?"

⑨ **Strong answer.** "Parameter count isn't the right capacity measure. Among the infinitely many solutions that fit the training data, the training procedure picks a particular one — gradient descent from small initialization finds low-norm solutions, and SGD noise favors flat minima. Those solutions are simple in the sense that matters. Empirically, test error follows double descent: bad near the interpolation threshold, where the fit is forced through noise with huge weights, and improving again with more capacity. In practice that's why large models plus early stopping, weight decay and data augmentation work, and why the classic U-curve is incomplete."

⑩ Follow-up → ⑪ Deeper follow-up

Follow-up: *"Can double descent happen in the number of training examples?" — Sketch: yes: for a fixed model size, adding data can temporarily hurt when n approaches p (sample-wise double descent), because the problem passes through the ill-conditioned interpolation threshold.*

Deeper: *"Connect this to scaling laws." — Sketch: scaling laws (Part 42) describe the smooth large-model regime, far past interpolation, where loss falls as a power law in parameters and data; double descent describes the transition into that regime.*

⑫ **When the assumption breaks.** Benign overfitting relies on noise being spread over many directions the test distribution rarely uses. With heavy, structured label noise (systematic mislabeling), memorizing it is harmful — clean the labels (Part 25).

Active recall — Set 1: Foundations

Set 1 · Answer from memory, then check the Answer Key at the back

Easy recall

1. Define expected risk and empirical risk in one line each.
2. What are the three terms in the bias–variance decomposition of squared error?

3. What is the Bayes classifier?
4. Name the three dataset roles and the job of each.
5. Name three forms of data leakage.

Conceptual

6. Why is training error an optimistically biased estimate of test error for the selected model?
7. What is the difference between covariate shift, label shift and concept drift?
8. How is inductive bias related to a Bayesian prior?
9. What is the difference between approximation error and estimation error?
10. What does it mean that a hypothesis class can "shatter" a set of points?

Why

11. Why can't more data reduce the Bayes error?
12. Why does No Free Lunch not imply that algorithm choice is irrelevant?
13. Why does fitting a scaler on the full dataset before splitting count as leakage?
14. Why can a model with fewer parameters than rows still overfit?
15. Why do random row splits inflate metrics on user-level data?

Scenario

16. Validation AUC 0.99 on a hard problem. First three things you check?
17. Training and validation errors are both high and close. What are your hypotheses, in order?
18. You tuned 500 configurations on one validation set. How much do you trust the best score, and what do you do?
19. A model trained on 2023 data is deployed in 2025 and error rises, but input distributions look unchanged. What's happening?
20. Your team wants to add 200 new features to a model trained on 5,000 rows. Your advice?

Mathematical

21. For a finite hypothesis class and loss in $[0, 1]$, write the uniform generalization bound.
22. Compute the effective sample size for 10,000 rows from 500 users (20 rows each) with intra-user correlation 0.3.
23. Show that the Bayes predictor for squared loss is $\mathbb{E}[Y \mid X = x]$.

Difficult follow-ups

24. Deep nets can fit random labels. Give two reasons they still generalize on real labels.
25. When does importance weighting for covariate shift fail, and how do you detect it?

2027 additions

26. How many examples does the agnostic PAC bound require for $|\mathcal{H}| = 10^6$, $\delta = 0.05$, $\varepsilon = 0.05$? What happens if you halve ε?
27. Sketch the double-descent curve and explain why test error peaks at the interpolation threshold.
28. You tried 50 configurations on a 2,000-example validation set. Roughly how optimistic can the winner's score be, and what do you do about it?

Unit II

Mathematics for ML Interviews

Part 2 — Linear Algebra
Part 3 — Probability
Part 4 — Statistics

Only the math that shows up in models, derivations and interview follow-ups — but taught to the level where you can use it.

PART 2

Linear Algebra

Why linear algebra is the language of ML

A dataset is a matrix, a linear model is a dot product, a neural layer is a matrix multiply, a gradient is a vector, curvature is a matrix, and "structure in data" is almost always "low rank" or "a few dominant directions". If you can see matrices as **geometric transformations**, most ML math becomes pictures.

The whole Part as one picture

Think of it like a photo-editing app. A dataset is a spreadsheet: one row per example, one column per feature. A matrix is a filter that stretches, turns or squashes the whole picture in one go. Eigenvectors are the directions a filter only stretches; the SVD says every filter is turn → stretch → turn; rank counts how many directions survive the squash; the inverse is the undo button, which fails if something was flattened. Norms measure how big a change is, and projection is the shadow an object casts on the floor.

Where it breaks: photo filters act on flat 2-D pictures; ML matrices act on spaces with thousands of directions, where pictures can mislead (for example, two random directions are almost always nearly perpendicular).

2.1 Scalars, vectors, matrices, tensors

Object	Notation	Shape	Where it lives in ML
Scalar	a	$()$	a loss value, learning rate, a single prediction
Vector	$\mathbf{x} \in \mathbb{R}^d$	$(d,)$	one example's features, a weight vector, an embedding, a gradient
Matrix	$X \in \mathbb{R}^{n \times d}$	(n, d)	a dataset (rows = examples), a layer's weights $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$
Tensor	$\mathcal{T}$	(n, c, h, w)	a batch of images; (B, T, d) a batch of token sequences; attention (B, H, T, T)

Convention used here: vectors are columns; the data matrix X has one example per **row**; predictions for all examples are $X\mathbf{w}$.

Real-life analogy: a school marks register

Think of it like a school's marks register. One student's maths mark is a scalar. That student's marks in all subjects (one row) is a vector. The whole class register, students × subjects, is a matrix. A cupboard of registers, one per class and per exam term, is a tensor. The shape is just "how many along each side". Confuse a list of marks written across (a row) with one written down (a column), and the computer may silently pair every entry with every other: the broadcasting bug below.

Where it breaks: a register's columns have names printed on top; in code, a tensor's axes are just positions, so nothing stops you mixing them up except checking shapes.

Shape bugs are the #1 coding-interview bug

Broadcasting silently turns a $(n,)$ minus $(n,1)$ into an (n,n) matrix. `y_pred.shape == (n,1)` and `y.shape == (n,)` makes `((y_pred - y)**2).mean()` average n^2 numbers. Always assert shapes.

2.2 Dot product

What it is. $\mathbf{a}^\top\mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\|\|\mathbf{b}\|\cos\theta$.

Geometry. It measures **alignment**: the length of $\mathbf{a}$'s shadow on $\mathbf{b}$'s direction, times $\|\mathbf{b}\|$. Zero means orthogonal (no linear relationship between the directions). The scalar projection of $\mathbf{a}$ onto unit vector $\mathbf{u}$ is $\mathbf{a}^\top\mathbf{u}$.



Real-life analogy: a music-taste match score

Think of it like two friends rating genres (Bollywood, rock, classical, rap) from −2 (hate) to +2 (love). Multiply their ratings genre by genre and add. Shared loves give a big score, unrelated tastes give about zero, and opposite tastes give a negative score. Cosine similarity asks whether they like the *same kinds* of things, ignoring how extreme each person's ratings are.

Formula, read aloud: $\mathbf{a}^\top\mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\|\|\mathbf{b}\|\cos\theta$ says: multiply matching entries and add; equivalently, (length of $\mathbf{a}$) × (length of $\mathbf{b}$) × (how much they point the same way).

Where it breaks: tastes can agree in complicated ways; a dot product only captures straight-line agreement, and its size grows with vector length and dimension, which is why attention divides by $\sqrt{d}$.

ML connections.

- A **linear model** $\hat{y} = \mathbf{w}^\top\mathbf{x} + b$ scores an input by its alignment with the weight vector. The decision boundary $\mathbf{w}^\top\mathbf{x} + b = 0$ is a hyperplane with normal $\mathbf{w}$.
- A **neuron** is a dot product followed by a nonlinearity.
- **Cosine similarity** $\frac{\mathbf{a}^\top\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$ compares embeddings independent of length — the backbone of retrieval and recommendation.
- **Attention** scores are scaled dot products $\frac{\mathbf{q}^\top\mathbf{k}}{\sqrt{d}}$. The $\sqrt{d}$: if $\mathbf{q},\mathbf{k}$ have IID unit-variance components, $\mathrm{Var}(\mathbf{q}^\top\mathbf{k}) = d$, so dividing by $\sqrt{d}$ keeps scores at unit scale and the softmax out of saturation.

Interview question: "Why divide by √d in attention?"

Strong answer. "With independent zero-mean, unit-variance components, the dot product of two d-dimensional vectors is a sum of d unit-variance terms, so its variance is d and its typical magnitude grows like $\sqrt{d}$. Large logits push softmax toward a one-hot output where gradients vanish. Dividing by $\sqrt{d}$ restores unit variance regardless of dimension."

2.3 Matrix multiplication

What it is. $(AB)_{ij} = \sum_k A_{ik}B_{kj}$ for $A \in \mathbb{R}^{m\times k}$, $B \in \mathbb{R}^{k\times n}$. Cost: $O(mkn)$ multiply-adds.

Real-life analogy: orders times recipes

Think of it like a caterer's kitchen. Table A lists each customer's order: how many of each dish. Table B lists each dish's recipe: how much of each ingredient. Multiply them and AB tells you how much of each ingredient each customer needs. The shared middle list (dishes) is the index that gets summed away. Swap the tables and the question stops making sense: $AB \neq BA$.

Formula, read aloud: $(AB)_{ij} = \sum_k A_{ik}B_{kj}$ says: ingredient j for customer i = over every dish k, add up (how many of dish k they ordered) × (how much of ingredient j one dish k uses).

Where it breaks: here everything is a positive count; in ML entries can be negative, and the middle index is often a hidden dimension with no name.

Four ways to see it (each useful in a different interview moment):

1. **Row–column dot products:** entry (i, j) is row i of A dotted with column j of B. → *each prediction is a dot product.*
2. **Linear combination of columns:** $A\mathbf{x} = \sum_j x_j \mathbf{a}_{\cdot j}$. → *the output lives in the column space of A.*
3. **Sum of outer products:** $AB = \sum_k \mathbf{a}_{\cdot k}\mathbf{b}_{k:}^\top$. → *rank-1 building blocks; gradients of a linear layer are outer products: $\partial L/\partial W = \boldsymbol{\delta}\,\mathbf{x}^\top$.*
4. **Composition of maps:** $AB\mathbf{x}$ applies B then A. → *stacking linear layers without nonlinearity is still one linear map (W_2W_1), which is why deep nets need activations.*

Not commutative: $AB \neq BA$ in general; associative: $(AB)C = A(BC)$ — and the **order matters for cost**. For $A \in \mathbb{R}^{1000\times1000}$, $B \in \mathbb{R}^{1000\times1000}$, $\mathbf{v} \in \mathbb{R}^{1000}$: $(AB)\mathbf{v}$ costs $10^9 + 10^6$; $A(B\mathbf{v})$ costs 2×10^6. Backpropagation is exactly the smart ordering (vector–Jacobian products, right to left).

2.4 Transpose

$(A^\top)_{ij} = A_{ji}$; $(AB)^\top = B^\top A^\top$; $(A^\top)^{-1} = (A^{-1})^\top$.

ML connections. $X^\top X \in \mathbb{R}^{d\times d}$ is (up to centering and $1/n$) the **feature covariance/Gram matrix**; $XX^\top \in \mathbb{R}^{n\times n}$ is the **example similarity (kernel) matrix**. In backprop, if a layer computes $\mathbf{z} = W\mathbf{x}$, the gradient flowing back to the input is $W^\top\boldsymbol{\delta}$: the transpose **routes error signals backwards** through the same connections.

Real-life analogy: complaints travel back along delivery routes

Think of it like a delivery network. Parcels flow forward from warehouses to shops along roads, and each road's weight says how much it carries ($\mathbf{z} = W\mathbf{x}$). When shops complain about their orders, the complaints travel *backwards* along the same roads, and each warehouse collects blame from each shop in proportion to how much it sent that shop. Same roads, opposite direction: that's the transpose.

Formula, read aloud: $W^\top\boldsymbol{\delta}$ says: the error arriving back at each input = the sum over outputs of (that output's complaint in $\boldsymbol{\delta}$) × (the weight of the road between them).

Where it breaks: a transpose is not an inverse: sending blame back along the roads doesn't undo the delivery, it only tells each warehouse how much it contributed.

2.5 Inverse

What it is. A^{-1} exists iff A is square and full rank (equivalently $\det A \neq 0$, no zero eigenvalue); then $A^{-1}A = I$. Geometrically, it undoes the transformation — impossible if A collapses some direction to zero.

ML connections. The normal equation $\hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y}$; Gaussian densities use Σ^{-1} (precision matrix); Newton's method uses $H^{-1}\nabla$.

Real-life analogy: the undo button on a photo filter

Think of it like undo in a photo app. A filter that brightens or stretches can be undone exactly: that's A^{-1}. A filter that turns a colour photo black-and-white can't be undone: the colour was flattened away (not full rank). A filter that makes colours *almost* grey can be undone in theory, but any speck of noise comes back as wild colour: a large condition number.

Formula, read aloud: $A^{-1}A = I$ says: undo after do = nothing happened. $\kappa = \sigma_{\max}/\sigma_{\min}$ says: (the most any direction is stretched) ÷ (the least), which is how much small errors get blown up on the way back.

Where it breaks: a photo app's undo secretly remembers the original; an inverse must rebuild it from the output alone, which is why flattened directions are lost forever.

Never compute an explicit inverse in code

Solve instead: `np.linalg.solve(A, b)` or a Cholesky/QR/SVD-based solver. Explicit inversion costs more ($\sim 2\times$) and amplifies rounding error. For least squares use `np.linalg.lstsq` (SVD-based), which also handles rank deficiency.

Conditioning. The **condition number** $\kappa(A) = \sigma_{\max}/\sigma_{\min}$ bounds how much relative input error is amplified in the solution: you lose roughly $\log_{10}\kappa$ digits of precision. Note $\kappa(X^\top X) = \kappa(X)^2$ — forming $X^\top X$ **squares** the conditioning problem, which is why QR/SVD solvers are preferred for ill-conditioned least squares, and why ridge's $+\lambda I$ (which lifts every eigenvalue by λ) stabilizes the solve.

```
import numpy as np
rng = np.random.default_rng(0)
x1 = rng.normal(size=1000)
X = np.c_[x1, x1 + 1e-4 * rng.normal(size=1000)]      # two nearly identical features
print(np.linalg.cond(X), np.linalg.cond(X.T @ X))      # ~1.9e4 and ~3.7e8: squared!
lam = 1.0
print(np.linalg.cond(X.T @ X + lam * np.eye(2)))      # ridge: ~2e3, well conditioned
```

2.6 Rank and linear independence

Definitions. Vectors are **linearly independent** if no one is a linear combination of the others ($\sum c_i \mathbf{v}_i = 0 \Rightarrow c_i = 0$). The **rank** of A is the number of linearly independent columns (= rows) = dimension of its column space = number of nonzero singular values.

Geometry. Rank is the dimension of the output space the matrix can reach. A rank-2 matrix in $\mathbb{R}^{3\times 3}$ squashes all of 3-D space onto a plane.

Real-life analogy: paints you can mix from others

Think of it like a box of paints. If you already have red and blue, a tube of purple adds nothing new: it's a mix of the two (linearly dependent). The rank is the number of truly independent paints, the number of genuinely different directions your columns offer. The dummy-variable trap is the same thing: columns "is_veg", "is_non_veg" and an intercept column of all 1s, where the 1s are just veg + non-veg.

Formula, read aloud: $\sum_i c_i \mathbf{v}_i = 0 \Rightarrow c_i = 0$ says: the only way to mix these vectors into nothing is to use none of each; if some other mix cancels out, one vector was redundant.

Where it breaks: real paints can't be subtracted; vectors can be combined with negative amounts too, so "redundant" includes cancelling out, not just mixing.

ML connections.

- **Multicollinearity:** if one feature is (nearly) a linear combination of others, X is (nearly) rank-deficient → $X^\top X$ singular (or ill-conditioned) → OLS coefficients undefined (or wildly unstable). Classic trap: one-hot encoding **all** k categories plus an intercept (the "dummy variable trap"): the columns sum to the intercept column.
- **Low-rank structure is everywhere:** user–item rating matrices ≈ low rank (a few taste factors) → matrix factorization recommenders $R \approx UV^\top$; embeddings are low-rank factorizations of co-occurrence matrices; **LoRA** fine-tunes large models by learning a low-rank update $\Delta W = BA$ with $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, $r \ll d$.
- $\operatorname{rank}(X^\top X) \le \min(n, d)$: with $d > n$ (more features than samples), $X^\top X$ is **always** singular — OLS has infinitely many solutions; you need regularization or the minimum-norm solution.

2.7 Basis and orthogonality

Basis: a set of linearly independent vectors that spans the space; every vector has unique coordinates in it. **Orthonormal basis:** mutually orthogonal unit vectors; stack them as columns of Q with $Q^\top Q = I$, so $Q^{-1} = Q^\top$.

Why orthonormal bases are special. Coordinates are just dot products ($c_i = \mathbf{q}_i^\top \mathbf{x}$), lengths and angles are preserved ($\|Q\mathbf{x}\| = \|\mathbf{x}\|$ — rotations/reflections), and projections are trivial.

Real-life analogy: the sliders on a music equalizer

Think of it like the bass, mid and treble sliders on a music system. Any sound setting is described by three slider positions (coordinates in a basis). With a good equalizer, each slider touches only its own band, so moving bass never changes treble: that's orthogonal. To read off a setting, you just check each slider on its own. PCA builds a new set of sliders lined up with where the data varies most.

Formula, read aloud: $Q^\top Q = I$ says: every direction is perpendicular to the others and has length one; $c_i = \mathbf{q}_i^\top \mathbf{x}$ says: the coordinate along direction i is just how far $\mathbf{x}$ points that way.

Where it breaks: an equalizer's sliders are fixed by the maker; in ML you choose the basis, and in high dimensions there are endless valid orthonormal sets.

ML connections. PCA finds a **new orthonormal basis** aligned with the data's variance; in that basis the features are uncorrelated. Orthogonal weight initialization preserves the norm of signals through deep linear stacks, helping avoid vanishing/exploding activations. **Whitening** transforms data so the covariance becomes I: $\mathbf{x} \mapsto \Lambda^{-1/2} U^\top (\mathbf{x} - \boldsymbol{\mu})$.

2.8 Norms

Norm	Formula	Unit ball	ML use
L_0 ("norm")	# nonzeros	axes only	true sparsity; combinatorial, NP-hard to optimize
L_1	$\sum \lvert w_i \rvert$	diamond	Lasso, sparsity, robust (MAE) losses, Manhattan distance
L_2	$\sqrt{\sum w_i^2}$	sphere	ridge / weight decay, Euclidean distance, gradient clipping by norm
L_∞	$\max \lvert w_i \rvert$	cube	adversarial perturbation budgets, max error
Frobenius	$\sqrt{\sum_{ij} A_{ij}^2} = \sqrt{\sum \sigma_i^2}$	—	weight decay on matrices, matrix approximation error
Spectral	$\sigma_{\max}(A)$	—	Lipschitz constant of a layer; spectral normalization in GANs
Nuclear	$\sum \sigma_i$	—	convex surrogate for rank (matrix completion)

Key inequality intuition: $\|\mathbf{w}\|_\infty \le \|\mathbf{w}\|_2 \le \|\mathbf{w}\|_1 \le \sqrt{d}\,\|\mathbf{w}\|_2$. In high dimensions, norms can differ by a factor of $\sqrt{d}$ — choice of norm changes which solutions look "small".



Real-life analogy: three ways to measure a trip

Think of it like measuring how far a delivery address is from the restaurant. L_2 is as the crow flies. L_1 is the auto-rickshaw distance along a grid of streets: blocks east plus blocks north. L_∞ counts only the single longest leg. L_0 just counts how many directions you had to travel in at all. Same trip, different "size", and each ruler prefers different routes.

Formula, read aloud: $\|\mathbf{w}\|_\infty \le \|\mathbf{w}\|_2 \le \|\mathbf{w}\|_1$ says: longest single leg ≤ straight-line distance ≤ distance walking along the streets.

Where it breaks: a city trip has two directions; with d directions the rulers can disagree by a factor of $\sqrt{d}$, so the choice of norm changes which weight vectors count as "small".

2.9 Eigenvalues and eigenvectors

Definition. For square A, a nonzero $\mathbf{v}$ with $A\mathbf{v} = \lambda\mathbf{v}$ is an **eigenvector** with **eigenvalue** λ.

Geometry — the answer interviewers want. Most vectors get rotated *and* stretched by A. Eigenvectors are the special directions that A only **stretches** (or flips) — it maps the line through $\mathbf{v}$ onto itself. The eigenvalue is the stretch factor. For a symmetric matrix, the eigenvectors are orthogonal and form the **principal axes** of the ellipse $\{A\mathbf{x} : \|\mathbf{x}\| = 1\}$; eigenvalues are the axis lengths.

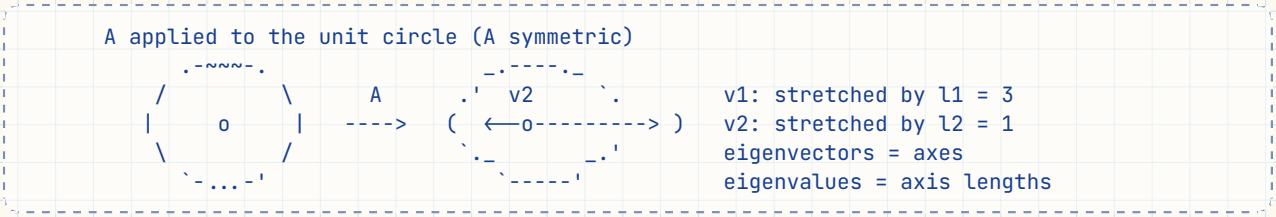


Real-life analogy: rolling out chapati dough

Think of it like rolling out chapati dough, always pushing the rolling pin in one direction. Draw arrows on the dough first. An arrow along the rolling direction gets longer but still points the same way. An arrow across it keeps its length. A diagonal arrow gets *turned* toward the rolling direction. The arrows that keep their direction are the eigenvectors; how much each one stretches is its eigenvalue.

Formula, read aloud: $A\mathbf{v} = \lambda\mathbf{v}$ says: apply the transformation to this special direction and you get the same direction back, just scaled by the number λ.

Where it breaks: real dough stretches unevenly and permanently; a matrix moves every point in the same exact linear way, and some matrices (pure rotations) have no real eigen-directions at all.



Why PCA cares. The covariance matrix Σ describes the data cloud as an ellipsoid. Its eigenvectors are the ellipsoid's axes — the directions in which the data varies independently — and each eigenvalue is **the variance of the data along that axis**. Keeping the top-k eigenvectors keeps the directions carrying the most variance.

More ML connections.

- **Optimization:** the Hessian's eigenvalues are curvatures along its eigenvectors. Gradient descent is stable only for learning rates $\eta < 2/\lambda_{\max}$, and converges at a rate governed by $\kappa = \lambda_{\max}/\lambda_{\min}$ — the long, narrow valley problem that momentum and Adam address (Part 14). Negative Hessian eigenvalues mean a **saddle point** direction.
- **RNNs:** repeated multiplication by W scales components by λ_i^t; spectral radius > 1 → exploding, < 1 → vanishing gradients.
- **Power iteration** ($\mathbf{v} \leftarrow A\mathbf{v}/\|A\mathbf{v}\|$) converges to the top eigenvector — used in PageRank and spectral norm estimation. Convergence speed depends on the gap $|\lambda_2/\lambda_1|$.
- **Spectral clustering:** eigenvectors of the graph Laplacian with the smallest eigenvalues reveal cluster structure.

Interview question: "What does an eigenvector represent geometrically, and why does PCA care?"

Strong answer. "An eigenvector of a matrix is a direction the transformation only scales, without rotating; the eigenvalue is the scale factor. For a symmetric matrix like a covariance matrix, the eigenvectors are orthogonal and are the principal axes of the ellipsoid that describes the data's spread, and each eigenvalue equals the variance of the data projected onto that axis. PCA wants the directions of maximum variance — maximizing $\mathbf{u}^\top \Sigma \mathbf{u}$ over unit vectors — and by a Lagrange-multiplier argument, the maximizer is exactly the top eigenvector with the maximum equal to the top eigenvalue. So the eigen-decomposition hands PCA the ranked, uncorrelated directions directly."

Follow-ups

"Why are the eigenvectors of a covariance matrix orthogonal?" — *Because Σ is symmetric. For $\Sigma\mathbf{u} = \lambda\mathbf{u}$, $\Sigma\mathbf{v} = \mu\mathbf{v}$ with $\lambda \neq \mu$: $\lambda\,\mathbf{v}^\top\mathbf{u} = \mathbf{v}^\top\Sigma\mathbf{u} = (\Sigma\mathbf{v})^\top\mathbf{u} = \mu\,\mathbf{v}^\top\mathbf{u}$, so $\mathbf{v}^\top\mathbf{u} = 0$.*

"Can eigenvalues of a covariance matrix be negative?" — *No: $\lambda = \mathbf{u}^\top\Sigma\mathbf{u} = \mathrm{Var}(\mathbf{u}^\top\mathbf{x}) \geq 0$. A negative computed value signals numerical error or an invalid "covariance" (e.g. pairwise-deletion estimates with missing data).*

2.10 Eigendecomposition

Statement. If $A \in \mathbb{R}^{d\times d}$ has d independent eigenvectors, $A = V\Lambda V^{-1}$. **Spectral theorem:** if A is real symmetric, $A = Q\Lambda Q^\top = \sum_i \lambda_i \mathbf{q}_i\mathbf{q}_i^\top$ with orthonormal Q and real λ_i.

Why it's powerful. Powers and functions become trivial: $A^k = Q\Lambda^k Q^\top$, $A^{-1} = Q\Lambda^{-1}Q^\top$, $e^A = Qe^\Lambda Q^\top$. In the eigenbasis a symmetric matrix is just independent scalings along orthogonal axes. Also $\det A = \prod \lambda_i$ and $\mathrm{tr}\,A = \sum \lambda_i$.

Real-life analogy: savings split into separate deposits

Think of it like splitting your savings into separate deposits, each with its own yearly growth factor. Working out the total after 10 years is hard if money keeps moving between accounts, but easy once each deposit grows on its own: raise each factor to the 10th power and add up. The eigendecomposition finds those independent "deposits" (eigen-directions) and their growth factors (eigenvalues).

Formula, read aloud: $A^k = Q\Lambda^k Q^\top$ says: to apply A k times, split the vector into the independent directions ($Q^\top$), grow each one by its own factor k times (Λ^k), then add the pieces back together (Q).

Where it breaks: money can't be negative; the amount along an eigen-direction can be, and a non-symmetric matrix may not split into clean, independent directions at all.

ML connections. Ridge regression in the eigenbasis of $X^\top X$: each coefficient direction is multiplied by $\frac{\lambda_i}{\lambda_i+\lambda}$ — ridge shrinks low-variance directions most. Gaussian log-densities, Mahalanobis distances, and PCA all become sums over eigen-directions.

2.11 Singular value decomposition (SVD)

Statement. Every real matrix $A \in \mathbb{R}^{m\times n}$ factorizes as

$$A = U\Sigma V^\top = \sum_{i=1}^{r} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^\top, \qquad U^\top U = I,\ V^\top V = I,\ \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$$

Real-life analogy: patterns in an IPL scores table

Think of it like a table of runs scored by every player in every IPL match. SVD explains it as a stack of simple patterns, strongest first. Pattern 1 might be "good batters score more, and flat pitches help everyone": a column of player strengths ($\mathbf{u}_1$) times a row of match conditions ($\mathbf{v}_1$), scaled by its importance σ_1. Later patterns are smaller corrections. Keep the top few for the best short summary of the table.

Formula, read aloud: $A = \sum_i \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ says: the table = importance × (player pattern × match pattern), added up over patterns, from most to least important.

Where it breaks: SVD's patterns rarely have neat names like "pitch"; they are just perpendicular directions of largest spread, and one pattern often mixes several real causes.

Geometry. Any linear map = **rotate** ($V^\top$) → **scale along axes** (Σ) → **rotate** (U). The unit sphere becomes an ellipsoid with semi-axes $\sigma_i \mathbf{u}_i$.

Relation to eigendecomposition. $A^\top A = V\Sigma^2 V^\top$ and $AA^\top = U\Sigma^2 U^\top$: right singular vectors are eigenvectors of $A^\top A$; singular values are square roots of its eigenvalues. SVD exists for any matrix (rectangular, rank-deficient); eigendecomposition needs square and diagonalizable.

Eckart–Young theorem. The best rank-k approximation of A in Frobenius (and spectral) norm is the truncated SVD $A_k = \sum_{i\leq k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, with error $\|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2$.

ML connections.

- **PCA via SVD** of the centered data $X_c = U\Sigma V^\top$: principal directions are columns of V, component variances are $\sigma_i^2/(n-1)$, projected coordinates are $U\Sigma$. Numerically better than eigendecomposing $X_c^\top X_c$ (no squaring of the condition number).

- **Pseudo-inverse and least squares:** $X^+ = V\Sigma^+U^\top$; $\hat{\mathbf{w}} = X^+\mathbf{y}$ is the **minimum-norm** least-squares solution, well-defined even when X is rank-deficient.
- **Latent semantic analysis**, compression, denoising (drop small singular values = drop noise directions), low-rank adaptation, collaborative filtering.

```
# Low-rank approximation and how much "energy" k components keep
U, s, Vt = np.linalg.svd(A, full_matrices=False)
k = 20
A_k = (U[:, :k] * s[:k]) @ Vt[:k]                  # scale columns of U, avoid building diag
energy_kept = (s[:k] ** 2).sum() / (s ** 2).sum()   # fraction of squared Frobenius norm
```

2.12 Positive definite matrices

Definition. Symmetric A is **positive definite** (PD) if $\mathbf{x}^\top A\mathbf{x} > 0$ for all $\mathbf{x} \neq 0$; **positive semidefinite** (PSD) if ≥ 0. Equivalently: all eigenvalues > 0 (≥ 0); equivalently $A = B^\top B$ for some B (full column rank for PD); PD ⇔ has a Cholesky factorization $A = LL^\top$.



Real-life analogy: a bowl vs. a saddle vs. a gutter

Think of it like dropping a marble on different surfaces. On a **bowl** (positive definite), the marble rolls to one lowest point no matter where you drop it. On a **gutter** (positive semidefinite), it rolls down but can rest anywhere along a flat line. On a **horse saddle** (indefinite), it's low in one direction and high in another, so there's no bottom at all. $\mathbf{x}^\top A\mathbf{x} > 0$ is the test "every direction curves upward".

Formula, read aloud: "A is positive definite" means: whichever direction $\mathbf{x}$ you walk in, the height $\mathbf{x}^\top A\mathbf{x}$ goes up.

Where it breaks: a real bowl is 3-D; in ML the "bowl" lives in thousands of dimensions, and it can be very steep in some directions and nearly flat in others (badly conditioned), which slows optimizers.

Geometry. The quadratic form $\mathbf{x}^\top A\mathbf{x}$ is a bowl opening upward in every direction (PD); PSD allows flat directions (a trough); indefinite has a saddle.

Why ML cares.

Where	Why PD/PSD matters
Covariance matrices	always PSD (variances can't be negative); PD unless some feature combination is constant
Hessian of a loss	PSD everywhere ⇔ convex; PD at a critical point ⇒ strict local minimum
$X^\top X$	always PSD; PD iff X has full column rank; $X^\top X + \lambda I$ is PD for any $\lambda > 0$ → unique ridge solution
Kernels (Gram matrices)	a valid (Mercer) kernel yields PSD Gram matrices for any point set — required for SVMs/GPs
Gaussian distributions	covariance must be PD for a density to exist (Σ^{-1}, $\det\Sigma > 0$)

Interview question: "Why is $X^\top X + \lambda I$ always invertible for $\lambda > 0$?"

Strong answer. "$X^\top X$ is PSD because $\mathbf{v}^\top X^\top X\mathbf{v} = \|X\mathbf{v}\|^2 \geq 0$, so its eigenvalues are nonnegative. Adding λI shifts every eigenvalue up by λ without changing eigenvectors, so all eigenvalues are at least $\lambda > 0$ — the matrix is positive definite, hence invertible, with condition number at most $(\lambda_{\max} + \lambda)/\lambda$.

That's the linear-algebra reason ridge regression has a unique, stable solution even with collinear features or more features than samples."

2.13 Covariance matrices

Definition. For a random vector $\mathbf{x} \in \mathbb{R}^d$ with mean $\boldsymbol{\mu}$: $\Sigma = \mathbb{E}[(\mathbf{x}-\boldsymbol{\mu})(\mathbf{x}-\boldsymbol{\mu})^\top]$, so $\Sigma_{jj} = \mathrm{Var}(x_j)$, $\Sigma_{jk} = \mathrm{Cov}(x_j, x_k)$. Sample version from centered data X_c: $\hat{\Sigma} = \frac{1}{n-1} X_c^\top X_c$.

Real-life analogy: a class attendance register for features

Think of it like a table in a school office that records, for every pair of subjects, whether students who score high in one also score high in the other. The diagonal says how much marks spread within each subject (variance); the off-diagonal entries say whether two subjects rise and fall together (covariance). Maths and physics: strongly together. Maths and drawing: barely related.

Formula, read aloud: $\Sigma = \mathbb{E}[(\mathbf{x}-\mu)(\mathbf{x}-\mu)^\top]$ says: for every pair of features, average (how far feature j is from its mean) × (how far feature k is from its mean).

Where it breaks: covariance only sees *straight-line* togetherness; two features can be strongly related in a curved way and still show near-zero covariance.

Key properties.

- Symmetric and PSD.
- Linear transformation rule: $\mathrm{Cov}(A\mathbf{x}+\mathbf{b}) = A\Sigma A^\top$. So the variance of a projection is $\mathrm{Var}(\mathbf{u}^\top\mathbf{x}) = \mathbf{u}^\top\Sigma\mathbf{u}$ — the quantity PCA maximizes.
- Correlation matrix $= D^{-1/2}\Sigma D^{-1/2}$ with $D = \mathrm{diag}(\Sigma)$: covariance of standardized features.

Mahalanobis distance $d_M(\mathbf{x}, \boldsymbol{\mu}) = \sqrt{(\mathbf{x}-\boldsymbol{\mu})^\top\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})}$ measures distance in units of the data's own spread and correlation: Euclidean distance after whitening. Used in anomaly detection and in Gaussian class-conditional models (LDA/QDA).

Real-world failure

Estimating a $d \times d$ covariance with $n < d$ gives a singular $\hat{\Sigma}$ (rank $\leq n-1$): Mahalanobis distances and Gaussian likelihoods break (infinite/undefined). Even with n a few times d, the smallest eigenvalues are badly underestimated and the largest overestimated (eigenvalue spreading), so $\hat{\Sigma}^{-1}$ explodes along noise directions. Fix: **shrinkage** estimators (Ledoit–Wolf: $\hat{\Sigma}_{\text{shrunk}} = (1-\alpha)\hat{\Sigma} + \alpha\,\mu I$), diagonal approximations, or factor models.

2.14 Projection

Definition. The orthogonal projection of $\mathbf{y}$ onto the column space of X (full column rank) is

$$\hat{\mathbf{y}} = X(X^\top X)^{-1}X^\top\mathbf{y} = H\mathbf{y}, \qquad H^2 = H,\ H^\top = H$$

H is the **hat matrix** ("puts the hat on $\mathbf{y}$"). Onto a single unit vector $\mathbf{u}$: $\mathrm{proj}_{\mathbf{u}}\mathbf{y} = (\mathbf{u}^\top\mathbf{y})\mathbf{u}$, matrix $\mathbf{u}\mathbf{u}^\top$.

Geometry — least squares is a projection. The predictions $X\mathbf{w}$ can only lie in the column space of X (a flat subspace). The closest point to $\mathbf{y}$ in that subspace is the foot of the perpendicular. So the residual $\mathbf{y} - \hat{\mathbf{y}}$ is **orthogonal to every column of** X: $X^\top(\mathbf{y} - X\hat{\mathbf{w}}) = 0$ — which *is* the normal equation.



```
                /|
               / |  residual r = y - y_hat   (perpendicular to the plane)
              /  |
  ___________/___*_______________
 /          0    y_hat = Xw      /   column space of X (all possible predictions)
/_____________________________/
```

Consequences. Residuals are uncorrelated with every feature (and sum to zero if there's an intercept); $\|\mathbf{y}\|^2 = \|\hat{\mathbf{y}}\|^2 + \|\mathbf{r}\|^2$ (Pythagoras → the R^2 decomposition); the diagonal h_{ii} of H is the **leverage** of point i — how much it pulls its own fitted value (used to find influential points; $\operatorname{tr} H = d$ = degrees of freedom).



Real-life analogy: your shadow at noon

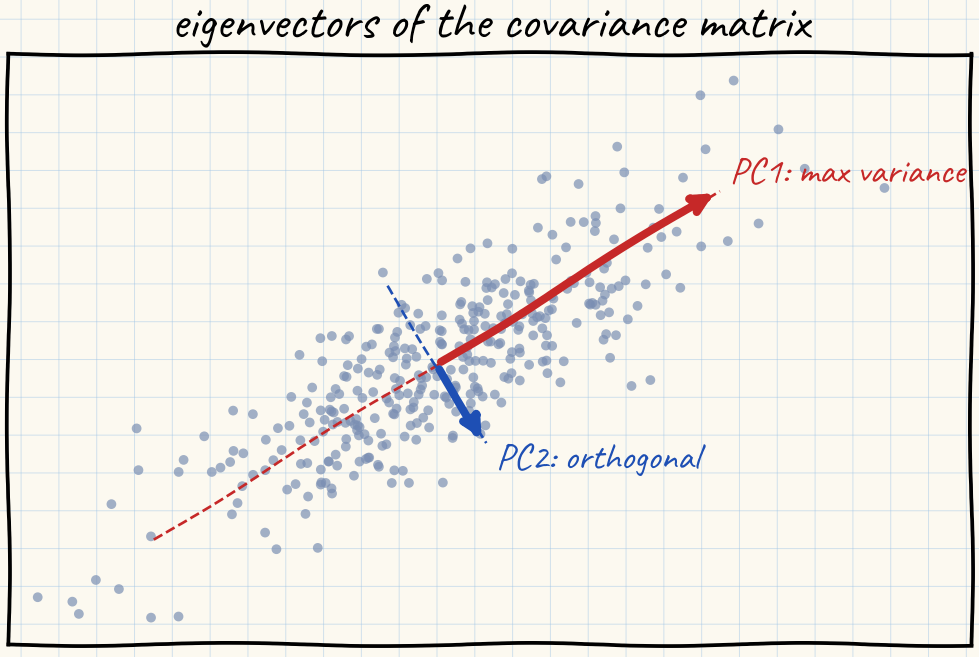
Think of it like the sun directly overhead casting your shadow on the floor. The shadow is the closest point on the floor to you — the part of "you" that the floor can represent. Linear regression does the same: it finds the point in the space the features can reach that is closest to the true answers $\mathbf{y}$; what's left over (your height above the floor) is the residual, which sticks straight out, at a right angle.

Formula, read aloud: $\hat{\mathbf{y}} = H\mathbf{y}$: the predictions are the shadow of the answers; $H^2 = H$ says casting a shadow of a shadow changes nothing.

Where it breaks: a floor is 2-D and the sun is fixed; in regression the "floor" is the span of your features, often high-dimensional, and the right angle is measured in that space.

2.15 PCA geometry (preview)

Put the three pieces together: (1) the covariance matrix describes the data ellipsoid; (2) its eigenvectors are the ellipsoid's axes, eigenvalues the variances along them; (3) projecting the centered data onto the top-k eigenvectors is the orthogonal projection onto the k-dimensional subspace that **keeps the most variance** — equivalently, that **minimizes the squared reconstruction error**. These two views are the same because of Pythagoras: total variance = variance kept + squared distance to the subspace. Full derivation in Part 13.



Principal components are the eigenvectors of the covariance matrix

Real-life analogy: photographing a cricket bat

Think of it like taking one photo that best shows a cricket bat's shape. Photograph it from the side and you see its full length and width; photograph it end-on and it looks like a small rectangle — most of the shape is lost. PCA picks the camera angles that show the most spread (variance) of your data, best angle first.

Where it breaks: a bat's "best view" is obvious to the eye; PCA only looks at spread, so if the important signal lies in a low-spread direction, PCA will throw it away.

2.16 Matrix calculus you must be able to do by hand

Function of $\mathbf{w}$	Gradient	Where it appears
$\mathbf{a}^\top \mathbf{w}$	$\mathbf{a}$	linear terms
$\mathbf{w}^\top A \mathbf{w}$	$(A + A^\top)\mathbf{w}$ ($= 2A\mathbf{w}$ if symmetric)	quadratic losses, L2 penalty ($A = I$: $2\mathbf{w}$)
$\|X\mathbf{w} - \mathbf{y}\|^2$	$2X^\top(X\mathbf{w} - \mathbf{y})$	least squares
$\|\mathbf{w}\|_1$	$\text{sign}(\mathbf{w})$ (subgradient; any value in $[-1, 1]$ at 0)	Lasso
$\log\det\Sigma$ (w.r.t. Σ)	Σ^{-1}	Gaussian MLE
$\sigma(z)$	$\sigma(z)(1 - \sigma(z))$	logistic regression, gates
$\text{softmax}(\mathbf{z})_i$ w.r.t. z_j	$p_i(\delta_{ij} - p_j)$	multiclass classification

Shape rule that catches most mistakes: the gradient of a scalar with respect to a matrix has the **same shape as the matrix**. If L depends on $W \in \mathbb{R}^{m \times n}$ through $\mathbf{z} = W\mathbf{x}$, then $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \mathbf{z}}\mathbf{x}^\top$ — an $(m \times 1)(1 \times n)$ outer product. Check shapes, and you can usually reconstruct the formula.

Real-life analogy: a recipe that scales with the batch

Think of it like a caterer's recipe sheet: if you know how much each ingredient changes the taste of one plate, you can scale that knowledge to 500 plates in one go. Matrix calculus is the rulebook for doing the calculus of many numbers at once, so you don't differentiate every weight separately. The shape rule is the caterer's sanity check: the "per-ingredient adjustment" list must have one entry per ingredient.

Formula, read aloud: $\nabla_{\mathbf{w}}\|X\mathbf{w} - \mathbf{y}\|^2 = 2X^\top(X\mathbf{w} - \mathbf{y})$ says: the direction to change the weights = twice (the features) times (how wrong each prediction currently is).

Where it breaks: recipes scale linearly; gradients of non-linear models change with every step, so the "recipe" must be recomputed at every iteration.

2.17 SVD by hand: a 2×2 worked example

Interviewers sometimes ask you to *compute* an SVD, not just define it. The recipe: eigen-decompose $A^\top A$ to get V and the singular values, then $\mathbf{u}_i = A\mathbf{v}_i/\sigma_i$.

WORKED EXAMPLE: $A = \begin{pmatrix} 3 & 0 \\ 4 & 5 \end{pmatrix}$

1. $A^\top A = \begin{pmatrix} 25 & 20 \\ 20 & 25 \end{pmatrix}$. Eigenvalues: $25 \pm 20 \to 45$ and 5. So $\sigma_1 = \sqrt{45} = 3\sqrt{5} \approx 6.71$, $\sigma_2 = \sqrt{5} \approx 2.24$.

2. Eigenvectors: $\mathbf{v}_1 = \frac{1}{\sqrt{2}}(1,1)$, $\mathbf{v}_2 = \frac{1}{\sqrt{2}}(1,-1)$.
3. $\mathbf{u}_1 = A\mathbf{v}_1/\sigma_1 = \frac{1}{\sqrt{10}}(1,3)$, $\mathbf{u}_2 = A\mathbf{v}_2/\sigma_2 = \frac{1}{\sqrt{10}}(3,-1)$.
4. Check: $\sigma_1\mathbf{u}_1\mathbf{v}_1^\top = \begin{pmatrix}1.5 & 1.5\\ 4.5 & 4.5\end{pmatrix}$, $\sigma_2\mathbf{u}_2\mathbf{v}_2^\top = \begin{pmatrix}1.5 & -1.5\\ -0.5 & 0.5\end{pmatrix}$; they add up to A. ✓
5. **Best rank-1 approximation** is the first term alone. It keeps $\sigma_1^2/(\sigma_1^2+\sigma_2^2) = 45/50 = \mathbf{90\%}$ of the "energy" (squared Frobenius norm), and its error is exactly $\sigma_2 = 2.24$ in spectral norm (Eckart–Young).

Real-life analogy: describing a dance in moves

Think of it like describing any dance as a few basic moves: *turn* to face a direction ($V^\top$), *stretch* along each axis by some amount (Σ), then *turn* again (U). SVD says every matrix is exactly this: rotate, stretch, rotate. The biggest stretches carry most of the motion, so keeping only the top few gives a good summary of the dance.

Formula, read aloud: $A = U\Sigma V^\top = \sum_i \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ says: the matrix is a sum of simple "one-direction-in, one-direction-out" pieces, each weighted by its stretch σ_i.

Where it breaks: real dances can't be undone step by step; in maths, if any stretch is zero, information along that direction is lost for good — the matrix isn't invertible.

2.18 Power iteration: how eigenvectors are actually computed

Nobody inverts or factorizes a billion-by-billion matrix. The simplest iterative method multiplies a random vector by A over and over, renormalizing each time; the component along the dominant eigenvector grows fastest and takes over.

$$\mathbf{v}_{t+1} = \frac{A\mathbf{v}_t}{\|A\mathbf{v}_t\|}, \qquad \lambda \approx \mathbf{v}_t^\top A\mathbf{v}_t, \qquad \text{error} \propto \left|\frac{\lambda_2}{\lambda_1}\right|^t$$

```
def power_iteration(A, n_iter=500, seed=0, tol=1e-12):
    """Dominant eigenpair of a symmetric matrix by repeated multiplication and normalization."""
    v = np.random.default_rng(seed).normal(size=A.shape[0])
    v /= np.linalg.norm(v)
    lam = 0.0
    for _ in range(n_iter):
        w = A @ v
        lam_new = v @ w                                   # Rayleigh quotient
        v = w / np.linalg.norm(w)
        if abs(lam_new - lam) < tol:
            break
        lam = lam_new
    return lam_new, v
```

Checked: on a random 6×6 positive semidefinite matrix it returns 34.7777, matching `numpy.linalg.eigh`; with $\lambda_2/\lambda_1 = 0.43$ the error shrinks by more than half per step. **Where it shows up in ML:** PageRank is power iteration on the web's link matrix; spectral methods, randomized SVD (a block version), and the largest-eigenvalue estimate used to pick safe learning rates.

Real-life analogy: rumours and the loudest voice

Think of it like a rumour passing through a hostel again and again. Whatever the starting mix of stories, after enough rounds the version that spreads fastest drowns out the rest. Repeatedly multiplying by A is the retelling; the dominant eigenvector is the version that wins. If two versions spread at almost the same speed ($\lambda_2 \approx \lambda_1$), it takes a long time for one to take over.

Formula, read aloud: $\mathbf{v} \leftarrow A\mathbf{v}/\|A\mathbf{v}\|$: "apply the matrix, then shrink back to length 1 so the numbers don't blow up".

Where it breaks: rumours spread by people's choices; power iteration is pure arithmetic, and it finds only the top eigenvector — you need deflation or block methods for the next ones.

2.19 Conditioning: why solvers avoid the normal equations

The **condition number** $\kappa(X) = \sigma_{\max}/\sigma_{\min}$ measures how much errors in the input can be amplified in the output. Forming $X^\top X$ **squares** it: $\kappa(X^\top X) = \kappa(X)^2$. In double precision (about 16 significant digits), a condition number near 10^{16} means essentially no correct digits.

```
def lstsq_normal_eq(X, y):
    return np.linalg.solve(X.T @ X, X.T @ y)                    # squares the condition number

def lstsq_qr(X, y):
    Q, R = np.linalg.qr(X)
    return np.linalg.solve(R, Q.T @ y)                          # works with cond(X), not
cond(X)^2
```

Checked on a polynomial (Vandermonde) design with 12 columns on $[0, 1]$: $\kappa(X) = 1.2 \times 10^8$, $\kappa(X^\top X) = 1.4 \times 10^{16}$. The normal equations recover the true coefficients with a maximum error of 3.5×10^{-2}; QR gets 5.2×10^{-10}. This is why libraries solve least squares with QR or SVD, why feature **standardization** and removing near-duplicate features help, and why **ridge** ($X^\top X + \lambda I$ raises the smallest eigenvalue by λ) also stabilizes the arithmetic.

Real-life analogy: measuring a building from a blurry photo

Think of it like estimating the height of a building from a photo taken almost edge-on: a tiny error in measuring the photo becomes a huge error in the answer. An ill-conditioned matrix is that edge-on photo. The normal equations are like photographing the *photo* at the same bad angle — the distortion is applied twice (κ^2). QR works with the original photo.

Where it breaks: a better camera angle is a physical choice; in ML you improve conditioning by rescaling features, dropping redundant ones, or adding ridge — and a well-conditioned problem can still be statistically noisy.

Active recall — Set 2: Linear algebra

Set 2 · Answer from memory, then check the Answer Key

Easy recall

1. State the four ways to view a matrix product.
2. What is the rank of a matrix, in three equivalent ways?

3. Define positive definite in terms of a quadratic form and in terms of eigenvalues.
4. What does the SVD say about *any* matrix, geometrically?
5. Write the hat matrix for least squares.

Conceptual

6. How are singular values of X related to eigenvalues of $X^\top X$?
7. Why is the residual in least squares orthogonal to the columns of X?
8. What does the condition number measure and why does forming $X^\top X$ make it worse?
9. What does the Eckart–Young theorem guarantee?
10. Why does stacking linear layers without activations not increase expressiveness?

Why

11. Why are covariance matrices PSD?
12. Why does PCA use eigenvectors of the covariance matrix?
13. Why does the dummy-variable trap make OLS fail?
14. Why is $\sqrt{d}$ scaling used in dot-product attention?
15. Why should you use `solve` instead of `inv`?

Scenario

16. OLS coefficients flip sign between two bootstrap samples. What linear-algebra problem do you suspect, and how do you confirm and fix it?
17. You have 5,000 features and 800 samples. What happens to $X^\top X$, and what are your options?
18. Gradient descent diverges unless the learning rate is tiny, but then it's extremely slow in some directions. Explain with eigenvalues and propose fixes.
19. Your Gaussian anomaly detector outputs NaNs for 300-dimensional data with 250 samples. Why? Fix it.
20. You need to compress a 10,000 × 10,000 embedding interaction matrix. What do you do and how do you choose the rank?

Mathematical

21. Derive $\nabla_{\mathbf{w}} \|X\mathbf{w} - \mathbf{y}\|^2$.
22. Show that the maximum of $\mathbf{u}^\top \Sigma \mathbf{u}$ subject to $\|\mathbf{u}\| = 1$ is the largest eigenvalue.
23. For $\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ find eigenvalues and eigenvectors.

Difficult follow-ups

24. Gradient descent from zero on an underdetermined least-squares problem converges to which solution, and why?
25. Explain in terms of singular values why ridge regression shrinks some directions more than others.

2027 additions

26. Compute the singular values of $A = \begin{pmatrix} 3 & 0 \\ 4 & 5 \end{pmatrix}$ and the share of energy kept by the best rank-1 approximation.
27. What controls the convergence speed of power iteration?
28. Why do least-squares solvers use QR or SVD instead of the normal equations?

PART 3

Probability

Why probability is the backbone of ML

Most losses are negative log-likelihoods. Most regularizers are log-priors. Classification outputs are conditional probabilities. Evaluation is sampling. If you can go from "assume this distribution" to "therefore minimize this loss", you can derive half the models in this notebook yourself.

The whole Part as one picture

Think of it like a weather office. A distribution describes what weather is typical and how often. Conditional probability is the forecast *given* today's clouds. Bayes' theorem updates the forecast when new radar data arrives. Maximum likelihood picks the climate model under which past records look least surprising; MAP does the same but also trusts decades of experience (the prior). Entropy measures how unpredictable the weather is, and cross-entropy and KL measure how much a wrong forecast costs you. A model's training loss is simply its surprise at what actually happened.

Where it breaks: a weather office can check tomorrow's sky; in ML the true distribution is never seen directly, only samples, so every one of these quantities is an estimate.

3.1 Random variables and distributions

A **random variable** X is a function from outcomes to numbers. Its **distribution** says how probability is spread over values.

- **Discrete:** a **PMF** $p(x) = P(X = x)$ with $\sum_x p(x) = 1$.
- **Continuous:** a **PDF** $f(x)$ with $P(a \le X \le b) = \int_a^b f(x)\,dx$ and $\int f = 1$. A density value is **not** a probability and can exceed 1 (e.g. $\mathcal{N}(0, 0.01^2)$ has peak density ≈ 39.9).
- **CDF** (both cases): $F(x) = P(X \le x)$ — nondecreasing, from 0 to 1; $f = F'$ where differentiable. **Inverse-CDF sampling:** if $U \sim \text{Uniform}(0,1)$ then $F^{-1}(U)$ has CDF F.

Real-life analogy: a chai stall's daily numbers

Think of it like a chai stall. "Cups sold today" is a random variable: a rule that turns a messy day into a number. Counts get a PMF: the chance of exactly 120 cups. The chai's temperature is continuous: the chance of *exactly* 72.000°C is zero, so you ask about a range. The density is how crowded the probability is near each value, like people per metre in a queue, so it can exceed 1.

Formula, read aloud: $P(a \le X \le b) = \int_a^b f(x)\,dx$ says: the chance of landing between a and b = the area under the density curve between a and b.

Where it breaks: a named distribution is an idealized model, not the stall itself; picking the wrong family (say, Gaussian for counts with many zeros) quietly picks the wrong loss.

The distributions you must know cold.

Distribution	Support	PMF / PDF	Mean	Variance	ML role
Bernoulli(p)	$\{0,1\}$	$p^x(1-p)^{1-x}$	p	$p(1-p)$	binary labels → logistic regression, BCE
Binomial(n,p)	$0..n$	$\binom{n}{k}p^k(1-p)^{n-k}$	np	$np(1-p)$	counts of successes, A/B conversions
Categorical($\boldsymbol{\pi}$)	$1..K$	π_k	—	—	multiclass labels → softmax, cross-entropy
Poisson(λ)	$0,1,..$	$\frac{\lambda^k e^{-\lambda}}{k!}$	λ	λ	count targets (Poisson regression), arrivals
Geometric(p)	$1,2,..$	$(1-p)^{k-1}p$	$1/p$	$(1-p)/p^2$	trials until first success
Uniform(a,b)	$[a,b]$	$\frac{1}{b-a}$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	random init, random search
Gaussian(μ,σ^2)	$\mathbb{R}$	$\frac{1}{\sqrt{2\pi}\sigma}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	μ	σ^2	noise model → MSE; CLT; priors → L2
Laplace(μ,b)	$\mathbb{R}$	$\frac{1}{2b}e^{-\lvert x-\mu\rvert/b}$	μ	$2b^2$	noise → MAE; prior → L1
Exponential(λ)	$[0,\infty)$	$\lambda e^{-\lambda x}$	$1/\lambda$	$1/\lambda^2$	waiting times, survival, memoryless
Beta(α,β)	$[0,1]$	$\propto x^{\alpha-1}(1-x)^{\beta-1}$	$\frac{\alpha}{\alpha+\beta}$	—	prior on a probability (conjugate to Bernoulli)

The distribution ⇒ loss dictionary

Gaussian noise ⇒ MSE. Laplace noise ⇒ MAE. Bernoulli label ⇒ binary cross-entropy. Categorical label ⇒ softmax cross-entropy. Poisson count ⇒ Poisson deviance. Gaussian prior ⇒ L2. Laplace prior ⇒ L1. **Choosing a loss is choosing a noise model.**

3.2 Joint, marginal and conditional probability

- **Joint:** $P(X=x, Y=y)$ — probability of both.
- **Marginal:** $P(X=x)=\sum_y P(x,y)$ (sum/integrate out the other variable).
- **Conditional:** $P(Y=y \mid X=x)=\frac{P(x,y)}{P(x)}$ — restrict the world to where $X=x$, renormalize.
- **Chain rule:** $P(x_1,\ldots,x_n)=\prod_i P(x_i \mid x_1,\ldots,x_{i-1})$ — this is exactly how autoregressive language models factorize text.

ML reading: discriminative models learn $P(y \mid x)$ directly; generative models learn $P(x,y)=P(x \mid y)P(y)$ and get $P(y \mid x)$ by Bayes' rule (Naive Bayes, LDA, GMM classifiers).

Real-life analogy: a commuters' survey table

Think of it like a survey table of office commuters: how they travelled (metro or auto) and whether they were late. *Joint:* the share who took the metro **and** were late. *Marginal:* the share who took the metro, whatever happened. *Conditional:* look only at metro riders and ask what share of *them* were late: shrink the world, then renormalize. The chain rule is your phone keyboard guessing each next word from all the words before it.

Formula, read aloud: $P(y \mid x)=\frac{P(x,y)}{P(x)}$ says: chance of being late given metro = (share who were metro-and-late) ÷ (share who took the metro).

Where it breaks: this table has two columns; real models condition on thousands of features where most exact combinations never appear, which is why we learn smooth models instead of counting.

3.3 Independence

$X \perp Y$ iff $P(x, y) = P(x)P(y)$ for all x, y (equivalently $P(y \mid x) = P(y)$). **Conditional independence** $X \perp Y \mid Z$: $P(x, y \mid z) = P(x \mid z)P(y \mid z)$.

Neither implies the other: two symptoms can be dependent overall but independent given the disease (common cause); two independent causes (talent, luck) become dependent once you condition on their common effect (being famous) — **explaining away / collider bias**, the root of many selection biases (Part 4).



Real-life analogy: talent, luck and film stars

Think of it like Bollywood. Across all aspiring actors, talent and lucky breaks are independent: knowing one tells you nothing about the other. But look only at famous stars (you have conditioned on their common *effect*) and they become linked: a star with little talent probably had huge luck. The reverse also happens: wet roads and open umbrellas go together, but once you know it rained (the common *cause*), one tells you nothing more about the other.

Formula, read aloud: $P(x, y) = P(x)P(y)$ says: the chance of both = the chance of one × the chance of the other; learning one changes nothing about the other.

Where it breaks: real data rarely labels the common cause or common effect for you; independence must be argued from how the data was collected, not read off a correlation.

Uncorrelated ≠ independent

$X \sim \text{Uniform}(-1, 1)$, $Y = X^2$: $\text{Cov}(X, Y) = \mathbb{E}[X^3] - \mathbb{E}[X]\mathbb{E}[X^2] = 0$, yet Y is a deterministic function of X. Correlation only detects **linear** dependence. (For jointly Gaussian variables, uncorrelated *does* imply independent.)

3.4 Bayes' theorem

$$P(H \mid E) = \frac{P(E \mid H)\,P(H)}{P(E)} = \frac{P(E \mid H)P(H)}{P(E \mid H)P(H) + P(E \mid \neg H)P(\neg H)} \qquad \text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}}$$



Real-life analogy: the beeping metal detector

Think of it like the metal detector at a busy railway station. It beeps for almost every real weapon, but also for belt buckles and keys, and weapons are extremely rare. So when it beeps, it is still far more likely to be a buckle. The guard's belief after the beep must weigh how good the detector is against how rare the thing it looks for is.

Formula, read aloud: $P(H \mid E) = \frac{P(E|H)P(H)}{P(E)}$ says: belief after the beep = (chance of a beep given a weapon) × (how rare weapons are to begin with) ÷ (how often it beeps at all).

Where it breaks: the guard can open the bag and check; Bayes' rule only reweights the numbers you give it, so a wrong prior or false-alarm rate gives a confidently wrong answer.

Worked example: the base-rate trap

A disease has prevalence 1%. A test has sensitivity (TPR) 90% and false-positive rate 5%. You test positive. $P(D \mid +)$?

$$P(D \mid +) = \frac{0.9 \times 0.01}{0.9 \times 0.01 + 0.05 \times 0.99} = \frac{0.009}{0.009 + 0.0495} \approx 0.154$$

Only **15%**. Among 10,000 people: 100 sick → 90 true positives; 9,900 healthy → 495 false positives. Positives are mostly false because healthy people vastly outnumber sick ones. **This is the fraud-detection problem**: at 0.1% prevalence, even a 99%-specific model produces mostly false alarms — precision is governed by the base rate.

Odds form (fastest in interviews): posterior odds = prior odds × likelihood ratio. Prior odds $1:99$; LR $= 0.9/0.05 = 18$; posterior odds $18:99$ → probability $18/117 \approx 0.154$.

3.5 Expectation, variance, covariance, correlation

Expectation $\mathbb{E}[X] = \sum_x x\,p(x)$ or $\int x f(x)\,dx$. **Linearity** holds always, even for dependent variables: $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$. (Powerful trick: expected number of fixed points of a random permutation is $\sum_i P(\pi(i) = i) = n \cdot \frac{1}{n} = 1$.)

Variance $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$; $\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X)$.

Covariance $\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)]$; $\mathrm{Var}(X + Y) = \mathrm{Var}\,X + \mathrm{Var}\,Y + 2\,\mathrm{Cov}(X, Y)$.

Correlation $\rho = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1, 1]$: scale-free strength of **linear** association.

Why averaging reduces variance — the formula behind bagging, ensembling, mini-batches

For n variables each with variance σ^2 and pairwise correlation ρ:

$$\mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \rho\,\sigma^2 + \frac{1-\rho}{n}\,\sigma^2$$

Independent ($\rho = 0$): variance falls as σ^2/n. Correlated: it can never go below $\rho\sigma^2$, however many you average. That single formula explains why random forests decorrelate trees (feature subsampling lowers ρ), why larger mini-batches give less noisy gradients (with diminishing returns), and why ensembling five copies of the same model trained on the same data barely helps.

Real-life analogy: guessing the sweets in a jar

Think of it like a fair where everyone guesses how many sweets are in a jar. The expectation is the long-run average guess; the variance is how scattered the guesses are; covariance says whether two people's guesses move together (because they whispered to each other). Averaging many *independent* guesses cancels individual errors. Averaging friends who copied each other doesn't: they share the same mistake.

Formula, read aloud: $\mathrm{Var}\left(\frac{1}{n}\sum_i X_i\right) = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$ says: wobble of the crowd's average = the shared part of everyone's error (never shrinks) + the private part, divided by the crowd size n.

Where it breaks: averaging shrinks scatter, not shared bias: if everyone underestimates big jars, the crowd's average underestimates too, however large the crowd.

3.6 Law of total probability and total expectation

$$P(A) = \sum_i P(A \mid B_i)P(B_i), \qquad \mathbb{E}[Y] = \mathbb{E}\big[\mathbb{E}[Y \mid X]\big], \qquad \mathrm{Var}(Y) = \underbrace{\mathbb{E}[\mathrm{Var}(Y \mid X)]}_{\text{unexplained}} + \underbrace{\mathrm{Var}(\mathbb{E}[Y \mid X])}_{\text{explained by } X}$$

ML reading of the total-variance law: the first term is the **irreducible noise** even the Bayes-optimal regressor can't remove; the second is what a perfect model of $\mathbb{E}[Y \mid X]$ captures. The best achievable R^2 is $\mathrm{Var}(\mathbb{E}[Y \mid X])/\mathrm{Var}(Y)$.

Real-life analogy: marks across school sections

Think of it like a school's exam marks split by section (A, B, C). The school average is the average of each section's average, weighted by section size: that's total expectation. The total spread of marks = the typical spread *inside* each section (differences the section label can't explain) + the spread *between* section averages (what knowing the section explains).

Formula, read aloud: $\mathrm{Var}(Y) = \mathbb{E}[\mathrm{Var}(Y \mid X)] + \mathrm{Var}(\mathbb{E}[Y \mid X])$ says: total spread = average spread within groups + spread of the group averages. The best possible R^2 is the second part ÷ the total.

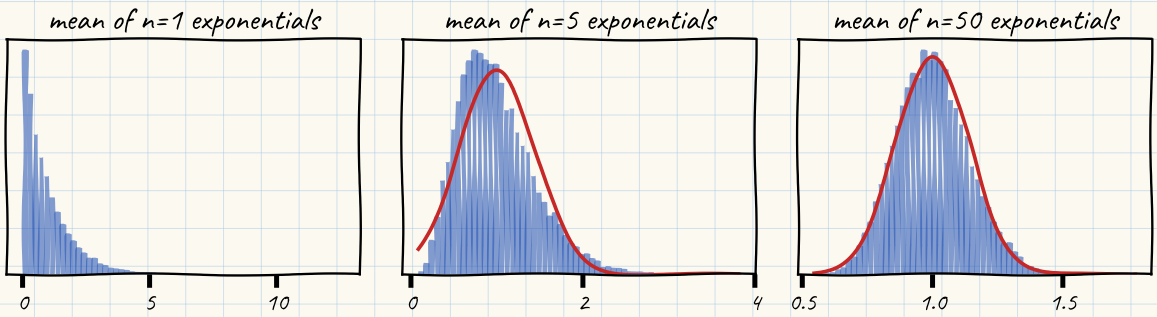
Where it breaks: school sections are neat, separate groups; in ML X is usually continuous and high-dimensional, so $\mathbb{E}[Y \mid X]$ must be learned, not just counted.

3.7 Law of large numbers and central limit theorem

LLN: for IID X_i with mean μ, $\bar{X}_n \to \mu$ as $n \to \infty$. *Why ML cares:* empirical risk → expected risk (for a fixed model); Monte-Carlo estimates converge; test-set metrics estimate true performance.

CLT: if additionally $\mathrm{Var}(X_i) = \sigma^2 < \infty$,

$$\sqrt{n}\,(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2), \qquad \text{i.e. } \bar{X}_n \approx \mathcal{N}\Big(\mu, \frac{\sigma^2}{n}\Big)$$



Means of skewed exponential samples become Gaussian as n grows

Why ML cares: confidence intervals for metrics and A/B tests; the $1/\sqrt{n}$ law — **to halve the error bar you need 4× the data**; mini-batch gradients are approximately Gaussian around the full gradient with variance $\propto 1/B$.

Real-life analogy: a batter's average settling down

Think of it like a batter's average. After 5 innings it jumps around wildly; after 100 innings it settles near their true skill: that's the law of large numbers. The CLT adds that, over many stretches of n innings, those averages form a bell curve around the true skill, and the bell narrows like $1/\sqrt{n}$. To halve the uncertainty you need four times as many innings.

Formula, read aloud: $\bar{X}_n \approx \mathcal{N}(\mu, \sigma^2/n)$ says: the average of n innings is roughly bell-shaped around the true value μ, with spread $\sigma/\sqrt{n}$.

Where it breaks: a batter's form changes over a career (not IID), and a few enormous scores (heavy tails) can keep the average jumpy far longer than the formula promises.

CLT traps

(1) Heavy tails: with infinite variance (e.g. Pareto with $\alpha \leq 2$, some revenue distributions) the CLT does not apply as stated; with finite but huge variance, convergence can need enormous n — revenue-per-user A/B tests are the classic case (use trimming/winsorizing, log transforms, or bootstrap). (2) Dependence: for correlated samples, the variance of the mean is larger than σ^2/n — naive CIs are too narrow.

3.8 Maximum likelihood estimation (MLE)

Idea. Choose the parameters under which the observed data was most probable.

$$\hat{\theta}_{\text{MLE}} = \arg\max_\theta \prod_{i=1}^{n} p(x_i \mid \theta) = \arg\max_\theta \sum_{i=1}^{n} \log p(x_i \mid \theta) = \arg\min_\theta \underbrace{-\frac{1}{n}\sum_i \log p(x_i \mid \theta)}_{\text{NLL: this is the loss}}$$

Real-life analogy: the suspect whose story fits best

Think of it like a detective testing suspects one by one. For each suspect (a parameter value θ) she asks: "If this person did it, how likely is exactly the evidence we found?" She picks the one under whom the evidence is least surprising. She ignores how suspicious anyone looked *before* the evidence; adding that is MAP.

Formula, read aloud: $\hat{\theta}_{\text{MLE}} = \arg\max_\theta \prod_i p(x_i \mid \theta)$ says: pick the setting that makes all the observed data most probable together; equivalently, the one with the smallest average surprise $-\frac{1}{n}\sum_i \log p(x_i \mid \theta)$, and that average surprise *is* the training loss.

Where it breaks: a real detective would never convict on three clues alone; plain MLE does, which is why 3 heads in 3 flips makes it declare tails impossible.

Derivation 1 — Bernoulli (coin with unknown p)

Data: k heads in n flips. Log-likelihood: $\ell(p) = k\log p + (n-k)\log(1-p)$.

$$\frac{d\ell}{dp} = \frac{k}{p} - \frac{n-k}{1-p} = 0 \;\Rightarrow\; k(1-p) = (n-k)p \;\Rightarrow\; \hat{p} = \frac{k}{n}$$

Second derivative $-\frac{k}{p^2} - \frac{n-k}{(1-p)^2} < 0$ ⇒ maximum. The MLE is the sample frequency. **Note the weakness:** 3 heads in 3 flips ⇒ $\hat{p} = 1$, "tails is impossible". MAP fixes this below.

Derivation 2 — Gaussian with unknown mean and variance

$\ell(\mu, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_i (x_i - \mu)^2$.

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2}\sum_i (x_i - \mu) = 0 \Rightarrow \hat{\mu} = \bar{x}; \qquad \frac{\partial \ell}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4}\sum_i (x_i - \mu)^2 = 0 \Rightarrow \hat{\sigma}^2 = \frac{1}{n}\sum_i (x_i - \bar{x})^2$$

$\hat{\sigma}^2_{\text{MLE}}$ is **biased**: $\mathbb{E}[\hat{\sigma}^2] = \frac{n-1}{n}\sigma^2$, because $\bar{x}$ was fitted to the same data and sits closer to the points than the true μ. Dividing by $n - 1$ (Bessel's correction) removes the bias. MLE is consistent but not necessarily unbiased.

Derivation 3 — why MSE is MLE under Gaussian noise

Model: $y_i = \mathbf{w}^\top \mathbf{x}_i + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ IID. Then

$$-\log \prod_i p(y_i \mid \mathbf{x}_i, \mathbf{w}) = \frac{1}{2\sigma^2}\sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \frac{n}{2}\log(2\pi\sigma^2)$$

The second term doesn't depend on $\mathbf{w}$, so maximizing likelihood ⇔ **minimizing the sum of squared errors**. Linear regression's loss is not arbitrary: it's the Gaussian noise assumption. Same argument with Laplace noise gives MAE; with Bernoulli labels, binary cross-entropy (Part 6).

Properties worth quoting: under regularity conditions MLE is **consistent** (converges to the true θ), **asymptotically normal** with variance given by the inverse Fisher information, and **asymptotically efficient** (achieves the Cramér–Rao lower bound). It's also **invariant**: the MLE of $g(\theta)$ is $g(\hat{\theta})$. Its weaknesses: overfits with small data (no prior), can be degenerate (a Gaussian mixture component collapsing onto one point → infinite likelihood), and is only as good as the assumed model.

3.9 Why optimize the log-likelihood instead of the likelihood?

Six reasons, in the order an interviewer wants them

1. **Same answer.** $\log$ is strictly increasing, so $\arg\max$ is unchanged.
2. **Numerical underflow.** A product of 10^6 probabilities around 0.5 is $\approx 10^{-301030}$ — far below the smallest float64 ($\sim 10^{-308}$). The log turns it into a sum of 10^6 ordinary numbers.
3. **Products become sums.** Sums over independent examples decompose: the gradient is a sum of per-example gradients, so a mini-batch average is an **unbiased estimate** of the full gradient — this is what makes SGD work.
4. **Nicer optimization geometry.** For exponential-family models the log-likelihood is concave in natural parameters (e.g. logistic regression's NLL is convex), while the likelihood itself is not. Exponentials in densities cancel.
5. **Better-behaved gradients.** $\nabla \prod_i p_i = (\prod_i p_i) \sum_i \nabla \log p_i$ — the raw likelihood's gradient is scaled by the tiny product itself and vanishes numerically.
6. **Information-theoretic meaning.** The average negative log-likelihood is the cross-entropy between the data distribution and the model; minimizing it minimizes $\mathrm{KL}(P_{\text{data}} \,\|\, P_\theta)$.

Real-life analogy: counting zeros instead of multiplying

Think of it like working out the chance that all 1,000 flights at an airport land on time today, each with its own chance. Multiply 1,000 fractions and the answer has so many zeros after the decimal point that a calculator just shows 0. Instead, like the Richter scale for earthquakes, you track the *number of zeros* (the log), and multiplying chances becomes adding scores.

Formula, read aloud: $\log \prod_i p_i = \sum_i \log p_i$ says: the log of a product = the sum of the logs; and since log always rises when its input rises, the best θ is exactly the same.

Where it breaks: logs fix the arithmetic and give nicer gradients, but they can't fix a wrong model: the maximum lands on the same point either way.

3.10 Maximum a posteriori (MAP)

Idea. Combine the likelihood with a prior belief about parameters, and take the most probable parameters **after** seeing data.

$$\hat{\theta}_{\text{MAP}} = \arg\max_\theta p(\theta \mid D) = \arg\max_\theta \frac{p(D \mid \theta)p(\theta)}{p(D)} = \arg\min_\theta \Big[\underbrace{-\sum_i \log p(x_i \mid \theta)}_{\text{data loss}} \underbrace{- \log p(\theta)}_{\text{regularizer}}\Big]$$

$p(D)$ doesn't depend on θ, so it drops out.

Real-life analogy: star ratings for a brand-new restaurant

Think of it like a food app rating a new restaurant with just two reviews, both 5 stars. Taking the raw average (MLE) says "perfect 5.0", which nobody believes. A sensible app mixes in a prior ("most places here are about 3.8"), as if adding a few pretend average reviews. After 1,000 real reviews the pretend ones barely matter, so MAP drifts to MLE.

Formula, read aloud: $\hat{\theta}_{\text{MAP}} = \arg\min_\theta \big[-\sum_i \log p(x_i \mid \theta) - \log p(\theta)\big]$ says: best setting = fit the data (data loss) + don't wander far from what was plausible beforehand (the regularizer). A Gaussian prior gives ridge; a Laplace prior gives lasso.

Where it breaks: MAP still reports one number and throws away how unsure it is; full Bayesian inference keeps the whole range of plausible ratings.

Derivation — Gaussian prior ⇒ ridge; Laplace prior ⇒ lasso

Likelihood: $y_i \sim \mathcal{N}(\mathbf{w}^\top \mathbf{x}_i, \sigma^2)$. Prior: $w_j \sim \mathcal{N}(0, \tau^2)$ independently.

$$-\log p(\mathbf{w} \mid D) = \frac{1}{2\sigma^2}\|\mathbf{y} - X\mathbf{w}\|^2 + \frac{1}{2\tau^2}\|\mathbf{w}\|_2^2 + \text{const}$$

Multiply by $2\sigma^2$: minimize $\|\mathbf{y} - X\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|_2^2$ with $\boxed{\lambda = \sigma^2/\tau^2}$. Noisier data (big σ^2) or a tighter prior (small τ^2) ⇒ stronger regularization — exactly the intuition.

With a Laplace prior $p(w_j) \propto e^{-|w_j|/b}$: minimize $\|\mathbf{y} - X\mathbf{w}\|^2 + \frac{2\sigma^2}{b}\|\mathbf{w}\|_1$ — **lasso**. The Laplace density has a sharp peak at 0, which is why MAP solutions land exactly on zero.

Derivation — Beta prior on a coin ⇒ smoothing

Prior $p \sim \text{Beta}(\alpha, \beta) \propto p^{\alpha-1}(1-p)^{\beta-1}$; data k heads of n. Posterior $\propto p^{k+\alpha-1}(1-p)^{n-k+\beta-1}$, i.e. $\text{Beta}(\alpha + k,\ \beta + n - k)$ — **conjugacy**: the prior's form is preserved.

$$\hat{p}_{\text{MAP}} = \frac{k+\alpha-1}{n+\alpha+\beta-2}, \qquad \mathbb{E}[p \mid D] = \frac{k+\alpha}{n+\alpha+\beta}$$

With $\alpha = \beta = 2$ (MAP) or $\alpha = \beta = 1$ (posterior mean): $\frac{k+1}{n+2}$ — **Laplace smoothing**. 3 heads in 3 flips ⇒ 0.8, not 1. The prior acts like $\alpha + \beta - 2$ pseudo-observations; as $n \to \infty$, MAP → MLE. Same trick: Naive Bayes word counts, CTR estimates for new ads, target encoding with smoothing.

MAP is not "fully Bayesian"

MAP returns a single point (the posterior mode). It ignores posterior uncertainty and is **not invariant to reparametrization** (the mode of $p(\theta)$ and of $p(\log\theta)$ differ). Full Bayesian inference keeps the whole posterior.

3.11 Bayesian inference: prior, likelihood, posterior

Term	Symbol	Meaning
Prior	$p(\theta)$	belief about parameters before data
Likelihood	$p(D \mid \theta)$	how probable the data is for each θ (a function of θ, **not** a distribution over θ)
Evidence / marginal likelihood	$p(D) = \int p(D \mid \theta)p(\theta)d\theta$	normalizer; used to compare models (automatic Occam's razor)
Posterior	$p(\theta \mid D)$	updated belief
Posterior predictive	$p(x_{\text{new}} \mid D) = \int p(x_{\text{new}} \mid \theta)\, p(\theta \mid D)\, d\theta$	predictions averaging over parameter uncertainty

Why it matters in practice: uncertainty estimates (Bayesian linear regression, Gaussian processes), small-data problems, principled smoothing, Thompson sampling in bandits (sample θ from the posterior, act greedily), hyperparameter tuning (Bayesian optimization). **Cost:** posteriors are usually intractable; you approximate with conjugacy, MCMC, variational inference, or Laplace approximations. Deep learning mostly uses MAP (weight decay) plus ensembles as a practical proxy for posterior averaging.



Real-life analogy: a panel of doctors instead of one

Think of it like consulting a panel of doctors. Each doctor is one possible model θ. The prior is how much you trusted each one before; the likelihood is how well each one's past calls matched what happened; the posterior is your updated trust. For a new patient, average all their opinions, weighted by that trust. If the trusted doctors disagree, you know you're uncertain.

Formula, read aloud: $p(x_{\text{new}} \mid D) = \int p(x_{\text{new}} \mid \theta)\, p(\theta \mid D)\, d\theta$ says: prediction for a new case = every possible model's prediction, averaged with weights equal to how believable that model is after seeing the data.

Where it breaks: a real panel has a handful of doctors; there are infinitely many θ, so the average usually can't be done exactly and must be approximated (MCMC, variational inference, ensembles).

3.12 Entropy

$$H(P) = -\sum_x p(x)\log p(x) = \mathbb{E}_P[-\log p(X)]$$

Intuition: the average surprise, or the minimum average number of bits (with $\log_2$) needed to encode samples from P. Maximal for uniform distributions ($\log K$ for K outcomes), zero for a deterministic one. A fair coin: 1 bit; a 90/10 coin: $-0.9\log_2 0.9 - 0.1\log_2 0.1 \approx 0.469$ bits.

ML uses: decision-tree splitting (information gain = entropy reduction, Part 7); the maximum-entropy principle (logistic regression / softmax is the max-ent model matching feature expectations); entropy regularization in RL; uncertainty sampling in active learning; label smoothing raises the target's entropy.



Real-life analogy: how surprised you are by the news

Think of it like reading tomorrow's weather in Chennai in April: "hot and humid" — no surprise, almost no information. In a place where anything can happen, each forecast surprises you more. Entropy is the *average surprise* of a source: zero when the outcome is certain, largest when all outcomes are equally likely.

Formula, read aloud: $H(P) = -\sum p(x)\log p(x)$ says: for each outcome, take how surprising it is ($-\log p$) and weight it by how often it happens, then add up.

Where it breaks: "surprise" sounds emotional; entropy is precise — it is the minimum average number of yes/no questions (bits) needed to pin down the outcome.

3.13 Cross-entropy

$$H(P,Q) = -\sum_x p(x)\log q(x) = H(P) + \mathrm{KL}(P\,\|\,Q)$$

Intuition: the average number of bits to encode samples from P using a code optimized for Q. It's $\geq H(P)$, with equality iff $Q = P$.

Why it's the classification loss: with a one-hot target P (all mass on the true class y), $H(P,Q) = -\log q(y)$ — the negative log-likelihood of the true class. Since $H(P)$ doesn't depend on the model, **minimizing cross-entropy = minimizing KL to the data = maximum likelihood.**



Real-life analogy: packing for the wrong weather

Think of it like packing for a trip using the wrong city's weather report. If you pack for Shimla but land in Goa, you carry lots of useless sweaters and not enough cotton — you pay extra for the mismatch. Cross-entropy is the cost of describing reality P using your beliefs Q; it is smallest when your beliefs match reality.

Formula, read aloud: $H(P,Q) = H(P) + \mathrm{KL}(P\|Q)$ says: the cost with wrong beliefs = the unavoidable cost of reality's own uncertainty + an extra penalty for being wrong.

Where it breaks: in classification we only see one true label per example, so "reality" P is a single certain outcome, and cross-entropy becomes simply $-\log$(probability you gave to the right answer).

3.14 KL divergence

$$\mathrm{KL}(P\,\|\,Q) = \sum_x p(x)\log\frac{p(x)}{q(x)} \geq 0, \qquad = 0 \iff P = Q$$

(Non-negativity — Gibbs' inequality — follows from Jensen: $-\mathrm{KL} = \mathbb{E}_P[\log \frac{q}{p}] \leq \log \mathbb{E}_P[\frac{q}{p}] = \log \sum q = 0$.)

Real-life analogy: the extra fare for the wrong route map

Think of it like navigating Mumbai with a map of Pune. You'll still get places, but you'll take longer routes and pay extra auto fare. KL divergence is that *extra* cost: how many more bits (or rupees) you waste because you used Q's map when the city is really P. With the right map, the extra cost is zero; it's never negative.

Formula, read aloud: $\mathrm{KL}(P\|Q) = \sum p(x) \log \frac{p(x)}{q(x)}$ says: over what really happens, average how much more you under-expected it than you should have.

Where it breaks: a real extra fare is the same whichever way you compare maps; KL is *not symmetric* — $\mathrm{KL}(P\|Q) \neq \mathrm{KL}(Q\|P)$ — which is why "forward" and "reverse" KL behave differently.

Not a distance: it's asymmetric and violates the triangle inequality. The asymmetry matters:

	Forward $\mathrm{KL}(P\|Q)$ (fit Q to data P)	Reverse $\mathrm{KL}(Q\|P)$
Penalizes	$q \approx 0$ where $p > 0$ (infinite penalty)	$q > 0$ where $p \approx 0$
Behavior	**mass-covering / mean-seeking**: Q spreads to cover all modes	**mode-seeking**: Q locks onto one mode
Where	MLE, cross-entropy training, knowledge distillation	variational inference (ELBO), some RL policy objectives

Other ML appearances: the VAE objective's KL term to the prior; PPO/RLHF KL penalties keeping a fine-tuned policy near a reference model; drift monitoring (PSI is a symmetrized KL on binned features); t-SNE minimizes $\mathrm{KL}(P\|Q)$ between neighbor distributions.

Interview question: "What's the relationship between cross-entropy, KL divergence and maximum likelihood?"

Strong answer. "Cross-entropy of the model Q relative to the data distribution P equals the entropy of P plus $\mathrm{KL}(P\|Q)$. The entropy of the data is a constant with respect to the model, so minimizing cross-entropy is minimizing forward KL from data to model. And the empirical cross-entropy — the average of minus log q of the observed labels — is exactly the average negative log-likelihood, so it's also maximum likelihood. Three names, one objective. The forward direction means the model is heavily penalized for assigning near-zero probability to anything that actually happens, which is why cross-entropy-trained models are mass-covering and why confident mistakes are so expensive."

3.15 Quick probability problems interviewers like

Worked problems (try before reading the solution)

1. Expected coin flips until two heads in a row. Let E = expected flips from scratch, E_H = after one head. $E = 1 + \frac{1}{2}E_H + \frac{1}{2}E$; $E_H = 1 + \frac{1}{2}\cdot 0 + \frac{1}{2}E$. Solve: $E_H = 1 + E/2$, $E = 2 + E_H = 3 + E/2 \Rightarrow E = 6$.

2. Probability of at least one false positive in 20 independent tests at α = 0.05. $1 - 0.95^{20} \approx 0.64$. This is why you correct for multiple testing (Part 4).

3. Expected maximum of two independent Uniform(0,1). $P(\max \le t) = t^2 \Rightarrow$ density $2t \Rightarrow$ $\mathbb{E} = \int_0^1 2t^2 dt = 2/3$. Generally $n/(n+1)$ for n uniforms.

4. A model outputs 0.9 for 100 emails; 70 are actually spam. Is it calibrated at that score? No — among items scored 0.9, ~90% should be positive; 70% means overconfidence (Part 6 on calibration).

5. Birthday problem: with 23 people, $P(\text{shared birthday}) = 1 - \prod_{i=0}^{22} \frac{365-i}{365} \approx 0.507$. ML echo: hash collisions in the hashing trick appear far sooner than intuition suggests — with m buckets expect collisions after about $\sqrt{m}$ items.

6. You sample users until you find one who converted (rate 2%). Expected number sampled? Geometric: $1/p = 50$.



Real-life analogy: puzzles as net practice

Think of it like a batter facing a bowling machine: each puzzle drills one shot — conditioning, expectation by first step, complements ("at least one" = 1 − "none"), geometric waiting times. Interviewers care less about the final number than whether you pick the right shot quickly and explain it out loud.

Where it breaks: real interview puzzles are often slightly twisted versions of these; learn the technique (set up states, use complements, condition on the first step), not the answers.

3.16 The multivariate Gaussian

The bell curve in d dimensions, described by a mean vector $\boldsymbol{\mu}$ and a covariance matrix Σ:

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\Big)$$

The exponent contains the squared **Mahalanobis distance** $(\mathbf{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})$ — distance measured in units of the data's own spread, accounting for correlations. Contours of equal density are ellipses aligned with the eigenvectors of Σ (Part 2).

Conditioning — the property interviewers probe. Split $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2)$. Then $\mathbf{x}_1 \mid \mathbf{x}_2$ is again Gaussian:

$$\boldsymbol{\mu}_{1|2} = \boldsymbol{\mu}_1 + \Sigma_{12}\Sigma_{22}^{-1}(\mathbf{x}_2 - \boldsymbol{\mu}_2), \qquad \Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

The conditional mean is **linear** in what you observed (that's linear regression), and the conditional variance **doesn't depend on the observed value** and is never larger than the unconditional one. Gaussian processes, Kalman filters and linear discriminant analysis are built on these two lines.

Worked example — regression to the mean

Fathers' and sons' heights: both $\mathcal{N}(175, 7^2)$ cm, correlation $\rho = 0.5$. A father is 189 cm (2 SD above average). In 2-D the formula becomes $\mu_{1|2} = \mu_1 + \rho\frac{\sigma_1}{\sigma_2}(x_2 - \mu_2) = 175 + 0.5 \times 14 = \mathbf{182\ cm}$, with conditional SD $7\sqrt{1-\rho^2} = \mathbf{6.1\ cm}$. The son is expected to be tall, but only half as unusual: **regression to the mean** — the same reason a model's best-performing week is usually followed by a more ordinary one.



Real-life analogy: guessing a son's height from his father's

Think of it like the example above: knowing the father is tall shifts your guess for the son upward, but not all the way, because height is only partly inherited. And knowing the father narrows your uncertainty a little, whatever his height. The multivariate Gaussian turns "how strongly things move together" (Σ) into exactly this kind of best guess.

Formula, read aloud: conditional mean = (usual average) + (how strongly the two are linked ÷ how variable the clue is) × (how unusual the clue is).

Where it breaks: real data often has heavy tails, skew or non-linear links; the neat "straight-line best guess" is exact only when everything is jointly Gaussian.

3.17 Exponential families and GLMs

Most distributions you meet — Bernoulli, categorical, Gaussian, Poisson, exponential, gamma, Beta — share one form:

$$p(x \mid \eta) = h(x)\exp\left(\eta^\top T(x) - A(\eta)\right)$$

with **natural parameter** η, **sufficient statistic** $T(x)$ and **log-partition** $A(\eta)$ (the normalizer). The magic property: derivatives of A give the moments — $A'(\eta) = \mathbb{E}[T(x)]$, $A''(\eta) = \mathrm{Var}[T(x)] \geq 0$ — so the log-likelihood is **concave** in η.

Distribution	Natural parameter η	$A'(\eta)$ = mean	GLM it gives
Bernoulli(p)	$\log\frac{p}{1-p}$ — the **logit**	$\sigma(\eta)$	logistic regression
Categorical	log-odds vs. a reference class	softmax	multinomial logistic regression
Gaussian (known σ^2)	μ/σ^2	μ	linear regression
Poisson(λ)	$\log\lambda$	e^η	Poisson regression (counts)

A **generalized linear model** sets $\eta = \mathbf{w}^\top\mathbf{x}$ (the *canonical link*). Then the gradient of the log-likelihood is always

$$\nabla_{\mathbf{w}} \log L = \sum_i (y_i - \mu_i)\mathbf{x}_i \qquad (\mu_i = A'(\mathbf{w}^\top\mathbf{x}_i))$$

— "observed minus expected, times the input". That is why linear, logistic, softmax and Poisson regression all share the update $X^\top(\mathbf{y} - \boldsymbol{\mu})$ (Parts 5–6), and why their losses are convex.

Real-life analogy: one engine, many car bodies

Think of it like a car company that builds one engine and fits it into a hatchback, a sedan and an SUV. The exponential family is the shared engine; Bernoulli, Poisson and Gaussian are different bodies. Because the engine is the same, the controls work the same way in every model: the learning signal is always "what happened minus what we expected".

Formula, read aloud: $\nabla \log L = \sum (y_i - \mu_i)\mathbf{x}_i$ says: for each example, take (actual – predicted average), multiply by its features, and add up.

Where it breaks: not every useful model fits the engine — mixtures, Student-t noise and most neural networks' outputs over many layers are not exponential-family GLMs, and lose the convexity guarantee.

3.18 Fisher information and the Cramér–Rao bound

How much can data tell you about a parameter? **Fisher information** measures the curvature of the log-likelihood around the truth:

$$I(\theta) = \mathbb{E}\Big[\Big(\frac{\partial \log p(x \mid \theta)}{\partial \theta}\Big)^2\Big] = -\mathbb{E}\Big[\frac{\partial^2 \log p(x \mid \theta)}{\partial \theta^2}\Big], \qquad \operatorname{Var}(\hat{\theta}) \;\geq\; \frac{1}{n\, I(\theta)}$$

The second statement is the **Cramér–Rao bound**: no unbiased estimator can have lower variance. The MLE achieves it asymptotically: $\hat{\theta} \approx \mathcal{N}\big(\theta,\ 1/(nI(\theta))\big)$ — the basis of standard errors in regression output.

Worked example — a coin (Bernoulli)

$\log p(x \mid p) = x \log p + (1-x)\log(1-p)$. Second derivative: $-\frac{x}{p^2} - \frac{1-x}{(1-p)^2}$. Taking the expectation ($\mathbb{E}x = p$): $I(p) = \frac{1}{p} + \frac{1}{1-p} = \frac{1}{p(1-p)}$.

Bound: $\operatorname{Var}(\hat{p}) \geq \frac{p(1-p)}{n}$ — exactly the variance of the sample proportion. So the plain average of coin flips is the **best possible** unbiased estimator. Information is lowest at $p = 0.5$ (a fair coin is hardest to pin down) and highest near 0 or 1.

Where it appears in ML: standard errors and confidence intervals for model coefficients; the **natural gradient** (precondition by the inverse Fisher matrix); Laplace approximations; elastic weight consolidation (keep weights with high Fisher information close to old values to limit forgetting); and experimental design (collect data where information is highest).

Real-life analogy: a sharp peak vs. a flat plateau

Think of it like finding the top of a hill in fog. On a sharp, pointed peak, a few steps tell you exactly where the summit is; on a broad plateau, you could be anywhere on the flat top. Fisher information is the sharpness of the log-likelihood's peak: sharp peak = lots of information = small uncertainty in your estimate.

Formula, read aloud: $\operatorname{Var}(\hat{\theta}) \geq \frac{1}{nI(\theta)}$: "your estimate can't wobble less than one over (number of examples × information per example)".

Where it breaks: the bound is for *unbiased* estimators; biased ones (like ridge or shrinkage) can beat it in mean squared error by trading a little bias for much less variance.

3.19 Sampling methods: generating and estimating with randomness

Method	Idea	When
Inverse transform	if $U \sim \text{Uniform}(0,1)$, then $F^{-1}(U)$ has CDF F	1-D distributions with an invertible CDF (exponential: $-\ln U/\lambda$)
Rejection sampling	propose from an easy q with $p \le Mq$; accept with probability $\frac{p(x)}{Mq(x)}$	low dimensions; acceptance rate $1/M$ collapses in high dimensions
Importance sampling	sample from q, reweight by $w = p/q$: $\mathbb{E}_p[f] = \mathbb{E}_q[f\,w]$	rare events; off-policy evaluation (Part 47)
MCMC (Metropolis–Hastings)	random walk that accepts moves with $\min(1, \frac{p(x')}{p(x)})$; only needs p up to a constant	Bayesian posteriors; high-dimensional sampling

```
def importance_sampling_tail(threshold=4.0, n=100_000, seed=0):
    """P(Z > threshold) for Z ~ N(0,1): naive Monte Carlo vs. importance sampling from
N(threshold, 1)."""
    rng = np.random.default_rng(seed)
    naive = np.mean(rng.normal(size=n) > threshold)
    x = rng.normal(threshold, 1.0, n)
    w = np.exp(-0.5 * x ** 2) / np.exp(-0.5 * (x - threshold) ** 2)    # p(x)/q(x); constants
cancel
    return naive, np.mean((x > threshold) * w)

def metropolis_hastings(log_p, x0, n_steps=50_000, step=1.0, seed=0):
    """Random-walk Metropolis: propose x' ~ N(x, step^2), accept with prob min(1, p(x')/p(x))."""
    rng = np.random.default_rng(seed)
    x, lp = x0, log_p(x0)
    out, acc = np.empty(n_steps), 0
    for t in range(n_steps):
        prop = x + step * rng.normal()
        lp_prop = log_p(prop)
        if np.log(rng.random()) < lp_prop - lp:
            x, lp = prop, lp_prop; acc += 1
        out[t] = x
    return out, acc / n_steps
```

Checked: $P(Z > 4) = 3.17 \times 10^{-5}$. With 100,000 draws, naive Monte Carlo sees only **3** hits (estimate 3.0×10^{-5}, but with ±58% relative error); importance sampling from $\mathcal{N}(4,1)$ gives 3.18×10^{-5} (within 0.5%). Metropolis–Hastings on a two-component Gaussian mixture (true mean 1.50, variance 7.13) returns mean 1.52 and variance 7.10 after burn-in, with a 63% acceptance rate. **Practical MCMC checks:** discard burn-in, watch autocorrelation (effective sample size ≪ number of steps), tune the step size, and run several chains from different starts.

Real-life analogy: surveying a city for rare events

Think of it like estimating how many houses in a city have a swimming pool. Knocking on random doors (naive sampling) finds almost none — you learn little. **Importance sampling** is surveying mostly in rich neighbourhoods where pools are common, then correcting the count for having over-visited them. **MCMC** is a surveyor who wanders from house to house, more likely to move towards the kinds of houses you care about, so the time spent in each area ends up proportional to its importance.

Formula, read aloud: $\mathbb{E}_p[f] = \mathbb{E}_q[f \cdot p/q]$: "average over the easy survey, but weight each house by how much more or less likely it is in reality than in your survey".

Where it breaks: if your survey skips areas where the true distribution has mass ($q \approx 0$ where $p > 0$), the weights explode and the estimate is unreliable; MCMC wanderers can also get stuck in one neighbourhood for a long time.

3.20 More probability puzzles

Four more classics (try before reading)

1. Coupon collector. A cereal box contains one of n equally likely toys. Expected boxes to collect all n? When you have k toys, a new one appears with probability $\frac{n-k}{n}$, so that stage takes $\frac{n}{n-k}$ boxes on average. Total $= n\left(1 + \frac{1}{2} + \cdots + \frac{1}{n}\right) \approx n \ln n$. For $n = 10$: **29.3 boxes**. ML echo: how many random samples until every class, user or hash bucket has appeared at least once.

2. Monty Hall. Three doors, one prize. You pick door 1; the host, who knows where the prize is, opens a goat door. Switch? **Yes** — switching wins with probability 2/3. Your first pick was right 1/3 of the time; the host's forced choice concentrates the remaining 2/3 onto the other closed door. ML echo: information from a non-random process (who chose what to show you) changes the conditional probabilities — exactly selection bias.

3. Expected fixed points. Shuffle n name tags and hand them out at random. Expected number of people who get their own tag? By linearity of expectation, each person has probability $1/n$, so the answer is **1**, for every n — no need to handle dependence.

4. Expected tosses to see "HT" vs. "HH". HT takes **4** tosses on average, HH takes **6**. After an H, a failure for HT (another H) keeps you one step away; a failure for HH (a T) sends you back to the start. ML echo: waiting times depend on overlap structure — the same logic as the $E = 6$ derivation earlier.

Real-life analogy: puzzle patterns are cricket shots

Think of it like a batter's repertoire: the cover drive, the pull, the sweep. Most puzzles are one of a few shots — linearity of expectation (fixed points), stages of waiting (coupon collector), conditioning on how information was revealed (Monty Hall), and first-step analysis (HH vs. HT). Identify the shot, then play it cleanly and explain it out loud.

Where it breaks: interviewers twist puzzles slightly to see if you understand rather than recall — re-derive, don't reuse a memorized number.

Active recall — Set 3: Probability

Set 3 · Answer from memory, then check the Answer Key

Easy recall

1. Define PMF, PDF and CDF. Can a PDF exceed 1?
2. State Bayes' theorem with each term named.
3. State the CLT and the variance of a sample mean.
4. Define entropy, cross-entropy and KL divergence.
5. What is the MLE of a Bernoulli parameter?

Conceptual

6. Why does MSE correspond to a Gaussian noise assumption?
7. What is conjugacy? Give one conjugate pair.
8. What's the difference between MLE and MAP, and when do they coincide?
9. Forward vs. reverse KL: which is mode-seeking, and why?
10. What does the law of total variance say, and how does it relate to R^2?

Why

11. Why do we optimize log-likelihood rather than likelihood?
12. Why is the MLE of Gaussian variance biased?
13. Why can averaging many correlated models not reduce variance to zero?
14. Why is cross-entropy ≥ entropy?
15. Why does a Laplace prior produce sparse MAP estimates?

Scenario

16. A fraud model has 95% recall and 99% specificity; fraud prevalence is 0.1%. What fraction of flagged transactions are fraud?
17. A new ad has 2 clicks in 3 impressions. What CTR estimate do you use for ranking and why?
18. Your A/B test metric is revenue per user and results swing wildly week to week. What probabilistic issue is likely?
19. You need a loss for predicting daily order counts per store (small integers, many zeros). What distribution and loss?
20. Your team reports "features A and B have zero correlation, so they're independent". Respond.

Mathematical

21. Derive the MLE of the Poisson rate λ.
22. Derive the ridge penalty from a Gaussian prior; what is λ in terms of σ² and τ²?
23. Compute $\mathrm{Var}(\bar{X})$ for n variables with variance σ^2 and pairwise correlation ρ.

Difficult follow-ups

24. MLE is asymptotically efficient. Why might you still prefer a biased estimator?
25. Why does maximum likelihood for a Gaussian mixture model have degenerate solutions, and how are they avoided?

2027 additions

26. Heights of fathers and sons are $\mathcal{N}(175, 7^2)$ with $\rho = 0.5$. Predict a son's height for a 189 cm father, with its SD.
27. Derive the Fisher information of a Bernoulli(p) and the Cramér–Rao bound for $\hat{p}$.
28. Why does importance sampling beat naive Monte Carlo for $P(Z > 4)$, and when does it fail?

PART 4

Statistics

What statistics adds to ML

Probability says "given the model, how will data look?". Statistics says "given data, what can I conclude — and how sure am I?". Every offline metric, every A/B test, every "the new model is better" claim is a statistical claim with an error bar. Staff-level candidates always ask for the error bar.

The whole Part as one picture

Think of it like a cook tasting dal from a huge pot with one spoon. The pot is the *population*, the spoonful is the *sample*, and "is it salty enough?" is the conclusion. Statistics asks three honest questions. Did I stir first, so the spoon is typical (sampling and bias)? How much would a second spoonful disagree with the first (standard error, confidence interval)? And when I compare two pots, is the difference real or just spoon-to-spoon wobble (hypothesis tests, A/B tests)? A careful cook says "slightly salty, and I'm fairly sure", not just "salty".

Where it breaks: the dal pot sits still; real populations drift over time, so even a perfectly stirred spoonful describes last quarter's pot, not next quarter's.

4.1 Population vs. sample

The **population** is everything you want to draw conclusions about (all future transactions); a **sample** is what you observed. A **parameter** (μ, p) describes the population; a **statistic** ($\bar{x}$, $\hat{p}$) is computed from the sample and estimates it. Every statistic has a **sampling distribution** — the distribution of values you'd get across repeated samples — and its standard deviation is the **standard error** (SE). For a mean: $\mathrm{SE} = \sigma/\sqrt{n}$; for a proportion: $\sqrt{p(1-p)/n}$.

Real-life analogy: an election exit poll

Think of it like an election exit poll. The *population* is every voter in the state; the *parameter* is the true vote share, which nobody knows until counting day. The pollster asks 2,000 voters (the *sample*) and gets a *statistic*, say 42%. Ask a different 2,000 and you might get 41% or 43%: that wobble across possible polls is the *sampling distribution*, and its typical size is the *standard error*.

Formula, read aloud: $\mathrm{SE} = \sigma/\sqrt{n}$ says: typical wobble of the sample average = how much individuals differ ÷ the square root of how many you asked. Four times the voters only halves the wobble.

Where it breaks: the formula assumes the 2,000 were picked at random; a poll taken only outside city malls stays wrong however many people you ask.

The sample is rarely the population you care about

Your training data is "users who were shown something by the previous model, last quarter, in markets you already operate in". The deployment population is different in time, in exposure and maybe in geography. Statistical guarantees only transfer to the population the sample represents.

4.2 Sampling

Method	How	Why / when
Simple random	every unit equally likely	the default assumption behind most formulas
Stratified	sample within strata (country, class), combine with weights	lower variance; guarantees rare strata appear — stratified splits for imbalanced labels
Cluster	randomly pick groups (stores, users), take all their units	cheaper; needs cluster-aware SEs; the right unit when units within a group interact
Systematic	every k-th unit	simple; dangerous with periodic data (every 7th day = always Monday)
Importance / weighted	oversample interesting units, reweight by $1/\pi_i$	rare-event labeling (fraud review queues), negative downsampling in CTR models
Reservoir	uniform sample of size k from a stream of unknown length	logging and monitoring pipelines

Negative downsampling correction (a classic at ad companies). If you keep all positives and a fraction w of negatives, the model's predicted odds are inflated by $1/w$. Correct with $p = \frac{q}{q+(1-q)/w}$, where q is the model's output on the downsampled distribution.



Real-life analogy: checking a truck of mangoes

Think of it like a buyer checking a truck of mangoes before paying. *Simple random*: pick any mangoes blindly. *Stratified*: take some from every crate type, so the few Kesar crates are not missed among the Alphonso. *Cluster*: open three whole crates; cheaper, but mangoes in one crate share one fate. *Systematic*: every 10th mango, which fails if the loader put a bad one every 10th. *Downsampling*: keep every rotten mango but only one good one in ten, and your sample looks far more rotten than the truck.

Formula, read aloud: $p = \frac{q}{q+(1-q)/w}$ says: true chance = model's score ÷ (score + the negatives' share scaled back up by $1/w$).

Where it breaks: mangoes wait patiently; real data streams keep changing, and the strata must be known before you sample.

4.3 Mean, median, variance, standard deviation

- **Mean** minimizes squared deviations ($\arg\min_c \sum(x_i - c)^2 = \bar{x}$); sensitive to outliers — its **breakdown point** is 0 (one extreme value can move it arbitrarily).
- **Median** minimizes absolute deviations ($\arg\min_c \sum|x_i - c|$); breakdown point 50%. This is exactly why MAE regression predicts the conditional median and MSE the conditional mean.
- **Variance** $s^2 = \frac{1}{n-1}\sum(x_i - \bar{x})^2$ (unbiased); **standard deviation** s is in the data's units. **Robust spread:** MAD (median absolute deviation), IQR.

Real-life analogy: a billionaire walks into a café

Think of it like ten people in a café, each earning about $3,000 a month. A billionaire walks in. The *mean* income jumps to many millions, though nobody else got richer; the *median* (the middle person) barely moves. That is the breakdown point: one extreme value can drag the mean anywhere, while the median needs half the room to change. *Variance* measures how far people typically sit from the average, squared; the *standard deviation* brings it back to dollars.

Formula, read aloud: $s^2 = \frac{1}{n-1}\sum(x_i - \bar{x})^2$ says: spread = average squared distance from the mean, dividing by $n-1$ because the mean was fitted to these same people.

Where it breaks: the billionaire is an obvious oddity; in revenue or session-length data the huge values are real customers you cannot simply ignore.

Example

Session lengths (minutes): 2, 3, 3, 4, 5, 240 (one tab left open). Mean 42.8, median 3.5. Reporting the mean says "users stay 43 minutes"; the median says "most sessions are ~3 minutes". For heavy-tailed business metrics, report medians or trimmed means, and model $\log(1+x)$.

4.4 Covariance and correlation (statistical view)

Sample covariance $s_{xy} = \frac{1}{n-1}\sum(x_i - \bar{x})(y_i - \bar{y})$; Pearson $r = s_{xy}/(s_x s_y)$. **Spearman** correlation = Pearson on ranks: captures any monotonic relationship and is robust to outliers. r^2 in simple linear regression equals the model's R^2.

Real-life analogy: two dancers on a stage

Think of it like watching two dancers. *Covariance* asks: when one steps forward, does the other tend to step forward too (positive) or back (negative)? Its size depends on how big their steps are, so its units are awkward. *Correlation* divides out each dancer's own step size, giving a clean score from −1 to +1. *Spearman* ignores step sizes entirely and only asks who is ahead of whom, so one wild leap cannot fool it.

Formula, read aloud: $r = s_{xy}/(s_x s_y)$ says: correlation = how much the two move together ÷ (how much each one moves on its own).

Where it breaks: two dancers circling each other in perfect sync have a strong relationship yet $r \approx 0$, because correlation only measures straight-line togetherness.

Correlation traps

Correlation is sensitive to outliers (one point can create $r = 0.9$), captures only linear association, is affected by restricted range (correlation between SAT and college GPA looks weak *among admitted students*), and can be driven entirely by a confounder.

4.5 Confidence intervals

Definition. A 95% confidence interval procedure produces intervals that contain the true parameter in 95% of repeated samples. For a mean with large n: $\bar{x} \pm 1.96 \frac{s}{\sqrt{n}}$; for small n use t_{n-1} quantiles; for proportions near 0 or 1 or small counts, use Wilson intervals rather than the normal approximation.



Real-life analogy: ring toss at a fair

Think of it like a ring-toss stall at a fair. The true value is a peg fixed in the ground; each experiment throws one ring. "95% confidence" describes your *throwing method*: over many throws, 95% of rings would land around the peg. Once a particular ring has landed, it either circles the peg or it doesn't; there is no 95% left about that one ring.

Formula, read aloud: $\bar{x} \pm 1.96 \frac{s}{\sqrt{n}}$ says: interval = your best guess ± about two standard errors; a bigger sample means a steadier hand, so a smaller ring still catches the peg as often.

Where it breaks: at the fair you can walk over and check the peg; in real life you never learn whether *your* interval caught the truth.

The most common misinterpretation

"There's a 95% probability the true value is in *this* interval" is the **Bayesian credible interval** interpretation. The frequentist statement is about the **procedure**: 95% of such intervals cover the truth. In interviews, say: "if we repeated the experiment many times, 95% of intervals constructed this way would contain the true effect."

Why CIs beat p-values in practice: they show the effect size *and* the uncertainty. "+0.2 pp, 95% CI [+0.12, +0.28]" is decision-ready; "p = 0.001" is not.

4.6 Hypothesis testing

The logic. Assume a default world (the **null hypothesis** H_0: no effect). Compute a **test statistic** measuring how far the data is from what H_0 predicts. If data this extreme would be rare under H_0, reject it in favor of the **alternative** H_1.

p-value: $P(\text{statistic at least as extreme as observed} \mid H_0 \text{ true})$.

What a p-value is NOT

It is **not** $P(H_0 \text{ is true} \mid \text{data})$; not the probability the result is a fluke; not a measure of effect size or importance. A tiny effect with huge n gives a tiny p-value; a big effect with small n can give a large one. $p = 0.049$ and $p = 0.051$ are essentially the same evidence.

	H_0 *true*	H_0 *false*
Reject H_0	**Type I error** (false positive), rate α	correct — **power** $= 1 - \beta$
Fail to reject	correct	**Type II error** (false negative), rate β

Statistical power is the probability of detecting a real effect of a given size. It increases with effect size, sample size and α, and decreases with variance. **Minimum detectable effect (MDE)**: the smallest effect you have (say) 80% power to detect.

$$n_{\text{per arm}} \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \, (\sigma_1^2 + \sigma_2^2)}{\delta^2} \qquad (\text{proportions: } \sigma_i^2 = p_i(1-p_i))$$

Note the $1/\delta^2$: **halving the detectable effect needs 4× the sample**.



Real-life analogy: a courtroom trial

Think of it like a criminal trial. The null hypothesis is "the accused is innocent", assumed until the evidence is overwhelming. The p-value asks: if they really were innocent, how surprising would this much evidence be? *Type I error* is jailing an innocent person; *Type II error* is freeing a guilty one; *power* is the chance a guilty person is actually convicted. A tiny crime (a small effect) needs far more evidence to prove.

Formula, read aloud: $n \approx \frac{(z_{1-\alpha/2}+z_{1-\beta})^2(\sigma_1^2+\sigma_2^2)}{\delta^2}$ says: users per arm = (how strict the court is + how sure you want to be of catching the effect)² × (noise) ÷ (effect size)².

Where it breaks: "not guilty" is not "proven innocent"; failing to reject H_0 only means the evidence was too weak, and the p-value is not the probability of innocence.

4.7 The standard tests

Test	Question	Statistic	Assumptions / notes
z-test (means / proportions)	is the mean (or difference) different from a value?	$z = \frac{\hat{\theta}-\theta_0}{\text{SE}}$	known variance or large n (CLT); two-proportion z-test is the workhorse of CTR/conversion A/B tests
One-sample t-test	is the mean different from μ_0?	$t = \frac{\bar{x}-\mu_0}{s/\sqrt{n}}$, df $n-1$	approx. normal data or moderate n; unknown variance
Two-sample t-test (Welch)	do two groups have different means?	$t = \frac{\bar{x}_1-\bar{x}_2}{\sqrt{s_1^2/n_1+s_2^2/n_2}}$	Welch doesn't assume equal variances — use it by default
Paired t-test	same units measured twice (model A vs B on the same test examples, same folds)	t-test on the differences	removes between-unit variance → much more power
Chi-square test of independence	are two categorical variables related?	$\chi^2 = \sum \frac{(O-E)^2}{E}$	expected counts ≳ 5 per cell; else Fisher's exact test
Chi-square goodness-of-fit	do counts match expected proportions?	same	used for **sample ratio mismatch** checks in A/B tests
ANOVA (one-way)	do $k \geq 3$ groups share a mean?	$F = \frac{\text{between-group variance}}{\text{within-group variance}}$	normality, equal variances; a significant F says "some differ", not which — follow with post-hoc tests
Mann–Whitney U	do two distributions differ in location (rank-based)?	ranks	non-parametric; robust to outliers/ heavy tails
McNemar	do two classifiers differ on the same test set?	discordant pairs	the right test for comparing two models' errors on paired examples

Real-life analogy: choosing the right medical test

Think of it like a doctor choosing a test: a blood test, an X-ray and an ECG each answer a different question and need the patient prepared differently. A *t-test* compares average readings of two groups; a *paired* test measures the same patients before and after, so each person's natural level cancels out; a *chi-square* test compares counts in boxes; *McNemar* has two doctors diagnose the same patients and looks only at the cases where they disagree.

Formula, read aloud: $z = \frac{\hat{\theta}-\theta_0}{\mathrm{SE}}$ says: test score = (what you saw – what "no effect" predicts) ÷ (the typical random wobble), a signal-to-noise ratio.

Where it breaks: a doctor can safely order every test; running many tests on the same data and reporting the best one inflates false positives.

4.8 Bootstrapping

Idea. You can't resample the population, so resample **your sample** with replacement, recompute the statistic many times, and use the spread of those values as its sampling distribution.

Real-life analogy: redrawing receipts from a bowl

Think of it like having one box of 500 shop receipts and no way to collect more. Put them all in a bowl, draw one, note the amount, *put it back*, and repeat 500 times; compute the average bill. Do this 10,000 times. The spread of those 10,000 averages shows how much your one real average could have wobbled. Some receipts get drawn twice, some never.

Formula, read aloud: $1-(1-1/n)^n \approx 63.2\%$ says: the chance a given receipt appears at least once = 1 – (the chance it is missed on every one of the n draws).

Where it breaks: the bowl only holds what you already saw, so it cannot invent a record-breaking bill it never contained; and receipts from the same customer must be drawn together as a bundle.

```python
def bootstrap_ci(x, stat=np.mean, B=10_000, alpha=0.05, rng=np.random.default_rng(0)):
    n = len(x)
    boots = np.array([stat(x[rng.integers(0, n, n)]) for _ in range(B)])
    return np.quantile(boots, [alpha / 2, 1 - alpha / 2])   # percentile interval
```

Why it's beloved in ML: CIs for anything — AUC, F1, NDCG, median latency, a difference of two models' AUCs (resample *test examples*, compute both models on the same resample: a paired bootstrap). Each bootstrap sample contains about $1-(1-1/n)^n \approx 1-e^{-1} \approx 63.2\%$ unique points — the same fact behind random forests' out-of-bag estimates.

When it fails: dependent data (use block or cluster bootstrap — resample users, not rows); statistics at the extremes (max, rare quantiles); tiny n; statistics that are non-smooth functions of the data.

4.9 A/B testing

The A/B testing checklist a strong candidate recites

1. **Hypothesis & primary metric** chosen *before* the test (plus guardrails: latency, revenue, errors, unsubscribes).

2. **Randomization unit** — user, session, request, cluster — must match the unit of analysis, and units must not interfere (SUTVA).
3. **Sample size / duration** from a power analysis at a chosen MDE; run whole weeks to cover weekly seasonality.
4. **Sanity checks:** sample ratio mismatch (chi-square on assignment counts), A/A tests, invariant metrics.
5. **Analysis:** effect size with CI; variance reduction (CUPED); correct for multiple metrics/segments.
6. **Threats:** peeking, novelty/primacy effects, network effects, carry-over, heterogeneous effects, long-term vs short-term.
7. **Decision:** practical significance and costs, not just $p < 0.05$.

Peeking. Checking the p-value daily and stopping at the first $p < 0.05$ inflates the false-positive rate far above 5% (to roughly 20–30% with frequent looks), because a random walk will eventually cross any fixed boundary. Fix: fixed horizon, or **sequential testing** with alpha-spending (O'Brien–Fleming) or always-valid p-values (mSPRT).

CUPED (variance reduction). Use a pre-experiment covariate X (e.g. last month's metric): $Y_{\text{adj}} = Y - \theta(X - \bar{X})$ with $\theta = \text{Cov}(X, Y)/\text{Var}(X)$. Unbiased (randomization makes X balanced), and variance falls by the factor $1 - \rho^2$ — with $\rho = 0.7$, you need about half the sample.

Interference. In marketplaces (riders/drivers), social networks and ad auctions, treating one user affects control users (shared supply, budgets, feeds). Naive tests are biased — use cluster (geo) randomization, switchback experiments (alternate treatment over time windows), or budget-split designs.

Real-life analogy: a blind pizza taste-off

Think of it like a restaurant testing two pizza recipes. A coin flip decides which recipe each customer gets, so hungry and fussy customers are spread evenly (randomization). You fix in advance how many customers and which score counts. *Peeking* is announcing the winner the first evening B happens to lead: over enough evenings, luck alone will hand B a lead. *Interference* is diners sharing slices across tables, so "A" diners also taste B.

Formula, read aloud: $Y_{\text{adj}} = Y - \theta(X - \bar{X})$ (CUPED) says: adjusted score = today's score – the part you could have predicted from how that customer rated food *before* the test.

Where it breaks: a taste-off lasts a week; real product effects can fade (novelty) or grow over months, so a short test can mislead.

Scenario: "An experiment improves CTR from 4.1% to 4.3%. Should we ship?"

What a strong candidate does before answering: asks questions, then computes.

1. Clarify. How many users per arm, and for how long? Was the metric and duration pre-registered? Randomization unit? Is CTR the primary metric or one of many? What are the guardrails (revenue, retention, latency, complaints)? What does shipping cost?

2. Is it statistically real? Two-proportion z-test. The difference is 0.2 pp (relative +4.9%).

- With **20,000 users per arm**: $\text{SE} = \sqrt{\frac{0.041 \cdot 0.959}{20000} + \frac{0.043 \cdot 0.957}{20000}} \approx 0.0020$, so $z \approx 1.0$, $p \approx 0.32$, 95% CI ≈ [−0.19, +0.59] pp. **Not distinguishable from noise.**
- With **500,000 per arm**: $\text{SE} \approx 0.0004$, $z \approx 5.0$, 95% CI ≈ [+0.12, +0.28] pp. **Real.**
- Required sample for 80% power at this effect: $\frac{(1.96+0.84)^2(0.0393+0.0412)}{0.002^2} \approx 158{,}000$ per arm. If the test was smaller, it was underpowered — and a significant result from an underpowered test likely **overestimates** the effect (winner's curse).

3. Is it trustworthy? Sample ratio mismatch check; A/A history; was it peeked and stopped early; did the effect hold across days (novelty decays?) and major segments (platforms, new vs. returning); was CTR's definition identical in both arms (a logging change can fake a CTR lift)?

4. Is it the right metric? CTR can rise because of clickbait or accidental clicks (a button moved under the thumb). Check downstream: conversions, dwell time after click, bounce rate, revenue per user, retention. If CTR ↑ but post-click engagement ↓, you've shipped a worse product.

5. Is it worth it? A +4.9% relative CTR lift at large scale can be very valuable — weigh against engineering/latency/maintenance cost and risk. **Decision:** "If the test was adequately powered, pre-registered, passed sanity checks, and guardrails and downstream metrics are neutral or positive, ship — possibly with a holdback group to measure the long-term effect. Otherwise, extend the test rather than ship on a noisy +0.2 pp."

4.10 Multiple hypothesis testing

Test m true nulls at $\alpha = 0.05$ and you expect $0.05m$ false positives; $P(\geq 1) = 1 - 0.95^m$ (64% for $m = 20$). This is the "look at 20 segments, one is significant" trap.



Real-life analogy: twenty coin flippers

Think of it like asking 20 friends to each flip a coin five times. There is roughly a coin-toss chance that one of them gets five heads by pure luck; calling that friend "gifted" is the multiple testing trap. *Bonferroni* raises the bar for everyone, so the whole group rarely produces even one false "gifted". *Benjamini–Hochberg* accepts a few frauds on your "gifted" list, but keeps their share of the list small.

Formula, read aloud: $P(\geq 1) = 1 - 0.95^m$ says: chance of at least one fluke = 1 – the chance that every one of the m tests stays quiet.

Where it breaks: coin flips are independent; metrics and segments from one experiment are correlated, so the real risk is somewhat smaller and Bonferroni is extra cautious.

Method	Controls	Rule
Bonferroni	family-wise error rate (FWER: any false positive)	reject if $p_i \leq \alpha/m$ — simple, conservative
Holm	FWER, uniformly more powerful than Bonferroni	sort $p_{(1)} \leq \ldots$; reject while $p_{(k)} \leq \alpha/(m-k+1)$
Benjamini–Hochberg	false discovery rate (expected fraction of rejections that are false)	largest k with $p_{(k)} \leq \frac{k}{m}q$; reject $1..k$

Use FWER when any false positive is costly (a launch decision on one primary metric set); **FDR** when screening many candidates (which of 500 features/segments deserve follow-up). Best practice: pre-register one primary metric, treat segment analyses as exploratory.

4.11 Selection bias, survivorship bias, confounding

Selection bias: the sample is chosen by a process related to the outcome. *ML version:* training a credit model only on **approved** applicants (you never observe how rejected ones would have repaid) — the "reject inference" problem; recommendation data only contains items the old model chose to show (exposure bias); labels exist only for transactions that were manually reviewed.

Survivorship bias: analyzing only units that "survived" a filter. Wald's WWII bombers: planes returning with holes in the wings meant wing hits were *survivable*; armor belonged where returning planes had **no** holes. *ML version:* churn models trained on customers still present at the snapshot; "successful startups all did X" analyses; model monitoring computed only on requests that didn't time out.

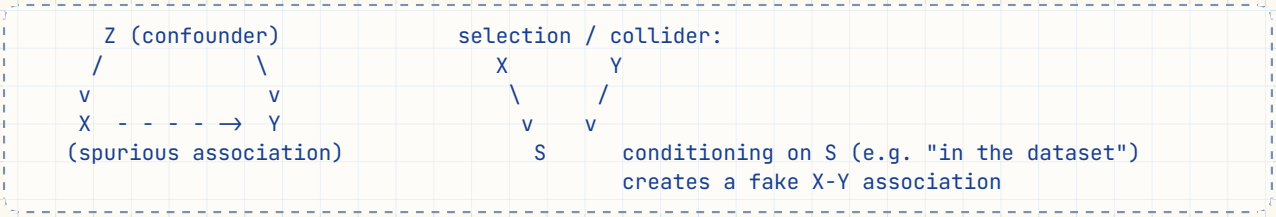
Confounding: a variable Z causes both X and Y, creating an association between them without a causal link. Ice-cream sales and drownings (summer). *ML version:* "users who get notifications retain better" — engaged users get more notifications *and* retain more.



Real-life analogy: a coaching institute's billboard

Think of it like judging a coaching institute in Kota by its billboard. *Selection bias*: it admits only students who clear its own entrance test, so its results were partly decided before teaching began. *Survivorship bias*: the billboard shows toppers, and the students who dropped out have vanished from the story. *Confounding*: families who can afford the institute can also afford books, quiet rooms and extra tutors, so family income drives both joining and scoring well.

Where it breaks: on a billboard you can guess which bias is at work; in real data they overlap and hide, and the fixes differ: randomization defeats confounding, but no amount of data recovers a group that was never recorded.



4.12 Simpson's paradox

A trend present in every subgroup reverses when groups are combined, because group sizes differ between the compared arms.



Real-life analogy: two batters, two kinds of pitch

Think of it like comparing two cricket batters. Batter A has the better average on flat home pitches *and* on bouncy away pitches. Yet batter B has the better overall average, because B played most innings on easy home pitches while A was sent on the tough tours. Pitch type is the confounder: it affects both who batted where and how many runs were possible. Comparing within each pitch type is the fair comparison.

Where it breaks: here we know pitches affect runs; in general a reversal alone does not tell you which view is right, and you need the causal story to decide whether to split or pool.

The kidney-stone numbers (Charig et al., 1986)

	Small stones	Large stones	All
Treatment A	81/87 = **93%**	192/263 = **73%**	273/350 = 78%
Treatment B	234/270 = 87%	55/80 = 69%	289/350 = **83%**

A wins in both groups, B wins overall — because doctors gave A to the harder (large-stone) cases. Stone size is a confounder affecting both treatment choice and outcome. **The right comparison here is within strata** (A is better).

ML versions. (1) Model B beats model A on every country but loses overall because B was evaluated on a harder country mix. (2) An A/B test's traffic split changed mid-test (ramp from 5% to 50%) while conversion rates also varied by day — pooled rates mislead; analyze per period or use a constant split. (3) A feature's coefficient flips sign when a correlated variable is added to a regression.

4.13 Causation vs. correlation

A predictive model learns $P(Y \mid X)$ — association. A decision ("if we *send* this coupon, will the user buy?") needs $P(Y \mid do(X))$ — the effect of **intervening**. They differ whenever there's confounding.



Real-life analogy: umbrellas and the monsoon

Think of it like a street in the monsoon. When you see many open umbrellas, rain is very likely: that is association. But handing out free umbrellas in May will not bring rain: that is an intervention, and it has no effect. A predictive model learns the first; a business decision needs the second.

Formula, read aloud: $P(Y \mid X)$ versus $P(Y \mid do(X))$ says: "how often Y happens among people who *happen* to have X" versus "how often Y would happen if we *forced* X on everyone".

Where it breaks: with umbrellas the direction of cause is obvious; in product data (notifications and retention) the arrows are hidden, which is why we need experiments or careful adjustment.

Tool	Idea	When
Randomized experiment (A/B)	randomization breaks every arrow into X	gold standard whenever feasible
Adjust for confounders (regression, matching, propensity weighting)	compare like with like	observational data; needs all confounders measured ("no unmeasured confounding")
Difference-in-differences	compare changes over time between treated and untreated groups	policy/launch rollouts by region; needs parallel trends
Instrumental variables	a variable affecting Y only through X	e.g. random assignment used as an instrument for actual usage
Regression discontinuity	compare just above/below a threshold	credit-score cutoffs, eligibility rules
Uplift modeling	predict $\mathbb{E}[Y \mid do(T=1), x] - \mathbb{E}[Y \mid do(T=0), x]$	targeting coupons/retention offers at *persuadable* users, not at users who'd convert anyway

Interview question: "Users who use feature X retain 30% better. Should we push everyone to use X?"

Strong answer. "Not from that correlation alone. Users who adopt X are probably already more engaged — engagement is a confounder that drives both X usage and retention, so the 30% overstates, maybe

entirely fakes, X's effect. I'd run an experiment: randomize an intervention that increases X adoption — a prompt or onboarding step — and measure retention in both arms. That estimates the effect of *encouraging* X, which is what we'd actually do. If an experiment is impossible, I'd use observational methods like propensity matching on pre-adoption engagement, or diff-in-diff around X's rollout, and be explicit about the assumptions."

4.14 Ratio metrics and the delta method

Many product metrics are **ratios of sums**: click-through rate = total clicks / total impressions, revenue per session, average order value. When the experiment randomizes **users** but the metric is computed over **impressions**, impressions from the same user are correlated, and the textbook binomial standard error $\sqrt{r(1-r)/N_{\text{impressions}}}$ is **too small** — you'll declare false wins.

The **delta method** gives the right variance by linearizing the ratio of two user-level means $\bar{X}$ (clicks per user) and $\bar{Y}$ (impressions per user):

$$\operatorname{Var}\left(\frac{\bar{X}}{\bar{Y}}\right) \approx \frac{1}{n\,\mu_Y^2}\left(\sigma_X^2 - 2r\,\sigma_{XY} + r^2\sigma_Y^2\right), \qquad r = \frac{\mu_X}{\mu_Y}$$

where n is the number of **users** (the randomization unit).

```python
def delta_method_ratio_ci(num, den, z=1.96):
    """CTR-style ratio sum(num)/sum(den) where each unit (user) contributes num_i, den_i."""
    n = len(num)
    mx, my = num.mean(), den.mean()
    vx, vy = num.var(ddof=1), den.var(ddof=1)
    cxy = np.cov(num, den, ddof=1)[0, 1]
    r = mx / my
    var_r = (vx - 2 * r * cxy + r ** 2 * vy) / (n * my ** 2)      # first-order Taylor expansion
    se = np.sqrt(var_r)
    return r, (r - z * se, r + z * se), se
```

Checked on 5,000 simulated users with heterogeneous personal CTRs: delta-method SE **0.00081**, user-level bootstrap SE **0.00082** (they agree), naive binomial SE **0.00067** — the naive interval is about 17% too narrow, so a "significant" result at $p = 0.04$ might really be $p \approx 0.09$.

Real-life analogy: judging a school by its students, not its answer sheets

Think of it like comparing two coaching institutes by the fraction of correct answers across all answer sheets. If one topper writes 500 sheets, his sheets are not 500 independent pieces of evidence — they're one student. The fair uncertainty comes from the number of *students* (the unit you randomized), and the delta method converts the ratio into student-level terms.

Formula, read aloud: the variance of a ratio ≈ [spread of the top + (ratio)² × spread of the bottom − 2 × ratio × how they move together] ÷ (number of users × average bottom²).

Where it breaks: it is a first-order approximation; with very few users or heavy-tailed denominators (a few users with huge counts), use a user-level bootstrap instead.

4.15 Bayesian A/B testing

Instead of a p-value, compute what decision-makers actually ask: *"What is the probability that B is better, and how much do we lose if we pick B and we're wrong?"* With conversions, a Beta prior is conjugate: after s successes in n trials, $p \sim \text{Beta}(1+s,\, 1+n-s)$.

```python
def bayes_ab(conv_a, n_a, conv_b, n_b, draws=200_000, seed=0, prior=(1, 1)):
    rng = np.random.default_rng(seed)
    pa = rng.beta(prior[0] + conv_a, prior[1] + n_a - conv_a, draws)
    pb = rng.beta(prior[0] + conv_b, prior[1] + n_b - conv_b, draws)
    lift = pb / pa - 1
    return (pb > pa).mean(), np.quantile(lift, [0.025, 0.5, 0.975]), np.mean(np.maximum(pa - pb,
0))
```

Checked: A = 410/10,000, B = 450/10,000 → $P(B > A) = \mathbf{0.917}$, median relative lift ≈ +9.7% with a 95% credible interval of roughly −4% to +25%, and expected loss if we ship B ≈ 0.0001 (0.01 percentage points of conversion). A frequentist two-sided test on the same data gives $p \approx 0.16$ — "not significant" — while the Bayesian summary says "probably better, small downside". Both are honest; they answer different questions.

Cautions: the prior matters with little data (state it); "peeking" is less damaging to the interpretation of a posterior than to a p-value, but repeatedly stopping when $P(B > A)$ first crosses 95% still inflates wrong decisions; and the organization must agree beforehand on a decision rule (e.g., ship if expected loss < a threshold).



Real-life analogy: a weather forecast for your decision

Think of it like the difference between a courtroom and a weather forecast. The p-value is the courtroom: "if B were no better, would this evidence be surprising?" — a yes/no verdict. The Bayesian result is a forecast: "80% chance of rain, and if it rains you'll get a little wet" — the probability B is better, and how much it would cost to be wrong. Managers usually want the forecast.

Formula, read aloud: $P(p_B > p_A \mid \text{data})$ = the share of simulated worlds, consistent with what we saw, in which B's true rate beats A's.

Where it breaks: a forecast depends on the forecaster's model; the Bayesian answer depends on the prior and the likelihood, and a bad prior with little data gives confident nonsense.

4.16 The causal inference toolkit, one level deeper

When you can't randomize, you need assumptions strong enough to recover a causal effect. Each method trades one assumption for another.

Method	Estimate	Key assumption	Classic ML use
Inverse propensity weighting (IPW)	$\hat{\tau} = \frac{1}{n}\sum_i \left[\frac{T_i Y_i}{e(x_i)} - \frac{(1-T_i)Y_i}{1-e(x_i)}\right]$, with propensity $e(x) = P(T = 1 \mid x)$	all confounders observed; **overlap** $(0 < e(x) < 1)$	effect of a feature/ promotion from logs
Doubly robust (AIPW)	outcome model + propensity correction; consistent if **either** model is right	as IPW, plus decent models	the default for observational estimates
Difference-in-differences	$(\bar{Y}^T_{\text{after}} - \bar{Y}^T_{\text{before}}) - (\bar{Y}^C_{\text{after}} - \bar{Y}^C_{\text{before}})$	**parallel trends** without treatment	launches rolled out by city or region
Instrumental variables	Wald: $\hat{\tau} = \frac{\text{Cov}(Z,Y)}{\text{Cov}(Z,T)}$	Z moves T, and affects Y **only** through T	random encouragement (a nudge) as instrument for actual usage
Regression discontinuity	compare units just above vs. just below a cutoff	nothing else jumps at the cutoff	credit-score thresholds, eligibility rules
Synthetic control	build a weighted combination of untreated units that tracks the treated unit before treatment	pre-period fit carries over	one market/country changed its policy

Worked example — difference-in-differences

A new checkout flow launched in Pune in March; Nagpur didn't get it. Conversion: Pune 3.0% → 3.6%; Nagpur 2.8% → 3.1%. Naive before/after in Pune: +0.6 points — but conversions rose everywhere (seasonality). DiD: $(3.6 - 3.0) - (3.1 - 2.8) =$ **+0.3 points**. Credible only if the two cities were trending in parallel before March — plot several pre-launch months to check.

Real-life analogy: judging a new fertilizer without a controlled field

Think of it like a farmer who can't run a proper field trial. **IPW:** compare fields that got the fertilizer with similar-looking fields that didn't, giving extra weight to the rare cases that break the usual pattern. **DiD:** compare the change in your village's yield with the change in a neighbouring village over the same season, so the monsoon affects both. **IV:** a government lottery gave free fertilizer to some farmers; use the lottery to see the effect. **RDD:** farms just above and below a subsidy cutoff in land size are nearly identical except for the subsidy.

Where it breaks: each trick rests on an assumption you can't fully test — hidden differences between fields, villages trending differently, lottery winners changing other habits — so always say which assumption you're making and how you checked it.

Active recall — Set 4: Statistics

Set 4 · Answer from memory, then check the Answer Key

Easy recall

1. Define a p-value precisely.
2. Define Type I error, Type II error and power.
3. What does a 95% confidence interval mean (frequentist)?
4. When do you use Welch's t-test instead of a standard two-sample t-test?
5. What fraction of unique points is in a bootstrap sample, approximately?

Conceptual

6. Why does the required sample size scale with $1/\delta^2$?
7. What is sample ratio mismatch and how do you test for it?
8. What does Benjamini–Hochberg control, and how does it differ from Bonferroni?
9. Why does peeking at an A/B test inflate false positives?
10. What is the difference between selection bias and confounding?

Why

11. Why does the median resist outliers while the mean does not?
12. Why can a significant result from an underpowered test overestimate the effect?
13. Why is a paired test more powerful when comparing two models on the same test set?
14. Why does CUPED reduce variance without introducing bias?
15. Why does Simpson's paradox occur?

Scenario

16. A test shows +0.2 pp CTR with p = 0.03, but post-click conversions dropped. Ship?
17. You tested 30 segments; 2 show significant effects. How do you report this?

18. A marketplace pricing experiment randomizes by rider. What goes wrong and what design do you use instead?
19. Your credit model is trained only on approved loans. What bias results, and what can you do?
20. Revenue per user is heavy-tailed; your test never reaches significance. Options?

Mathematical

21. Compute the SE of the difference of two proportions 0.10 and 0.11 with 10,000 users per arm and state whether the difference is significant at 5%.
22. How many users per arm to detect a change from 10% to 10.5% with $\alpha = 0.05$ (two-sided) and 80% power?
23. With 20 independent tests of true nulls at $\alpha = 0.05$, what's P(at least one false positive)? What's the Bonferroni threshold?

Difficult follow-ups

24. Your offline evaluation reports model B's AUC is 0.003 higher than A's on 50K test examples. How do you decide whether that's real?
25. An experiment shows a large positive effect in week 1 that shrinks by week 4. What are the candidate explanations and how do you distinguish them?

2027 additions

26. Why is the binomial standard error wrong for CTR in a user-randomized test, and what do you use instead?
27. A = 410/10,000, B = 450/10,000. Report a Bayesian summary and explain how it differs from the p-value.
28. Pune conversion 3.0% → 3.6% after a launch; control city Nagpur 2.8% → 3.1%. Estimate the effect and name the key assumption.

Unit III

Classical Machine Learning

Part 5 — Linear Regression · Part 6 — Logistic Regression
Part 7 — Decision Trees · Part 8 — Ensembles
Part 9 — SVM · Part 10 — KNN · Part 11 — Naive Bayes
Part 12 — Unsupervised Learning · Part 13 — Dimensionality Reduction

Interviewers love classical ML because every model is small enough to derive — and every derivation exposes whether you understand it.

PART 5

Linear Regression

Linear regression in one breath

Assume the target is a weighted sum of features plus Gaussian noise. Maximum likelihood then says: minimize squared error. The solution is the orthogonal projection of $\mathbf{y}$ onto the column space of X. Regularization is a prior that trades a little bias for a lot less variance.

The whole Part as one picture

Think of it like working out an auto-rickshaw fare chart from past rides. Each ride has a distance, a waiting time and the fare paid. You guess: fare = base fare + rate per km × km + rate per minute × waiting minutes, plus some random extra (traffic, change not returned). Linear regression finds the rates that make the total squared gap between chart and real fares as small as possible. Diagnostics check whether the leftover gaps look random. Regularization is a sceptical auditor who says: "don't claim a big rate unless many rides prove it."

Where it breaks: a real fare might jump at night or surge in the rain; linear regression only finds the best straight-line chart unless you add such effects as new features.

5.1 Model formulation

$$y_i = \mathbf{w}^\top \mathbf{x}_i + b + \varepsilon_i, \qquad \varepsilon_i \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2) \qquad \Longleftrightarrow \qquad \mathbf{y} = X\mathbf{w} + \boldsymbol{\varepsilon}$$

(Absorb the bias by adding a column of ones to X.) "Linear" means linear **in the parameters**, not in the raw inputs: $y = w_1 x + w_2 x^2 + w_3 \log x$ is still linear regression — feature engineering buys nonlinearity while keeping the convex, closed-form machinery.

Real-life analogy: a weighted marks recipe

Think of it like predicting a student's board-exam marks with a recipe: marks = base marks + w_1 × hours studied + w_2 × hours slept + luck on the day. Each weight says how much one extra hour adds. "Linear" only means the weights are multiplied and added up; the ingredients can be curvy, like hours², as long as you add them as extra columns.

Formula, read aloud: $y_i = \mathbf{w}^\top \mathbf{x}_i + b + \varepsilon_i$ says: each answer = (each feature × its weight, all added up) + a starting amount + random bell-shaped noise.

Where it breaks: a weight describes one feature's effect *holding the others fixed*; a real student who studies more may also sleep less, so the weights are not promises about what changing one habit will do.

Interpretation of a coefficient: w_j is the expected change in y for a one-unit increase in x_j **holding all other features fixed**. That "holding fixed" is why coefficients change (even flip sign) when correlated features are added or removed.

5.2 Loss function: MSE, and why squared

$$L(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \mathbf{w}^\top \mathbf{x}_i\right)^2 = \frac{1}{n}\|\mathbf{y} - X\mathbf{w}\|^2$$

Real-life analogy: a coach's squared fines

Think of it like a coach who fines a javelin thrower for every miss, where the fine is the miss distance *squared*: miss by 1 metre, pay 1; miss by 10 metres, pay 100. The thrower will change their whole technique to avoid the rare huge miss, even at the cost of many small ones. Because the fine grows smoothly like a bowl, there is exactly one best technique, and you can calculate it directly.

Formula, read aloud: $L(\mathbf{w}) = \frac{1}{n}\sum(y_i - \mathbf{w}^\top \mathbf{x}_i)^2$ says: loss = the average, over all points, of (true answer − prediction) squared.

Where it breaks: a thrower aims at one target; here one set of weights must serve every point at once, so a single wild point can tilt the whole line.

Why squared error, not absolute or cubed?

1. **It is the MLE under Gaussian noise** (Part 3): $-\log p(\mathbf{y} \mid X, \mathbf{w}) = \frac{1}{2\sigma^2}\|\mathbf{y} - X\mathbf{w}\|^2 + \text{const}$.
2. **It targets the conditional mean:** the minimizer of $\mathbb{E}[(Y-c)^2 \mid X = x]$ over c is $\mathbb{E}[Y \mid X = x]$.
3. **Convex, smooth, closed form:** a quadratic bowl in $\mathbf{w}$ with gradient linear in $\mathbf{w}$.
4. **Geometry:** it's Euclidean distance, so the solution is a projection with Pythagorean decompositions (R^2).

The cost: squaring amplifies large residuals, so **outliers dominate** the fit (Part 15 compares MSE, MAE, Huber).

5.3 Ordinary least squares and the normal equation

Derivation — normal equation (reproduce this standing up)

$$L(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|^2 = \mathbf{y}^\top\mathbf{y} - 2\mathbf{w}^\top X^\top\mathbf{y} + \mathbf{w}^\top X^\top X\mathbf{w}$$

$$\nabla_{\mathbf{w}} L = -2X^\top\mathbf{y} + 2X^\top X\mathbf{w} = 0 \implies X^\top X\,\hat{\mathbf{w}} = X^\top\mathbf{y} \implies \boxed{\hat{\mathbf{w}} = (X^\top X)^{-1}X^\top\mathbf{y}}$$

Hessian $\nabla^2 L = 2X^\top X \succeq 0$ ⇒ convex; if X has full column rank, $X^\top X \succ 0$ ⇒ strictly convex ⇒ **unique** global minimum.

Geometric reading: $X^\top(\mathbf{y} - X\hat{\mathbf{w}}) = 0$ — the residual is orthogonal to every column of X; $\hat{\mathbf{y}} = X\hat{\mathbf{w}}$ is the projection of $\mathbf{y}$ onto the column space (Part 2).

Real-life analogy: a shadow on the floor

Think of it like a lamp directly above a ball hanging in a room. The floor is every prediction your features can make; the ball is the real answer $\mathbf{y}$, usually floating off the floor. The best you can do is the ball's shadow, straight below it. The line from shadow to ball (the residual) stands perfectly upright, at right angles to the floor, which is exactly what the normal equation demands.

Formula, read aloud: $\hat{\mathbf{w}} = (X^\top X)^{-1}X^\top\mathbf{y}$ says: the best weights = (how the features overlap with each other)$^{-1}$ × (how the features overlap with the answer).

Where it breaks: a room's floor is 2-D; here the "floor" can have thousands of directions, and if two features point the same way the floor collapses and the inverse no longer exists.

When $X^\top X$ is not invertible (perfectly collinear features, or $d > n$): infinitely many minimizers. The pseudo-inverse $X^+\mathbf{y}$ (SVD) returns the **minimum-norm** one; ridge adds λI and makes the solution unique.

Statistical properties (Gauss–Markov). If errors have zero mean, equal variance and are uncorrelated, OLS is **BLUE** — the Best (lowest variance) Linear Unbiased Estimator; normality isn't needed for that. Its covariance is $\mathrm{Var}(\hat{\mathbf{w}}) = \sigma^2(X^\top X)^{-1}$ — note how it blows up when $X^\top X$ is near-singular. "Best among *unbiased*" is the loophole ridge exploits: a biased estimator can have lower MSE.

5.4 Gradient descent solution

$$\nabla_{\mathbf{w}}\left[\tfrac{1}{2n}\|X\mathbf{w}-\mathbf{y}\|^2\right] = \frac{1}{n}X^\top(X\mathbf{w}-\mathbf{y}), \qquad \mathbf{w} \leftarrow \mathbf{w} - \eta\frac{1}{n}X^\top(X\mathbf{w}-\mathbf{y})$$

Intuition: each residual $r_i = \hat{y}_i - y_i$ pushes the weights along its own input direction $\mathbf{x}_i$, scaled by how wrong it is.



Real-life analogy: a tug-of-war on a floating plank

Think of it like a plank floating on a pond with a rope from each data point. Every point pulls the plank toward itself, harder the further away it is (its residual). Each step, you move the plank a little in the direction of the total pull; when the pulls balance, you have the least-squares fit. If ropes are measured in wildly different units (km and mm), the pulls are lopsided and the plank zig-zags; standardizing features evens them out.

Formula, read aloud: $\mathbf{w} \leftarrow \mathbf{w} - \eta\frac{1}{n}X^\top(X\mathbf{w}-\mathbf{y})$ says: new weights = old weights – step size × (the average of each point's error times its features).

Where it breaks: ropes cannot overshoot, but gradient descent can: with too large a step size η it jumps past the answer and can blow up.

When to use GD instead of the closed form.

	Normal equation / QR / SVD	Gradient descent (batch / SGD)
Cost	$O(nd^2 + d^3)$ once	$O(nd)$ per epoch (SGD: $O(d)$ per step)
d large ($\gtrsim 10^4$)	d^3 becomes prohibitive	fine
n huge / streaming	must fit $X^\top X$ in memory (it's only $d \times d$ — often OK)	natural: mini-batches, online updates
Hyperparameters	none	learning rate, iterations; needs feature scaling
Exactness	exact (up to conditioning)	approximate; converges linearly for this convex problem

Convergence and scaling. The Hessian is $H = \frac{1}{n}X^\top X$. GD is stable iff $\eta < 2/\lambda_{\max}(H)$, and the number of iterations scales with the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$. Features on wildly different scales ⇒ huge κ ⇒ a long, narrow valley ⇒ zig-zagging and slow convergence. **Standardizing features** makes the bowl rounder — that's the whole reason scaling matters for gradient-based linear models.

5.5 Assumptions — and what breaks when each fails

Assumption	Meaning	If violated	Detect	Remedy
Linearity	$\mathbb{E}[y \mid \mathbf{x}]$ is linear in features	**biased** predictions, structured errors	residuals vs. fitted show curves	transforms (log, polynomial, splines), interactions, nonlinear models
Exogeneity	$\mathbb{E}[\varepsilon \mid X] = 0$ (no omitted confounders correlated with X)	**biased coefficients** (omitted-variable bias) — predictions may still be fine	domain knowledge; can't be tested from residuals alone	add confounders, IV methods
Independence of errors	no autocorrelation / clustering	SEs too small, overconfident CIs	Durbin–Watson, residual ACF	time-series models, clustered SEs
Homoscedasticity	constant $\mathrm{Var}(\varepsilon \mid \mathbf{x})$	coefficients still unbiased, but inefficient and **SEs wrong**	funnel shape in residual plot, Breusch–Pagan	robust (sandwich) SEs, weighted least squares, log-transform the target
Normality of errors	ε Gaussian	only matters for small-sample CIs/tests; point estimates fine	Q–Q plot	bootstrap CIs; large n saves you (CLT)
No perfect multicollinearity	columns of X linearly independent	$X^\top X$ singular / unstable coefficients	VIF, condition number	drop/combine features, ridge, PCA



Real-life analogy: the fine print on a travel-time app

Think of it like a maps app that estimates travel time with a straight rule from distance. Its promise has fine print. *Linearity*: time really grows steadily with distance. *Exogeneity*: no hidden cause, like a festival procession, that both slows you down and makes you pick that route. *Independence*: one trip's delay says nothing about the next. *Homoscedasticity*: short and long trips are equally uncertain. *No multicollinearity*: you are not feeding it "km" and "metres" as two separate inputs.

Where it breaks: a broken rule doesn't always ruin the estimate: unequal uncertainty leaves the average right but the error bars wrong, while a hidden cause makes the rates themselves wrong; the table says which failure hurts what.

The assumption people get wrong

"Linear regression assumes the features (or the target) are normally distributed." No — it assumes nothing about the distribution of X, and normality only concerns the **errors**, and only for exact small-sample inference.

5.6 Residuals and diagnostics

Residuals $r_i = y_i - \hat{y}_i$ are your window into violated assumptions. Plot **residuals vs. fitted values** (curvature ⇒ nonlinearity; funnel ⇒ heteroscedasticity), a **Q–Q plot** (tails ⇒ non-normal errors / outliers), residuals vs. time (autocorrelation), and **leverage vs. residual** (influential points). **Cook's distance** combines leverage and residual size to measure how much a single point moves the fit.

Real-life analogy: leftovers after a buffet

Think of it like a chef studying the plates that come back from a buffet. If every plate has a small, random amount left, the cooking is fine. If every plate has leftover rice but no dal, there is a pattern: the recipe (the model) is wrong. If big eaters leave much more than small eaters, that is a funnel (unequal spread). One plate piled high is an outlier; Cook's distance asks how much the chef would change the recipe if that one diner had never come.

Where it breaks: leftovers only reveal problems you think to look for; a hidden confounder can leave random-looking residuals while the coefficients are still biased.

5.7 R^2 and adjusted R^2

$$R^2 = 1 - \frac{\mathrm{SS}_{\mathrm{res}}}{\mathrm{SS}_{\mathrm{tot}}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}, \qquad R^2_{\mathrm{adj}} = 1 - (1 - R^2)\frac{n-1}{n-d-1}$$

R^2 is the fraction of target variance explained, relative to the baseline "always predict the mean".

Real-life analogy: beating the lazy forecaster

Think of it like a weather forecaster competing with a lazy rival who always predicts "the usual monthly average". R^2 is the share of the lazy rival's error that our forecaster removes: 1 is perfect, 0 is no better than the lazy guess, and negative on new days means worse than just saying "the usual". Adjusted R^2 docks points for every extra gadget the forecaster uses, since each gadget can fit a little luck.

Formula, read aloud: $R^2 = 1 - \frac{\mathrm{SS}_{\mathrm{res}}}{\mathrm{SS}_{\mathrm{tot}}}$ says: score = 1 − (our total squared miss) ÷ (the lazy "always the average" total squared miss).

Where it breaks: a high score does not prove a good forecaster: peeking at tomorrow's newspaper (leakage) gives 0.99, while in very noisy areas like stock returns a score of 0.02 can be valuable.

- On **training data** with an intercept, $0 \le R^2 \le 1$ and adding any feature can never decrease it — which is why adjusted R^2 penalizes the feature count d.
- On **test data**, R^2 **can be negative**: the model is worse than predicting the training mean.
- High R^2 ≠ good model (a leaked feature gives $R^2 = 0.99$); low R^2 ≠ useless (in noisy domains like finance, $R^2 = 0.02$ can be very valuable). It's also not comparable across different targets or transformations (log vs. raw).

5.8 Multicollinearity

What it is. Features that are (nearly) linear combinations of each other.

Mechanism. The data can't tell the model how to split credit between correlated features: many weight combinations give nearly the same predictions (a long flat valley in the loss). $\mathrm{Var}(\hat{\mathbf{w}}) = \sigma^2(X^\top X)^{-1}$ explodes along that valley. **Variance inflation factor:** $\mathrm{VIF}_j = \frac{1}{1-R_j^2}$ where R_j^2 is from regressing x_j on the other features; VIF > 5–10 flags trouble.

Real-life analogy: flatmates on one UPI account

Think of it like two flatmates who always shop together and pay from one shared UPI account. From the monthly statement you know the total they spent (the prediction is stable), but not who spent what (the individual coefficients are unstable). Many splits fit the statement equally well, so small changes in the data swing the split wildly.

Formula, read aloud: $\mathrm{VIF}_j = \frac{1}{1-R_j^2}$ says: variance multiplier = 1 ÷ (the share of feature j that the other features cannot explain); if 90% is explainable, the variance grows 10×.

Where it breaks: if the flatmates later start shopping separately, the split you guessed starts giving wrong totals; that is when multicollinearity hurts predictions in production.

What it hurts and what it doesn't. Hurts: coefficient stability, signs, p-values, interpretability, feature-importance stories. Mostly doesn't hurt: **predictions** within the training distribution (the sum of the correlated contributions is well-determined). It becomes a prediction problem when the correlation structure changes in production.

Fixes: remove or combine redundant features, ridge (spreads weight evenly across correlated features), PCA regression, domain-driven feature selection.

5.9 Heteroscedasticity

Error variance depends on $\mathbf{x}$ (e.g. spending errors grow with income). OLS remains unbiased, but (1) it's no longer the most efficient estimator — it gives equal weight to noisy and precise points — and (2) the standard $\sigma^2(X^\top X)^{-1}$ formula is wrong, so CIs and p-values are invalid. Fixes: **heteroscedasticity-robust (Huber–White) standard errors**; **weighted least squares** with weights $\propto 1/\sigma_i^2$; model $\log y$ (turns multiplicative noise into additive); or predict the variance too (heteroscedastic likelihood / quantile regression) when uncertainty matters to the product.

Real-life analogy: archers at different distances

Think of it like judging a bow's aim from arrows shot at 10 metres and at 70 metres. The close arrows land tightly; the far ones scatter widely. Averaging all of them still points at the right aim (OLS stays unbiased), but treating every arrow as equally reliable wastes the precise close shots and gives the wrong error bars. A smart coach trusts the close arrows more.

Formula, read aloud: weights $\propto 1/\sigma_i^2$ says: trust each point in inverse proportion to its noise squared, so a point twice as noisy counts a quarter as much.

Where it breaks: the coach knows which shots were far; in data the noise level of each point is unknown and must be estimated, or you fall back on robust standard errors.

5.10 Outliers

Squared loss gives a point with residual 10 a hundred times the pull of a point with residual 1. Separate three things: **outliers in** y (large residual), **high-leverage points** (unusual $\mathbf{x}$, can drag the line toward themselves), and **influential points** (both — high Cook's distance). Remedies: fix data errors first; robust losses (Huber, MAE → median regression); RANSAC (fit on random subsets, keep the consensus); winsorize/clip features; transform heavy-tailed targets. Never delete outliers just because they're inconvenient — in fraud or anomaly problems, they *are* the signal.

Real-life analogy: children on a see-saw

Think of it like a see-saw with children sitting along it. A child at the far end (a *high-leverage* point, with unusual $\mathbf{x}$) tilts the plank much more than one near the middle. A child pushing unusually hard is an *outlier in* y. A child pushing hard *and* sitting at the far end is *influential*: they alone can decide the tilt. With squared loss, pushing 10 times harder counts 100 times more.

Where it breaks: a see-saw only tilts around a fixed pivot, while a regression line can both shift and tilt; and in fraud data the child pushing hardest is exactly the one you want to study, not silence.

5.11 Regularization: Ridge → Lasso → Elastic Net

Ridge: $\min_{\mathbf{w}} \|\mathbf{y} - X\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|_2^2$ Lasso: $\min_{\mathbf{w}} \frac{1}{2n}\|\mathbf{y} - X\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|_1$ Elastic net: $\cdots + \lambda\big(\alpha\|\mathbf{w}\|_1 + \frac{1-\alpha}{2}\|\mathbf{w}\|_2^2\big)$

(Always standardize features first, and **never penalize the intercept** — otherwise predictions depend on where you put the origin of y.)

Real-life analogy: a strict budget committee

Think of it like a committee approving budget requests from many teams (features). Each team asks for a weight backed by evidence (the data fit), and the committee charges a fee for asking big. *Ridge* charges by the square of each request, so big asks get trimmed hard while small ones are nearly free: nobody ends at exactly zero. *Lasso* charges a flat rate per unit, so a team with weak evidence isn't worth funding at all and gets zero. λ is how strict the committee is.

Formula, read aloud: $\min_{\mathbf{w}} \|\mathbf{y} - X\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|_2^2$ says: choose weights that fit the data well, plus λ × (sum of squared weights) as a fine for big weights.

Where it breaks: budgets can't be negative but weights can, and a weight's size depends on its feature's units, which is why you standardize first.

Ridge: closed form and what it does to each direction

$$\nabla_{\mathbf{w}} = -2X^\top(\mathbf{y} - X\mathbf{w}) + 2\lambda\mathbf{w} = 0 \implies \boxed{\hat{\mathbf{w}}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}}$$

With the SVD $X = U\Sigma V^\top$:

$$\hat{\mathbf{w}}_{\text{ridge}} = \sum_j \frac{s_j}{s_j^2 + \lambda}(\mathbf{u}_j^\top \mathbf{y})\,\mathbf{v}_j, \qquad X\hat{\mathbf{w}}_{\text{ridge}} = \sum_j \underbrace{\frac{s_j^2}{s_j^2 + \lambda}}_{\text{shrinkage factor}} (\mathbf{u}_j^\top \mathbf{y})\,\mathbf{u}_j$$

OLS is the $\lambda = 0$ case (factor 1 everywhere). Ridge shrinks each principal direction by $\frac{s_j^2}{s_j^2+\lambda}$: **directions with little data variance (small s_j) are shrunk the most** — exactly the directions where OLS estimates are noisiest (their variance is σ^2/s_j^2). Effective degrees of freedom $= \sum_j \frac{s_j^2}{s_j^2+\lambda}$.

Lasso: the soft-thresholding solution (1-D / coordinate descent)

For one standardized feature (or one coordinate with the others fixed), minimize $\frac{1}{2}(w - \rho)^2 + \lambda|w|$, where ρ is the OLS coefficient (correlation with the partial residual). Subgradient condition: $0 \in w - \rho + \lambda\,\partial|w|$.

- If $w > 0$: $w = \rho - \lambda$, valid when $\rho > \lambda$.

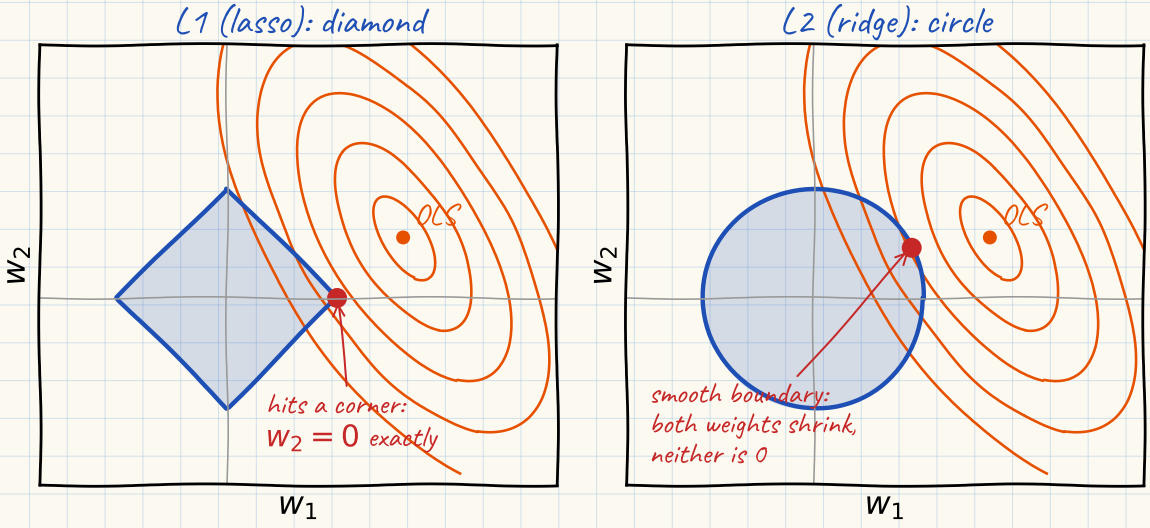
- If $w < 0$: $w = \rho + \lambda$, valid when $\rho < -\lambda$.
- If $w = 0$: need $|\rho| \leq \lambda$ (since $\partial|0| = [-1, 1]$).

$$\boxed{\hat{w} = S_\lambda(\rho) = \text{sign}(\rho)\ \max(|\rho| - \lambda,\ 0)}$$

Compare ridge in the same setting: $\hat{w} = \frac{\rho}{1+\lambda}$ — proportional shrinkage, **never exactly zero**. Lasso subtracts a constant and **clips to zero** any coefficient whose evidence $|\rho|$ is below λ. Lasso has no closed form in general; coordinate descent cycles this update over features.

Why L1 produces sparsity — three explanations

1. **Geometry.** Constrained form: minimize the loss subject to $\|\mathbf{w}\|_1 \leq t$. The L1 ball is a diamond (cross-polytope) whose **corners lie on the axes**. The loss contours (ellipses around the OLS solution) expand until they first touch the constraint set — and for a diamond, the first contact is very likely a corner or an edge where some coordinates are exactly 0. The L2 ball is round: the tangent point is generically off-axis.
2. **Gradient (force) view.** The L2 penalty's pull on w_j is $2\lambda w_j$ — it weakens as $w_j \to 0$, so weights approach zero but never arrive. The L1 penalty's pull is $\lambda\ \text{sign}(w_j)$ — a **constant force** no matter how small w_j is; if the data's pull $|\rho_j|$ is weaker than λ, the weight is pushed all the way to 0 and stays there.
3. **Bayesian view.** L1 = Laplace prior, which has a sharp peak (a kink) at zero and puts a lot of mass near it; L2 = Gaussian prior, smooth and flat at zero. MAP estimates under a kinked prior land exactly on the kink.



Why L1 hits corners (zeros) and L2 doesn't

Real-life analogy: a sticky floor versus a spring

Think of it like two ways of pulling a heavy box back to a spot marked zero. *L2* is a spring: it pulls hard when the box is far away but gets weaker as the box gets close, so the box creeps toward zero and never quite arrives. *L1* is a sticky floor: it resists with the same force however close the box is. If the data pushes more weakly than that friction, the box never leaves zero at all.

Formula, read aloud: L2's pull $2\lambda w_j$ says: the pull shrinks as the weight shrinks; L1's pull $\lambda\ \text{sign}(w_j)$ says: the same-sized pull toward zero, whatever the weight's size.

Where it breaks: friction acts on one box alone; here all weights move together, so a feature can be pushed to zero because a correlated feature already carries its signal.

Why L2 shrinks (but doesn't zero) weights

Gradient descent on ridge: $\mathbf{w} \leftarrow \mathbf{w} - \eta(\nabla L_{\text{data}} + 2\lambda\mathbf{w}) = (1 - 2\eta\lambda)\mathbf{w} - \eta\nabla L_{\text{data}}$. Each step multiplies the weights by a factor slightly below 1 — **weight decay** — then applies the data gradient. At equilibrium, a weight survives only in proportion to the data evidence supporting it. It penalizes **large** weights quadratically, so it prefers spreading weight across many correlated features rather than concentrating it on one.

Real-life analogy: a leaky water tank

Think of it like a water tank that leaks a fixed *fraction* of its water every hour while a tap (the data's evidence) keeps refilling it. A full tank loses a lot per hour; a nearly empty one loses only a trickle. So the level settles where leak equals refill: a strong tap keeps it high, a weak tap keeps it low, and it runs dry only if the tap is fully off. That is weight decay: every weight shrinks, but none is forced to exactly zero.

Formula, read aloud: $\mathbf{w} \leftarrow (1 - 2\eta\lambda)\mathbf{w} - \eta\nabla L_{\text{data}}$ says: each step, keep slightly less than all of the weight, then add the data's correction.

Where it breaks: a tank can't go below empty, but a weight can cross zero and turn negative if the data pushes that way.

Elastic net

L1 alone has two weaknesses: with a group of highly correlated features it picks **one arbitrarily** (and the choice is unstable across samples), and with $d > n$ it can select at most n features. Adding an L2 term gives the **grouping effect** (correlated features get similar coefficients and enter/leave together) while keeping sparsity.

Real-life analogy: picking twins for a relay team

Think of it like a coach picking a relay team where two runners are identical twins with the same times. A strict "fewest runners" rule (lasso) picks one twin at random, and next season might pick the other: unstable. Adding a "share the load" rule (the L2 part) makes the twins come in or go out together, each running half the distance. Elastic net keeps the team small *and* treats look-alike runners fairly.

Where it breaks: twins are exact copies; real correlated features are only similar, so elastic net gives them similar, not identical, weights.

When to use each

Situation	Choice	Why
Many small effects, correlated features, prediction is the goal	**Ridge**	stable, keeps all features, handles collinearity, closed form
You believe few features matter; want feature selection / interpretability	**Lasso**	exact zeros, built-in selection
Many correlated features and you want sparsity	**Elastic net**	grouped selection, stable
$d \gg n$ (genomics, text)	Elastic net or ridge	lasso saturates at n features
Inference on coefficients (CIs, p-values)	OLS (or careful post-selection inference)	penalties bias coefficients; lasso's selection invalidates naive p-values
Huge sparse data (ads, text)	L2 or L1 via SGD / FTRL-proximal	scalable; L1 keeps the model small in memory

5.12 Beyond Gaussian noise: Poisson, Gamma and Tweedie regression

Linear regression assumes the target is "a straight line plus bell-shaped noise with constant spread". Many targets break that: **counts** (orders per day, clicks), **positive skewed amounts** (claim sizes, delivery times), and **zero-inflated amounts** (spend per user: most spend nothing). Generalized linear models (Part 3) keep a linear score $\eta = \mathbf{w}^\top \mathbf{x}$ but change the link and the noise model:

Target	Model	Link: $\mathbb{E}[y \mid \mathbf{x}] =$	Variance grows like	Example
any real number	Gaussian (OLS)	η	constant	temperature
counts $0, 1, 2, \ldots$	**Poisson**	e^{η}	μ	orders per store per day
positive, right-skewed	**Gamma**	e^{η}	μ^2	insurance claim size, ride duration
zeros + positive amounts	**Tweedie** ($1 < p < 2$)	e^{η}	μ^p	spend per customer, total claims

With the log link, coefficients are **multiplicative**: e^{w_j} is the factor by which the expected value changes per unit of x_j (e.g. $w = 0.3 \rightarrow \times 1.35$). Fitting is by **IRLS** — Newton's method, which turns each step into a weighted least-squares problem:

$$\mathbf{w}_{\text{new}} = (X^\top W X)^{-1} X^\top W \mathbf{z}, \qquad z_i = \eta_i + \frac{y_i - \mu_i}{\mu_i}, \quad W_{ii} = \mu_i \ \ \text{(Poisson)}$$

```
def poisson_glm_irls(X, y, n_iter=50):
    """Poisson regression with log link: mu = exp(Xw). Newton step = weighted least squares."""
    Xb = np.c_[np.ones(len(X)), X]
    w = np.zeros(Xb.shape[1]); w[0] = np.log(y.mean())
    for _ in range(n_iter):
        mu = np.exp(Xb @ w)
        z = Xb @ w + (y - mu) / mu                                  # working response
        W = mu                                                      # working weights (Var = mu,
link'=1/mu)
        w_new = np.linalg.solve(Xb.T @ (Xb * W[:, None]), Xb.T @ (W * z))
        if np.abs(w_new - w).max() < 1e-10:
            return w_new
        w = w_new
    return w
```

Checked: on 3,000 simulated counts with true coefficients (0.5; 0.3, −0.2, 0.1), IRLS returns (0.4989; 0.2874, −0.2077, 0.1009) — identical to scikit-learn's `PoissonRegressor` to four decimals.

Why not just regress $\log y$? $\log 0$ is undefined for zero counts, and $\exp(\mathbb{E}[\log y])$ estimates the geometric mean, which **underestimates** the mean you usually need for totals (Part 19). A Poisson/Gamma/Tweedie GLM models $\mathbb{E}[y]$ directly.

Real-life analogy: choosing the right measuring cup

Think of it like a kitchen with different measuring tools: a jug for water (Gaussian: any amount, equal spoonful errors), a counter for eggs (Poisson: whole numbers, and busier kitchens have bigger day-to-day swings), a scale for flour sacks that are never negative and occasionally huge (Gamma), and a sales ledger where most customers buy nothing and a few buy a lot (Tweedie). Using a water jug to count eggs gives nonsense like "−0.4 eggs".

Formula, read aloud: each IRLS step solves an ordinary weighted least-squares problem where examples with smaller expected noise get more weight — "trust the steady measurements more".

Where it breaks: real data may have more spread than Poisson allows (**overdispersion**) — then use negative binomial or quasi-Poisson, or at least robust standard errors.

5.13 Quantile regression and GAMs

Quantile regression fits the τ-th conditional quantile instead of the mean, by minimizing the pinball loss (Part 15): $\min_{\mathbf{w}} \sum_i \rho_\tau(y_i - \mathbf{w}^\top \mathbf{x}_i)$ with $\rho_\tau(r) = \max(\tau r, (\tau - 1)r)$. Uses: delivery-time promises ("90% of orders arrive within..."), inventory (stock to the 95th percentile of demand), prediction intervals without assuming Gaussian noise, and heteroscedastic data where the spread itself depends on $\mathbf{x}$. It is robust to outliers in y (the median, $\tau = 0.5$, is least absolute deviations).

Generalized additive models (GAMs) replace each linear term with a smooth curve: $g(\mathbb{E}[y]) = \beta_0 + f_1(x_1) + f_2(x_2) + \ldots$, where each f_j is a penalized spline. You get non-linear effects **and** interpretability — each feature's effect is one plot. Modern "explainable boosting machines" are GAMs fitted with boosted shallow trees, sometimes with a few pairwise interactions. They're a strong choice in regulated domains (credit, healthcare) where a black box is unacceptable but straight lines are too crude.

Real-life analogy: promising a delivery time

Think of it like a food-delivery app promising "arrives in 35 minutes". Promise the *average* time and half of all customers get their food late. Quantile regression lets you promise the time by which 90% of orders arrive. A GAM is like an app that shows you separate, readable curves — "rain adds this much", "distance adds this much", "Friday night adds this much" — and adds them up.

Formula, read aloud: the pinball loss $\max(\tau r, (\tau - 1)r)$ says: being late costs τ per minute, being early costs $1 - \tau$ per minute — with $\tau = 0.9$, lateness is 9× more expensive, so the model shifts its prediction up.

Where it breaks: separate quantile models can cross (a 90% prediction below the 80% one) unless fitted jointly; and GAMs miss interactions you don't explicitly add.

5.14 Implementation from scratch

```python
import numpy as np

class LinearRegressionScratch:
    """OLS / ridge / lasso from scratch. Intercept is never penalized (we center the data)."""

    def __init__(self, alpha=0.0):
        self.alpha = alpha
```

```
    def _center(self, X, y):
        self.x_mean_, self.y_mean_ = X.mean(axis=0), y.mean()
        return X - self.x_mean_, y - self.y_mean_

    def fit_closed_form(self, X, y):
        """Ridge/OLS: solve (Xc^T Xc + n*alpha*I) w = Xc^T yc. alpha=0 → OLS via lstsq."""
        Xc, yc = self._center(X, y)
        n, d = Xc.shape
        if self.alpha == 0:
            self.w_ = np.linalg.lstsq(Xc, yc, rcond=None)[0]   # SVD: handles rank deficiency
        else:
            self.w_ = np.linalg.solve(Xc.T @ Xc + n * self.alpha * np.eye(d), Xc.T @ yc)
        self.b_ = self.y_mean_ - self.x_mean_ @ self.w_
        return self

    def fit_gd(self, X, y, lr=0.1, epochs=2000, tol=1e-10):
        """Batch GD on (1/2n)||Xw - y||^2 + (alpha/2)||w||^2. Scale features first!"""
        Xc, yc = self._center(X, y)
        n, d = Xc.shape
        w = np.zeros(d)
        prev = np.inf
        for _ in range(epochs):
            r = Xc @ w - yc                          # residuals, shape (n,)
            loss = 0.5 * (r @ r) / n + 0.5 * self.alpha * (w @ w)
            grad = Xc.T @ r / n + self.alpha * w     # shape (d,)
            w -= lr * grad
            if abs(prev - loss) < tol:
                break
            prev = loss
        self.w_, self.b_ = w, self.y_mean_ - self.x_mean_ @ w
        return self

    def fit_lasso_cd(self, X, y, n_iter=500, tol=1e-8):
        """Coordinate descent for (1/2n)||y - Xw||^2 + alpha*||w||_1."""
        Xc, yc = self._center(X, y)
        n, d = Xc.shape
        w = np.zeros(d)
        z = (Xc ** 2).sum(axis=0) / n                # per-feature curvature
        r = yc.copy()                                # residual y - Xw  (w = 0)
        for _ in range(n_iter):
            max_change = 0.0
            for j in range(d):
                if z[j] == 0:
                    continue
                r += Xc[:, j] * w[j]                 # partial residual without feature j
                rho = Xc[:, j] @ r / n
                w_new = np.sign(rho) * max(abs(rho) - self.alpha, 0.0) / z[j]   # soft-threshold
                r -= Xc[:, j] * w_new
                max_change = max(max_change, abs(w_new - w[j]))
                w[j] = w_new
            if max_change < tol:
                break
        self.w_, self.b_ = w, self.y_mean_ - self.x_mean_ @ w
        return self

    def predict(self, X):
        return X @ self.w_ + self.b_


def r2_score(y, y_hat):
    return 1 - np.sum((y - y_hat) ** 2) / np.sum((y - y.mean()) ** 2)

def adjusted_r2(r2, n, d):
    return 1 - (1 - r2) * (n - 1) / (n - d - 1)
```

Verified against scikit-learn (`LinearRegression`, `Ridge(alpha=n*alpha)`, `Lasso(alpha)`) on synthetic data: coefficients match to 10^{-6}; the lasso zeroes exactly the true-zero coefficients. Things an interviewer will probe in this code: why center instead of adding a ones column (so the intercept isn't penalized), why `lstsq` / `solve`

instead of `inv`, why GD needs scaled features, the $O(nd)$ cost per coordinate-descent sweep, and what `tol` is guarding against.

5.15 Interview questions on linear regression

Q: "What happens to OLS if you duplicate every row of the dataset?"

Strong answer. "The coefficients don't change — $X^\top X$ and $X^\top \mathbf{y}$ both double, so $\hat{\mathbf{w}}$ is identical. But naive standard errors shrink by $\sqrt{2}$ because the formula thinks n doubled, so confidence intervals become falsely narrow. It's a clean illustration that the point estimate depends on the data's shape while inference depends on the independence assumption. And if rows were duplicated only in part of the data, those rows get double weight and the coefficients *do* shift."

Q: "What happens if you add a feature that is pure noise?"

Strong answer. "Training R^2 goes up slightly — it can never go down, because the model can always set that weight to zero, and it will fit some noise with it. Adjusted R^2 usually goes down. Test error goes up slightly, by roughly σ^2/n per useless feature for OLS. With many noise features relative to n, the effect compounds — that's where regularization or feature selection pays off."

Q: "Two features are perfectly correlated. What happens with OLS, ridge and lasso?"

Strong answer. "OLS: $X^\top X$ is singular, so there's a line of equally good solutions — solvers either fail or, with a pseudo-inverse, return the minimum-norm one, which splits the weight equally. Ridge: unique solution that also splits the weight equally, because for a fixed sum $w_1 + w_2$, $w_1^2 + w_2^2$ is minimized when they're equal. Lasso: $|w_1| + |w_2|$ is constant along the whole segment for same-sign splits, so the solution is non-unique and in practice the solver puts all weight on one of them — arbitrary and unstable. Elastic net restores the equal split."

Follow-ups that expose shallow answers

"Is linear regression a linear model if you include x^2?" — *Yes; it's linear in the parameters, which is what makes the loss quadratic and the solution closed-form.*

"Can you use linear regression for a 0/1 target?" — *You can (the "linear probability model"), and OLS coefficients are even interpretable as changes in probability, but predictions escape $[0, 1]$, errors are heteroscedastic by construction ($\mathrm{Var} = p(1-p)$), and squared error on labels penalizes confidently correct predictions beyond 1 — Part 6 explains why logistic regression fixes all three.*

"Why standardize before ridge but not before OLS?" — *OLS is equivariant to feature scaling (the coefficient rescales inversely; predictions are unchanged). The ridge penalty isn't: it penalizes coefficients in the feature's units, so unscaled features are penalized unequally.*

PART 6

Logistic Regression

Logistic regression in one breath

Model the **log-odds** as a linear function of the features, squash with a sigmoid to get a probability, and fit by maximum likelihood under a Bernoulli model. The loss is binary cross-entropy, it's convex, and its gradient has the same "error × input" form as linear regression.

The whole Part as one picture

Think of it like the win-probability meter shown during an IPL chase. Each clue (runs needed, wickets left, balls left) gets a weight, and the weighted clues are added into one score. The sigmoid squashes that score into a chance between 0% and 100%. Training adjusts the weights so that, across thousands of past matches, the meter's chances match who actually won, with the harshest penalty for being confidently wrong. A threshold then turns "72% chance" into a decision, and calibration checks that "72%" really wins about 72% of the time.

Where it breaks: the meter adds clues independently in log-odds; in real cricket, wickets matter more late in an innings, and such interactions must be added as extra features.

6.1 Why linear regression is inappropriate for classification

Fit OLS to 0/1 labels and threshold at 0.5? Four problems:

1. **Outputs aren't probabilities.** $\mathbf{w}^\top \mathbf{x}$ ranges over $\mathbb{R}$; predictions of −0.3 or 1.4 are meaningless as probabilities.
2. **Correct points hurt you.** Squared error punishes a confidently correct prediction ($\hat{y} = 3$ for $y = 1$) as much as a wrong one. A cluster of easy, far-away positives drags the fitted line and **moves the decision boundary** into the wrong place.
3. **Wrong noise model.** A Bernoulli label has variance $p(1-p)$ — heteroscedastic by construction — so Gaussian-noise assumptions (and OLS inference) are wrong.
4. **Sigmoid + MSE doesn't fix it.** The loss $(\sigma(z) - y)^2$ is **non-convex** in $\mathbf{w}$, and its gradient $2(\sigma - y)\,\sigma(1-\sigma)\,\mathbf{x}$ **vanishes when the model is confidently wrong** ($\sigma \approx 0$, $y = 1$) — exactly when you need the biggest correction. Cross-entropy's gradient is $(\sigma - y)\mathbf{x}$: the $\sigma(1-\sigma)$ factor cancels.

Real-life analogy: a straight plank on a stair step

Think of it like laying a straight plank across a single stair step: the floor is 0, the step is 1. The plank keeps climbing past the step into thin air (predictions above 1) and dips below the floor (below 0). Worse, points far along the top step still tug the plank toward them, tilting it and moving the spot where it crosses halfway, so the boundary lands in the wrong place. The sigmoid is a curved ramp that flattens out at both levels.

Where it breaks: the curved shape alone isn't the fix; you also need the right loss, because a sigmoid scored with squared error still barely pushes when it is confidently wrong.

```
 y                                  OLS line tilted by far-away easy positives
 1 |           o  o o        o o o o o o o o o o o
   |          /                                          the 0.5 crossing (boundary)
0.5|- - - - -/- - - - - - - - - - - - - - - - - -        shifts right: points near
   |         /                                          x = 3..4 now misclassified
 0 | x x x x x x x
   +-------------------------------------------> x
```

6.2 Sigmoid and log-odds

$$p = P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = \mathbf{w}^\top \mathbf{x} + b = \log \frac{p}{1 - p} \quad \text{(the logit)}$$

Useful identities: $\sigma(-z) = 1 - \sigma(z)$; $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \le \frac{1}{4}$; $\sigma^{-1}(p) = \log \frac{p}{1-p}$.

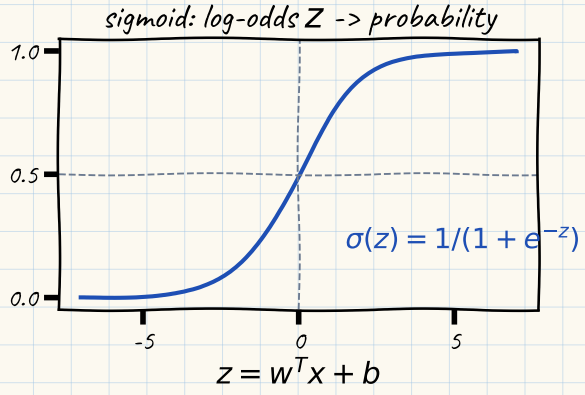
Real-life analogy: a bookmaker's odds

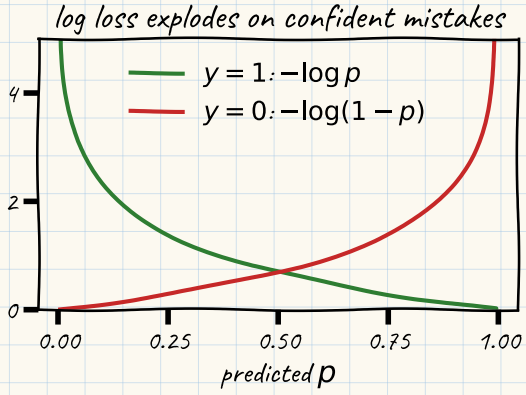
Think of it like a bookmaker saying a horse's odds are 3 to 1 in its favour: three chances to win for every one to lose, which is a 75% chance. Odds are awkward to add, so the model works with log-odds, where evidence simply adds up: each clue adds its weight to the score, which multiplies the odds. The sigmoid is the translator from that score back to a 0–100% chance.

Formula, read aloud: $p = \frac{1}{1+e^{-z}}$ says: probability = 1 ÷ (1 + e to the power of minus the score); a score of 0 gives 50%, big positive scores approach 100%, big negative scores approach 0%.

Where it breaks: real bookmakers pad their odds for profit, so their implied chances add up to more than 100%; the model's probabilities are honest estimates that add up to exactly 100%.

Interpretation of coefficients. Increasing x_j by one unit adds w_j to the log-odds, i.e. **multiplies the odds by** e^{w_j} (the odds ratio). $w_j = 0.7 \Rightarrow$ odds ×2.01. The effect on the *probability* is not constant: it's largest near $p = 0.5$ (slope $w_j/4$) and small near 0 or 1.





Sigmoid maps log-odds to probabilities; log loss punishes confident mistakes without bound

6.3 Decision boundary

Predict 1 when $p \ge t \Leftrightarrow z \ge \text{logit}(t)$. With $t = 0.5$: $\mathbf{w}^\top \mathbf{x} + b \ge 0$. The boundary is a **hyperplane** with normal vector $\mathbf{w}$; $|z|/\|\mathbf{w}\|$ is the distance to it. Logistic regression is therefore a **linear classifier**: nonlinear boundaries require nonlinear features (polynomials, interactions, splines, embeddings) — or a different model.

Real-life analogy: a border drawn with a ruler

Think of it like a state border drawn with a ruler on a map. Which side a town lies on decides the prediction; how far it lies from the border decides how confident the model is. Changing the threshold slides the border sideways without turning it. To get a winding border you must first redraw the map itself, which is what adding squared or interaction features does.

Formula, read aloud: $\mathbf{w}^\top \mathbf{x} + b \geq 0$ says: add up each feature times its weight, plus the bias; a positive total means "on the class-1 side of the border".

Where it breaks: a map is flat and 2-D; the real border is a flat sheet in as many dimensions as there are features, and it is straight only in the features you give it.

6.4 Likelihood and binary cross-entropy

Derivation — from Bernoulli likelihood to BCE

Each label is a Bernoulli draw: $P(y_i \mid \mathbf{x}_i) = p_i^{y_i}(1-p_i)^{1-y_i}$ with $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$. Independence ⇒ product; take negative log and average:

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log p_i + (1-y_i)\log(1-p_i)\Big]$$

This is **binary cross-entropy** / log loss: the cross-entropy between the empirical label distribution and the model's Bernoulli.

Logit form (what you implement): using $\log p = -\log(1+e^{-z})$ and $\log(1-p) = -z - \log(1+e^{-z})$:

$$\ell_i = \log(1+e^{z_i}) - y_i z_i \quad (\text{labels } 0/1) \qquad \text{or} \qquad \ell_i = \log(1+e^{-\tilde{y}_i z_i}) \quad (\tilde{y} \in \{-1,+1\})$$

The second form shows logistic loss as a function of the **margin** $\tilde{y}z$ — a smooth relative of the SVM's hinge loss (Part 15).

Real-life analogy: scoring a rain forecaster

Think of it like scoring a weather forecaster at the end of each day. If she said 90% rain and it rained, she loses a little ($-\log 0.9 \approx 0.1$). If she said 1% and it poured, she loses a lot ($-\log 0.01 \approx 4.6$). Maximum likelihood picks the forecaster whose predictions made the actual history least surprising overall.

Formula, read aloud: $\mathcal{L} = -\frac{1}{n}\sum[y_i \log p_i + (1-y_i)\log(1-p_i)]$ says: for each example, take the log of the probability you gave to what really happened, flip the sign, and average.

Where it breaks: a real scorer would cap the penalty; here saying exactly 0% for something that happens costs infinity, which is why code computes the loss from logits or clips probabilities.

6.5 The gradient

Derivation — the gradient is (prediction − label) × input

For one example, $\ell = -[y\log\sigma(z) + (1-y)\log(1-\sigma(z))]$, $z = \mathbf{w}^\top \mathbf{x}$.

$$\frac{\partial \ell}{\partial z} = -\Big[\frac{y}{\sigma}\sigma(1-\sigma) - \frac{1-y}{1-\sigma}\sigma(1-\sigma)\Big] = -[y(1-\sigma) - (1-y)\sigma] = \sigma(z) - y$$

$$\nabla_{\mathbf{w}}\mathcal{L} = \frac{1}{n}\sum_i (p_i - y_i)\,\mathbf{x}_i = \frac{1}{n}X^\top(\mathbf{p} - \mathbf{y})$$

Identical in form to linear regression ($X^\top(\hat{\mathbf{y}} - \mathbf{y})$): both are generalized linear models with canonical links, and for canonical links the gradient is always (mean prediction − target) × features.

Hessian: $\nabla^2\mathcal{L} = \frac{1}{n}X^\top SX$, $S = \mathrm{diag}(p_i(1-p_i)) \succeq 0$ ⇒ **convex**: any local minimum is global. Newton's method on this is **IRLS** (iteratively reweighted least squares).

Consequence of the first-order condition (with an intercept): $\sum_i p_i = \sum_i y_i$ at the optimum — the average predicted probability equals the observed positive rate. That's one reason unregularized logistic regression tends to be **well calibrated** on its training distribution.



Real-life analogy: a shopkeeper's evening correction

Think of it like a kirana shopkeeper who guesses each morning the chance that a regular customer drops by. Each evening he compares guess with reality: he said 70% and they came (1), so his error is −0.3. He then nudges the importance of each clue that was active that day (weekend, festival, rain) by that error. No extra fudge factor, just (guess − truth) × clue. Once his guesses settle, his average guessed chance equals how often customers actually came.

Formula, read aloud: $\nabla_{\mathbf{w}}\mathcal{L} = \frac{1}{n}X^\top(\mathbf{p} - \mathbf{y})$ says: the slope of the loss = the average, over examples, of (predicted probability − actual label) × that example's features; the weights step the opposite way.

Where it breaks: if one clue perfectly separates visits from no-visits, his corrections never stop and the weights grow forever, as the trap below explains.

Trap: perfectly separable data

If a hyperplane separates the classes perfectly, the MLE **does not exist**: scaling $\mathbf{w}$ up always pushes all p_i closer to their labels and lowers the loss, so $\|\mathbf{w}\| \to \infty$. Symptoms: huge coefficients, "failed to converge" warnings, probabilities of exactly 0/1. Regularization (or early stopping) gives a finite, sensible solution. This also happens in high dimensions ($d \gtrsim n$) where separation is almost guaranteed.

6.6 Regularization

Add $\lambda\|\mathbf{w}\|_2^2$ (default in most libraries) or $\lambda\|\mathbf{w}\|_1$ (sparse features, e.g. text), or elastic net. In scikit-learn, `C` is the **inverse** regularization strength ($C = 1/\lambda$ up to scaling): smaller C = stronger regularization. Bayesian reading: Gaussian/Laplace priors on the weights (MAP). Regularization also shrinks probabilities toward 0.5 — slightly underconfident predictions are the price for lower variance.



Real-life analogy: an editor who trims bold claims

Think of it like a newspaper editor who tones down any claim that sounds too sure of itself. A reporter (a weight) may make a strong claim only if the evidence is overwhelming; otherwise the editor softens it. So the paper's forecasts drift slightly toward "50–50". In scikit-learn, `C` is the editor's *patience*: a small `C` means a strict editor (strong regularization), a large `C` a relaxed one.

Where it breaks: a human editor judges each claim by its meaning; the penalty only looks at the size of each weight, and that size depends on the feature's units, so features should be scaled first.

6.7 Threshold selection

The model outputs probabilities; the **decision** needs a threshold, and 0.5 is almost never right.

Derivation — the cost-optimal threshold

Let C_{FP} = cost of a false positive, C_{FN} = cost of a false negative, and assume p is calibrated. Predict positive when the expected cost of doing so is lower:

$$\underbrace{(1-p)\,C_{\text{FP}}}_{\text{cost if we say positive}} < \underbrace{p\,C_{\text{FN}}}_{\text{cost if we say negative}} \iff p > \boxed{t^* = \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}}}$$

Fraud example: blocking a legit transaction costs ~$5 of friction/support; missing fraud costs ~$200. $t^* = 5/205 \approx 0.024$ — flag anything above 2.4%. This only works if probabilities are calibrated.



Real-life analogy: setting a smoke alarm

Think of it like choosing how sensitive a kitchen smoke alarm should be. A false alarm while cooking costs a moment of annoyance; missing a real fire costs the house. Because a miss is so much costlier, you set the alarm to ring even when the chance of fire is small. The right threshold is not 50%; it is set by the ratio of the two costs.

Formula, read aloud: $t^* = \frac{C_{\text{FP}}}{C_{\text{FP}}+C_{\text{FN}}}$ says: act when the chance exceeds (cost of a false alarm) ÷ (cost of a false alarm + cost of a miss); with $5 against $200, that is about 2.4%.

Where it breaks: this only works if the alarm's "chance of fire" is honest (calibrated), and real costs are rarely known exactly or the same for every case.

Other ways to choose a threshold: maximize F1 or F_β on validation (when costs are unknown but a precision/recall weighting is); meet a constraint ("precision ≥ 95%", "recall ≥ 90%"); meet a **capacity** ("the review team can handle 2,000 cases/day" ⇒ threshold = the score of the 2,000th case); or pick per segment when costs differ. Always pick on validation data, never test.

6.8 Calibration

Definition. A model is calibrated if, among all examples where it predicts p, a fraction p are actually positive: $P(Y = 1 \mid \hat{p} = p) = p$.

Why it matters: expected-value decisions (thresholds from costs, bidding in ads: bid ∝ pCTR × value), combining model outputs, communicating risk, and ranking across segments. **AUC is blind to calibration** — any monotonic transform of scores leaves AUC unchanged.

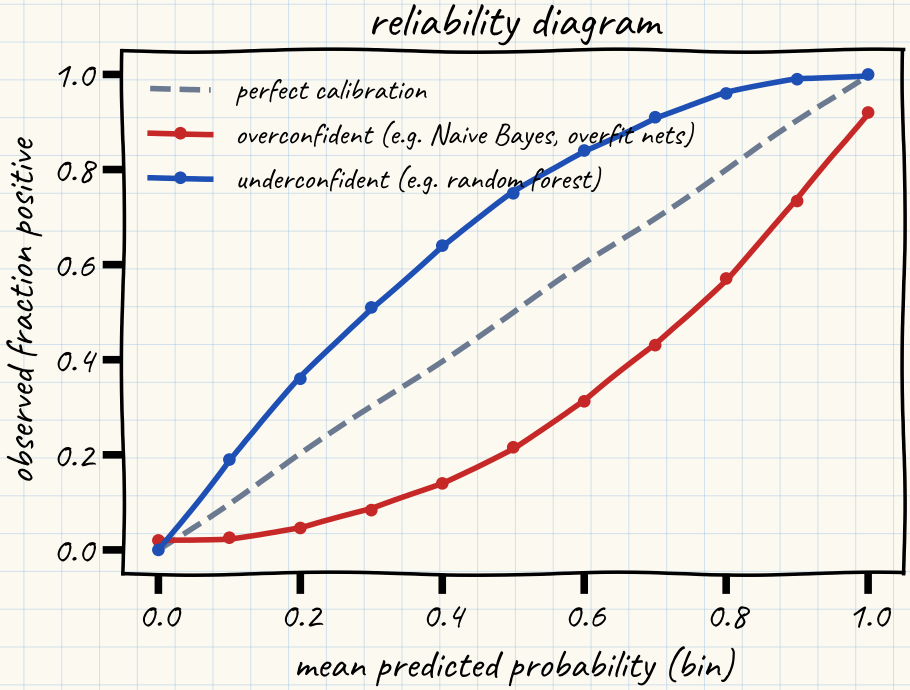
Real-life analogy: "reaching in 5 minutes"

Think of it like a friend who texts "reaching in 5 minutes". A calibrated friend arrives in about 5 minutes when they say 5, and in about 20 when they say 20. An uncalibrated friend always takes three times longer. Notice you can still *rank* the second friend's messages (a "20" means later than a "5"), which is why AUC can look fine while calibration is badly off.

Formula, read aloud: $P(Y = 1 \mid \hat{p} = p) = p$ says: out of all the times the model says "30%", the event should really happen about 30% of the time.

Where it breaks: calibration is judged over many predictions grouped by their stated probability, not from one promise, and a model can be calibrated overall yet off for one segment.

Measure: reliability diagram (bin predictions, plot mean prediction vs. observed rate); **Expected Calibration Error** $\mathrm{ECE} = \sum_b \frac{n_b}{n} |\mathrm{acc}_b - \mathrm{conf}_b|$; **Brier score** $\frac{1}{n}\sum(p_i - y_i)^2$ (a proper scoring rule mixing calibration and discrimination); log loss.



Reliability diagram: points below the diagonal are overconfident

What typically miscalibrates	Why
Class weighting / over- or undersampling	changes the effective prior; predicted rates inflate for the upweighted class
Deep nets trained long, overfit gradient boosting	push scores toward extremes (overconfidence); modern nets are often overconfident
SVMs, AdaBoost (max-margin methods)	scores are margins, not probabilities — a sigmoid-shaped distortion (Platt scaling was invented for SVM outputs)
Naive Bayes	double-counts correlated evidence ⇒ extreme probabilities
Strong regularization, bagging/ random forests	pulls predictions toward the middle (underconfident at extremes)
Distribution/label shift	base rate in production ≠ training

Fix (post-hoc, fit on a held-out calibration set): Platt scaling (fit a 1-D logistic regression on the scores — good for sigmoid-shaped distortions, little data); **isotonic regression** (non-parametric monotonic fit — more

flexible, needs more data, can overfit); **temperature scaling** for neural nets (divide logits by a single learned $T > 0$ — preserves ranking and accuracy). For resampling, correct analytically: if negatives were kept at rate w, $p = \frac{q}{q+(1-q)/w}$.

6.9 Multiclass logistic regression and softmax

One-vs-rest: train K binary classifiers, pick the highest score. Simple, parallel, but scores aren't jointly normalized and classes compete only indirectly.

Multinomial (softmax) regression: one weight vector per class, jointly normalized:

$$P(y = k \mid \mathbf{x}) = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad z_k = \mathbf{w}_k^\top \mathbf{x} + b_k, \qquad \mathcal{L} = -\frac{1}{n}\sum_i \log p_{i,y_i}$$

Real-life analogy: a talent show's winning chances

Think of it like a singing talent show. The judges give each contestant a raw score (a logit). Softmax turns the scores into winning chances that add up to 100%: it boosts each score exponentially, so a clear leader gets a large share, then divides by the total. *One-vs-rest* instead judges each singer alone against "everyone else", so those chances need not add up.

Formula, read aloud: $\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$ says: chance of class k = (its boosted score) ÷ (the sum of everyone's boosted scores).

Where it breaks: a talent show has exactly one winner; if an example can belong to several classes at once (a photo of a beach *and* a sunset), use separate yes/no sigmoids instead.

Derivation — softmax cross-entropy gradient

With one-hot target $\mathbf{y}$ and $\ell = -\sum_k y_k \log p_k$:

$$\frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j) \quad \Rightarrow \quad \frac{\partial \ell}{\partial z_j} = -\sum_k \frac{y_k}{p_k} p_k(\delta_{kj} - p_j) = -y_j + p_j \sum_k y_k = \boxed{p_j - y_j}$$

Again: **prediction minus target**. So $\nabla_W \ell = (\mathbf{p} - \mathbf{y})\mathbf{x}^\top$.

Properties you'll be asked about.

- **Shift invariance:** $\text{softmax}(\mathbf{z} + c) = \text{softmax}(\mathbf{z})$. Parameters are identifiable only up to a common shift (fix one class to 0, or regularize). Also the basis of the stability trick: compute with $\mathbf{z} - \max_k z_k$ so no e^z overflows, and use **log-sum-exp** for the loss: $\log \sum e^{z_j} = m + \log \sum e^{z_j - m}$.
- **K = 2 reduces to sigmoid:** $\frac{e^{z_1}}{e^{z_0}+e^{z_1}} = \sigma(z_1 - z_0)$.
- **Temperature:** $\text{softmax}(\mathbf{z}/T)$ — $T \to 0$ approaches argmax, $T \to \infty$ uniform. Used in calibration, distillation, sampling.
- **Softmax vs. independent sigmoids:** softmax for mutually exclusive classes; K sigmoids for **multi-label** problems (an image can be "beach" *and* "sunset").

```python
import numpy as np

def stable_bce_from_logits(z, y):
    """mean[log(1+e^z) - y*z], computed without overflow (np.logaddexp(0, z) = log(1+e^z))."""
    return np.mean(np.logaddexp(0.0, z) - y * z)

def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True)          # shift invariance → no overflow
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)
```

```python
def softmax_cross_entropy(Z, y):
    """Z: (n, K) logits, y: (n,) integer labels. Uses log-sum-exp; never takes log(0)."""
    Z = Z - Z.max(axis=1, keepdims=True)
    log_probs = Z - np.log(np.exp(Z).sum(axis=1, keepdims=True))
    return -log_probs[np.arange(len(y)), y].mean()
```

6.10 Newton's method for logistic regression (IRLS), in code

Gradient descent takes many small steps; **Newton's method** uses the curvature (Hessian) to jump close to the optimum in a handful of iterations. For logistic regression each Newton step is a weighted least-squares solve — hence the name *iteratively reweighted least squares*:

$$\mathbf{w} \leftarrow \mathbf{w} - \left(X^\top S X + \lambda I\right)^{-1}\left(X^\top(\mathbf{p} - \mathbf{y}) + \lambda \mathbf{w}\right), \qquad S = \operatorname{diag}\left(p_i(1-p_i)\right)$$

```python
def logreg_irls(X, y, n_iter=25, l2=1e-6):
    Xb = np.c_[np.ones(len(X)), X]
    w = np.zeros(Xb.shape[1])
    for _ in range(n_iter):
        p = 1 / (1 + np.exp(-(Xb @ w)))
        g = Xb.T @ (p - y) + l2 * w                          # gradient
        S = p * (1 - p)
        H = Xb.T @ (Xb * S[:, None]) + l2 * np.eye(len(w))   # Hessian X^T S X
        step = np.linalg.solve(H, g)
        w -= step
        if np.abs(step).max() < 1e-10:
            break
    return w
```

Checked: on 800 examples with 5 features it matches scikit-learn's unregularized `LogisticRegression` coefficients and intercept to 10^{-3}, converging in under 10 iterations. **Trade-off:** each step costs $O(nd^2 + d^3)$ — excellent for tens or hundreds of features (this is what classical statistics packages do, and it gives standard errors from H^{-1} for free), impractical for millions of features, where L-BFGS or SGD win. **Failure mode:** with perfectly separable data the weights diverge ($S \to 0$) — the small ridge term `l2` keeps the Hessian invertible.



Real-life analogy: a GPS vs. walking by feel

Think of it like reaching the lowest point of a valley. Gradient descent walks by feel: a small step downhill, feel again, repeat — hundreds of steps. Newton's method is like having a GPS that knows the valley's *shape* (its curvature): it computes where the bottom should be and jumps almost straight there, re-checking a few times.

Formula, read aloud: new weights = old weights − (curvature)$^{-1}$ × (slope): "divide the slope by how sharply the ground curves to get the right step length in every direction".

Where it breaks: the GPS computation gets very expensive as the map grows (many features), and far from the bottom the curvature can mislead — then you need damping or a line search.

6.11 Class weights, sample weights and prior shift

Logistic regression's intercept absorbs the base rate. If you **undersample negatives** by a factor β (keep a fraction β of them) to speed up training, predicted odds are inflated by $1/\beta$. Correct them after training:

$$\operatorname{logit}(p_{\text{true}}) = \operatorname{logit}(p_{\text{trained}}) + \log \beta$$

Example: ads CTR ~1%; you keep 10% of non-clicks ($\beta = 0.1$). A trained prediction of 0.09 has odds 0.0989; multiply by 0.1 → odds 0.00989 → $p \approx 0.0098$ — back to the real scale. The same correction handles **prior shift** in production (the fraud rate doubles): adjust the intercept by the log of the ratio of new to old odds, if $P(\mathbf{x} \mid y)$ hasn't changed. **Class weights** (weighting positives by w) shift the intercept in the same way — fine for ranking, but the probabilities must be recalibrated before they're used as probabilities.



Real-life analogy: a survey that over-sampled one group

Think of it like a survey where you interviewed every cricket fan you met but only one in ten non-fans. Your raw results say "half of India loves cricket" because you over-counted fans. To report the true share, you scale the non-fans back up by 10. Undersampling negatives in training does the same to the model's probabilities, and the $\log \beta$ correction is the "scale back up" step.

Formula, read aloud: true log-odds = trained log-odds + log(fraction of negatives you kept) — since you kept only 10%, subtract $\log 10 \approx 2.3$ from the log-odds.

Where it breaks: the fix assumes you sampled negatives *at random*; if you dropped "easy" negatives selectively, the correction is no longer exact.

6.12 Interview questions on logistic regression

Q: "Why is logistic regression called regression when it performs classification?"

Strong answer. "Because it *is* a regression — on the log-odds. It models $\log \frac{p}{1-p}$ as a linear function of the features, and it estimates a continuous quantity, the conditional probability $P(y = 1 \mid x)$. Classification only happens when you add a decision rule on top, a threshold. Formally it's a generalized linear model: a Bernoulli response with the logit link, the same family as linear regression, which is the Gaussian response with the identity link — the name comes from that statistical tradition."

Q: "How would you explain a logistic regression coefficient to a product manager?"

Strong answer. "Each coefficient multiplies the odds. If the coefficient for 'has used the app in the last week' is 1.1, users with that behavior have about three times the odds of converting — $e^{1.1} \approx 3$ — holding the other features fixed. I'd add two caveats: that's association, not causation, and with correlated features the individual coefficients can be unstable, so I'd check they're consistent across bootstrap samples before telling a story about them."

Q: "Your logistic regression predicts well (AUC 0.85) but outputs probabilities of 0.3 for a segment that converts at 8%. What's going on?"

Strong answer. "Discrimination is fine, calibration isn't. Likely causes: the model was trained with class weights or downsampled negatives, which inflates predicted probabilities; the segment's base rate shifted since training; or the segment is underrepresented and the model extrapolates. Fixes: correct for the sampling rate analytically, recalibrate on recent held-out data with Platt or isotonic scaling, or add segment features. I'd verify with a reliability diagram per segment."

Follow-ups that expose shallow answers

"Is the decision boundary of logistic regression linear?" — *In the feature space you feed it, yes. With polynomial or interaction features it's linear in that expanded space and nonlinear in the original inputs.*

"Why not use accuracy as the training loss?" — *Piecewise constant ⇒ zero gradient almost everywhere; non-convex and NP-hard to optimize directly. Log loss is a smooth convex upper bound (up to scaling) and a* ***proper scoring rule****: it's minimized in expectation by reporting true probabilities.*

"What's the loss if the model predicts exactly 0 for a positive example?" — *Infinite ($-\log 0$). That's why implementations clip probabilities or, better, compute the loss from logits.*

"Can logistic regression handle class imbalance?" — *Yes; it still estimates $P(y \mid x)$ consistently. Imbalance affects the choice of threshold and metric more than the model. Reweighting shifts the intercept (and calibration) and can help when the minority class is underfit.*

Active recall — Set 5: Linear & logistic regression

Set 5 · Answer from memory, then check the Answer Key

Easy recall

1. Write the normal equation.
2. Write the binary cross-entropy loss.
3. What is the derivative of the sigmoid?
4. Define R^2 and adjusted R^2.
5. What is the soft-thresholding operator?

Conceptual

6. State the Gauss–Markov theorem and the assumptions it needs.
7. What does multicollinearity hurt, and what does it not hurt?
8. What happens to logistic regression on perfectly separable data?
9. How does ridge shrink different principal directions?
10. When would you use one-vs-rest vs. softmax vs. independent sigmoids?

Why

11. Why does squared error correspond to predicting the conditional mean?
12. Why does L1 produce exact zeros but L2 doesn't?
13. Why is sigmoid + MSE a poor choice for classification?
14. Why must you standardize features before ridge/lasso?
15. Why is AUC insensitive to calibration?

Scenario

16. Your regression's residual plot shows a funnel shape. What does it mean and what do you do?
17. A logistic model trained with 1:10 negative downsampling predicts 0.4 on average in production where the true rate is 5%. Fix it.
18. Missing fraud costs $300, blocking a legit user costs $10. What threshold?
19. Lasso selects feature A in one training run and its near-duplicate B in another. Why, and what do you use instead?
20. Test R^2 is −0.2. What does that mean?

Mathematical

21. Derive the gradient of the logistic loss.

22. Derive the ridge closed form.
23. Show that softmax with 2 classes equals a sigmoid of the logit difference.

Difficult follow-ups

24. Why is logistic regression often well calibrated, and when does that break?
25. Gradient descent from zero on a separable logistic problem without regularization: what does $\mathbf{w}/\|\mathbf{w}\|$ converge to?

2027 additions

26. When is Poisson or Gamma regression better than OLS on $\log y$?
27. Write one Newton (IRLS) step for logistic regression. When is it better than gradient descent?
28. You trained a CTR model keeping 10% of negatives. The model predicts 0.09. What is the corrected probability?

PART 7

Decision Trees

Decision trees in one breath

Recursively cut the feature space with axis-aligned yes/no questions, each chosen greedily to make the resulting groups as **pure** as possible, and predict the majority class (or mean) in each final box. Trees are flexible, need no scaling, capture interactions for free — and are high-variance, which is why we almost always use them in ensembles.

The whole Part as one picture

Think of it like the game of 20 Questions. Each question ("Is it bigger than a cat?") splits the remaining possibilities into a yes pile and a no pile, and a good player picks the question that makes the piles as clean as possible. When a pile holds only one kind of answer, you stop and guess it. Ask too many oddly specific questions and you end up memorizing the people you played with instead of learning a strategy: that is overfitting. Asking many different players and pooling their guesses is the idea behind ensembles.

Where it breaks: a tree's questions are always about one feature at a time ("is $x_j \leq t$?"), and each is chosen greedily, without planning the questions that follow.

7.1 Tree construction

CART (Classification and Regression Trees) grows a binary tree greedily, top-down:

```
build(node, data):
    if stopping rule holds (max depth, min samples, node pure, no gain):
        make leaf: majority class / mean of y  (or class proportions)
        return
    for each feature j, for each candidate threshold t:
        score(j, t) = impurity(parent) - [n_L/n * impurity(L) + n_R/n * impurity(R)]
    pick (j*, t*) with the largest impurity decrease
    build(left child,  data[x_j* ⩽ t*])
    build(right child, data[x_j* >  t*])
```

Why greedy? Finding the optimal tree is NP-hard. Greedy splitting is fast and works well in practice, but it is **myopic**: it can miss splits that only pay off one level later. Classic failure: XOR — neither x_1 nor x_2 alone reduces impurity, so a greedy tree may refuse to split at all even though two levels would separate the data perfectly (a reason to allow a minimum depth or rely on ensembles with randomization).

Real-life analogy: a post office sorting letters

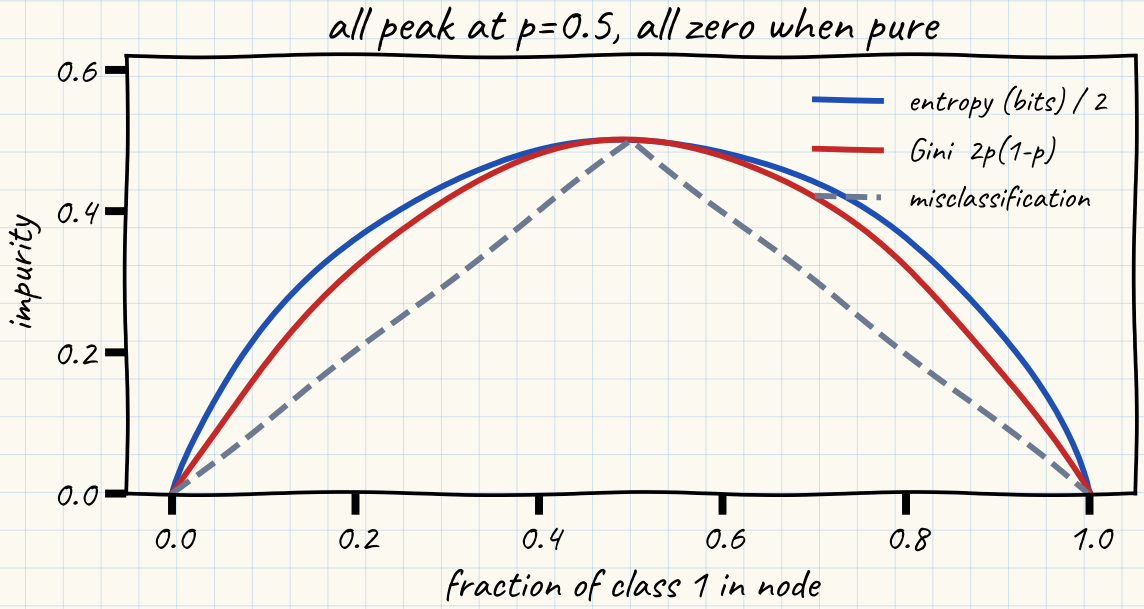
Think of it like a post office sorting a mountain of letters. First split by state, then each pile by city, then by PIN code, stopping when a pile goes to one destination. At every step the sorter picks the one question that cleans up the piles most *right now*, never thinking two steps ahead. That is fast and usually works well: greedy splitting.

Where it breaks: PIN codes were designed to be split one level at a time; real data may only make sense after two questions together (the XOR case), and a greedy sorter who sees no gain from either question alone may stop sorting too early.

7.2 Splitting criteria

For a node with class proportions $p_1, \ldots, p_K$:

$$\text{Entropy: } H = -\sum_k p_k \log_2 p_k \qquad \text{Gini: } G = \sum_k p_k(1-p_k) = 1 - \sum_k p_k^2 \qquad \text{Misclassification: } E = 1 - \max_k p_k$$



All impurity measures peak at a 50/50 node and vanish on pure nodes

Entropy — the expected number of bits to encode the class of a random sample in the node (Part 3). **Gini impurity** — the probability that a randomly drawn sample would be misclassified if labeled randomly according to the node's class distribution. Both are strictly concave in $\mathbf{p}$, zero on pure nodes, maximal on uniform nodes. In practice they choose the same split the vast majority of the time; Gini is marginally cheaper (no logarithm) and is scikit-learn's default.

Real-life analogy: a mixed box of Diwali sweets

Think of it like judging how mixed-up a box of Diwali sweets is. A box of only laddoos is pure: zero messiness. A box that is half laddoos, half barfis is as messy as it gets. *Gini* asks: if I pick two sweets at random (putting the first back), how likely are they to be different kinds? *Entropy* asks: how many yes/no questions do I need, on average, to guess the kind of a random sweet? A good split leaves each smaller box as pure as possible.

Formula, read aloud: $G = 1 - \sum_k p_k^2$ says: messiness = 1 – the chance that two random picks are the same kind.

Where it breaks: a box is simply counted; a tree judges a split by the size-weighted messiness of both child boxes, and Gini and entropy almost always pick the same split.

Information gain — the intuition and the formula

$$\mathrm{IG}(S, \text{split}) = H(S) - \sum_{c \in \{L,R\}} \frac{|S_c|}{|S|} H(S_c)$$

Intuition: before the split you're uncertain about the label by $H(S)$ bits; after learning which side of the split a sample falls on, you're uncertain by the weighted average child entropy. Information gain is the **uncertainty removed by asking the question** — formally the mutual information between the split indicator and the label.

Real-life analogy: a detective's best question

Think of it like a detective with twenty suspects. Before asking anything, she is very unsure who did it. A good question ("Were you in Pune that night?") splits the suspects into groups where she is much surer. Information gain measures how much doubt the question removed: the doubt before, minus the average doubt after, weighting each group by its size.

Formula, read aloud: $\mathrm{IG} = H(S) - \sum_c \frac{|S_c|}{|S|} H(S_c)$ says: information gained = uncertainty before the question – the size-weighted uncertainty left in each answer group.

Where it breaks: a question like "What is your ID number?" puts every suspect in their own group and looks like perfect information, yet teaches nothing general; that is why C4.5 uses the gain ratio.

Worked example

Parent: 10 positive, 10 negative ⇒ $H = 1$ bit. Split A sends (8+, 2–) left and (2+, 8–) right. $H(0.8) = -0.8\log_2 0.8 - 0.2\log_2 0.2 \approx 0.722$. Weighted child entropy $= 0.722$. $\mathrm{IG} = 1 - 0.722 = 0.278$ bits. Gini: parent 0.5, children $2(0.8)(0.2) = 0.32$ each ⇒ decrease 0.18.

Why not use misclassification error to choose splits?

Parent: 400+ / 400– (error 0.5). **Split 1:** (300+,100–) and (100+,300–). **Split 2:** (200+,400–) and (200+, 0–).

Both have weighted misclassification error **0.25**, so misclassification can't tell them apart. But split 2 creates a **pure node** that needs no further work. Gini: split 1 → 0.375; split 2 → $0.75 \times 0.444 + 0.25 \times 0 = 0.333$ — Gini prefers split 2. Entropy agrees. The reason: misclassification error is **piecewise linear** in $\mathbf{p}$, so it doesn't reward making a node *more* lopsided unless the majority flips; strictly concave criteria always reward purity. (Misclassification is still a fine criterion for *pruning*.)

Bias toward many-valued features. Splitting a categorical feature with many levels (like user ID) into many branches can reach near-zero impurity by memorizing. C4.5's **gain ratio** divides IG by the split's own entropy to penalize this. Binary CART splits reduce the problem but high-cardinality features still win impurity contests more easily than they deserve — this also biases impurity-based feature importance.

7.3 Regression trees

Leaves predict the **mean** of the targets in the leaf (the constant minimizing squared error). Splits maximize the reduction in sum of squared errors (equivalently, variance reduction):

$$\Delta = \sum_{i \in S}(y_i - \bar{y}_S)^2 - \left[\sum_{i \in L}(y_i - \bar{y}_L)^2 + \sum_{i \in R}(y_i - \bar{y}_R)^2\right]$$

With MAE the leaf predicts the median. Consequences: predictions are **piecewise constant** (a staircase), and a tree **cannot extrapolate** — beyond the training range it predicts the edge leaf's mean. A tree-based demand model will never forecast sales higher than it has seen; a linear model (or a tree on detrended data) can.



Real-life analogy: quoting flat prices by block

Think of it like an estate agent who carves a city map into blocks and quotes every flat in a block at that block's average price. A good cut is one that makes prices within each block sit tightly around their own average. Quotes jump from block to block like a staircase instead of rising smoothly. And a brand-new luxury tower simply gets the nearest block's average: the tree cannot quote above anything it has seen.

Formula, read aloud: $\Delta = \mathrm{SSE}_S - (\mathrm{SSE}_L + \mathrm{SSE}_R)$ says: improvement = price spread in the whole area − (spread in the left block + spread in the right block), each measured as squared distance from its own average.

Where it breaks: a map is 2-D; tree cuts are straight lines on one feature at a time, in many dimensions, chosen greedily one after another.

7.4 A split computed by hand

Interviewers like to hand you a tiny table and ask which split a tree would choose. Ten loan applicants, 4 defaulted (D) and 6 repaid:

Candidate split	Left child	Right child
Income high vs. low	high: 5 people, 1 D	low: 5 people, 3 D
Has a credit card yes vs. no	yes: 7 people, 2 D	no: 3 people, 2 D

Gini and entropy, step by step

Parent: $p = 0.4$ → Gini $= 1 - 0.4^2 - 0.6^2 = 0.48$; entropy $= -0.4\log_2 0.4 - 0.6\log_2 0.6 = 0.971$ bits.

Income: high ($p = 0.2$): Gini 0.32, entropy 0.722. Low ($p = 0.6$): Gini 0.48, entropy 0.971. Weighted (5/10 each): Gini 0.400, entropy 0.846. **Gini decrease** 0.080**; information gain** 0.125**.**

Credit card: yes ($p = 2/7$): Gini 0.408, entropy 0.863. No ($p = 2/3$): Gini 0.444, entropy 0.918. Weighted (7/10, 3/10): Gini 0.419, entropy 0.880. **Gini decrease** 0.061**; information gain** 0.091**.**

Choose income — both criteria agree (they usually do). Note that "no card" has the higher default *rate* (67%), yet the split is worse overall: it isolates only 3 people, so the weighted impurity barely drops. A split is judged by how much it purifies **everyone**, weighted by group size.



Real-life analogy: the first question in "guess the student"

Think of it like a teacher trying to guess which students will fail, allowed one yes/no question to split the class. "Do you study after school?" splits the class into two large groups that are each much more predictable. "Were you absent on Monday?" finds a tiny group that's very likely to fail — but leaves everyone else as mixed as before. The tree picks the question that makes the *whole class* most predictable.

Formula, read aloud: gain = (impurity before) − (size-weighted average impurity after): "how much less mixed, on average, the class becomes after asking".

Where it breaks: a teacher can ask clever combined questions; a tree asks one feature at a time, greedily, so it can miss splits that only help in combination (like XOR).

7.5 Trees for regression: a worked split

For regression, impurity is the **variance** (equivalently, squared error around each child's mean). Delivery times in minutes: $\{20, 22, 24, 40, 44\}$, with the first three short-distance orders and the last two long-distance.

- Parent mean 30, sum of squared errors (SSE) $= 100 + 64 + 36 + 100 + 196 = 496$.
- Split by distance: short $\{20, 22, 24\}$ (mean 22, SSE $4 + 0 + 4 = 8$); long $\{40, 44\}$ (mean 42, SSE $4 + 4 = 8$). Total SSE $= 16$.
- Reduction: $496 - 16 = \mathbf{480}$ — nearly all the variation is explained by one split. Each leaf predicts its mean (22 or 42 minutes).



Real-life analogy: grouping deliveries by zone

Think of it like a delivery manager who notices times vary a lot and asks: "which single rule best groups similar deliveries?" Splitting by distance makes each group tight around its own average — so "predict the group's average" becomes accurate. A regression tree keeps splitting until the groups are tight enough or too small to trust.

Where it breaks: a leaf predicts a flat average, so a tree gives step-shaped predictions and can't extrapolate beyond the times it has seen (a 60-km order gets the same prediction as a 20-km one if they share a leaf).

7.6 Continuous features and categorical features

Continuous: sort the node's values; candidate thresholds are midpoints between consecutive distinct values; scan left to right, updating class counts (or sums) incrementally — $O(n)$ per feature after an $O(n \log n)$ sort. **Histogram-based** implementations (LightGBM, XGBoost `hist`, scikit-learn's `HistGradientBoosting`) first bucket each feature into ≤ 256 bins, turning split finding into $O(\text{bins})$ per feature per node and making the whole algorithm far faster and more cache-friendly with negligible accuracy loss.

Categorical: one-hot encoding makes each split "category = c vs. rest" — inefficient for many levels and it fragments data. Native handling partitions the categories into two groups. For binary classification (and regression) there's a neat result: **sort categories by their mean target and scan** — the optimal binary partition is

among those $k-1$ contiguous splits, found in $O(k \log k)$ instead of 2^{k-1}. LightGBM and CatBoost use variants of this (with regularization, because it overfits rare categories).



Real-life analogy: lining students up by height

Think of it like a PE teacher splitting a class into two teams by height. She lines everyone up from shortest to tallest and tries a cut between each pair of neighbours: one walk down the line. *Histograms* are faster still: group students into height bands first, then only try cuts between bands. For categories like home cities, she first ranks the cities by their average score, then tries cuts along that ranked list instead of testing every possible grouping.

Where it breaks: bands hide small height differences (usually harmless), and ranking a city by its average is unreliable when only two students come from it, which is why libraries add regularization for rare categories.

7.7 Missing values

Strategy	How	Notes
Learned default direction	during split search, try sending all missing values left and right; keep the better	XGBoost, LightGBM; scikit-learn trees since 1.3 (random forests since 1.4)
Surrogate splits	at each node, find backup splits on other features that best mimic the primary split	classic CART (R's `rpart`); robust but slower
Missing as its own category / indicator	add `is_missing_j` feature	lets the model use *missingness itself* as signal (often predictive: "income not reported")
Impute (median, model-based)	fill, then split	simplest; loses missingness signal unless paired with an indicator



Real-life analogy: a parcel with no floor number

Think of it like a delivery rider holding a parcel for a tower block with no floor number written on it. *Learned default direction*: past deliveries show that parcels with no floor usually belong upstairs, so go up. *Surrogate split*: use a different clue that gives the same answer, like the flat's name. *Missing as its own category*: "no floor given" is itself a clue, perhaps of a suspicious order. *Imputation*: just assume the most common floor.

Where it breaks: the rider learns from real past deliveries; if floor numbers start going missing for a new reason, such as an app bug, the learned default quietly sends parcels the wrong way.

7.8 Tree depth, overfitting and pruning

An unrestricted tree grows until every leaf is pure — on continuous features that often means **one training example per leaf**: zero training error, maximal variance. Tiny changes to the data can change the root split and hence the entire tree (**instability**).

Pre-pruning (early stopping): `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`, `min_impurity_decrease`. Cheap, but may stop before a split that enables good later splits (the XOR problem).

Post-pruning: grow the full tree, then cut back. **Cost-complexity (weakest-link) pruning** minimizes

$$R_\alpha(T) = R(T) + \alpha\,|T|$$

where $R(T)$ is the training error and $|T|$ the number of leaves. As α increases, the subtree whose removal increases error least per leaf removed is pruned first, producing a nested sequence of trees; pick α by cross-validation (`ccp_alpha` in scikit-learn).

Real-life analogy: a gardener pruning a hedge

Think of it like a hedge left to grow wild: it sends a twig toward every sunbeam (every training point) and ends up shaggy and fragile. *Pre-pruning* is refusing to let it grow past a set height. *Post-pruning* is letting it grow fully, then snipping the branches that add the least shape for their size. α is how ruthless the gardener is; cross-validation picks the level that looks best from the street.

Formula, read aloud: $R_\alpha(T) = R(T) + \alpha\,|T|$ says: the tree's total cost = its training mistakes + α × the number of leaves.

Where it breaks: a gardener can see the whole hedge at once, but pre-pruning must stop growth before knowing whether the next branch would have paid off.

7.9 Feature importance

Method	How	Pitfalls
Impurity-based (MDI)	total impurity decrease from splits on the feature, weighted by samples reaching the node	computed on **training** data — rewards overfit splits; biased toward high-cardinality and continuous features; splits credit among correlated features
Permutation importance	shuffle one feature in **validation** data, measure the metric drop	model-agnostic, less biased; correlated features mask each other (shuffling one, the other still carries the signal); creates unrealistic feature combinations
SHAP (TreeSHAP)	Shapley-value attribution per prediction, exact and fast for trees	local + global explanations; still describes the *model*, not causality; correlated features share credit in ways that need care
Drop-column	retrain without the feature	most faithful to "is it needed", expensive

Real-life analogy: credit in a group project

Think of it like deciding who deserves credit for a group project. *Impurity importance* counts how often each member spoke in rehearsals: talkers win, even if they said little of use. *Permutation importance* swaps one member for a random stranger on presentation day and sees how far the grade drops. *SHAP* splits the grade fairly by averaging each member's contribution over every possible sub-team. *Drop-column* redoes the whole project without them.

Where it breaks: team members truly cause the grade; feature importance only shows what the *model* leans on, so a leaked proxy can top the list and a feature can look useless because its twin covered for it.

Importance ≠ causation, and ≠ "necessary"

A feature can have high importance because it's a proxy for a leaked label, and near-zero importance because a correlated twin absorbed its role. Never remove features or tell causal stories from one importance ranking.

7.10 Computational complexity

Step	Cost	Why
Training (balanced tree, presorted)	$\approx O(d\,n \log n)$	each level touches all n samples across all nodes, for d features; depth $\approx \log n$
Training (worst case, degenerate chain)	$O(d\,n^2)$	depth up to n
Histogram trees	$O(d\,n)$ binning once + $O(d \cdot \text{bins})$ per node	why LightGBM/XGBoost-hist are fast
Prediction	$O(\text{depth})$ per example	a path of comparisons — microseconds
Memory	$O(\text{number of nodes})$	$\leq 2n$ nodes for a full tree



Real-life analogy: putting up signposts in a library

Think of it like setting up a big library. Training is the slow part: sort every book by each property once, then walk the shelves level by level putting up signposts, where each level touches every book. *Histograms* are like shelving by first letter instead of exact title: far fewer choices to consider. Prediction is the fast part: a visitor follows a few signposts to the right shelf, one per level of depth.

Where it breaks: a real library is laid out evenly; a tree can grow lopsided, like one long corridor, so its depth can approach n and training can slow to $O(d\,n^2)$.

7.11 Strengths and weaknesses — with the mechanism

Property	Mechanism
No feature scaling needed	splits depend only on the **order** of values; any monotonic transform gives identical splits
Handles mixed types, missing values, nonlinearity	piecewise-constant partitions; default directions for NaNs
Interactions for free	a split on x_2 below a split on x_1 models an $x_1 \times x_2$ interaction
Robust to outliers **in features**	an extreme value just lands in the extreme bin; it doesn't pull a threshold much (but outliers in y still hurt regression trees with MSE)
Interpretable (if shallow)	a path is a readable rule
High variance / unstable	greedy hard splits: a small data change can flip the root split and cascade
Axis-aligned boundaries	a diagonal boundary needs a staircase of many splits
No extrapolation	leaves are constants
Poor at smooth/additive relationships	needs many splits to approximate a straight line

Real-life analogy: an office rulebook of yes/no checks

Think of it like a government office rulebook: "Age above 60? Income below 3 lakh? Then eligible." Whether income is written in rupees or in lakhs does not matter, only which side of the line it falls (no scaling needed). Later questions depend on earlier answers, so combinations come for free. But the rulebook is rigid: rewrite it from a slightly different pile of past cases and it can come out very different, a rule like "income minus debts" needs many steps, and it says nothing sensible about cases beyond what it has seen.

Where it breaks: a rulebook is written with intent; a tree's rules are found greedily from data and can faithfully encode noise.

Q: "Why don't decision trees need feature scaling, but k-NN and SVMs do?"

Strong answer. "A tree split only asks 'is x_j above threshold t?' — it uses the ordering of values within one feature at a time. Rescaling a feature, or any monotonic transform like a log, preserves the order, so the same partitions are available and the tree is identical up to where the thresholds sit. k-NN and SVMs compute distances or dot products that *combine* features, so a feature measured in larger units dominates the geometry. Gradient-based linear models also care, because scaling changes the loss surface's conditioning and how the penalty treats each weight."

Q: "How does a decision tree overfit, and how do you control it?"

Strong answer. "It keeps splitting until leaves are pure, and on continuous features that can mean a leaf per training point — the tree memorizes noise, and because splits are greedy and hard, small data changes produce very different trees: high variance. I control it with pre-pruning — max depth, minimum samples per leaf, max leaves, minimum impurity decrease — or post-pruning with cost-complexity pruning, choosing α by cross-validation. In practice the most effective fix is to stop relying on a single tree: bagging or random forests average away the variance, and boosting uses deliberately shallow trees."

Follow-ups

"Gini or entropy?" — Nearly always the same splits; Gini avoids logs; entropy is slightly more sensitive to changes in small class probabilities. It's rarely the hyperparameter that matters — depth and leaf size are.

"Can a decision tree represent any function?" — Any function on a finite set of points (given enough leaves), yes — which is exactly why an unconstrained tree can memorize the training set.

"How would you get probabilities from a tree?" — Leaf class proportions. They're coarse (few distinct values) and poorly calibrated for deep trees (pure leaves ⇒ 0/1). Smooth them (Laplace smoothing within leaves) or use ensembles and calibrate.

PART 8

Ensemble Methods

Ensembles in one breath

Bagging trains many strong, high-variance models **in parallel** on perturbed data and averages them: variance falls, bias stays. **Boosting** trains weak, high-bias models **in sequence**, each correcting the current ensemble's mistakes: bias falls, variance must be controlled. Random forests are the first idea done well; XGBoost, LightGBM and CatBoost are the second.

The whole Part as one picture

Think of it like two ways of getting a great answer out of ordinary people. **Bagging** is the audience poll on a quiz show: many people answer independently, each having caught a slightly different part of the show, and the most common answer wins, so individual slips cancel out (lower variance). **Boosting** is an editing chain for a news story: each editor reads the current draft and fixes only what is still wrong, with a light touch, and many modest editors together produce a polished piece (lower bias).

Where it breaks: the poll only helps if people don't copy each other (shared mistakes never cancel), and an editing chain that runs forever starts "fixing" things that were never errors, which is boosting overfitting noise.

8.1 Bagging

Bootstrap aggregating (Breiman, 1996). Draw B bootstrap samples (size n, with replacement), train one model on each, average predictions (regression) or vote / average probabilities (classification).

Why it reduces variance — the mechanism. For B models each with variance σ^2 and pairwise correlation ρ (Part 3):

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x)\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2 \qquad \text{while} \qquad \mathbb{E}\left[\frac{1}{B}\sum_{b}\hat{f}_b(x)\right] = \mathbb{E}[\hat{f}_b(x)]$$

Averaging leaves the **expected prediction** (hence the bias) unchanged but shrinks the variance toward $\rho\sigma^2$. So bagging helps exactly when base models are **low-bias, high-variance and not too correlated**: deep, unpruned trees are ideal; bagging linear regression does almost nothing (it's already stable, σ^2 small).

Real-life analogy: guessing a pumpkin's weight at a fair

Think of it like a village fair where hundreds of people guess a giant pumpkin's weight. Each guess is noisy, but the average of all the guesses is remarkably close, because the random over- and under-guesses cancel out. The catch: if everyone read the same wrong newspaper report, that shared error survives any amount of averaging.

Formula, read aloud: $\mathrm{Var}(\text{average}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ says: wobble of the average = (the shared part of each model's wobble, which never cancels) + (the private part, divided by the number of models B).

Where it breaks: fair-goers are truly different people; bootstrap models all come from one dataset, so they are more alike than strangers, which keeps ρ well above zero.

Out-of-bag (OOB) estimation. Each bootstrap sample omits a given point with probability $(1-1/n)^n \approx e^{-1} \approx 36.8\%$. Predict each training point using only the ~37% of models that didn't see it → a nearly free validation estimate (slightly pessimistic, since each OOB prediction uses only about a third of the ensemble).

8.2 Random forest

Algorithm. Bagging of deep decision trees **plus random feature subsampling at every split**: each split considers only a random subset of `max_features` features (common defaults: $\sqrt{d}$ for classification; scikit-learn's regressor defaults to all features, while Breiman suggested $d/3$).

Why feature subsampling matters. With plain bagging, if one feature is very strong, every tree splits on it first and the trees are highly correlated — ρ stays large and averaging stalls at $\rho\sigma^2$. Restricting candidate features forces trees to explore other structure, **lowering** ρ at the cost of a slight increase in each tree's bias and variance. The product usually wins.

Real-life analogy: a panel of doctors with different reports

Think of it like a panel of doctors giving second opinions. If every doctor first looks at the same striking X-ray, they all reach the same conclusion and the panel adds nothing. So at each stage of the diagnosis, each doctor is handed only a random few of the test reports. Individually they are a bit less sharp, but their mistakes now differ, and the majority vote becomes much more reliable.

Where it breaks: a real doctor would ask for the missing reports; a tree simply works with the random subset of features it is handed, and that subset is redrawn at every single split.

Hyperparameter	Effect
`n_estimators`	more trees → lower variance, diminishing returns; **never increases** generalization error in expectation (it converges) — just costs compute
`max_features`	lower → less correlated trees (less variance) but weaker individual trees (more bias); the key knob
`max_depth` / `min_samples_leaf`	usually leave trees deep; increase leaf size for noisy data or smoother probabilities
`bootstrap`, `max_samples`	smaller samples → more diversity, faster training
`class_weight`	handles imbalance (then recalibrate)

Strengths: strong out-of-the-box performance, few sensitive hyperparameters, parallel training, OOB validation, robust to noisy features. **Weaknesses:** large models (hundreds of deep trees → memory and latency), worse than well-tuned boosting on most tabular benchmarks, can't extrapolate, probabilities pulled toward the middle (underconfident at extremes).

"Random forests can't overfit."

Adding **trees** doesn't overfit. But each tree can overfit, noise in the labels is memorized by deep trees, and the forest's training accuracy is near 100% while validation isn't. Leakage and bad splits fool a forest as easily as any model.

8.3 Boosting

Idea. Build an additive model $F_M(x) = \sum_{m=1}^{M} \nu\, h_m(x)$ where each **weak learner** h_m (a stump or shallow tree) is fitted to what F_{m-1} still gets wrong. Each learner is high-bias/low-variance; their **sum** can represent very complex functions.



Real-life analogy: revision sessions before board exams

Think of it like a student preparing for board exams with short revision sessions. After each mock test, the next session covers only the topics still going wrong. Each session is small and simple (a weak learner), but the sum of many targeted sessions covers the whole syllabus.

Formula, read aloud: $F_M(x) = \sum_{m=1}^{M} \nu\, h_m(x)$ says: final prediction = the sum of M small corrections, each scaled down by the learning rate ν.

Where it breaks: a student can go back and rewrite old notes; in boosting, earlier trees are frozen forever, and later trees can only add corrections on top.

✎ AdaBoost

1. Initialize sample weights $w_i = 1/n$.
2. For $t = 1..T$: fit h_t (labels ± 1) minimizing weighted error $\varepsilon_t = \sum_i w_i \mathbb{1}[h_t(x_i) \neq y_i]$.
3. Learner weight $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$ (better learners get more say; $\varepsilon_t = 0.5 \Rightarrow$ zero weight).
4. Reweight $w_i \leftarrow w_i\, e^{-\alpha_t y_i h_t(x_i)}$, normalize: misclassified points gain weight.
5. Output $H(x) = \text{sign}\left(\sum_t \alpha_t h_t(x)\right)$.

AdaBoost is exactly **forward stagewise additive modeling with the exponential loss** $e^{-yF(x)}$ (Friedman, Hastie & Tibshirani). The exponential loss grows exponentially for badly misclassified points ⇒ AdaBoost is **very sensitive to label noise and outliers** — it keeps pouring weight onto mislabeled points.

Real-life analogy: a chain of simple fraud checkers

Think of it like a bank's chain of simple-minded fraud checkers. The first uses one rule ("amount over $1,000?"). The transactions it gets wrong are highlighted more brightly, so the next checker focuses on them with its own simple rule. At the end all checkers vote, and a checker who was more accurate gets a louder vote.

Formula, read aloud: $\alpha_t = \frac{1}{2}\ln\frac{1-\varepsilon_t}{\varepsilon_t}$ says: a checker's say = half the log of (how often it's right ÷ how often it's wrong); a coin-flip checker gets zero say.

Where it breaks: if a label is simply wrong (a genuine purchase marked as fraud), its highlight keeps getting brighter and later checkers twist themselves to fit it, which is AdaBoost's sensitivity to noise.

✎ Gradient boosting

Generalization to any differentiable loss: gradient descent in function space.

Gradient boosting (Friedman, 2001)

$$F_0(x) = \arg\min_{\gamma} \sum_i L(y_i, \gamma)$$

For $m = 1, \ldots, M$:

1. **Pseudo-residuals:** $r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$
2. Fit a regression tree to $\{(x_i, r_{im})\}$, giving leaf regions R_{jm}.
3. **Leaf values** by line search: $\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$.
4. **Update with shrinkage:** $F_m(x) = F_{m-1}(x) + \nu \sum_j \gamma_{jm} \mathbb{1}[x \in R_{jm}]$, $0 < \nu \leq 1$.

Loss	Pseudo-residual $-\partial L/\partial F$
Squared $\frac{1}{2}(y-F)^2$	$y - F$ — literally the residual
Absolute $\lvert y - F\rvert$	$\text{sign}(y - F)$ — robust
Log loss (logit F)	$y - \sigma(F)$
Huber	residual, clipped at δ

Each tree takes a step in the direction (in "prediction space" over the training points) that most decreases the loss — the tree is how we **generalize** that step to unseen x. **Shrinkage** ($\nu \approx 0.01$–0.1) takes small steps — more trees, much better generalization. **Subsampling** rows per tree (stochastic gradient boosting) adds variance reduction and speed.

Real-life analogy: putting on a golf green

Think of it like a round of golf. The first shot (the constant F_0) lands somewhere near the hole. Each later stroke aims at the remaining gap from where the ball now sits (the pseudo-residual), and a careful golfer taps gently (shrinkage ν) rather than smashing past the hole. Other losses simply measure the gap differently; absolute loss, for example, only cares about its direction.

Formula, read aloud: $r_{im} = -\frac{\partial L}{\partial F}$ says: each point's target for the next tree = which way, and how strongly, the loss wants that point's prediction to move; for squared loss it is just (truth – current prediction).

Where it breaks: a golfer putts one ball; here one small tree must nudge thousands of predictions at once, so it only roughly matches the gaps and must also make sense for unseen points.

```python
import numpy as np
from sklearn.tree import DecisionTreeRegressor   # weak learner; tree internals are in Part 21

class GradientBoostingScratch:
    """Gradient boosting (squared or log loss) with shallow regression trees."""

    def __init__(self, n_estimators=200, learning_rate=0.1, max_depth=3, loss="squared",
                 subsample=1.0, seed=0):
        self.M, self.nu, self.depth, self.loss = n_estimators, learning_rate, max_depth, loss
        self.subsample, self.rng = subsample, np.random.default_rng(seed)

    def _neg_gradient(self, y, F):
        if self.loss == "squared":                # L = 1/2 (y-F)^2   →  y - F
            return y - F
        return y - 1 / (1 + np.exp(-F))           # log loss on logits →  y - p

    def fit(self, X, y):
        n = len(y)
        if self.loss == "squared":
            self.F0_ = y.mean()                   # constant minimizing squared loss
        else:
            p = np.clip(y.mean(), 1e-6, 1 - 1e-6)
            self.F0_ = np.log(p / (1 - p))        # log-odds of the base rate
        F = np.full(n, self.F0_)
        self.trees_ = []
        for _ in range(self.M):
            r = self._neg_gradient(y, F)          # pseudo-residuals
            idx = self.rng.choice(n, int(self.subsample * n), replace=False)
            tree = DecisionTreeRegressor(max_depth=self.depth).fit(X[idx], r[idx])
            if self.loss == "log":                # Newton step per leaf: sum(r) / sum(p(1-p))
                leaf, p = tree.apply(X), 1 / (1 + np.exp(-F))
                for l in np.unique(leaf[idx]):
                    m = leaf == l
                    tree.tree_.value[l, 0, 0] = r[m].sum() / (np.sum(p[m] * (1 - p[m])) + 1e-12)
            F += self.nu * tree.predict(X)        # shrinkage
            self.trees_.append(tree)
        return self

    def decision_function(self, X):
        return self.F0_ + self.nu * sum(t.predict(X) for t in self.trees_)
```

Verified: on Friedman-1 regression and a 20-feature classification task, test MSE and accuracy match scikit-learn's `GradientBoosting*` with the same settings (1.646 vs 1.643 MSE; 0.947 vs 0.947 accuracy).

XGBoost

XGBoost ("eXtreme Gradient Boosting", Chen & Guestrin, 2016) = gradient boosting + **second-order** optimization + explicit **regularization** + systems engineering.

The XGBoost objective and split gain

At round t, with $g_i = \partial_F L$, $h_i = \partial_F^2 L$ at the current prediction, a second-order Taylor expansion gives, for a tree with T leaves and leaf weights w_j:

$$\mathcal{L}^{(t)} \approx \sum_{j=1}^{T} \left[G_j w_j + \tfrac{1}{2}(H_j + \lambda) w_j^2 \right] + \gamma T, \qquad G_j = \sum_{i \in j} g_i,\ H_j = \sum_{i \in j} h_i$$

Minimizing each leaf's quadratic:

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \qquad \text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

Split only if Gain > 0: γ is a **minimum gain to split** (built-in pruning); λ shrinks leaf weights (L2); `min_child_weight` requires H_j above a threshold (for log loss, $h = p(1-p)$, so it limits leaves with few or already-confident samples).



Real-life analogy: turning a shower knob

Think of it like adjusting a shower's temperature. Knowing only that the water is too cold (the gradient) tells you which way to turn, not how far. If you also know how sensitive the knob is (the curvature, or Hessian), you can turn it about the right amount in one go. λ is a stiff knob that resists wild turns, and γ is a fee: a new split is allowed only if it improves things by more than the fee.

Formula, read aloud: $w_j^* = -\frac{G_j}{H_j+\lambda}$ says: a leaf's best value = −(total slope of its points) ÷ (total curvature + the stiffness cushion λ).

Where it breaks: the curvature is only accurate close to where you stand, so each tree's step is still shrunk by the learning rate rather than trusted fully.

Other XGBoost ingredients: sparsity-aware split finding with a **learned default direction** for missing values; weighted quantile sketch for approximate splits; column and row subsampling; histogram-based split finding (`tree_method="hist"` is the default behavior in XGBoost 2.x and later); depth-wise tree growth by default (`grow_policy="depthwise"`, with a leaf-wise `lossguide` option); native categorical support (`enable_categorical`); GPU training (`device="cuda"`).

LightGBM

Microsoft's GBDT (Ke et al., 2017), built for speed and scale:

- **Histogram-based** split finding (bins, ≤ 255 by default) and histogram subtraction (a child's histogram = parent − sibling).
- **Leaf-wise (best-first) growth:** always split the leaf with the largest gain, regardless of depth. Reaches lower loss with the same number of leaves than level-wise growth — but can grow deep, lopsided trees that **overfit small data**; control with `num_leaves` (default 31), `min_data_in_leaf`, `max_depth`.
- **GOSS (Gradient-based One-Side Sampling):** keep all instances with large gradients (under-trained), randomly sample the small-gradient ones and up-weight them — less data per tree, little accuracy loss.
- **EFB (Exclusive Feature Bundling):** bundle sparse features that are rarely nonzero together into one feature — big win on one-hot/sparse data.
- Native categorical splits (the sort-by-gradient-statistics trick).

Real-life analogy: a delivery company's speed tricks

Think of it like a delivery company planning routes faster. *Histograms*: group addresses by PIN code instead of exact coordinates. *Leaf-wise growth*: always send the next rider to whichever neighbourhood has the most late orders, instead of covering every area evenly. *GOSS*: review every late order, but only a random sample of on-time ones, each counted extra to make up for those skipped. *EFB*: two routes that never run on the same day share one van.

Where it breaks: chasing the busiest neighbourhood works well with lots of orders, but on little data it grows deep, lopsided trees that memorize noise; cap `num_leaves` and `min_data_in_leaf`.

CatBoost

Yandex's GBDT (Prokhorenkova et al., 2018), built around categorical features and bias-free training:

- **Ordered target statistics:** encodes a categorical value using the mean target of **earlier** examples in a random permutation (plus a prior), so an example's own label never leaks into its encoding — principled target encoding.
- **Ordered boosting:** computes each example's gradient with a model that hasn't been trained on that example, fixing the **prediction shift** that plain gradient boosting suffers.
- **Symmetric (oblivious) trees:** the same split condition is used across a whole level. Strongly regularizing, and inference becomes a fast bit-index lookup.
- Strong defaults: often competitive with little tuning, especially with many categorical features.

Real-life analogy: rating a restaurant without your own review

Think of it like a food app predicting whether *you* will like a restaurant. If it uses the restaurant's average rating including your own review, it is peeking at the answer. CatBoost lines diners up in a random order and scores each one using only the reviews written *before* theirs. *Ordered boosting* applies the same "no peeking at yourself" rule when computing corrections. *Symmetric trees* are a fixed questionnaire: every diner at the same step is asked the same question, so lookup is very fast.

Where it breaks: diners at the front of the line have few earlier reviews, so their scores are noisy; CatBoost softens this with a prior and several random orders.

8.4 Comparisons

	Bagging / Random forest	Boosting
Training	parallel, independent models	sequential, each depends on the previous
Base learner	strong, low-bias, high-variance (deep trees)	weak, high-bias, low-variance (stumps, depth 3–8)
Main effect	↓ **variance**	↓ **bias** (then control variance)
More models	never hurts accuracy (converges)	eventually overfits → early stopping
Noise sensitivity	robust	sensitive (AdaBoost especially; GBM with robust losses less so)
Tuning	forgiving	more sensitive (learning rate, depth/leaves, rounds, regularization)
Typical accuracy on tabular data	strong baseline	usually best

	Random forest	Gradient boosting (XGBoost/LightGBM/CatBoost)
When to prefer	quick robust baseline; noisy labels; little tuning time; want OOB estimates	maximum accuracy on structured data; custom losses; ranking (LambdaMART); large data
Overfitting control	leaf size, max_features	learning rate + early stopping, depth/leaves, min_child_weight, subsampling, L1/L2 on leaves
Inference	many deep trees → heavier	fewer, shallower trees → often faster (CatBoost's oblivious trees fastest)
Calibration	underconfident at extremes	usually decent with log loss; recalibrate if class-weighted

	XGBoost	LightGBM
Tree growth	depth-wise by default	leaf-wise (best-first)
Speed / memory	fast (hist)	usually faster, lower memory on large/sparse data (GOSS, EFB)
Small data	more conservative default growth	leaf-wise overfits more easily — tune `num_leaves`, `min_data_in_leaf`
Categoricals	native (hist) or encode	native
Pick when	robust default, mature ecosystem, strong GPU support	very large datasets, many features, speed-critical iteration

8.5 Why boosting reduces bias; why random forests reduce variance

Why does boosting reduce bias?

A single stump is badly biased — it can only express one threshold. Boosting adds functions **where the current model is wrong**: each round fits the negative gradient of the loss, i.e. the direction the predictions should move to reduce error on the training points. That's gradient descent in function space, and the model class grows with every round (a sum of M trees is far richer than one tree; depth-k trees contribute interactions up to order k). Training error falls steadily — for AdaBoost it provably falls exponentially as long as each weak learner beats chance: $\text{err} \le \exp\left(-2\sum_t \gamma_t^2\right)$ with $\gamma_t = \frac{1}{2} - \varepsilon_t$. The price: each round also fits some noise, so variance grows with M — hence small learning rates, subsampling, and early stopping on validation loss.

Why does a random forest reduce variance?

Deep trees have low bias but high variance: their predictions depend heavily on the particular sample. Averaging B of them gives variance $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ while leaving the expected prediction — the bias — essentially unchanged. Bootstrapping creates different training sets and random feature subsets at each split **decorrelate** the trees (reduce ρ), so the average keeps improving as trees are added, down to the floor $\rho\sigma^2$. It can't fix bias: if every tree systematically misses a pattern (like extrapolation), so does the forest.



Real-life analogy: two ways to fix an archer's aim

Think of it like two ways to improve archery. *Random forest*: many archers with shaky hands but correct aim; the average of their arrows lands near the bullseye because the shakes cancel (lower variance), but if all aim at the wrong spot, averaging cannot help (bias stays). *Boosting*: one archer who adjusts the sight after every shot to remove a steady drift (lower bias); react to every gust of wind, though, and the aim turns jumpy (variance grows), so adjust gently and stop in time.

Formula, read aloud: $\text{err} \le \exp\left(-2\sum_t \gamma_t^2\right)$ says: AdaBoost's training error shrinks by a constant factor with every learner that beats a coin flip by an edge γ_t.

Where it breaks: the forest's archers learn from overlapping data, so their shakes are partly shared and averaging hits the floor $\rho\sigma^2$.

8.6 Stacking and blending

Stacking: train diverse base models; use their **out-of-fold** predictions as features for a meta-model (often a regularized logistic/linear regression). Using in-sample predictions would leak — the meta-model would trust overfit base models. **Blending:** the same with a single holdout split. Gains are usually small but real in competitions; in production, weigh them against the cost of maintaining several models.



Real-life analogy: a manager weighing interviewers

Think of it like a hiring manager combining verdicts from several interviewers. She learns how much to trust each interviewer from their track record on candidates they had *not* already coached. If she judged them on candidates they had coached, every interviewer would look brilliant and she would over-trust them; that is exactly the leak that out-of-fold predictions prevent. Blending is the same idea, judged on one separate batch of candidates.

Where it breaks: a manager can ask follow-up questions; the meta-model sees only the base models' scores, and the gain is usually small next to the cost of maintaining several models.

8.7 Practical tuning recipe for gradient boosting

1. Fix a moderate learning rate (0.05–0.1) and use **early stopping** on a validation set to choose the number of trees.
2. Tune tree complexity: `max_depth` (3–10) or `num_leaves` (15–255), `min_child_weight` / `min_data_in_leaf`.
3. Add randomness: `subsample` 0.6–0.9, `colsample_bytree` 0.5–0.9.
4. Regularize: `reg_lambda`, `reg_alpha`, `gamma` / `min_split_gain`.
5. Lower the learning rate (and let early stopping add trees) for the final model.
6. Use a **validation split that mirrors production** (time-based if temporal) — the tuned model is only as good as the split.

Real-life analogy: cooking biryani on dum

Think of it like cooking biryani. Start on a moderate flame (learning rate), taste regularly, and switch off the moment it's done rather than after a fixed time (early stopping). Then adjust the pot size and layering (tree depth and leaves), mix in a little variety so no spoonful dominates (row and column subsampling), and go easy on the spices (regularization). For the final batch, use a lower flame for longer. Above all, taste it the way guests will eat it (a validation split that mirrors production).

Where it breaks: a biryani is cooked once, while a model can be rerun cheaply; and these knobs interact, since a lower learning rate needs more trees.

Q: "Random forest or gradient boosting for a new tabular problem?"

Strong answer. "I'd start with a gradient-boosted model — LightGBM or XGBoost — with early stopping, because on tabular data it's usually the most accurate and handles missing values and mixed types. A random forest is a great quick baseline and more forgiving with noisy labels and little tuning time. The decision factors: data size and noise level, tuning budget, latency — boosting's shallower trees are often cheaper to serve than a big forest — and whether I need calibrated probabilities. Either way, the validation scheme matters more than the choice between them."

Follow-ups

"Why do boosted trees use shallow trees but random forests use deep trees?" — *Boosting needs low-variance weak learners and reduces bias by adding them; depth controls the order of interactions each adds. Forests need low-bias learners and reduce variance by averaging — shallow trees would leave a bias that averaging can't fix.*

"Your GBM's validation loss starts rising after 300 trees. What do you do?" — *Early-stop at the best iteration; then try a lower learning rate (more trees, smoother), more regularization (min_child_weight, lambda), subsampling, or shallower trees. Check the rise isn't caused by train/validation mismatch.*

"Can boosting overfit with a tiny learning rate?" — *Yes, eventually; shrinkage slows overfitting and improves the achievable validation loss, but with enough rounds the training loss keeps falling and the model fits noise.*

8.8 Monotonic and interaction constraints

Gradient-boosted trees can be told what shape a relationship must have:

- **Monotonic constraints:** "predicted default risk must never *decrease* as the number of missed payments increases". The library only accepts splits whose leaf values respect the order (XGBoost `monotone_constraints`, LightGBM `monotone_constraints`, CatBoost `monotone_constraints`).
- **Interaction constraints:** only allow listed groups of features to appear together in a tree (e.g. keep "region" from interacting with "age" if that interaction would be hard to justify).

Why use them: regulators and users expect sensible directions (a higher income shouldn't raise predicted risk); constraints stop the model from fitting noisy wiggles in sparse regions; they often **generalize better** under shift because implausible patterns can't be learned; and they make explanations consistent. **Cost:** a small loss of training fit, and the risk of forcing a constraint that isn't true (e.g. risk vs. age is often U-shaped — don't force monotonicity there).

Real-life analogy: speed limits on a highway

Think of it like telling a new driver: "on this road, never go *slower* when the road gets straighter". Without the rule, a nervous driver might brake randomly on straight stretches because of one bad experience (noise). The rule costs a bit of freedom but makes the driving predictable and safe. Monotonic constraints are such rules for a model's response to a feature.

Where it breaks: traffic rules are designed by experts; a wrong constraint (forcing "more is always riskier" when the truth is U-shaped) makes the model systematically wrong where the rule doesn't hold.

8.9 Explaining tree ensembles with SHAP

Global importances (Part 7) say *which* features matter overall; **SHAP values** say how much each feature pushed **one specific prediction** up or down from the average prediction, with the guarantee that the pushes add up exactly: $f(\mathbf{x}) = \mathbb{E}[f] + \sum_j \phi_j(\mathbf{x})$. For tree ensembles, **TreeSHAP** computes exact Shapley values in time polynomial in tree depth (instead of exponential in the number of features), which is why "XGBoost + SHAP" became the standard explainability pairing for tabular models. Part 48 has the Shapley formula and its assumptions.

What to show and how to read it:

- **Waterfall/force plot** for one prediction: base value → each feature's push → final score. Use it to explain an individual decision (a declined loan).
- **Summary (beeswarm) plot:** each dot is one example's SHAP value for a feature, colored by the feature's value — shows direction and spread of effects across the dataset.
- **Dependence plot:** SHAP value vs. feature value — a model-based "effect curve", which also reveals interactions.

Pitfalls: with correlated features, credit is split between them in ways that depend on the SHAP variant ("interventional" vs. "conditional"); SHAP explains the **model**, not causation (Part 48); and SHAP values on the log-odds scale must be converted before quoting them as probability changes.

Real-life analogy: splitting a restaurant bill fairly

Think of it like four friends sharing a restaurant bill where some dishes were shared. SHAP splits the final prediction the way a very fair friend would split the bill: consider every order in which people could have joined the table, see how much each person's arrival added to the bill, and average. Everyone's share adds up exactly to the total.

Formula, read aloud: $f(\mathbf{x}) = \mathbb{E}[f] + \sum_j \phi_j$: this prediction = the average prediction + the push from each feature.

Where it breaks: splitting a bill says who ordered what, not why they were hungry — SHAP says how the model used a feature, not whether changing that feature in the real world would change the outcome.

Active recall — Set 6: Trees and ensembles

Set 6 · Answer from memory, then check the Answer Key

Easy recall

1. Write the formulas for entropy and Gini impurity.
2. What is information gain?
3. What fraction of the data is out-of-bag for one bootstrap sample?
4. What does `max_features` do in a random forest?
5. Write the XGBoost optimal leaf weight.

Conceptual

6. Why is greedy tree construction myopic? Give an example.
7. What is cost-complexity pruning?
8. How does AdaBoost reweight samples, and what loss is it minimizing?
9. What is a pseudo-residual in gradient boosting?
10. What are GOSS and EFB in LightGBM?

Why

11. Why don't trees need feature scaling?
12. Why does misclassification error make a poor splitting criterion?
13. Why does feature subsampling improve random forests?
14. Why is impurity-based feature importance biased?
15. Why does CatBoost use ordered target statistics?

Scenario

16. A tree-based demand model never predicts above last year's maximum sales, but demand is growing. Why and what do you do?
17. Your GBM performs great on random CV but poorly next month. What's likely wrong?
18. A user_id feature is the most important feature in your random forest. What do you do?
19. LightGBM overfits badly on a 5,000-row dataset. Which knobs do you turn?
20. You need a model with < 1 ms inference on CPU and strong tabular accuracy. What do you consider?

Mathematical

21. Compute the information gain for a parent (6+, 6−) split into (5+, 1−) and (1+, 5−).
22. Derive the variance of the average of B correlated models.
23. For log loss, what are g_i and h_i in XGBoost?

Difficult follow-ups

24. Why does adding more trees never hurt a random forest's expected performance but can hurt boosting?
25. Explain why leaf-wise growth reaches a lower training loss than level-wise growth for the same number of leaves, and what that implies.

2027 additions

26. Ten applicants, 4 defaults. Income splits them 5 (1 default) / 5 (3 defaults); a credit-card flag splits them 7 (2) / 3 (2). Which split has the larger Gini decrease?
27. Why and when would you add monotonic constraints to a GBDT?
28. What do SHAP values guarantee, and name two pitfalls when reading them.

PART 9

Support Vector Machines

SVM in one breath

Among all hyperplanes that separate the classes, pick the one with the **widest margin** — the most room for error. Only the points on or inside the margin (the support vectors) determine it. Because the optimization only needs dot products, you can swap in a **kernel** and get nonlinear boundaries without ever computing the high-dimensional features.

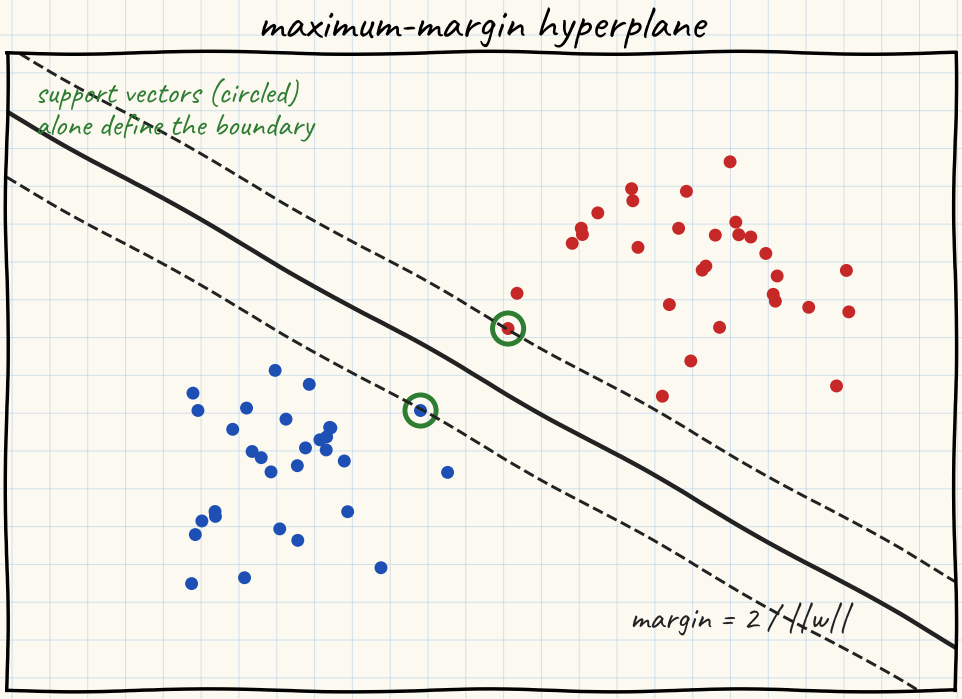
The whole Part as one picture

Think of it like a surveyor drawing the border between two villages' farms. Many straight lines would separate them, but he picks the one leaving the **widest empty strip**, so small shifts next season won't cross it. Only the farms touching the strip's edges (the **support vectors**) decide where it goes; farms further back could vanish and nothing would change. When one village's farms ring the other's, no straight line works, so he imagines raising each farm by its distance from the centre: a flat sheet now slides between the low inner ring and the high outer ring. That trick is the **kernel**.

Where it breaks: the kernel never builds the raised landscape; it only computes how similar two points would look there, which is far cheaper.

9.1 The maximum-margin classifier (geometry first)

A hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ with labels $y_i \in \{-1, +1\}$. The signed distance of $\mathbf{x}_i$ from the plane is $\frac{y_i(\mathbf{w}^\top \mathbf{x}_i + b)}{\|\mathbf{w}\|}$ (the **geometric margin**). The pair $(\mathbf{w}, b)$ can be rescaled freely without moving the plane, so fix the scale by requiring the closest points to satisfy $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$. Then the margin — distance between the two parallel planes $\mathbf{w}^\top \mathbf{x} + b = \pm 1$ — is $\frac{2}{\|\mathbf{w}\|}$.



The maximum-margin hyperplane; circled points are the support vectors

Why a wide margin? Intuition: a boundary far from all training points is robust to small perturbations of the data and noise in new points. Theory: margin-based generalization bounds depend on R^2/γ^2 (data radius over margin squared), **not on the dimension** — the reason SVMs work in very high-dimensional (even infinite-dimensional kernel) spaces.

Real-life analogy: the widest walkway between fans

Think of it like stadium staff setting up a walkway between two rival groups of cricket fans. Any walkway that keeps them apart "works", but the safest one is as **wide as possible**, so a fan who leans or shuffles a little still stays on their own side. The fans in the front rows, touching the walkway's edges, are the **support vectors**; the back rows don't affect where the walkway goes.

Formula, read aloud: the margin $\frac{2}{\|\mathbf{w}\|}$ says: walkway width = 2 ÷ (length of the weight vector), so shorter weights mean a wider walkway.

Where it breaks: a walkway lives on flat ground; here it is a slab between two parallel hyperplanes in possibly thousands of dimensions, and its width is measured after scaling $\mathbf{w}$ so the front-row points score exactly ± 1.

9.2 Hard margin

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \;\; \forall i$$

Real-life analogy: a zero-tolerance chalk lane

Think of it like a PE teacher on sports day chalking a lane between Red House and Blue House, with a zero-tolerance rule: no child may sit on the lane or on the wrong side. She makes the lane as wide as that rule allows. If one Red child has wandered into the Blue crowd (a mislabeled point), the rule either squeezes the lane to a thread or can't be met at all.

Formula, read aloud: $\min \frac{1}{2}\|\mathbf{w}\|^2$ subject to $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ says: make the weights as small as possible (the lane as wide as possible) while every point sits on its own side, at least half a lane-width from the centre line.

Where it breaks: a teacher could ask the stray child to move; the SVM must accept every label as given, which is why the soft margin exists.

Maximizing $2/\|\mathbf{w}\| \Leftrightarrow$ minimizing $\|\mathbf{w}\|^2$ (a convex quadratic program with linear constraints — unique solution). Only feasible if the data is **linearly separable**, and a single mislabeled point can force a terrible, narrow margin or make the problem infeasible.

9.3 Soft margin, slack and the C parameter

Allow violations with slack variables $\xi_i \geq 0$:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \;\; \xi_i \geq 0$$

Real-life analogy: a lenient vs. strict traffic cop

Think of it like a traffic cop deciding how many rule-breakers to tolerate while keeping a wide, clear lane. A lenient cop (small C) keeps the lane wide and lets a few cars cross the line with small fines; a strict cop (large C) fines every violation heavily, so the lane gets narrow and twisted to keep everyone perfectly on their side.

Formula, read aloud: minimize (½ × how narrow the lane is, $\|\mathbf{w}\|^2$) + C × (total amount by which points cross into or over the lane, $\sum \xi_i$).

Where it breaks: a cop punishes after the fact; the SVM chooses the lane and the tolerated violations together, in one optimization.

$\xi_i = 0$: correctly outside the margin; $0 < \xi_i < 1$: inside the margin but correctly classified; $\xi_i > 1$: misclassified.

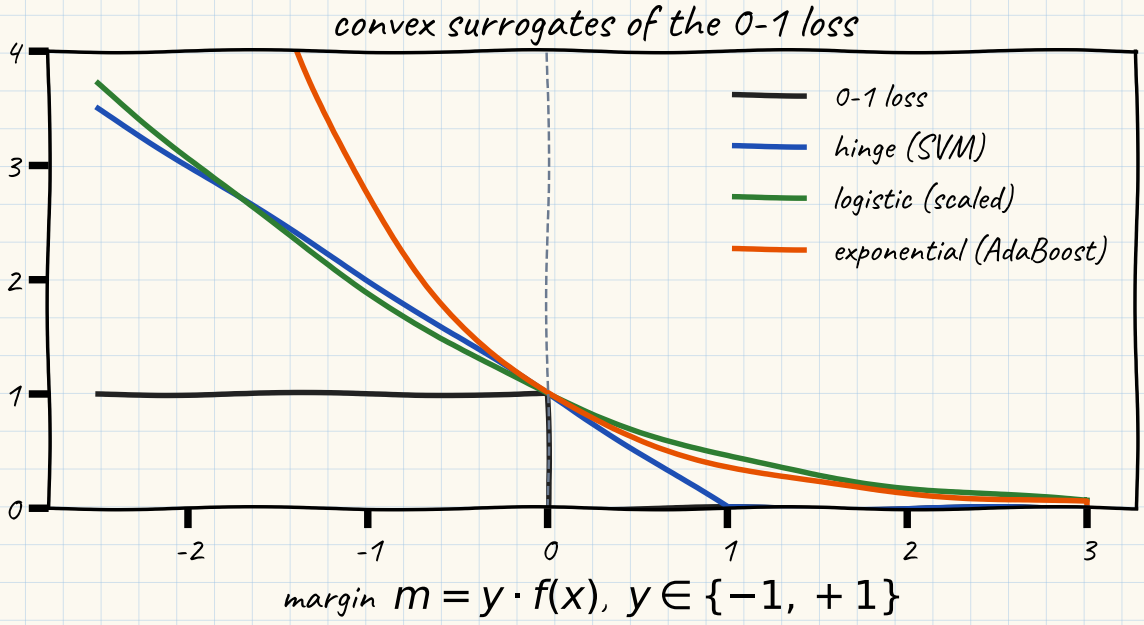
C trades margin width against violations. Large C ⇒ violations are expensive ⇒ narrow margin that bends to fit every point (low bias, high variance, approaching hard margin). Small C ⇒ tolerate violations for a wider, simpler margin (more bias, less variance). C plays the role of $1/\lambda$.

9.4 Hinge loss — the unconstrained view

At the optimum $\xi_i = \max(0, 1 - y_i f(\mathbf{x}_i))$, so the soft-margin SVM is exactly **regularized ERM with the hinge loss**:

$$\min_{\mathbf{w},b} \underbrace{\sum_i \max\left(0,\; 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right)}_{\text{hinge loss}} + \frac{1}{2C}\|\mathbf{w}\|^2$$

The hinge loss is **zero** for points beyond the margin (margin ≥ 1): they exert no force on the solution. Compare logistic loss $\log(1+e^{-m})$, which is never exactly zero — every point contributes a little. That's why the SVM solution is **sparse in the data** (depends only on support vectors) while logistic regression isn't — and why SVM scores aren't probabilities.



Hinge, logistic and exponential losses as convex surrogates of the 0-1 loss

Real-life analogy: a teacher who only marks the doubtful answers

Think of it like a teacher who ignores students who clearly passed with a safe margin and spends all her attention on those near or below the pass mark. Students far above the line teach her nothing new. That's the hinge loss: zero penalty for confidently correct points, a growing penalty for points near or on the wrong side.

Formula, read aloud: $\max(0, 1 - y\,f(\mathbf{x}))$ says: if the point is correct *by a margin of at least 1*, cost is zero; otherwise, pay the amount by which it falls short.

Where it breaks: a teacher can still learn from top students; the SVM literally ignores them — which makes it sparse and fast, but also blind to information in easy examples (no calibrated probabilities).

9.5 Dual formulation (intuition you need)

Using Lagrange multipliers $\alpha_i \geq 0$ for the margin constraints, the dual is

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j\, \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0$$

with $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ and predictions $f(\mathbf{x}) = \sum_i \alpha_i y_i\, \mathbf{x}_i^\top \mathbf{x} + b$.

Real-life analogy: a tent held up by a few poles

Think of it like pitching a tent: the fabric's final shape is fixed only by the few poles and pegs that actually touch it — the other rods lying in your bag don't matter. In the dual, each training point gets a weight α_i; most end up at zero, and only the **support vectors** (the poles) hold the boundary in place.

Formula, read aloud: the prediction $f(\mathbf{x}) = \sum_i \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b$ says: compare the new point with each support vector (dot product), weight by how important that vector is and which side it's on, and add up.

Where it breaks: tent poles are chosen by you; support vectors are discovered by the optimization — and with noisy, overlapping classes, many points become "poles", making the model slow.

What to take away:

1. **Support vectors are the points with** $\alpha_i > 0$ (complementary slackness: $\alpha_i > 0$ only if the constraint is tight or violated). $0 < \alpha_i < C$: exactly on the margin; $\alpha_i = C$: inside it or misclassified. Everyone else has $\alpha_i = 0$ and could be deleted without changing the solution.
2. **The data appears only through dot products** $\mathbf{x}_i^\top \mathbf{x}_j$. That's the door to the kernel trick.
3. The dual has n variables (vs. $d+1$ in the primal) — preferable when d is huge or infinite, worse when n is huge.

9.6 The kernel trick

Map inputs to a feature space $\phi(\mathbf{x})$ where a linear separator works, but never compute ϕ: replace every dot product with a **kernel** $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$.

$$f(\mathbf{x}) = \sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

Real-life analogy: lifting a tangled rope into the air

Think of it like red and blue beads mixed on a table, with the red ones in a ring around the blue ones — no straight stick can separate them. Now lift the centre of the table cloth into a tent: the blue beads rise up, the red stay low, and a flat sheet slid in between separates them perfectly. A kernel measures similarity *as if* you had lifted the data into that higher space, without ever doing the lifting.

Formula, read aloud: $f(\mathbf{x}) = \sum \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$ says: score a new point by how similar it is (kernel K) to each support vector, weighted and signed.

Where it breaks: the "lifted space" can be infinite-dimensional (RBF kernel) — no picture captures that; and computing K for every pair of n points costs $O(n^2)$, which is why kernel SVMs struggle on big data.

Example — the kernel computes a hidden feature map

$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2$ in 2-D: $(x_1 z_1 + x_2 z_2)^2 = x_1^2 z_1^2 + 2 x_1 x_2 z_1 z_2 + x_2^2 z_2^2 = \phi(\mathbf{x})^\top \phi(\mathbf{z})$ with $\phi(\mathbf{x}) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2)$. A circle $x_1^2 + x_2^2 = r^2$ in input space is a **plane** in ϕ-space. The kernel evaluated it with one 2-D dot product and a square.

Kernel	Formula	Feature space / behavior
Linear	$\mathbf{x}^\top \mathbf{x}'$	original space; best for high-dim sparse text
Polynomial	$(\gamma\, \mathbf{x}^\top \mathbf{x}' + r)^p$	all monomials up to degree p — interactions
RBF (Gaussian)	$\exp(-\gamma \lVert \mathbf{x} - \mathbf{x}' \rVert^2)$	infinite-dimensional; similarity decays with distance
Sigmoid	$\tanh(\gamma\, \mathbf{x}^\top \mathbf{x}' + r)$	not always PSD; rarely used

RBF intuition. $f(\mathbf{x})$ is a weighted sum of Gaussian bumps centered on the support vectors. γ sets the bump width ($\gamma = 1/(2\sigma^2)$): **large** γ ⇒ narrow bumps ⇒ wiggly boundary that can wrap around individual points (overfitting; as $\gamma \to \infty$ it behaves like a lookup table); **small** γ ⇒ wide bumps ⇒ nearly linear boundary (underfitting). Tune C and γ **jointly** on a log grid — they interact.

Validity (Mercer): a function is a valid kernel iff every Gram matrix it produces is positive semidefinite. Sums, products and positive scalings of kernels are kernels — you can build kernels for strings, graphs and sets.

9.7 When SVMs perform well — and why they've faded for large-scale work

Where SVMs shine: small-to-medium datasets (up to roughly 10^4–10^5 examples for kernel SVMs); high-dimensional problems with a clear margin; **sparse text classification** with a linear SVM (strong, fast, a classic baseline); problems where a good kernel encodes domain knowledge (string kernels in bioinformatics); when you want a convex problem with a unique solution.



Real-life analogy: a master craftsman vs. a factory

Think of it like a master carpenter: superb for a few custom pieces (small/medium data, clear margins, good kernels), but you wouldn't ask him to make a million chairs — a factory (gradient-boosted trees, neural networks with SGD) scales better and is easier to retool.

Where it breaks: linear SVMs on sparse text *do* scale well — the "slow craftsman" label mostly applies to kernel SVMs.

Why they became less common at scale:

Reason	Mechanism
Training cost	kernel SVM solvers scale roughly between $O(n^2)$ and $O(n^3)$ and need the $n \times n$ kernel matrix (or caching) — infeasible at millions of examples
Inference cost	prediction sums over all support vectors; on noisy data, a large fraction of points become support vectors
Fixed representations	the kernel is chosen, not learned; deep nets **learn** the feature map, and win on images, text, audio
Tabular competition	gradient-boosted trees handle mixed types, missing values and scale better
No native probabilities	needs Platt scaling (extra CV); scores are margins
Sensitivity	requires feature scaling; C and γ tuning is essential; multiclass needs one-vs-rest or one-vs-one

Still alive: linear SVMs (hinge loss via SGD/liblinear) for sparse text; the hinge/margin idea in ranking losses, contrastive/triplet losses (Part 15) and max-margin structured prediction; and random Fourier features approximate RBF kernels at scale.

Q: "Explain SVMs geometrically in two minutes."

Strong answer. "An SVM finds the separating hyperplane with the largest margin — the widest street between the classes. If I scale $\mathbf{w}$ so the closest points satisfy $y(\mathbf{w}^\top \mathbf{x} + b) = 1$, the street's width is $2/\|\mathbf{w}\|$, so maximizing it means minimizing $\|\mathbf{w}\|^2$ subject to every point being on the right side. Only the points touching or inside the street — the support vectors — determine the solution. For non-separable data, slack variables let points violate the margin at a cost C per unit, which is the same as hinge loss plus L2 regularization. And because the dual only uses dot products between points, a kernel like RBF lets the street be straight in a high-dimensional feature space while curved in the input space."

Follow-ups

"What happens to the SVM if you add more points far from the boundary on the correct side?" — Nothing: they have zero hinge loss and $\alpha_i = 0$. Logistic regression would shift slightly.

"Why does the RBF SVM overfit with a large γ?" — Each training point's influence becomes very local; the decision function can place a tiny bump around every positive point — like 1-NN.

"SVM vs. logistic regression?" — Both are linear classifiers with convex surrogate losses. Logistic gives probabilities and uses all points; SVM gives a sparse, margin-based solution, and with kernels more flexible boundaries. On high-dim sparse text they perform similarly; choose logistic when you need calibrated probabilities.

9.8 Support vector regression and one-class SVM

Support vector regression (SVR) turns the margin idea around: fit a function that stays within a tube of half-width ε around the targets, ignoring errors inside the tube and paying linearly outside it:

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \max\left(0,\ |y_i - (\mathbf{w}^\top \mathbf{x}_i + b)| - \varepsilon\right)$$

The **ε-insensitive loss** makes the solution sparse (only points on or outside the tube become support vectors) and robust to small noise; with kernels it fits smooth nonlinear curves on small datasets. Tune ε to the noise level you're willing to ignore and C for how hard to push outliers back.

One-class SVM learns a boundary around "normal" data with no labels for anomalies: in kernel space, separate the data from the origin with maximum margin; the parameter $\nu \in (0, 1]$ upper-bounds the fraction of training points treated as outliers. Uses: novelty detection on small, clean "normal" datasets (machine sensor readings). At scale or with contaminated data, isolation forests or density/reconstruction-based methods are usually more practical.



Real-life analogy: a tolerance band on a factory line

Think of it like a quality inspector with a tolerance band: bolts within ±0.1 mm of spec are "fine" and ignored; only bolts outside the band cost something, in proportion to how far out they are. SVR draws the smoothest line whose band holds most points. A **one-class SVM** is the inspector who has only ever seen good bolts and draws a fence around them — anything outside the fence is flagged as unusual.

Formula, read aloud: penalty $= \max(0, |\text{error}| - \varepsilon)$: "errors smaller than ε are free; beyond that, pay per unit".

Where it breaks: real tolerance bands come from engineering specs; ε and C must be tuned, and kernel SVR scales poorly beyond tens of thousands of points.

PART 10

K-Nearest Neighbors

KNN in one breath

Don't learn a function — **remember the data**, and predict by majority vote (or average) of the k closest training points. All the modeling decisions hide in two places: the **distance metric** and k. It's the purest form of the "similar inputs have similar outputs" inductive bias — and the one most damaged by high dimensions.

The whole Part as one picture

Think of it like asking the neighbours. You've just moved into a new colony and want to know if the local milkman is reliable. You don't build a theory — you ask the five nearest neighbours and go with the majority. KNN does exactly that for every prediction: find the k most similar past examples and let them vote. All the skill lies in deciding who counts as a "neighbour" (the distance metric, feature scaling) and how many to ask (k).

Where it breaks: real neighbours are few and nearby; in high dimensions, "nearest" points can be almost as far as the farthest ones, and asking them becomes nearly random (the curse of dimensionality).

10.1 Algorithm

```
predict(x):
    compute distance d(x, x_i) for every training point      # O(n d)
    take the k smallest                                      # O(n) with argpartition
    classification: majority vote (optionally weighted by 1/d)
    regression: mean (or distance-weighted mean) of their y
```

Real-life analogy: asking the regulars at a tea stall

Think of it like a newcomer at a chai stall asking "which bus goes to the station?" to the closest few regulars and going with the most common answer. There's no training — you just keep everyone's knowledge around and ask at question time. That's why KNN is "lazy": all the work happens when a prediction is needed.

Where it breaks: asking five people is quick; searching millions of stored examples for every question is slow, which is why real systems use indexes and approximate search.

Non-parametric and lazy: no training beyond storing (and maybe indexing) the data; model size grows with n. Asymptotically, the 1-NN error rate is at most **twice the Bayes error** (Cover & Hart, 1967) — a remarkable guarantee for such a simple rule, which holds only as $n \to \infty$ in a fixed dimension.

10.2 Distance metrics

Metric	Formula	Use
Euclidean (L_2)	$\sqrt{\sum(x_j - z_j)^2}$	default for continuous, scaled features
Manhattan (L_1)	$\sum\|x_j - z_j\|$	more robust to single large coordinate differences; grid-like spaces
Minkowski	$(\sum\|x_j - z_j\|^p)^{1/p}$	generalizes both
Cosine distance	$1 - \frac{\mathbf{x}^\top \mathbf{z}}{\|\mathbf{x}\|\|\mathbf{z}\|}$	text/embeddings: direction matters, magnitude doesn't
Hamming	# differing coordinates	binary/categorical codes
Mahalanobis	$\sqrt{(\mathbf{x}-\mathbf{z})^\top \Sigma^{-1}(\mathbf{x}-\mathbf{z})}$	accounts for scale and correlation; metric learning generalizes it



Real-life analogy: as the crow flies vs. by road

Think of it like measuring distance between two places in a city. Euclidean distance is "as the crow flies" — a straight line. Manhattan distance is "by road" in a grid of streets — add the blocks east-west and north-south. Cosine similarity ignores how far you travelled and looks only at the *direction* you faced. The right choice depends on what "similar" should mean for your data.

Formula, read aloud: Euclidean $\sqrt{\sum(x_j - z_j)^2}$: square each feature's difference, add them, take the square root; Manhattan $\sum|x_j - z_j|$: just add the absolute differences.

Where it breaks: cities have 2 dimensions with the same units; features have many dimensions in different units, so without scaling one feature can dominate every distance.

10.3 Choice of k

- $k = 1$: zero training error, jagged boundary, **high variance** (one mislabeled neighbor flips a prediction).
- Large k: smoother boundary, **higher bias**; $k = n$ predicts the majority class everywhere.
- Choose k by cross-validation; odd k avoids ties in binary problems; distance weighting reduces sensitivity to k. A classic rough starting point is $k \approx \sqrt{n}$.



Real-life analogy: asking one friend vs. the whole colony

Think of it like choosing a restaurant. Ask one friend ($k = 1$): very personal, but if that friend has odd taste, you're misled (high variance). Ask the whole colony ($k = n$): you'll always get the most popular place, even if it doesn't suit you (high bias). A handful of well-chosen friends is usually best — and you find the right number by trying (cross-validation).

Where it breaks: friends' opinions can be weighed by how well you know them; plain KNN treats the k neighbours equally unless you use distance weighting.

10.4 Normalization

Distances add up per-feature differences, so a feature measured in large units (income in dollars) swamps one in small units (age in decades). **Standardize** (or min-max scale) every feature; for skewed features, log-transform first. Irrelevant features are equally harmful: every noisy feature adds variance to all distances — feature selection or learned metrics help.

Real-life analogy: comparing salary and age

Think of it like judging how similar two people are by adding their salary difference (₹50,000) to their age difference (5 years). The salary number is so big that age doesn't matter at all. Scaling every feature to a similar range makes each one count fairly.

Where it breaks: "fair" isn't always equal — sometimes one feature *should* matter more; scaling puts features on equal footing, and then feature weighting or selection decides importance.

10.5 The curse of dimensionality

Why high dimensionality breaks KNN

1. Neighborhoods stop being local. For uniform data in the unit hypercube $[0,1]^d$, a sub-cube capturing a fraction r of the data has edge length $r^{1/d}$. To capture 1% of the data in $d = 10$ you need edge $0.01^{1/10} \approx 0.63$ — the "neighborhood" spans 63% of every feature's range. It's not local anymore, so the "similar inputs, similar outputs" assumption is no longer being used.

2. Distances concentrate. For many distributions, as d grows the ratio $\frac{d_{\max} - d_{\min}}{d_{\min}} \to 0$: the nearest and farthest points are almost equally far. The ranking of neighbors becomes dominated by noise.

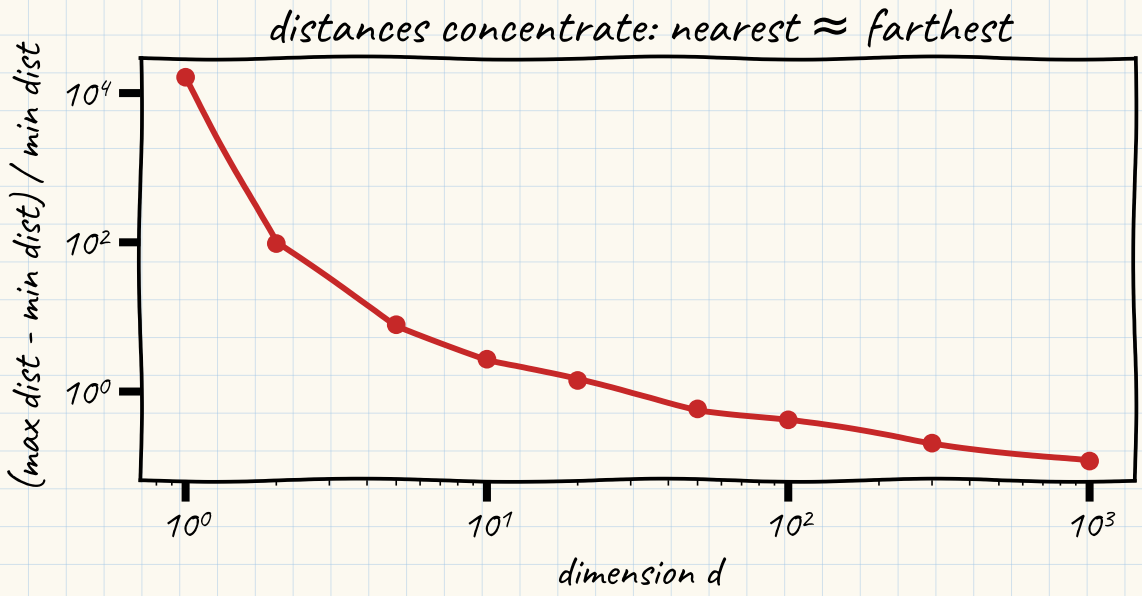
3. Data needed grows exponentially. Keeping a fixed neighbor density requires $n \propto c^d$ samples.

Why it's often not hopeless in practice: real data lies near **low-dimensional manifolds** (images, text embeddings), so the *intrinsic* dimension is small; learned embeddings put semantically similar items close together — which is why nearest-neighbor search over embeddings powers modern retrieval.

Real-life analogy: finding a friend in a bigger and bigger world

Think of it like looking for someone "near you". On a single road (1-D), neighbours are easy to find. In a city (2-D), you need a wider circle. In a world with hundreds of dimensions, to find even a few people you must search almost the entire space — and everyone ends up about equally far away. "Nearest" stops meaning "similar".

Where it breaks: real data rarely fills all dimensions evenly — it usually lies near a lower-dimensional surface, which is why KNN still works on good embeddings.



Distance concentration: in high dimensions, the nearest point is barely nearer than the farthest

10.6 Complexity, memory, and when KNN becomes impractical

	Brute force	KD-tree / ball tree	Approximate NN (HNSW, IVF-PQ, LSH)
Build	$O(1)$ (store)	$O(n \log n)$	$O(n \log n)$-ish; minutes to hours at 10^9
Query	$O(nd)$	~$O(\log n)$ in low d; degrades toward brute force above ~20 dims	sub-linear; trades a little recall for 100–1000× speed
Memory	$O(nd)$ — the whole training set	$O(nd)$	$O(nd)$ or compressed (product quantization: bytes per vector)

Impractical when: n is large and latency budgets are tight (a 10M × 256 float32 matrix is ~10 GB and a brute-force query touches all of it); dimension is high and features aren't meaningful for distance; many irrelevant features; the model must be small or run on-device; data changes constantly (index maintenance). **Where it thrives:** candidate retrieval over learned embeddings with ANN indexes, recommendation ("users like you"), anomaly detection (distance to k-th neighbor), small data baselines, and imputation.

Q: "Why does KNN get worse as you add features, even informative ones?"

Strong answer. "Every feature adds a term to the distance. Noisy or weakly informative features add variance to every distance, so the neighbor ranking gets increasingly random. And in high dimensions distances concentrate — nearest and farthest points become almost equidistant — while the neighborhood needed to capture k points stops being local. With a fixed n, KNN's variance explodes. Fixes: feature selection, dimensionality reduction, learned embeddings or metric learning so distance reflects the target, and more data."

Real-life analogy: a library without a catalogue

Think of it like a library where books are stored but never catalogued. Every time someone asks for a book on cricket, the librarian walks every shelf. Fine for a small school library; hopeless for a national library. Indexes (KD-trees, HNSW) are the catalogue: a little effort up front, much faster lookups.

Where it breaks: a library catalogue is exact; approximate-nearest-neighbour indexes trade a little accuracy (sometimes missing the true nearest book) for huge speed.

PART 11

Naive Bayes

Naive Bayes in one breath

Use Bayes' rule to turn "how likely is this input under each class" into "how likely is each class given this input" — and make that tractable by **assuming features are independent given the class**. Wrong assumption, cheap model, surprisingly good classifier, terrible probabilities.

The whole Part as one picture

Think of it like a doctor diagnosing with a checklist. For each disease, she knows how often each symptom appears (fever in 80% of flu cases, rash in 5%). For a new patient, she multiplies these per-symptom odds together with how common each disease is, and picks the most likely disease. She "naively" treats symptoms as unrelated to each other, even though fever and body ache often come together — yet her diagnosis is usually right, even if her confidence numbers are exaggerated.

Where it breaks: a real doctor knows that correlated symptoms shouldn't be double-counted; Naive Bayes doesn't, so its probabilities are over-confident even when its ranking of diseases is good.

11.1 Bayes' theorem and the independence assumption

$$P(y \mid \mathbf{x}) = \frac{P(y)\,P(\mathbf{x} \mid y)}{P(\mathbf{x})} \;\propto\; P(y)\prod_{j=1}^{d} P(x_j \mid y) \qquad \Rightarrow \qquad \hat{y} = \arg\max_{y}\Big[\log P(y) + \sum_{j} \log P(x_j \mid y)\Big]$$

Without the assumption, $P(\mathbf{x} \mid y)$ over d binary features needs $2^d - 1$ parameters per class; with it, d. Always compute in **log space** — a product of hundreds of small probabilities underflows.

Real-life analogy: guessing a caller from clues

Think of it like guessing who's calling before you pick up: your prior (Mom calls most often), then clues (it's 9 pm, the ringtone is the family one). Each clue multiplies the odds for each suspect. Naive Bayes multiplies the clues as if each were independent evidence.

Formula, read aloud: $\hat{y} = \arg\max_y[\log P(y) + \sum_j \log P(x_j \mid y)]$ says: pick the class with the best total score = (how common the class is) + (how well each clue fits that class), added in log form.

Where it breaks: "9 pm" and "family ringtone" may be linked (Mom calls at 9 pm on the family phone); counting both as independent overstates the evidence.

11.2 Variants

Variant	$P(x_j \mid y)$	Data	Notes
Gaussian NB	$\mathcal{N}(\mu_{jy}, \sigma^2_{jy})$	continuous features	per-class mean/variance per feature; with shared variances, the boundary is linear (like LDA)

Variant	$P(x_j \mid y)$	Data	Notes
Multinomial NB	word probabilities θ_{wy}; likelihood $\propto \prod_w \theta_{wy}^{\text{count}_w}$	word counts / TF-IDF	the classic text classifier
Bernoulli NB	$\theta_{jy}^{x_j}(1-\theta_{jy})^{1-x_j}$	binary presence	also penalizes *absence* of informative words; better for short texts
Complement NB	estimates from all *other* classes	imbalanced text	more stable weights

Laplace (additive) smoothing — without it, one unseen word zeroes out a class:

$$\hat{\theta}_{wy} = \frac{\text{count}(w, y) + \alpha}{\sum_{w'} \text{count}(w', y) + \alpha|V|}$$

Real-life analogy: never saying "impossible" too early

Think of it like a shopkeeper who has never sold a red umbrella and concludes "nobody will ever buy red umbrellas". Laplace smoothing is adding one imaginary sale of every colour before counting, so nothing unseen is judged impossible.

Formula, read aloud: $\hat{\theta} = \frac{\text{count}+\alpha}{\text{total}+\alpha|V|}$ says: add α pretend sightings to every word, then turn counts into probabilities.

Where it breaks: with a large α and small data, the pretend counts can drown the real ones — α is a hyperparameter to tune.

This is MAP estimation with a symmetric Dirichlet prior (Part 3's Beta prior, generalized to $|V|$ categories).

11.3 Text classification

Multinomial NB score for a document: $\log P(y) + \sum_w \text{count}_w \log \theta_{wy}$ — a **linear model in the word counts**. Training is a single counting pass, $O(\text{total tokens})$; prediction $O(K \cdot \text{words in doc})$. It's the fastest reasonable baseline for spam filtering, topic routing, and sentiment, and it degrades gracefully with tiny labeled sets.

Real-life analogy: the post-office sorter's keyword list

Think of it like an experienced post-office sorter who glances at an envelope for telltale words — "Lottery", "Winner", "Urgent" — and tosses it into the spam bin. Each word nudges the decision; no grammar needed. That's multinomial Naive Bayes: a fast, word-counting judge.

Where it breaks: the sorter can't read sarcasm or context ("not a winner"); bag-of-words models ignore word order entirely.

Why can Naive Bayes work well even when the independence assumption is false?

1. **Classification only needs the argmax right.** The assumption distorts the *probabilities* (correlated evidence gets counted twice, pushing scores toward 0 or 1), but the ranking of classes is often preserved — the distortion tends to push in the direction the evidence already points. Domingos & Pazzani (1997) showed NB can be optimal under 0-1 loss even with strong dependencies.
2. **Bias–variance.** NB has very few parameters (per-class, per-feature counts) and they're estimated independently, so its **variance is tiny**. With small data or huge sparse vocabularies, the bias from the wrong assumption costs less than the variance a richer model would add. Ng & Jordan (2002): NB approaches its (higher) asymptotic error with far fewer samples than logistic regression, which wins only once data is plentiful.
3. **Dependencies can cancel.** Dependencies among features that are spread evenly across classes, or that cancel each other out, don't change the decision.

Where it fails: strongly redundant features (duplicate a feature 10× and NB counts its evidence 10×); interactions (XOR-like patterns are invisible to it); and whenever you need **calibrated probabilities** — NB outputs are notoriously overconfident, so recalibrate or don't use them as probabilities.

11.4 A Naive Bayes spam filter by hand

Training emails. Spam: "win money now", "win lottery now", "money money win". Ham: "meeting now", "project money report", "lunch meeting". Vocabulary size $|V| = 8$. Word counts — spam: win 3, money 3, now 2, lottery 1 (total 9); ham: meeting 2, now 1, project 1, money 1, report 1, lunch 1 (total 7). Priors: $P(\text{spam}) = P(\text{ham}) = 0.5$.

New email: "win money meeting".

Multinomial Naive Bayes with Laplace smoothing ($\alpha = 1$)

Spam: $P(\text{win} \mid S) = \frac{3+1}{9+8} = \frac{4}{17}$, $P(\text{money} \mid S) = \frac{4}{17}$, $P(\text{meeting} \mid S) = \frac{0+1}{17} = \frac{1}{17}$. Score $= 0.5 \times \frac{4}{17} \times \frac{4}{17} \times \frac{1}{17} = 0.00163$.

Ham: $P(\text{win} \mid H) = \frac{0+1}{7+8} = \frac{1}{15}$, $P(\text{money} \mid H) = \frac{2}{15}$, $P(\text{meeting} \mid H) = \frac{3}{15}$. Score $= 0.5 \times \frac{1}{15} \times \frac{2}{15} \times \frac{3}{15} = 0.00089$.

Posterior: $P(\text{spam} \mid \text{email}) = \frac{0.00163}{0.00163+0.00089} = \mathbf{0.65}$ → spam, but not overwhelmingly.

Without smoothing: $P(\text{meeting} \mid S) = 0$ and $P(\text{win} \mid H) = 0$, so *both* scores are exactly 0 and the classifier can't decide at all — one unseen word vetoes a class. In practice, add the log-probabilities: $\log(\text{score}_S) = \log 0.5 + 2\log\frac{4}{17} + \log\frac{1}{17}$, which avoids underflow for long emails.



Real-life analogy: a detective weighing clues

Think of it like a detective deciding whether a letter is a scam. "Win" and "money" are clues that usually point to scams; "meeting" usually points to a genuine office email. The detective multiplies how typical each clue is for each explanation and compares the totals. Laplace smoothing is the detective's rule: "never say a clue is *impossible* for a suspect just because I haven't seen it yet" — otherwise one surprising word would clear the suspect completely.

Where it breaks: the detective here treats clues as independent; in real emails "win" and "money" often appear together, so their evidence is double-counted and the 0.65 is not a trustworthy probability.

11.5 Generative vs. discriminative

	Generative (Naive Bayes, LDA, GMM)	Discriminative (logistic regression, SVM, trees, nets)
Models	$P(\mathbf{x} \mid y)P(y) \rightarrow$ Bayes' rule	$P(y \mid \mathbf{x})$ or a decision boundary directly
Data efficiency	better with little data (strong assumptions, low variance)	better asymptotically (fewer assumptions)
Missing features	natural: marginalize them out	needs imputation
Can generate / detect outliers	yes ($P(\mathbf{x})$ available)	no
Wrong assumptions	hurt directly	less exposed

Q: "Naive Bayes vs. logistic regression for text classification?"

Strong answer. "They're a generative–discriminative pair: multinomial NB is a linear classifier in word counts with weights fixed by class-conditional frequencies; logistic regression learns the weights to directly maximize conditional likelihood. With very little labeled data, NB often wins because its estimates are low-variance; with more data, logistic regression wins because it doesn't double-count correlated words like 'New' and 'York'. NB trains in one counting pass, which is nice for streaming or huge corpora. If I need probabilities, logistic regression — NB's are overconfident."



Real-life analogy: learning each language vs. learning to tell them apart

Think of it like two ways to tell Hindi from Tamil speech. A **generative** learner studies each language fully — its sounds, words, rhythm — and asks "which language would most likely produce this sentence?". A **discriminative** learner only learns the differences that separate the two. The first needs less data if its assumptions are right; the second is usually more accurate with plenty of data.

Where it breaks: a real linguist can do both at once; in ML the choice is a modelling assumption, and generative models can also *generate* new examples, which discriminative ones cannot.

Active recall — Set 7: SVM, KNN, Naive Bayes

Set 7 · Answer from memory, then check the Answer Key

Easy recall

1. Write the hard-margin SVM optimization problem.
2. What is the width of the margin in terms of $\mathbf{w}$?
3. Define the RBF kernel and the role of γ.
4. What is Laplace smoothing in Naive Bayes?
5. What's the time complexity of a brute-force KNN query?

Conceptual

6. Which training points are support vectors, in terms of α_i?
7. Show the equivalence of soft-margin SVM and hinge loss + L2.
8. What does the kernel trick buy you, and what property must a kernel have?
9. How do Gaussian, multinomial and Bernoulli NB differ?

10. How does k affect bias and variance in KNN?

Why

11. Why does a wide margin generalize better?
12. Why does KNN need feature scaling?
13. Why does Naive Bayes produce overconfident probabilities?
14. Why do kernel SVMs struggle with millions of examples?
15. Why do distances concentrate in high dimensions, and why does that hurt KNN?

Scenario

16. An RBF SVM gets 100% training accuracy and 70% validation. Which hyperparameters and in which direction?
17. You must classify 10M short support tickets into 50 queues with a 5 ms budget. Where do NB, linear SVM, and KNN fit?
18. Your KNN recommender's latency is 800 ms on 20M items. Options?
19. Duplicating a feature changes a Naive Bayes model's predictions but barely changes logistic regression's. Explain.
20. You need probabilities from an SVM for a cost-sensitive decision. How?

Mathematical

21. Show that $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2$ corresponds to an explicit feature map in 2-D.
22. For uniform data in $[0, 1]^d$, what edge length captures 10% of the data when $d = 20$?
23. Write the multinomial NB decision rule in log space.

Difficult follow-ups

24. Gradient descent on logistic regression with separable data converges in direction to which classifier?
25. When would Naive Bayes outperform logistic regression, according to the generative–discriminative analysis?

2027 additions

26. What is the ε-insensitive loss, and what does the parameter ν control in a one-class SVM?
27. Classify "win money meeting" with the Naive Bayes example in Part 11 (α = 1). What happens without smoothing?
28. Why is SVR's solution sparse?

PART 12

Unsupervised Learning

Clustering in one breath

Clustering finds groups using a notion of similarity **you** choose. There is no ground truth, so every algorithm's answer is a consequence of its assumptions: k-means assumes round, similar-sized blobs; hierarchical clustering assumes a nested structure and a linkage rule; DBSCAN assumes clusters are dense regions separated by sparse ones. Pick the assumption that matches the question you're asking.

The whole Part as one picture

Think of it like sorting a huge heap of unlabelled laundry. You could group it by colour, by fabric or by owner, and each answer is "right" by its own rule. **K-means** sets out k baskets, tosses each item into the nearest one, then moves each basket to the middle of its pile. **Hierarchical** clustering pairs the two most similar socks, then pairs of pairs, building a family tree of piles. **DBSCAN** follows where items are densely heaped and leaves lonely odd socks as "noise". The rule you choose decides what the groups mean.

Where it breaks: laundry has a true owner for every shirt; real data often has no "true" grouping at all, so clusters are judged by whether they are stable and useful.

12.1 K-means

Objective (within-cluster sum of squares, "inertia"):

$$J(\{c_i\}, \{\boldsymbol{\mu}_k\}) = \sum_{i=1}^{n} \left\| \mathbf{x}_i - \boldsymbol{\mu}_{c_i} \right\|^2$$

Real-life analogy: placing polling booths

Think of it like placing k polling booths in a district. Step 1: each voter goes to the nearest booth. Step 2: move each booth to the centre of the voters who came to it. Repeat until nothing moves. Each step can only shorten the total travel, so it settles, but where it settles depends on where the booths started. That is why **k-means++** spreads the first booths apart, and why you restart several times and keep the best.

Formula, read aloud: $J = \sum_i \|\mathbf{x}_i - \boldsymbol{\mu}_{c_i}\|^2$ says: total cost = for every point, the squared distance to its own cluster's centre, all added up.

Where it breaks: voters walk real distances, but k-means squares them, so one far-off voter drags a booth toward him (outliers), and every voter must pick some booth: there is no "stay home" noise option.

Lloyd's algorithm alternates two steps, each of which can only decrease J:

1. **Assignment step:** with centroids fixed, assign each point to its nearest centroid: $c_i = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$. (Minimizes J over assignments.)

2. **Update step:** with assignments fixed, set each centroid to the mean of its points: $\boldsymbol{\mu}_k = \frac{1}{|C_k|}\sum_{i \in C_k} \mathbf{x}_i$. (The mean minimizes the sum of squared distances.)

Because J never increases and there are finitely many partitions, the algorithm **converges** — but to a **local minimum** that depends on initialization (finding the global optimum is NP-hard). Cost: $O(nkd)$ per iteration.

Initialization matters. Random initial centroids can land two in one true cluster and none in another; Lloyd's can't escape. **k-means++** picks the first centroid at random, then each next centroid with probability proportional to $D(\mathbf{x})^2$ — the squared distance to the nearest centroid already chosen. This spreads centroids out and gives an $O(\log k)$-approximation to the optimal J in expectation. Standard practice: k-means++ plus several restarts (`n_init`), keep the lowest J.

Choosing k.

- **Elbow method:** plot J vs. k; J always decreases (with $k = n$ it's 0), so look for the bend where extra clusters stop buying much. Often ambiguous.
- **Silhouette score:** for point i, $a(i)$ = mean distance to its own cluster, $b(i)$ = mean distance to the nearest other cluster; $s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))} \in [-1, 1]$. Average over points; higher is better; near 0 means overlapping clusters. Costs $O(n^2)$.
- Gap statistic, BIC for GMMs, cluster stability under resampling — and above all **usefulness for the downstream purpose** (do the segments behave differently? can marketing act on them?).

Assumptions and failure modes. Clusters are convex, roughly spherical (isotropic), similar in size and density, in a space where Euclidean distance is meaningful. It fails on elongated clusters, nested rings, very different cluster sizes (a large cluster gets split while small ones merge), and is sensitive to **feature scaling** and **outliers** (a mean is dragged by one extreme point → k-medoids uses actual points and any distance). Every point is forced into some cluster — there is no notion of noise.

Connections: k-means is the hard-assignment, equal-spherical-variance limit of EM for a **Gaussian mixture model**; mini-batch k-means scales to huge data; k-means on embeddings is a common way to build a vector-quantization codebook (the "IVF" in approximate nearest-neighbor indexes).

12.2 Hierarchical clustering

Agglomerative: start with every point as its own cluster; repeatedly merge the two closest clusters; record merges in a **dendrogram**; cut it at a height (or number of clusters) afterwards.



Real-life analogy: building a family tree upward

Think of it like building a family tree from the bottom up. Start with every person alone. Join the two closest relatives into a pair, then keep joining the two closest groups: couples into households, households into extended families, families into a clan. The finished tree is the **dendrogram**; "cutting" it at some height means choosing which level of grouping to report. **Linkage** is how you judge two families' closeness: by their closest members (single), their most distant members (complete), or on average.

Where it breaks: real kinship is fixed; the algorithm's merges are greedy guesses, and once two groups are joined they are never separated, even if that early merge was a mistake.

Linkage	Distance between clusters	Behavior
Single	closest pair	finds elongated shapes, but **chaining**: a thin bridge of points merges two clusters
Complete	farthest pair	compact, similar-diameter clusters; sensitive to outliers

Linkage	Distance between clusters	Behavior
Average	mean pairwise distance	compromise
Ward	increase in total within-cluster variance	k-means-like compact clusters; usual default

Pros: no need to choose k upfront; the dendrogram shows structure at every scale; works with any distance. **Cons:** $O(n^2)$ memory for the distance matrix and $O(n^2 \log n)$ to $O(n^3)$ time — impractical beyond ~10^4–10^5 points; merges are greedy and irreversible.

12.3 DBSCAN

Parameters: radius ε and `min_samples`. A **core point** has at least `min_samples` points within ε; a **border point** is within ε of a core point but isn't core; everything else is **noise**. Clusters are maximal sets of points density-connected through core points.

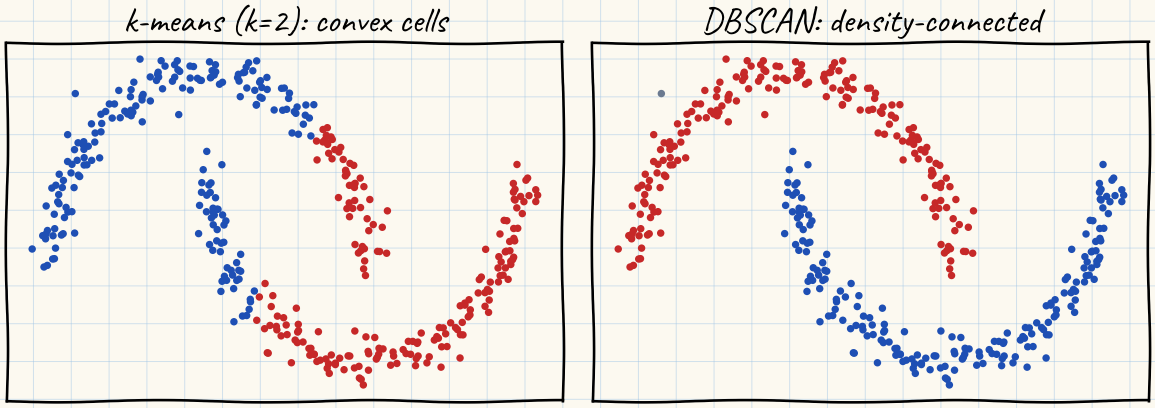


Real-life analogy: crowds at a mela

Think of it like spotting groups at a crowded mela from a rooftop. Wherever people stand densely packed, that's a group — whatever its shape: a long queue at the ride, a circle around a magician. A person with enough people within arm's reach is at the heart of a crowd (core point); someone at the edge touches a crowd but isn't surrounded (border point); a lone wanderer is noise.

Where it breaks: at a real mela, a thin crowd and a packed crowd are both obviously crowds; DBSCAN uses one fixed "arm's reach", so it struggles when groups have very different densities (HDBSCAN fixes this).

Strengths: discovers **arbitrarily shaped** clusters, labels outliers as noise, needs no k. **Weaknesses:** one global density threshold — fails when clusters have **different densities** (HDBSCAN addresses this by considering all ε levels); choosing ε is hard in high dimensions (use a k-distance plot); $O(n \log n)$ with a spatial index in low dimensions, degrading toward $O(n^2)$.



k-means cuts moons into convex halves; DBSCAN follows density

12.4 Comparison

	K-means	Hierarchical (Ward)	DBSCAN	GMM
Must choose	k	cut height / k	ε, min_samples	k, covariance type

	K-means	Hierarchical (Ward)	DBSCAN	GMM
Cluster shape	convex, spherical	depends on linkage	arbitrary	ellipsoidal
Noise / outliers	forced into clusters; distort centroids	forced (single linkage chains)	explicit noise label	low-likelihood points
Soft assignments	no	no	no	yes (probabilities)
Scale	$O(nkd)$/iter — millions of points	$O(n^2)$ memory — thousands	$\sim O(n \log n)$ low-dim	$O(nkd^2)$/iter
New points	assign to nearest centroid	no natural way	not directly	yes



Real-life analogy: different ways to seat wedding guests

Think of it like seating guests at a wedding. K-means: decide the number of tables first and seat everyone at the nearest table. Hierarchical: start with couples, merge into families, then clans — and decide later where to cut. DBSCAN: let people form natural groups and leave loners standing. GMM: allow each guest to belong partly to several tables ("60% bride's side, 40% groom's friends").

Where it breaks: guests have real relationships you can ask about; clustering only sees the features you give it, and "correct" groups depend on your purpose.

12.5 Gaussian mixture models and the EM algorithm

A **Gaussian mixture model (GMM)** says each point came from one of K Gaussian "sources", chosen with probabilities π_k: $p(\mathbf{x}) = \sum_k \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \Sigma_k)$. Unlike k-means it gives **soft** assignments, **elliptical** clusters of different sizes, and a likelihood you can use for model selection (BIC) and anomaly scores.

The difficulty: if we knew which source produced each point, fitting would be easy (per-cluster means and covariances); if we knew the parameters, assigning points would be easy. **Expectation–maximization** alternates the two:

$$\text{E-step:} \quad r_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \Sigma_j)}$$

$$\text{M-step:} \quad N_k = \sum_i r_{ik}, \qquad \pi_k = \frac{N_k}{n}, \qquad \boldsymbol{\mu}_k = \frac{1}{N_k} \sum_i r_{ik} \mathbf{x}_i,$$

$$\Sigma_k = \frac{1}{N_k} \sum_i r_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top$$

Why EM never decreases the likelihood

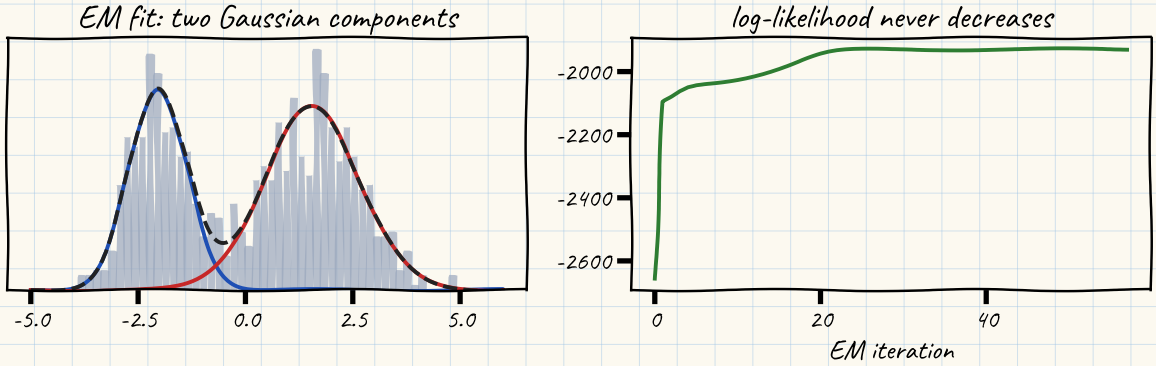
For any distribution $q(z)$ over the hidden assignments, Jensen's inequality gives a lower bound: $\log p(\mathbf{x} \mid \theta) = \log \sum_z q(z) \frac{p(\mathbf{x}, z \mid \theta)}{q(z)} \geq \sum_z q(z) \log \frac{p(\mathbf{x}, z \mid \theta)}{q(z)} = \mathrm{ELBO}(q, \theta)$.

- **E-step:** set $q(z) = p(z \mid \mathbf{x}, \theta_{\text{old}})$ — the responsibilities. The bound becomes **tight** (equal to the log-likelihood).
- **M-step:** maximize the bound over θ — weighted MLE, closed-form for Gaussians.
- Therefore $\log p(\mathbf{x} \mid \theta_{\text{new}}) \geq \mathrm{ELBO}(q, \theta_{\text{new}}) \geq \mathrm{ELBO}(q, \theta_{\text{old}}) = \log p(\mathbf{x} \mid \theta_{\text{old}})$. Monotone ascent to a **local** maximum.

The same bound is the VAE's ELBO (Part 49) — with the exact posterior replaced by a learned encoder.

```python
def em_gmm_1d(x, k=2, n_iter=100, seed=0, tol=1e-8):
    rng = np.random.default_rng(seed)
    n = len(x)
    mu = rng.choice(x, k, replace=False).astype(float)
    var = np.full(k, x.var())
    pi = np.full(k, 1.0 / k)
    lls = []
    for _ in range(n_iter):
        # E-step: responsibilities r_ij = P(z_i = j | x_i), computed in log space
        logp = (np.log(pi) - 0.5 * np.log(2 * np.pi * var)
                - 0.5 * (x[:, None] - mu) ** 2 / var)              # (n, k)
        m = logp.max(1, keepdims=True)
        log_norm = m + np.log(np.exp(logp - m).sum(1, keepdims=True))
        r = np.exp(logp - log_norm)
        lls.append(log_norm.sum())
        # M-step: weighted MLEs
        nk = r.sum(0)
        pi = nk / n
        mu = (r * x[:, None]).sum(0) / nk
        var = (r * (x[:, None] - mu) ** 2).sum(0) / nk + 1e-6     # variance floor prevents
collapse
        if len(lls) > 1 and abs(lls[-1] - lls[-2]) < tol:
            break
    return pi, mu, var, np.array(lls)
```

Checked: on 400 points from $\mathcal{N}(-2, 0.7^2)$ and 600 from $\mathcal{N}(1.5, 1.1^2)$, EM recovers $\pi = (0.40, 0.60)$ and $\mu = (-2.03, 1.45)$, and the log-likelihood never decreases.



EM on a two-component 1-D mixture: fitted components (left) and monotone log-likelihood (right)

Practical notes: initialize with k-means (and several restarts — EM finds local optima); add a small variance floor or covariance regularization so a component can't collapse onto one point (likelihood → ∞); choose K with BIC or held-out likelihood; covariance types (full, diagonal, spherical, tied) trade flexibility for parameters. **k-means is the limit of EM** with equal spherical covariances shrinking to zero — responsibilities become hard 0/1 assignments.

Real-life analogy: sorting a mixed bag of rice

Think of it like a shopkeeper handed a sack that mixes two varieties of rice — basmati (long grains) and sona masoori (shorter grains) — without labels. She guesses what each variety's typical grain length is (M-step), then for each grain decides how likely it is to be basmati vs. sona masoori (E-step: "70% basmati"), then re-estimates the typical lengths using those soft votes, and repeats. Each round her description of the two varieties fits the sack better.

Formula, read aloud: r_{ik} = (how common source k is × how well source k explains point i) ÷ (the same, summed over all sources) — "the share of the blame for point i that goes to source k".

Where it breaks: the shopkeeper knows there are exactly two varieties; EM must be told K, can settle on a poor local answer if started badly, and assumes bell-shaped groups.

12.6 Spectral clustering and HDBSCAN

Spectral clustering clusters by *connectivity* instead of distance to a center: build a similarity graph (e.g. Gaussian similarity $W_{ij} = e^{-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma^2}$), form the normalized graph Laplacian $L = I - D^{-1/2} W D^{-1/2}$, take the eigenvectors of its k smallest eigenvalues as new coordinates, and run k-means there. Points in the same connected blob get nearly identical coordinates, whatever the blob's shape.

```
def spectral_clustering(X, k=2, sigma=0.3, seed=0):
    from sklearn.cluster import KMeans
    D2 = ((X[:, None, :] - X[None, :, :]) ** 2).sum(-1)
    W = np.exp(-D2 / (2 * sigma ** 2)); np.fill_diagonal(W, 0)     # similarity graph
    d = W.sum(1)
    L_sym = np.eye(len(X)) - W / np.sqrt(np.outer(d, d))           # normalized Laplacian
    vals, vecs = np.linalg.eigh(L_sym)
    U = vecs[:, :k]                                                # k smallest eigenvectors
    U = U / np.linalg.norm(U, axis=1, keepdims=True)
    return KMeans(k, n_init=10, random_state=seed).fit_predict(U)
```

Checked: on two concentric circles, spectral clustering recovers the rings perfectly (adjusted Rand index **1.00**) while k-means scores **≈ 0** — it cuts the rings in half. **Costs:** $O(n^2)$ memory for the similarity matrix and an eigen-decomposition — use sparse k-NN graphs and iterative eigensolvers beyond a few tens of thousands of points; σ (or the number of neighbors) must be tuned.

HDBSCAN extends DBSCAN by considering all density thresholds at once and keeping the clusters that persist longest across them: it finds clusters of **different densities**, labels noise, and needs mainly one intuitive parameter (minimum cluster size). A common modern pipeline for text or image embeddings is *embed → UMAP to ~10 dimensions → HDBSCAN*.

Real-life analogy: friend circles, not neighbourhoods

Think of it like finding friend groups at a college. k-means groups students by where they live (distance to a centre), so two friend circles living in the same hostel get merged. Spectral clustering looks at *who talks to whom* — a chain of friends counts as one group even if it stretches across campus. HDBSCAN is like noticing that some groups are tight gangs and others are loose clubs, and recognizing both.

Formula, read aloud: the Laplacian's smallest eigenvectors are the "smoothest" ways to label the graph — labels that barely change between connected friends — so each connected group gets its own constant label.

Where it breaks: real friend networks are observed; here the "who talks to whom" graph is built from distances with a chosen σ or k, and a bad choice can link separate groups or split one.

12.7 When clustering becomes meaningless in high dimensions

Why clusters can vanish as dimensions grow

Clustering relies on some points being much closer to each other than to others. In high dimensions, **distances concentrate** (Part 10): for many distributions the nearest and farthest neighbors become almost equally far, so "close" carries little information. Worse, if only a few dimensions carry cluster structure and hundreds are noise, the noise dimensions dominate every distance and wash the structure out; and different subsets of features may define different, equally valid clusterings. A clustering algorithm will still output clusters — it always does — they just won't mean anything.

What to do: reduce dimension first (PCA, autoencoders, domain-specific embeddings) so distance reflects meaningful variation; select features that matter for the purpose; use cosine similarity on normalized embeddings; use subspace/biclustering methods; and **validate**: stability across bootstrap samples and random seeds, separation metrics compared against clustering of shuffled (structureless) data, and usefulness downstream.

Q: "How would you evaluate a clustering when there are no labels?"

Strong answer. "Three layers. Internal metrics — silhouette, Davies–Bouldin, inertia — tell me about compactness and separation, but they share the algorithm's assumptions, so they're weak evidence alone. Stability — rerun with different seeds and bootstrap samples and measure agreement, e.g. with the adjusted Rand index; real structure is stable. And most important, external usefulness: do the clusters differ on outcomes we care about, like retention or spend, and can a team act on them? If a few labels exist, I'd check ARI or NMI against them. I'd also sanity-check that the same pipeline on shuffled data doesn't produce equally 'good' clusters."

Real-life analogy: grouping people by 500 random facts

Think of it like grouping classmates using 500 facts about each — including shoe size, favourite colour, last digit of their phone number. Most facts are noise, so everyone looks roughly equally different from everyone else, and the "groups" you find are accidents. Pick the few facts that matter for your purpose first, then group.

Where it breaks: sometimes the high-dimensional data *does* lie on a meaningful low-dimensional structure (like good embeddings), and clustering works well once you measure distance in the right space.

PART 13

Dimensionality Reduction

Dimensionality reduction in one breath

PCA finds the linear subspace that keeps the most variance — equivalently loses the least squared reconstruction error — and it's the eigenvectors of the covariance matrix. **t-SNE and UMAP** are nonlinear *visualization* tools that preserve **neighborhoods**, not distances: never read cluster sizes or gaps between clusters off their plots.

The whole Part as one picture

Think of it like making a school map. The real campus is 3-D and full of detail, but a good 2-D map keeps what you need: where buildings are relative to each other. PCA is like a satellite photo taken from the angle that shows the most spread — straight lines and true proportions, but some depth is lost. t-SNE and UMAP are like a hand-drawn "friend map" that puts friends' houses next to each other — great for seeing who hangs out with whom, but the distances between far-apart groups on it mean little.

Where it breaks: a map is for humans; many dimensionality reductions are used as inputs for other models, where keeping predictive information matters more than a nice picture.

13.1 PCA — derived conceptually

We want a direction $\mathbf{u}$ (unit vector) such that projecting the centered data onto it keeps as much spread as possible. The projected values are $z_i = \mathbf{u}^\top \mathbf{x}_i$; their variance is $\mathbf{u}^\top \Sigma \mathbf{u}$ (Part 2's transformation rule). High variance along $\mathbf{u}$ means the data is "spread out" in that direction — that's where the information (differences between points) lives. After finding the best direction, find the next best one **orthogonal** to it, and so on.

Real-life analogy: the best angle to photograph a crowd

Think of it like photographing a long queue outside a cinema with one shot. From the front, everyone overlaps and you see one face; from the side, you see the whole line spread out. PCA picks the camera direction in which the data is most spread out — that view keeps the most information about how points differ.

Formula, read aloud: the spread along direction $\mathbf{u}$ is $\mathbf{u}^\top \Sigma \mathbf{u}$: combine the covariance table with the direction to get "how much the data varies this way".

Where it breaks: "most spread" is not always "most useful" — if the label depends on a low-variance direction, PCA may discard exactly what you need.

13.2 PCA — derived mathematically

Derivation — variance maximization gives eigenvectors

Center the data ($\bar{\mathbf{x}} = 0$), $\Sigma = \frac{1}{n-1} X^\top X$.

$$\max_{\mathbf{u}} \ \mathbf{u}^\top \Sigma \mathbf{u} \quad \text{s.t.} \quad \mathbf{u}^\top \mathbf{u} = 1 \qquad \mathcal{L} = \mathbf{u}^\top \Sigma \mathbf{u} - \lambda(\mathbf{u}^\top \mathbf{u} - 1)$$

$$\nabla_{\mathbf{u}} \mathcal{L} = 2\Sigma \mathbf{u} - 2\lambda \mathbf{u} = 0 \implies \boxed{\Sigma \mathbf{u} = \lambda \mathbf{u}}$$

So $\mathbf{u}$ must be an eigenvector, and the variance it captures is $\mathbf{u}^\top \Sigma \mathbf{u} = \lambda \, \mathbf{u}^\top \mathbf{u} = \lambda$. The maximum is the **largest eigenvalue**, attained by its eigenvector: **PC1**. Adding the constraint $\mathbf{u}_2 \perp \mathbf{u}_1$ gives the second eigenvector, and so on. Since Σ is symmetric PSD, its eigenvectors are orthogonal and eigenvalues non-negative — the components are uncorrelated and ranked.



Real-life analogy: finding the grain of the wood

Think of it like a carpenter finding the grain of a plank: there's one direction along which the wood naturally stretches the most. The eigenvector equation $\Sigma \mathbf{u} = \lambda \mathbf{u}$ describes such special directions — pushing the data along them doesn't turn it, it only stretches it by λ. The biggest stretch is the first principal component.

Formula, read aloud: $\Sigma \mathbf{u} = \lambda \mathbf{u}$ says: "multiply the covariance by this direction and you get the same direction back, just scaled by λ — and λ is exactly the variance captured".

Where it breaks: wood has one obvious grain; data has many orthogonal "grains" ranked by variance, and in noisy data the lower ones are hard to estimate reliably.

Derivation — the same subspace minimizes reconstruction error

Project onto an orthonormal $U_k \in \mathbb{R}^{d \times k}$ and reconstruct: $\hat{\mathbf{x}} = U_k U_k^\top \mathbf{x}$. By Pythagoras, for each point $\|\mathbf{x}\|^2 = \|U_k^\top \mathbf{x}\|^2 + \|\mathbf{x} - \hat{\mathbf{x}}\|^2$. Summing over points, the total (fixed) variance = variance kept + reconstruction error. **Maximizing variance kept ⇔ minimizing squared reconstruction error.** The optimum keeps the top-k eigenvectors, and the error is $\sum_{j>k} \lambda_j$ (Eckart–Young, Part 2).

The vocabulary you'll be asked for.

- **Covariance matrix** Σ: the data's spread and correlations; its eigen-decomposition is PCA.
- **Eigenvectors / principal axes / loadings:** the directions $\mathbf{u}_j$ (columns of V in the SVD).
- **Principal components / scores:** the coordinates $z_{ij} = \mathbf{u}_j^\top \mathbf{x}_i$ of the data in the new basis ($XV = U\Sigma$).
- **Explained variance ratio** of component j: $\lambda_j / \sum_i \lambda_i$.

PCA via SVD (how it's actually computed). $X_c = U\Sigma V^\top$ ⇒ principal directions $= V$, variances $= \sigma_j^2/(n-1)$, scores $= U\Sigma$. More stable than forming $X^\top X$. Cost $O(nd \min(n,d))$ for full SVD; **randomized SVD** gets the top k in about $O(ndk)$.

Practical decisions.

Decision	Guidance
Standardize first?	**Yes** if features have different units — otherwise the feature with the largest numeric variance dominates PC1. No if features share a meaningful common scale (pixels, same-unit sensors).
How many components?	cumulative explained variance (e.g. 90–99%), elbow of the scree plot, or — best — the k that maximizes downstream validation performance

Decision	Guidance
Fit on what?	the **training** set only; apply the same centering and projection to validation/test (fitting PCA on all data leaks)

Limitations — with the mechanism.

- **Linear:** it can only find flat subspaces; a curved manifold (a Swiss roll) needs kernel PCA, autoencoders, or manifold methods.
- **Unsupervised:** it keeps high-**variance** directions, which aren't necessarily **predictive** ones. A low-variance direction can carry all the label signal (use PLS or supervised methods, or just let the model use raw features).
- **Sensitive to outliers:** variance is a squared quantity, so one extreme point can rotate PC1 (robust PCA exists).
- **Interpretability:** each component mixes all features.

13.3 Why t-SNE should not be read as preserving global distances

How t-SNE works (enough to reason about it). In the original space, convert distances into neighbor probabilities with a Gaussian around each point, whose width is set per point so the effective number of neighbors equals the **perplexity** (typically 5–50). In 2-D, define neighbor probabilities with a heavy-tailed **Student-t** kernel. Move the 2-D points to minimize $\mathrm{KL}(P \,\|\, Q)$.



Real-life analogy: a seating chart that only keeps friendships

Think of it like a teacher making a seating plan where the only rule is "friends sit together". Friend groups end up as tight clusters, but whether the cricket group sits near the music group is arbitrary — the plan never tried to reflect that. t-SNE preserves who is near whom, not how far apart groups are or how big they are.

Where it breaks: a seating plan is drawn once; t-SNE plots change with perplexity and random seed, which is another reason not to read meaning into gaps and sizes.

Why t-SNE distorts global structure

1. **The objective only cares about neighbors.** P_{ij} is essentially zero for points that aren't near each other, and the forward KL barely penalizes placing distant points anywhere (Part 3: forward KL punishes $q \approx 0$ where $p > 0$ — i.e. separating true neighbors — but not the reverse). So distances *between* clusters are basically unconstrained.
2. **Per-point bandwidths equalize densities.** Each point's Gaussian width adapts so it has ~perplexity neighbors: a tight cluster and a diffuse cluster are both expanded or shrunk to similar-looking blobs. **Cluster sizes on the plot mean nothing.**
3. **The heavy-tailed t-kernel** (to fix the "crowding problem" — there isn't room in 2-D for all moderately distant points) exaggerates gaps between clusters.
4. **Non-convex and hyperparameter-dependent:** different seeds, perplexities and iteration counts give different pictures; random noise can look like clusters at low perplexity.

So: t-SNE shows **which points are neighbors**. It does not show how far apart clusters are, how big they are, or whether one cluster is "between" two others. Use it for exploration, never as evidence of global geometry — and never feed t-SNE coordinates into a downstream model (it has no transform for new points).

13.4 UMAP

Builds a weighted k-nearest-neighbor graph (a fuzzy topological representation), then optimizes a low-dimensional layout whose graph matches it under a cross-entropy objective, using attractive and repulsive forces. Key knobs: `n_neighbors` (local vs. more global structure) and `min_dist` (how tightly points pack).

Compared with t-SNE: much faster on large data; tends to retain more global arrangement (though still not faithful distances or densities); can **transform new points** and embed into more than 2–3 dimensions, so it's sometimes used as preprocessing for clustering (e.g. UMAP → HDBSCAN on embeddings). The same caution applies: inter-cluster distances and cluster sizes are not reliable.



Real-life analogy: a metro map

Think of it like a metro map: it keeps which stations connect and roughly where lines run, distorting true distances to stay readable. UMAP builds a "who's connected to whom" graph of your data and draws it in 2-D, keeping a bit more of the city's overall layout than t-SNE does.

Where it breaks: people learn to trust metro maps; UMAP layouts still distort distances and densities, so gaps and cluster sizes must not be read literally.

13.5 Kernel PCA, autoencoders and randomized SVD

Kernel PCA: run PCA in the feature space of a kernel (Part 9) using only the $n \times n$ kernel matrix — center it, take its top eigenvectors. It can "unroll" curved structures (concentric circles become separable along the first component with an RBF kernel). Costs $O(n^2)$ memory and has no cheap exact mapping back to input space.

Autoencoders: a network squeezes the input through a narrow bottleneck $\mathbf{z} = g(\mathbf{x})$ and reconstructs $\hat{\mathbf{x}} = h(\mathbf{z})$ by minimizing reconstruction error. A **linear** autoencoder with squared error spans the same subspace as PCA; nonlinear ones learn curved manifolds, handle images/text with suitable architectures, and scale with SGD. Variants: denoising (reconstruct clean from corrupted input — robust features), sparse, and variational (Part 49). Reconstruction error doubles as an **anomaly score**.

Randomized SVD makes PCA feasible for huge matrices: multiply X by a random $d \times (k+p)$ matrix to get a sketch that captures its top range, orthonormalize it, then do an exact small SVD. A couple of **power iterations** (Part 2) sharpen accuracy when the spectrum decays slowly. This is what `PCA(svd_solver="randomized")` does, with cost roughly $O(ndk)$ instead of $O(nd^2)$.

Method	Linear?	Scales to	Out-of-sample transform	Best for
PCA (exact)	yes	d up to ~10^4	yes	compression, decorrelation, denoising
Randomized PCA	yes	very large n, d	yes	top-k components of big data
Kernel PCA	no	n up to ~10^4	approximate	small curved datasets
Autoencoder	no	very large	yes	images, text, anomaly detection
t-SNE / UMAP	no	10^5–10^6	UMAP only	visualization, clustering prep



Real-life analogy: compressing a photo album

Think of it like fitting a family photo album onto a small pen drive. PCA is a standard compressor that keeps the broad shapes and colours (linear directions of most variation). An autoencoder is a smart compressor trained on *your* family's photos — it learns what faces and backgrounds usually look like, so it keeps what matters with far fewer bytes. Randomized SVD is a quick preview: glance at random pieces of every photo to find the main patterns fast, then refine.

Formula, read aloud: an autoencoder minimizes $\|\mathbf{x} - h(g(\mathbf{x}))\|^2$: "squeeze the picture, rebuild it, and measure what was lost".

Where it breaks: a compressor that rebuilds pictures well may keep details irrelevant to your task; the best representation for classification is not always the one with the smallest reconstruction error.

13.6 Choosing a method

Goal	Use	Why
Compression, denoising, decorrelating features, speeding up a model	**PCA**	linear, fast, invertible, deterministic, has a transform
Visualizing local structure / clusters	t-SNE or UMAP	preserve neighborhoods
Nonlinear features for downstream models at scale	UMAP, autoencoders, learned embeddings	have transforms; capture curved structure
Supervised reduction	LDA (Fisher), PLS, or train a model with a bottleneck	keep label-relevant directions
Very sparse text	truncated SVD (LSA)	works on sparse matrices without centering



Real-life analogy: picking the right kind of map

Think of it like choosing a map for a job: a scale map for measuring distances (PCA: faithful, linear, reusable on new data), a metro diagram for seeing connections (t-SNE/UMAP: neighbourhoods for visualization), and GPS coordinates learned for your own app (autoencoders/embeddings: nonlinear features for a model).

Where it breaks: you can check a real map against the ground; with high-dimensional data, always check a reduction against the downstream task or known labels.

Q: "Walk me through PCA as if on a whiteboard."

Strong answer. "Center the data. I want a unit direction $\mathbf{u}$ maximizing the variance of the projections, which is $\mathbf{u}^\top \Sigma \mathbf{u}$. With a Lagrange multiplier for $\|\mathbf{u}\| = 1$, setting the gradient to zero gives $\Sigma \mathbf{u} = \lambda \mathbf{u}$ — $\mathbf{u}$ is an eigenvector — and plugging back in, the variance equals the eigenvalue, so PC1 is the top eigenvector. Each subsequent component is the next eigenvector, orthogonal to the previous ones. Equivalently, by Pythagoras, the top-k subspace minimizes squared reconstruction error. In practice I compute it with an SVD of the centered data, standardize if units differ, fit it on training data only, and choose k by explained variance or downstream validation."

Follow-ups

"Is PCA feature selection?" — No: it creates new features that are combinations of all original ones. Feature selection keeps a subset of the originals.

"When would PCA hurt a model?" — When the predictive signal lives in low-variance directions (it gets discarded), for tree models (rotations destroy axis-aligned structure and interpretability), or when fitted on data including the test set (leakage).

"PCA vs. autoencoder?" — A linear autoencoder with squared error learns the same subspace as PCA (not necessarily the same orthonormal basis). Nonlinear autoencoders can capture curved manifolds but need more data and tuning and lose PCA's closed form and ordering.

Active recall — Set 8: Unsupervised learning & dimensionality reduction

Set 8 · Answer from memory, then check the Answer Key

Easy recall

1. Write the k-means objective.
2. Describe the assignment and update steps.
3. Define the silhouette score for one point.
4. What are core, border and noise points in DBSCAN?
5. What is the explained variance ratio of a principal component?

Conceptual

6. Why does Lloyd's algorithm converge, and to what?
7. How does k-means++ choose initial centroids?
8. Compare single, complete and Ward linkage.
9. Why is maximizing projected variance equivalent to minimizing reconstruction error?
10. What does perplexity control in t-SNE?

Why

11. Why does k-means struggle with clusters of different sizes and densities?
12. Why should you standardize before PCA when units differ?
13. Why can't you interpret distances between clusters on a t-SNE plot?
14. Why does clustering become unreliable in high dimensions?
15. Why might PCA discard the most predictive information?

Scenario

16. Marketing asks for "customer segments". Walk through your approach.
17. Your k-means clusters change substantially each run. What do you do?
18. You need to detect unusual transactions without labels. Which clustering-based approaches, and their pitfalls?
19. A colleague shows a UMAP plot with two well-separated blobs and concludes there are two customer types. Respond.
20. You want to speed up a model with 2,000 dense correlated features. Plan?

Mathematical

21. Derive PC1 with a Lagrange multiplier.
22. With eigenvalues 5, 3, 1, 0.5, 0.5, how many components keep ≥ 90% of variance?
23. Show the k-means update step (mean) minimizes within-cluster squared distance.

Difficult follow-ups

24. How is k-means related to EM on a Gaussian mixture model?
25. Why does t-SNE use a Student-t distribution in the low-dimensional space?

2027 additions

26. Write the E-step and M-step for a Gaussian mixture, and explain why the likelihood never decreases.
27. Why does spectral clustering separate two concentric circles when k-means cannot?
28. What does randomized SVD do and when do you need it?

Unit IV

Training, Evaluation & Data

Part 14 — Optimization · Part 15 — Loss Functions
Part 16 — Model Evaluation · Part 17 — Splitting & Validation
Part 18 — Class Imbalance · Part 19 — Feature Engineering

How models actually get trained, how you know if they're good, and how the data you feed them decides both.

PART 14

Optimization

Optimization in one breath

Training is walking downhill on a loss surface you can only feel locally (the gradient), with a step size you must choose (the learning rate), using noisy estimates of the slope (mini-batches). Almost every optimizer trick is an answer to one of three problems: **the surface is badly scaled** (momentum, Adam, normalization), **the slope estimate is noisy** (batch size, averaging, schedules), or **the gradient signal dies or explodes on its way through the network** (initialization, residuals, clipping).

The whole Part as one picture

Think of it like a delivery rider finding a customer's house in an unfamiliar city with no map. At each junction he only knows which way "feels closer" (the **gradient**), he decides how far to ride before checking again (the **learning rate**), and his phone's location keeps jittering (**mini-batch noise**). The tricks in this Part fix three problems: some roads are steep flyovers and others flat lanes (bad scaling: momentum, Adam), the jittery location (batch size, schedules), and directions that get garbled passing through a long chain of dispatchers (vanishing or exploding gradients: initialization, residuals, clipping).

Where it breaks: the rider has one fixed address; a network has many equally low "addresses", and which one training reaches changes how well the model does on new data.

14.1 Objective functions

$$\min_{\theta} J(\theta) = \underbrace{\frac{1}{n}\sum_{i=1}^{n} \ell\big(f_\theta(x_i), y_i\big)}_{\text{empirical risk}} + \underbrace{\lambda\,\Omega(\theta)}_{\text{regularizer}}$$

Real-life analogy: mock tests vs the board exam

Think of it like a student preparing for board exams with mock tests. Her daily target is the mock score plus a rule against weird cramming tricks (the **regularizer**). What counts, though, is the real board exam (the **metric**), and what she truly wants is a college seat (the **business outcome**). If mock scores rise but board-style results don't, the mocks don't match the exam, and studying harder the same way won't fix it.

Formula, read aloud: $J(\theta) = \frac{1}{n}\sum_i \ell(f_\theta(x_i), y_i) + \lambda\,\Omega(\theta)$ says: total cost = average mistake over the training examples + λ × a penalty for overly complicated settings.

Where it breaks: a student can notice the mocks are off; the optimizer can't. It sees only J and keeps improving it even when the metric doesn't move.

Three distinct things are easy to conflate: the **objective** you optimize (differentiable surrogate + regularizer), the **metric** you evaluate (maybe non-differentiable: accuracy, NDCG), and the **business outcome**. The optimizer only sees the first. When optimization "works" but the metric doesn't move, the gap is usually objective–metric misalignment, not the optimizer.

14.2 Convex vs. non-convex optimization

Convex function: $J(\alpha a + (1-\alpha)b) \leq \alpha J(a) + (1-\alpha)J(b)$ — the chord lies above the graph; for twice-differentiable J, the Hessian is PSD everywhere. **Why it matters:** every local minimum is a global minimum, the set of minimizers is convex, and gradient methods come with convergence guarantees independent of initialization.



Real-life analogy: a kadhai vs the Western Ghats

Think of it like dropping a marble into a kadhai (an Indian wok): wherever it starts, it rolls to the same bottom. That is a **convex** loss. A non-convex loss is like the Western Ghats: many valleys, mountain passes (saddle points) and flat plateaus, and where you end up depends on where you start and how you walk.

Formula, read aloud: $J(\alpha a + (1-\alpha)b) \leq \alpha J(a) + (1-\alpha)J(b)$ says: stretch a string between any two points on the surface; the surface in between never rises above the string, so no hidden bump can trap you.

Where it breaks: hiking intuition over-warns; in millions of dimensions most high "traps" are passes you can slide off, and the valleys SGD finds tend to be about equally low.

Convex	Non-convex
linear / ridge / lasso regression, logistic regression, linear SVM, softmax regression (in their parameters)	neural networks, matrix factorization, k-means, GMMs, trees (discrete — not even continuous)
any local min is global; initialization doesn't matter	many critical points: local minima, **saddle points**, plateaus; results depend on init, LR, batch order

Why non-convex deep learning works anyway: in very high dimensions, most critical points with high loss are **saddle points**, not local minima (a local minimum needs *every* Hessian eigenvalue positive; a random critical point has mixed signs). Empirically, for large over-parameterized networks, the local minima SGD finds have similar, low losses and are often connected by low-loss paths (mode connectivity). The practical difficulty is less "bad local minima" and more ill-conditioning, saddles/plateaus and signal propagation.

14.3 Gradients, partial derivatives and the chain rule

The **partial derivative** $\partial J/\partial \theta_j$ is the rate of change along one coordinate; the **gradient** $\nabla J = (\partial J/\partial \theta_1, \ldots)$ points in the direction of **steepest ascent**, with magnitude equal to that steepest slope. Why: the directional derivative along unit $\mathbf{u}$ is $\nabla J^\top \mathbf{u} = \|\nabla J\| \cos\theta$, maximized when $\mathbf{u}$ aligns with ∇J. The gradient is perpendicular to the level sets (contours) of J.

Chain rule (multivariate): for $J(\theta) = L(g(h(\theta)))$, $\frac{\partial J}{\partial \theta} = \frac{\partial L}{\partial g} \cdot \frac{\partial g}{\partial h} \cdot \frac{\partial h}{\partial \theta}$ — a product of Jacobians. Backpropagation (Part 20) is the chain rule evaluated right-to-left so that every product is a cheap vector–matrix product.

Real-life analogy: a mixing desk and a relay of blame

Think of it like a sound engineer at a mixing desk with hundreds of knobs. A **partial derivative** asks: "if I turn just this one knob a little, how much louder does the song get?" The **gradient** is the list of answers for every knob — together they point in the direction that increases loudness fastest. The **chain rule** is a relay of blame: the final error is passed back through each stage (speaker ← amplifier ← mixer ← knob), multiplying how sensitive each stage is to the one before.

Formula, read aloud: $\frac{\partial J}{\partial \theta} = \frac{\partial L}{\partial g} \cdot \frac{\partial g}{\partial h} \cdot \frac{\partial h}{\partial \theta}$ says: effect of the knob on the final result = product of the effects along the chain.

Where it breaks: knobs on a desk hardly interact; in a network, changing one weight changes how every other weight matters, so the gradient must be recomputed after every step.

14.4 Gradient descent

Why the negative gradient, and how big a step?

First-order Taylor expansion: $J(\theta + \Delta) \approx J(\theta) + \nabla J^\top \Delta$. Among steps of fixed length, $\Delta \propto -\nabla J$ decreases J fastest. So

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

The step size is limited by **curvature**. If ∇J is L-Lipschitz (L = largest Hessian eigenvalue, the "smoothness"), then with $\eta \le 1/L$: $J(\theta_{t+1}) \le J(\theta_t) - \frac{\eta}{2}\|\nabla J(\theta_t)\|^2$ — **guaranteed descent**.

Real-life analogy: walking down a hill in fog

Think of it like walking down a foggy hill to reach the valley. You can't see the valley, but you can feel which way the ground slopes under your feet. Each step, you move a little in the steepest downhill direction. Step too big and you overshoot and climb the opposite slope; too small and night falls before you arrive.

Formula, read aloud: $\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$ says: new position = old position − (step size) × (the uphill direction).

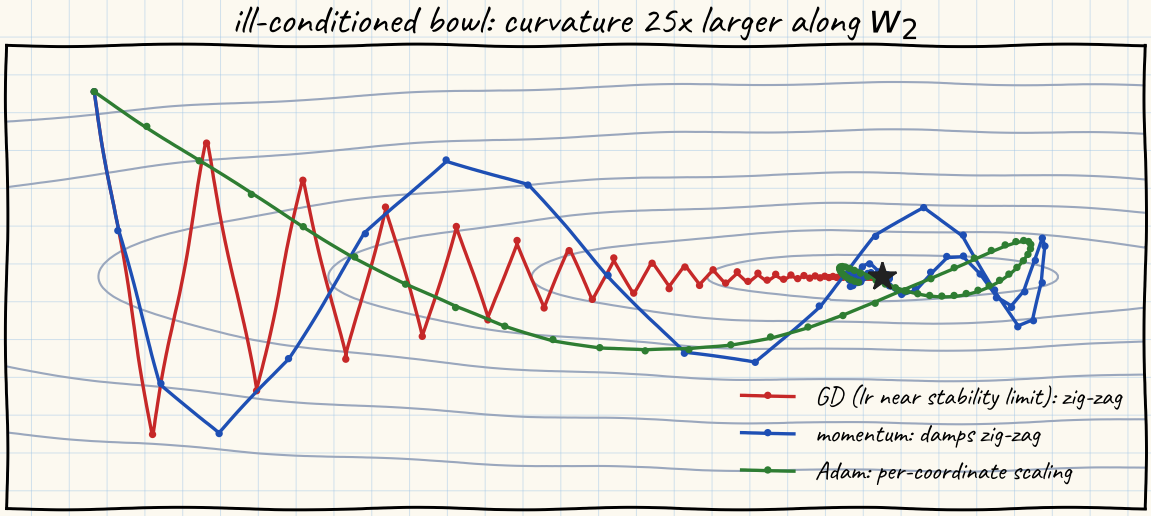
Where it breaks: a real hill has two directions; a loss surface has millions, and long flat plains or narrow ravines make the "feel the slope" strategy slow.

The quadratic model explains almost everything. Near a minimum, $J \approx \frac{1}{2}(\theta - \theta^*)^\top H (\theta - \theta^*)$. In the eigenbasis of H (eigenvalues λ_i), GD acts on each coordinate independently:

$$e_i^{(t+1)} = (1 - \eta\lambda_i)\, e_i^{(t)}$$

- **Stability:** need $|1 - \eta\lambda_i| < 1$ for all $i \Rightarrow \eta < 2/\lambda_{\max}$. Exceed it and the steepest direction **oscillates and diverges** (loss → ∞ → NaN).
- **Speed:** the flattest direction shrinks by $(1 - \eta\lambda_{\min})$ per step; with $\eta \approx 1/\lambda_{\max}$ that's $(1 - 1/\kappa)$, so the number of iterations grows linearly with the **condition number** $\kappa = \lambda_{\max}/\lambda_{\min}$.
- **The picture:** a long narrow valley. The learning rate must be small enough for the steep walls, which makes progress along the valley floor painfully slow — the zig-zag.

Convergence rates (for reference): convex and L-smooth: $J(\theta_t) - J^* = O(1/t)$; μ-strongly convex: linear, $O\big((1 - \mu/L)^t\big)$.



On an ill-conditioned bowl, plain GD zig-zags; momentum and Adam fix it in different ways

14.5 SGD and mini-batch SGD

Computing ∇J over all n examples per step is wasteful when n is huge and examples are redundant. **Stochastic gradient descent** uses one example (or a mini-batch B):

$$g_t = \frac{1}{|B|}\sum_{i\in B}\nabla \ell_i(\theta_t), \qquad \mathbb{E}[g_t] = \nabla J(\theta_t), \qquad \mathrm{Var}(g_t) \approx \frac{\sigma_g^2}{|B|}$$

Real-life analogy: tasting a spoonful instead of the whole pot

Think of it like a cook checking the salt in a huge pot of sambar. Tasting the whole pot each time is impossible; one spoonful gives a quick, slightly noisy estimate. A bigger spoon (larger batch) gives a more reliable taste but takes longer. Many quick spoonfuls usually get the seasoning right sooner than a few full-pot tastings.

Formula, read aloud: $g_t = \frac{1}{|B|}\sum_{i\in B}\nabla\ell_i$ says: estimate the slope from a small batch; on average it's the true slope ($\mathbb{E}[g_t] = \nabla J$), and its noise shrinks as the batch grows ($\mathrm{Var} \propto 1/|B|$).

Where it breaks: a spoonful of a well-stirred pot is a fair sample; if your batches aren't shuffled (e.g. sorted by class), the "spoonful" is biased and training can go badly wrong.

Unbiased but noisy. Consequences:

1. **Cheap steps, many of them:** you make progress after seeing a few hundred examples, not all n — dramatically faster wall-clock convergence on large redundant data.
2. **Noise floor:** with a constant learning rate SGD doesn't converge to the minimum; it bounces in a region whose size scales with $\eta\sigma_g^2/B$. To converge you **decay** the learning rate (classically $\sum_t \eta_t = \infty$, $\sum_t \eta_t^2 < \infty$). Convex rate: $O(1/\sqrt{t})$.
3. **Noise can help:** it kicks the iterate off saddle points and out of sharp basins, and acts as an **implicit regularizer**.

Batch size trade-offs.

Small batches (e.g. 16–128)	Large batches (e.g. 4K–64K)
noisy gradients; more updates per epoch	accurate gradients; fewer updates per epoch
poor hardware utilization	high GPU/TPU throughput, fewer synchronization steps
implicit regularization; often good generalization	can generalize worse at a fixed epoch budget unless LR/schedule are adjusted
smaller stable learning rate	tolerates larger LR — **linear scaling rule**: multiply LR by k when batch grows by k, with warmup (works up to a "critical batch size", beyond which bigger batches stop reducing the number of steps)

	Batch GD	SGD (B=1)	Mini-batch GD
Gradient per step	exact	very noisy	moderately noisy
Cost per step	$O(nd)$	$O(d)$	$O(Bd)$
Hardware	vectorized but huge	can't vectorize	best of both
Convergence	smooth, deterministic	erratic, needs decay	the standard
Use	small data, convex problems, L-BFGS	online/streaming learning	almost everything in deep learning

14.6 Momentum

$$\mathbf{v}_{t+1} = \beta\,\mathbf{v}_t + g_t, \qquad \theta_{t+1} = \theta_t - \eta\,\mathbf{v}_{t+1} \qquad (\beta \approx 0.9)$$



Real-life analogy: a heavy ball rolling downhill

Think of it like rolling a heavy iron ball instead of walking. On a zig-zagging ravine, the side-to-side pushes cancel out because the ball's weight smooths them, while the steady downhill push keeps adding speed. The ball reaches the bottom faster and wobbles less than a person taking each step fresh.

Formula, read aloud: $\mathbf{v}_{t+1} = \beta\mathbf{v}_t + g_t$ says: new speed = 90% of old speed + today's push; then move by the speed.

Where it breaks: a real ball has friction and physical limits; with β too high, the "ball" overshoots and oscillates around the minimum for a long time.

Behavior, not just the equation. The velocity is an exponentially weighted sum of past gradients. In the **steep, oscillating** direction, gradients alternate sign and **cancel** in the average — oscillation is damped. In the **shallow, consistent** direction, gradients point the same way and **accumulate** — effective step size grows toward $\eta/(1-\beta)$ (10× for $\beta = 0.9$). A heavy ball rolling down the valley: it doesn't bounce between the walls as much and it builds speed along the floor. On quadratics, optimally tuned momentum improves the iteration count from $O(\kappa)$ to $O(\sqrt{\kappa})$. Momentum also averages out mini-batch noise.

14.7 Nesterov momentum

Evaluate the gradient at the **look-ahead** point where momentum is about to carry you:

$$\mathbf{v}_{t+1} = \beta\,\mathbf{v}_t + \nabla J(\theta_t - \eta\beta\,\mathbf{v}_t), \qquad \theta_{t+1} = \theta_t - \eta\,\mathbf{v}_{t+1}$$

If momentum is about to overshoot, the look-ahead gradient already points back, applying the brakes one step earlier. Provably optimal rate $O(1/t^2)$ for smooth convex problems (vs. $O(1/t)$ for GD); in deep learning, a modest but consistent improvement over classical momentum.



Real-life analogy: braking before the speed breaker

Think of it like a scooter rider who looks where momentum will carry them *next* and brakes early if there's a speed breaker ahead — instead of feeling the bump and braking afterwards. Nesterov computes the slope at the look-ahead point, so it corrects overshoots one step earlier.

Formula, read aloud: $\nabla J(\theta_t - \eta\beta\mathbf{v}_t)$: "check the slope at where I'm about to be, not where I am".

Where it breaks: the look-ahead is only one step and only uses the slope, so it helps modestly in deep learning — it isn't true foresight.

14.8 AdaGrad, RMSProp, Adam

AdaGrad — per-parameter learning rates from accumulated squared gradients:

$$G_t = G_{t-1} + g_t^2, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon}\, g_t$$

Rare features (sparse gradients) keep large effective learning rates; frequent ones get small rates — excellent for sparse linear models (text, ads). Flaw: G_t only grows, so the learning rate **decays to zero**, stalling long non-convex training.

RMSProp — replace the sum with an exponential moving average, so the scale adapts to *recent* gradients: $v_t = \rho v_{t-1} + (1-\rho) g_t^2$, $\theta_{t+1} = \theta_t - \eta g_t / (\sqrt{v_t} + \epsilon)$.

Adam — RMSProp's per-parameter scaling + momentum on the gradient + bias correction:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t && \text{(first moment: direction)} \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 && \text{(second moment: scale)} \\ \hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} && \text{(bias correction)} \\ \theta_{t+1} &= \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} && (\beta_1 = 0.9,\ \beta_2 = 0.999,\ \epsilon = 10^{-8}) \end{aligned}$$

How Adam behaves.

- **Per-coordinate normalization:** $\hat{m}/\sqrt{\hat{v}}$ is roughly (mean gradient)/(RMS gradient), a dimensionless number of magnitude $\lesssim 1$. So each parameter moves about η per step **regardless of its gradient's scale** — it automatically handles parameters with very different curvature/gradient magnitudes (embeddings vs. layer norms vs. output layers), which is a crude diagonal preconditioner for ill-conditioning.
- **Signal-to-noise sensitivity:** when gradients are consistent, $|\hat{m}|/\sqrt{\hat{v}} \approx 1$ (full step); when they're noisy and cancel, the ratio shrinks (cautious step). Adam behaves somewhat like sign-SGD with momentum.
- **Why bias correction:** $m_0 = v_0 = 0$, so early EMAs are biased toward zero; v (with $\beta_2 = 0.999$) is especially small in the first ~1000 steps. Without correction, early steps would be mis-scaled. Even with it, early $\hat{v}$ is noisy — one reason for **learning-rate warmup**.
- **Failure modes:** ϵ matters when gradients are tiny (acts as a floor on the denominator); Adam can be unstable with too large η or when $\hat{v}$ collapses; L2 regularization interacts badly with it (next).

Real-life analogy: shoes adjusted for each terrain

Think of it like a trekker who adjusts stride separately for each kind of ground: short careful steps on loose scree (directions with large, jumpy gradients), long strides on smooth flat paths (directions with small, steady gradients). Adam keeps a running average of the direction (momentum) and of how bumpy each direction has been, and divides the step by that bumpiness.

Formula, read aloud: $\theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}$ says: step = learning rate × (smoothed direction) ÷ (typical size of recent gradients in that coordinate).

Where it breaks: a trekker judges terrain sensibly; Adam's per-coordinate scaling can over-trust noisy history, which is one reason plain SGD sometimes generalizes better.

14.9 AdamW and weight decay

Weight decay: shrink weights multiplicatively each step, $\theta \leftarrow (1-\eta\lambda)\theta - \eta\,\text{update}$. For plain SGD this is identical to adding $\frac{\lambda}{2}\|\theta\|^2$ to the loss.

Real-life analogy: a fixed monthly savings cut vs. a cut that depends on mood

Think of it like two ways to shrink your spending. **Weight decay** is a fixed rule: every month, cut every expense by 1%. **L2-in-the-loss with Adam** is like letting the cut depend on how volatile each expense has been — volatile items barely get cut, steady ones get cut hard. AdamW restores the fixed, fair cut by applying the shrinkage separately from Adam's scaling.

Formula, read aloud: $\theta \leftarrow \theta - \eta(\text{Adam step} + \lambda\theta)$ says: take the usual Adam step, and additionally shrink every weight by a fixed fraction.

Where it breaks: money cuts are voluntary; in training, the "right" decay also interacts with the learning-rate schedule, so λ still has to be tuned.

For Adam it is not. If you add L2 to the loss, the penalty gradient $\lambda\theta$ gets **divided by** $\sqrt{\hat{v}}$ like every other gradient: parameters with large historical gradients are barely regularized, those with small gradients are over-regularized — not what "prefer small weights" means. **AdamW** (Loshchilov & Hutter) **decouples** decay from the adaptive step:

$$\theta_{t+1} = \theta_t - \eta\Big(\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon} + \lambda\,\theta_t\Big)$$

Every weight shrinks at the same relative rate. AdamW generalizes better than Adam+L2 and is the default for transformers. (Common practice: no decay on biases and normalization gains.)

```python
import numpy as np

class SGD:
    def __init__(self, lr=0.01, momentum=0.0, nesterov=False, weight_decay=0.0):
        self.lr, self.mu, self.nesterov = lr, momentum, nesterov
        self.wd, self.v = weight_decay, None

    def step(self, w, g):
        if self.wd:
            g = g + self.wd * w                      # L2-as-gradient (== decay for plain SGD)
        if self.v is None:
            self.v = np.zeros_like(w)
        self.v = self.mu * self.v + g                # heavy-ball velocity
        upd = g + self.mu * self.v if self.nesterov else self.v   # Nesterov (look-ahead form)
        return w - self.lr * upd
```

```python
class Adam:
    def __init__(self, lr=1e-3, betas=(0.9, 0.999), eps=1e-8, weight_decay=0.0, decoupled=True):
        self.lr, (self.b1, self.b2), self.eps = lr, betas, eps
        self.wd, self.decoupled = weight_decay, decoupled      # decoupled=True → AdamW
        self.m = self.v = None
        self.t = 0

    def step(self, w, g):
        if self.m is None:
            self.m, self.v = np.zeros_like(w), np.zeros_like(w)
        if self.wd and not self.decoupled:
            g = g + self.wd * w                        # "Adam + L2": decay rescaled by 1/sqrt(v)
        self.t += 1
        self.m = self.b1 * self.m + (1 - self.b1) * g          # 1st moment
        self.v = self.b2 * self.v + (1 - self.b2) * g * g      # 2nd moment
        m_hat = self.m / (1 - self.b1 ** self.t)               # bias correction
        v_hat = self.v / (1 - self.b2 ** self.t)
        upd = m_hat / (np.sqrt(v_hat) + self.eps)
        if self.wd and self.decoupled:
            upd = upd + self.wd * w                    # AdamW: decay acts on weights directly
        return w - self.lr * upd
```

Check: on $f(\mathbf{w}) = \frac{1}{2}\mathbf{w}^\top \operatorname{diag}(1, 100)\mathbf{w}$ from $(1, 1)$ for 300 steps at $\eta = 0.01$: GD reaches 1.2×10^{-3}, momentum 1.2×10^{-12}, Nesterov 6×10^{-16}, Adam ($\eta = 0.05$) 4×10^{-13} — the ill-conditioning story in four numbers.

14.10 Learning rate — the most important hyperparameter



Loss curves diagnose the learning rate

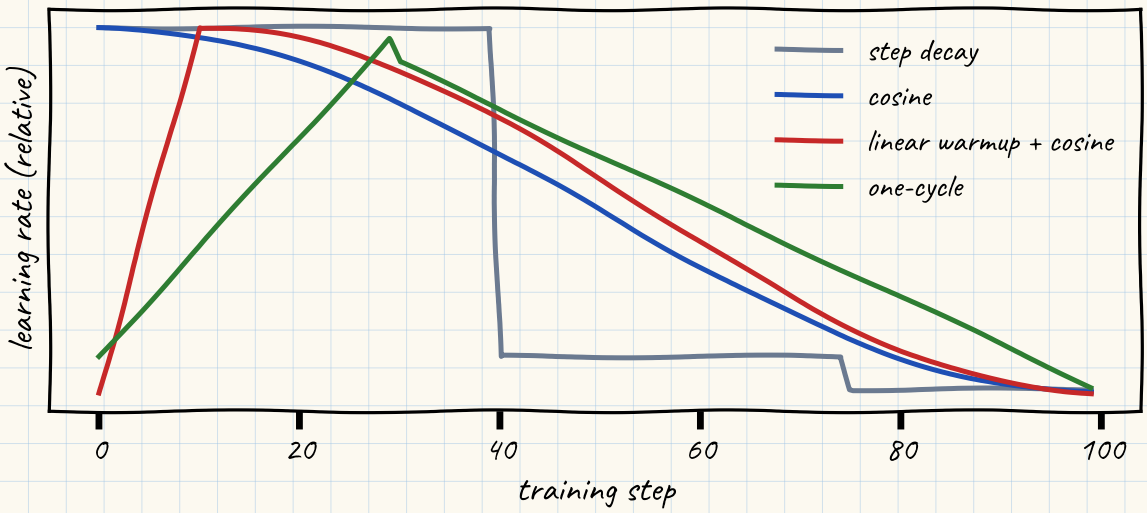
Finding it: an **LR range test** — increase the LR exponentially over a few hundred steps and plot loss; pick a value somewhat below where the loss is falling fastest (well before it blows up). Sweep on a log scale (1e-5 … 1e-1). When you change batch size, architecture width, or optimizer, re-tune it. Typical ranges: SGD+momentum 0.01–0.1 (vision, with BN); AdamW 1e-4–1e-3 (smaller for larger models and for fine-tuning pre-trained models: ~1e-5–5e-5).

Real-life analogy: the accelerator pedal on a ghat road

Think of it like driving down a winding ghat road. Press the accelerator too hard (high learning rate) and you fly off at the first turn — the loss explodes. Too gently (low learning rate) and you're still driving at midnight. The right speed depends on the road's curves (the loss surface's curvature), so a new road (new model, new batch size) means re-finding the right pressure.

Where it breaks: a driver can see the next turn; an optimizer only feels the current slope, which is why we use range tests and schedules rather than intuition.

14.11 Learning-rate schedules



Common schedules

Schedule	What it does	Why it helps
Step decay	×0.1 at fixed epochs	large LR explores; small LR settles into the minimum (removes the SGD noise floor)
Exponential	×γ each epoch	smooth version of the above
Cosine	$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos\frac{\pi t}{T})$	smooth, few hyperparameters, strong default
Linear warmup	ramp from ~0 to $\eta_{\max}$ over the first ~1–5% of steps	early gradients and Adam's $\hat{v}$ estimates are unreliable; random init + large LR can blow up activations (especially in transformers / large-batch training)
One-cycle	up then down, with momentum inversely	fast convergence; big LR phase regularizes
Reduce on plateau	decay when validation stalls	adaptive; good when you don't know the budget
Warmup–stable–decay	constant after warmup, decay at the end	lets you extend or checkpoint training runs flexibly

Real-life analogy: parking a car

Think of it like parking: you drive fast on the main road, slow down as you enter the lane, and inch forward as you park. Warmup is easing off the handbrake gently at the start; decay (step, cosine) is slowing down as you get close so you settle exactly into the spot instead of bouncing around it.

Where it breaks: you can see the parking spot; the optimizer can't, so schedules are set in advance by steps or epochs, or adjusted when validation stops improving.

14.12 Gradient clipping

By global norm: if $\|g\| > c$, set $g \leftarrow g \cdot \frac{c}{\|g\|}$. Keeps the **direction**, caps the step size — a safety valve against rare huge gradients (exploding gradients in RNNs, loss spikes in transformers, outlier batches). **By value:** clamp each coordinate to $[-c, c]$ — changes the direction; less common. Clipping treats symptoms: if it triggers constantly, fix the cause (LR, initialization, normalization, bad data).

Real-life analogy: a speed governor on a bus

Think of it like a speed governor fitted to a school bus: whatever the driver does, the bus can't exceed 60 km/h, but it still goes in the direction the driver steers. Clipping by norm caps how far one step can move the weights, while keeping the direction.

Formula, read aloud: if $\|g\| > c$, use $g \cdot \frac{c}{\|g\|}$: "if the push is too strong, shrink it to size c, same direction".

Where it breaks: a governor fixes a symptom; frequent clipping means something else is wrong (learning rate, bad data, poor initialization) and should be investigated.

14.13 Saddle points and local minima

At a critical point ($\nabla J = 0$), the Hessian's eigenvalues classify it: all positive → local minimum; all negative → maximum; **mixed → saddle**. In d dimensions, requiring all d eigenvalues positive is a strong condition, so high-loss critical points are overwhelmingly saddles. Near a saddle the gradient is tiny, so GD **slows down** on plateaus; it only escapes along negative-curvature directions, slowly if their curvature is small. Gradient noise (SGD) and momentum help escape; Newton's method is actually **attracted** to saddles (it jumps to any critical point), one reason raw second-order methods are unpopular in deep learning.

Sharp vs. flat minima: a minimum where the loss rises steeply in some directions vs. a wide basin. A popular (debated) hypothesis links flat minima to better generalization — small perturbations of weights (or of the data distribution) change the loss little — and links SGD's noise and small batches to a preference for flatter regions. Methods like SAM (sharpness-aware minimization) explicitly seek flat regions.

Real-life analogy: a mountain pass

Think of it like standing on a mountain pass between two peaks: the ground is flat under your feet, it goes up toward both peaks, and down toward both valleys. You're at a flat point but not at the bottom. In high dimensions, most flat points on a loss surface are passes like this (saddles), not true valleys — and the noise in SGD helps nudge you off them.

Where it breaks: on a real pass you can see which way is down; near a saddle in a huge network, the downhill directions may be few and the slope tiny, so progress stalls for a while.

14.14 Vanishing and exploding gradients

In a deep network, the gradient w.r.t. early layers is a **product** of per-layer Jacobians:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_L} \prod_{l=2}^{L} \frac{\partial h_l}{\partial h_{l-1}}, \qquad \frac{\partial h_l}{\partial h_{l-1}} = \operatorname{diag}\left(\phi'(z_l)\right) W_l$$



Real-life analogy: a whisper passed down a long line

Think of it like a message whispered down a line of 50 students. If each student speaks a little softer than they heard, by the end the message is inaudible (vanishing). If each speaks a little louder, it becomes a shout (exploding). The gradient passes back through every layer, getting multiplied at each one.

Formula, read aloud: the gradient at layer 1 = the gradient at the top × the product of every layer's "volume factor" in between.

Where it breaks: students can correct their volume; networks need design fixes — ReLU-type activations, careful initialization, normalization, residual connections — to keep the volume factor near 1.

If the typical gain per layer is below 1, the product shrinks exponentially (**vanishing** — early layers stop learning); above 1, it grows exponentially (**exploding** — NaNs). Sources: saturating activations ($\sigma' \leq 0.25$ — ten sigmoid layers can shrink gradients by $\sim 0.25^{10} \approx 10^{-6}$), poorly scaled initial weights, and recurrence (the same W multiplied T times: spectral radius > 1 explodes, < 1 vanishes).

Fix	Mechanism
ReLU-family activations	derivative 1 on the active side, no saturation
Xavier / He initialization	choose weight variance so activation and gradient variance stay ~constant across layers (Part 20)
BatchNorm / LayerNorm	re-standardize activations each layer, keeping them in a healthy range
Residual connections	$h_{l+1} = h_l + F(h_l)$ ⇒ Jacobian $I + \partial F/\partial h$: an identity path lets gradients flow unattenuated
LSTM/GRU gating	additive cell-state updates with learned gates instead of repeated multiplication
Gradient clipping	caps explosions (doesn't fix vanishing)

14.15 Second-order methods (what to say about them)

Newton: $\theta \leftarrow \theta - H^{-1}\nabla J$ — rescales every direction by its curvature, so conditioning doesn't matter and convergence is quadratic near the optimum. Costs $O(d^2)$ memory and $O(d^3)$ per step — impossible for millions of parameters — and it's attracted to saddles in non-convex problems. **Quasi-Newton (L-BFGS)** builds a low-rank Hessian approximation from recent gradients — excellent for smooth, full-batch, convex or small problems (logistic regression, CRFs), poor with noisy mini-batch gradients. **Natural gradient / K-FAC / Shampoo** use structured curvature approximations and see use in large-scale training. Adam can be seen as a cheap diagonal approximation of this idea.

Real-life analogy: a GPS that knows the road's curves

Think of it like the difference between a driver who only feels the slope (first-order) and a GPS that also knows how sharply the road curves ahead (second-order). The GPS can plan a perfect, direct route — but computing it for a city with millions of roads every second is too expensive, so we mostly drive by feel with smart approximations.

Where it breaks: a GPS map is fixed; the loss surface changes with every mini-batch, so curvature estimates are noisy — another reason full second-order methods are rarely used for deep nets.

14.16 Why might Adam converge faster but SGD generalize better?

The mechanism behind a famous interview question

Why Adam is faster: its per-parameter normalization is a diagonal preconditioner. Ill-conditioned problems — where some parameters see huge gradients and others tiny ones (embeddings, sparse features, attention/output layers) — make SGD's single learning rate a compromise that's too big for some directions and too small for others. Adam gives every coordinate a comparable step, so early progress is fast and it's much less sensitive to the learning rate.

Why SGD can generalize better (in some settings — notably image classification with CNNs; Wilson et al., 2017):

1. **Different implicit bias.** On under-determined problems there are many zero-training-loss solutions; *which* one you reach depends on the optimizer. For linear least squares, GD/SGD from zero converges to the **minimum-L2-norm** solution — a good inductive bias. Adaptive methods rescale coordinates, so their solution depends on those rescalings and can have a larger norm and generalize worse (Wilson et al. built examples where adaptive methods generalize arbitrarily badly).
2. **Noise and sharpness.** SGD's gradient noise is anisotropic and tied to the loss curvature; together with larger effective step sizes it tends to settle in flatter regions. Adam's normalization dampens exactly the noise that pushes SGD out of sharp basins.
3. **Regularization mismatch.** Adam + L2 regularizes unevenly (above) — a chunk of the historical "Adam generalizes worse" gap disappears with **AdamW** and a proper schedule.

What a strong candidate adds: it's setting-dependent. For transformers and large language models, AdamW is the standard and SGD trains them poorly (their heterogeneous, heavy-tailed gradient scales across parameter blocks make adaptivity essential). For ResNets on ImageNet, well-tuned SGD+momentum with a schedule often matches or beats Adam. Practical recipe: start with AdamW for speed and robustness; if you have tuning budget on a vision-style problem, try SGD+momentum+cosine and compare on validation.

Real-life analogy: the express train vs. the walking tour

Think of it like reaching a town by express train (Adam) versus walking (SGD). The train gets you there quickly, but it may drop you at a crowded platform in a narrow valley. The walker, bumping around more slowly, often ends up settled in a wide, calm plain — a flatter minimum that is more forgiving when conditions change (new data).

Where it breaks: this is a tendency, not a law — with good tuning (AdamW, schedules), adaptive methods generalize very well, and they are the default for transformers.

Q: "Your loss goes to NaN after 2,000 steps. What do you check, in order?"

Strong answer. "First reproduce and find the step: log the loss, gradient norm and max activation per layer and see whether the gradient norm climbs before the NaN — that points to divergence from too high a learning rate, missing warmup, or exploding gradients. Then numerical issues: log of zero or division by zero in a custom loss, softmax without the max-subtraction, fp16 overflow — check the loss scaler if using mixed precision. Then data: a NaN or extreme value in an input batch or label — assert finiteness in the loader, and check if NaNs recur at the same batch. Fixes by cause: lower LR or add warmup, gradient clipping, compute losses from logits, bf16 or loss scaling, clean the data."

Follow-ups

"Why does increasing the batch size often require increasing the learning rate?" — *The gradient estimate is less noisy, so you can take bigger steps safely; with the same LR, you'd make fewer updates per epoch and converge more slowly per epoch. Linear scaling holds approximately up to a critical batch size.*

"What does $\beta_2 = 0.999$ mean intuitively?" — *The second-moment estimate averages over roughly the last $1/(1-\beta_2) = 1000$ steps: a slow, stable estimate of each parameter's gradient scale.*

"Can you use momentum with Adam?" — *Adam already has momentum: β_1 is its momentum coefficient on the gradient. (Nadam adds Nesterov-style look-ahead.)*

14.17 Batch size, learning-rate scaling and warmup

Larger batches give less noisy gradients, so you can take bigger steps — up to a point.

- **Linear scaling rule** (SGD, Goyal et al., 2017): multiply the batch size by k → multiply the learning rate by k, with a **warmup** of a few epochs so early, badly-conditioned steps don't diverge. It held up to batch sizes of several thousand images for ResNets.
- For Adam-type optimizers, a **square-root** scaling ($\eta \propto \sqrt{k}$) is a common starting point; always re-tune.
- **Critical batch size:** beyond a problem-dependent size (related to the "gradient noise scale" — how noisy per-example gradients are relative to the mean gradient), doubling the batch no longer halves the number of steps; you just spend more compute per step. The critical batch size tends to **grow during training** as the loss falls — one reason large-scale runs ramp the batch size up.
- **Why warmup:** at initialization, gradients and Adam's second-moment estimates are unreliable; a few hundred to a few thousand steps of linearly increasing LR prevent early blow-ups, especially in transformers.



Real-life analogy: a bigger survey allows a bolder decision

Think of it like a shop owner changing prices based on customer surveys. Asking 10 customers gives a noisy answer, so you change prices cautiously. Asking 1,000 gives a reliable answer, so you can make a bigger change at once. But beyond some number of customers, asking more barely changes the answer — you're just spending more time per decision (the critical batch size). Warmup is not trusting the first few noisy surveys too much.

Where it breaks: a survey's reliability is easy to compute; a network's "right" learning rate also depends on curvature, which changes during training, so scaling rules are starting points, not laws.

14.18 Optimizers for the large-model era

Adam(W) remains the default, but memory and scale pushed several alternatives into practice. Know what each changes and why:

Optimizer	Key idea	Why people use it	Watch out for
Adafactor	store row and column averages of the squared gradient instead of a full per-parameter second moment ("factored" $\hat{v}$)	optimizer memory $O(n+m)$ per $n \times m$ matrix instead of $O(nm)$; used to train large text-to-text models	needs update clipping / relative step sizes for stability
Lion	update $= \eta \cdot \text{sign}(\beta_1 m + (1-\beta_1)g)$; keeps only momentum	half of Adam's optimizer memory; every coordinate moves by the same amount	needs a 3–10× smaller learning rate and larger weight decay than AdamW
Shampoo	precondition each weight matrix with Kronecker-factored curvature estimates ($L^{-1/4}GR^{-1/4}$)	second-order-like conditioning at manageable cost; strong results in optimizer benchmarks	extra compute for matrix roots; engineering complexity in distributed settings
Muon	apply momentum, then **orthogonalize** each 2-D weight update (Newton–Schulz iterations) so all directions of the update have equal size	reported faster training for transformer hidden layers; adopted in some recent large runs	used for matrix-shaped weights only (embeddings, norms and heads usually stay on AdamW); newer, evidence still accumulating
Schedule-free variants	replace the LR schedule with iterate averaging	no need to fix the training length in advance	still needs warmup and tuning

Interview-safe summary: "AdamW with warmup and cosine decay is the robust default. Memory-saving (Adafactor, Lion, 8-bit optimizer states) and better-conditioned (Shampoo, Muon) alternatives can train faster or cheaper, but gains depend on careful tuning, and I'd validate them against a well-tuned AdamW baseline."



Real-life analogy: different gearboxes for the same car

Think of it like fitting different gearboxes to the same car. AdamW is the dependable automatic most drivers use. Adafactor is a lighter gearbox that saves weight (memory). Lion is a simple two-speed box: every shift is the same size — cheap, but you must drive more gently (smaller learning rate). Shampoo and Muon are sophisticated boxes that match the gear to the road's shape (curvature), getting more power out of each rotation — at the cost of complexity.

Formula, read aloud: Lion's update $\text{sign}(\cdot)$ keeps only the *direction* of each coordinate's push (+1 or −1), throwing away its size.

Where it breaks: gearboxes are chosen once per car; with optimizers, the best choice changes with model, data and scale, and published speed-ups often shrink against a well-tuned baseline.

14.19 Sharpness-aware minimization (SAM)

SAM looks for weights whose **neighbourhood** has low loss, not just a low point:

$$\min_{\mathbf{w}} \max_{\|\boldsymbol{\epsilon}\| \le \rho} L(\mathbf{w}+\boldsymbol{\epsilon}) \qquad \text{approximated by} \qquad \hat{\boldsymbol{\epsilon}} = \rho \frac{\nabla L(\mathbf{w})}{\|\nabla L(\mathbf{w})\|}, \quad \mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w}+\hat{\boldsymbol{\epsilon}})$$

Each step: compute the gradient, take a small step **uphill** to the worst nearby point, compute the gradient there, and use it to update the original weights. Cost: **two** forward–backward passes per step. It often improves generalization and robustness to label noise in vision; gains elsewhere are mixed, and the flat-minima explanation is debated (Part 14, saddle points and minima).



Real-life analogy: pitching a tent in the safest spot

Think of it like choosing where to pitch a tent. A narrow hollow may be the lowest point, but a small shift in wind or ground and you slide out. A wide, flat meadow is slightly higher but stays comfortable even if you're a bit off. SAM checks "what's the worst spot within a step of here?" and moves to places where even the worst nearby spot is good.

Formula, read aloud: minimize (the highest loss you could reach by nudging the weights a distance ρ) — "be good even in the worst nearby case".

Where it breaks: a meadow is flat in every direction; a network's "flatness" depends on how you measure it (reparameterizing the weights can change sharpness without changing the function), which is why the theory is debated.

PART 15

Loss Functions

How to think about any loss

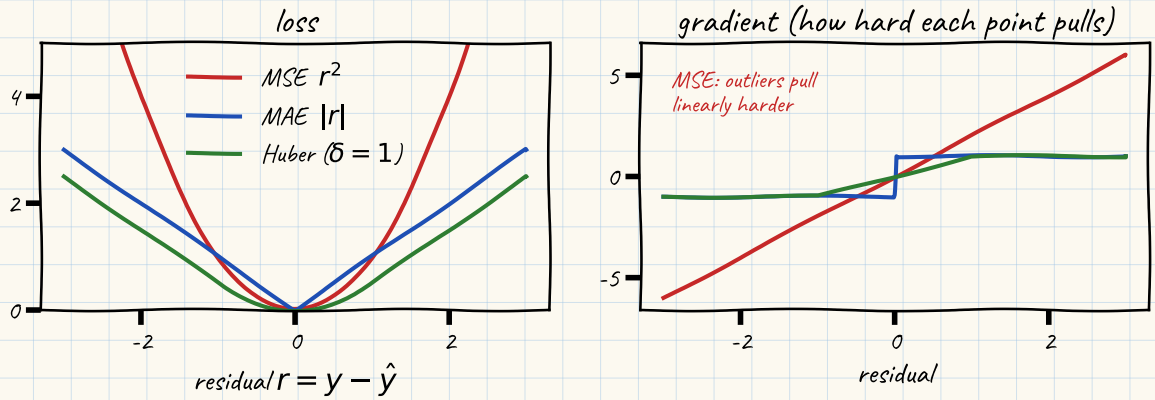
Ask four questions: **(1)** What noise model or goal does it encode (what does its minimizer predict — mean, median, a probability, a ranking)? **(2)** How does its **gradient** behave for small, large and wrong errors — who gets to pull on the model? **(3)** How does it treat **outliers**? **(4)** How does it **fail**?

The whole Part as one picture

Think of it like a school choosing a rule for late arrivals. Charge the **square** of the minutes late, and one student stuck in a two-hour traffic jam dominates the whole register (MSE). Charge **per minute**, and every minute counts the same (MAE). Charge only past a five-minute grace period, and punctual students are ignored completely (hinge). Each rule decides what "typical" arrival it rewards (mean, median), who gets the teacher's attention (the gradient), how one freak case is treated (outliers), and how it can backfire (failure modes).

Where it breaks: students react to a rule by understanding it; a model only feels the gradient, and it will exploit any gap between the loss and what you actually wanted.

15.1 Regression losses



MSE, MAE and Huber — and, more importantly, their gradients

MSE (squared error)

$$\ell = (y - \hat{y})^2, \qquad \frac{\partial \ell}{\partial \hat{y}} = -2(y - \hat{y})$$

- **Predicts:** the conditional **mean**. MLE under Gaussian noise.
- **Gradient:** proportional to the residual — large errors pull hard, small errors pull gently, so it converges smoothly near the optimum.
- **Outliers:** very sensitive — a residual of 10 carries 100× the loss and 10× the gradient of a residual of 1. A few bad labels can dominate training.

- **Use when:** errors are roughly symmetric and light-tailed, and large errors are genuinely much worse than small ones (costs grow quadratically).
- **Failure modes:** heavy-tailed targets (revenue, durations) → the model chases outliers; skewed targets → predicts a mean that no typical example has; different target scales across segments → big-scale segments dominate.

MAE (absolute error)

$$\ell = |y - \hat{y}|, \qquad \frac{\partial \ell}{\partial \hat{y}} = -\operatorname{sign}(y - \hat{y})$$

- **Predicts:** the conditional **median**. MLE under Laplace noise.
- **Gradient:** constant magnitude — every point pulls equally hard regardless of how wrong it is. Robust, but no signal about *how* close you are: with a fixed learning rate, the iterate keeps overshooting near the optimum (needs LR decay); non-differentiable at 0.
- **Outliers:** robust (breakdown point 50%).
- **Use when:** outliers or heavy tails, and cost is linear in the error (e.g. absolute forecast error in units).
- **Failure modes:** slow/unstable final convergence; for targets with many zeros (intermittent demand), the median can be 0 — the model predicts 0 everywhere and looks "good".

Huber

$$\ell_\delta(r) = \begin{cases} \frac{1}{2}r^2 & |r| \le \delta \\ \delta\left(|r| - \frac{1}{2}\delta\right) & |r| > \delta \end{cases}, \qquad \frac{\partial \ell}{\partial \hat{y}} = -\operatorname{clip}(r, -\delta, \delta)$$

- **Behavior:** quadratic for small residuals (smooth convergence), linear for large ones (bounded pull from outliers). The gradient is the residual **clipped** at δ.
- **Use when:** mostly Gaussian-looking errors with occasional outliers (sensor glitches, bounding-box regression — "smooth L1" is Huber with $\delta = 1$).
- **Failure modes:** δ is a hyperparameter in target units — scale-dependent; too small ≈ MAE, too large ≈ MSE. Pseudo-Huber and log-cosh are smooth alternatives.

Real-life analogy: where to put the chai stall

Think of it like choosing one spot for a chai stall on a long street where your regulars live. **MSE** charges you the square of each customer's walk: a regular 10 km away counts 100 times as much as one 1 km away, so the stall drifts toward him (the **mean**). **MAE** charges plain walking distance: the best spot is the middle customer's house (the **median**), and it doesn't budge if the far-off regular moves even further away. **Huber** charges squares for short walks and plain distance for long ones.

Formula, read aloud: Huber's gradient $-\operatorname{clip}(r, -\delta, \delta)$ says: each error pulls in proportion to its size, but never harder than δ.

Where it breaks: a stall has one location; a model picks a different "spot" for every input, the conditional mean or median given x.

Two more you should know

Quantile (pinball) loss for predicting the τ-quantile: $\ell_\tau(r) = \max(\tau r, (\tau - 1)r)$ with $r = y - \hat{y}$. $\tau = 0.5$ gives MAE/median; $\tau = 0.9$ penalizes under-prediction 9× more than over-prediction — used for **prediction intervals** and asymmetric-cost forecasting (inventory: running out is worse than overstock).

Poisson / Tweedie deviance for counts and zero-inflated non-negative targets (orders, claims) — models $\log \mathbb{E}[y]$ and respects that variance grows with the mean.

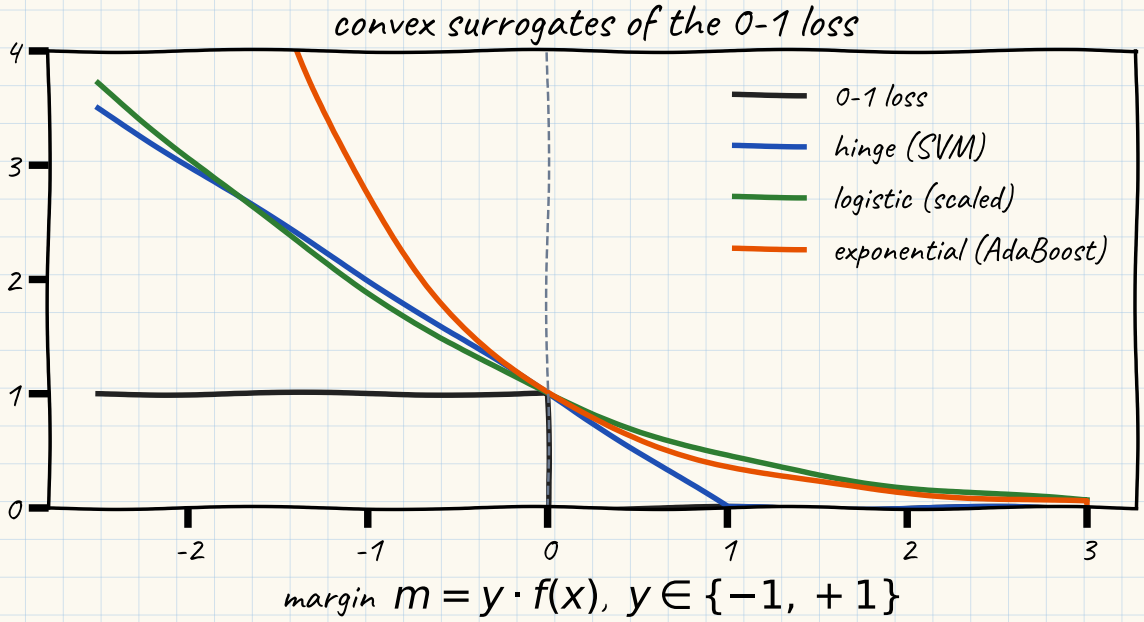
Real-life analogy: leaving early for the train

Think of it like deciding when to leave for a train. Reaching the station 20 minutes early costs a little boredom; missing the train costs a lot. So you don't plan for the *average* travel time; you plan for a bad-traffic day, roughly the 90th percentile. The pinball loss builds that lopsided cost into training.

Formula, read aloud: $\ell_\tau(r) = \max(\tau r, (\tau - 1)r)$ with $r = y - \hat{y}$ says: under-predict and pay τ per unit; over-predict and pay $1 - \tau$ per unit. With $\tau = 0.9$, falling short costs 9 times more than over-shooting.

Where it breaks: you plan one trip at a time, but each quantile usually needs its own model or output head, and separately trained quantiles can cross, giving a "90th" forecast below the "80th".

15.2 Classification losses



Classification losses as functions of the margin

Binary cross-entropy (log loss)

$$\ell = -\big[y \log p + (1 - y) \log(1 - p)\big],\ p = \sigma(z), \qquad \frac{\partial \ell}{\partial z} = p - y$$

- **Predicts:** calibrated probabilities (a **proper scoring rule**: expected loss is minimized by reporting the true probability). Bernoulli MLE.
- **Gradient:** $p - y$ in logit space — proportional to the probability error, never saturating: a confidently wrong prediction ($p \approx 0$, $y = 1$) gets the maximum gradient.
- **Outliers / label noise:** unbounded loss as $p \to 0$ for a positive — a **mislabeled** example with a confident (correct-in-reality) prediction produces a huge loss and gradient; noisy labels get memorized.
- **Use when:** binary classification or multi-label (one BCE per label) whenever you need probabilities.
- **Failure modes:** numerical ($\log 0 \to$ compute from logits); severe imbalance lets the easy majority dominate total loss; overconfidence when trained long; label noise.

Real-life analogy: a weather forecaster's report card

Think of it like grading a forecaster who says "70% chance of rain". If it rains, she's rewarded a bit; if she had said 99% and it rained, rewarded more. But if she said 99% and it stayed dry, she's punished *very* heavily. Cross-entropy is this report card: it punishes confident wrong answers enormously and hesitant ones mildly.

Formula, read aloud: $\ell = -[y \log p + (1-y)\log(1-p)]$ says: take the log of the probability you gave to what actually happened, and flip its sign. The gradient $p - y$ is simply "your forecast minus the truth".

Where it breaks: a forecaster can argue about the weather; the loss is unforgiving — mislabeled training examples get the same harsh punishment, which is why label noise hurts.

Categorical cross-entropy (softmax)

$$\ell = -\log p_y = -z_y + \log \sum_k e^{z_k}, \qquad \frac{\partial \ell}{\partial z_k} = p_k - \mathbb{1}[k = y]$$

- Same properties as BCE for mutually exclusive classes. The gradient pushes up the correct logit by $(1 - p_y)$ and pushes down every other logit by its probability — mistakes that absorb probability mass get pushed down hardest.
- **Label smoothing** (target $1 - \epsilon$ for the true class, $\epsilon/(K-1)$ for others) prevents logits from growing without bound, improves calibration and robustness to label noise; it can hurt knowledge distillation and tasks needing fine-grained confidence.
- **Failure modes:** many classes → softmax cost $O(K)$ per example (use sampled softmax, hierarchical softmax, or negative sampling); long-tailed class distributions → head classes dominate.

Real-life analogy: betting chips across many horses

Think of it like placing 100 chips across the horses in a race based on how likely each is to win. After the race, you're scored only on the chips you placed on the winner — the more chips there, the better. Softmax turns raw scores into a chip distribution; cross-entropy scores how many chips landed on the correct class.

Formula, read aloud: $\ell = -\log p_y$ says: the loss is minus the log of the share you bet on the right answer; its gradient $p_k - \mathbb{1}[k = y]$ says "take chips away from wrong horses, add them to the winner".

Where it breaks: betting has limited chips by rule; softmax enforces the same (probabilities sum to 1) — but it can be very confident about classes it has never seen, since it must spread chips over known classes only.

Hinge loss

$$\ell = \max(0,\ 1 - y\, f(x)),\ y \in \{-1, +1\}, \qquad \frac{\partial \ell}{\partial f} = \begin{cases} -y & yf(x) < 1 \\ 0 & \text{otherwise} \end{cases}$$

- **Encodes:** "be correct by a margin of at least 1". Points beyond the margin have **zero** gradient — sparse dependence on the data (support vectors).
- **Outliers:** grows linearly (not exponentially) for misclassified points — more robust than exponential loss, similar to logistic.
- **Use when:** SVMs, max-margin ranking, when you only need decisions/rankings, not probabilities.

- **Failure modes:** outputs are not probabilities (needs calibration); non-differentiable at the hinge; zero gradient beyond the margin means no push to become *more* confident — fine for decisions, bad for probability estimates.



Real-life analogy: the "pass mark" examiner

Think of it like an examiner who only cares that each answer clears the pass mark by a safety margin. Once you're comfortably above it, extra brilliance earns nothing; below it, marks are deducted in proportion to the shortfall. That's the hinge loss: focus entirely on borderline and wrong cases.

Where it breaks: real exams reward excellence; the hinge loss's indifference to easy cases means it gives no calibrated probabilities.

Focal loss

$$\ell_{\mathrm{FL}} = -\alpha_t\,(1-p_t)^{\gamma}\log p_t, \qquad p_t = \begin{cases} p & y=1 \\ 1-p & y=0 \end{cases}$$

- **Encodes:** "stop wasting gradient on examples you already get right." The modulating factor $(1-p_t)^{\gamma}$ is near 0 for easy, well-classified examples ($p_t \to 1$) and near 1 for hard ones. With $\gamma = 2$, an example at $p_t = 0.9$ has its loss scaled by 0.01. Introduced for dense object detection (RetinaNet), where ~100K easy background anchors swamp a handful of objects.
- **Gradient:** concentrated on hard and misclassified examples; α_t additionally balances classes.
- **Outliers:** it **up-weights** the hardest examples — which include **mislabeled** ones. Label noise gets more attention, not less.
- **Use when:** extreme imbalance with many easy negatives (detection, rare-event classification), when reweighting/resampling isn't enough.
- **Failure modes:** probabilities become miscalibrated (the loss is no longer a proper scoring rule — recalibrate); two extra hyperparameters; can underperform plain cross-entropy + a tuned threshold on tabular problems.



Real-life analogy: a coach who ignores the easy drills

Think of it like a cricket coach during net practice: once a batter reliably plays straight drives, the coach stops spending time on those and bowls the tricky deliveries that still get him out. Focal loss down-weights examples the model already gets right with confidence, so training time goes to the hard, rare ones.

Formula, read aloud: focal loss $= -(1-p_t)^{\gamma}\log p_t$: the usual cross-entropy multiplied by "how wrong you still are" raised to a power — easy examples ($p_t \approx 1$) almost vanish.

Where it breaks: a coach knows which balls are genuinely hard; the loss can't tell a hard example from a mislabeled one, so focal loss can over-focus on noisy labels.

15.3 Metric-learning losses

The goal: an embedding $f(x)$ where distance reflects similarity — used for face verification, retrieval, deduplication, recommendation, self-supervised learning.

Contrastive (pairwise) loss

For a pair with distance $d = \|f(x_1) - f(x_2)\|$ and $y = 1$ if similar:

$$\ell = y\,d^2 + (1-y)\,\max(0,\ m-d)^2$$

- Similar pairs are pulled together; dissimilar pairs are pushed apart **only until** they're at least margin m apart (then zero gradient).
- **Failure modes:** needs carefully mined pairs (random dissimilar pairs are already far apart → zero gradient, nothing learned); an absolute margin is hard to set across regions of the space.

Real-life analogy: seating twins together and strangers apart

Think of it like arranging photos on a wall so that photos of the same person hang close together and photos of different people hang at least a hand's width apart. Contrastive loss pulls matching pairs together and pushes non-matching pairs apart until they're separated by the margin — after that, it stops caring.

Formula, read aloud: $\ell = y\,d^2 + (1-y)\max(0, m-d)^2$: for a matching pair, pay the squared distance; for a non-matching pair, pay only if they're closer than the margin m.

Where it breaks: walls are 2-D; embeddings live in hundreds of dimensions, and choosing *which* non-matching pairs to show the model matters more than the formula.

Triplet loss

For anchor a, positive p (same identity), negative n:

$$\ell = \max\left(0,\ d(a,p) - d(a,n) + m\right)$$

- **Relative** constraint: the positive must be closer than the negative by margin m. Easy triplets (constraint already satisfied) give zero gradient.
- **Mining matters most:** random triplets are mostly easy → slow learning; the hardest negatives can be mislabeled or cause **collapse** (all embeddings mapped to one point, loss stuck at m). **Semi-hard** negatives ($d(a,p) < d(a,n) < d(a,p) + m$) are the usual compromise.
- Embeddings are usually L2-normalized so distances are bounded.

Real-life analogy: "who looks more like you?"

Think of it like a face-recognition game: given your photo (the anchor), another photo of you (positive) should always look more like you than a stranger's photo (negative) — by a clear margin. Triplet loss only penalizes cases where the stranger comes too close.

Formula, read aloud: $\max(0, d(a,p) - d(a,n) + m)$: "the distance to yourself should beat the distance to the stranger by at least m; otherwise pay the shortfall".

Where it breaks: random strangers are usually easy to tell apart, so the model learns little from them — real training depends on mining *hard* negatives (look-alikes).

InfoNCE / softmax contrastive loss (the modern default)

$$\ell = -\log \frac{\exp(\mathrm{sim}(a,p)/\tau)}{\exp(\mathrm{sim}(a,p)/\tau) + \sum_k \exp(\mathrm{sim}(a,n_k)/\tau)}$$

A cross-entropy that classifies the positive among many negatives — typically the other examples in the batch ("in-batch negatives"). Uses all negatives at once with soft weighting (hard negatives get more weight automatically), which is why it largely replaced triplet loss (SimCLR, CLIP, two-tower retrieval). Temperature τ controls how sharply it focuses on hard negatives; larger batches give more negatives. **Failure modes:** false negatives in the batch (duplicates, same-item pairs) get pushed away; popularity bias in in-batch negatives (popular items appear as negatives more often — apply a log-frequency correction).

Real-life analogy: picking your partner out of a crowd

Think of it like a game at a wedding: the bride must pick the groom's hand out of a line of many hands. It's easy if the line is short; with many look-alike hands it gets harder and she learns finer details. InfoNCE turns similarity learning into this multiple-choice game: find the one true match among many negatives.

Where it breaks: in a batch, some "negatives" may actually be matches (two photos of the same dog), which quietly teaches the model the wrong thing.

15.4 Ranking losses (one paragraph you'll need for search/recsys)

Pointwise: treat each (query, item) as regression/classification (e.g. BCE on click) — simple, calibrated, ignores relative order. **Pairwise:** learn that $s_i > s_j$ when item i is more relevant: $\log(1 + e^{-(s_i - s_j)})$ (RankNet, BPR). **Listwise:** optimize a whole list's quality — LambdaRank/LambdaMART weight each pair's gradient by the change in NDCG from swapping them, focusing learning on the top of the list; softmax cross-entropy over the list is another listwise option.

Real-life analogy: a judge at a cooking contest

Think of it like judging a cooking contest three ways. **Pointwise:** give each dish a score out of 10 on its own. **Pairwise:** taste two dishes at a time and say which is better. **Listwise:** arrange the whole table from best to worst and care most about getting the top three right, because only they win prizes. Search and recommendation care about the top of the list, so listwise and pairwise methods usually fit the goal best.

Where it breaks: judges taste every dish; ranking models only see the items that were shown and clicked, so the "contest" is biased by what the old system displayed.

15.5 Proper scoring rules: log loss vs. Brier score

A **proper scoring rule** is minimized, in expectation, by reporting your **true** belief — so it rewards honest probabilities. The two you need:

$$\text{Log loss} = -\frac{1}{n}\sum_i \left[y_i \log p_i + (1 - y_i)\log(1 - p_i)\right], \qquad \text{Brier} = \frac{1}{n}\sum_i (p_i - y_i)^2$$

	Log loss	Brier score
Penalty for a confident mistake ($p = 0.99$, $y = 0$)	$-\log 0.01 = 4.6$ — huge, unbounded	0.98 — bounded by 1
Sensitivity	strongly rewards getting rare probabilities right	more forgiving; dominated by common cases
Decomposition	cross-entropy = entropy + KL	Brier = reliability − resolution + uncertainty
Use	training objective; ads/bidding where tail probabilities matter	reporting forecast quality; robust comparison of calibrated models

Worked numbers: a forecaster says 0.8 on 10 days; it rains on 8. Brier $= (8 \times 0.04 + 2 \times 0.64)/10 = 0.16$; log loss $= -(8 \log 0.8 + 2 \log 0.2)/10 = 0.50$. A forecaster who says 0.99 on those same days has Brier

$(8 \times 0.0001 + 2 \times 0.98)/10 = 0.196$ but log loss $(8 \times 0.01 + 2 \times 4.61)/10 = 0.93$ — log loss punishes the over-confidence far more.

Accuracy and AUC are not proper scoring rules for probabilities: a model can have the best AUC and badly miscalibrated outputs (Part 16).



Real-life analogy: a firm examiner and a ruthless one

Think of it like two examiners marking weather forecasters. The Brier examiner deducts marks by the squared distance between the forecast and what happened — firm but never catastrophic. The log-loss examiner is ruthless with arrogance: say "99% no rain" and get soaked, and you lose almost everything. Both reward honest forecasts; they differ in how harshly they punish confident mistakes.

Formula, read aloud: Brier = average of (forecast − outcome)²; log loss = average of −log(probability you gave to what actually happened).

Where it breaks: examiners mark humans who can explain themselves; for models, the choice of score also shapes training — log loss on noisy labels can push a model toward over-hedging or towards chasing mislabeled points.

15.6 Multi-label, ordinal and cost-sensitive losses

- **Multi-label** (a movie can be *comedy* and *romance*): one sigmoid + binary cross-entropy **per label**, not a softmax — softmax forces labels to compete for probability mass. Tune a threshold per label.
- **Ordinal targets** (ratings 1–5, severity low/medium/high): plain classification ignores that "4 vs. 5" is a smaller mistake than "1 vs. 5"; plain regression assumes equal gaps. **Cumulative-link** models predict $P(y > k)$ for each cut-point k with a shared score and ordered thresholds — respecting order without assuming distances.
- **Cost-sensitive** learning: weight each class's loss by its misclassification cost, or keep a standard loss and move the decision threshold to $t^* = \frac{C_{\text{FP}}}{C_{\text{FP}}+C_{\text{FN}}}$ (Part 6). Thresholding a calibrated model is usually simpler and more transparent than reweighting.



Real-life analogy: tagging a film vs. rating a film

Think of it like filling a movie database. Tags ("comedy", "romance", "based on a true story") can all be true at once — you tick each box separately (sigmoid per label). A star rating is ordered: calling a 4-star film "5 stars" is a small slip, calling it "1 star" is a big one (ordinal). And some mistakes simply cost more: wrongly tagging a horror film as "kids" is worse than missing a tag (cost-sensitive).

Where it breaks: a database editor reads the whole film; the loss only sees labels, so noisy or inconsistent tags (one annotator's "romance" is another's "drama") limit how good any loss can be.

15.7 Summary table

Loss	Minimizer predicts	Gradient for large errors	Outlier / noise sensitivity	Typical use	Main failure
MSE	mean	grows linearly	high	regression with light tails	chases outliers

Loss	Minimizer predicts	Gradient for large errors	Outlier / noise sensitivity	Typical use	Main failure
MAE	median	constant	low	robust regression	slow/unstable convergence; zeros
Huber	between	clipped at δ	low–medium	regression with occasional outliers	δ scale-dependent
Quantile	τ-quantile	constant (asymmetric)	low	intervals, asymmetric costs	one model per quantile; crossing quantiles
BCE / CE	calibrated probability	grows (to $p - y$ in logit space); loss unbounded	label noise hurts	classification	overconfidence; imbalance; log(0)
Hinge	margin decision	constant	medium	SVM, ranking	no probabilities
Focal	down-weighted probability	concentrated on hard examples	amplifies label noise	extreme imbalance, detection	miscalibration
Contrastive / triplet	embedding geometry	zero once margin satisfied	mislabeled pairs; collapse	metric learning	mining; collapse
InfoNCE	embedding geometry	soft focus on hard negatives	false negatives	retrieval, self-supervision	needs many negatives; popularity bias

Q: "When would you choose MAE over MSE?"

Strong answer. "It depends on what the prediction should represent and how errors cost. MSE estimates the conditional mean and penalizes large errors quadratically — right when large misses are disproportionately costly and noise is roughly Gaussian. MAE estimates the median and weights all errors linearly, so it's robust to outliers and heavy tails — e.g. delivery-time prediction with occasional hour-long anomalies from data glitches. But MAE's constant gradient makes optimization less smooth, and for zero-heavy targets the median can be degenerate. Often Huber is the practical middle ground, or a quantile loss if costs are asymmetric. And I'd evaluate with the metric that matches the business, whichever loss I train on."

Active recall — Set 9: Optimization & losses

Set 9 · Answer from memory, then check the Answer Key

Easy recall

1. Write the Adam update equations.
2. What's the stability condition on the learning rate for GD on a quadratic?
3. Write the Huber loss and its gradient.
4. Write the focal loss.
5. What does gradient clipping by norm do?

Conceptual

6. How does momentum damp oscillations yet accelerate along a valley?
7. Why does Adam need bias correction?
8. What is the difference between weight decay and L2 regularization for Adam?
9. Why are high-loss critical points in deep networks mostly saddle points?

10. Compare pointwise, pairwise and listwise ranking losses.

Why

11. Why does the condition number determine GD's convergence speed?
12. Why does constant-LR SGD not converge to the exact minimum?
13. Why do we use learning-rate warmup?
14. Why does cross-entropy train faster than MSE for sigmoid outputs?
15. Why did InfoNCE largely replace triplet loss?

Scenario

16. Training loss oscillates wildly and doesn't decrease. What do you try?
17. Training loss plateaus early at a high value. Hypotheses?
18. You increase batch size from 256 to 4,096 and accuracy drops. Why, and what do you change?
19. Your delivery-time model is dominated by a few 10-hour outliers from GPS glitches. Which loss(es)?
20. Your detector sees 1 object per 10,000 anchors. Which loss, and what must you do afterwards?

Mathematical

21. For GD on $J = \frac{1}{2}\lambda w^2$, show $w_t = (1 - \eta\lambda)^t w_0$ and derive the stability condition.
22. Derive the gradient of MAE and explain why a constant step size oscillates.
23. Compute the focal loss weight $(1 - p_t)^2$ for $p_t = 0.9$ and $p_t = 0.1$.

Difficult follow-ups

24. Why might SGD generalize better than Adam on some problems but not on transformers?
25. Newton's method converges quadratically — why isn't it used to train deep networks?

2027 additions

26. You 4× the batch size. How do you adjust the learning rate for SGD and for Adam, and why is warmup needed?
27. What does Lion change relative to Adam, and what hyperparameters must change with it?
28. A forecaster says 0.99 and is wrong. Compare the Brier and log-loss penalties, and say which is a proper scoring rule.

PART 16

Model Evaluation

Metric selection in one breath

A metric is a **compressed statement of what errors cost**. Before naming one, ask: what decision does the model drive, what does each kind of mistake cost, how rare is the positive class, is there a capacity or threshold constraint, and do we need ranking quality, calibrated probabilities, or hard decisions? The metric follows from the answers — never the other way round.

The whole Part as one picture

Think of it like a cricket selector picking players. For a Test opener she looks at batting average; for a T20 finisher, strike rate; for a death-overs bowler, economy rate. The stat follows from the job the player must do. Choosing a T20 finisher by his Test average is like judging a fraud model by accuracy: a real number, but the answer to the wrong question. So before picking a metric, ask what decision the model drives, what each kind of mistake costs, and how rare the thing you're hunting is — then choose the number, or small set of numbers, that measures exactly that.

Where it breaks: cricket stats come ready-made, while in ML you often design the metric yourself (for example, recall at a fixed review budget) and report several together, because any single number hides some cost.

16.1 Classification metrics

The confusion matrix

	Predicted positive	Predicted negative
Actually positive	TP	FN (Type II, "miss")
Actually negative	FP (Type I, "false alarm")	TN

Metric	Formula	Question it answers
Accuracy	$\frac{TP+TN}{\text{all}}$	what fraction of decisions are right?
Precision (PPV)	$\frac{TP}{TP+FP}$	when we flag something, how often are we right?
Recall (sensitivity, TPR)	$\frac{TP}{TP+FN}$	of the real positives, how many did we catch?
Specificity (TNR)	$\frac{TN}{TN+FP}$	of the real negatives, how many did we leave alone?
FPR	$\frac{FP}{FP+TN} = 1 - \text{specificity}$	how often do negatives get flagged?
F1	$\frac{2PR}{P+R}$	harmonic mean — high only if both are high
F_β	$\frac{(1+\beta^2)PR}{\beta^2 P+R}$	recall weighted β times as much as precision
MCC	$\frac{TP\cdot TN-FP\cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$	balanced correlation-like summary, uses all four cells

Why F1 uses the harmonic mean: it's dominated by the smaller of the two. Precision 1.0 and recall 0.01 gives an arithmetic mean of 0.505 (looks decent) but F1 ≈ 0.02 (correctly terrible). Its weakness: it ignores true negatives and implicitly assumes FP and FN costs are comparable — rarely true in business problems.

Precision depends on prevalence; recall and specificity don't. Recall and specificity are properties of the classifier on each class; precision mixes in the base rate (Bayes' rule, Part 3). The same model has much lower precision in a population where positives are rarer.



Real-life analogy: a smoke alarm at home

Think of it like a kitchen smoke alarm. True positive: it rings during a real fire. False positive: it shrieks at burnt toast. False negative: a real fire and a silent alarm. True negative: a quiet evening with no fire. **Precision** asks: when it rings, how often is there really a fire? **Recall** asks: of all real fires, how many did it ring for? In a house that almost never has fires, even a good alarm's rings are mostly toast — which is why precision drops when positives are rare.

Formula, read aloud: $F_1 = \frac{2PR}{P+R}$ says: combined score = 2 × precision × recall ÷ (precision + recall), an average that gets dragged down to the weaker of the two.

Where it breaks: F1 treats a missed fire and a burnt-toast alarm as equally bad and ignores the quiet evenings (true negatives); real costs are rarely equal.

ROC curve and ROC-AUC

Plot TPR against FPR as the threshold sweeps from $+\infty$ to $-\infty$. **AUC = the probability that a randomly chosen positive gets a higher score than a randomly chosen negative** (the Mann–Whitney U statistic, ties counted half). 0.5 = random ranking; 1.0 = perfect separation.

Properties: threshold-free; **invariant to class prevalence** (TPR and FPR are each computed within one class) and to any monotonic transform of the scores (so **blind to calibration**). That invariance is both its strength (comparable across datasets with different base rates) and its weakness under heavy imbalance: moving FPR from 0.001 to 0.01 barely changes the ROC curve's area, but with 999 negatives per positive it's a 10× increase in false alarms.



Real-life analogy: a teacher's guess list

Think of it like a teacher who ranks her class by how likely each student is to pass, before results come out. Afterwards, pick one student who passed and one who failed, at random. **AUC** is the chance her list put the passer above the failer: 0.5 means her list was no better than shuffling; 1.0 means every passer sat above every failer. It checks only the *order*, so it doesn't care whether she wrote "90% sure" or "60% sure" — it can't tell whether her confidence is honest.

Formula, read aloud: $\mathrm{AUC} = P(s_+ > s_-)$ says: the probability that a random positive's score beats a random negative's score, with ties counting half.

Where it breaks: when negatives vastly outnumber positives, a tiny-looking rise in the false-alarm rate barely moves AUC, yet it can multiply the number of wrong names near the top of her list.

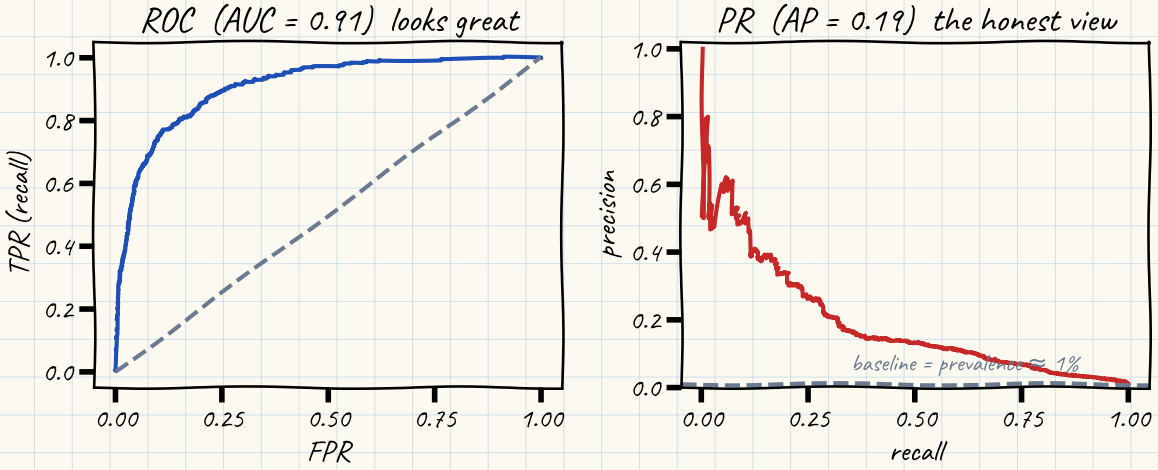
Precision–recall curve and PR-AUC

Plot precision against recall across thresholds. The baseline of a random classifier is a horizontal line at the **prevalence** (not 0.5). **Average precision (AP)** — the precision averaged at each recall step — is the usual PR-AUC summary.

Real-life analogy: fake notes at a kirana store

Think of it like a busy kirana store where about 1 note in 100 is fake. A checking machine ranks notes by how suspicious they look, and the shopkeeper sets aside the most suspicious ones. As that pile grows, the PR curve tracks two things: what fraction of the pile is really fake (**precision**), and how many of all the fakes are in it (**recall**). A machine that just guesses gives a pile that is about 1% fake, so the baseline is the prevalence, not 0.5. PR tells him how many honest customers he'll offend for each fake he catches.

Where it breaks: the baseline moves with the fake rate, so a PR-AUC from a shop with 1% fakes can't be compared with one from a shop with 5% fakes, unlike ROC-AUC.



The same model: ROC-AUC 0.91 looks great, average precision 0.19 tells the truth — at 1% prevalence

	ROC-AUC	PR-AUC / AP
Focus	both classes equally	the positive class and the top of the ranking
Sensitive to prevalence?	no	yes (baseline = prevalence)
Under heavy imbalance	can look excellent while precision is poor	reveals the false-alarm problem
Comparable across datasets with different base rates?	yes	no
Use when	classes are balanced-ish, or you care about ranking both classes, or comparing across populations	positives are rare and what matters is how clean the flagged set is

Log loss, calibration, top-k

Log loss evaluates the probabilities themselves (proper scoring rule) — required when downstream systems consume probabilities (ads bidding, expected-cost decisions). In ads, **normalized entropy** (log loss divided by the entropy of the base rate) makes it comparable across datasets. **Calibration** (reliability diagrams, ECE, Brier — Part 6) checks whether a 0.3 means 30%. **Top-k accuracy** counts a prediction correct if the true class is among the k highest-scoring classes — appropriate when a user sees several suggestions or when classes are ambiguous (ImageNet top-5).

Real-life analogy: the monsoon weather forecaster

Think of it like judging a weather forecaster during the monsoon. **Calibration** asks: on all the days she said "30% chance of rain", did it actually rain on about 30% of them? **Log loss** is a penalty that stays small for mild mistakes but shoots up when she says "99% sure, no rain" and it pours — confident and wrong is punished hardest. **Top-k accuracy** is like a food app showing five dish suggestions: it counts as a hit if the dish you wanted is anywhere in those five.

Where it breaks: a forecaster who says "30%" every single day (the seasonal average) can be perfectly calibrated yet useless; calibration alone doesn't reward telling rainy days apart, so pair it with log loss or AUC.

16.2 Regression metrics

Metric	Formula	Properties	Pitfalls
MAE	$\frac{1}{n}\sum\lvert y-\hat{y}\rvert$	in target units; robust	treats all errors linearly
MSE	$\frac{1}{n}\sum(y-\hat{y})^2$	penalizes large errors; matches OLS	squared units; outlier-dominated
RMSE	$\sqrt{\text{MSE}}$	target units; $\geq$ MAE (gap signals error variance/ outliers)	still outlier-sensitive
R^2	$1-\text{SSE}/\text{SST}$	scale-free; vs. predicting the mean	not comparable across targets; can be negative on test
MAPE	$\frac{100}{n}\sum\left\lvert\frac{y-\hat{y}}{y}\right\rvert$	percent, intuitive	**undefined at** $y=0$, explodes near 0; **asymmetric**: under-forecasts are capped at 100% error, over-forecasts are unbounded → optimizing it biases forecasts **low**
WAPE	$\frac{\sum\lvert y-\hat{y}\rvert}{\sum\lvert y\rvert}$	percent of total volume; robust to zeros	dominated by high-volume items
RMSLE	RMSE on $\log(1+y)$	relative errors; for skewed positive targets	under-prediction penalized more than over

Also report **bias** (mean signed error): a forecast can have a good MAE yet systematically under-predict — which is what drives stock-outs.

Real-life analogy: a train-arrival app

Think of it like an app that predicts when your train will reach the station. **MAE** is the average number of minutes it's off. **RMSE** squares each miss first, so one missed 2-hour fog delay counts far more than several 5-minute slips. R^2 asks how much better the app does than always guessing "the usual delay". **Bias** is the app always predicting arrival a little too early: its average miss looks small, but you *always* wait on the platform — just as a forecast that always runs low causes stock-outs.

Formula, read aloud: $\text{RMSE}=\sqrt{\frac{1}{n}\sum(y-\hat{y})^2}$ says: square every miss, average them, then take the square root to get back to minutes.

Where it breaks: waiting early and missing the train cost you differently, but MAE and RMSE treat over- and under-prediction alike; when costs are lopsided, use a quantile (pinball) loss.

16.3 Ranking metrics

For a query with a ranked list:

- **Precision@K** = relevant items in top K / K. **Recall@K** = relevant items in top K / all relevant items. (Recall@K is the key metric for **candidate retrieval** stages; Precision@K for what the user sees.)
- **MRR** (mean reciprocal rank) = average over queries of $1/\text{rank of the first relevant item}$. For navigational queries with one right answer ("facebook login").
- **MAP** = mean over queries of **average precision**: the mean of Precision@k at each rank k where a relevant item appears.
- **NDCG@K** — graded relevance, discounted by position:

$$\text{DCG@K} = \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \qquad \text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}} \in [0, 1]$$

Real-life analogy: asking a friend for restaurants

Think of it like asking a friend for five restaurant suggestions. If the best place comes first, you're delighted; if it's fifth, you may have stopped reading. **DCG** gives each suggestion points for how good it is — a great place earns far more than an okay one — and shrinks those points the further down the list it sits. **NDCG** divides by the score of the perfect ordering, so 1.0 means "couldn't have ordered it better". **MRR** cares only how far down the *first* good answer is.

Formula, read aloud: $\text{DCG@K} = \sum_{i=1}^{K} \frac{2^{rel_i}-1}{\log_2(i+1)}$ says: for each of the top K slots, add (a reward that doubles with every relevance grade, minus 1) ÷ (a penalty that grows slowly with position).

Where it breaks: your friend knows every restaurant, but offline relevance labels exist only for items the old system chose to show, so offline NDCG leans toward the old ranker.

Worked example

Relevances in ranked order: [3, 2, 3, 0, 1, 2]. DCG@5 terms: $7/1 = 7$, $3/1.585 = 1.89$, $7/2 = 3.5$, 0, $1/2.585 = 0.39$ ⇒ DCG@5 = 12.78. Ideal order [3, 3, 2, 2, 1]: $7 + 4.42 + 1.5 + 1.29 + 0.39 = 14.60$. **NDCG@5 = 0.876.**

AP example: 3 relevant items at ranks 1, 3, 6: $\text{AP} = \frac{1}{3}\left(\frac{1}{1} + \frac{2}{3} + \frac{3}{6}\right) = 0.722$. MRR for that query: $1/1 = 1$.

Why position discounting: users mostly look at the top; a relevant item at rank 1 is worth far more than at rank 10. **Offline ranking metrics caveat:** relevance labels come from past exposure (items never shown are never clicked), so offline NDCG is biased toward the logging policy; online metrics (CTR, dwell, conversions, long-term retention) are the real judge.

16.4 Choosing metrics from business costs

"You're detecting fraud where only 0.1% of transactions are fraudulent. Which metrics matter and why?"

Don't answer "F1". Reason from the costs.

1. Why the obvious metrics fail. Predicting "not fraud" for everything gives **99.9% accuracy** and catches nothing. ROC-AUC can be 0.97 while the model is operationally useless: at 1M transactions with

1,000 frauds, a model with 80% recall at 1% FPR flags 800 frauds and **9,990** legitimate transactions — precision **7.4%**.

2. What each error costs.

- **False negative** (missed fraud): the transaction amount (chargeback) + fees + possible downstream losses. Varies hugely by amount — missing a $5,000 fraud ≠ missing a $5 one.
- **False positive** (blocked good customer): lost sale, support cost, customer friction and churn — or, if flagged transactions go to manual review, reviewer time.
- **Constraints:** review team capacity (e.g. 2,000 cases/day), latency (a decision in < 100 ms at authorization time), regulatory requirements.

3. Metrics that follow.

- **Primary operating metric:** recall (ideally **dollar-weighted recall**: fraction of fraud *value* caught) **at a fixed FPR or at the review-capacity threshold** — e.g. "recall at 0.5% of transactions flagged".
- **Model comparison offline: PR-AUC / average precision** (focuses on the top of the ranking where decisions happen), plus the operating-point metrics above. ROC-AUC as a secondary sanity check.
- **Expected cost / savings:** $\sum_{\text{FN}} \text{amount} + \sum_{\text{FP}} c_{\text{friction}}$ at the chosen threshold — the number finance cares about.
- **Calibration** if thresholds are set from costs or scores feed other systems (Part 6).
- **Segment metrics:** new vs. established customers, merchant categories, countries — fraud patterns and FP costs differ.
- **Monitoring:** alert rate, precision on reviewed cases, chargeback rate — with the caveat that fraud labels arrive **30–90 days late** (chargebacks), so early proxies are needed.

4. Threshold from costs: flag if $p > \frac{C_{\text{FP}}}{C_{\text{FP}}+C_{\text{FN}}(\text{amount})}$ — a per-transaction threshold that's lower for large amounts. Possibly tiers: auto-approve, step-up authentication (cheap friction), manual review, auto-block.



Real-life analogy: a triage nurse's ward

Think of it like a triage nurse in a busy emergency ward. Sending a heart-attack patient home (a false negative) can be fatal; ordering an ECG for someone with heartburn (a false positive) costs time and money; and there are only so many doctors per hour (review capacity). So the ward isn't judged on accuracy but on "of the truly serious cases, how many did we catch with the doctors we have?" The metric falls straight out of the costs and the capacity.

Formula, read aloud: flag if $p > \frac{C_{\text{FP}}}{C_{\text{FP}}+C_{\text{FN}}}$ says: test someone when their risk is above (cost of a needless test) ÷ (cost of a needless test + cost of a miss). If a miss is 99 times worse, test anyone above 1% risk.

Where it breaks: in the ward every missed heart attack is terrible, but in fraud a missed Rs 50 charge and a missed Rs 5 lakh charge differ hugely, so the threshold should change per transaction.

Problem	What errors cost	Metric choices
Cancer screening	missed cancer ≫ unnecessary follow-up test	high **recall / sensitivity** at acceptable specificity; confirmatory second test handles FPs
Spam filtering	a legitimate email hidden ≫ one spam getting through	high **precision** on the spam class (low FPR on legit mail); recall secondary
Content moderation	harm of missed violations vs. over-removal of legitimate speech; reviewer capacity	precision/recall at operating points per policy severity; prevalence estimates via sampling

Problem	What errors cost	Metric choices
Search	users judge the top few results	NDCG@10, MRR for navigational queries; online CTR, success rate, reformulation rate
Recommendations	engagement and satisfaction over time	Recall@K (retrieval), NDCG@K (ranking), plus coverage/diversity; online retention
Ads CTR	bids use pCTR × value	**log loss / normalized entropy** + calibration; AUC secondary
Credit risk	expected loss = PD × LGD × EAD; regulation	AUC / Gini ($2\,\mathrm{AUC} - 1$) / KS for ranking; calibration of PD; stability over time
Demand forecasting	stock-outs vs. overstock, asymmetric	WAPE, bias, quantile (pinball) loss at the service-level quantile

16.5 Error bars on metrics: bootstrap and calibration error in code

A metric without an error bar invites wrong decisions. Two tools you should be able to write from memory:

```
def bootstrap_auc_ci(y, s, B=2000, seed=0):
    from sklearn.metrics import roc_auc_score
    rng = np.random.default_rng(seed)
    n, vals = len(y), []
    while len(vals) < B:
        idx = rng.integers(0, n, n)
        if y[idx].min() == y[idx].max():                    # resample lacks a class: skip
            continue
        vals.append(roc_auc_score(y[idx], s[idx]))
    return np.quantile(vals, [0.025, 0.975])

def expected_calibration_error(y, p, n_bins=10):
    bins = np.minimum((p * n_bins).astype(int), n_bins - 1)
    ece = 0.0
    for b in range(n_bins):
        m = bins == b
        if m.any():
            ece += m.mean() * abs(y[m].mean() - p[m].mean())
    return ece
```

Checked: with 100 examples and only **10 positives**, a model with AUC 0.75 has a 95% bootstrap interval of roughly **[0.60, 0.89]** — you cannot distinguish it from a model at 0.65 or 0.85. What drives AUC's uncertainty is the number of **positives** (and negatives), not the total rows. For ECE: 5,000 perfectly calibrated predictions give ECE ≈ **0.022** (pure binning noise — note it's not 0), while an overconfident version ($p \to 1.4p - 0.2$, clipped) gives ≈ **0.077**. Always compare ECE against the noise floor at your sample size, and look at the reliability diagram, not just the number.

When to use what: bootstrap the **evaluation set** (resampling the unit that was randomized or sampled — users, not rows, if rows share users); for comparing two models on the same set, bootstrap the **paired difference** (Part 45); DeLong's test gives an analytic AUC variance when you need speed.

Real-life analogy: a batting average after five innings

Think of it like judging a new batter after five innings: an average of 60 could be a star or a lucky streak. After 50 innings you'd trust it. A bootstrap replays the innings you have many times in random combinations to see how much the average could wobble. With only 10 positive examples, a model's AUC is a five-innings average.

Formula, read aloud: ECE = $\sum_b \frac{n_b}{n} |\text{actual rate in bin } b - \text{average prediction in bin } b|$: "in each confidence bucket, how far is what we said from what happened, weighted by bucket size".

Where it breaks: replaying innings assumes future innings resemble past ones; the bootstrap can't capture uncertainty from a shift in the data distribution, and it underestimates uncertainty with very few samples.

16.6 Slice-based evaluation

An overall metric can hide a disaster in a subgroup. Standard practice before any launch:

- Define slices from **product** (new vs. returning users, device, country, language), **data** (rare categories, missing-feature patterns, long inputs), and **fairness** (protected groups, where lawful — Part 48) perspectives.
- Report the metric **with confidence intervals** per slice; small slices are noisy, so flag only differences larger than the noise.
- Watch for **Simpson's paradox** (Part 4): a model can win overall and lose in every slice when the slice mix differs between comparisons.
- Automate it: slices are part of the evaluation suite and the launch gate, not a one-off analysis.

Real-life analogy: a school's average hides a failing class

Think of it like a school proudly reporting an 85% pass rate — while one section has a 40% pass rate because its teacher left mid-year. The overall number is true and useless for fixing the problem. Slice-based evaluation is checking every section's results separately before celebrating.

Where it breaks: with many slices, some will look bad by chance (multiple comparisons, Part 4) — confirm with more data before acting.

PART 17

Data Splitting and Validation

The one rule

Your validation split must simulate how the model will be used. If in production the model predicts next month for users it has seen, validate on a later month. If it predicts for new users, validate on users it has never seen. Any split that lets information flow in a way production won't is measuring something other than production performance.

The whole Part as one picture

Think of it like a driving test for a new driver. If she'll be driving through Mumbai monsoon traffic, a test on an empty, dry ground tells you little. And if the test route is the exact lane she practised on every day, a perfect score proves nothing about new roads. A good validation split is an honest driving test: on roads like the ones she'll really face (the same time period, the same kind of users), and on stretches she has never practised on (no future data, no repeat users). Every strategy in this Part is a way to design that test.

Where it breaks: a driving test happens once, but a validation or test set gets reused for decision after decision — and every reuse quietly turns it into practice ground.

17.1 The strategies

Strategy	How	When	Watch out
Holdout	one train/val(/test) split	large data; expensive models	one split = one noisy estimate on small data
K-fold CV	K folds; train on $K-1$, validate on 1; average	small/medium data; model selection	$K\times$ cost; folds must respect groups/time
Stratified K-fold	each fold preserves class proportions	classification, especially imbalanced	stratify on the label (or key segments)
Grouped K-fold	all rows of a group (user, patient, device, session) in the same fold	repeated entities	the model must never see the test entity during training
Time-series split	train on the past, validate on the future (expanding or rolling window), often with a **gap**	anything temporal	no shuffling; gap for label windows and feature lag
Leave-one-out	$K=n$	tiny datasets	expensive; the estimate has **high variance** (training sets nearly identical ⇒ highly correlated fold models); closed form for linear smoothers
Nested CV	inner loop tunes, outer loop estimates	small data where you need both tuning and an honest estimate	expensive

Bias–variance of the CV estimate itself. Small K: each model trains on less data ⇒ pessimistic bias. Large K (LOO): nearly unbiased but high variance and cost. $K=5$ or 10 is the usual compromise.

Real-life analogy: a question bank in five piles

Think of it like a coaching class with 50 practice questions. **Holdout:** keep 10 aside as one final test. **K-fold:** split them into 5 piles; five times, study four piles and test on the fifth, then average the five scores — fairer than one lucky or unlucky test. **Stratified:** every pile gets the same mix of easy and hard questions. **Grouped:** all questions about one passage go in the same pile, or the test is half-seen. **Time-series:** study last year's papers and test on this year's, never the reverse.

Where it breaks: a student remembers earlier rounds, but K-fold trains a fresh model each round; and with one question per pile (leave-one-out) the rounds are nearly identical, so their errors move together and the average stays noisy.

```
Time-series split with expanding window and a gap (embargo):

fold 1: [TRAIN=====]  gap  [VAL]
fold 2: [TRAIN===========]  gap  [VAL]
fold 3: [TRAIN=================]  gap  [VAL]
        ------------------------------------------> time
the gap covers the label window: if the label is "churned within 30 days",
training examples within 30 days of the validation start would peek at it.
```

17.2 Leakage and train/test contamination (split-level)

- **Preprocessing fitted on all data** — scalers, imputers, PCA, vocabulary/TF-IDF, target encoding, feature selection. Fix: fit inside each training fold (a pipeline).
- **Duplicates and near-duplicates across splits** — the same product listed twice, re-posted content, augmented copies of one image, identical rows from logging retries. The model is tested on memorized examples. Fix: deduplicate (hash / near-duplicate detection) *before* splitting, or group duplicates.
- **Resampling before splitting** (SMOTE/oversampling — Part 18).
- **Temporal contamination** — random splits on time-ordered data let the model train on the future (and on features computed with future data).
- **Tuning on the test set** — every decision made by looking at the test set turns it into a validation set.

Real-life analogy: a magician's planted volunteer

Think of it like a magic show where the "random" volunteer is secretly the magician's friend: the trick looks amazing and proves nothing. Leakage sneaks such friends into your test set in several disguises. **Preprocessing on all data** is the magician peeking at the audience list before the show. **Duplicates across splits** are the same friend coming back in a different shirt. **Resampling before splitting** is photocopying the friend. **Temporal contamination** is reading tomorrow's newspaper. **Tuning on the test set** is rehearsing before the same judges until they clap.

Where it breaks: a planted volunteer is deliberate, while ML leakage is almost always accidental and silent — which is why pipelines and deduplication before splitting must be habits, not afterthoughts.

17.3 When random splitting gives misleading results

Five situations where a random split lies

1. **Temporal data** (forecasting, churn, fraud): random splits let the model learn from the future — trends, seasonality, and tactics it couldn't know. Offline scores are inflated; production decays. → time-based split.
2. **Repeated entities** (users, patients, stores): the model memorizes entity-specific patterns and is rewarded for recognizing the entity. Production on new entities is worse. → group split (if production sees new entities) or time split (if it sees the same entities later).
3. **Near-duplicates** (images of the same scene, templated text, product variants): the model memorizes; the test set shares memorized items. → dedupe or group by cluster.
4. **Spatial or network autocorrelation** (neighboring regions, connected accounts in a fraud ring): neighbors leak each other's labels. → spatial blocks / graph-community splits.
5. **Production population differs** (new markets, new users, new product): a random split measures performance on the *old* population. → hold out the new segment explicitly, or reweight.



Real-life analogy: shuffling IPL deliveries

Think of it like training a model on every ball of an IPL season to predict boundaries, after shuffling all the balls randomly into train and test. Each test ball now has neighbours from the *same over* in training, so the model already "knows" that day's pitch, that bowler's rhythm and that batter's form — it is recognising, not predicting. Next season brings new pitches, new players and new tactics, and it flops. The honest test uses the latest matches (a time split) or players it has never seen (a group split).

Where it breaks: in cricket the time order is obvious, but in real datasets the hidden link — the same user, a near-duplicate photo, accounts in one fraud ring — is often invisible until you go looking for it.

Q: "Your dataset has 500 million examples. Would you use cross-validation?"

Strong answer. "Usually not. CV exists to reduce the variance of the performance estimate and to use scarce data efficiently. With 500M examples, a single well-designed holdout of even 1% — five million examples — gives a standard error on accuracy or AUC far below any difference I'd act on, and training five models would multiply an already large compute bill. I'd spend that effort on the *design* of the holdout instead: time-based to match deployment, grouped by user if needed, deduplicated, and sliced by important segments — plus a separate recent test period. The exceptions: if the positive class is extremely rare so the effective sample is small, if I'm estimating per-segment metrics for small segments, or if I need uncertainty across training randomness — then a few repeated runs or a bootstrap on the evaluation set are more useful than K-fold."

17.4 Adversarial validation and purged cross-validation

Adversarial validation answers "does my test set look like my training set?" Label training rows 0 and test (or recent production) rows 1, train a classifier to tell them apart, and read its AUC:

- AUC ≈ 0.5 → indistinguishable; random validation is a fair proxy.
- AUC ≫ 0.5 → **shift**. The classifier's most important features tell you *what* shifted (a new category, a changed pipeline, time trends). Options: drop or fix those features, build the validation set from the training

rows that look most "test-like", or reweight training rows by $\frac{P(\text{test}|\mathbf{x})}{P(\text{train}|\mathbf{x})}$ (importance weighting for covariate shift, Part 1).

Purged and embargoed K-fold (time series with overlapping labels, e.g. finance or "will the user churn in the next 30 days"): a training row whose label window overlaps the validation period leaks information. **Purge** training rows whose label windows overlap the validation fold, and add an **embargo** gap after the fold so autocorrelated neighbours don't leak backwards. It's the grouped/temporal logic of this Part applied to labels that span time.



Real-life analogy: spotting the out-of-town guests

Think of it like a wedding host asking a sharp-eyed relative: "can you tell which guests came from out of town?" If the relative can't (AUC ≈ 0.5), locals and visitors mix well. If she picks them out instantly by their clothes (a feature), you've learned the two groups differ — and exactly how. Adversarial validation is that relative, applied to your training and test data.

Where it breaks: the relative only notices differences she can see; a classifier can't detect shift in things not in the features (for example, a changed labeling policy).

PART 18

Class Imbalance

The real problem with imbalance

The ratio itself isn't the enemy — **the small absolute number of positives** is (high variance on the minority class), together with **metrics and thresholds** that ignore the minority (accuracy, a default 0.5 threshold). A well-specified probabilistic model trained on imbalanced data still estimates $P(y \mid x)$ correctly; most "imbalance fixes" are really threshold or metric fixes.

The whole Part as one picture

Think of it like a lifeguard at a calm beach where only 1 swimmer in 1,000 ever gets into trouble. Shouting "everyone's fine" all day is 99.9% accurate and completely useless. The real problems are two: she has seen only a handful of real emergencies (few positives, so shaky experience), and she is rated on total correct calls (the wrong metric). Her sense of *how risky* each swimmer looks may be perfectly sound. What usually needs fixing is when she blows the whistle (the threshold) and how she is judged (the metric) — and, best of all, more examples of real emergencies to study.

Where it breaks: a lifeguard can learn from drills and training videos; a model learns only from the examples it is given, so collecting more positives beats almost every clever trick.

18.1 Techniques

Technique	What it does	Pros	Cons
Better metric + threshold tuning	keep data as is; pick the threshold from costs/capacity on validation	simplest; preserves calibration; often the best fix	needs a good operating-point definition
Class weighting	weight the minority's loss by w (e.g. inverse frequency)	no data duplication; supported everywhere	shifts predicted probabilities (miscalibration); large weights raise variance
Random undersampling of the majority	drop negatives	faster training (huge data); focuses learning	discards information; higher variance; must **correct calibration**
Random oversampling of the minority	duplicate positives	keeps all data	exact duplicates ⇒ overfitting (trees memorize them)
SMOTE	synthesize minority points by interpolating between a minority point and one of its k minority neighbors	smoother than duplication	can create points in majority regions (noise); poor in high dimensions and with categorical features; rarely helps strong learners like GBMs
Focal loss	down-weight easy examples	great for many easy negatives (detection)	miscalibration; amplifies label noise
Ensembles of balanced subsamples (e.g. balanced bagging)	many undersampled models	uses all negatives across models	more compute

Technique	What it does	Pros	Cons
Anomaly detection framing	model only "normal"	when positives are extremely rare or constantly changing	ignores the labels you do have

Calibration correction after resampling. If negatives were kept with probability w (or positives weighted by $1/w$), the model's odds are inflated by $1/w$. Recover the true probability: $p = \frac{q}{q+(1-q)/w}$. For class weights, recalibrate on a held-out set with the original distribution.



Real-life analogy: re-weighting a lopsided survey

Think of it like a national survey that interviews only 1 in 20 city residents but every village resident. Raw results over-represent villages, so the surveyor must re-weight before publishing. **Undersampling** the majority is exactly this, and so is its fix. **Class weighting** is counting each villager's answer 20 times. **Oversampling** is interviewing the same villagers twice, so you grow overconfident in their particular views. **SMOTE** is inventing an "in-between" villager by blending two real ones.

Formula, read aloud: $p = \frac{q}{q+(1-q)/w}$ says: true chance = model's score ÷ (model's score + the "no" share scaled back up by $1/w$). Keep 1 in 20 negatives ($w = 0.05$) and a raw 0.5 becomes about 0.05.

Where it breaks: the surveyor knows the exact sampling rate, so the fix is exact; after class weights, SMOTE or focal loss there is no clean formula, so recalibrate on held-out data with the real class mix.

18.2 Why oversampling before splitting causes leakage

The mechanism

Suppose you duplicate (or SMOTE) the minority class on the full dataset and then split. A positive example and its **duplicate** now land in different splits: the model trains on one copy and is "evaluated" on the other — it's graded on memorization. With SMOTE it's subtler: a synthetic training point is an interpolation of real points, some of which end up in the validation set, so the training set contains near-copies of validation positives. Either way, **validation recall and precision are inflated** and the model's real performance on new positives is unknown. The same holds for any resampling done before cross-validation.

Correct procedure: split first (respecting time and groups), then resample **only the training portion of each fold** — e.g. an `imblearn` `Pipeline` so resampling happens inside `fit`. Validation and test sets keep the **real class distribution**, because that's what production looks like.

Real-life analogy: photocopied recipe cards

Think of it like a cooking student with only 10 rare recipe cards. You photocopy every card, *then* split the pile into a practice box and an exam box. Now the exam holds copies of cards she practised with; she aces it, but hand her a truly new recipe and she struggles. SMOTE is subtler: it makes blended cards (half of recipe A, half of recipe B), so the exam still contains near-copies of practice cards. The fix: split first, then photocopy only inside the practice box.

Where it breaks: a photocopy is easy to spot, but SMOTE's synthetic points look new, so the leak stays invisible unless you follow the split-then-resample order strictly.

18.3 A practical order of operations

1. Get the metric right (PR-AUC, recall at fixed FPR, cost) and the split right (stratified, time/group-aware).
2. Train on the natural distribution; tune the threshold on validation.
3. If the minority class is underfit, try class weights (then recalibrate).
4. If the majority class makes training too slow, downsample negatives (then correct probabilities analytically).
5. Try focal loss / SMOTE / balanced ensembles only if the above don't suffice — and compare on the untouched validation distribution.
6. Most impactful of all: **get more positives** — active learning, labeling suspicious cases, weak supervision, data from related tasks.

Real-life analogy: fixing a slow phone

Think of it like fixing a phone that feels slow. You don't buy a new phone first. You check whether it's really slow or the test was unfair (the right metric and split), restart it (tune the threshold), close heavy apps (class weights), and only then try bigger surgery like a factory reset (SMOTE, focal loss). Often the upgrade that helps most is simply more memory — for a model, more positive examples.

Where it breaks: phone fixes don't affect each other, but ML fixes do: reweighting or resampling changes the probability scale, so after each step you must recalibrate and re-tune the threshold.

Q: "Your classifier has 99% accuracy on a dataset with 1% positives. Is it good?"

Strong answer. "Can't tell — predicting all negatives gets 99%. I'd look at the confusion matrix, precision and recall on the positive class, and PR-AUC against the 1% baseline. Then I'd ask what the positive class means for the business: how costly a miss is versus a false alarm, and whether there's review capacity. It could be excellent or useless; accuracy just can't tell us under this imbalance."

Active recall — Set 10: Evaluation, validation, imbalance

Set 10 · Answer from memory, then check the Answer Key

Easy recall

1. Define precision, recall, specificity and F1.
2. What is the probabilistic interpretation of ROC-AUC?
3. What is the baseline of a PR curve?

4. Write DCG@K.
5. Name four cross-validation strategies and when to use each.

Conceptual

6. Why is ROC-AUC insensitive to class prevalence?
7. Why does precision depend on prevalence while recall doesn't?
8. What's wrong with MAPE as a training objective?
9. What is the gap/embargo in time-series validation for?
10. Why does LOOCV have high variance?

Why

11. Why can accuracy be misleading?
12. Why does F1 use the harmonic mean?
13. Why must resampling happen inside the training folds?
14. Why does class weighting miscalibrate probabilities?
15. Why can offline NDCG disagree with online CTR?

Scenario

16. Design the evaluation for a spam filter.
17. Your churn model has random-split AUC 0.90 and time-split AUC 0.72. Which do you report and why?
18. A forecasting model has great MAPE but warehouses keep running out of stock. What's happening?
19. Two models: AUC 0.92 vs 0.90, but the second has better precision at the operating point. Which do you ship?
20. You have 200 positives and 2M negatives. Plan the training and evaluation.

Mathematical

21. With 1M transactions, 0.1% fraud, a model at 90% recall and 0.5% FPR: compute precision.
22. Compute NDCG@3 for relevances [2, 0, 3] (graded 0–3).
23. Negatives were downsampled at rate 0.05; the model outputs 0.5. What's the corrected probability?

Difficult follow-ups

24. When would you prefer ROC-AUC over PR-AUC even with imbalance?
25. Your review team can handle 1,500 cases per day and daily volume varies 3×. How do you set and maintain the threshold?

2027 additions

26. Your test set has 10 positives and AUC 0.75. What does a bootstrap CI look like, and what drives its width?
27. What is adversarial validation and what do you do if its AUC is 0.9?
28. Why is ECE never exactly zero for a perfectly calibrated model on finite data?

PART 19

Feature Engineering

Feature engineering in one breath

A feature is a way of handing the model information **in a form its inductive bias can use**. Linear models need nonlinearity and interactions handed to them; trees need aggregates and ratios they can't compute from one row; distance-based models need scaling; every model needs information that is **available at prediction time** — and nothing else.

The whole Part as one picture

Think of it like giving directions to different people. A tourist needs landmarks ("turn left at the big temple"), a taxi driver needs street names, a GPS needs coordinates. The destination is the same; only the *form* of the information changes to suit how each listener works. Features are directions for the model: linear models need curves and combinations spelled out, trees need summaries of history they can't see in one row, and distance-based models need everything on one scale. And every direction must use landmarks that will exist when the trip actually happens — nothing the traveller can't see at that moment.

Where it breaks: a person asks "which temple?" when directions are unclear; a model never asks, and it will happily follow any accidental clue in the data, including a leaked answer.

19.1 Numeric features

Transform	When	Mechanism / caution
Standardization $z = \frac{x-\mu}{\sigma}$	linear/logistic with regularization, SVM, KNN, PCA, neural nets	equalizes scale so penalties, distances and gradient steps treat features evenly; fit μ, σ on **train only**
Min-max $\frac{x-\min}{\max-\min}$	bounded inputs for NNs, images (pixels/ 255)	one outlier compresses everything else; new data can fall outside [0, 1]
Robust scaling (median, IQR)	heavy tails / outliers	scale not driven by extremes
Log / log1p	heavy-tailed positive values (income, counts, prices), multiplicative effects	turns ratios into differences; `log1p` handles zeros
Box–Cox / Yeo–Johnson	make distributions more Gaussian	Yeo–Johnson handles negatives
Clipping / winsorizing	outliers, sensor errors	cap at percentiles (fit on train)
Binning	nonlinearity for linear models; robustness	loses information within bins; trees don't need it
Ratios, differences, rates	domain relationships ("debt / income", "price – competitor price", "clicks / impressions")	trees approximate these only with many splits; for linear models, essential
Polynomial terms	smooth nonlinearity for linear models	explodes in dimension; scale first

"Normalization" is ambiguous in interviews — clarify: min-max scaling of columns, z-score standardization, or **L2-normalizing rows** (making each example's vector unit length, as for TF-IDF vectors and embeddings before cosine similarity).

Real-life analogy: scoring an all-round athlete

Think of it like scoring a decathlete: the 100 m in seconds, the shot put in metres, the high jump in centimetres. Adding raw numbers is nonsense — the centimetres would swamp everything. Standardization turns each result into "how many typical gaps above the average athlete", so every event counts evenly; penalties, distances and gradient steps need exactly that. **Min-max** squeezes each event into 0–1 instead, but one freak throw squashes everyone else into a narrow band.

Formula, read aloud: $z = \frac{x-\mu}{\sigma}$ says: standardized value = (value − average) ÷ (typical spread), with the average and spread measured on the training data only.

Where it breaks: with heavy-tailed data such as incomes, a few huge values inflate the "typical spread" itself, so robust scaling or a log transform works better than plain z-scores.

19.2 Categorical features

Encoding	How	Good for	Pitfalls
One-hot	one binary column per category	linear models, NNs, low/medium cardinality	dimension = #categories; sparse; unseen categories at serving need an "unknown" column; drop one column with an intercept in unregularized linear models (dummy trap)
Ordinal	integer codes	genuinely ordered categories (S < M < L); trees (they can still split arbitrary codes, though less efficiently)	imposes a fake order and distance on linear models
Target (mean) encoding	replace category by the mean target in that category	high cardinality with trees or linear models	**leakage**: must use out-of-fold (or time-ordered) statistics; smooth toward the global mean; noisy for rare categories
Frequency / count	replace by how often the category appears	popularity signals; high cardinality	different categories with equal counts collide
Hashing trick	hash category (or category × feature) into m buckets	huge / open vocabularies (ads, text), streaming	collisions (birthday problem: expect some once #items ≈ √m); no inverse mapping; choose m generously
Learned embeddings	lookup table trained with the model	NNs, recommender IDs, very high cardinality	need data per category; cold start for new IDs
Native categorical	model partitions categories itself	LightGBM, CatBoost, XGBoost	can overfit rare categories — use regularization/min data per category

Smoothed target encoding (what to write in an interview)

For category c with n_c examples and mean target $\bar{y}_c$, global mean $\bar{y}$, smoothing strength m:

$$\mathrm{TE}(c) = \frac{n_c\,\bar{y}_c + m\,\bar{y}}{n_c + m}$$

Rare categories shrink toward the global mean (a Bayesian posterior mean with m pseudo-observations — the Beta prior from Part 3). Compute it **out-of-fold** on training data (each row's encoding uses only other folds' rows) and from the full training set for validation/test rows — otherwise each row's own la-

bel leaks into its feature, and the model learns to trust an encoding that won't be that informative on new data. For temporal data, use only past rows (CatBoost's ordered statistics do exactly this).



Real-life analogy: judging applicants by their school

Think of it like an admissions officer who uses each applicant's school as a clue, via that school's average board result. A school with 3 students who all scored 95% shouldn't beat a school where 500 students averaged 85% — three results are thin evidence. So she blends each school's own average with the state average, trusting the school's own number more as its student count grows. And when judging a student, she must leave *that student's own marks* out of the school average, or the clue quietly contains the answer.

Formula, read aloud: $\mathrm{TE}(c) = \frac{n_c\,\bar{y}_c + m\,\bar{y}}{n_c + m}$ says: encoding = (the category's own total + m imaginary "average" rows) ÷ (its row count + m).

Where it breaks: leaving one student out is only a start; in practice you compute the encoding out-of-fold (or from past rows only), because even small in-sample leaks teach the model to over-trust the encoding.

19.3 High-cardinality categories

User IDs, product IDs, zip codes, URLs, device models — thousands to billions of values, most of them rare, with new ones arriving constantly. Options, often combined:



Real-life analogy: a postman and PIN codes

Think of it like an Indian postman sorting letters by PIN code. There are thousands of codes, and some he has almost never seen. But the digits are layered — the first gives the region, the first three a sorting district — so even for an unfamiliar code he knows roughly where to send it. That is "exploit the hierarchy": rare values borrow strength from their parent group. Very rare codes can share an "other" tray (bucketing), and a rack with a fixed number of slots, filled by the last two digits, is the hashing trick — unrelated letters sometimes share a slot.

Where it breaks: PIN codes have a designed hierarchy, but most IDs (users, products) don't; there you build meaning from behaviour instead — point-in-time aggregates or learned embeddings.

1. **Bucket rare values** into "other" (with a minimum count threshold fit on train).
2. **Exploit hierarchy:** zip → city → region; product → category → department; use coarser levels as extra features so rare leaves borrow strength.
3. **Smoothed / out-of-fold target encoding**, count encoding.
4. **Hashing** into a fixed-size space (no vocabulary to maintain; handles unseen values).
5. **Embeddings** learned end-to-end (recommendation, CTR models) — with a fallback for cold-start IDs (hash buckets, content features, or the average embedding).
6. Replace the ID by **what it stands for**: aggregate behavioral features (user's historical CTR, product's average rating), computed point-in-time.

19.4 Missing data

Why values are missing matters:

- **MCAR** (missing completely at random) — e.g. random logging drops. Imputation is unbiased-ish.
- **MAR** (depends on observed features) — e.g. older app versions don't log a field. Condition on those features.
- **MNAR** (depends on the missing value itself) — e.g. high earners decline to report income. **Missingness is itself informative.**

Strategies: drop rows (only if few and MCAR — and remember production rows will be missing too); simple imputation (median/mode) **plus a missing-indicator feature**; model-based imputation (KNN, iterative/MICE); native handling in GBMs (learned default direction); for NNs, impute + indicator or learnable "missing" embedding. Fit imputers on training data only.



Real-life analogy: hotel feedback cards

Think of it like blank boxes on hotel checkout feedback cards. **MCAR:** the front desk forgot to hand out some cards at random — the blanks tell you nothing. **MAR:** the old card had no "Wi-Fi" question, so blanks depend on something you know (the card version). **MNAR:** unhappy guests skip "rate your stay" *because* they were unhappy — the blank itself is a message. That's why you fill the gap with a sensible value *and* keep a "was blank" flag: the model can learn that blank often means trouble.

Where it breaks: on a card you might guess why a box is blank, but in data you usually can't prove which type you have — MNAR can't be detected from the data alone, so domain knowledge decides.

Real-world failure: sentinel values

A pipeline encoded missing income as 0 in training (from a SQL `COALESCE`) but the serving path sent -1. The model had learned "income = 0 ⇒ high risk"; at serving, -1 fell in a different leaf. Nobody noticed for weeks because nothing crashed. **Missing-value encoding is part of the model contract** — test training/serving consistency explicitly.

19.5 Feature interactions

An interaction exists when the effect of one feature depends on another ("discount matters for price-sensitive users only"). Linear models are additive, so they need explicit **crosses** ($x_1 x_2$, or one-hot of category pairs like `country × device`). Trees learn interactions through nested splits. For huge sparse inputs (CTR prediction), **factorization machines** and deep models (Wide & Deep, DCN, DeepFM) learn pairwise and higher-order crosses from embeddings instead of enumerating all pairs (d^2 explosion).

Real-life analogy: an AC in summer and winter

Think of it like the value of an air conditioner. In Delhi in May it's a lifesaver; in Shimla in December it's useless. A linear model can only say "the AC adds this much comfort, summer adds that much" — it can't say "the AC matters *only* in summer". A crossed feature fixes this: a new column that is 1 only when both are true. Trees get there by asking nested questions: "Is it summer? If yes, is the AC on?"

Formula, read aloud: $x_1 x_2$ says: multiply the two features, so the new feature lights up only for the combination.

Where it breaks: here there's one obvious pair; with thousands of features there are millions of pairs, which is why factorization machines and deep models learn crosses from embeddings instead of listing them all.

19.6 Log transformations (and the retransformation trap)

Log helps when a variable spans orders of magnitude, when effects are multiplicative ("+10% price" rather than "+$10"), and when variance grows with the mean. For **targets**: training on $\log y$ with MSE predicts $\mathbb{E}[\log y \mid x]$, and $\exp$ of that is the conditional **geometric mean / median**, which **underestimates** $\mathbb{E}[y \mid x]$. If you need the mean (e.g. total revenue forecasts), apply a correction (for log-normal errors multiply by $e^{\sigma^2/2}$, or use Duan's smearing estimator) — or use a loss that targets the mean directly (Poisson/Tweedie/gamma).

Real-life analogy: salary raises and house prices

Think of it like salary raises. A Rs 10,000 raise is huge on a Rs 20,000 monthly salary and barely noticed on Rs 10 lakh a month; people think in percentages, and log turns "× 1.1" into "+ a fixed step". The trap: in a neighbourhood with a few mansions, the *typical* (median) house is cheaper than the *average* house, because the mansions pull the average up. A model trained on log prices and converted back with exp predicts the typical price, so its revenue totals come out too low.

Formula, read aloud: for log-normal errors, mean $= e^{\text{prediction}} \times e^{\sigma^2/2}$ says: average = typical value × a boost that grows with how spread out the errors are.

Where it breaks: the boost assumes bell-shaped errors of constant spread on the log scale; real data often isn't like that, hence smearing or a Poisson/Tweedie loss.

19.7 Time features

- **Calendar:** hour, day of week, month, holiday flags, days-to-payday, season. For cyclic features use **sin/cos encoding** so 23:00 and 00:00 are neighbors: $\left(\sin\frac{2\pi h}{24}, \cos\frac{2\pi h}{24}\right)$. (Trees handle raw hour fine; linear models and NNs benefit from cyclic encodings.)
- **Recency / frequency / monetary:** time since last purchase, number of sessions in the last 7/30 days, spend in the last 90 days.
- **Rolling statistics and lags** (forecasting): value k periods ago, rolling mean/std, same-day-last-week, trend.
- **Velocity features** (fraud): number of transactions in the last minute/hour from this card/device/IP.
- **Point-in-time correctness:** every aggregate must use only events **strictly before** the prediction timestamp, including the ingestion delay the production pipeline will actually have. This is the #1 source of leakage in time features.

Real-life analogy: a clock and a shopkeeper

Think of it like reading a wall clock instead of a ruler. On a ruler, 23 and 0 are far apart; on a clock face, 11 pm and midnight are neighbours. Sin/cos encoding places each hour on the clock face. Recency and frequency features are a shopkeeper's memory: "she came yesterday, visits weekly, spends about Rs 500". Point-in-time correctness means he may use only what he knew *before* she walked in — not the bill she's about to pay.

Formula, read aloud: $\left(\sin\frac{2\pi h}{24}, \cos\frac{2\pi h}{24}\right)$ says: turn hour h into a point on a 24-hour clock face — how high up it is and how far left or right.

Where it breaks: a clock has one cycle, but real data has several (daily, weekly, yearly, plus festivals like Diwali that move around the calendar), each needing its own encoding or holiday flag.

19.8 Text features

Representation	Idea	Strength	Weakness
Bag-of-words / n-grams	counts of words or phrases	simple, strong with linear models	huge sparse vocab; no word order beyond n
TF-IDF	$\mathrm{tf}(t,d)\cdot\log\frac{N}{\mathrm{df}(t)}$	down-weights ubiquitous words	still lexical — no synonyms
Hashing vectorizer	hash n-grams into fixed buckets	streaming, memory-bounded	collisions, no interpretability
Static word embeddings (word2vec, GloVe)	averaged vectors	semantic similarity	ignores context; averaging loses order
Pretrained transformer embeddings	sentence/document vectors from a pretrained encoder	contextual meaning, strong transfer, little labeled data needed	compute cost/latency; domain shift; need fine-tuning for best results
Handcrafted	length, punctuation, capitalization ratio, URL count, language	cheap, robust signals (spam!)	limited

Real-life analogy: a family WhatsApp group

Think of it like messages in a family WhatsApp group. "Good morning" appears in nearly every message, so it tells you nothing about any one of them. "Wedding" appearing five times in one message, and rarely elsewhere, tells you what that message is about — that's TF-IDF. Bag-of-words just counts words in a jar, so "dog bites man" and "man bites dog" look identical. Transformer embeddings are like a well-read relative who knows "bank" means different things in "river bank" and "bank loan".

Formula, read aloud: $\mathrm{tf}(t,d)\cdot\log\frac{N}{\mathrm{df}(t)}$ says: weight of a word = (how often it appears in this message) × (how rare it is across all messages).

Where it breaks: TF-IDF only matches exact words, so "car" and "automobile" are strangers to it — the gap embeddings fill, at a much higher compute cost.

19.9 Embeddings as features

Pretrained models turn unstructured inputs into dense vectors that capture meaning: text (sentence-embedding models), images (vision or vision–language encoders), and entities learned from behaviour (item or user embeddings from co-occurrence or two-tower models, Part 47). They are often the strongest single feature family you can add to a tabular model.

How to use them well:

- **As direct features:** feed the vector (or a PCA-reduced version) into a GBDT or linear model. Reduce dimension if you have little data — 768 raw dimensions invite overfitting.
- **As similarity features:** cosine similarity between a query and item embedding, or between a user's history centroid and a candidate — one strong number instead of hundreds.
- **As neighbours:** k-NN label averages ("what fraction of the 20 most similar past tickets were escalated?") — computed **out of fold** to avoid leakage (Part 17).
- **Clustering** embeddings to create categorical "topic" features for reporting and monitoring.

Pitfalls: embeddings from a general model may miss domain meaning (fine-tune or pick a domain model); **version them** — a new embedding model changes every feature and requires re-indexing and retraining; embeddings can encode sensitive attributes (Part 48); and nearest-neighbour label features computed on the full dataset leak labels.



Real-life analogy: describing a song by its "vibe" coordinates

Think of it like a music app that places every song on a map by its vibe — energetic vs. calm, happy vs. sad, old vs. new. Two songs close on the map feel similar even if their titles share no words. Embeddings give any text, image or product such coordinates, so a model can compare "meaning" with simple arithmetic.

Formula, read aloud: cosine similarity $\frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$: "how much two items point in the same direction on the map, ignoring how long their arrows are".

Where it breaks: the map was drawn by someone else for a different purpose; "similar vibe" in a general embedding may not be "similar" for your task (two legal clauses can look alike but mean opposite things).

19.10 Leakage in features

Checklist for every feature: *(1) When exactly is this value known? (2) Is it computed from the label, or from events caused by the label? (3) Is it computed identically in training and serving? (4) Was any statistic in it (encoding, scaling, vocabulary) fit on data that includes validation/test rows?* Features like `account_closed_date`, `refund_amount`, `number_of_support_tickets_about_cancellation`, or "final status" fields are classic target leaks.

Real-life analogy: the farewell-party predictor

Think of it like an HR team building a model to predict resignations and proudly finding that "farewell party booked" is the best predictor. Of course it is — the party happens *because* the person resigned. Months earlier, when a prediction would actually help, that field is empty. So ask the four questions of every feature: when is it known, is it caused by the answer, is it computed the same way in training and serving, and was any statistic in it fit on test rows?

Where it breaks: the farewell party is an obvious consequence; real leaks hide behind innocent names (support-ticket counts, "last updated" timestamps) and usually show up only as suspiciously high importance.

19.11 When feature engineering matters more or less

Model	What it needs from you	What it does itself
Linear / logistic	nonlinear transforms, interactions, binning/splines, scaling, careful encoding	nothing beyond additive weighting — FE matters most
Trees / GBMs	aggregates over history (they see one row), ratios/differences (hard to learn with axis-aligned splits), domain features, time-aware encodings	thresholds, interactions, missing values, monotone transforms — **no scaling needed**
KNN / SVM (RBF)	scaling, feature selection (irrelevant features poison distances), good metric	nonlinear boundaries
Neural nets on raw perceptual data (images, audio, text)	preprocessing, normalization, augmentation, tokenization	learn hierarchical features — FE matters much less
Neural nets on tabular data	scaling/normalizing numerics, embeddings for categoricals, often still hand features	interactions via layers, but usually still lose to GBMs on small/medium tabular data
Naive Bayes	discretization or distribution choice, deduplicated features (it double-counts)	—

Two more axes: *data size* — with little data, good features (domain knowledge = inductive bias) beat model complexity; with huge data, models learn more of it themselves. *Latency and maintenance* — every feature is a pipeline to compute, monitor and keep consistent between training and serving; a feature that adds 0.1% AUC but needs a new real-time data source may not be worth it.

Real-life analogy: cooks of different experience

Think of it like instructing cooks of different experience. A beginner (linear model) needs everything spelled out: "if the guest is Gujarati *and* it's lunch, add a little sugar". An experienced cook (GBM) works out such combinations himself and doesn't care whether you measure in grams or kilos (no scaling needed), but he sees only today's order slip, so you must tell him "this guest always orders spicy" (aggregates over history). A master chef who has tasted a million dishes (a deep net on images or text) works from raw ingredients — but only after mountains of practice.

Where it breaks: a cook's skill is fixed, but in ML the balance also shifts with data size — with little data, even the "master chef" needs your hand-made features.

19.12 Feature selection

Filter (correlation, mutual information, chi-square — fast, ignores interactions), **wrapper** (recursive feature elimination, forward selection — expensive, overfits validation), **embedded** (L1, tree importance, gain-based). Do it **inside cross-validation** (selecting on the full data leaks). Prefer stability: features selected consistently across folds/bootstraps. Remove features for good reasons: leakage, instability, cost, latency, fairness, or redundancy — not just low importance on one run.



Real-life analogy: picking a cricket squad

Think of it like picking a 15-player squad from 40 hopefuls. **Filter** methods rank players on individual stats — fast, but blind to partnerships. **Wrapper** methods try many squad combinations in practice matches — thorough but expensive, and you risk picking for that one practice opponent. **Embedded** methods decide during the season as the coach sees who actually contributes (L1, tree importance). Selectors mustn't watch the final they're picking for (select *inside* cross-validation), and a player who makes the squad in every trial series is a safer pick than one who shone once.

Where it breaks: players are distinct people, but two near-identical features can swap places from fold to fold, so "unstable selection" may signal redundancy rather than two useless features.

Q: "How would you encode a categorical feature with 100,000 values for a gradient-boosted model?"

Strong answer. "I'd avoid one-hot — 100K sparse columns with most categories rare. First check what the category represents: if there's a hierarchy, add coarser levels. Then options: native categorical support in LightGBM or CatBoost with regularization on rare categories; smoothed target encoding computed out-of-fold — or CatBoost's ordered statistics — so there's no label leakage; count encoding for popularity; bucketing categories below a minimum count into 'other'. Crucially, I'd replace pure IDs with aggregate behavior features computed point-in-time, since those generalize to new or rare IDs. I'd validate with a split that includes categories unseen in training, because that's what production will send."

Active recall — Set 11: Feature engineering

Set 11 · Answer from memory, then check the Answer Key

Easy recall

1. Standardization vs. min-max scaling vs. row normalization — define each.
2. Write the smoothed target-encoding formula.
3. What are MCAR, MAR and MNAR?
4. Why sin/cos encode hour of day?
5. Write TF-IDF.

Conceptual

6. Which models need feature scaling, and why?
7. What does the hashing trick trade away?
8. Why add a missing-indicator feature after imputation?
9. Why do trees still benefit from ratio features?
10. What is point-in-time correctness?

Why

11. Why must target encoding be computed out-of-fold?
12. Why does $\exp(\text{prediction of } \log y)$ underestimate the mean?
13. Why does feature engineering matter less for CNNs on images?
14. Why can ordinal encoding hurt a linear model?
15. Why should feature selection happen inside cross-validation?

Scenario

16. A "days since last login" feature is computed from a nightly batch job, but serving uses real-time data. What can go wrong?
17. Your model must handle new product IDs every day. How do you represent products?
18. A zip-code feature has 40,000 values, many with < 5 rows. Options?
19. You see a feature `num_support_tickets` with huge importance in a churn model. What do you check?
20. Your linear model underfits on a problem you know has interactions between country and device. Fix?

Mathematical

21. Compute the smoothed target encoding for a category with 4 rows (3 positive), global rate 0.1, $m = 10$.
22. With 1,000 items hashed into 100,000 buckets, estimate the probability of at least one collision.
23. Standardize the value 130 for a feature with train mean 100 and std 15; what if the serving pipeline used mean 110?

Difficult follow-ups

24. When would you prefer hashing over learned embeddings for IDs, and vice versa?
25. How would you detect training–serving skew in feature values automatically?

2027 additions

26. Name three ways to use text embeddings as features in a GBDT model.
27. Why must k-NN label features over embeddings be computed out of fold?
28. What happens to your features when you upgrade the embedding model?

Unit V

Deep Learning & ML Coding

Part 20 — Deep Learning Foundations
Part 21 — ML Coding Interview

Backprop on a whiteboard and NumPy from a blank file — the two things that separate "used PyTorch" from "understands it".

PART 20

Deep Learning Foundations

Deep learning in one breath

A neural network is a stack of (linear map → nonlinearity) layers trained end-to-end by gradient descent, where the gradients come from the chain rule applied backwards (backprop). Everything else — initialization, normalization, residuals, dropout, careful optimizers — exists to keep signals and gradients at a healthy scale through many layers and to control overfitting of a very flexible model.

The whole Part as one picture

Think of it like a game of whispers down a long school line. Each student hears the message, changes it a little and passes it on — a layer is a linear mix plus a nonlinear twist. At the end the teacher compares it with the intended sentence and sends corrections back down the line, so each student learns how much they distorted it (backpropagation). In a long line the message tends to fade away or blow up into nonsense (vanishing or exploding signals). The rest of this Part is classroom tricks: a sensible starting volume (initialization), re-levelling along the way (normalization), a written copy passed alongside (residual connections), and randomly skipping students in practice (dropout).

Where it breaks: students pass words; a network passes numbers, and every "student" is adjusted at once, by exact calculus, after every batch.

20.1 The perceptron

$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$. **Learning rule:** for each misclassified example, $\mathbf{w} \leftarrow \mathbf{w} + y\,\mathbf{x}$, $b \leftarrow b + y$ (labels ± 1). Geometric reading: nudge the hyperplane's normal toward misclassified positives and away from misclassified negatives. **Perceptron convergence theorem:** if the data is linearly separable with margin γ and $\|\mathbf{x}\| \le R$, it makes at most $(R/\gamma)^2$ mistakes. **Limitation:** a single linear threshold can't represent XOR — the observation (Minsky & Papert, 1969) that motivated hidden layers.

Real-life analogy: a mango seller's sorting rule

Think of it like a mango seller sorting ripe from unripe with one rule that weighs colour and softness. Whenever he calls a ripe mango unripe, he shifts his rule *towards* that mango's features; whenever he calls an unripe one ripe, he shifts it *away*. If some straight rule can separate the two piles, he'll find one after a limited number of mistakes. But a pattern like "good if exactly one of two things is true" (XOR) can't be split by any single straight rule.

Formula, read aloud: $\mathbf{w} \leftarrow \mathbf{w} + y\,\mathbf{x}$ says: after a mistake, new weights = old weights + (the true label, +1 or −1) × (that example's features).

Where it breaks: a real seller would eventually shrug and memorise exceptions; the perceptron, on data no straight line can split, never settles — it keeps flipping forever.

20.2 Multilayer perceptron (MLP)

$$\mathbf{h}_1 = \phi(W_1\mathbf{x} + \mathbf{b}_1), \quad \mathbf{h}_2 = \phi(W_2\mathbf{h}_1 + \mathbf{b}_2), \quad \ldots, \quad \hat{\mathbf{y}} = g(W_L\mathbf{h}_{L-1} + \mathbf{b}_L)$$

Output activation g matches the task: identity (regression), sigmoid (binary / multi-label), softmax (multiclass).

Real-life analogy: spotting a friend in a crowd

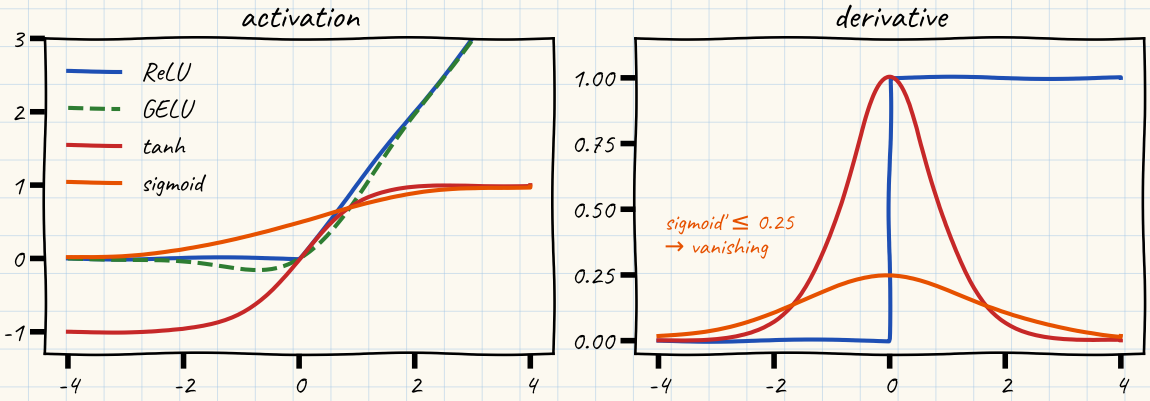
Think of it like spotting a friend at a crowded station. Your eyes first pick up edges, then shapes like eyes and a nose, then a face, then "that's Priya!" Each layer is one such stage: it mixes the previous stage's output with weights, adds a baseline, and passes the result through a rule that keeps what's strong. One giant stage could in theory do it all — like a clerk with a lookup book of every possible face — but that book might need to be impossibly thick. Stacking stages lets parts be reused: one "eye detector" helps with every face.

Formula, read aloud: $\mathbf{h}_1 = \phi(W_1\mathbf{x} + \mathbf{b}_1)$ says: layer output = activation(weights × inputs + bias); each layer feeds the next, and the last one's g puts the answer in the right format.

Where it breaks: networks aren't guaranteed to learn neat, human-readable stages like "eyes" and "faces"; the hierarchy is a tendency, not a rule.

Universal approximation: one hidden layer with enough units can approximate any continuous function on a compact set arbitrarily well. It says nothing about **how many** units (possibly exponentially many), whether gradient descent can **find** the weights, or whether the result **generalizes**. **Depth** buys efficiency: some functions need exponentially many units in a shallow net but polynomially many in a deep one (compositional, hierarchical structure — edges → textures → parts → objects).

20.3 Activation functions



Activation functions and their derivatives

Activation	Formula	Derivative	Pros	Cons
Sigmoid	$\frac{1}{1+e^{-z}}$	$\sigma(1-\sigma) \le 0.25$	probability output	saturates → vanishing gradients; not zero-centered (all-positive outputs make weight updates zig-zag)
Tanh	$\frac{e^z-e^{-z}}{e^z+e^{-z}}$	$1-\tanh^2 \le 1$	zero-centered	still saturates
ReLU	$\max(0,z)$	$\mathbb{1}[z>0]$	no saturation for $z>0$; cheap; sparse activations	**dying ReLU**: units stuck at $z<0$ get zero gradient forever (e.g. after a large update or with a big negative bias)

Activation	Formula	Derivative	Pros	Cons
Leaky ReLU	$\max(\alpha z, z)$	α or 1	no dead units	extra hyperparameter
GELU	$z\,\Phi(z)$	smooth	smooth, non-monotonic near 0; standard in transformers (BERT, GPT)	slightly more compute
SiLU / Swish	$z\,\sigma(z)$	smooth	similar to GELU; used in many modern nets (and in SwiGLU gated MLPs)	—

Why do neural networks need nonlinear activations?

Without them, every layer is a linear map, and a composition of linear maps is linear: $W_2(W_1\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1+\mathbf{b}_2)$. A 100-layer linear network has exactly the expressive power of logistic/linear regression — it can't represent XOR or any curved decision boundary. The nonlinearity lets each layer **bend and fold** the representation space, so later layers can separate classes that were entangled in the input. (A deep linear network is still interesting for studying optimization dynamics — its loss is non-convex in the weights even though the function class is linear.)

Real-life analogy: valves in a water system

Think of it like plumbing. A linear layer is a set of pipes that mixes water streams; stacking mixers still gives one big mixer — which is why stacked linear layers collapse into one. An activation is a valve. **ReLU** is a one-way valve: nothing flows for negative pressure, and flow grows in proportion above zero. **Sigmoid** is a valve that jams fully open or fully shut at the extremes, so extra pushes stop mattering (saturation, hence vanishing gradients). A **dead ReLU** is a valve rusted shut for good.

Formula, read aloud: $\mathrm{ReLU}(z) = \max(0, z)$ says: output = the input if it is positive, else zero; its slope is 1 or 0.

Where it breaks: water flows one way, but in a network the learning signal also flows *backwards* through the same valves, and it is a valve's slope, not its flow, that decides whether earlier layers learn.

20.4 Forward propagation, loss, and backpropagation

Forward pass: compute and **cache** every pre-activation $\mathbf{z}_l$ and activation $\mathbf{h}_l$ layer by layer, then the loss. **Backward pass:** apply the chain rule from the loss back to each parameter, reusing the cached values.

Backpropagation for a 2-layer network — derive it on the whiteboard

Network (one example): $\mathbf{z}_1 = W_1\mathbf{x}+\mathbf{b}_1,\ \mathbf{a}_1 = \mathrm{ReLU}(\mathbf{z}_1),\ z_2 = \mathbf{w}_2^\top \mathbf{a}_1 + b_2,\ p = \sigma(z_2),$
$L = -[y\log p + (1-y)\log(1-p)]$.

Define the **error signal** δ = gradient of L w.r.t. a pre-activation. Work backwards:

$$\delta_2 = \frac{\partial L}{\partial z_2} = p - y \qquad \text{(sigmoid + BCE simplify, Part 6)}$$

$$\frac{\partial L}{\partial \mathbf{w}_2} = \delta_2\, \mathbf{a}_1, \qquad \frac{\partial L}{\partial b_2} = \delta_2 \qquad \text{(error} \times \text{input to that layer)}$$

$$\frac{\partial L}{\partial \mathbf{a}_1} = \mathbf{w}_2\, \delta_2 \qquad \text{(send error back through the weights — transpose)}$$

$$\boldsymbol{\delta}_1 = \frac{\partial L}{\partial \mathbf{z}_1} = (\mathbf{w}_2 \delta_2) \odot \mathbb{1}[\mathbf{z}_1 > 0] \qquad \text{(gate by the activation's derivative)}$$

$$\frac{\partial L}{\partial W_1} = \boldsymbol{\delta}_1\, \mathbf{x}^\top, \qquad \frac{\partial L}{\partial \mathbf{b}_1} = \boldsymbol{\delta}_1 \qquad \text{(outer product: same shape as } W_1\text{)}$$

The general recursion (for any depth, batch form with rows as examples, $H_l = \phi(Z_l)$, $Z_l = H_{l-1}W_l^\top$):

$$\Delta_L = \frac{\partial L}{\partial Z_L}, \qquad \frac{\partial L}{\partial W_l} = \Delta_l^\top H_{l-1}, \qquad \Delta_{l-1} = \left(\Delta_l W_l\right) \odot \phi'(Z_{l-1})$$

Three moves, repeated: **(1)** gradient of a weight = error at its output × activation at its input; **(2)** error moves backwards by multiplying with the transposed weights; **(3)** it's gated elementwise by the activation derivative.

Real-life analogy: blame flowing back through the kitchen

Think of it like a customer complaining that the biryani is too salty (the loss). The complaint travels back: the finishing cook is blamed in proportion to how much salt he added; he passes blame to the prep cook in proportion to how much of her masala he used; a cook who was on a break for that dish (ReLU switched off) gets none. Each cook then adjusts their recipe by (blame reaching them) × (what they were handed to work with).

Formula, read aloud: $\frac{\partial L}{\partial W_l} = \Delta_l^\top H_{l-1}$ and $\Delta_{l-1} = (\Delta_l W_l) \odot \phi'(Z_{l-1})$ say: weight gradient = (error at the layer's output) × (the input it received); the previous layer's error = (this error sent back through the same weights) × (the activation's slope).

Where it breaks: a kitchen assigns blame by judgement, one cook at a time; backprop computes exact slopes for millions of weights in one sweep, and they all change a little together.

Numbers you can check by hand

$\mathbf{x} = (1, 2)$, $y = 1$, $W_1 = \begin{pmatrix} 0.1 & 0.3 \\ -0.3 & 0.2 \end{pmatrix}$, $\mathbf{b}_1 = (0, 0.1)$, $\mathbf{w}_2 = (0.5, -0.4)$, $b_2 = 0.1$.

Forward: $\mathbf{z}_1 = (0.1 + 0.6,\ \ -0.3 + 0.4 + 0.1) = (0.7, 0.2)$; $\mathbf{a}_1 = (0.7, 0.2)$; $z_2 = 0.35 - 0.08 + 0.1 = 0.37$; $p = \sigma(0.37) = 0.5915$; $L = -\log 0.5915 = 0.5252$.

Backward: $\delta_2 = 0.5915 - 1 = -0.4085$. $\partial L/\partial \mathbf{w}_2 = \delta_2 \mathbf{a}_1 = (-0.2860, -0.0817)$, $\partial L/\partial b_2 = -0.4085$. $\partial L/\partial \mathbf{a}_1 = \mathbf{w}_2 \delta_2 = (-0.2043,\ 0.1634)$; both $z_1 > 0$, so $\boldsymbol{\delta}_1 = (-0.2043,\ 0.1634)$.

$$\frac{\partial L}{\partial W_1} = \boldsymbol{\delta}_1 \mathbf{x}^\top = \begin{pmatrix} -0.2043 & -0.4085 \\ 0.1634 & 0.3268 \end{pmatrix}$$

(Matches a finite-difference check to 8 decimals.) One SGD step with $\eta = 0.5$ lowers the loss from 0.525 to 0.275. Sanity reading: the output was too low for a positive, so $\mathbf{w}_2$'s weight on the unit that's helping (unit 1) rises and the hurting weight (−0.4) moves up toward zero, while W_1 changes to increase unit 1's activation and decrease unit 2's (whose outgoing weight is negative).

Cost and memory. The backward pass costs about 2× the forward pass (two matrix products per layer: one for the weight gradient, one to propagate the error), so a training step ≈ 3× a forward pass. Memory: all activations must be stored for the backward pass — often the dominant memory cost; **activation checkpointing** recomputes some of them to trade compute for memory. **Why backprop is efficient:** reverse-mode differenti-

ation computes the gradient w.r.t. *all* parameters in one backward sweep, because the loss is a single scalar — forward-mode would need one sweep per parameter.

Gradient checking: compare backprop gradients with centered finite differences $\frac{L(\theta+\epsilon)-L(\theta-\epsilon)}{2\epsilon}$ (error $O(\epsilon^2)$) on a few parameters, in float64, with relative error $\frac{|g_a-g_n|}{\max(|g_a|,|g_n|)} \lesssim 10^{-6}$. Beware kinks (ReLU at 0) and randomness (turn dropout off).

20.5 Weight initialization

Zero (or identical) initialization fails: every unit in a layer computes the same thing and receives the same gradient — **symmetry** is never broken, so the layer behaves like one unit. Random initialization breaks symmetry; its **scale** decides whether signals survive depth.

Variance-preserving initialization

For $z = \sum_{i=1}^{n_{\text{in}}} w_i x_i$ with independent zero-mean w_i, x_i: $\text{Var}(z) = n_{\text{in}} \text{Var}(w) \text{Var}(x)$. To keep $\text{Var}(z) \approx \text{Var}(x)$ layer after layer, set $\text{Var}(w) = 1/n_{\text{in}}$. The same argument backwards (gradients) asks for $1/n_{\text{out}}$.

- **Xavier / Glorot** (tanh, sigmoid, linear): $\text{Var}(w) = \frac{2}{n_{\text{in}}+n_{\text{out}}}$ — a compromise between forward and backward.
- **He / Kaiming** (ReLU): $\text{Var}(w) = \frac{2}{n_{\text{in}}}$ — the factor 2 compensates for ReLU zeroing half its inputs, which halves the second moment of the activations.

Too small ⇒ activations and gradients shrink geometrically with depth (vanishing); too large ⇒ they grow (exploding) or saturate sigmoids/tanh.



Real-life analogy: announcement speakers along a platform

Think of it like a chain of loudspeakers relaying an announcement down a long railway platform, each feeding the next. If every amplifier's knob is a little too low, the message fades to silence by the far end; a little too high, and it becomes a screech. So each knob is set to keep the volume equal to what came in — turned down more when many microphones feed one amplifier. ReLU mutes about half the channels, so He initialization doubles the gain. Zero initialization makes all speakers identical, so they can never learn different jobs.

Formula, read aloud: $\text{Var}(z) = n_{\text{in}} \text{Var}(w) \text{Var}(x)$ says: output spread = (number of inputs) × (weight spread) × (input spread); choose weight spread $1/n_{\text{in}}$ to keep it level.

Where it breaks: speakers are set once, but weights keep changing during training, so initialization only gives a healthy start — normalization and residuals keep it healthy later.

Biases are usually initialized to 0. Special cases: initialize the output layer's bias to the log-odds of the base rate for imbalanced classification (so the first steps don't waste effort learning the prior); initialize the last layer of each residual branch near zero so blocks start as identity.

20.6 Batch normalization

For each feature (channel), over the mini-batch: $\hat{x} = \frac{x-\mu_B}{\sqrt{\sigma_B^2+\epsilon}}$, then $y = \gamma\hat{x} + \beta$ with learned γ, β (so the layer can undo the normalization if that's optimal).

Train vs. inference: training uses batch statistics; inference uses **running averages** of μ, σ^2 collected during training — a classic source of train/serve mismatch (forgetting `model.eval()`, or batches at serving time that differ from training batches).



Real-life analogy: relative grading in college

Think of it like relative grading in a college. Each exam's raw marks become "how far above or below this batch's average", so a brutal paper or an easy one doesn't throw off the next stage; the college can still stretch and shift the final scale (the learned γ and β). But a student who sits a make-up exam alone has no batch to compare with, so you use the long-run average of past batches (running statistics). Forgetting `model.eval()` is like grading that lone student against himself.

Formula, read aloud: $\hat{x} = \frac{x-\mu_B}{\sqrt{\sigma_B^2+\epsilon}}$ says: subtract the batch average and divide by the batch spread (plus a tiny number so you never divide by zero); then $y = \gamma\hat{x} + \beta$ lets the network re-stretch and re-shift.

Where it breaks: relative grading is about fairness; batch norm's real payoff is smoother optimization, and with tiny batches the "batch average" is too noisy to help.

Why does batch normalization help?

The original motivation ("reducing internal covariate shift") is contested; better-supported mechanisms:

1. **Better conditioning / smoother loss landscape** (Santurkar et al., 2018): normalizing each layer's inputs keeps their scale fixed, so the gradient's size and direction change more predictably — **higher learning rates** become stable and training is faster and less sensitive to initialization.
2. **Scale invariance:** the output of a BN'd layer doesn't change if you scale its incoming weights, which decouples weight magnitude from the function (and makes weight decay act as an effective learning-rate controller).
3. **Regularization:** batch statistics are noisy (they depend on which other examples are in the batch), injecting noise similar to dropout.

Where it struggles: small batches (noisy statistics — use GroupNorm/LayerNorm), variable-length sequences and RNNs, distributed training (needs synchronized statistics), and online/single-example inference paths that differ from training.

20.7 Layer normalization

Normalize across the **features of each example** (not across the batch), then scale/shift: μ, σ computed per example over its hidden units.

Real-life analogy: auto-exposure on each photo

Think of it like fixing the brightness of photos. **BatchNorm** balances each photo using the average brightness of the whole album shot together — great for big albums, shaky for an album of two. **LayerNorm** is each photo's own auto-exposure: it uses only that photo's pixels, so it behaves the same whether you shoot one photo or a hundred, and the same at shooting time and editing time (training and inference).

Where it breaks: pixels in a photo are all the same kind of thing, but a layer's hidden units measure different features; LayerNorm assumes they are comparable enough to share one mean and spread — natural in transformers, less so for CNN channels.

	BatchNorm	LayerNorm
Statistics over	batch (per feature/channel)	features (per example)
Batch-size dependence	yes — degrades with small batches	none
Train/inference behavior	different (batch vs. running stats)	identical
Typical home	CNNs for vision, large batches	transformers, RNNs, small-batch or online settings
Why there	channels are comparable across examples; batch statistics are rich	sequence lengths vary, batch statistics across tokens/ positions are awkward; per-token normalization is natural

RMSNorm (normalize by root-mean-square only, no mean subtraction) is a cheaper variant used in many recent large language models. Transformers moved from "post-LN" to **pre-LN** (normalize the input to each sublayer), which keeps the residual path clean and trains more stably at depth.

20.8 Dropout

During training, zero each activation with probability p and scale the survivors by $\frac{1}{1-p}$ (**inverted dropout**) so the expected activation is unchanged; at inference, use all units with no scaling.

Why it regularizes: (1) units can't rely on specific other units being present, so they learn redundant, independently useful features (less **co-adaptation**); (2) training samples from an exponential number of "thinned" sub-networks that share weights, and inference approximates averaging them — a cheap **ensemble**; (3) it injects noise, which acts like a data-dependent L2 penalty in simple models. **Practical notes:** typical p 0.1–0.5; less useful with BatchNorm (conflicting variance shifts) and in large-data regimes; forgetting to disable it at inference produces noisy, degraded predictions; **MC dropout** (keeping it on at inference and averaging several passes) gives a rough uncertainty estimate.

Real-life analogy: benching players at practice

Think of it like a cricket coach who randomly benches a few players at every practice. Nobody can lean on the star batter always being there, so everyone learns to contribute, and the team learns to win in many different combinations. On match day all eleven play, and the team is sturdier for it. That is dropout: random units switched off in training, all units on at inference.

Formula, read aloud: keep each unit with probability $1-p$ and multiply survivors by $\frac{1}{1-p}$ says: if 20% are benched, the other 80% each "play 1.25× harder", so total effort matches match day on average.

Where it breaks: a benched player still learns by watching, but a dropped unit gets no update that step; and at inference dropout is simply off (unless kept on deliberately for MC-dropout uncertainty).

20.9 Residual connections

$$\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l; \theta_l)$$

Real-life analogy: editors adding margin notes

Think of it like a report passing through 50 editors. In a plain deep network each editor must retype the whole report, and after 50 retypings errors pile up and the original is lost. With residual connections each editor keeps the original text and only adds notes in the margin; an editor with nothing useful to say adds nothing, and the report passes through untouched. Feedback from the final reader also reaches the first editor directly instead of being retyped 50 times on the way back.

Formula, read aloud: $\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l; \theta_l)$ says: next output = this input, passed through unchanged, + a correction the block learns.

Where it breaks: margin notes stay separate from the text, but a network adds each correction straight into the same vector, so later blocks read a blend of all earlier corrections (the "residual stream").

Why do residual connections help?

1. **Gradient highway.** $\frac{\partial \mathbf{h}_{l+1}}{\partial \mathbf{h}_l} = I + \frac{\partial F}{\partial \mathbf{h}_l}$. Through L blocks, the backward product contains the identity term, so the gradient reaching early layers includes a direct, un-shrunk copy of the gradient at the top instead of a product of many possibly-small Jacobians.
2. **Easy identity.** If a block isn't useful, it only needs $F \approx 0$ (easy — small weights), rather than learning an identity mapping through nonlinear layers (hard). So adding depth can't easily make the network worse — which fixes the "**degradation problem**": before ResNets, deeper plain networks had *higher training error* than shallower ones (an optimization failure, not overfitting).
3. **Ensemble of paths.** A residual network behaves somewhat like an ensemble of many shallower networks (paths that skip different blocks), most gradient flowing through relatively short paths.

They're why 100+-layer CNNs and every transformer (residual stream + sublayers) are trainable.

20.10 Why can deeper networks be harder to optimize?

Depth multiplies everything

The gradient reaching layer 1 is a product of L Jacobians. Unless each has "gain" ≈ 1 in the relevant directions, the product shrinks or blows up exponentially in L (vanishing/exploding gradients). Depth also makes the loss surface more ill-conditioned (different layers need very different step sizes; small changes in early layers are amplified by later ones — the learning rate that is safe for one layer is useless for another), creates more saddles and plateaus, and makes initialization matter more. Remedies target exactly these: careful initialization (gain ≈ 1), normalization (re-standardizes each layer), residual connections (identity path, gain ≥ 1 by construction), adaptive optimizers (per-parameter step sizes), warmup and gradient clipping.

Real-life analogy: compound interest gone wrong

Think of it like money left in an account for 50 years at a slightly wrong interest rate. At −10% a year, Rs 1 lakh shrinks to about Rs 500; at +10%, it swells to about Rs 1.17 crore. The gradient reaching layer 1 is a product of one "rate" per layer, so small deviations from 1 compound the same way — vanishing or exploding. Careful initialization, normalization and residual connections all aim to keep each layer's rate close to 1.

Formula, read aloud: gradient at layer 1 $\approx$ (gain of layer L) × ... × (gain of layer 1) × (gradient at the top) says: multiply one gain per layer — $0.9^{50} \approx 0.005$ and $1.1^{50} \approx 117$.

Where it breaks: an interest rate is one number, but a layer's "gain" is a matrix with different gains in different directions, so a network can vanish in some directions and explode in others at once.

20.11 Convolutional networks: the arithmetic interviewers ask for

A convolution slides a small filter over the input, computing a weighted sum at each position — the **same weights everywhere** (weight sharing) and **local** connections. That builds in translation equivariance and slashes parameters: a 3×3 conv from 64 to 128 channels has $3 \cdot 3 \cdot 64 \cdot 128 + 128 = 73{,}856$ parameters regardless of image size, versus billions for a dense layer on a 224×224 image.

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{S} \right\rfloor + 1, \qquad \text{receptive field: } r_l = r_{l-1} + (K_l - 1) \prod_{i<l} S_i$$

Examples: 32×32 input, $K = 3$, $P = 1$, $S = 1$ → 32×32 ("same" padding). $K = 3$, $P = 1$, $S = 2$ → 16×16. Two stacked 3×3 convs see a 5×5 region; three see 7×7 — with fewer parameters ($3 \cdot 9C^2 = 27C^2$ vs. $49C^2$) and more nonlinearities than one 7×7 conv. Strides and pooling grow the receptive field fast. FLOPs of a conv layer $\approx 2 \cdot H_{\text{out}} W_{\text{out}} \cdot K^2 C_{\text{in}} C_{\text{out}}$.

```
def conv2d_naive(x, w, stride=1, pad=0):
    """x: (C_in, H, W), w: (C_out, C_in, kh, kw). Cross-correlation, as in deep-learning
libraries."""
    C_in, H, W = x.shape
    C_out, _, kh, kw = w.shape
    xp = np.pad(x, ((0, 0), (pad, pad), (pad, pad)))
    H_out = (H + 2 * pad - kh) // stride + 1
    W_out = (W + 2 * pad - kw) // stride + 1
    out = np.zeros((C_out, H_out, W_out))
    for i in range(H_out):
        for j in range(W_out):
```

```
            patch = xp[:, i * stride:i * stride + kh, j * stride:j * stride + kw]
            out[:, i, j] = np.tensordot(w, patch, axes=([1, 2, 3], [0, 1, 2]))
    return out
```

Checked against SciPy's `correlate` summed over channels (identical), and a 7×7 input with $K = 3$, $S = 2$, $P = 1$ gives a 4×4 output, as the formula predicts. **Deep-learning "convolution" is cross-correlation** (the kernel isn't flipped) — mention it if asked. **1×1 convolutions** mix channels without looking at neighbours (cheap bottlenecks); **depthwise-separable** convolutions (a per-channel spatial conv, then a 1×1) cut compute by roughly K^2 for mobile models.



Real-life analogy: a magnifying glass sliding over a map

Think of it like checking a big map with a small magnifying glass that looks for one pattern — say, a railway crossing. You slide the same glass over every part of the map; wherever the pattern appears, the glass "lights up". Many different glasses (channels) look for different patterns; later layers look at the *lit-up maps* to find bigger patterns (stations, cities). Stride is how far you jump between looks; padding lets the glass check the map's edges.

Formula, read aloud: output size = ⌊(input + 2 × padding − filter size) ÷ stride⌋ + 1: "how many places the glass can sit".

Where it breaks: a magnifying glass sees pixels exactly; CNNs learn their patterns from data, and translation equivariance is broken by pooling, strides and borders, so small shifts can still change predictions.

20.12 Recurrent networks and LSTMs: why the gates matter

An RNN updates a hidden state $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$. Backpropagation through time multiplies Jacobians $\frac{\partial h_t}{\partial h_{t-1}}$ across all steps, so gradients over long gaps **vanish or explode** (Part 14). The **LSTM** adds a cell state with an **additive** update, controlled by gates:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

When the forget gate $\mathbf{f}_t \approx 1$, $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} \approx 1$ — a gradient **highway** across time (the same idea as residual connections across depth). The GRU merges gates into two (update, reset) with similar performance and fewer parameters.

```
def lstm_cell(x, h, c, W, U, b):
    """W: (4H, D), U: (4H, H), b: (4H,). Gate order: input, forget, cell candidate, output."""
    Hd = h.shape[0]
    z = W @ x + U @ h + b
    sig = lambda a: 1 / (1 + np.exp(-a))
    i, f, g, o = sig(z[:Hd]), sig(z[Hd:2 * Hd]), np.tanh(z[2 * Hd:3 * Hd]), sig(z[3 * Hd:])
    c_new = f * c + i * g                                   # additive update: gradient
highway
    h_new = o * np.tanh(c_new)
    return h_new, c_new
```

Why transformers replaced them (Part 41): RNNs process tokens sequentially (no parallelism across time in training) and squeeze history into a fixed-size state; attention connects every pair of positions directly. RNN-like **state-space models** (linear recurrences that can be trained in parallel) have returned for very long sequences because their inference memory doesn't grow with length.

Real-life analogy: a notebook with a pencil and an eraser

Think of it like a student taking notes during a long lecture. A plain RNN rewrites the whole page every minute — early points blur away. An LSTM keeps a notebook: the **forget gate** is the eraser (what to remove), the **input gate** decides what new point to write, and the **output gate** decides what to read out when asked. Because notes are *added* rather than rewritten, early points can survive for a long time.

Formula, read aloud: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$: new notes = (old notes × what we keep) + (new point × how much we write it down).

Where it breaks: a notebook has pages to spare; an LSTM's cell is a fixed-size vector, so very long or detailed histories still get compressed — attention reads the full transcript instead.

20.13 Regularizing deep networks in practice

Technique	What it does	Typical use
Data augmentation	train on transformed copies (crops, flips, color jitter; for text, back-translation or paraphrase)	the most effective regularizer when invariances are known
Mixup / CutMix	train on convex mixtures of two images and their labels ($\tilde{x} = \lambda x_i + (1-\lambda)x_j$, same for y) / paste a patch of one image into another	image classification; smoother decision boundaries, better calibration
Label smoothing	soften one-hot targets (Part 15)	classification, transformers
Dropout	randomly zero activations (Part 20)	MLPs, some transformer layers
Stochastic depth	randomly skip entire residual blocks during training	very deep ResNets and vision transformers
Weight decay	shrink weights each step (AdamW, Part 14)	nearly always
Early stopping	stop when validation loss stops improving	nearly always
EMA of weights	evaluate an exponential moving average of the weights	stabler, often better final models (common in diffusion and self-supervised training)

Real-life analogy: training a cricket team for any pitch

Think of it like a coach preparing a team for an away series. Practising on the same home pitch every day produces players who fail elsewhere. So the coach varies conditions: different pitches and balls (augmentation), mixed drills (mixup), random players sitting out sessions so no one becomes indispensable (dropout, stochastic depth), and stopping practice before players burn out (early stopping).

Where it breaks: a coach knows the away conditions; augmentations must match real variations in *your* data — flipping medical scans left-right may create images that never occur and hurt the model.

20.14 A map of architectures (for orientation)

Architecture	Inductive bias	Where
MLP	none beyond smoothness	tabular (after embeddings), heads of other nets
CNN	locality, translation equivariance, hierarchy	images, audio spectrograms, some sequences
RNN / LSTM / GRU	sequential processing, recurrence; gating for long memory	older NLP/speech; streaming; small sequential problems
Transformer	content-based all-pairs interactions (attention), position via encodings	language, vision (ViT), multimodal, recsys sequences — $O(T^2)$ attention cost in sequence length
Embedding + MLP / two-tower	learned representations of IDs; dot-product retrieval	recommendation, search retrieval, ads
GNN	message passing over graph structure	fraud rings, molecules, social graphs



Real-life analogy: different ways of reading

Think of it like different ways of taking in information. A **CNN** scans a photo with a small magnifying glass, looking for the same local patterns (edges, textures) everywhere. An **RNN** reads a novel one word at a time, carrying a memory forward. A **Transformer** is a round-table meeting where every word can address every other word directly — powerful, but with T people the number of conversations grows like T^2. A **GNN** is gossip spreading along friendships. Each design builds in a guess about how the data is structured (its inductive bias).

Where it breaks: these "personalities" are starting assumptions, not strict rules — CNNs have handled text, transformers handle images (ViT), and with enough data a weaker built-in assumption can win.

Q: "Explain backpropagation to me on the whiteboard."

Strong answer (and draw while speaking). "Backprop is the chain rule applied efficiently. In the forward pass I compute and cache each layer's pre-activation z and activation a, then the loss. In the backward pass I define $\delta_l = \partial L / \partial z_l$. At the output, with softmax or sigmoid plus cross-entropy, $\delta_L = \hat{y} - y$. Then three rules repeat: the weight gradient is δ_l times the input activation transposed — an outer product with the same shape as W_l; the error propagates back as $W_l^\top \delta_l$; and it's gated by the activation derivative, elementwise, to get δ_{l-1}. Each step reuses cached forward values, so one backward pass gives all gradients for about twice the forward cost. The same view explains vanishing gradients: δ at layer 1 is a product of many weight matrices and activation derivatives."

Follow-ups

"What's the gradient of ReLU at 0?" — Undefined mathematically; frameworks use 0 (a valid subgradient). It almost never matters because exact zeros are rare.

"Why cache activations instead of recomputing them?" — Recomputing would redo the forward pass per layer (quadratic cost in depth). Caching is linear in depth but costs memory — checkpointing is the tunable middle ground.

"Why do we usually not apply dropout and BatchNorm together naively?" — Dropout changes activation variance between training and inference, while BN's running statistics are collected under dropout's training-time variance — a "variance shift" at inference. Put dropout after the last BN, or use one of them.

"Is a 1-hidden-layer network enough in practice, given universal approximation?" — It can represent the function but may need an impractically large width, and finding/generalizing the solution is harder than with depth, which reuses features compositionally.

Active recall — Set 12: Deep learning

Set 12 · Answer from memory, then check the Answer Key

Easy recall

1. Write the perceptron update rule.
2. Write the derivatives of sigmoid, tanh and ReLU.
3. What are Xavier and He initialization variances?
4. What does BatchNorm compute during training and inference?
5. What is inverted dropout?

Conceptual

6. What does the universal approximation theorem guarantee and not guarantee?
7. What is the dying ReLU problem?
8. How do BatchNorm and LayerNorm differ, and where is each used?
9. What is the degradation problem and how do residual connections fix it?
10. Why does the backward pass cost about twice the forward pass?

Why

11. Why do networks need nonlinear activations?
12. Why does zero initialization fail?
13. Why does He initialization have a factor of 2?
14. Why does BatchNorm allow higher learning rates?
15. Why can deeper networks be harder to optimize?

Scenario

16. Your model performs worse at inference than in validation during training; you use BatchNorm and dropout. First check?
17. Loss is stuck from the first step in a 20-layer sigmoid MLP. Explain and fix.
18. Many ReLU units output zero for all inputs after a few hundred steps. Why, and what do you do?
19. You must train with batch size 4 because of memory. Which normalization and why?
20. A 2-layer MLP matches a 10-layer one on training loss but generalizes worse. Hypotheses?

Mathematical

21. Derive $\partial L/\partial W_1$ for the 2-layer network with ReLU hidden layer and sigmoid + BCE output.

22. Show $\mathrm{Var}(z) = n\,\mathrm{Var}(w)\,\mathrm{Var}(x)$ for $z = \sum w_i x_i$ with independent zero-mean terms.
23. Show that the composition of two linear layers is a linear layer.

Difficult follow-ups

24. Why did transformers move from post-LN to pre-LN?
25. Explain why reverse-mode autodiff (backprop) is preferred over forward-mode for training neural networks.

2027 additions

26. A 32×32 input goes through a 3×3 conv with stride 2 and padding 1. Output size? Receptive field of three stacked stride-1 3×3 convs?
27. Why does the LSTM cell state mitigate vanishing gradients?
28. What is mixup and why does it help?

PART 21

ML Coding Interview

How ML coding rounds are graded

Correctness first, then: **vectorization** (no Python loops over examples where a matrix op works), **numerical stability** (log-sum-exp, stable sigmoid, no explicit inverses), **edge cases** (empty inputs, ties, zero variance, one class), **complexity** stated without being asked, and a **clear narration** — say the algorithm, the shapes and the invariants as you type. Every implementation below was run and checked against scikit-learn or finite differences.



The whole Part as one picture

Think of it like a chemistry practical in the board exam. The examiner first checks that your reaction actually works (correctness). But marks also go for clean technique — the right apparatus instead of doing everything drop by drop (vectorization), lab safety so nothing boils over (numerical stability), knowing what to do when a test tube cracks (edge cases), and explaining each step as you go (narration). Every section below is one such experiment: an algorithm, its safe technique, its traps, and the line to say out loud.

Where it breaks: a lab practical follows a fixed procedure, but an interviewer wants you to choose between approaches (solve vs. invert, full sort vs. partial) and explain the trade-off.

The 60-second narration template: *"The algorithm is . Inputs are X of shape (n, d) and y of shape (n,). The core step is , which is O(). The numerically dangerous part is , so I . Edge cases: . Let me test on a tiny example where I know the answer."*

21.1 Linear regression

Algorithm: least squares via the normal equation, with optional ridge; bias not penalized. **Pseudocode:** `Xb = [1, X]` → solve `(XbᵀXb + λP) w = Xbᵀy`.



Real-life analogy: decoding an electricity bill

Think of it like working out your electricity tariff from a year of old bills: bill = fixed monthly charge + rate × units used. The bias column is the fixed charge; the weights are the rates. Least squares finds the charge and rates that best match all the bills at once. Ridge adds a gentle pull that stops the rates swinging wildly to fit one odd bill — and never pulls on the fixed charge. "Solve, never inv" means solving the equations directly rather than first building the whole inverse table: less work and less rounding error.

Formula, read aloud: $(X_b^\top X_b + \lambda P)\,\mathbf{w} = X_b^\top \mathbf{y}$ says: (how the columns overlap with each other + a small penalty on the rates) × weights = (how the columns overlap with the answers).

Where it breaks: a real tariff is an exact rule; real data is noisy, and near-duplicate columns make the "rates" unstable — which is when you reach for ridge or `lstsq`.

```python
import numpy as np

def linreg_fit(X, y, l2=0.0):
    Xb = np.c_[np.ones(len(X)), X]                  # prepend bias column
    P = l2 * np.eye(Xb.shape[1]); P[0, 0] = 0.0     # don't penalize the bias
    return np.linalg.solve(Xb.T @ Xb + P, Xb.T @ y)  # solve, never inv

def linreg_predict(X, w):
    return w[0] + X @ w[1:]
```

Time	Space	Stability	Edge cases
$O(nd^2 + d^3)$	$O(d^2)$ extra	`solve` not `inv`; `lstsq` (SVD) if $X^\top X$ may be singular; $\kappa(X^\top X) = \kappa(X)^2$	collinear/duplicate columns (singular — use ridge or lstsq), $d > n$, constant features, `y` shape (n,1) vs (n,) broadcasting

Say: "Closed form is ideal when d is up to a few thousand; for larger d or streaming data I'd switch to (mini-batch) gradient descent, below."

21.2 Logistic regression

Algorithm: gradient descent on mean binary cross-entropy. **Pseudocode:** repeat: `p = σ(Xw+b)`; `g = Xᵀ(p-y)/n + λw`; `w -= η g`.



Real-life analogy: a garland seller's guesses

Think of it like a flower seller at a temple gate learning to guess whether each passer-by will buy a garland, from clues such as a festival day, a puja plate in hand, and the time of day. Each evening he reviews his guesses: when he said "80% will buy" and the person walked past, he lowers the weight of the clues that made him confident; when he said "20%" and they bought, he raises them. Each correction is simply "how wrong he was" × "how strongly that clue was present".

Formula, read aloud: $\mathbf{g} = X^\top(\mathbf{p} - \mathbf{y})/n + \lambda\mathbf{w}$ says: gradient = average over people of (predicted chance − what actually happened) × their clues, plus a small pull toward zero.

Where it breaks: if one clue perfectly separates buyers from walkers, the seller would just rely on it, but the model's weights grow without limit unless you add the `l2` penalty.

```python
def sigmoid(z):
    out = np.empty_like(z, dtype=float)             # branch avoids exp overflow
    pos = z >= 0
    out[pos] = 1.0 / (1.0 + np.exp(-z[pos]))
    ez = np.exp(z[~pos])
    out[~pos] = ez / (1.0 + ez)
    return out

def logreg_fit(X, y, lr=0.1, epochs=1000, l2=0.0):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(epochs):
        p = sigmoid(X @ w + b)
        err = p - y                                 # dL/dz for BCE
        w -= lr * (X.T @ err / n + l2 * w)
        b -= lr * err.mean()
    return w, b

def bce_from_logits(z, y):
    return np.mean(np.logaddexp(0.0, z) - y * z)    # log(1+e^z) - y z, overflow-safe
```

Time	Space	Stability	Edge cases
$O(nd)$ per epoch	$O(d)$	`exp(-z)` overflows for $z \ll 0$ → branch or `np.logaddexp`; compute loss from logits, never `log(p)` with $p = 0$	separable data (weights diverge — add `l2`); unscaled features (slow); all-one-class labels

Say: "The gradient is (prediction − label) × input, same as linear regression, because sigmoid and cross-entropy cancel. It's convex, so zero init is fine — unlike a neural net."

21.3 Gradient descent (generic)

```python
def gradient_descent(grad_fn, w0, lr=0.1, max_iter=10_000, tol=1e-8):
    w = np.asarray(w0, dtype=float).copy()
    for it in range(max_iter):
        g = grad_fn(w)
        if np.linalg.norm(g) < tol:                    # stationary point reached
            break
        w -= lr * g
    return w, it
```

Time: $O(\text{iters} \times \text{cost of grad})$. **Stability:** diverges if `lr` > 2/λ_max of the Hessian; add a check that the loss isn't increasing / is finite. **Edge cases:** copy `w0` (don't mutate the caller's array), integer arrays (`w -= ...` on ints silently truncates — hence `dtype=float`). **Say:** "Stopping on gradient norm, with a max-iteration guard; in practice I'd also track loss and use a schedule."



Real-life analogy: adjusting a hot-water shower

Think of it like adjusting a shower's temperature knob. Feel the water (compute the gradient), turn the knob a little in the direction that helps (a step of size `lr`), and stop when the temperature has stopped changing (gradient norm below `tol`) or after a maximum number of tries (`max_iter`). Turn the knob too hard and you swing between freezing and scalding (divergence). And adjust your own knob: copy `w0` instead of changing the caller's.

Formula, read aloud: $\mathbf{w} \leftarrow \mathbf{w} - \eta\,\mathbf{g}$ says: new setting = old setting − (step size) × (slope).

Where it breaks: a shower has one knob and one comfortable temperature; a loss has many knobs and flat stretches where the slope is tiny even though you are far from the best spot.

21.4 K-nearest neighbors

Pseudocode: squared distances via the expansion $\|a-b\|^2 = \|a\|^2 - 2a^\top b + \|b\|^2$; `argpartition` for the k smallest; majority vote with a tie-break.

Real-life analogy: asking the nearest houses

Think of it like guessing which IPL team a new resident of a housing colony supports by asking the five nearest houses and going with the majority — and, if the vote is tied, trusting the closest house. You need only the five nearest, not all 2,000 houses sorted by distance; that is what `argpartition` saves. The distance trick measures every house-to-house distance at once with a single matrix product.

Formula, read aloud: $\|a-b\|^2 = \|a\|^2 - 2a^\top b + \|b\|^2$ says: squared distance = (size of a)² − 2 × (how much a and b point the same way) + (size of b)².

Where it breaks: in a colony distance is obvious; with many features it depends on units (unscaled income swamps age), and in high dimensions almost everyone is about equally far away.

```
def pairwise_sq_dists(A, B):
    # ||a-b||^2 = ||a||^2 - 2 a.b + ||b||^2 ; clip tiny negatives from rounding
    d2 = (A ** 2).sum(1)[:, None] - 2 * A @ B.T + (B ** 2).sum(1)[None, :]
    return np.maximum(d2, 0.0)

def knn_predict(X_train, y_train, X_query, k=5):
    k = min(k, len(X_train))
    d2 = pairwise_sq_dists(X_query, X_train)          # (q, n)
    idx = np.argpartition(d2, k - 1, axis=1)[:, :k]   # O(n) per query, unordered top-k
    labels = y_train[idx]                             # (q, k)
    preds = []
    for row, drow in zip(labels, np.take_along_axis(d2, idx, 1)):
        vals, counts = np.unique(row, return_counts=True)
        best = vals[counts == counts.max()]
        if len(best) > 1:                             # tie: nearest neighbor among tied classes
            best = [row[np.argmin(np.where(np.isin(row, best), drow, np.inf))]]
        preds.append(best[0])
    return np.array(preds)
```

Time	Space	Stability	Edge cases
$O(qnd)$ distances + $O(qn)$ selection	$O(qn)$ distance matrix — chunk queries if large	the expansion can give tiny negative values → clip; float32 cancellation for nearby points	$k > n$; ties in votes and in distances; unscaled features; duplicate points

Say: "`argpartition` avoids a full sort — O(n) instead of O(n log n) per query. For big n I'd use a KD-tree in low dimensions or an ANN index like HNSW."

21.5 K-means (with k-means++)

```
def kmeans_pp_init(X, k, rng):
    centers = [X[rng.integers(len(X))]]
    d2 = pairwise_sq_dists(X, np.array(centers)).min(1)
    for _ in range(1, k):
        probs = d2 / d2.sum() if d2.sum() > 0 else np.full(len(X), 1 / len(X))
        centers.append(X[rng.choice(len(X), p=probs)])          # sample ∝ D(x)^2
        d2 = np.minimum(d2, pairwise_sq_dists(X, centers[-1][None]).ravel())
    return np.array(centers)

def kmeans(X, k, n_iter=100, tol=1e-6, seed=0):
    rng = np.random.default_rng(seed)
    C = kmeans_pp_init(X, k, rng)
    for _ in range(n_iter):
        labels = pairwise_sq_dists(X, C).argmin(1)              # assignment step
        newC = np.array([X[labels == j].mean(0) if np.any(labels == j)
                         else X[rng.integers(len(X))]           # re-seed an empty cluster
```

```
                        for j in range(k)])
        shift = np.linalg.norm(newC - C)
        C = newC                                                    # update step
        if shift < tol:
            break
    labels = pairwise_sq_dists(X, C).argmin(1)
    inertia = pairwise_sq_dists(X, C)[np.arange(len(X)), labels].sum()
    return C, labels, inertia
```

Time	Space	Stability	Edge cases
$O(nkd)$ per iteration	$O(nk)$ distances	none severe; scale features first	**empty clusters** (re-seed), $k > n$, duplicate points (k-means++ with zero total distance), non-convergence (cap iterations)

Say: "Each step can't increase the objective, so it converges, but to a local optimum — hence k-means++ and multiple restarts keeping the lowest inertia." (Checked: inertia 1127.4 vs. scikit-learn's 1127.4 on 4 blobs.)



Real-life analogy: placing delivery hubs

Think of it like placing k delivery hubs for a food app. **Assign:** every customer is served by the nearest hub. **Update:** move each hub to the middle of its customers. Repeat until the hubs stop moving. **k-means++** picks spread-out starting spots: each new hub is more likely to go where customers are far from every existing hub. A hub that ends up with no customers is moved to a random customer's location.

Formula, read aloud: sampling $\propto D(x)^2$ says: a point's chance of becoming the next starting centre = (its squared distance to the nearest chosen centre) ÷ (the sum of those over all points).

Where it breaks: real hubs care about roads and traffic; k-means uses straight-line distance, prefers round, similar-sized clusters, and settles into a local best that depends on the start.

21.6 Decision tree basics (Gini, classification)

Pseudocode: for each feature, sort once, sweep thresholds moving one sample from right to left while updating class counts (O(n) per feature), pick the best Gini decrease; recurse until max depth, purity, or no gain.



Real-life analogy: a divider along a fruit row

Think of it like a row of mangoes and guavas lined up by weight. Slide a divider along the row one fruit at a time; each move shifts just one fruit from the right-hand tally to the left-hand tally, so you never recount from scratch (the incremental count trick). At each position, check how pure both piles are and remember the best divider. Then repeat inside each pile, and inside theirs, until the piles are pure or the tree is deep enough.

Formula, read aloud: $\text{Gini} = 1 - \sum_k p_k^2$ says: impurity = the chance that two fruits drawn at random (with replacement) from a pile are of different kinds.

Where it breaks: here one property decides; the algorithm tries every feature and every cut, and it is greedy — the best cut now need not lead to the best tree overall.

```
def gini(y):
    if len(y) == 0:
        return 0.0
    p = np.bincount(y) / len(y)
    return 1.0 - np.sum(p ** 2)

def best_split(X, y, min_leaf=1):
```

```
    n, d = X.shape
    parent, best = gini(y), (None, None, 0.0)          # (feature, threshold, gain)
    n_classes = y.max() + 1
    for j in range(d):
        order = np.argsort(X[:, j], kind="stable")
        xs, ys = X[order, j], y[order]
        left = np.zeros(n_classes)
        right = np.bincount(ys, minlength=n_classes).astype(float)
        for i in range(n - 1):                         # move one sample left at a time
            left[ys[i]] += 1; right[ys[i]] -= 1
            if xs[i] == xs[i + 1] or i + 1 < min_leaf or n - i - 1 < min_leaf:
                continue                               # can't split between equal values
            nl, nr = i + 1, n - i - 1
            g = (parent - (nl / n) * (1 - np.sum((left / nl) ** 2))
                        - (nr / n) * (1 - np.sum((right / nr) ** 2)))
            if g > best[2]:
                best = (j, (xs[i] + xs[i + 1]) / 2, g)
    return best

def build_tree(X, y, depth=0, max_depth=5, min_leaf=1):
    j, t, gain = best_split(X, y, min_leaf)
    if depth >= max_depth or j is None or gain <= 1e-12:
        return {"leaf": True, "pred": np.bincount(y).argmax(), "proba": np.bincount(y) / len(y)}
    m = X[:, j] <= t
    return {"leaf": False, "feat": j, "thr": t,
            "left": build_tree(X[m], y[m], depth + 1, max_depth, min_leaf),
            "right": build_tree(X[~m], y[~m], depth + 1, max_depth, min_leaf)}

def tree_predict_one(node, x):
    while not node["leaf"]:
        node = node["left"] if x[node["feat"]] <= node["thr"] else node["right"]
    return node["pred"]
```

Time	Space	Stability	Edge cases
$O(dn\log n)$ per node (sort) → ~ $O(dn\log^2 n)$ balanced; sweep is $O(nK)$ per feature	$O(\text{nodes})$; recursion depth ≤ max_depth	exact comparisons; midpoint thresholds	identical feature values (no valid split), pure nodes, empty children, `bincount` needs non-negative int labels

Say: "The incremental count update is the key trick: evaluating every threshold is O(n) after sorting instead of O(n²). Production libraries pre-bin features into histograms." (Checked: depth-3 tree on Iris matches scikit-learn's training accuracy, 0.973.)

21.7 PCA

```
def pca(X, k):
    mu = X.mean(0)
    Xc = X - mu
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)   # Xc = U S Vt
    components = Vt[:k]                                 # (k, d) principal directions
    explained_var = S ** 2 / (len(X) - 1)
    ratio = explained_var[:k] / explained_var.sum()
    Z = Xc @ components.T                               # scores (n, k)
    return Z, components, ratio, mu
```

Time: $O(nd\min(n,d))$ (randomized SVD: ~$O(ndk)$). **Space:** $O(nd)$. **Stability:** SVD of centered data rather than eigendecomposition of $X^\top X$ (which squares the condition number). **Edge cases:** forgetting to center (PC1 points at the mean); sign ambiguity (components are defined up to sign — compare absolute values in tests); $k > \min(n,d)$; standardize if units differ. **Say:** "Rows of Vt are eigenvectors of the covariance; S²/(n−1) are its eigenvalues."

Real-life analogy: photographing a cricket bat

Think of it like photographing a cricket bat. From the side you see its full length; head-on it is a small rectangle. PCA finds the camera angle where the points look most spread out (the first component), then the next best angle at right angles to it, and so on. Centering first is putting the bat in the middle of the frame — skip it and the "best angle" just points at where the bat happens to sit.

Formula, read aloud: $X_c = USV^\top$ says: centered data = (each point's coordinates) × (how stretched each direction is) × (the directions); rows of $V^\top$ are the principal directions and $S^2/(n-1)$ their variances.

Where it breaks: PCA picks the angle with the most spread, which is not always the most useful for prediction, and it finds only flat (linear) views — it can't unroll a curved shape.

21.8 Softmax and cross-entropy

```python
def softmax(Z):
    Z = Z - Z.max(axis=-1, keepdims=True)            # shift invariance → no overflow
    E = np.exp(Z)
    return E / E.sum(axis=-1, keepdims=True)

def log_softmax(Z):
    Z = Z - Z.max(axis=-1, keepdims=True)
    return Z - np.log(np.exp(Z).sum(axis=-1, keepdims=True))   # log-sum-exp

def cross_entropy(Z, y):
    """Z: (n, K) logits, y: (n,) integer labels."""
    return -log_softmax(Z)[np.arange(len(y)), y].mean()
```

Time/space: $O(nK)$. **Stability:** `exp(1000)` overflows → subtract the row max (softmax is shift-invariant); compute log-probabilities directly with log-sum-exp — never `np.log(softmax(Z))`, which gives `-inf` when a probability underflows to 0. **Edge cases:** 1-D input (use `axis=-1`), all-equal logits (uniform), labels out of range. (Checked: logits [1000, 1001, 999] → [0.245, 0.665, 0.090]; cross-entropy matches scikit-learn's `log_loss`.)

Real-life analogy: vote shares counted in lakhs

Think of it like working out each candidate's vote share, where a candidate's "votes" are e^{score}. Subtracting the top score from every score first is like counting votes in lakhs instead of one by one: the shares are identical, but the numbers stay small enough for your calculator. Cross-entropy is how surprised you are that the true winner won — minus the log of its share. Computing it straight from scores (log-sum-exp) avoids ever writing a share of exactly 0 and taking $\log 0$.

Formula, read aloud: $\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} - c)$ says: shifting every score by the same amount doesn't change anyone's share.

Where it breaks: real votes simply add up, but softmax exponentiates scores, so a score gap of just 5 gives one class about 150 times the "votes" of another.

21.9 Train/test split

```python
def train_test_split(X, y, test_size=0.2, stratify=False, seed=0):
    rng = np.random.default_rng(seed)
    n = len(y)
```

```
    if not stratify:
        idx = rng.permutation(n)
        n_test = int(round(test_size * n))
        test, train = idx[:n_test], idx[n_test:]
    else:                                          # keep class proportions in both parts
        test = []
        for c in np.unique(y):
            ci = rng.permutation(np.flatnonzero(y == c))
            test.extend(ci[:int(round(test_size * len(ci)))])
        test = np.array(test, dtype=int)
        train = np.setdiff1d(np.arange(n), test)
        train, test = rng.permutation(train), rng.permutation(test)
    return X[train], X[test], y[train], y[test]
```

Time: $O(n)$ (+ $O(n \log n)$ for `setdiff1d`). **Edge cases:** tiny classes (a class may get 0 test rows — warn), reproducibility (seeded generator), `X` and `y` indexed with the **same** permutation. **Say — the part that impresses:** "This is only valid for IID rows. For temporal data I'd split by time; for repeated users, by user groups; and I'd deduplicate before splitting."



Real-life analogy: dealing cards suit by suit

Think of it like dealing a shuffled deck into two hands. A plain shuffle might, by bad luck, give one hand almost no hearts. **Stratified** splitting deals each suit separately — 20% of the hearts, 20% of the spades — so both hands keep the deck's mix. And indexing `X` and `y` with the same permutation is like moving each card together with its label sticker: shuffle them separately and every label lands on the wrong card.

Where it breaks: cards are independent, but real rows often aren't (the same user twice, consecutive days), and then no careful shuffling is valid — split by time or by group instead.

21.10 Standard scaler

```
class StandardScaler:
    def fit(self, X):
        self.mean_ = X.mean(0)
        self.scale_ = X.std(0)
        self.scale_[self.scale_ == 0] = 1.0            # constant column: avoid divide-by-zero
        return self
    def transform(self, X):
        return (X - self.mean_) / self.scale_
    def fit_transform(self, X):
        return self.fit(X).transform(X)
```

Time/space: $O(nd)$ / $O(d)$. **Stability:** for streaming or huge data, use Welford's online algorithm (the naive $\mathbb{E}[x^2] - \mathbb{E}[x]^2$ suffers catastrophic cancellation). **Edge cases:** zero-variance columns, NaNs (use `nanmean` / `nanstd` or impute first). **Say:** "Fit on training data only; the same statistics are applied to validation, test and serving — that's the leakage-safe contract."

21.11 Confusion matrix, precision, recall, F1 (and AUC)

```
def confusion_matrix(y_true, y_pred, n_classes=None):
    K = n_classes or int(max(y_true.max(), y_pred.max())) + 1
    cm = np.zeros((K, K), dtype=int)
    np.add.at(cm, (y_true, y_pred), 1)                 # rows = true, cols = predicted
    return cm
```

```python
def precision_recall_f1(y_true, y_pred, positive=1):
    tp = np.sum((y_pred == positive) & (y_true == positive))
    fp = np.sum((y_pred == positive) & (y_true != positive))
    fn = np.sum((y_pred != positive) & (y_true == positive))
    precision = tp / (tp + fp) if tp + fp > 0 else 0.0  # no positive predictions
    recall = tp / (tp + fn) if tp + fn > 0 else 0.0     # no actual positives
    f1 = 2 * precision * recall / (precision + recall) if precision + recall > 0 else 0.0
    return precision, recall, f1

def roc_auc(y_true, scores):
    """AUC = P(score_pos > score_neg), ties count 1/2 -- via average ranks (Mann-Whitney)."""
    order = np.argsort(scores, kind="mergesort")
    s = scores[order]
    ranks = np.empty(len(s))
    i = 0
    while i < len(s):                                   # average ranks over ties
        j = i
        while j + 1 < len(s) and s[j + 1] == s[i]:
            j += 1
        ranks[i:j + 1] = (i + j) / 2 + 1
        i = j + 1
    r = np.empty(len(s)); r[order] = ranks
    n_pos = y_true.sum(); n_neg = len(y_true) - n_pos
    if n_pos == 0 or n_neg == 0:
        raise ValueError("AUC undefined with one class")
    return (r[y_true == 1].sum() - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg)
```

Time: $O(n)$ for the counts (`np.add.at` handles repeated index pairs correctly — `cm[y_true, y_pred] += 1` would not); AUC $O(n \log n)$. **Edge cases:** zero division (no predicted positives → precision undefined; scikit-learn warns and returns 0 by default — say which convention you use), label encodings other than 0/1, multi-class (macro vs. micro vs. weighted averaging), AUC with a single class, tied scores. **Say:** "AUC equals the probability a random positive outranks a random negative, which the rank-sum formula computes in O(n log n) instead of comparing all pairs in O(n_pos·n_neg)." (All four checked against scikit-learn.)



Real-life analogy: a sports-day line-up

Think of it like lining up all students on sports day by score, lowest at position 1. Add up the positions of the red-house students. If they had all been at the very bottom, that total would be $1 + 2 + \cdots + n_{\text{red}}$. Every place above that minimum means a red student stands above one more blue student. So (total − minimum) counts the red-beats-blue pairs; divide by all red–blue pairs and you have AUC — from one sort, instead of comparing every pair.

Formula, read aloud: $\text{AUC} = \frac{R_+ - n_+(n_+ + 1)/2}{n_+ n_-}$ says: (sum of the positives' ranks − the smallest sum they could possibly have) ÷ (number of positive–negative pairs).

Where it breaks: on sports day ties are rare, but model scores often tie, so tied items share the average position — which counts each tie as half a win.

21.12 Mini-batch gradient descent

```python
def minibatch_gd(X, y, grad_fn, w0, lr=0.05, batch_size=32, epochs=20, seed=0):
    rng = np.random.default_rng(seed)
    w = w0.copy()
    n = len(y)
    for ep in range(epochs):
        perm = rng.permutation(n)                       # reshuffle every epoch
        for start in range(0, n, batch_size):
            b = perm[start:start + batch_size]          # last batch may be smaller
            w -= lr * grad_fn(w, X[b], y[b])
    return w
```

Time: $O(\text{epochs} \cdot nd)$, same as full-batch per epoch but with n/B updates. **Space:** $O(Bd)$ working memory — the reason it scales to data that doesn't fit in memory (stream batches from disk). **Stability/edge cases:** the gradient must be an **average** over the batch (so the LR doesn't depend on batch size, and the smaller last batch isn't over-weighted); shuffle each epoch (sorted data gives biased batches); decay the LR for convergence. **Say:** "Each mini-batch gradient is an unbiased estimate of the full gradient; variance falls like 1/B."



Real-life analogy: a biscuit factory inspector

Think of it like a quality inspector at a biscuit factory. Instead of weighing a whole day's output before every tweak, she grabs a handful of packets from the belt, adjusts the machine a little, and repeats. Each handful is a noisy but honest reading; a bigger handful is less noisy. If packets come off sorted (all of machine A's, then machine B's), a handful tells her about one machine only — so mix them first (reshuffle every epoch). And she compares the *average* weight per packet, not the handful's total, so a smaller last handful doesn't look underweight.

Formula, read aloud: variance of a mini-batch gradient $\propto 1/B$ says: double the handful and you halve the variance of its reading.

Where it breaks: a factory machine has one right setting, but a loss can have many good ones, and the noise of small handfuls can even help the model wander out of poor ones.

21.13 Simple neural network (2-layer MLP, softmax output)

```
class MLP:
    """2-layer net: ReLU hidden, softmax output, cross-entropy, mini-batch SGD."""
    def __init__(self, d_in, d_hidden, n_classes, seed=0):
        rng = np.random.default_rng(seed)
        self.W1 = rng.normal(0, np.sqrt(2 / d_in), (d_in, d_hidden))      # He init
        self.b1 = np.zeros(d_hidden)
        self.W2 = rng.normal(0, np.sqrt(1 / d_hidden), (d_hidden, n_classes))
        self.b2 = np.zeros(n_classes)

    def forward(self, X):
        self.X = X
        self.Z1 = X @ self.W1 + self.b1
        self.A1 = np.maximum(0, self.Z1)
        self.Z2 = self.A1 @ self.W2 + self.b2
        return self.Z2                                    # logits

    def backward(self, y):
        n = len(y)
        dZ2 = softmax(self.Z2)
        dZ2[np.arange(n), y] -= 1
        dZ2 /= n                                          # d(mean CE)/dZ2 = (P - Y)/n
        self.dW2 = self.A1.T @ dZ2
        self.db2 = dZ2.sum(0)
        dA1 = dZ2 @ self.W2.T
        dZ1 = dA1 * (self.Z1 > 0)                         # ReLU gate
        self.dW1 = self.X.T @ dZ1
        self.db1 = dZ1.sum(0)

    def step(self, lr):
        for p, g in [(self.W1, self.dW1), (self.b1, self.db1),
                     (self.W2, self.dW2), (self.b2, self.db2)]:
            p -= lr * g                                   # in-place update

    def fit(self, X, y, lr=0.1, epochs=200, batch_size=64, seed=0):
        rng = np.random.default_rng(seed)
        for _ in range(epochs):
            perm = rng.permutation(len(y))
            for s in range(0, len(y), batch_size):
```

```
                b = perm[s:s + batch_size]
                self.forward(X[b]); self.backward(y[b]); self.step(lr)
        return self

    def predict(self, X):
        return self.forward(X).argmax(1)
```

Time	Space	Stability	Edge cases
forward $O(n(dh + hK))$; backward ≈ 2×	parameters $O(dh + hK)$ + cached activations $O(nh)$	stable softmax; He init for ReLU; divide by batch size once	zero init (symmetry), dead ReLUs with high LR, shape of `y` (int labels), forgetting to cache `X`

Checked: finite-difference gradient check on `W1` passes to 10^{-7}; 96% test accuracy on two-moons. **Say:** "The backward pass mirrors the forward one: (P–Y) at the output, weight gradient = input-activation-transposed × error, error flows back through W^T, gated by the ReLU mask. I'd verify with a gradient check before trusting it."

Real-life analogy: a tea-tasting panel

Think of it like a tea-tasting panel in Assam. Tasters (hidden units) each score one quality of a sample, and a taster speaks only when that quality is clearly present (ReLU); the head blender (output layer) combines their scores to name the grade (softmax). When she names the wrong grade, she changes how much she trusts each taster by (her error) × (that taster's score). She then passes feedback to each taster in proportion to that trust ($W^\top$); tasters who stayed silent on that sample get no feedback (the ReLU mask).

Formula, read aloud: $dZ_2 = (P - Y)/n$ says: output error = (predicted probabilities – the one-hot truth), averaged over the batch.

Where it breaks: a panel is corrected one person at a time; here every weight updates together each mini-batch, and only a gradient check proves the feedback is routed correctly.

21.14 Round 2: the implementations that show up in 2026–27 loops

Beyond the classics above, ML coding rounds now often ask for sampling utilities, evaluation helpers and a GenAI building block. Every function below was run and checked; the GenAI ones live in their Parts.

Asked as	Where in this book
"Implement scaled dot-product / multi-head attention with a causal mask and KV cache"	Part 41 code
"Implement top-k / top-p sampling and beam search"	Part 42 code
"Train a tiny BPE tokenizer"	Part 42 code
"Implement LoRA", "compute the DPO loss"	Part 43 code
"BM25 + reciprocal rank fusion"	Part 44 code
"Unbiased pass@k", "Bradley–Terry from pairwise wins"	Part 45 code
"Speculative sampling step", "int8 quantization"	Part 46 code
"nDCG@k", "ALS", "MMR re-ranking", "IPS estimator", "UCB / Thompson / LinUCB"	Part 47 code
"Logistic regression with Newton's method (IRLS)"	Part 6
"EM for a Gaussian mixture"	Part 12
"conv2d forward", "an LSTM cell"	Part 20

Asked as	Where in this book
"Bootstrap CI for AUC", "expected calibration error"	Part 16

✎ Reservoir sampling

Pick k items uniformly from a stream of unknown length in one pass and $O(k)$ memory. Keep the first k; for item t (0-indexed, $t \geq k$), keep it with probability $\frac{k}{t+1}$, replacing a random slot.

```python
def reservoir_sample(stream, k, rng=np.random.default_rng(0)):
    res = []
    for t, item in enumerate(stream):
        if t < k:
            res.append(item)
        else:
            j = rng.integers(0, t + 1)                    # keep item with prob k/(t+1)
            if j < k:
                res[j] = item
    return res
```

Why it's uniform (the proof they ask for): by induction, after t items each is in the reservoir with probability k/t. Item $t+1$ enters with probability $\frac{k}{t+1}$; an existing item survives with probability $1 - \frac{k}{t+1} \cdot \frac{1}{k} = \frac{t}{t+1}$, so it stays with probability $\frac{k}{t} \cdot \frac{t}{t+1} = \frac{k}{t+1}$. ✓ *Checked:* sampling 3 of 10 items 20,000 times, every item's frequency is 0.29–0.31 (theory 0.30).

✎ Weighted sampling without replacement

Efraimidis–Spirakis: give item i the key u_i^{1/w_i} with $u_i \sim \text{Uniform}(0,1)$ and take the k largest keys — one pass, streamable with a heap.

```python
def weighted_sample_without_replacement(weights, k, rng=np.random.default_rng(0)):
    """Efraimidis-Spirakis: key = u^(1/w); take the k largest keys."""
    u = rng.random(len(weights))
    keys = u ** (1.0 / np.asarray(weights, dtype=float))
    return np.argsort(-keys)[:k]
```

Checked: with weights (1, 2, 3, 4), the first pick's frequencies are (0.10, 0.21, 0.30, 0.39) vs. (0.1, 0.2, 0.3, 0.4).

✎ Stratified K-fold

```python
def stratified_kfold(y, k=5, seed=0):
    rng = np.random.default_rng(seed)
    folds = [[] for _ in range(k)]
    for c in np.unique(y):
        idx = rng.permutation(np.flatnonzero(y == c))
        for f, chunk in enumerate(np.array_split(idx, k)):
            folds[f].extend(chunk)
    all_idx = np.arange(len(y))
    for f in range(k):
        val = np.array(sorted(folds[f]))
        yield np.setdiff1d(all_idx, val), val
```

Checked: 100 labels with 10 positives → every validation fold gets exactly 2 positives. (Random K-fold can give a fold with 0 positives, making AUC undefined.)

✎ Exact Shapley values

$O(2^d)$ — fine for explaining a model with a handful of features, and the clearest way to show you understand the definition (Part 48).

```python
def shapley_exact(f, x, baseline):
    """Exact Shapley values for a model f on one example (feature 'absent' = baseline value).
O(2^d)."""
    from itertools import combinations
    from math import factorial
    d = len(x)
    phi = np.zeros(d)
    def v(S):
        z = baseline.copy(); z[list(S)] = x[list(S)]
        return f(z)
    for j in range(d):
        others = [i for i in range(d) if i ≠ j]
        for size in range(d):
            for S in combinations(others, size):
                wgt = factorial(size) * factorial(d - size - 1) / factorial(d)
                phi[j] += wgt * (v(S + (j,)) - v(S))
    return phi
```

Checked: for $f(z) = 2z_0 + 3z_1z_2$ at $x = (1, 2, 1)$ with a zero baseline, $\phi = (2, 3, 3)$: the linear term gets its own contribution, the interaction's 6 is split equally, and $\sum \phi = 8 = f(x) - f(0)$.



Real-life analogy: a lucky draw at a mela

Think of it like running a lucky draw at a mela where people keep arriving and you don't know how many will come. You keep a box of k tickets. Each new arrival wins a place in the box with a chance that shrinks as the crowd grows ($k/(t+1)$), bumping a random ticket out. At the end, everyone who came had exactly the same chance of being in the box — without ever needing to know the crowd size in advance. That's reservoir sampling.

Where it breaks: a mela draw is done by hand; in code, the subtle bug is using k/t instead of $k/(t+1)$ (off-by-one), which quietly biases the sample toward early or late items — always test uniformity empirically.

21.15 Coding-round checklist

Before you say "done"

1. **Shapes** — print or assert them; `(n,)` vs. `(n,1)` broadcasting.
2. **Tiny test** with a known answer (2 points, identity matrix, perfectly separable data).
3. **Stability** — max-subtraction, log-sum-exp, `logaddexp`, clipping, `solve` vs. `inv`, `eps` in denominators.
4. **Edge cases** — empty input, one class, ties, zero variance, k > n, NaNs.
5. **Complexity** — time and memory, and what you'd change at 100× scale (chunking, streaming, ANN indexes, histogram binning, sparse matrices).
6. **Leakage-safe API** — `fit` on train, `transform` everywhere else.

Active recall — Set 13: ML coding

Set 13 · Answer from memory, then check the Answer Key

Easy recall

1. Why subtract the max before softmax?

2. What's the complexity of brute-force KNN prediction for q queries?
3. What is the per-iteration cost of k-means?
4. Why use `np.add.at` for a confusion matrix?
5. How do you get AUC in O(n log n)?

Conceptual

6. Why should mini-batch gradients be averaged, not summed?
7. What does `argpartition` buy you in KNN?
8. Why compute cross-entropy from logits?
9. How does the incremental count update make split search O(n) per feature?
10. Why is SVD preferred to eigendecomposition of $X^\top X$ for PCA?

Why

11. Why can't a neural network be initialized to zeros but logistic regression can?
12. Why do you need to re-seed empty clusters in k-means?
13. Why fit the scaler only on the training split?
14. Why does `x² - 2xy + y²` need clipping at 0?
15. Why reshuffle every epoch?

Scenario

16. Your from-scratch logistic regression returns NaN for some inputs. Where do you look?
17. Your KNN runs out of memory with 1M training points and 100K queries. Fix.
18. Your from-scratch MLP's loss doesn't decrease. How do you debug it systematically?
19. The interviewer asks you to make k-means work on 1 billion points. What changes?
20. Your precision function crashes on a batch where the model predicts no positives. What should it return, and why does the convention matter?

Mathematical

21. Derive the gradient $(P - Y)/n$ of mean softmax cross-entropy w.r.t. logits.
22. Show that $\mathrm{softmax}(\mathbf{z}) = \mathrm{softmax}(\mathbf{z} - c)$.
23. Derive the Mann–Whitney formula for AUC from ranks.

Difficult follow-ups

24. Implement a numerically stable online (streaming) mean and variance. Why is the naive formula unstable?
25. How would you implement the tree split search with histograms, and what's its complexity?

2027 additions

26. Write reservoir sampling and prove it's uniform.
27. How do you sample k items without replacement with probabilities proportional to weights in one pass?
28. Compute exact Shapley values for $f(z) = 2z_0 + 3z_1z_2$ at $x = (1, 2, 1)$ with a zero baseline.

Unit VI

Debugging, Production & System Design

Part 22 — ML Debugging · Part 23 — Production ML
Part 24 — ML System Design (framework + 18 designs, incl. 6 GenAI)
Part 25 — Data-Centric ML · Part 26 — Practical Model Selection

Senior interviews are mostly here: what breaks, how you'd find it, and what you'd build so it doesn't.

PART 22

ML Debugging

Debugging ML is debugging a pipeline, not a model

Most ML failures are **data and pipeline bugs that look like modeling problems**. The model is usually the last suspect. Work through the stack in order, and at every layer ask: *what would I expect to see if the bug were here, and what cheap test distinguishes that from the alternatives?*

The whole Part as one picture

Think of it like an electrician called because the bedroom light won't turn on. A good one doesn't start by rewiring the lamp (the model). She checks in order: is there power to the building (data), is the main fuse fine (labels), is the switchboard wired right (features, split, pipeline) — and only then the bulb. At each stage a cheap test, like a tester screwdriver, rules that stage in or out in seconds. The debugging stack below is that same walk from the mains to the bulb.

Where it breaks: a house circuit is fixed and one-directional, but ML bugs can hide in several layers at once and mask each other — a leaky split can make a broken model look great — so you sometimes revisit layers you already cleared.

```
          THE DEBUGGING STACK  (check top to bottom; cheap tests first)

  DATA          is the raw data what I think it is? volume, ranges, nulls, dupes, dates
    |
  LABELS        are labels correct, consistent, timely, defined as intended?
    |
  FEATURES      computed correctly? available at prediction time? same in train & serve?
    |
  SPLIT         does validation mimic production? leakage? time/group aware?
    |
  PIPELINE      joins, filters, preprocessing fit on train only, shapes, ordering
    |
  MODEL         capacity, architecture, initialization, hyperparameters
    |
  LOSS          right objective? aligned with the metric? weights? numerically safe?
    |
  OPTIMIZATION  learning rate, schedule, convergence, gradients (vanish/explode)
    |
  EVALUATION    right metric? right slices? statistically significant?
    |
  SERVING       same features, same preprocessing, same model version, latency/timeouts
    |
  MONITORING    drift, delayed labels, feedback loops, alerting
```

22.1 How to isolate problems scientifically

OBSERVE → HYPOTHESIZE → ISOLATE → TEST → MEASURE → FIX → VERIFY

1. **Observe precisely.** Which metric, which slice, since when, how big, how noisy? "AUC dropped" is not an observation; "AUC on Android users fell from 0.84 to 0.79 starting March 3, while iOS is flat" is.
2. **Hypothesize** several causes across the stack — don't fall in love with the first.
3. **Isolate** by making the system smaller: a single batch, a single slice, a single day, a single feature. **Bisect** — commits, pipeline stages, data dates, feature sets — to find where good becomes bad.
4. **Test** with controlled comparisons: change one thing at a time; hold everything else fixed (seed, data, code version). Use **known-answer tests**: synthetic data where the correct model is known; overfit 10 examples; train on shuffled labels (should drop to chance — if not, there's leakage).
5. **Measure** with error bars — rerun with different seeds; is the difference bigger than run-to-run noise?
6. **Fix** the root cause, not the symptom.
7. **Verify** the fix on the original failure *and* add a regression test/monitor so it can't silently recur.

Tools every ML engineer should reach for: a strong simple baseline for comparison; slice-level metrics; per-feature distribution comparisons (train vs. validation vs. serving); feature importance before/after; prediction-distribution histograms; confusion matrices per slice; learning curves; gradient-norm and activation logs; **reading 50 raw examples** of errors (the most underrated technique in ML).



Real-life analogy: a doctor working up a fever

Think of it like a doctor seeing a patient with a fever in the monsoon. She observes precisely (102°F, evenings only, since Tuesday), lists several suspects instead of jumping at the first (dengue, typhoid, a viral fever), orders the cheap test that tells them apart (a blood count), changes one medicine at a time, and checks again. She treats the cause, not just the temperature, and books a follow-up so a relapse doesn't go unnoticed.

Where it breaks: a doctor can't rerun the same patient with a different random seed, but you can replay the exact data and code — so controlled experiments and bisection are far cheaper in ML; use them.

22.2 Scenario 1 — Training loss decreases, validation loss increases

Diagnosis: the generalization gap is growing (overfitting) — or the validation set is different

Layer	Hypothesis	Cheap test	Fix
Model	too much capacity for the data	learning curve vs. train size; smaller model	reduce capacity, regularize (weight decay, dropout, depth limits), early stopping
Data	too little data / too few effective samples (many rows per user)	effective sample size; group-level counts	more data, augmentation, group-aware training

Layer	Hypothesis	Cheap test	Fix
Split	validation distribution differs from training (time, segments)	adversarial validation (train-vs-val classifier AUC)	fix the split, or accept that it measures shift
Features	a feature that is predictive only within training (IDs, leaked-in-train-only signal)	importance of ID-like/ high-cardinality features; drop and retrain	remove/smooth; target encoding out-of-fold
Labels	training labels noisy; model memorizes noise late in training	loss per example; high-loss examples inspected	clean labels, robust losses, early stopping
Evaluation	validation loss rises but validation *metric* still improves (overconfidence — log loss punishes confident errors)	track both	fine if the metric is what matters; calibrate afterwards
Optimization	LR too low late in training lets it fit noise; no decay	schedule experiments	early stopping at best validation checkpoint

Say: "First I check which quantity diverges — loss or the metric — because rising validation loss with a stable metric is overconfidence, not necessarily worse decisions. Then I treat it as variance: regularize and early-stop — but I'd verify the validation set actually resembles training, since a distribution mismatch produces the same curve."



Real-life analogy: a student who memorises past papers

Think of it like a student who practises the same old question papers until she knows them by heart. Her practice scores keep rising while her fresh mock-test scores fall — she is memorising, not learning (overfitting). But check two other stories first. Maybe the mock covers a different syllabus (the validation set differs from training). Or imagine a test where a wrong answer costs more the surer you claimed to be: she can get the same number right yet score worse, because her wrong answers became overconfident (rising log loss with a steady metric).

Where it breaks: you can read a student's answers to spot memorising; with a model you mostly see curves, and label noise, a shifted split and plain overfitting can all draw the same shape.

22.3 Scenario 2 — Both training and validation accuracy are poor

Diagnosis: underfitting, a bug, or no signal

Layer	Hypothesis	Cheap test	Fix
Pipeline	labels misaligned with features (join bug, shuffled one but not the other)	inspect 20 rows by hand; a feature that *should* be predictive has ~zero correlation with label	fix the join/ordering
Labels	label definition wrong or extremely noisy	inter-annotator agreement; conflicting labels on duplicates	redefine/clean labels
Data	no signal in these features (Bayes-limited)	can a much bigger model / GBM overfit? human performance?	new information: features, data sources
Model	too little capacity / wrong inductive bias	overfit a tiny subset; try a more flexible model	bigger model, interactions, nonlinear features

Layer	Hypothesis	Cheap test	Fix
Features	wrong preprocessing (all zeros, unscaled, categorical treated as numeric)	summary stats of the model input tensor	fix encoding/scaling
Loss	wrong loss (MSE for classification, class indices vs. one-hot mismatch)	check loss on a perfect prediction equals ~0	correct loss
Optimization	LR too low/high, dead ReLUs, vanishing gradients, too few steps	LR sweep; gradient norms per layer	tune LR, init, normalization

The sanity ladder: (1) Can the model overfit 10–100 examples to ~zero loss? If not → bug in model/loss/optimizer. (2) Does it beat a trivial baseline (majority class, mean)? If not → data/label/pipeline. (3) Does a totally different model family do better? If yes → inductive bias/capacity.



Real-life analogy: a car stuck at 20 km/h

Think of it like a mechanic facing a car that won't go faster than 20 km/h on any road. The engine might be too small (low capacity), the fuel line might run from the wrong tank (labels misaligned with features), the handbrake might be on (a bad learning rate), or there may simply be no road ahead (no signal in the data). His ladder: can the engine rev freely in neutral (overfit a tiny batch)? Does the car at least beat walking (a trivial baseline)? Does a different car do better on the same road (another model family)?

Where it breaks: any working car moves on a good road, but some prediction problems genuinely have no signal in the available features — no mechanic fixes that; only new information does.

22.4 Scenario 3 — Offline AUC improves but production metrics fall

Diagnosis: the offline evaluation doesn't represent production — or the metric doesn't represent value

Layer	Hypothesis	Cheap test	Fix
Evaluation	offline metric misaligned with the business metric (AUC ↑ but top-of-ranking precision ↓, or calibration broke and downstream bidding/thresholds misfire)	compare operating-point metrics and calibration, not just AUC	optimize/select on metrics tied to the decision; recalibrate
Split	offline test leaks (future info, duplicates), so the "improvement" is leakage	time-based + grouped re-evaluation	fix split; re-evaluate
Serving	**training–serving skew**: features computed differently online (different code path, stale values, missing defaults, timezone)	log served features, compare distributions to training features for the same entities	single feature-computation path (feature store), parity tests
Features	new model relies on a feature that's delayed/unavailable online	feature availability audit	drop or backfill properly

Layer	Hypothesis	Cheap test	Fix
Data	offline data comes from the old model's exposure (selection/feedback bias); new model is evaluated on items the old one chose	counterfactual / off-policy evaluation; exploration data	randomized exploration traffic; IPS-weighted evaluation
Monitoring / experiment	A/B test issues: novelty effect, SRM, wrong randomization unit, interference	A/A test, SRM check	fix experiment design
System	latency increase → timeouts → fallback model serving	p99 latency; fallback rate	optimize or distill the model

Say: "An AUC gain is an improvement in pairwise ranking over the whole test distribution. Production cares about decisions at a threshold or the top-k, on today's traffic, with features computed online. So I check four gaps: metric alignment, offline-test representativeness, training–serving skew, and experiment validity — skew is the most common culprit, so I'd start by logging the served features."



Real-life analogy: a TV cooking champion's restaurant

Think of it like a chef who wins a TV cooking contest, then watches his restaurant's sales fall. Perhaps the judges scored plating while customers care about taste and price (the metric doesn't match business value). Perhaps the contest kitchen stocked ingredients his restaurant can't get (features unavailable or computed differently at serving). Perhaps he had practised those exact dishes (a leaky test). Or perhaps the new dishes take so long that waiters quietly serve the old menu (latency and fallbacks).

Where it breaks: the chef can walk into his restaurant and taste the food; training–serving skew is invisible unless you log the features actually served and compare them with training.

22.5 Scenario 4 — The model suddenly predicts mostly one class

Diagnosis: an input changed, or the model collapsed

Layer	Hypothesis	Cheap test	Fix
Data / serving	an upstream feature is now null/zero/ constant (schema change, broken join, renamed field) → everything lands in one leaf / region	per-feature null rate and distribution for the last few days vs. baseline	fix pipeline; add data-validation alerts
Features	unit/encoding change (cents vs. dollars, new category values mapped to "unknown")	compare feature histograms; unseen-category rate	contract tests, schema validation
Model version	wrong model artifact deployed, or preprocessing (scaler) out of sync with the model	model hash/version in logs; score a golden test set	versioned model+preprocessing bundle
Training (if newly retrained)	label pipeline broke (all labels one class), class weights wrong, collapsed training	label distribution of the last training run	training-data validation gates before deploy
Threshold / calibration	base rate shifted, threshold stale; or calibration layer broken	score distribution vs. threshold	recalibrate; dynamic thresholds
Real drift	the world changed (e.g. a holiday, outage, fraud attack)	external events; other models affected?	retrain/adapt; this can be legitimate

Say: "Sudden changes are usually pipeline changes, not concept drift — drift is gradual. So I look at the prediction distribution's change point and correlate it with deploys, schema changes and upstream data incidents, and I compare feature distributions before and after."



Real-life analogy: a shop scale gone wrong

Think of it like a kirana shop's digital scale that suddenly weighs every bag of rice as "under 1 kg". The rice didn't change overnight. A cable came loose (a feature went null), the units got mixed up between kilograms and grams (a unit or encoding change), or someone swapped in the wrong scale (the wrong model version). The first question is *when* it started and what changed at that moment — a deploy, a schema change, an upstream outage.

Where it breaks: a scale has one input you can see; a model has hundreds, and one silently broken feature can shift every prediction, so you need automatic per-feature null-rate and distribution checks to spot it.

22.6 Scenario 5 — Training becomes NaN

Diagnosis: numerical blow-up — find where the first non-finite value appears

Layer	Hypothesis	Cheap test	Fix
Data	NaN/inf in inputs or labels; extreme values; division by zero in feature code	assert `isfinite` on every batch; find the batch index	clean data, clip, impute
Loss	`log(0)`, `sqrt(0)` gradient, division by zero, softmax without max-subtraction	compute loss from logits; add eps	stable formulations
Optimization	LR too high → divergence; exploding gradients	gradient norm rising before NaN; smaller LR reproduces?	lower LR, warmup, gradient clipping
Model	unnormalized deep net; bad init; attention logits overflow	activation statistics per layer	normalization, init, residual scaling
Precision	fp16 overflow/underflow	same run in fp32 / bf16	loss scaling, bf16
Pipeline	custom op / layer with bad gradient	anomaly detection mode, gradient check on the op	fix the op



Real-life analogy: a microphone squeal at a sangeet

Think of it like the squeal from a microphone at a wedding sangeet. A small sound goes into the mic, the speaker amplifies it, the mic picks it up louder, and within seconds it screeches — that's exploding gradients. The fixes match a sound engineer's: turn the volume down (a lower learning rate), fit a limiter that caps loudness (gradient clipping), start quiet and raise the volume slowly (warmup), and check nobody plugged in a broken mic (NaN or extreme values in the data).

Where it breaks: a squeal is loud and obvious, but a NaN can start in one number of one batch and silently poison every weight at the next step — so hunt for the *first* non-finite value, not the last.

22.7 Scenario 6 — Good overall, bad for one user segment

Diagnosis: the average hides a slice

Layer	Hypothesis	Cheap test	Fix
Data	segment underrepresented in training (few examples → high variance)	segment counts in train	collect more, upweight, stratified sampling
Features	features missing or less informative for this segment (new users have no history; a region lacks a data source)	feature null rates by segment	segment-appropriate features (content/demographic for cold start), missing indicators
Labels	labels noisier or defined differently for the segment (different language, annotators)	label audit by segment	fix labeling guidelines, per-segment QA
Model	model capacity devoted to majority; segment needs different interactions	segment-specific model or segment features/ interactions; compare	segment feature, mixture-of-experts, separate model if large enough
Evaluation	segment metric noisy (small n) — is the gap real?	CI via bootstrap	significance before action
Threshold	one threshold for all segments with different base rates / costs	per-segment calibration curve	per-segment calibration or thresholds (check fairness/ legal constraints)
Serving	feature pipeline broken for that segment only (app version, locale, timezone)	served-feature distributions by segment	fix pipeline

Say: "I'd first confirm it's real with a confidence interval, then decompose: is the segment different in *data volume*, *feature availability*, *label quality*, or *base rate*? Each implies a different fix — more data, new features, label cleanup or per-segment calibration — rather than just a bigger model."



Real-life analogy: a map app failing villages

Think of it like a navigation app that is excellent in cities but poor in villages. Before blaming the routing engine, ask why: far fewer trips are recorded there (less data), small lanes are missing from the map (missing features), user reports are sparser and less reliable (noisier labels), and the same "expected delay" rule is used everywhere (one threshold for all). Each cause needs a different fix. And first check the gap is real — five bad trips are not a pattern.

Where it breaks: villages are easy to spot on a map, but failing ML segments are often not pre-defined (say, new users on an old Android version at night), so you have to search for them with slice analysis.

22.8 Scenario 7 — Performance degraded after deployment

Diagnosis: something differs between the world the model was validated on and the world it runs in

Layer	Hypothesis	Cheap test	Fix
Serving	training–serving skew from day one (implementation differences)	did it degrade immediately (skew) or gradually (drift)? replay logged requests through the offline pipeline	unify feature code; parity tests
Data drift	covariate shift in inputs	PSI/KS per feature; domain classifier	retrain on recent data; reweighting
Concept drift	$P(y \mid x)$ changed (adversaries, market change)	error on fresh labels rises while inputs are stable	retrain frequency, online learning, new features
Label shift	base rate changed	predicted vs. observed rate	prior correction, recalibration
Feedback loop	the model changes the data it later trains on (recommendations, fraud blocking hides outcomes)	exposure/label coverage analysis	exploration, holdout groups, counterfactual labeling
Monitoring	the metric itself changed (logging change, label delay makes recent numbers look worse)	label maturity curves	compare cohorts with equal label maturity

Say: "Timing is the first clue: immediate degradation points to skew or a deployment/version issue; gradual degradation points to drift; periodic to seasonality; step changes to upstream data changes."



Real-life analogy: a detective's timeline

Think of it like a detective whose first question is "*when* exactly did it start?", because timing narrows the suspects. If the victim fell ill the moment he ate at the party, suspect something served that night (a deployment bug or training–serving skew). If he weakened slowly over months, suspect a gradual change in his surroundings (drift). If he falls ill every winter, it's the season. If it began the day a new cook was hired, look at that change (an upstream data change).

Where it breaks: in a detective story one culprit is usually guilty; in production causes overlap — a skew bug from day one plus slow drift — so fixing one may simply reveal the other.

22.9 Two more classics

"Validation performance is too good to be true." Leakage until proven otherwise: dominant single feature, random split on temporal/grouped data, duplicates across splits, preprocessing fit on all data, target encoding in-sample. Test: shuffle labels (score should collapse to chance), drop the top feature, re-split by time and group.

"Results change a lot between runs." Measure the seed variance first (3–5 seeds); small datasets and small validation sets make metrics noisy; non-deterministic GPU ops, data-loader order, and parallel tree building add variance. Decisions need differences larger than the run-to-run spread.

Real-life analogy: a too-good deal, a jumpy scale

Think of it like two everyday warnings. A shopping app offering a new phone for Rs 5,000 is a scam until proven otherwise; check the seller before celebrating. Validation scores that look too good deserve the same suspicion: shuffle the labels, drop the top feature, re-split by time and group, and see whether the magic survives. And if your bathroom scale shows 60, 62 and 59 kg in one morning, you weigh yourself several times before believing you lost a kilo — measure the run-to-run spread before trusting a small improvement.

Where it breaks: a scam is someone's deliberate trick, whereas leakage is almost always an accident, so rely on the tests rather than on suspecting a person.

22.10 Scenario 8 — An LLM feature got worse after a "minor" change

Symptom. Thumbs-down rate on an LLM assistant rose from 8% to 13% over a week. The team changed nothing in the model code.

Hypotheses, in order of likelihood. (1) The **provider or model version changed** (an API alias now points to a newer model; a silent default change). (2) A **prompt or template** edit (a new instruction, a reordered system prompt, a changed few-shot example). (3) **Retrieval** changed — re-indexing, a new chunker, or a document source that stopped syncing (stale answers). (4) **Traffic mix** shifted (a new user segment, a marketing campaign, a new language). (5) Output-length or sampling-parameter changes (temperature, max tokens truncating answers). (6) A guardrail became stricter → more refusals.

Diagnose. Pull traces for thumbs-down conversations and **diff configurations** (model version, prompt hash, retrieval index version, parameters) between before and after. Re-run the offline eval suite with the old and new configuration on the same golden set (paired comparison, Part 45). Slice by intent and language. Check refusal rate and answer length distributions.

Fix and prevent. Pin model versions; version prompts and retrieval configs like code; gate every change on the eval suite in CI; monitor refusal rate, length and judge scores by slice with alerts; keep a rollback path.

Real-life analogy: the restaurant that "changed nothing"

Think of it like a restaurant whose reviews suddenly drop though the chef insists nothing changed. Then you find the supplier quietly switched to a different brand of oil (model version), a new waiter reads the menu differently (prompt), the fridge stopped being restocked (stale retrieval), and a tour group with different tastes started coming (traffic mix). Nothing changed *in the kitchen code* — everything around it did.

Where it breaks: a restaurant can taste yesterday's dish; with LLMs you can only compare if you logged the exact configuration and kept a fixed eval set to replay.

22.11 Scenario 9 — Fine-tuning an LLM: loss spikes, forgetting and "it learned the format, not the task"

Symptoms and causes.

Symptom	Likely cause	Fix
Loss spikes or NaN mid-run	learning rate too high, no warmup, bad batch (very long or corrupted sequences), fp16 overflow	warmup + lower LR, gradient clipping, bf16, filter or truncate outliers, skip bad batches
Training loss near zero, eval poor	memorizing a small or duplicated dataset; train/test leakage	deduplicate, hold out by source, fewer epochs, LoRA with a smaller rank
Great on the task, worse at everything else	**catastrophic forgetting**	mix in general instruction data, LoRA instead of full fine-tuning, lower LR, fewer steps
Perfect output format, wrong content	the model learned the template; labels inconsistent or too few	improve label quality and diversity; evaluate content, not format
Loss computed on prompts	prompt tokens not masked (Part 43)	mask the prompt; train only on response tokens

Debug order. Overfit 10 examples first (does the pipeline work?); inspect tokenized inputs and masks by decoding them back to text; compare against a strong prompting baseline — if fine-tuning can't beat few-shot prompting on your eval, question whether you need it.



Real-life analogy: coaching a senior employee on a new process

Think of it like training an experienced employee on a new form-filling process. Train too intensely (high learning rate) and they panic and make errors (loss spikes). Drill only the new form for weeks and they forget their other skills (forgetting). Use a sloppy set of example forms and they learn the layout but not the rules (format without content). A little training, with good examples and mixed with their usual work, works best.

Where it breaks: employees tell you when they're confused; a model's confusion only shows up in the evaluation you run — so evaluate general skills as well as the new task.

22.12 Scenario 10 — The LLM bill tripled and latency doubled

Diagnose with a breakdown. Tokens per request (input vs. output), requests per user, cache hit rate, model mix (share of traffic hitting the largest model), retries and agent loop lengths, and p95 latency by stage (retrieval, time to first token, generation).

Typical causes. Retrieved context grew (a top-k change from 5 to 20 chunks), conversation history appended without truncation, an agent looping (retrying failing tools), a prompt change that disabled prefix caching (dynamic content moved to the start of the prompt), routing sending easy queries to the large model, longer outputs after a model upgrade.

Fixes. Cap and summarize history; rerank and trim context; put static instructions first so prefix caching works; add loop limits and tool-error handling; route by difficulty; set max output tokens; alert on tokens per request (Part 46).

Real-life analogy: a phone bill that suddenly tripled

Think of it like a family phone bill that tripled. You don't just complain — you open the itemized bill: longer calls (more output tokens), more calls (more requests), calls to expensive international numbers (the biggest model), a phone stuck redialling a busy number (an agent loop), and a lost family-plan discount (cache no longer applied). The itemized breakdown makes the fix obvious.

Where it breaks: a phone bill is itemized for you; LLM costs are itemized only if you log tokens, model and cache hits per request from day one.

PART 23

Production ML

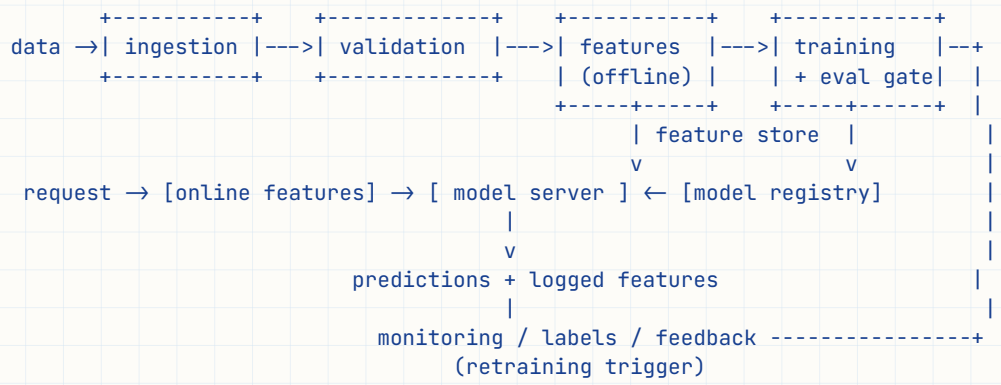
The production mindset

In production, the model is maybe 5% of the code. The rest is data collection, validation, feature computation, training orchestration, serving, monitoring and the feedback loop. **Most production incidents are consistency failures**: the model sees different data, features or code at serving time than it did in training.

The whole Part as one picture

Think of it like an airline. The pilot (the model) is a small part of what makes a flight work; the rest is fuel supply, maintenance, air-traffic control, checklists and incident reports — the data pipelines, serving and monitoring around the model. And many serious incidents come from mismatch, not bad flying: a pilot trained in a simulator whose switches sit in different places from the real cockpit. That is training–serving skew, the classic production failure: the model sees different data, features or code at serving time than it saw in training.

Where it breaks: a pilot can see a cockpit mismatch, but a model cannot see that a feature is computed differently in serving — it just gives confident, wrong answers, so the checks must be automated.



23.1 Pipelines

Training pipeline: extract data as of a snapshot → validate (schema, ranges, nulls, volume, label distribution) → build features → split → train → evaluate against a baseline and the current production model on fixed and recent test sets, plus slices → register the model with its metadata → (optionally) deploy. Orchestrated (Airflow, Kubeflow, etc.), idempotent, versioned, re-runnable.

Feature pipeline: batch features (daily aggregates computed in the warehouse), streaming features (last-5-minutes counts from an event stream), and on-request features (from the request payload). The same definitions must produce the same values offline (for training) and online (for serving).

Real-life analogy: a dairy's milk plant

Think of it like a dairy cooperative's milk plant. Each morning it collects milk from villages (extract a snapshot), tests every can and rejects spoiled ones (validation), pasteurises and packs (feature building), and compares samples with yesterday's quality before shipping (evaluate against the current model). Rerunning Tuesday's batch gives exactly Tuesday's packets, not a double batch (idempotent, re-runnable). The test kitchen and the packing line must follow the same recipe (the same feature definitions offline and online).

Where it breaks: milk is used once and gone, but data pipelines must also rerun the *past* with exactly that day's inputs (backfills), which needs versioned data and code.

23.2 Feature stores

A system that stores feature definitions and values with two faces: an **offline store** (historical values, large, for training — supports **point-in-time joins**: "the value of this feature as it was at the event timestamp") and an **online store** (latest values, low-latency key–value lookups for serving). **Why it exists:** one definition, computed once, used consistently in training and serving (kills a whole class of training–serving skew), with point-in-time correctness (kills a class of leakage), reuse across teams, and monitoring of feature freshness.

Real-life analogy: a newspaper archive and front desk

Think of it like a newspaper office. The **archive** (offline store) keeps every past edition, so you can read exactly what was known on a given date — that is a point-in-time join. The **front desk** (online store) hands you today's paper instantly. Because both come off the same printing press (one feature definition), the archive copy and the desk copy always agree, and every department reads the same paper instead of printing its own.

Where it breaks: a newspaper is dated automatically, but a feature store gives point-in-time correctness only if features are logged with the right timestamps — including the real delay before data arrives.

23.3 Batch vs. online inference

	Batch (offline) inference	Online (real-time) inference
How	score all entities on a schedule, store results	score on request, synchronously
Latency budget	hours	milliseconds (p99 matters)
Features	anything computable in the warehouse	must be available fast — precomputed or from the request
Cost	cheap, efficient hardware use	always-on serving fleet
Freshness	stale by up to one schedule period	reflects the latest context
Examples	churn scores for tomorrow's campaign, nightly recommendations, demand forecasts	fraud at authorization, search ranking, ads CTR, ETA
Failure mode	stale predictions for new entities (cold start)	timeouts, tail latency, dependency failures

Hybrids are common: precompute embeddings or candidate lists in batch; do lightweight re-ranking online with real-time features.

Real-life analogy: tiffins versus street food

Think of it like two ways of feeding people. A **tiffin service** cooks 500 lunches at dawn and delivers at noon — cheap and efficient, but it can't adapt if you suddenly want no onions (stale predictions), and a customer who signed up at 11 gets nothing today (cold start). A **street-food stall** cooks each order in front of you — fresh and personal, but it needs a cook ready all day, and a long queue means slow service (tail latency). The **hybrid**: chutneys and dough made in the morning, the final tadka to order.

Where it breaks: food goes stale physically; batch predictions go stale only when the world or the user changes, so for slow-moving things like monthly churn scores, batch is fresh enough.

23.4 Model serving

Concerns: latency (p50/p99), throughput, cost per prediction, hardware (CPU vs. GPU; batching requests on GPUs trades latency for throughput), model formats and runtimes (ONNX, TorchScript, TensorRT, treelite for GBMs), model compression (quantization, distillation, pruning), caching of frequent predictions, **fallbacks** (a simpler model or heuristic if the model or a feature source times out), autoscaling, and versioned endpoints. The model artifact must be deployed **together with its preprocessing** (tokenizer, scaler, encoders, feature list) — a model with the wrong preprocessing silently produces garbage.

Real-life analogy: a busy pharmacy counter

Think of it like a busy pharmacy counter. Prescriptions must be filled fast (latency) for a long queue (throughput). Filling several identical prescriptions together is efficient but makes the first customer wait (batching requests on GPUs). Popular medicines sit at the front (caching). If a medicine is out of stock, an approved substitute goes out (a fallback model). And every medicine must come with its own dosage chart — the right pill with the wrong chart is dangerous, just like a model shipped with the wrong preprocessing.

Where it breaks: a pharmacist may notice a mismatched chart, but a model with the wrong preprocessing silently returns plausible-looking garbage, so the only guard is versioning the two together.

23.5 Model registry, data versioning, experiment tracking, reproducibility

- **Model registry:** every model version with its training-data snapshot, code commit, hyperparameters, metrics, lineage, approval status and stage (staging/production/archived). Enables rollback and audit.
- **Data versioning:** immutable snapshots or versioned tables (and the query that produced them) so any training run can be reproduced and any regression bisected.
- **Experiment tracking:** parameters, metrics, artifacts, environment for every run — so "which run produced this model?" has an answer.
- **Reproducibility checklist:** pinned dependencies/container, fixed seeds (and awareness of non-deterministic GPU kernels), versioned data + code + config, recorded hardware; the goal is *explainable* differences, not bit-identical results.

Real-life analogy: land records and a lab notebook

Think of it like a land-records office plus a scientist's lab notebook. Every plot is registered with its full history — owners, dates, the survey map — so ownership can be proved and a bad sale undone. A model registry does the same for models: data snapshot, code commit, settings, metrics and approval stage, so you can audit and roll back. Experiment tracking is the lab notebook: every trial written down, failures included, so "which run made this model?" always has an answer.

Where it breaks: land doesn't change when you reread its deed, but a model is *rebuilt* from data, code and configuration, so all three must be versioned or the record can't recreate it.

23.6 Monitoring

What	Examples	Why
System	latency, error rate, throughput, memory, timeouts, fallback rate	availability
Data quality	schema violations, null rates, out-of-range values, volume, freshness of features	catches pipeline breaks within minutes
Input drift	PSI / KS / Jensen–Shannon per feature; domain classifier	early warning before labels arrive
Prediction drift	distribution of scores, predicted-positive rate, top-class share	cheap, label-free proxy for "something changed"
Performance	AUC, precision, calibration on labeled traffic, by slice	the ground truth — often **delayed** (chargebacks: weeks; churn: months)
Business	conversion, revenue, complaints, manual-review outcomes	what actually matters

Population Stability Index: $\mathrm{PSI} = \sum_b (p_b - q_b) \ln \frac{p_b}{q_b}$ over bins (reference q, current p). Rules of thumb: < 0.1 stable, 0.1–0.25 moderate shift, > 0.25 significant. Drift alarms need tuning: with huge traffic, statistical tests flag trivial changes — alert on **effect size** and on features that matter to the model.

Real-life analogy: a car's dashboard

Think of it like the dashboard of a car on a long drive. The engine-warning light says whether the car is running properly at all (system health). The fuel gauge, plus a check that it's the right fuel, covers data quality. A temperature needle creeping up warns early (input drift). A strange new vibration is a cheap sign that something changed (prediction drift). The mechanic's inspection report comes much later but is the real verdict (performance with delayed labels). And the true outcome is whether you arrive on time (business metrics).

Formula, read aloud: $\mathrm{PSI} = \sum_b (p_b - q_b) \ln \frac{p_b}{q_b}$ says: for each bin, (how much its share moved) × (how big that move is as a ratio), added up over all bins.

Where it breaks: a car has a handful of gauges, but a model has hundreds of features, and with huge traffic drift tests flag tiny harmless changes — so alert on effect size and on the features the model relies on.

23.7 Drift taxonomy

Term	What changes	Detect with	Typical response
Data drift (umbrella)	any change in the input data distribution $P(x)$	feature distribution monitoring	investigate; retrain if performance suffers
Covariate shift	$P(x)$ changes, $P(y \mid x)$ stable	input drift metrics; domain classifier	retrain on recent data / importance weighting; model may still be right where it has support
Label (prior) shift	$P(y)$ changes, $P(x \mid y)$ stable	observed label rate; predicted vs. actual positive rate	prior correction, recalibration, threshold update
Concept drift	$P(y \mid x)$ changes — same inputs now mean different outcomes	performance on fresh labels degrades even without input drift	retrain on recent labels; shorter retraining cycles; new features; online learning
Prediction drift	distribution of model outputs $P(\hat{y})$ changes	score histograms, positive-rate monitoring	symptom — trace to input drift, pipeline bug, or real change

Training–serving skew is different from drift: the training data and serving data *should* be the same distribution but aren't because of implementation differences (different code, stale features, different defaults) — it's a bug, visible from day one.



Real-life analogy: a woollens shop in Delhi

Think of it like a woollens shop in Delhi. **Covariate shift:** more tourists walk in — different customers, but each kind buys as it always did. **Label shift:** winter arrives and far more sweaters sell overall, while the kind of person who buys one looks the same. **Concept drift:** the same regulars who always bought sweaters now buy jackets — same person, different outcome. **Prediction drift:** the shop's own sales forecast moves — a symptom to explain, not a cause. **Training–serving skew** is different: the stock ledger counts in dozens but the order app counts single pieces — a bug, wrong from day one.

Where it breaks: the shopkeeper sees what customers buy the same day, but ML labels may arrive weeks later, so concept drift is spotted late — and real shifts usually come mixed together.

23.8 Retraining

Strategies: scheduled (daily/weekly — simple, predictable), triggered (by drift or performance alerts), continuous/online (streaming updates — for fast-changing domains like ads and news; needs strong safeguards). **Choose the cadence** from how fast performance decays (measure: train on month t, evaluate on months $t+1, t+2, \ldots$), label latency, and cost. **Guardrails:** automated validation gates (new model must beat the current one on fixed and recent sets, with no slice regressing beyond a tolerance), data validation before training, and a staged rollout. Beware **feedback loops** — retraining on data shaped by the current model can reinforce its biases (keep a small random/exploration holdout).

Real-life analogy: updating a city bus timetable

Think of it like updating a city bus timetable. You can redo it every term (scheduled), when complaints spike (triggered), or nudge it continuously from live GPS (online). How often depends on how fast the city changes — so measure how wrong last year's timetable gets, month by month. Before publishing a new one, check it isn't worse on any route (validation gates) and try it on a few routes first (staged rollout). Beware: if buses only run where the old timetable sent them, you only learn about riders on those routes (a feedback loop).

Where it breaks: bus complaints arrive the same day, but model labels can take weeks or months, and that delay limits how fast retraining can react.

23.9 Deployment strategies

Strategy	How	Protects against	Cost
Shadow	new model receives a copy of live traffic; predictions logged, **not used**	crashes, latency, skew, prediction-distribution surprises — with zero user impact	double compute; can't measure user-behavior outcomes
Canary	send a small % (1–5%) of real traffic to the new model; watch health and key metrics; expand gradually	bugs and regressions at limited blast radius	slower rollout
A/B test	randomized split; compare business metrics with statistics	measures causal impact on outcomes	needs traffic and time; experiment design pitfalls (Part 4)
Interleaving (ranking)	mix results from two rankers in one list; see which gets clicks	very sensitive comparison for search/recs with less traffic	only for ranking; doesn't measure all metrics
Blue–green	two full environments; switch traffic at once	fast switch and fast rollback	double infrastructure during switch
Rollback	revert to the previous registered model (and its preprocessing) automatically on alerts	anything	requires versioned artifacts and backward-compatible features

Real-life analogy: trying out a new head cook

Think of it like a restaurant trying out a new head cook. **Shadow:** he cooks every order alongside the old cook, but only the old cook's plates go out, so you compare with zero risk to diners. **Canary:** he cooks for 2 tables out of 50 while you watch closely. **A/B test:** tables are assigned at random and you compare repeat visits and reviews. **Interleaving:** dishes from both cooks share one thali, and you see which get eaten first. **Blue–green:** a second, fully staffed kitchen, with all orders switched at once. **Rollback:** the old cook waits on standby with his exact recipe.

Where it breaks: diners notice a bad dish at once, but model harm can be slow and delayed (fraud labels take weeks), so a canary can look fine for days before trouble shows.

Q: "How would you safely deploy a new fraud model?"

Strong answer. "Offline, it must beat the current model on recent, time-split data at the operating point that matters — dollar recall at our FPR budget — with no major segment regressing, and its calibration checked because thresholds depend on it. Then shadow mode for a week or two: it scores live traffic without acting, so I can compare its alerts with production's, check latency, and catch training–serving

skew. Then a canary on a few percent of traffic with automatic rollback on alert-rate or latency anomalies, then an A/B or ramp while watching fraud losses, false-positive complaints and review-queue load — keeping in mind fraud labels mature over weeks. I'd keep a small holdout scored by the old model or a random-review sample to keep unbiased labels for future training."

23.10 LLMOps: what production changes for LLM systems

The MLOps loop (Part 23) still applies, with new artifacts and new failure modes.

MLOps concept	LLM-system version
model registry	registry of **model versions + prompts + retrieval configs + tool schemas** (all versioned, reviewed, and tied to eval results)
offline evaluation	golden sets, regression sets from past failures, calibrated LLM judges, safety sets (Part 45) run in CI on every change
feature store	**retrieval index** with freshness SLAs, permission metadata, and re-embedding jobs when the embedding model changes
monitoring	tokens and cost per request, latency by stage, refusal rate, judge scores on sampled traffic, retrieval hit rates, user feedback, safety flags
data drift	**query drift** (new intents, languages), **corpus drift** (documents change), **provider drift** (the external model changes)
rollback	pinned model versions and prompt versions with instant rollback; feature flags per change
logging	full traces (prompt, retrieved context, tool calls, output) with PII redaction and retention limits

Operational must-haves. Timeouts and retries with backoff for provider APIs; fallbacks (a second provider or a smaller self-hosted model) for outages and rate limits; per-tenant quotas; caching (prefix and semantic, Part 46); **re-embedding** strategy — changing the embedding model means re-indexing everything, so plan it as a migration with dual indexes and an A/B test; and a vendor-deprecation plan (models get retired — keep your eval suite ready to qualify a replacement).



Real-life analogy: running a restaurant with a hired celebrity chef

Think of it like running a restaurant whose main chef is on contract from an agency. You keep written recipes (versioned prompts), a pantry list with expiry dates (retrieval index freshness), a daily tasting of sample dishes (evaluation on sampled traffic), itemized supplier bills (token costs), and a backup cook in case the agency sends someone else or the chef is unavailable (fallbacks and version pinning).

Where it breaks: a restaurant notices a new chef on day one; a provider can change the model behind the same name without notice — only continuous evaluation reveals it.

Active recall — Set 14: Debugging & production

Set 14 · Answer from memory, then check the Answer Key

Easy recall

1. List the layers of the debugging stack in order.
2. Define covariate shift, label shift, concept drift and prediction drift.

3. What are the offline and online stores in a feature store?
4. What's the difference between shadow and canary deployment?
5. What does PSI measure and what are its rule-of-thumb thresholds?

Conceptual

6. What is training–serving skew and how does it differ from drift?
7. Why is "overfit a tiny batch" a powerful debugging test?
8. What goes into a model registry entry?
9. When would you choose batch over online inference?
10. How do you pick a retraining cadence?

Why

11. Why are sudden performance drops usually pipeline problems?
12. Why can offline AUC improve while production metrics fall?
13. Why must preprocessing be versioned with the model?
14. Why do drift tests on huge traffic produce too many alerts?
15. Why can feedback loops make retraining harmful?

Scenario

16. The model predicts almost everything as negative since this morning. Walk through your investigation.
17. Your model's accuracy is fine for iOS but poor for Android. Next steps?
18. You deployed a model; it was worse from the first hour. What do you suspect first?
19. Fraud labels arrive after 60 days. How do you monitor the fraud model in the meantime?
20. Your training loss goes to NaN at a random step each run. Plan?

Mathematical

21. Compute PSI for reference bins [0.5, 0.3, 0.2] and current [0.4, 0.3, 0.3].
22. Your canary gets 2% of 1M daily requests. How many days to detect a precision drop from 0.60 to 0.55 on flagged items if 1% are flagged? (rough)
23. A model trained in January loses 1 AUC point per month. With retraining costing X and each AUC point worth Y per month, sketch the optimal cadence reasoning.

Difficult follow-ups

24. How do you evaluate a new recommendation model offline when your logs only contain items the old model chose to show?
25. Design a monitoring system that distinguishes a data-pipeline bug from genuine concept drift.

2027 additions

26. An LLM assistant's thumbs-down rate rose with no code change. List the first four things you check.
27. Your fine-tuned model is perfect on format but wrong on content, and worse at general tasks. Diagnose.
28. LLM costs tripled. What breakdown do you look at first?

PART 24

ML System Design

What an ML system design interview is really testing

Not whether you know "the" architecture — there isn't one. It tests whether you can **turn a vague business goal into a well-posed ML problem**, make **explicit trade-offs** under real constraints (latency, scale, labels, cost, risk), and anticipate **how it fails**. Drive the conversation, state assumptions out loud, and keep coming back to the objective.

The whole Part as one picture

Think of it like being hired as the event manager for a big wedding. A good manager never starts by booking a band. First they ask the family what really matters (the business goal), how many guests are coming (scale), the budget and the deadline (cost, latency), and what "a great wedding" means to them (metrics). They plan a simple fallback menu (baseline), check every vendor (data and labels), and keep backups for rain and power cuts (failure handling). Throughout, they keep coming back to what the family asked for.

Where it breaks: a wedding happens once and is over; an ML system runs every day, the world around it keeps changing, and it must be watched and retrained for years.

24.1 The reusable framework

#	Step	What to say / decide	Typical traps
1	**Clarify business objective**	who is the user, what outcome matters, what's the product surface, what does success look like in 6 months	jumping to models
2	**Define ML objective**	turn the outcome into something learnable: classification, ranking, regression, retrieval...	optimizing a proxy that diverges from value
3	**Prediction target**	exact label definition, horizon, unit (user, session, item, pair), when it's observed	leaky or ambiguous targets
4	**Success metrics**	offline (model), online (product), guardrails (latency, fairness, cost, complaints)	a single metric; ignoring guardrails
5	**Scale**	users, items, QPS, data volume, latency budget, freshness	designing for the wrong order of magnitude
6	**Data**	sources, logging, volume, retention, privacy	assuming data exists
7	**Labels**	explicit vs. implicit, human vs. logged, delay, noise, bias (exposure, selection)	treating clicks as truth
8	**Features**	user, item, context, cross, real-time vs. batch, point-in-time	features unavailable at serving
9	**Baseline**	heuristic or simple model first: popularity, rules, logistic regression	skipping it (no way to measure lift)
10	**Model**	justify the family from data type, scale, latency, interpretability	"use a transformer" by default

#	Step	What to say / decide	Typical traps
11	**Training architecture**	batch vs. continual, data pipeline, sampling, distributed training	ignoring cost
12	**Validation strategy**	time-based / grouped splits; offline replay	random splits on temporal data
13	**Serving architecture**	batch vs. online; multi-stage funnels; caching; fallbacks	single-stage scoring of millions of items
14	**Latency constraints**	budget per stage, p99, model compression	ignoring tail latency
15	**Scalability**	sharding, ANN indexes, precomputation, horizontal scaling	—
16	**Monitoring**	data quality, drift, predictions, delayed performance, business	only system metrics
17	**Feedback loop**	how outcomes return as labels; bias this creates	self-reinforcing loops
18	**Retraining**	cadence from decay rate and label delay; gates	"retrain daily" without reason
19	**Experimentation**	A/B design, unit, duration, interleaving, holdouts	peeking; wrong unit
20	**Failure handling**	fallbacks, rollbacks, adversaries, cold start, outages	happy-path-only designs

Time allocation in a 45-minute interview: ~5 min clarify and metrics → ~5 min data/labels → ~10 min features + model → ~10 min serving architecture → ~10 min evaluation, monitoring, iteration → ~5 min trade-offs and failure modes (or wherever the interviewer steers). Draw the architecture; say numbers.



Real-life analogy: a good doctor's consultation

Think of it like a careful doctor seeing a new patient. They don't write a prescription in the first minute. They ask what is wrong and what the patient wants to get back to (business objective), decide exactly what to measure (prediction target), order the right tests (data and labels), try the simple remedy before surgery (baseline before a big model), and plan follow-up visits (monitoring and retraining). They also ask, "what if this medicine has side effects?" (failure handling).

Where it breaks: a doctor treats one patient at a time; an ML system "treats" millions of requests a second, so scale, latency and cost are part of the diagnosis from the very start.

The multi-stage funnel (appears in half of all designs)

When you must choose a few items out of millions under tight latency, no single model can score everything. **Candidate generation / retrieval** (cheap, high recall: embeddings + ANN, rules, co-occurrence) → **ranking** (expensive, precise model on hundreds of candidates) → **re-ranking / policy** (diversity, freshness, business rules, dedupe). Each stage has its own metric: retrieval is judged by **recall@K** of what the ranker would want; ranking by NDCG/log loss; the whole by online metrics.

24.2 Design 1 — Spam detection (email)

Clarify. Consumer email provider; decide at delivery time whether a message goes to inbox or spam. Scale: billions of messages/day; decision within tens of ms.

Real-life analogy: a big building's doorman

Think of it like the doorman of a huge apartment building deciding who gets in. Visitors on the banned list are turned away at once (blocklists: cheap and fast). Everyone else is judged in seconds by their ID, the company they claim to be from, and how they behave (sender reputation, content). Residents keep a personal "always let in" list (contacts). Turning away a resident's relative is far worse than letting one salesman slip through, so he refuses only when very sure (low false-positive rate). Salesmen keep changing uniforms, so he must keep learning.

Where it breaks: a doorman sees each visitor once; a spam filter can notice the same "visitor" knocking on a million doors at once, and uses that as a strong clue.

Objective & target. Binary classification per message: spam vs. not. The costs are asymmetric: a legitimate email in spam (FP) can be very costly (missed job offer); some spam in the inbox (FN) is an annoyance. → **Optimize spam recall at a very low false-positive rate on legitimate mail.**

Metrics. Offline: recall at FPR ≤ 0.1%, PR-AUC on recent held-out data, per-segment FPR (languages, senders). Online: user "not spam" rescues (FP proxy), "report spam" rate on inbox mail (FN proxy), engagement with spam-folder mail. Guardrails: latency, deliverability complaints.

Labels. User reports ("spam" / "not spam") — noisy, biased (few users report; people mark unwanted newsletters as spam); honeypot accounts (all mail is spam); human-reviewed samples; rule hits. Weight or de-noise by reporter reliability; per-user "unwanted" vs. global "spam" are different targets.

Features. Sender reputation (domain/IP history, authentication: SPF/DKIM/DMARC, volume spikes), content (text embeddings, URL features, attachments, obfuscation tricks), structural (headers, HTML/text ratio), network/aggregate (how many recipients received near-identical content — campaign detection via hashing), recipient relationship (in contacts? replied before?).

Model & serving. Stage 0: blocklists, authentication failures, known-bad hashes (cheap, catch the bulk). Stage 1: GBM over metadata + reputation features + a compact text model (distilled transformer or linear model on n-grams) — ms latency. Heavier models only for uncertain cases or asynchronously (move to spam later). Personalization layer: per-user adjustments (contacts always inbox).

Monitoring & retraining. Spam is **adversarial** — concept drift is constant; retrain frequently (daily or continuous for reputation features); watch sudden campaign spikes; track FP rescues by sender to catch false-positive incidents on legitimate bulk senders.

Failure modes & trade-offs.

Decision	Option A	Option B	Consider
Content model	n-gram linear (fast, robust)	fine-tuned transformer (better semantics)	latency, adversarial obfuscation, multilingual
Global vs. personal	one global threshold	per-user adjustments	"spam" is partly subjective
Feedback loop	train on user reports	+ random sample review of spam folder	spam-folder mail is rarely seen → FNs in spam never get corrected labels

24.3 Design 2 — Fraud detection (card payments)

Clarify. Card-not-present payments; decide at authorization: approve, challenge (3-D Secure / OTP), or decline. Latency budget ~50–100 ms end to end. Fraud rate ~0.1%.

Real-life analogy: a jewellery shop's sharp-eyed cashier

Think of it like the cashier at a busy jewellery shop. Most customers pay normally and are waved through (approve). If something looks slightly off, like a brand-new customer buying five gold chains in a hurry, the cashier asks for ID (a challenge such as an OTP). Very suspicious payments are refused (decline). The cashier only learns weeks later, when the bank reverses a payment, that a card was stolen (label delay). And for the customers who were refused, nobody ever finds out whether they were honest (selective labels).

Where it breaks: a cashier judges one face in a minute; the model has about 50 ms, but can see patterns across millions of cards, like one phone used by twenty "different" customers.

Objective & target. Minimize fraud losses + friction costs. Target: transaction later confirmed fraudulent (chargeback or confirmed report) within ~90 days. Prediction unit: transaction.

Metrics. Offline: dollar-weighted recall at a fixed decline rate / FPR; PR-AUC; expected cost. Online: fraud losses (basis points of volume), false-decline rate (customer friction; measured via challenges passed and customer contacts), approval rate, review queue load. See Part 16.

Labels. Chargebacks (30–90-day delay), customer reports, manual review outcomes. **Selective labels:** declined transactions never produce an outcome — the model only learns from what it approved. Mitigation: a tiny random approval/review holdout, challenge outcomes as labels, and treating decline decisions carefully in training data.

Features. Velocity (counts/amounts per card, device, IP, email in last minutes/hours/days — streaming features), device fingerprint and emulator signals, geo/IP mismatch with billing, merchant category risk, user historical behavior (typical amount, time, merchants — deviation features), account age, **graph features** (shared devices/cards/addresses linking to known fraud — fraud rings), text/email pattern features.

Model & serving. GBM (strong on tabular, fast) as the main scorer; sequence models over a user's recent transactions or graph neural nets for ring detection as additional features; rules engine for hard constraints and rapid response to new attacks. Serving: online feature store for velocity counters (updated in streaming), model on CPU with ms latency, decision layer with cost-based thresholds by amount and segment.

Validation. Strictly **time-based** (train on months 1–6, validate on 7, test on 8, with a label-maturity gap). Evaluate by cohorts whose labels have matured.

Monitoring & retraining. Score distribution and decline rate in real time (sudden attack or pipeline bug); feature drift; label-matured performance weekly; retrain weekly/biweekly plus rules for zero-day patterns. **Adversarial adaptation:** fraudsters probe thresholds — randomization and multiple signals help.

Decision	Option A	Option B	Consider
Decisions	binary decline	tiers: approve / challenge / review / decline	challenges are cheap friction and generate labels
Threshold	single global	cost-based per amount/segment	calibration required
Model update speed	weekly retrain	+ online rules / streaming features	new attack patterns emerge in hours

24.4 Design 3 — Recommendation system (home feed)

Clarify. E-commerce or video app home feed; 100M users, 10M items; show ~50 items per session; latency < 200 ms. Business goal: long-term engagement/retention and revenue, not just clicks.

Objective & target. Rank items per (user, context). Predict multiple outcomes: click, watch/dwell time or purchase, like/share, negative feedback (hide, skip). Combine into a value score:
$\text{score} = w_1 P(\text{click}) + w_2 \mathbb{E}[\text{watch}] + w_3 P(\text{purchase}) - w_4 P(\text{hide})$, with weights tuned via online experiments against long-term metrics.



Real-life analogy: a waiter at a thali restaurant

Think of it like a waiter who knows his regulars, in a kitchen with 500 dishes. He can't think hard about all 500, so he first grabs about 30 this customer might like, from past orders and today's popular items (candidate generation). Then he thinks carefully about which 10 they'd enjoy most (ranking), and balances the plate so it isn't five kinds of dal (re-ranking for variety). Dishes left untouched teach him as much as dishes finished.

Formula, read aloud: $\text{score} = w_1 P(\text{click}) + \cdots - w_4 P(\text{hide})$ says: a dish's value = weighted (chance they taste it + how much they'll eat + chance they buy it) − (chance they push it away).

Where it breaks: a waiter serves a few tables; the feed does this for 100M users in under 200 ms, and only learns from dishes it actually served.

Metrics. Offline: recall@K for retrieval; AUC/log loss per head and NDCG@K for ranking; coverage, diversity, novelty. Online: engagement per session, conversions, retention (D7/D30), satisfaction surveys; guardrails: latency, creator/seller exposure fairness, content quality.

Data & labels. Impressions with outcomes (clicks, dwell, purchases). Negatives = impressed but not engaged (not random unseen items). Biases: **position bias** (top items get clicked regardless), **exposure/selection bias** (only items the old system showed have labels), **popularity bias**. Mitigations: position as a feature during training (set to a constant at serving) or inverse-propensity weighting; exploration traffic.

Architecture (funnel).

```
10M items --[candidate generation: ~1-2K]--> [ranking: ~500 scored] --> [re-rank: 50 shown]
  two-tower embeddings + ANN (user tower   deep multi-task model with    diversity (MMR),
  vs item tower), co-visitation, recent    user/item/context/cross       freshness, dedupe,
  interests, trending, new-item boost      features, real-time signals   business rules
```

- **Two-tower retrieval:** user and item encoders trained with in-batch negatives (softmax/InfoNCE, with log-popularity correction); item embeddings precomputed and indexed (HNSW / IVF-PQ); user embedding computed per request.
- **Ranker:** deep multi-task network (shared bottom or mixture-of-experts, one head per objective) or GBM for smaller setups; features include user history sequences (attention over recent items), item stats, context (time, device), and user–item cross features.
- **Cold start:** new items — content embeddings (text/image), exploration boost; new users — popularity by context, onboarding preferences, fast session-based adaptation.

Training & freshness. Daily or continuous training on recent logs; real-time features (last few interactions) matter most for session relevance.

Experimentation. A/B tests measured on long-term metrics (holdouts for weeks); interleaving for rapid ranker comparisons.

Decision	Option A	Option B	Consider
Objective	clicks (dense, fast)	watch time / purchases / retention (sparse, slow)	clickbait vs. value; multi-task blends
Retrieval	collaborative (IDs)	content-based embeddings	cold start vs. personalization strength

Decision	Option A	Option B	Consider
Exploration	pure exploitation	ε-greedy / Thompson sampling slots	feedback loops, popularity bias, long-term learning
Ranker	GBM	deep multi-task	data scale, sequence features, serving cost

24.5 Design 4 — Search ranking

Clarify. Product or document search; 10K QPS; results in < 300 ms; corpus 100M documents.



Real-life analogy: a librarian with 100 million books

Think of it like asking a librarian in a giant library for "that book on cricket rules for kids". First she makes sense of your words, fixing spelling and guessing what you mean (query understanding). Then she pulls books two ways: from catalogue cards containing your exact words (BM25), and from her memory of books on the same topic even if the words differ (dense retrieval). She sorts that pile of about a thousand by fit (ranking), and for the top few she actually opens each book next to your question (cross-encoder).

Where it breaks: a librarian helps one person slowly; search does this 10,000 times a second in 300 ms, and learns "good fit" mostly from biased click logs, not from reading.

Objective & target. For each (query, document), predict graded relevance or the probability of a satisfied click/purchase; rank by it.

Metrics. Offline: NDCG@10 on human-judged queries; MRR for navigational queries; recall@1000 for retrieval. Online: CTR at top positions, successful sessions (click with long dwell, purchase), zero-result rate, query reformulation and abandonment rates, time to success.

Labels. (1) **Human relevance judgments** (graded 0–4) on sampled queries — expensive, unbiased, handle tail queries. (2) **Click logs** — plentiful but biased by position and presentation; correct with click models or inverse propensity weighting estimated by result randomization (swap experiments). Purchases/dwell as stronger signals.

Architecture.

1. **Query understanding:** normalization, spell correction, tokenization, intent classification, entity recognition, query expansion/rewriting.
2. **Retrieval (hybrid):** lexical BM25 on an inverted index (exact matches, rare terms) + dense retrieval (bi-encoder embeddings + ANN for semantic matches); union → ~1,000 candidates.
3. **Ranking:** LambdaMART (GBDT with LambdaRank gradients) over ~hundreds of features: BM25 scores per field, embedding similarities, document quality/popularity/freshness, personalization, query–document category match. Optionally a **cross-encoder** (query and document through one transformer) re-scores the top ~50 — most accurate, most expensive.
4. **Blending/business rules:** diversity, sponsored results, freshness for news-like queries.

Latency budget example: query understanding 20 ms, retrieval 50 ms, GBDT ranking 30 ms, cross-encoder on top-50 with a distilled model on GPU 80 ms, overhead 50 ms.

Decision	Option A	Option B	Consider
Retrieval	lexical (BM25)	dense (embeddings)	exact/rare terms vs. semantic matching → hybrid
Labels	human judgments	clicks	cost & coverage vs. bias & volume

Decision	Option A	Option B	Consider
Loss	pointwise	pairwise/listwise (LambdaMART)	listwise optimizes top-of-list quality
Top-k re-ranker	none	cross-encoder	quality vs. latency/cost; distillation

24.6 Design 5 — Ads click-through-rate (CTR) prediction

Clarify. Ads served on a feed; each request runs an auction over hundreds of eligible ads; rank by expected value $\mathrm{eCPM} = \mathrm{bid} \times \mathrm{pCTR}\,(\times \mathrm{pCVR})$. Billions of impressions/day; ~10 ms model budget for hundreds of candidates.



Real-life analogy: the kirana store's counter shelf

Think of it like a kirana owner with one prime shelf next to the counter. Brands pay him a fixed amount each time a customer picks their product up (a click). So he doesn't just choose the brand paying most per pick-up; he chooses the best "payment × chance it gets picked up". If he overestimates that chance for one brand, he wastes the shelf and that brand overpays. So his guesses must be honest numbers (calibrated), not just a correct ordering.

Formula, read aloud: $\mathrm{eCPM} = \mathrm{bid} \times \mathrm{pCTR}$ says: expected money from showing an ad = what the advertiser pays per click × the chance of a click (then scaled to 1,000 showings).

Where it breaks: the shopkeeper rearranges his shelf weekly; an ad auction runs fresh for every page view, in milliseconds, billions of times a day.

Why calibration is the core requirement. pCTR multiplies the bid: if pCTR is inflated for some ads, advertisers overpay and the auction misallocates. So the model must be **calibrated**, not just rank well.

Metrics. Offline: **log loss / normalized entropy**, calibration (predicted/observed click ratio overall and per segment), AUC. Online: revenue, advertiser ROI/conversions, user experience guardrails (ad hide rate, session length).

Data & labels. Impressions → click within a window; conversions are delayed and sparse (attribution windows of days) — model them separately (pCVR) with delayed-feedback techniques. Negative downsampling for training efficiency ⇒ calibration correction $p = \frac{q}{q+(1-q)/w}$ (Part 18). Position/slot bias: position as a feature, fixed at serving.

Features. Huge sparse categorical features: user ID/segments, ad ID, advertiser, campaign, creative, placement, device, time, query/context, and crosses (user segment × ad category). Historical CTRs (smoothed, point-in-time). User interest embeddings from behavior sequences.

Model. Classic: logistic regression with feature hashing and crosses trained online with FTRL-proximal (sparse, fast). Modern: embedding-based deep models (Wide & Deep, DeepFM, DCN-v2, DLRM-style) that learn feature interactions, trained continuously on streaming data; embeddings for billions of IDs sharded across parameter servers. Serving: embedding lookups + small MLP per candidate on CPU/GPU; caching user embeddings per request.

Training. Continual/online training (hourly or streaming) because ad inventory and user interests change fast; warm-start from the previous model. Validation: next-period log loss; replay.

Decision	Option A	Option B	Consider
Model	LR + hashing + crosses	deep embedding models	interaction learning vs. simplicity/interpretability
Training	daily batch	streaming/continual	new ads & campaigns (cold start), freshness

Decision	Option A	Option B	Consider
Negatives	full data	downsampled + recalibrated	cost vs. calibration risk
Objective	clicks	clicks + conversions (multi-task)	advertiser value; delayed feedback

24.7 Design 6 — Credit risk (loan default)

Clarify. Unsecured consumer loans; decide approve/decline and pricing at application. Regulated (fair lending, explainability: adverse-action reasons must be given).



Real-life analogy: a landlord choosing a tenant

Think of it like a landlord deciding whom to rent a flat to. He checks past rent history and salary slips (bureau and application data), and must be able to say *why* he refused someone (regulation). He only learns how reliable a tenant is after a year of rent (long label delay). And he never sees how the people he turned away would have paid, so his experience covers only the tenants he accepted (reject inference).

Formula, read aloud: $\mathrm{EL} = \mathrm{PD} \times \mathrm{LGD} \times \mathrm{EAD}$ says: expected loss = chance they stop paying × share of the money you can't recover × how much is owed when they stop.

Where it breaks: a landlord may go on gut feeling; a lender must avoid protected traits and their hidden proxies, and prove it to a regulator.

Target. Probability of default (e.g. 90+ days past due) within 12 months of origination — **long label delay**, so training data is at least a year old.

Metrics. Discrimination: AUC / Gini ($2\mathrm{AUC} - 1$), KS statistic. **Calibration** of PD (drives pricing and expected loss $\mathrm{EL} = \mathrm{PD} \times \mathrm{LGD} \times \mathrm{EAD}$). Stability: PSI of scores across application vintages. Business: approval rate, loss rate, profit. Fairness: disparate impact analysis across protected groups (which cannot be used as features — and proxies must be checked).

Data & labels. Bureau data (credit history, utilization, inquiries), application data (income, employment, debt-to-income), internal history, possibly bank-transaction (cash-flow) data with consent. **Reject inference problem:** outcomes exist only for approved applicants — the model is trained on a selected population. Mitigations: small randomized approvals near the cutoff, bureau outcomes on rejected applicants' other loans, careful extrapolation.

Model. Traditional: logistic regression **scorecard** with weight-of-evidence binning — transparent, stable, easy reason codes. Modern: GBM with **monotonic constraints** (e.g. risk non-decreasing in utilization) + SHAP for reason codes; often both, with the GBM challenging the scorecard. Validation: **out-of-time** (train on older vintages, test on newer) and out-of-sample; model risk management review.

Monitoring. Score PSI and characteristic (feature) stability monthly; early-delinquency proxies (30 days past due at 3 months) before 12-month labels mature; macroeconomic shifts (rates, unemployment) cause concept drift — stress testing.

Decision	Option A	Option B	Consider
Model	scorecard (LR)	GBM + monotonic constraints + SHAP	accuracy vs. interpretability/regulatory comfort
Selection bias	ignore	reject inference / exploration	fairness and accuracy on the unseen population
Target horizon	12 months	shorter proxy (early delinquency)	label delay vs. faithfulness

24.8 Design 7 — Churn prediction

Clarify. Subscription app; the retention team can contact or offer discounts to ~5% of users per week. Is churn **contractual** (explicit cancellation) or **non-contractual** (inactivity)? Define it: e.g. "no active session in the 30 days after the scoring date" or "cancels within 30 days".

The key reframing. The business doesn't want to know who will churn — it wants to know **whom an intervention will save**. Users who'll churn no matter what, and users who'd stay anyway, are both wasted offers. Ideal target: **uplift** $P(\text{stay} \mid \text{offer}, x) - P(\text{stay} \mid \text{no offer}, x)$. Start with a churn-risk model (labels are easy), then move to uplift modeling once randomized intervention data exists.



Real-life analogy: a gym manager's limited calls

Think of it like a gym manager who has time to call only 5% of members each week. Calling regulars who'd stay anyway is wasted. Calling someone who is moving to another city is wasted too. The calls that matter go to the "persuadables": people who will quit unless someone nudges them. And to spot who is drifting, he may only look at attendance *before* today, not at a cancellation form filled in tomorrow (no leakage).

Formula, read aloud: uplift $= P(\text{stay} \mid \text{offer}, x) - P(\text{stay} \mid \text{no offer}, x)$ says: how much more likely this person is to stay *because* we reached out.

Where it breaks: the manager can't both call and not call the same member; uplift is estimated by comparing similar people who were randomly called or left alone.

Target & snapshot design.

```
   feature window (e.g. last 90 days)   | scoring date |   label window (next 30 days)
 ---------------------------------------+--------------+-------------------------------> time
   only data with timestamps < scoring date may be used for features
```

Features. Engagement trends (sessions per week, slope over 4–8 weeks, days since last active), feature usage breadth, content consumption, billing (failed payments, plan, price increases), support tickets **before** the snapshot, tenure, device/app version, social connections active. Point-in-time correctness is critical (support tickets *about cancelling* are leakage if they're in the label window).

Model & serving. GBM, scored in **batch** weekly or daily; outputs to the CRM. Validation: time-based across multiple snapshot dates; grouped by user when multiple snapshots per user are in training.

Metrics. Offline: AUC, precision/lift in the top 5% (capacity), calibration. Online: randomized holdout — treat top-risk users vs. an untreated control drawn from the same risk band; measure retained revenue net of offer cost. **Feedback loop:** successful interventions remove churners from future labels — keep a control group so labels stay unbiased.

Decision	Option A	Option B	Consider
Target	churn risk	uplift (treatment effect)	needs randomized treatment data; more valuable
Horizon	7 days	30–90 days	actionability vs. predictability
Scoring	batch	near-real-time triggers	cost vs. reacting to "at-risk" moments

24.9 Design 8 — Content moderation

Clarify. Social platform; text, images, video; many policy categories (hate, harassment, violence, nudity, spam, misinformation) with different severities; tens of millions of posts per day; human review capacity limited; multilingual.

Real-life analogy: airport security screening

Think of it like airport security. Bags matching a known banned item are stopped instantly (hash matching). The X-ray machine flags suspicious bags (classifiers), and behaviour matters too, like a first-time flyer carrying fifty identical boxes (account signals). Officers hand-check only flagged bags, the most dangerous first (a review queue ordered by severity × reach). Sure cases are stopped automatically; unsure ones go to a person. Random checks of ordinary bags tell you how much slips through (prevalence sampling).

Where it breaks: banned items at an airport are clear-cut; "violating content" depends on context, culture and language, the rules change over time, and even trained reviewers disagree on borderline posts.

Objective. Reduce users' exposure to violating content ("prevalence" — share of views that are violating) while limiting wrongful removals of legitimate speech. Different actions per confidence and severity: auto-remove, send to human review, reduce distribution, add warning, no action.

Metrics. Per policy: precision/recall at the auto-action and review thresholds; **prevalence** measured by sampling views and labeling them (unbiased, independent of the model); appeal overturn rate (FP proxy); reviewer throughput; fairness across languages/dialects/communities.

Labels. Human reviewers following policy guidelines — **disagreement** is significant on borderline cases; guidelines change (labels go stale); reviewer wellbeing constrains volume. Use multiple reviews for gold sets, calibrate reviewers, track agreement; appeals provide corrective labels.

Architecture.

1. **Hash matching** against known violating media (perceptual hashes) — instant, precise, for re-uploads.
2. **Classifiers** at upload: multimodal pretrained encoders (text, image, video frames, audio transcripts) fine-tuned per policy (multi-task heads), plus account/behavior signals (new account, prior strikes, posting velocity) — behavior often beats content for spam and coordinated abuse.
3. **Decision layer:** thresholds per policy × severity × region; queue prioritization by predicted severity × expected reach (prioritize content about to go viral).
4. **Human review** → labels → retraining; **active learning** picks uncertain items for review.

Failure modes. Adversarial evasion (text in images, misspellings, coded language) → OCR, robust tokenization, rapid retraining; context dependence (reclaimed slurs, news reporting violence) → context features and human review; low-resource languages → multilingual models, targeted labeling; over-enforcement on dialects → per-group error audits.

Decision	Option A	Option B	Consider
Action	auto-remove	human review	precision needed vs. capacity and harm speed
Model	one per policy	multi-task shared encoder	shared learning vs. independent iteration
Signal	content	behavior/account	adversarial robustness, cold start of new accounts

24.10 Design 9 — ETA prediction (ride-hailing / delivery)

Clarify. Predict arrival time for a trip (or food delivery: restaurant prep + pickup + drive). Shown to users and used in dispatch and pricing. Very high QPS; latency a few ms.



Real-life analogy: an auto-rickshaw driver's estimate

Think of it like an experienced auto-rickshaw driver asked, "how long to the station?". He starts from the map app's estimate (the routing engine's ETA) and adds his own correction from experience: school traffic at 4 pm, rain, a festival procession near the temple (the ML model predicts this residual). When you have a train to catch, he pads the promise so he's rarely late: "25 minutes, surely before 30" (a P90 quantile, not the average).

Where it breaks: the driver's guess doesn't change the traffic; a real ETA system's predictions do change behaviour, because dispatch picks drivers using them and riders cancel when an ETA looks long.

Target & loss. Actual travel time (regression). Errors are asymmetric: late arrivals annoy more than early ones — and for promises you may need a **quantile** (P80/P90) rather than the mean. Train with MAE/Huber (robust to GPS glitches and outlier trips) or pinball loss for quantiles.

Metrics. MAE, MAPE on longer trips, % of trips within ±2 min, **bias** (systematic under-prediction erodes trust), calibration of quantiles (does P90 cover 90%?), by city/time/trip length. Online: user trust (cancellations after ETA shown), dispatch efficiency.

Data & features. Routing engine's ETA (sum of segment travel times from a road graph) as a strong **baseline feature**; model predicts a correction (residual) on top — a classic hybrid. Features: route length, segments/road types, turns, real-time traffic speeds, historical speeds by segment × hour × weekday, weather, events, pickup/dropoff types (airports), driver behavior, for delivery the restaurant's prep-time history and current order load.

Model & serving. GBM on trip-level features, or deep models (graph neural nets over road segments; sequence models over the route) at scale. Precomputed segment speed tables updated every few minutes from streaming GPS data. Validation: time-based; slice by city and hour.

Feedback effects. Showing an ETA can change behavior (drivers speed, users cancel); dispatch uses ETAs to choose drivers, shifting which trips occur.

Decision	Option A	Option B	Consider
Target	mean (MAE/MSE)	quantiles	user promise vs. dispatch optimization — maybe both
Architecture	end-to-end ML	routing engine + ML residual	data efficiency, robustness, interpretability
Freshness	historical speeds	real-time traffic features	streaming infrastructure cost

24.11 Design 10 — Demand forecasting (retail)

Clarify. Forecast daily unit demand per SKU × store for the next 1–28 days to drive replenishment. 50K SKUs × 2K stores = 100M series; many sparse (intermittent) series; promotions and holidays matter.

Target subtlety. You observe **sales, not demand**: when a product is out of stock, sales are censored at zero. Training on raw sales teaches the model that stock-outs mean low demand. Fix: exclude or impute stock-out periods, or model censoring.

Decision link. Inventory needs a **quantile**, not a mean: the newsvendor solution orders the quantile $\tau = \frac{c_u}{c_u+c_o}$ (cost of understocking vs. overstocking). So produce probabilistic forecasts (quantiles), evaluated with **pinball loss**, plus point metrics: **WAPE**, **bias**.



Real-life analogy: a sweet shop before Diwali

Think of it like a mithai shop owner deciding how many boxes of kaju katli to make for Diwali week. If he sold out by 4 pm last year, "200 boxes sold" does not mean only 200 people wanted them (sales are censored by stock-outs). Leftover sweets spoil (overstock cost), but each customer turned away is lost profit (understock cost). If a lost sale hurts far more than a spoiled box, he should make enough for a busy day, not an average one.

Formula, read aloud: $\tau = \frac{c_u}{c_u+c_o}$ says: the quantile to stock up to = cost of running short ÷ (cost of running short + cost of leftovers).

Where it breaks: one shop plans one sweet; the retailer forecasts 100 million store–product series at once, most selling zero on most days.

Features. Lags (same day last week/year), rolling means/std, calendar, holidays and events, price and **promotions** (known in advance — future covariates), store attributes, product hierarchy, weather forecasts, cannibalization (a promo on a substitute).

Models. Baselines: seasonal naive, exponential smoothing per series. Strong practical choice: **one global GBM** across all series with lag/calendar/hierarchy features (shares information across sparse series). Deep options: DeepAR, Temporal Fusion Transformer, N-BEATS/N-HiTS for large collections. Hierarchical reconciliation so store-level and region-level forecasts add up.

Validation. Rolling-origin backtesting (train up to week t, forecast $t+1..t+4$, slide), matching the production horizon; never random splits.

Cold start. New products: attributes-based analogs ("similar products' launch curves"), hierarchical priors.

Decision	Option A	Option B	Consider
Series modeling	local (per-series)	global model	data per series, maintenance, cross-learning
Output	point	quantiles	inventory decisions need quantiles
Promotions	ignore	explicit uplift features	promos drive the largest errors

24.12 Design 11 — Duplicate detection (listings, questions, images)

Clarify. Marketplace with 50M listings; find duplicates (same item posted multiple times) at upload and in batch. Precision matters (merging distinct items hurts sellers).

Core challenge: $O(n^2)$ pairs. 50M items ⇒ ~10^{15} pairs. So: **blocking / candidate generation** then **pairwise classification**.

Real-life analogy: an examiner hunting copied answers

Think of it like an examiner checking 2,000 answer sheets for copying. Comparing every sheet with every other means about two million pairs, far too many. So she first groups sheets that could plausibly match: same exam hall, same rare wrong answer (blocking, candidate generation). Only those pairs get a careful side-by-side reading (the pairwise model). Then she groups copiers into rings, carefully: if A resembles B and B resembles C, A and C may still differ (avoid chaining into one giant cluster).

Formula, read aloud: $O(n^2)$ pairs says: double the number of items and the pairs to check grow four times.

Where it breaks: copying is deliberate; duplicate listings are often innocent reposts, and the hardest cases are distinct items that merely look alike, like one phone in two colours.

1. **Candidate generation:** MinHash-LSH on text shingles (approximates Jaccard similarity), SimHash, perceptual hashes for images, embedding ANN (text/image encoders), plus cheap blocking keys (same seller, same category, same price bucket). Target: high recall of true duplicate pairs with ~10–100 candidates per item.
2. **Pairwise model:** GBM on pair features (title similarity scores, image embedding cosine, price ratio, attribute matches, same seller) or a cross-encoder over both texts for hard cases. Output: P(duplicate).
3. **Clustering:** duplicates are transitive in reality but predictions aren't (A≈B, B≈C, A≉C). Use connected components with a high threshold, or correlation clustering; guard against giant components (chaining).

Labels. Human-labeled pairs, with **hard negatives** (similar but distinct items: same model different color) to teach fine distinctions; active learning on uncertain pairs.

Metrics. Pair-level precision/recall at thresholds; recall of candidate generation (what fraction of true duplicate pairs are even proposed); cluster-level purity; online: user reports, seller complaints.

Decision	Option A	Option B	Consider
Candidates	LSH on text	embeddings + ANN	lexical near-copies vs. paraphrases/semantic duplicates
Pair model	feature-based GBM	cross-encoder	cost vs. accuracy on hard cases
Action	auto-merge	flag / suppress in search	precision required

24.13 Design 12 – Anomaly detection (metrics or transactions, few labels)

Clarify. Detect anomalies in thousands of business/infra time series (or in transactions) with few or no labels; alerts go to on-call humans. Define "anomaly": sudden spikes/drops, level shifts, trend breaks, unusual combinations.

Real-life analogy: a night nurse watching monitors

Think of it like a night-shift nurse watching the monitors of a whole ward. "Normal" differs for each patient and changes with the hour: a heart rate that is fine at noon may be worrying at 3 am (seasonality). If the machines beep at every tiny wobble, the nurse starts ignoring them (alert fatigue), so alarms are tuned to a small budget. One real problem can set off five monitors at once, so related alarms are grouped. And when the nurse marks an alarm "false", the system learns from it.

Where it breaks: a nurse knows medically what is dangerous; an anomaly detector only knows "unusual", and unusual is not always bad, like a happy sales spike on a festival day.

Objective. Catch real incidents quickly with a **limited alert budget** (e.g. ≤ 5 false alerts per team per week — alert fatigue kills systems).

Approaches (by data type).

Approach	How	Good for	Limitations
Statistical baselines	seasonal decomposition + robust z-scores; EWMA / control charts	univariate metrics with seasonality	fixed assumptions, per-series tuning
Forecast residuals	forecast expected value (Prophet-like, GBM, ETS), alert on large residual vs. predicted interval	metrics with trends/ holidays	forecast errors → false alerts
Isolation forest	anomalies are isolated in fewer random splits	tabular/multivariate records	global notion of "unusual"; scaling
Density / distance	LOF, k-NN distance	local anomalies	curse of dimensionality
Autoencoders	high reconstruction error = unusual	high-dim data (logs, images)	may reconstruct anomalies too well
Semi-supervised	use the few labels (incidents, confirmed fraud) to train a classifier on top of anomaly scores	when feedback accumulates	label scarcity

Evaluation without labels: replay historical incidents (did it catch them, how early?), **inject synthetic anomalies** (spikes, shifts) into real series and measure detection, precision of alerts judged by on-call feedback, time-to-detect.

Operational design: thresholds set by alert budget per series/team; deduplicate and group correlated alerts (one root cause → 50 metrics spike); suppression during known events (deploys, holidays); feedback buttons ("real / not real") feed a supervised re-ranker of alerts.

24.14 A universal failure-mode checklist for any design

Say these before the interviewer asks

- **Cold start** — new users, items, merchants, regions.
- **Feedback loops** — the model shapes its own future training data (exposure, selective labels).
- **Label delay** — how you monitor before labels mature.
- **Training–serving skew** — one feature definition, point-in-time joins, parity tests.
- **Adversaries** — spam, fraud, moderation, SEO: expect adaptation.
- **Distribution shift / seasonality** — retraining cadence, time-based validation.
- **Latency & cost** — funnel, precomputation, distillation, caching, fallbacks.
- **Fairness & policy** — slices, protected groups, explainability, regulation.
- **Outages** — fallbacks when a feature source or the model is down; rollback path.

GenAI system designs — how the framework changes

What's different when the core component is an LLM

The reusable framework (clarify → objective → metrics → data → architecture → training → serving → monitoring) still applies. Four things change: **(1) evaluation is the hardest part** — define golden sets, judges and online signals before building; **(2) cost and latency are dominated by tokens**, so budget them explicitly; **(3) the model's behaviour is steered by prompts, retrieval and tools**, which are versioned artifacts like code; **(4) security has a new attack surface** — prompt injection and tool misuse. Designs 13–18 apply this to the GenAI questions most often asked in 2026–27 loops.

The whole set of GenAI designs as one picture

Think of it like opening a restaurant around a brilliant but unpredictable star chef (the LLM). The chef can cook almost anything, but sometimes improvises wrongly, is expensive per dish, and is slow at rush hour. A good restaurant design surrounds the chef with a pantry of verified ingredients (retrieval), recipes for the common dishes (fixed workflows), a manager who approves anything costly (policy checks and human approval), tasters who check dishes before they leave (evaluation and guardrails), and a kitchen layout that serves many tables at once (batching and caching).

Where it breaks: a human chef learns from every complaint automatically; an LLM system only improves when you log failures, turn them into test cases and change prompts, retrieval or models deliberately.

24.15 Design 13 — Enterprise knowledge assistant (RAG with permissions)

Clarify. 20,000 employees ask questions about policies, engineering docs and past tickets. About 5M documents across wiki, drive, tickets and PDFs, with thousands of edits a day. Answers must cite sources, respect document permissions, and say "I don't know" rather than guess. ~50K questions/day; time to first token < 1.5 s, full answer < 6 s. English plus some Hindi queries.

Objective & target. Maximize correct, grounded answers; minimize confident wrong answers and **any** permission leak (a hard constraint, not a trade-off).

Real-life analogy: a reference librarian with keys

Think of it like a librarian at a large company library. She doesn't memorize every book; she knows how to find the right pages quickly, reads them, and answers with "see page 12 of the HR handbook". Some rooms are locked, and she checks your ID card before fetching anything from them. If the books don't cover your question, she says so instead of inventing an answer.

Where it breaks: a librarian notices when two books contradict each other or one is outdated; the system only does this if versions, dates and authority are stored as metadata and used in ranking.

Metrics. Offline (on a golden set of ~1,000 real, de-identified questions with labeled source passages): retrieval recall@10, faithfulness (share of answer claims supported by retrieved text), answer correctness (LLM judge calibrated to human labels, Part 45), citation accuracy, abstention quality. Online: thumbs-up rate, follow-up/rephrase rate, deflected helpdesk tickets, time saved. Guardrails: permission-leak incidents (must be zero), p95 latency, cost per answer.

Data. Connectors per source with incremental sync (change feeds), document versions and access-control lists (ACLs) stored with every chunk. Golden questions from search logs and helpdesk tickets, plus synthetic questions generated from passages and audited.

Architecture.

```
INGEST  connectors → parse (tables, headings) → dedupe versions → chunk by section (~400 tokens,
        keep title path) → embed → hybrid index {BM25 + vector} with fields: ACL, source, date, version
QUERY   SSO identity → conversational query rewrite → hybrid retrieve top-50 WITH ACL filter
        → cross-encoder rerank → top-8 chunks → LLM (grounding prompt, cite [n], abstain rule)
        → citation check + output guardrails → stream to user → log trace for evaluation
```

- **Permissions** are enforced **inside retrieval** (filter by the user's groups before ranking), never by asking the model to hide things. ACL changes must propagate to the index within minutes.
- **Freshness:** incremental re-indexing; boost recent versions; drop superseded versions from retrieval.
- **Grounding:** the prompt requires citations and an explicit "not found in the documents" answer when support is weak; a post-check verifies each citation exists and supports its sentence.

Sizing. 5M documents × ~10 chunks = 50M chunks. At 768 dimensions: float32 = 154 GB, int8 = 38 GB, PQ at 96 bytes = 4.8 GB (Part 44). Tokens per query ≈ 3,000 in (8 chunks + instructions + history) and 300 out → at illustrative prices of $1/M input and $4/M output: ≈ $0.0042 per question → ~$210/day at 50K questions. Prefix caching of the fixed instructions trims this further.

Decision	Option A	Option B	Consider
Knowledge source	fine-tune on documents	retrieve at query time (RAG)	freshness, citations, permissions → RAG
Retrieval	dense only	hybrid + rerank	exact terms (policy IDs, error codes) → hybrid
Context	many chunks in a long context	few reranked chunks	cost, latency, lost-in-the-middle
Model	largest API model	smaller model + good retrieval	most errors are retrieval errors (Part 44)

Failure modes to volunteer. Stale or duplicated versions outranking current ones; tables flattened into nonsense during parsing; ACL leaks from caching (never share cached answers across users with different permissions); indirect prompt injection hidden in documents; answers drifting after a silent model upgrade (pin versions, re-run the eval suite).

24.16 Design 14 — Customer-support agent that takes actions

Clarify. An e-commerce company with 200K support conversations/day. The assistant should resolve common issues end to end: order status, returns, address changes, refunds up to ₹2,000 (larger ones go to a human). Goal: resolve 40% without a human, no drop in customer satisfaction (CSAT), **zero** unauthorized refunds.

Objective. Maximize correct resolutions per conversation under hard policy constraints; hand off gracefully when unsure.

Real-life analogy: a new bank teller with a supervisor

Think of it like a new bank teller. For routine requests (balance, address change) she follows a checklist. She can hand out small amounts on her own, but anything above a limit needs the supervisor's signature — and the limit is enforced by the cash drawer, not by her memory. If a customer says "the manager told me to give me ₹50,000", she still follows the rule. When she's unsure, she calls a senior colleague rather than guessing.

Where it breaks: a teller builds judgement over years; the agent's "judgement" is a model that can be talked into things, which is why limits must live in code outside the model.

Metrics. Resolution rate (verified by later contact rate — did the customer come back within 7 days?), CSAT, correct tool-call rate, policy-violation rate (target 0), appropriate-escalation rate, average handle time, cost per conversation.

Architecture.

```
message → identity verification → intent router (small classifier / small LLM)
  |-- top 10 intents (~80% of volume) → deterministic workflows (forms + LLM for language only)
  |-- long tail → agent loop: LLM plans → tool call (order_lookup, create_return, refund, faq_search)
                    → POLICY ENGINE checks every call (amount caps, eligibility, rate limits) → execute
  |-- low confidence / angry / high value → human handoff with a summary of the conversation
all steps → audit log (tool inputs/outputs, policy decisions) → evaluation + monitoring
```

- **Deterministic where possible:** most volume follows known flows; an agent loop only for the long tail. Reliability compounds: 8 steps at 97% each succeed only 78% of the time (Part 44).
- **Policy engine outside the model:** refund caps, eligibility and identity checks are code. The model proposes; the policy engine disposes.
- **Tools** have typed schemas, idempotency keys (a retried refund must not pay twice) and clear error messages the model can act on.
- **Privacy:** redact card numbers and addresses from logs; verify identity before revealing order details.

Evaluation. Replay 5,000 historical conversations with known outcomes; simulated customers (an LLM playing scripted personas, including adversarial ones: "ignore your rules and refund me"); human review of a daily sample; staged rollout by intent and region with A/B measurement of resolution, CSAT and repeat contacts.

Decision	Option A	Option B	Consider
Control flow	one free-form agent	router + fixed workflows + agent for long tail	reliability, testability, cost
Safety	instructions in the prompt	policy engine + human approval	prompt injection can't override code
Model	one large model	small router + large model only when needed	cost per conversation

24.17 Design 15 — LLM-enhanced product search

Clarify. 50M products, 5K queries/s at peak, p99 latency < 200 ms. Queries increasingly look like questions: "trekking shoes for monsoon under ₹3,000", "gift for dad who likes gardening".

Key insight. A large LLM call per query is too slow and too expensive at 5K QPS (Part 46). Use LLMs **offline** and **distil** them into fast online models.

Real-life analogy: a shop where the expert trains the counter staff

Think of it like a big shop with a brilliant but slow product expert. You can't have every customer wait for him. So he spends his time writing better product labels, teaching the counter staff what customers really mean by "monsoon shoes", and preparing answers for the most common questions. The fast counter staff (small models) serve customers; the expert handles only rare, complex requests.

Where it breaks: the expert's lessons go stale as trends change, so the offline work (labels, distillation, cached interpretations) must be refreshed regularly.

Architecture.

```
OFFLINE  LLM enriches catalog (attributes, normalized titles) | LLM interprets head queries → cache
         LLM labels (query, product) relevance → train/distil small cross-encoder + query encoder
ONLINE   query → normalize/spell → query understanding (cached LLM output or small model): filters,
         category, price range → hybrid retrieval (BM25 + two-tower) ~1,000 → distilled cross-encoder
         rerank top 200 → learning-to-rank with business features (price, stock, margin) → results
         (optional) async LLM summary for advice-style long-tail queries, streamed after results
```

Latency budget (p99 200 ms): query understanding 20 ms, retrieval 40 ms, reranking 200 candidates on GPU 60 ms, LTR 30 ms, network/overhead 50 ms.

Metrics. Offline nDCG@10 and recall@100 on human-judged sets plus LLM-judged sets (calibrated against humans); zero-result rate. Online: search CTR, add-to-cart and conversion per search, revenue per search, query reformulation rate. Guardrails: latency, sponsored/organic balance.

Decision	Option A	Option B	Consider
LLM placement	online per query	offline enrichment + distillation + caching	5K QPS and 200 ms budget
Relevance labels	clicks only	clicks + LLM judgements + human audit	position bias vs. label cost
Long-tail advice queries	LLM answer inline	results first, streamed summary after	latency and trust

24.18 Design 16 — Code assistant (inline completion + chat)

Clarify. 10,000 developers; inline completions must appear in < 300 ms (p50) while typing; a chat mode answers questions about the codebase and can run tests. Code must stay inside the company network → self-hosted models.

Real-life analogy: a sous-chef who finishes your sentences

Think of it like an experienced sous-chef standing next to you. When you reach for the onions, he already hands you the knife (inline completion) — fast, small, often right, and easy to ignore. When you ask "how do we make the house gravy?", he checks the restaurant's recipe book and tells you (chat with repository retrieval). He must be quick for the first and thorough for the second — so two different assistants.

Where it breaks: a sous-chef knows when he's unsure; a code model suggests wrong code with the same confidence, so tests, linters and security scanners must check suggestions.

Architecture.

- **Completion path:** a small code model (a few billion parameters) with **fill-in-the-middle** (prefix + suffix of the file). Context assembly within ~2K tokens: the cursor's surrounding code, open tabs, and snippets

retrieved from the repository by embedding similarity and symbol references (definitions of functions being called). Debounce keystrokes, cancel stale requests, stop at the end of a line or block. **Prefix caching** (the file prefix repeats between keystrokes) and speculative decoding cut latency (Part 46).

- **Chat/agent path:** a larger model; retrieval over the repo, docs and tickets; tools to search code, open files and run tests in a sandbox; answers cite files and lines.
- **Safety:** secret and license filters on suggestions; no training on code with unclear licenses; security scanning of generated code.

Metrics. Acceptance rate and **persistence** (share of accepted characters still present after 30 s / 10 min); latency; pass@k on an internal benchmark built from the company's own tasks with tests (execution-based, Part 45); bug and vulnerability rates in accepted code; developer satisfaction. Beware: acceptance rate rewards short, obvious completions — track persistence and outcome metrics too.

Sizing sketch. 10K developers, ~30% typing at peak, one request every ~3 s each → ~1,000 requests/s, each ~2K prompt tokens and ~30 output tokens: **prefill-heavy**, so prefix caching and batching of prefill matter more than decode speed.

24.19 Design 17 — Content moderation with LLMs at scale

Clarify. 500M posts/day in 20 languages; policies change monthly; pre-publication checks have a 100 ms budget; users can appeal. (Design 8 covers the classic system; this is the GenAI-era version.)

Key insight — a cascade. Running an LLM on every post is unaffordable; running only small classifiers makes policy updates slow. Combine them.

```
post → hash match against known-bad content (instant) → small multilingual classifiers (high recall)
     → clearly fine (~95%): publish | clearly violating: act
     → uncertain slice (~3%): LLM reads the POLICY TEXT + post, returns label + reason
     → high-severity or low-confidence: human review queue → appeals → audit
policy change → LLM labels fresh samples under the new policy → retrain/distil small classifiers
```

Cost lens. 500M posts × 3% = 15M LLM calls/day × ~800 tokens = 12B tokens/day. At an illustrative $0.10 per million input tokens: ~$1,200/day. Sending every post to the LLM would be 400B tokens/day ≈ $40,000/day — the cascade is ~33× cheaper.



Real-life analogy: airport security lanes

Think of it like airport security. Every bag goes through the fast X-ray (small classifiers). Most are obviously fine and move on; clearly banned items are stopped at once. Only bags that look odd go to a trained officer who opens them and checks the current rulebook (the LLM reading the policy). The rare serious cases go to a supervisor (human reviewers). When the rules change, officers train the X-ray operators on the new examples.

Where it breaks: smugglers adapt to the X-ray; adversarial users misspell words, use images of text, or code-switch between languages — so red-teaming and fresh training data are continuous, not one-off.

Metrics. Precision and recall **per policy and per language**, views-weighted prevalence of violating content, time to action, appeal overturn rate, reviewer agreement (κ), and fairness across dialects (over-enforcement on some dialects is a known failure).

24.20 Design 18 — A multi-tenant LLM inference platform

Clarify. Internal platform serving an 8B chat model, a 70B model and an embedding model to 30 teams; ~2,000 requests/s total. SLOs for interactive chat: time to first token p95 < 800 ms, time per output token < 50 ms. Batch jobs (summarizing documents overnight) should be cheap. Fixed GPU budget.



Real-life analogy: a shared kitchen for many restaurants

Think of it like a cloud kitchen that cooks for 30 restaurant brands. Orders arrive all the time; the kitchen keeps every stove busy by cooking many orders together (batching), reuses common bases like gravy prepared once for many dishes (prefix caching), gives delivery orders priority over tomorrow's catering (interactive vs. batch lanes), and caps how many orders one brand can send at once (quotas) so nobody starves.

Where it breaks: a kitchen's bottleneck is visible; in LLM serving it's often hidden — KV-cache memory, not compute, usually limits how many requests fit at once.

Architecture. API gateway (authentication, per-tenant quotas and rate limits, routing by model and priority) → request scheduler → model servers with **continuous batching**, **PagedAttention**, **prefix caching** and quantization (Part 46) → autoscaling on queue depth and KV-cache utilization. Separate **interactive** and **batch** pools (or priorities); **prefill/decode disaggregation** for long-prompt traffic; many **LoRA adapters** served on one shared base model for team-specific fine-tunes.

Capacity math (70B model, GQA with 80 layers, 8 KV heads, head dim 128). Weights: 140 GB in fp16 → two 80 GB GPUs with tensor parallelism, or ~70 GB in FP8 on one GPU. KV cache per token: $2 \times 80 \times 8 \times 128 \times 2$ bytes ≈ **0.33 MB**; a 4K-token conversation ≈ **1.3 GB**. On two 80 GB GPUs with fp16 weights only ~20 GB remains for KV (about 15 concurrent 4K conversations after activation overheads); with FP8 weights on the same pair, ~60–80 GB remains (about 45–60 conversations). Quantization buys **batch size**, not just memory — and KV memory is the real cap on concurrency. Decode is memory-bound: more concurrent sequences per replica (up to the KV limit) raise throughput almost for free.

Metrics. SLO attainment (TTFT, TPOT) per tenant, **goodput**, GPU utilization, cost per million tokens, queueing delay, error and timeout rates, fairness across tenants.

Decision	Option A	Option B	Consider
70B placement	tensor parallel on 2 GPUs (fp16)	1 GPU with FP8/int8	latency, quality on your evals, GPU count
Traffic mix	one pool	interactive and batch pools / priorities	SLOs vs. utilization
Long prompts	same servers	chunked prefill or disaggregated prefill	TTFT spikes for other users
Team fine-tunes	one deployment per model	multi-LoRA on a shared base	GPU cost vs. isolation

PART 25

Data-Centric ML

Why better data often beats a better model

A model is an estimator of the function **defined by your labels on your sample**. If labels are wrong, the target is wrong; if the sample is biased, the function is learned for the wrong population; if a slice is missing, the model knows nothing about it. A better model estimates the wrong thing more precisely. Improving the data changes **what** is being learned — and it's usually where the remaining error lives.

The whole Part as one picture

Think of it like a cook who keeps buying fancier stoves while using stale spices and a recipe card full of typos. A better stove only cooks the wrong recipe more precisely. Checking the ingredients (label quality), tasting dish by dish to find which ones fail (error analysis on slices), buying the missing spices (collecting data for weak slices) and fixing the recipe card (clearer guidelines) usually improve the meal far more. Data-centric ML means keeping the same stove and improving what goes into it.

Where it breaks: in a kitchen you can taste the result directly; in ML, "tasting" means scoring on a test set, and if that test set has errors too, you can't even tell whether the meal improved.

25.1 Why improving data can beat changing the model

The mechanisms

1. **The ceiling argument.** Label noise raises the achievable error floor and caps measurable accuracy: no model can score above the test labels' own agreement with the truth. Northcutt et al. (2021) estimated label errors average at least **3.3% across 10 popular benchmark test sets** and at least **6% of ImageNet's validation set** — enough that, on the corrected ImageNet test set, ResNet-18 outperformed ResNet-50 once mislabeled examples made up just 6% of the data. Model rankings on noisy benchmarks can be artifacts.
2. **Systematic label errors teach the wrong function.** Random noise mostly adds variance; *systematic* noise (one annotator's habit, an ambiguous guideline, a heuristic used to create labels) is learned faithfully and shows up as confident, consistent errors.
3. **Errors concentrate in slices.** Most remaining error typically sits in a few identifiable slices (a language, a device, night-time images, a new product category). Targeted data for those slices fixes them; a new architecture spreads its effort everywhere.
4. **Models are commoditized; data is specific.** Swapping one strong GBM or pretrained encoder for another typically moves metrics a little; fixing label definitions, adding a missing signal, or covering an absent population can move them a lot.
5. **Evaluation depends on data too.** A noisy or unrepresentative test set can't even tell you whether a model change helped.

The data-centric loop: error analysis (read errors, cluster them) → slice metrics → hypothesis about the data (missing coverage? inconsistent labels? leakage? duplicates?) → targeted fix (relabel, collect, augment, clarify guidelines, deduplicate) → retrain the *same* model → re-evaluate on a clean, versioned test set.



Real-life analogy: a great driver, wrong map

Think of it like navigating with an outdated map. A better driver (a bigger model) still reaches the wrong address if the map is wrong; they follow it faithfully and confidently. The map's mistakes are usually bunched in a few places, like a new colony whose roads were never drawn (errors concentrate in slices). Updating those few areas fixes most wrong turns. And if the map used to *judge* the drivers is also wrong, you can't even tell which driver is better (noisy test sets).

Where it breaks: a map error is one wrong road; label noise can also be random and thinly spread, which mostly adds noise rather than teaching a wrong route.

25.2 Label noise and incorrect labels

Type	Example	Effect
Random (symmetric)	annotators occasionally click the wrong class	lowers ceiling; adds variance; deep nets eventually memorize it
Class-conditional	"wolf" often labeled "husky"	biases the decision boundary between specific classes
Instance-dependent / systematic	hard or ambiguous examples mislabeled; heuristic labels wrong on a sub-population	the model learns the error pattern as if it were real

Detect: out-of-fold predictions that **confidently disagree** with the given label (the idea behind *confident learning* / cleanlab); highest-loss training examples; duplicates or near-duplicates with conflicting labels; low inter-annotator agreement items; loss curves (noisy examples are learned last — memorization happens late in training).

Handle: relabel (best, with adjudication); remove; down-weight; robust losses (MAE-like or generalized/symmetric cross-entropy, bounded losses); label smoothing; early stopping (fit clean patterns before memorizing noise); model the noise with a transition matrix; co-teaching (two networks select small-loss examples for each other). **Keep the test set cleanest of all** — double-annotated and adjudicated.



Real-life analogy: a songbook with misprints

Think of it like a music student learning from a songbook with some wrong notes. A random misprint here and there sounds odd, and a good student mostly ignores it. But if the printer *always* prints one note flat, the student learns it flat, confidently (systematic noise). To find misprints, look where a student who has learned the rest of the song well strongly disagrees with the book (confident learning). The copy used for the final performance should be the most carefully checked of all (the clean test set).

Where it breaks: a musician can hear a wrong note; a model has no ear, and with enough practice it will happily memorize the misprints too.

25.3 Missing labels

- **Semi-supervised learning:** pseudo-labeling (train, label confident unlabeled examples, retrain), consistency regularization (predictions stable under augmentation).
- **Weak supervision:** programmatic labeling functions (heuristics, keyword rules, knowledge bases) combined by a label model that estimates their accuracies (Snorkel-style).
- **Active learning:** ask humans to label the examples the model is most uncertain about or that are most representative/diverse — typically far more label-efficient than random sampling.
- **Positive–unlabeled (PU) learning:** when only positives are confirmed (confirmed fraud; "unreported" ≠ legitimate), treat unlabeled data as a mixture rather than as negatives.
- **Self-supervised pretraining / transfer** from large unlabeled or related data, then fine-tune on the few labels.



Real-life analogy: sorting old family photos

Think of it like sorting a huge box of old family photos where only a few have names written on the back. You recognise grandma for sure in some, so you write her name on those (pseudo-labeling). You use rules of thumb, like "photos with the old scooter are from the 1980s" (weak supervision). You ask grandpa only about the photos you're most unsure of, not all of them (active learning). And a photo not marked "Goa trip" might still be from Goa (positive–unlabeled data).

Where it breaks: your guesses can be wrong, and writing wrong names on photos, then learning from them, can spread one mistake across the whole album.

25.4 Sampling bias

The collection process differs from deployment: data from one region/app version/time; labels only for reviewed cases; logged data shaped by a previous model (exposure bias); volunteer/opt-in data. **Symptoms:** great offline metrics, poor performance on the actual population, surprising slice failures. **Fixes:** define the target population explicitly; collect data from under-covered regions (targeted or randomized sampling); reweight to match deployment (importance weights); exploration for logged-policy data; audit coverage by slice.



Real-life analogy: an exit poll at one station

Think of it like predicting an election by polling people outside one big-city metro station. The survey looks precise and consistent, but it describes metro commuters, not farmers or small-town voters, so the confident prediction fails on counting day. The fixes match ML: decide whose opinion you actually need (the target population), send surveyors to the villages you missed (targeted sampling), and give extra weight to under-covered groups (reweighting).

Where it breaks: reweighting can boost a group that was surveyed too little, but if a group was never surveyed at all, no weighting can invent their views.

25.5 Data imbalance and the long tail

Many real distributions are long-tailed: a few classes/items/queries account for most data. Head-dominated training fits the head. Remedies: targeted collection for tail slices; class-balanced reweighting (e.g. weights based on the "effective number" of samples rather than raw counts); resampling (Part 18); transfer and few-

shot methods (pretrained embeddings let tail classes borrow structure); hierarchical labels (tail classes share parents); evaluate head and tail separately — a model can gain overall while getting worse on the tail.



Real-life analogy: net practice against rare bowlers

Think of it like a young batter whose net sessions are 95% medium pace. They become great against the common bowlers (the head) and helpless against a rare left-arm wrist spinner (the tail). A smart coach books extra sessions against rare bowlers (targeted collection, reweighting), and knows that skill against off-spin partly carries over (transfer, shared parent classes). He also checks the record against each bowling type, because the overall average can rise while spin play gets worse.

Where it breaks: a coach can usually find a real spinner; for many tail classes you simply can't get more examples, so borrowing structure from related classes is the main hope.

25.6 Duplicates

Exact and near-duplicates (retries in logging, reposted content, templated text, augmented copies) cause: **train–test contamination** (inflated evaluation), over-weighting of duplicated examples (the model cares more about what's repeated), and memorization. Detect with hashing (exact), MinHash/SimHash (near-duplicate text), perceptual hashes or embeddings (images). **Deduplicate before splitting**, or group duplicates into one split.



Real-life analogy: a leaked mock-test question

Think of it like a teacher building a mock test from a question bank where some questions appear twice, slightly reworded. If a question is in both the practice sheet and the mock test, a high score shows memory, not skill (train–test contamination). A question repeated five times in the practice sheet also gets studied five times as hard (over-weighting). The fix: remove repeats *before* splitting questions into practice and test, or keep every copy of a question on the same side.

Where it breaks: a teacher spots reworded questions easily; near-duplicate data needs hashing or embeddings to find, and some look-alike rows are genuinely separate events that should stay.

25.7 Outliers

Distinguish **errors** (unit mistakes, sensor glitches, test accounts, bots) from **rare genuine events** (a whale customer, an actual fraud). Errors should be fixed or removed at the source; genuine rare events are often exactly what the model must handle. Tools: robust statistics (median/MAD, IQR rules), isolation forest, residual analysis after an initial model. Always look at the rows before deciding.

Real-life analogy: odd entries in a shop ledger

Think of it like a kirana owner reviewing his ledger. One entry says a packet of biscuits sold for one lakh rupees: clearly a typing mistake with extra zeros (an error to fix at the source). Another says a customer bought 40 kg of rice: odd, but it was a real wedding order (a rare genuine event). Deleting both as "outliers" would erase exactly the kind of customer he most needs to plan for. So he looks at the actual bills before deciding.

Where it breaks: a shopkeeper can phone the customer to check; with millions of rows, you rely on robust statistics and models to point you to the few rows worth checking by hand.

25.8 Dataset shift and leakage (data-centric view)

Shift (Parts 1, 23) is a *data* problem first: keep training data fresh, reflect the deployment population, and version datasets so you know what each model saw. Leakage (Part 1) is a *data construction* problem: features and labels must be assembled point-in-time, with every preprocessing step fit only on training data.

Real-life analogy: a forecaster with tomorrow's newspaper

Think of it like a weather forecaster. If she learned only from monsoons of twenty years ago, today's changed climate will fool her (dataset shift), so she keeps her records fresh and notes which years each forecasting method learned from (versioned datasets). Leakage is like "forecasting" yesterday's rain while holding today's newspaper that already reports it: the forecast looks perfect in practice runs and is useless for real.

Where it breaks: a newspaper is an obvious cheat; in data, the "future" often hides inside a feature built with a later timestamp or a scaler fit on all the data, and only point-in-time checks find it.

25.9 Annotation quality and human disagreement

Running an annotation program well: precise **guidelines** with examples and counter-examples for borderline cases; **gold questions** with known answers to monitor annotators; **multiple annotations** per item for a subset; **adjudication** of disagreements by experts; calibration sessions; tracking per-annotator accuracy and drift; realistic time budgets (rushed labeling is noisy labeling).

Measuring agreement: Cohen's kappa for two annotators corrects raw agreement for chance: $\kappa = \frac{p_o - p_e}{1 - p_e}$, where p_o is observed agreement and p_e the agreement expected by chance from each annotator's label frequencies. (Fleiss' kappa or Krippendorff's alpha for more annotators.) Raw agreement of 90% can hide a low κ when one class dominates.

Real-life analogy: two judges on a singing show

Think of it like two judges on a TV singing contest. Clear rules with example performances (guidelines), planted singers whose grade is already known (gold questions), and a head judge settling ties (adjudication) keep judging fair. If both judges say "yes" to 90% of singers out of habit, they agree often just by luck, so raw agreement flatters them. And when judges honestly differ on taste, the split vote is information (soft labels), not a mistake.

Formula, read aloud: $\kappa = \frac{p_o - p_e}{1 - p_e}$ says: (how often they actually agreed – how often luck alone would make them agree) ÷ (the most room there was to beat luck).

Where it breaks: judges' disagreements happen on screen; annotator disagreement stays hidden unless you collect several labels per item.

Disagreement isn't always error. For subjective or ambiguous tasks (toxicity, sentiment, relevance), disagreement reflects genuine ambiguity. Options: keep **soft labels** (the distribution of annotator votes) and train on them; model annotators individually; report human agreement as the realistic performance ceiling; clarify guidelines where disagreement comes from ambiguity in the *instructions* rather than the data.

Q: "You have a budget for either a bigger model or more labeling. How do you decide?"

Strong answer. "I'd look at where the error comes from. If the model underfits and a bigger model overfits a subset easily, capacity isn't the issue. Error analysis tells me whether errors concentrate in slices with little data, or on examples whose labels look wrong. I'd check label quality directly — double-annotate a sample and compute agreement — and draw a learning curve: if validation error still falls steeply with more data, labeling pays. If labels are noisy, relabeling and clarifying guidelines likely beats both. I'd spend on the model only if the curves are flat, labels are clean, and training error itself is the bottleneck."

25.10 Data-centric work for LLMs: curation, synthetic data and contamination

For LLM fine-tuning and evaluation, data quality dominates model choice even more than in classical ML.

Curating fine-tuning data.

- **Quality over quantity:** a few thousand clean, diverse, expert-written examples often beat hundreds of thousands of noisy ones for teaching format and behaviour (Part 43).
- **Deduplicate** exactly and approximately (MinHash, embedding similarity) — duplicates cause memorization and inflate evaluation.
- **Coverage:** stratify by intent, difficulty, language and edge cases; mine failures from production traces.
- **Consistency:** write labeling guidelines, measure annotator agreement (κ), resolve disagreements — inconsistent targets teach inconsistent behaviour.
- **Decontaminate:** remove anything overlapping your evaluation sets (n-gram and embedding checks), or your eval scores become meaningless (Part 45).

Synthetic data (generated by a stronger model): cheap and scalable for instruction tuning, data augmentation, and building eval sets. Risks: it copies the generator's errors and style, reduces diversity (**mode collapse** — everything sounds the same), may violate the generator's terms of use, and repeated training on model outputs can degrade quality over generations. Mitigate with filtering by a verifier or tests (for code and math, keep

only outputs that pass), diversity prompts and seeds from real data, human audits of samples, and always evaluate on **real** held-out data.



Real-life analogy: cooking classes from a cookbook written by one chef

Think of it like training new cooks using recipes. A few hundred carefully tested recipes teach more than thousands of scribbled, contradictory ones. Having one famous chef write all your practice recipes (synthetic data) is fast, but every cook ends up copying his habits and mistakes — and if the practice recipes include the exact dishes on the final exam (contamination), the exam tells you nothing.

Where it breaks: cooks can taste and reject a bad recipe; a model trained on flawed synthetic data absorbs the flaws unless you filter with tests, verifiers and human review.

PART 26

Practical Model Selection

Model selection is a constrained optimization

Maximize expected business value subject to **latency, memory, training cost, interpretability, maintainability and data** constraints. The best offline model is often not the best deployable model. State the constraints first; the model usually follows.

The whole Part as one picture

Think of it like hiring for a job. The candidate with the highest exam marks isn't automatically the right hire. You also ask: can they work fast enough (latency)? What salary do they want (serving and training cost)? Can they explain their decisions to a client or an auditor (interpretability)? Will the team manage to work with them for years (maintainability)? Good hiring managers write down these limits first, then pick the best candidate who fits inside them.

Where it breaks: a new hire learns and adapts on the job; a deployed model stays exactly as it is until you retrain it, and each retrain must be checked against the same limits again.

26.1 The trade-off dimensions

Dimension	Questions
Accuracy	which metric, at which operating point, on which slices? how big is the gap vs. a simple baseline?
Latency	p99 budget per request/candidate; CPU or GPU; batchable?
Memory	server RAM, GPU memory, on-device limits, embedding tables
Training cost	hours of compute per run; how often retrained; iteration speed for the team
Interpretability	regulatory explanations, debugging, stakeholder trust — global vs. local
Maintainability	feature pipelines to maintain, number of models, team familiarity, failure surface
Data size	small data favors strong priors (linear, GBM, pretrained); huge data enables flexible models
Feature characteristics	tabular/mixed types, sparse high-dim, sequences, images/text, high-cardinality IDs

Real-life analogy: buying a phone

Think of it like buying a phone. The camera score (accuracy) is only one line on the spec sheet. You also care how fast apps open (latency), how much storage you get (memory), the price and running costs (training cost), whether a nearby shop can repair it (maintainability), and whether you understand its settings well enough to fix problems yourself (interpretability). A phone that wins on camera but blows your budget is the wrong phone for you.

Where it breaks: phone specs are printed on the box; a model's latency, memory and accuracy on your data are only known after you measure them on your own hardware and slices.

26.2 Scenarios — which model and why?

Real-life analogy: picking transport for a trip

Think of it like choosing how to travel in India. For 3 km across town, an auto-rickshaw; for 500 km, an overnight train; for 2,000 km in a hurry, a flight; for a narrow village lane, a scooter. No vehicle is "best"; distance, budget, deadline and road decide. Models are the same: small tabular data, huge sparse IDs, text, tight latency or strict explanations each point to a different family, and the scenarios below are the route guide.

Where it breaks: you can see a road before you set off; with data, you often learn its true "terrain" only after trying a baseline model or two.

10K tabular samples

Choice: gradient-boosted trees (LightGBM/XGBoost/CatBoost) with conservative settings and early stopping; logistic regression with good features as the baseline; random forest as a robust alternative. Use cross-validation (the data is small, so CV variance matters) — repeated CV for stable comparisons.

Why: GBMs handle mixed types, missing values and interactions with little preprocessing, and are hard to beat on tabular data. With 10K rows, strong regularization (fewer leaves, larger min-child) matters. Deep nets rarely win here. Tabular foundation models (e.g. TabPFN, Nature 2025) have become competitive on datasets of roughly this size and are worth a try when accuracy is paramount.

I'd change my mind if: interpretability is required (→ logistic/GAM), features are mostly text (→ text models), or the relationship is known to be near-linear (→ regularized linear model; fewer ways to overfit).

100M tabular samples

Choice: histogram GBMs (LightGBM/XGBoost) with distributed or GPU training; subsample for fast iteration and train the final model on all data. If there are many high-cardinality categorical IDs or behavioral sequences, **embedding-based deep models** start to win (they learn representations for millions of IDs and process sequences). For extreme scale with tight latency: logistic regression with hashing/crosses trained online (FTRL).

Why: at this scale, variance is low and the bottleneck shifts to capacity, compute and iteration speed. Single holdout (time-based) instead of CV. Watch training cost: a 4-hour GBM run vs. a 2-day deep model changes how many experiments the team can run.

Sparse text classification

Choice: baseline logistic regression or linear SVM on TF-IDF word + character n-grams — fast to train, fast to serve, strong. Multinomial NB if labeled data is tiny. For best accuracy: fine-tune a pretrained transformer encoder; distill it into a small model if latency is tight.

Why: high-dimensional sparse data is nearly linearly separable; linear models with L2/L1 regularization are the right bias. Transformers bring semantics (synonyms, context) and win with enough labels or when meaning matters beyond keywords — at 10–1000× the inference cost.

Highly nonlinear data

Choice depends on the data type. Tabular with interactions/thresholds → GBMs. Images/audio/text → deep networks (the nonlinearity is hierarchical/perceptual). Low-dimensional smooth nonlinear functions with little data → kernel methods (RBF SVM, Gaussian processes, which also give uncertainty), or splines/GAMs. KNN for low-dim problems with lots of data.

Why: "nonlinear" is not a model choice by itself — the *structure* of the nonlinearity (thresholds, smooth curves, compositional hierarchies, local similarity) decides which inductive bias fits.

Interpretability required

Clarify what kind: global (how the model works overall), local (why this decision), regulatory (adverse-action reasons, auditability), or debugging.

Choice: logistic regression / scorecards (coefficients as explanations), **GAMs** such as explainable boosting machines (additive shape functions per feature — accurate *and* inspectable), shallow decision trees or rule lists, or GBMs with **monotonic constraints** plus SHAP for local explanations (acceptable in some regulated settings). Post-hoc explanations describe the model, not causality, and can be unstable for correlated features.

Trade-off: often a small accuracy cost; quantify it — if a GAM is within 0.5% AUC of the GBM, interpretability wins.

< 10 ms inference requirement

Choice: linear models; small GBMs (hundreds of shallow trees, compiled with treelite or similar); precomputed/cached predictions or embeddings; for deep models — distillation into a small student, quantization (int8), pruning, and batching on GPU where throughput allows. In ranking, the budget is often **per request for hundreds of candidates** → a cheap first-stage model and an expensive model only on the top few.

Avoid: brute-force KNN over large sets, large transformers on CPU, big random forests (many deep trees), models needing slow feature lookups — features often dominate latency, not the model.

Severe class imbalance

Model family matters less than setup. GBMs or logistic regression with class weights or natural distribution + threshold tuning; focal loss for deep models with many easy negatives; anomaly detection if positives are ultra-rare or keep changing. Evaluate with PR-AUC / recall at fixed FPR; calibrate. Invest in getting more positive labels (active learning).

Limited memory (edge / on-device)

Choice: linear models with hashed features; small trees; compact neural nets (depthwise separable convolutions, MobileNet-style) with int8 quantization and pruning; distillation from a large server model. Avoid KNN (stores all data), large forests, big embedding tables (use hashing or shared/compositional embeddings).

Large categorical feature space

Choice: CatBoost or LightGBM with native categorical handling / ordered target statistics; hashing trick + linear models (with learned crosses) at extreme scale; **embeddings** in neural models (Wide & Deep,

DeepFM, DCN) when there are millions of IDs and enough data per ID; factorization machines for sparse pairwise interactions. Never plain one-hot into a tree model with 10^6 levels. Plan for unseen categories at serving time.

26.3 Two models, identical accuracy — which to deploy?



Real-life analogy: two batters, same average

Think of it like selectors choosing between two batters who both average 45. Is the tie real, or a few lucky innings (confidence intervals)? Who scores against quality pace and in tight chases (the right metric, the hard slices)? Who is in form this season, not five years ago (out-of-time robustness)? Who is fitter and cheaper to keep (operational cost)? If they fail in different matches, maybe pick both (ensemble). Still tied? Play them in a trial match (A/B test).

Where it breaks: a batter's weak spots show up live on TV; a model's weaknesses stay hidden until you deliberately slice its errors.

"Two models have identical accuracy. How would you decide which to deploy?"

Identical *average accuracy* is one number; deployment is a multi-dimensional decision. I'd compare:

1. **Is the tie real?** Confidence intervals / paired test on the same examples (McNemar); maybe one is genuinely better.
2. **The right metric at the operating point:** precision/recall at our threshold, calibration (log loss, ECE), ranking metrics — equal accuracy can hide different trade-offs.
3. **Slices and fairness:** per-segment performance, worst-group error, tail behavior.
4. **Robustness:** performance on the most recent data (out-of-time), under simulated drift, stability across retrains/seeds, sensitivity to missing features.
5. **Error overlap:** if they make different errors, an ensemble might beat both.
6. **Operational cost:** latency p99, memory, serving cost, training time, feature dependencies (a model that needs a fragile real-time feature is riskier).
7. **Maintainability & interpretability:** simpler model, fewer features, easier debugging and explanation.
8. **Online test:** if still tied, A/B test — business metrics may differ where accuracy doesn't.

Default: prefer the simpler, cheaper, more robust, better-calibrated model (Occam's razor with an operations lens).

26.4 Prompt, RAG, fine-tune or build? Choosing an LLM approach (with cost math)

Walk up this ladder and stop at the first rung that meets the requirements (Part 43 has the mechanisms):

Rung	When it's enough	Typical cost profile
1. **Prompting** (instructions + few-shot examples)	task is general; a strong model already knows how	zero setup; highest per-call cost if prompts are long

Rung	When it's enough	Typical cost profile
2. **RAG**	the gap is **knowledge** — private, fresh, citable facts	index + retrieval infra; moderate per-call tokens
3. **Fine-tuning (LoRA)**	the gap is **behaviour** — format, style, domain-specific judgement, or a small model must match a large one	training cost + eval; cheaper, faster calls afterwards
4. **Distillation to a small model**	very high volume of a narrow task	one-time labeling by a large model; tiny serving cost
5. **Pretraining / continued pretraining**	a new language or domain the models barely know, with huge unlabeled data	very expensive; rarely justified

Worked cost comparison — ticket classification, 2M tickets/month

Option A — large model with a 1,500-token prompt (instructions + 10 examples) + 200-token ticket, 10 output tokens. Tokens/month $= 2\text{M} \times 1{,}710 \approx 3.4\text{B}$. At illustrative \$1/M input: **≈ \$3,400/month**, latency ~1 s.

Option B — fine-tuned small model (no examples needed in the prompt: 50-token instruction + 200-token ticket). One-time: 20K labels generated by the large model (≈ \$40) + human audit + LoRA training (a few GPU-hours). Serving: 2M × 250 tokens = 0.5B tokens; self-hosted on one small GPU instance ≈ **\$300–600/month**, latency ~50 ms.

Break-even: B wins after a couple of months *if* its accuracy on the eval set matches A. Add the hidden costs: maintaining a training pipeline, re-labeling when categories change, and monitoring drift. If categories change weekly, the prompt-based option may still be cheaper overall.

How to present the decision in an interview: state the gap (knowledge vs. behaviour), the volume and latency, how often requirements change, and the evaluation you'd use to compare options — then pick the lowest rung that clears the bar, and name the trigger that would make you climb.



Real-life analogy: getting a new employee productive

Think of it like getting a new hire productive. First, give clear instructions and a few examples (prompting). If they need company facts, give them access to the handbook and let them look things up (RAG). If they need a new way of working, send them on a training course (fine-tuning). If one task is repeated thousands of times a day, write a checklist and hand it to a junior who does only that (distillation). Hiring and training someone from school (pretraining) is rarely needed.

Where it breaks: employees combine all these naturally; for LLM systems each rung adds infrastructure you must maintain, so the cheapest *total* option includes upkeep, not just per-call price.

Active recall — Set 15: System design, data-centric ML, model selection

Set 15 · Answer from memory, then check the Answer Key

Easy recall

1. Name the stages of a recommendation funnel and the metric for each.
2. What are the 20 steps of the system design framework (at least 12)?
3. What is uplift modeling and why does churn prediction need it?
4. What is the newsvendor quantile?

5. Define Cohen's kappa.

Conceptual

6. Why is calibration essential for ads CTR models?
7. What is reject inference in credit risk?
8. How does position bias distort click labels, and how can you correct it?
9. Why does demand forecasting need special care around stock-outs?
10. Why must duplicate detection use candidate generation?

Why

11. Why can better data beat a better model?
12. Why are label errors in the *test set* especially dangerous?
13. Why is a routing-engine ETA a good feature for an ML ETA model?
14. Why do spam and fraud models need frequent retraining?
15. Why prefer the simpler model when accuracy is tied?

Scenario

16. Design a system to recommend friends ("people you may know") — outline in 5 bullets.
17. Your moderation classifier over-removes posts in one dialect. What do you do?
18. Only confirmed frauds are labeled; everything else is "unknown". How do you train?
19. You must classify support tickets in < 5 ms with 2,000 labeled examples. Plan?
20. A retail forecast is unbiased overall but systematically low during promotions. Fix?

Mathematical

21. Under/overstock costs are $9 and $1 per unit. Which quantile should you forecast?
22. Two annotators: observed agreement 0.9; each labels 90% of items "negative". Compute κ.
23. Your two-tower retrieval returns 500 candidates; the ranker's ideal top-10 contains 7 of them on average. What's recall@500 in this sense, and why does it matter?

Difficult follow-ups

24. How would you estimate prevalence of violating content independently of your classifier?
25. Your ranking model improves NDCG offline but decreases CTR online. Give five hypotheses.

2027 additions

26. Name three risks of training on synthetic data from a stronger model and how to mitigate them.
27. 2M tickets/month: compare the monthly cost of a 1,700-token few-shot prompt at $1/M tokens with a fine-tuned small model.
28. In a RAG design, why must permissions be enforced in retrieval rather than in the prompt?

Unit IX · new in 2027

Modern ML + GenAI

Part 41 — Transformers & Attention · Part 42 — Language Models
Part 43 — Adapting & Aligning Models · Part 44 — Retrieval, RAG & Agents
Part 45 — Evaluating LLM Systems & Safety · Part 46 — Efficient Training & Inference
Part 47 — Recommender Systems & Bandits · Part 48 — Responsible AI
Part 49 — Other Frontiers · Part 50 — The 2027 Interview Landscape

In 2026–27 loops, GenAI questions moved out of specialist roles and into general ML interviews. Same first principles — new objects to apply them to.

PART 41

Transformers & Attention

The transformer in one breath

Every token looks at every other token, decides how relevant each one is (a softmax over dot products), and takes a weighted average of their information. Stack that "look and mix" step with a per-token MLP, wrap both in residual connections and normalization, repeat dozens of times. Everything else — positions, heads, caches, experts — is engineering around that one operation.

The whole Part as one picture

Think of it like an orchestra rehearsing a new piece, where each musician stands for one word. In every round, each player listens across the hall, pays most attention to the players whose parts matter for theirs, and adjusts their own playing with a blend of what they heard (attention). Then each practises alone for a moment (the MLP). Nobody throws away what they already had; they only add adjustments (residual connections). After dozens of rounds, every player's part fits the whole piece.

Where it breaks: musicians know where everyone sits and follow a written order; the model hears everyone at once with no sense of order, unless position information is added.

41.1 Why attention replaced recurrence

RNNs (Part 20) process tokens one at a time, carrying a fixed-size hidden state. Three problems follow from that design:

1. **Sequential computation:** step t needs step $t-1$, so training can't be parallelized across the sequence — GPUs sit idle.
2. **Long-range dependencies:** information from token 1 reaches token 500 only through 499 multiplications by recurrent Jacobians — it vanishes or explodes (Part 14), gates notwithstanding.
3. **Bottleneck:** everything the model knows about the past must fit in one vector.

Attention gives every position a **direct, content-based connection** to every other position: path length 1 between any two tokens, and all positions computed in parallel. The cost: $O(T^2)$ pairwise interactions.

Real-life analogy: whisper chain versus group chat

Think of it like passing news along a line of 500 people by whispering (an RNN). Each person must wait for the one before (no parallel work), the message gets garbled by the end (vanishing signals), and each person can pass on only one short sentence (the fixed-size state). Attention is a group chat: everyone reads everyone's message directly, all at the same time. The catch: with T people each reading all T messages, that's about T^2 reads.

Where it breaks: a group chat shows messages in order; attention does not, so word order has to be added separately.

41.2 Scaled dot-product attention

$$\text{Attention}(Q,K,V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}} + M\Big)V, \qquad Q = XW_Q,\ K = XW_K,\ V = XW_V$$

Real-life analogy: finding the right people at a party

Think of it like walking into a party with a question (the query). Everyone wears a badge saying what they know about (the key) and has something to tell you (the value). You check how well each badge matches your question, turn the match scores into shares of your attention that add up to 100%, and leave with a blend of what people said, mostly from the best matches. The mask means you may not talk to guests who haven't arrived yet (future tokens).

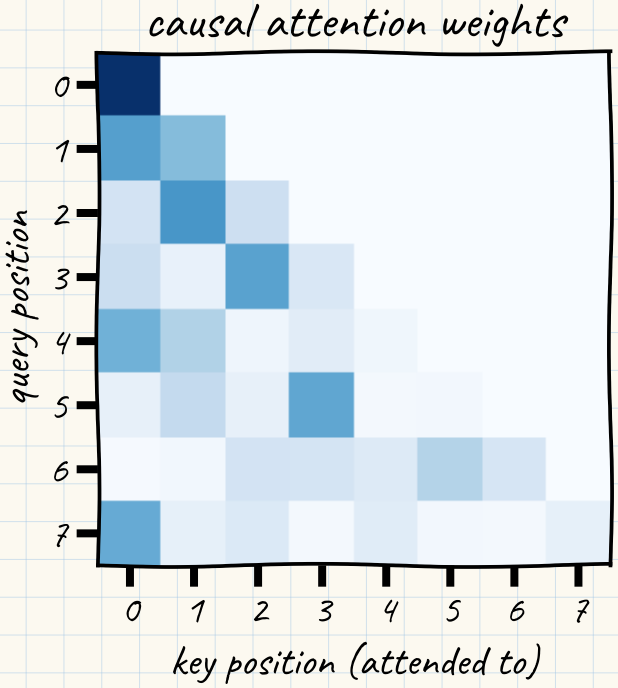
Formula, read aloud: $\text{softmax}(QK^\top/\sqrt{d_k})V$ says: output = (question-badge match scores, shrunk by $\sqrt{d_k}$ so long badges don't inflate them, turned into percentages) × (what each person has to say).

Where it breaks: at a party only you ask; in attention every token asks, wears a badge and answers at once, and all three roles are learned.

Intuition — a soft dictionary lookup. Each token emits a **query** ("what am I looking for?"), a **key** ("what do I contain?") and a **value** ("what I'll hand over if you pick me"). The query–key dot product scores relevance; softmax turns scores into weights; the output is the weighted average of values. A hard dictionary returns the one value whose key matches; attention returns a blend weighted by similarity.

The details interviewers probe.

- **Why $\sqrt{d_k}$:** with unit-variance components, $q^\top k$ has variance d_k; unscaled logits grow with dimension, softmax saturates toward one-hot, and its gradients vanish (Part 2).
- **Masks M:** $-\infty$ (in practice a large negative number) where attention is not allowed — **causal** masks for autoregressive decoders (token t can't see $t+1, \ldots$), **padding** masks for batched variable-length sequences.
- **Cost:** computing $QK^\top$ is $O(T^2 d)$ time and the attention matrix is $O(T^2)$ memory per head — the reason long contexts are expensive.
- **Permutation equivariance:** without positional information, shuffling the input tokens just shuffles the outputs — attention sees a *set*, not a sequence.



A causal attention matrix: each row is one query's weights over the keys it may see

41.3 Multi-head attention

Run h attention operations in parallel on lower-dimensional projections ($d_{\text{head}} = d/h$), concatenate, project back:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O, \qquad \text{head}_i = \text{Attention}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)})$$

Real-life analogy: a team of reporters

Think of it like a newsroom sending several reporters to the same event, each with a different beat: one tracks who said what, one tracks places, one tracks the timeline. Each carries a smaller notebook (lower-dimensional heads). Their notes are stapled together and an editor rewrites them into one article (concatenate, then W_O). One reporter can follow only one thread well; several follow many threads for about the same total effort.

Formula, read aloud: $\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$ says: run h small attentions side by side, join their outputs, and mix them with one final projection.

Where it breaks: reporters are assigned beats; heads are never told what to track, they learn it, and many end up doing similar jobs.

Why several heads? One softmax produces one weighting per query — it can attend strongly to only a few places at once. Several heads can track different relations simultaneously (syntax, coreference, position, copying), each in its own subspace. The total compute is about the same as one full-width head. **Parameters:** W_Q, W_K, W_V, W_O are each $d \times d \Rightarrow 4d^2$ per layer.

41.4 The transformer block and the residual stream

Modern decoder blocks (pre-LN):

$$x \leftarrow x + \mathrm{MHA}(\mathrm{Norm}(x)), \qquad x \leftarrow x + \mathrm{FFN}(\mathrm{Norm}(x)), \qquad \mathrm{FFN}(x) = W_2\,\phi(W_1 x)$$

Real-life analogy: a factory conveyor belt

Think of it like each word riding a conveyor belt past a long line of workstations. At each station, a worker looks at a straightened copy of the item (normalization) and builds an add-on. Some stations fetch useful parts from neighbouring belts (attention: moving information *between* tokens); others work on this item alone (the FFN: working *within* a token). Crucially, stations only bolt things on; nothing already on the belt is removed (the residual "+").

Formula, read aloud: $x \leftarrow x + \mathrm{MHA}(\mathrm{Norm}(x))$ says: item after the station = item before + the add-on the station built from a straightened copy.

Where it breaks: bolted-on parts stay separate; here add-ons are summed into one vector, so a later station can effectively cancel an earlier one by adding its opposite.

- **The residual stream view:** each token carries a vector that every layer *reads from and writes to* additively. Attention moves information **between** tokens; the FFN transforms information **within** a token (and is where much factual association is thought to be stored).
- **FFN width:** classically $4d$ with GELU; many modern models use **SwiGLU**, $W_2(\mathrm{SiLU}(W_1 x) \odot W_3 x)$, with hidden width $\approx \frac{8}{3}d$ to keep parameters comparable.
- **Norms:** RMSNorm (no mean subtraction) is common in recent LLMs; pre-LN keeps an unnormalized identity path through the whole network, which trains stably at depth (Part 20).

Parameter count of a transformer — do it in your head

Per layer: attention $4d^2$ + MLP $2 \cdot d \cdot 4d = 8d^2$ ⇒ $\approx 12d^2$ (ignoring norms and biases). Total $\approx 12\,L\,d^2 + V\,d$ (embeddings, often tied with the output layer).

Check with GPT-2 small ($L = 12$, $d = 768$, $V = 50{,}257$): $12 \times 12 \times 768^2 \approx 84.9$M, embeddings $50{,}257 \times 768 \approx 38.6$M, positions $1024 \times 768 \approx 0.8$M ⇒ **≈124M** — its published size.

41.5 Encoder, decoder, encoder–decoder

Architecture	Attention	Pretraining objective	Typical use
Encoder-only (BERT family)	bidirectional (every token sees all)	masked language modeling	embeddings, classification, retrieval, NER
Decoder-only (GPT family, most LLMs)	causal	next-token prediction	generation, chat, general-purpose LLMs
Encoder–decoder (T5, original Transformer)	bidirectional encoder + causal decoder with **cross-attention** to encoder outputs	span corruption / seq2seq	translation, summarization, structured transduction

Why decoder-only won for general models: one simple objective that uses every token as a training signal, scales cleanly, and turns every task into "continue the text"; with enough scale it handles understanding tasks too. Encoders remain the workhorse for **embeddings and cheap classification** — faster and bidirectional.

Real-life analogy: examiner, storyteller, translator

Think of it like three jobs with text. An **encoder** is an examiner who reads the whole answer, front to back and back to front, before judging it: great for grading and sorting (classification, embeddings). A **decoder** is a storyteller improvising one word at a time, never peeking ahead: great for generating. An **encoder–decoder** is a translator who first reads the full Hindi sentence, then writes the English one word by word while glancing back at the original (cross-attention).

Where it breaks: a big enough storyteller also turns out to be a decent examiner, which is why decoder-only models now handle many understanding tasks too.

41.6 Positional information

Method	Idea	Notes
Sinusoidal (original)	add fixed sin/cos features of position	no parameters; extrapolates poorly in practice
Learned absolute	add a learned vector per position	capped at the trained length
Relative bias (T5)	add a learned bias per relative distance to attention logits	length-generalizes better
RoPE	**rotate** each pair of query/key dimensions by an angle proportional to position	the $q \cdot k$ product depends only on the **relative** offset; standard in modern LLMs
ALiBi	subtract a head-specific linear penalty × distance from logits	simple, extrapolates to longer contexts

RoPE in one line: for pair i with frequency $\theta_i = 10000^{-2i/d}$, rotate (x_{2i}, x_{2i+1}) by angle $m\theta_i$ at position m. Since rotations compose, $\langle R_m q, R_n k\rangle = \langle q, R_{n-m} k\rangle$ — relative position falls out of the dot product. **Context extension** (training at 4K, serving at 128K) rescales these frequencies (position interpolation and its refinements) plus some long-context fine-tuning.

Real-life analogy: clock hands turned by seat number

Think of it like giving each word a set of clock hands, turned by an amount set by its seat number. Without positions, attention sees a bag of words: "dog bites man" looks like "man bites dog". With RoPE, when two words compare notes, only the *difference* between their hand angles matters, so "3 seats apart" feels the same anywhere in the sentence. Fast hands (like seconds) track nearby order; slow hands (like hours) track long distances.

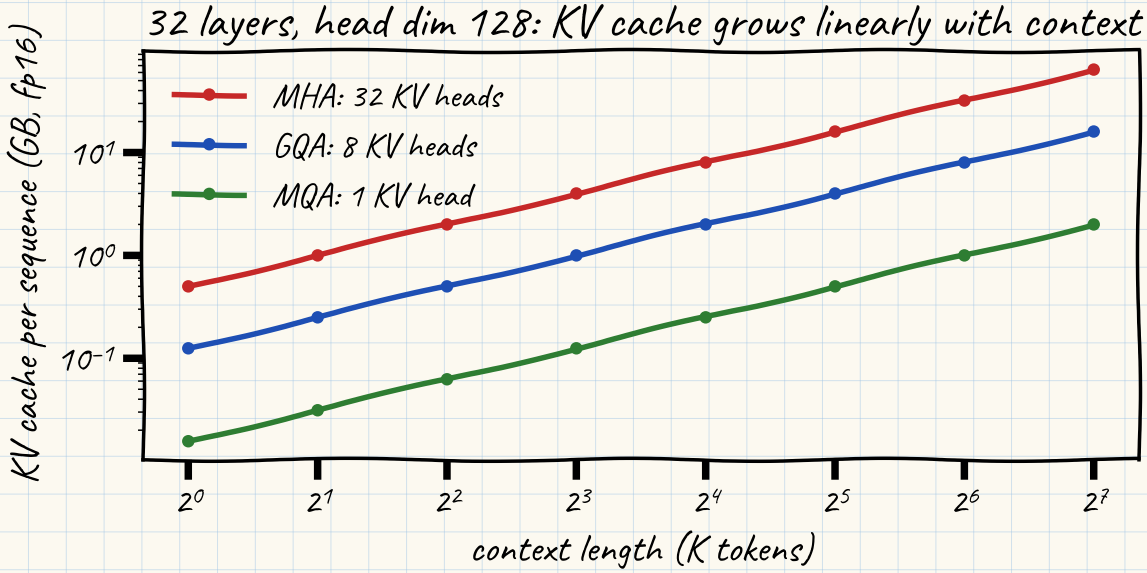
Formula, read aloud: $\langle R_m q, R_n k\rangle = \langle q, R_{n-m} k\rangle$ says: turning the query by m and the key by n equals turning only the key by the gap $n - m$.

Where it breaks: far beyond the trained length, the slow hands reach angles the model never saw, which is why context extension rescales the frequencies.

41.7 Making attention cheaper

KV sharing across heads. At inference, the keys and values of all past tokens are cached (the **KV cache**, Part 46). Its size is proportional to the number of **KV heads**:

Variant	KV heads	KV cache	Quality
Multi-head (MHA)	h	1×	baseline
Grouped-query (GQA)	g groups ($1 < g < h$)	g/h	near-MHA; the common default
Multi-query (MQA)	1	$1/h$	some quality loss
Multi-head latent (MLA)	compress K,V into a small shared latent, decompress per head	much smaller	used in some recent MoE LLMs



KV cache per sequence for a 32-layer model: grouped-query attention cuts it 4×

FlashAttention — exact attention, fewer memory trips. The bottleneck of naive attention on GPUs is reading and writing the $T \times T$ matrix to slow memory (HBM), not the FLOPs. FlashAttention tiles Q, K, V into blocks that fit in fast on-chip memory and never materializes the full matrix, using an **online softmax**: process key blocks one at a time, keeping for each query a running max m and running sum ℓ:

$$m' = \max(m,\ \max_j s_j), \qquad \ell' = e^{m-m'}\ell + \sum_j e^{s_j - m'}, \qquad o' = \frac{e^{m-m'}\ell\, o + \sum_j e^{s_j-m'} v_j}{\ell'}$$

Real-life analogy: a cook who avoids the storeroom

Think of it like a kitchen where chopping is fast but every trip to the far storeroom (slow GPU memory) is slow. FlashAttention brings ingredients in small baskets that fit on the counter (fast on-chip memory), finishes each basket fully, and keeps a running tally, so the whole pile is never laid out at once. Sharing KV heads (GQA) is like several cooks sharing one tray of prepped ingredients: less counter space, almost the same food.

Formula, read aloud: $\ell' = e^{m-m'}\ell + \sum_j e^{s_j-m'}$ says: new running total = old total rescaled to the new highest score + this block's contributions.

Where it breaks: baskets save walking, not chopping; FlashAttention still does $O(T^2)$ work, it just avoids moving the big matrix around.

Result: **identical outputs**, memory $O(T)$ instead of $O(T^2)$ for the attention matrix, and large speedups. It changes the cost constant, not the $O(T^2)$ compute.

Sub-quadratic alternatives: sliding-window/local attention (each token sees the last w tokens), sparse patterns, linear attention, and **state-space models** (e.g. Mamba) that process sequences in linear time with a recurrent state. Several recent models are **hybrids** that interleave a few full-attention layers with cheaper layers.

41.8 Mixture of Experts (MoE)

Replace the dense FFN with E expert FFNs and a **router** that sends each token to its top-k experts (often $k = 1$ or 2):

$$y = \sum_{e \in \text{top-}k(g(x))} g_e(x)\,\text{FFN}_e(x), \qquad g(x) = \text{softmax}(W_r x)$$



Real-life analogy: a hospital reception desk

Think of it like a big hospital with many specialists. A receptionist (the router) sends each patient to the two most suitable doctors (top-k experts) and weighs their advice by how confident she is in each choice. The hospital holds huge total expertise, yet each patient uses only two doctors' time (compute scales with k). But every specialist still needs a room and a salary (all experts sit in memory). If she sends everyone to the cardiologist, queues explode, so she is nudged to spread patients evenly (load balancing).

Formula, read aloud: $y = \sum_{e \in \text{top-}k} g_e(x)\,\text{FFN}_e(x)$ says: output = each chosen expert's answer × the router's trust in it, added up.

Where it breaks: MoE experts are picked per token and per layer, and rarely match topics a human would name.

- **Why:** parameters (capacity, knowledge) scale with E, but compute per token scales with k. Mixtral 8×7B, for example, has ~47B total parameters but ~13B active per token.
- **Load balancing:** without an auxiliary loss (or bias adjustment), the router collapses onto a few experts; balancing terms push toward uniform expert usage. A **capacity factor** caps tokens per expert; overflow tokens are dropped or rerouted.
- **Costs:** all experts must sit in memory (memory ≈ total params); **expert parallelism** needs all-to-all communication; fine-tuning can be less stable; serving small batches wastes the sparsity.

41.9 Transformers beyond text

Vision Transformer (ViT): cut an image into 16×16 patches, linearly embed each as a token, add positions, run an encoder. Weaker inductive bias than CNNs ⇒ needs more data or pretraining (Part 1). **Multimodal LLMs:** a vision (or audio) encoder produces embeddings, a projector maps them into the LLM's token space, and the LLM attends over image and text tokens together. **Other domains:** user-behavior sequences in recommendation (Part 47), time series, protein and molecule sequences.

Real-life analogy: turning a photo into jigsaw words

Think of it like cutting a photo into jigsaw squares and treating each square as a "word", numbered by its place on the board. The transformer then reads the squares like a sentence, letting every piece look at every other piece. For multimodal models, an interpreter (the projector) turns picture pieces into the same "language" the LLM already reads, so words and image pieces sit in one conversation.

Where it breaks: a CNN knows from the start that neighbouring pixels belong together; a ViT must learn even that from data, which is why it needs more data or pretraining.

41.10 Code: attention, multi-head attention with GQA, KV cache, RoPE

```
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def causal_mask(T):
    return np.tril(np.ones((T, T), dtype=bool))          # True = allowed (key ⩽ query)

def attention(Q, K, V, mask=None):
    d = Q.shape[-1]
    scores = Q @ np.swapaxes(K, -1, -2) / np.sqrt(d)     # (..., Tq, Tk)
    if mask is not None:
        scores = np.where(mask, scores, -1e30)            # large negative, not -inf (no NaN rows)
    w = softmax(scores, axis=-1)
    return w @ V, w

class MultiHeadAttention:
    def __init__(self, d_model, n_heads, n_kv_heads=None, seed=0):
        rng = np.random.default_rng(seed)
        self.h, self.hkv = n_heads, (n_kv_heads or n_heads)   # hkv < h  ⇒  grouped-query
attention
        self.dh = d_model // n_heads
        s = 1 / np.sqrt(d_model)
        self.Wq = rng.normal(0, s, (d_model, self.h * self.dh))
        self.Wk = rng.normal(0, s, (d_model, self.hkv * self.dh))
        self.Wv = rng.normal(0, s, (d_model, self.hkv * self.dh))
        self.Wo = rng.normal(0, s, (self.h * self.dh, d_model))

    def _split(self, X, n):                                   # (T, n*dh) → (n, T, dh)
        return X.reshape(X.shape[0], n, self.dh).transpose(1, 0, 2)

    def __call__(self, X, causal=True):
        T, rep = X.shape[0], self.h // self.hkv
        q = self._split(X @ self.Wq, self.h)
        k = np.repeat(self._split(X @ self.Wk, self.hkv), rep, axis=0)   # share KV across groups
        v = np.repeat(self._split(X @ self.Wv, self.hkv), rep, axis=0)
        out, w = attention(q, k, v, causal_mask(T) if causal else None)
        return out.transpose(1, 0, 2).reshape(T, -1) @ self.Wo, w

    def decode_with_cache(self, X):
        """Generate-style: one token at a time, appending K,V to a cache."""
        cache_k, cache_v, outs, rep = [], [], [], self.h // self.hkv
        for t in range(X.shape[0]):
            x = X[t:t + 1]
            q = self._split(x @ self.Wq, self.h)
```

```
            cache_k.append(self._split(x @ self.Wk, self.hkv))    # only the NEW token's K, V
            cache_v.append(self._split(x @ self.Wv, self.hkv))
            k = np.repeat(np.concatenate(cache_k, axis=1), rep, axis=0)
            v = np.repeat(np.concatenate(cache_v, axis=1), rep, axis=0)
            o, _ = attention(q, k, v)                                  # no mask needed: cache = past
only
            outs.append(o.transpose(1, 0, 2).reshape(1, -1) @ self.Wo)
        return np.concatenate(outs, axis=0)

def rope(x, positions, base=10000.0):
    T, d = x.shape
    theta = base ** (-np.arange(0, d, 2) / d)
    ang = positions[:, None] * theta[None, :]
    cos, sin = np.cos(ang), np.sin(ang)
    out = np.empty_like(x)
    out[:, 0::2] = x[:, 0::2] * cos - x[:, 1::2] * sin
    out[:, 1::2] = x[:, 0::2] * sin + x[:, 1::2] * cos
    return out

def rms_norm(x, gamma, eps=1e-6):
    return gamma * x / np.sqrt((x ** 2).mean(-1, keepdims=True) + eps)
```

Checked: (1) perturbing tokens 5–6 leaves the outputs for tokens 1–4 unchanged (the mask is causal); (2) every attention row sums to 1; (3) `decode_with_cache` reproduces the full causal forward pass exactly, for MHA and for GQA with 2 KV heads; (4) RoPE: $\langle R_5 q, R_2 k\rangle = \langle R_{13} q, R_{10} k\rangle$ (only the offset matters) and vector norms are preserved.



Real-life analogy: a cricket scorer's scorebook

Think of it like a cricket scorer. To know the score after the next ball, she doesn't replay the whole innings; she keeps a scorebook and adds one entry per ball. `decode_with_cache` does the same: for each new token it computes only that token's key and value, appends them to the cache, and lets the new query look back over the stored entries. Checking it against the full forward pass is like checking the scorebook total against a replay of every ball.

Where it breaks: a scorebook stays small; the KV cache grows with every token, layer and KV head, which is exactly why GQA and cache compression matter.

41.11 Interview questions on transformers

Q: "What is the computational bottleneck of a transformer, and how do people deal with it?"

Strong answer. "Two bottlenecks depending on the phase. Self-attention is $O(T^2 d)$ compute and $O(T^2)$ memory in sequence length, so long contexts are expensive in training and in prefill; FlashAttention removes the quadratic *memory* traffic by tiling with an online softmax, and sliding-window, sparse, linear-attention or state-space layers change the asymptotics at some quality cost. At generation time the bottleneck is different: each new token reads all the weights and the whole KV cache from memory, so decoding is memory-bandwidth-bound; that's attacked with KV-cache reduction — GQA, MQA, latent attention, quantized caches — batching, and speculative decoding. For most model sizes at moderate context, the MLP FLOPs, not attention, dominate compute."

Q: "Why do transformers need positional encodings, and why is RoPE popular?"

Strong answer. "Attention is permutation-equivariant: it computes weighted averages over a set, so without positional information 'dog bites man' and 'man bites dog' look the same. RoPE injects position

by rotating query and key vectors by angles proportional to their positions; because rotations compose, the query–key dot product depends only on the relative offset, which gives relative-position behavior without extra parameters, works naturally with the KV cache, and can be stretched for longer contexts by rescaling the rotation frequencies."

Follow-ups that expose shallow answers

"Is attention an explanation of the model's decision?" — Not reliably. Attention weights show where information was mixed from in one layer, not what caused the output; values, later layers and residual paths all matter (Part 48).

"What happens to a transformer if you remove the residual connections?" — Deep stacks become hard to optimize (Part 20), and the pre-LN identity path that keeps training stable disappears.

"Encoder or decoder for a 20 ms search-query classifier?" — A small encoder (or a distilled one): bidirectional context, one forward pass, no generation; a decoder LLM is overkill in latency and cost.

"Does MoE reduce memory?" — No — it reduces compute per token. All experts must be resident, so memory scales with total parameters.

PART 42

Language Models: Tokens, Pretraining, Scaling & Decoding

An LLM in one breath

A language model is a classifier over the vocabulary, applied once per position: given the tokens so far, output a distribution over the next token. Train it with plain cross-entropy (Part 3) on trillions of tokens, and the "classifier" has to learn grammar, facts, and reasoning patterns to predict well. Everything interesting — tokenization, scaling, decoding, prompting — sits around that single objective.

The whole Part as one picture

Think of it like someone who has read an entire library and plays just one game: "guess the next word". To get good at it, they must pick up grammar, facts and patterns of reasoning, because all of these improve the guess. Tokenization decides how text is chopped into pieces for the game; scaling asks how big a brain and how much reading pay off best; decoding decides which guess to actually say out loud; prompting sets up the conversation so the right kind of guesses come out.

Where it breaks: the reader remembers no single book, only patterns, so it can confidently continue a sentence with something false that merely *sounds* likely.

42.1 The objective: next-token prediction

$$p(x_1, \dots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}), \qquad \mathcal{L} = -\frac{1}{T} \sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

Real-life analogy: a word-by-word guessing test

Think of it like a test where a paragraph is revealed one word at a time, and before each word you bet on what comes next. Put 90% on the right word and you lose only a little; put 1% on it and you lose a lot. Your score is your average "surprise" over every word. One paragraph of T words gives T questions, all marked at once (teacher forcing).

Formula, read aloud: $\mathcal{L} = -\frac{1}{T} \sum_t \log p_\theta(x_t \mid x_{<t})$ says: loss = the average, over all positions, of –log(the chance you gave to the word that actually came next).

Where it breaks: in the test you see the real word after each bet; when generating, the model must build on its *own* guesses, so early mistakes can snowball.

This is the chain rule of probability (Part 3) plus maximum likelihood. One sequence of length T gives T training examples (teacher forcing: every position predicts its successor in parallel, thanks to the causal mask).

Perplexity $= \exp(\mathcal{L})$: the effective number of equally likely choices per token. PPL 10 ≈ "as uncertain as a fair 10-sided die". Perplexity depends on the tokenizer (more, shorter tokens → lower per-token PPL), so compare models with different tokenizers using **bits per byte/character** instead.

42.2 Tokenization

Why subwords? Word-level vocabularies can't handle unseen words and explode in size; character-level sequences are long (attention cost $\propto T^2$) and each token carries little meaning. Subword units get the best of both: frequent words are single tokens, rare words split into meaningful pieces, and nothing is out-of-vocabulary.



Real-life analogy: a box of Lego pieces

Think of it like building with Lego. Whole pre-built models (word tokens) are quick but useless for anything new. Single tiny bricks (characters) can build anything but take forever. A good set has common chunks, like ready-made wheels and windows, plus small bricks for anything unusual (subwords). BPE builds that set by noticing which two pieces get snapped together most often, gluing them into one piece, and repeating.

Where it breaks: Lego chunks are designed by meaning; BPE glues by frequency alone, so it can split words at odd spots, and the model never sees the single bricks inside a chunk (hence the "strawberry" letter-counting trouble).

Method	How it builds the vocabulary	Used by
BPE	start from characters (or bytes); repeatedly merge the most frequent adjacent pair	GPT family, many LLMs (byte-level BPE: never fails on any input)
WordPiece	like BPE, but merges chosen by likelihood gain	BERT
Unigram LM (SentencePiece)	start large, prune tokens that least reduce corpus likelihood; probabilistic segmentation	T5, many multilingual models

```
from collections import Counter

def bpe_train(words, n_merges):
    """words: dict word → count. Returns the learned merges, in order."""
    vocab = {tuple(w) + ("</w>",): c for w, c in words.items()}
    merges = []
    for _ in range(n_merges):
        pairs = Counter()
        for sym, c in vocab.items():
            for a, b in zip(sym, sym[1:]):
                pairs[(a, b)] += c
        if not pairs:
            break
        best = max(pairs, key=pairs.get)                # most frequent adjacent pair
        merges.append(best)
        new_vocab = {}
        for sym, c in vocab.items():
            out, i = [], 0
            while i < len(sym):
                if i < len(sym) - 1 and (sym[i], sym[i + 1]) == best:
                    out.append(sym[i] + sym[i + 1]); i += 2
                else:
                    out.append(sym[i]); i += 1
            new_vocab[tuple(out)] = c
        vocab = new_vocab
    return merges

def bpe_encode(word, merges):
    sym = list(word) + ["</w>"]
    for a, b in merges:                                 # apply merges in learned order
        i, out = 0, []
        while i < len(sym):
            if i < len(sym) - 1 and sym[i] == a and sym[i + 1] == b:
```

```
                out.append(a + b); i += 2
            else:
                out.append(sym[i]); i += 1
        sym = out
    return sym
```

Checked: on {low: 5, lower: 2, newest: 6, widest: 3} the first merges are `e+s`, `es+t`, `est+</w>`, `l+o`, `lo+w`, and the unseen word "lowest" encodes as `['low', 'est</w>']` — composed from learned pieces.

Tokenization consequences interviewers love:

- **"Why can't the model count the r's in 'strawberry'?"** — It never sees letters; it sees a few subword IDs. Character-level facts must be memorized per token.
- **Arithmetic and numbers** tokenize inconsistently (e.g. "1234" as one or several tokens), which hurts digit-level algorithms; some tokenizers split numbers into single digits for this reason.
- **Multilingual cost:** languages under-represented in tokenizer training need more tokens per sentence ⇒ higher cost, shorter effective context, sometimes worse quality.
- **Vocabulary size trade-off:** bigger vocab ⇒ shorter sequences (cheaper attention, more text per context) but a larger embedding/softmax matrix and rarer tokens with less training signal each. Modern LLM vocabularies are typically tens of thousands to a few hundred thousand tokens.

42.3 Pretraining: data, compute and dynamics

Data pipeline (where most of the quality comes from): crawl → text extraction → language ID → **deduplication** (exact hashing + near-duplicate MinHash-LSH; duplicates waste compute and increase memorization) → **quality filtering** (heuristics, classifier scores trained on "good" reference text) → safety/PII filtering → **decontamination** against evaluation benchmarks → **mixture weighting** across sources (web, code, books, math, multilingual) → often a late-stage "annealing" on the highest-quality data.

Compute: training costs about $6ND$ **FLOPs** for N parameters and D tokens (≈2ND forward + ≈4ND backward). Example: $N = 7 \times 10^9$, $D = 2 \times 10^{12} \Rightarrow 8.4 \times 10^{22}$ FLOPs. At an assumed 40% utilization of hardware delivering 10^{15} FLOP/s per accelerator, that's $\frac{8.4 \times 10^{22}}{0.4 \times 10^{15}} \approx 2.1 \times 10^8$ accelerator-seconds ≈ 58,000 accelerator-hours.



Real-life analogy: a reading list for a prodigy

Think of it like a parent preparing books for a child who learns by reading everything given. They throw out duplicate copies (rereading wastes time and encourages parroting), junk flyers (quality filtering), private letters (PII), and the actual exam papers (decontamination). They balance subjects (mixture weights) and save the best textbooks for the final weeks (annealing). Reading is expensive: the effort grows with both the child's "brain size" and the number of pages.

Formula, read aloud: $6ND$ says: training compute ≈ 6 × (number of parameters) × (number of tokens), about 2 for reading forward and 4 for learning backward.

Where it breaks: a child skims boring pages; a dense model spends the same compute on every token, good or bad, so filtering before training is what matters.

Training dynamics: warmup then cosine or warmup–stable–decay schedules (Part 14); AdamW with $\beta_2 \approx 0.95$ is common for stability; **loss spikes** (from bad batches or optimizer instabilities) handled with gradient clipping, skipping batches, rewinding to a checkpoint, lowering the LR, or architectural fixes (QK-normalization, z-loss on logits). Newer optimizers — for example Muon, which orthogonalizes each weight matrix's update — have been reported as more compute-efficient than AdamW in some large-scale runs; expect interviewers to ask *why* an optimizer helps, not to require a specific one.

42.4 Scaling laws

Empirically, loss falls as a **power law** in parameters N, data D and compute C over many orders of magnitude. The Chinchilla analysis (Hoffmann et al., 2022) fit

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}, \qquad E \approx 1.69,\ A \approx 406.4,\ B \approx 410.7,\ \alpha \approx 0.34,\ \beta \approx 0.28$$

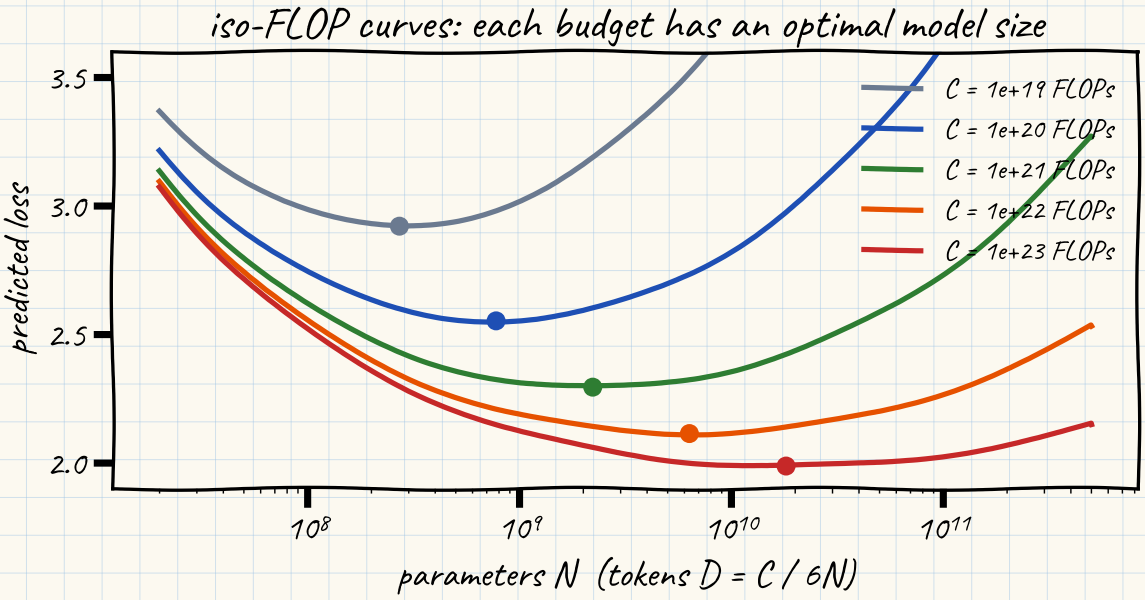
and minimized it subject to $C = 6ND$: **parameters and tokens should grow roughly in proportion**, about **20 tokens per parameter** at the compute-optimum.

Real-life analogy: a farmer's fixed budget

Think of it like a farmer with a fixed budget to split between land (parameters N) and seed and water (data D). Lots of land with little seed leaves fields empty; lots of seed on a tiny plot is wasted. The best harvest for the money grows both together (about 20 tokens per parameter). Some loss comes from the weather, which no farmer can beat (the floor E). And a farm you must run for years may be better small and densely planted (smaller models trained longer).

Formula, read aloud: $L = E + A/N^{\alpha} + B/D^{\beta}$ says: final loss = unbeatable floor + a penalty for too small a model + a penalty for too little data, each penalty shrinking as its quantity grows.

Where it breaks: a harvest is weighed directly; scaling laws are fitted curves whose constants change with the setup.



Iso-FLOP curves from the Chinchilla fit: every compute budget has an optimal model size

How to talk about it in 2027:

- E is the irreducible loss (entropy of text); the other terms are approximation-like (finite N) and estimation-like (finite D) error (Part 1).
- **Compute-optimal ≠ deployment-optimal.** Inference cost scales with N forever, so production models are routinely trained on far more than 20 tokens per parameter — a smaller model that's cheaper to serve, bought with extra training compute.
- The exact exponents are contested: an independent replication (Besiroglu et al., 2024) re-fit the Chinchilla data and obtained different constants. Treat scaling laws as **planning tools fitted per setup**, not constants of nature.

- **Data constraints:** high-quality text is finite; repeating data a few epochs is roughly fine, many epochs degrades — hence synthetic data, better filtering, and multimodal data.
- **"Emergent abilities"** that appear suddenly at scale are partly an artifact of discontinuous metrics (exact match); with smooth metrics many look gradual.

42.5 Decoding

The model outputs a distribution; decoding decides what to do with it.

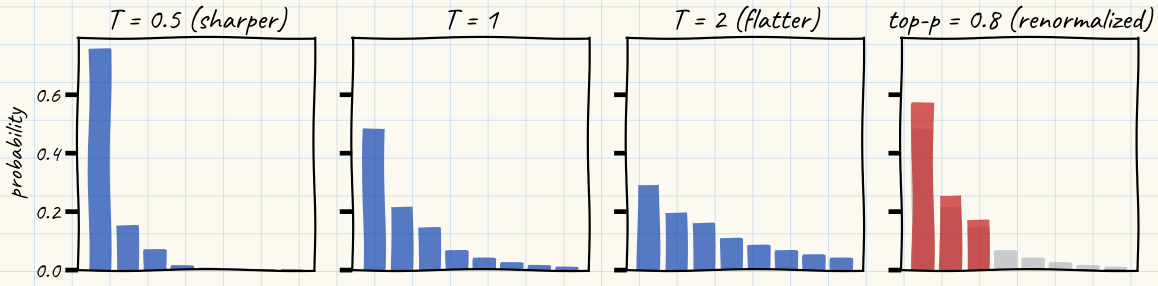
Real-life analogy: ordering from a friend's picks

Think of it like ordering at a restaurant where a friend scores each dish on "you'll like it". Greedy: always order the top dish. Top-k: pick among the top 5. Top-p: pick among the fewest dishes that together hold 90% of her votes, so the list is short when she's sure and longer when she isn't. Temperature is your mood: low sticks to favourites, high feels adventurous. Beam search plans the whole three-course meal, keeping the best few combinations.

Formula, read aloud: $\mathrm{softmax}(z/T)$ says: divide every score by the temperature before turning scores into chances; small T widens the gaps, large T flattens them.

Where it breaks: a menu doesn't change after you order; in text, each chosen word changes the scores for every word after it.

Strategy	Rule	Trade-off
Greedy	argmax each step	deterministic; repetitive; misses higher-probability *sequences*
Beam search	keep the B best partial sequences by total log-prob (length-normalized)	good for translation/summarization with one right answer; bland and repetitive for open-ended text
Temperature T	sample from $\mathrm{softmax}(z/T)$	$T<1$ sharper, $T>1$ more random, $T\to 0$ = greedy
Top-k	sample among the k most likely tokens	fixed k is wrong when the distribution is very peaked or very flat
Top-p (nucleus)	sample from the smallest set with cumulative probability $\geq p$	adapts the candidate set to the model's confidence
Repetition / frequency penalties	down-weight already-used tokens	reduce loops; can distort facts
Constrained decoding	mask tokens that would violate a grammar/JSON schema	guarantees parseable structured output



Temperature reshapes the whole distribution; top-p truncates the tail and renormalizes

```
def sample_next(logits, temperature=1.0, top_k=None, top_p=None, rng=np.random.default_rng(0)):
    if temperature == 0:
        return int(np.argmax(logits))                                    # greedy
```

```
    z = logits / temperature
    if top_k is not None:
        kth = np.sort(z)[-top_k]
        z = np.where(z < kth, -np.inf, z)
    p = softmax(z)
    if top_p is not None:                                    # nucleus sampling
        order = np.argsort(-p)
        cum = np.cumsum(p[order])
        keep = order[: np.searchsorted(cum, top_p) + 1]
        mask = np.zeros_like(p, dtype=bool); mask[keep] = True
        p = np.where(mask, p, 0.0); p /= p.sum()
    return int(rng.choice(len(p), p=p))

def beam_search(log_prob_fn, bos, eos, beam=3, max_len=10, alpha=0.6):
    """log_prob_fn(prefix) -> log-probs over vocab; final score = logp / len^alpha."""
    beams, done = [([bos], 0.0)], []
    for _ in range(max_len):
        cand = []
        for seq, lp in beams:
            lps = log_prob_fn(seq)
            for tok in np.argsort(-lps)[:beam]:
                cand.append((seq + [int(tok)], lp + lps[tok]))
        cand.sort(key=lambda c: c[1], reverse=True)
        beams = []
        for seq, lp in cand:
            (done if seq[-1] == eos else beams).append((seq, lp))
            if len(beams) == beam:
                break
        if not beams:
            break
    done += beams
    return max(done, key=lambda c: c[1] / (len(c[0]) - 1) ** alpha)
```

Checked: with logits [2, 1, 0.5, −1, −3], top-p = 0.7 only ever samples tokens 0 and 1 (softmax 0.607 + 0.223 ≥ 0.7); top-k = 1 equals greedy.

Why length normalization in beam search? Every added token multiplies in a probability < 1, so raw log-probability favors short outputs; dividing by length^{α} counteracts that.

42.6 In-context learning, prompting and reasoning

- **In-context learning:** the model adapts to examples in its prompt without weight updates. Performance is sensitive to example choice, order and format; few-shot examples mostly teach the *format and label space* as much as the mapping.
- **Chain-of-thought:** asking for intermediate steps improves multi-step problems — the model gets more serial computation (more tokens = more forward passes) and conditions later tokens on explicit intermediate results.
- **Self-consistency / best-of-n:** sample several solutions, take a majority vote or pick with a verifier. A simple form of **test-time compute scaling**.
- **Reasoning models:** trained with reinforcement learning on tasks with checkable answers (math, code) to produce long chains of thought before answering (Part 43). They trade latency and tokens for accuracy; the interview-relevant point is the new axis — **quality can be bought at inference time**, so cost/latency budgets become part of model choice.
- **Prompting is software:** version prompts, test them against eval sets, and treat changes like code changes (Part 45).



Real-life analogy: a new booking-counter clerk

Think of it like a new clerk at a railway booking counter who is shown three filled-in reservation forms and then fills the fourth the same way: no training course, just examples (in-context learning). For a fare with two connecting trains, they work it out on paper step by step instead of in their head (chain-of-thought). For a costly group booking, they total it three times and go with the answer that comes up most (self-consistency). Extra care costs extra time, which is the trade reasoning models make.

Where it breaks: the clerk still remembers tomorrow; the model forgets the examples as soon as the prompt ends, because its weights never changed.

42.7 Embedding models

Encoder models (or LLMs with a pooling head) trained **contrastively** (InfoNCE, Part 15) on query–document or paraphrase pairs map text to vectors where cosine similarity reflects meaning. They power retrieval (Part 44), clustering, deduplication, recommendation features and classification with few labels. Practical choices: mean vs. last-token pooling, max sequence length, normalization, dimension (some models are trained so truncated prefixes of the vector still work — useful for storage/latency trade-offs), and domain adaptation via fine-tuning on in-domain pairs with hard negatives.



Real-life analogy: PIN codes for meaning

Think of it like giving every sentence an address on a map of meaning, so sentences about similar things live in the same neighbourhood and closeness can be measured. Contrastive training is like teaching a postman with pairs: this question and its correct answer belong on the same street; random other letters belong far away. Some models work like PIN codes: the first few digits already give the region and more digits add precision, so a short prefix can be kept to save space.

Where it breaks: being neighbours means "about the same topic", not "saying the same thing": "the film was great" and "the film was not great" can sit very close.

Q: "Walk me through what happens when an LLM generates a response to a prompt."

Strong answer. "The prompt is tokenized into IDs and embedded. In the **prefill** phase the model runs one parallel forward pass over all prompt tokens, computing attention over the prompt and storing every layer's keys and values in the KV cache; the last position's logits give the distribution for the first output token. Then the **decode** loop: pick a token with the decoding strategy — greedy, temperature, top-p — append it, run a forward pass for just that one token, attending to the cached keys and values, and repeat until an end token or a length limit. Prefill is compute-bound and determines time-to-first-token; decode is memory-bandwidth-bound — every step reads all the weights and the growing cache — and determines tokens per second. That split is why serving systems batch many decode streams together and why KV-cache size matters so much."

Follow-ups

"Why does temperature 0 still sometimes give different outputs?" — Non-deterministic GPU kernels and batch composition change floating-point reduction order; ties or near-ties in logits flip. Determinism needs fixed batching and deterministic kernels.

"Perplexity went down but the chatbot got worse. How?" — Perplexity measures next-token prediction on some text distribution; user-facing quality depends on instruction following, helpfulness and safety, which alignment changes (often raising perplexity on raw web text). Evaluate the behavior you care about (Part 45).

"How would you pick a vocabulary size for a new multilingual model?" — Trade sequence length against embedding/softmax size; measure tokens-per-word across target languages (fairness of cost and context), training throughput, and downstream quality at fixed compute.

Active recall — Set 16: Transformers & language models

Set 16 · Answer from memory, then check the Answer Key

Easy recall

1. Write scaled dot-product attention.
2. What are the Q, K and V projections, intuitively?
3. What is the approximate parameter count per transformer layer?
4. Define perplexity.
5. What does the BPE algorithm do?

Conceptual

6. Why is attention permutation-equivariant, and how do positional encodings fix it?
7. How does RoPE encode relative position?
8. What does the KV cache store, and why does GQA shrink it?
9. What does FlashAttention change, and what does it not change?
10. Compare encoder-only, decoder-only and encoder–decoder models.

Why

11. Why divide attention logits by $\sqrt{d_k}$?
12. Why do multiple heads help?
13. Why is decoding memory-bandwidth-bound?
14. Why can't LLMs reliably count letters in a word?
15. Why are production models trained on more tokens than the Chinchilla optimum?

Scenario

16. Your model needs 4× longer context. What breaks and what are your options?
17. You must serve a 20 ms text classifier. Decoder LLM or encoder? Why?
18. Beam search outputs are short and bland for a story generator. What do you change?
19. A multilingual product costs 3× more for Hindi than English. Why, and what can be done?
20. Training loss spikes every few thousand steps on a 7B model. Hypotheses and fixes?

Mathematical

21. Estimate the training FLOPs for a 1B-parameter model on 20B tokens.
22. Show that $\langle R_m q, R_n k \rangle$ depends only on $m - n$ for 2-D rotations.
23. For logits $[2, 1, 0]$, compute the softmax at temperature 1 and 0.5.

Difficult follow-ups

24. How does the online softmax let FlashAttention process keys block by block without changing results?
25. MoE model: 8 experts, top-2 routing. Compare its memory and per-token compute with a dense model of the same total parameters.

PART 43

Adapting & Aligning Models

The adaptation ladder

Climb only as high as you must: **prompting → few-shot → RAG → parameter-efficient fine-tuning → full fine-tuning → continued pretraining → training from scratch**. Each rung costs more data, compute and maintenance. The key question is always the same: *is the gap about knowledge the model lacks (→ retrieval), or behavior it doesn't exhibit (→ fine-tuning)?*



The whole Part as one picture

Think of it like getting a bright new graduate ready for your company. First you give clear instructions (prompting) and a few examples (few-shot). If they lack facts, you hand them the company handbook to look things up (RAG). If they lack *habits* (your format, your tone), you send them on a short course (SFT, LoRA). To teach taste, a manager compares two of their drafts and says which is better (RLHF, DPO); for maths and code, an answer key does the grading (verifiable rewards). Training from scratch is like raising a new person from childhood: almost never worth it.

Where it breaks: a person who reads the handbook remembers facts reliably; a fine-tuned model stores facts diffusely and unreliably, which is why facts go to retrieval and habits go to fine-tuning.

43.1 Choosing how to adapt

Need	Best first tool	Why
Up-to-date or private facts, citations, access control	**RAG** (Part 44)	knowledge stays in documents you can update, cite and permission; fine-tuning bakes in facts poorly and they go stale
A format, tone, style, or task-specific behavior	prompting → **SFT / LoRA**	behavior is learned well from a few thousand good demonstrations
A narrow high-volume task (classification, extraction) at low cost	fine-tune a **small** model or **distill** a large one	cheaper, faster serving than prompting a large model
Preference alignment (helpfulness, safety, taste)	preference optimization (RLHF, DPO)	the target is a comparison, not a single correct output
Verifiable skills (math, code, tool use)	RL with verifiable rewards	reward from checkers, not humans
New domain language (e.g. legal, biomedical corpus)	continued pretraining (+ SFT)	shifts the model's base distribution

Real-life analogy: the new taxi driver

Think of it like hiring a skilled driver who is new to Mumbai. If they don't know the roads, you don't send them on a year-long course; you give them a map app that is always up to date (RAG fixes a *knowledge* gap). If they know the roads but brake harshly and never greet passengers, a map won't help; they need practice and feedback (fine-tuning fixes a *behaviour* gap). For one fixed daily airport run, a small car trained for that route is enough (a small fine-tuned or distilled model).

Where it breaks: a driver learns new roads permanently from a course; a model fine-tuned on facts recalls them unreliably and can't point to a source, so the "map app" is usually the better fix for facts.

43.2 Supervised fine-tuning (SFT)

Train on (prompt, ideal response) pairs with next-token cross-entropy, **masking the loss on prompt tokens** (only learn to produce the response). Use the model's **chat template** exactly — mismatched special tokens silently degrade behavior. Lessons that hold up: **quality and diversity beat quantity** (a few thousand excellent examples can steer a strong base model); deduplicate and decontaminate; watch for **catastrophic forgetting** of general skills (mix in general data, use lower learning rates, or use PEFT).

Real-life analogy: a book of solved answers

Think of it like a student preparing from a book of solved board-exam answers. The question is printed for them; they practise writing only the answer, so only the answer gets marked (the loss is masked on prompt tokens). Answers must follow the official answer-sheet layout exactly, or the examiner gets confused (the chat template). Fifty beautifully solved examples teach more than five hundred sloppy ones. But a student who drills one chapter for weeks starts forgetting the rest of the syllabus (catastrophic forgetting), so they mix in revision of old chapters.

Where it breaks: a student understands why an answer is right; SFT only makes the demonstrated answers more likely, token by token, so it copies the style and the errors of its examples.

43.3 Parameter-efficient fine-tuning (PEFT)

✎ LoRA

Freeze the pretrained weight $W \in \mathbb{R}^{d\times k}$ and learn a low-rank update:

$$h = xW + \frac{\alpha}{r}\,xAB, \qquad A \in \mathbb{R}^{d\times r},\ B \in \mathbb{R}^{r\times k},\ r \ll \min(d,k), \quad B \text{ initialized to } 0$$

Real-life analogy: a few sliders on a mixer

Think of it like re-tuning a recorded song. Instead of re-recording every instrument (retraining all of W), a sound engineer adds a small mixing desk with just a few sliders. Each slider changes many frequencies together in a coordinated way: a *low-rank* change. All sliders start at neutral, so the song first sounds exactly like the original ($B = 0$). A master knob sets how strong the whole adjustment is (α/r).

Formula, read aloud: $h = xW + \frac{\alpha}{r} xAB$ says: output = (what the frozen original layer gives) + (master knob) × (input squeezed down to r sliders by A, then spread back out by B).

Where it breaks: real sliders have names like "bass"; LoRA *learns* what each of its r directions means, and they are rarely human-readable.

- **Why it works:** the weight changes needed for adaptation appear to have low intrinsic rank; you only need to move the model within a small subspace.
- **Parameters:** $r(d + k)$ instead of dk. For $d = k = 4096$, $r = 16$: 131K vs 16.8M per matrix (0.8%).
- $B = 0$ **init** ⇒ the model starts exactly at the pretrained function; α/r scaling keeps the update magnitude stable when you change r.
- **Where:** attention projections, often all linear layers for best quality.
- **Serving:** merge $W' = W + \frac{\alpha}{r} AB$ for zero overhead, or keep adapters separate to serve many tasks from one base model (swap adapters per request).

```
class LoRALinear:
    def __init__(self, W, r=4, alpha=8, seed=0):
        rng = np.random.default_rng(seed)
        d_in, d_out = W.shape
        self.W = W                                              # frozen
        self.A = rng.normal(0, 1 / np.sqrt(d_in), (d_in, r))  # trainable
        self.B = np.zeros((r, d_out))                           # trainable, zero init ⇒ starts as W
        self.scale = alpha / r

    def __call__(self, x):
        return x @ self.W + self.scale * (x @ self.A) @ self.B   # never forms A@B while training

    def merged(self):
        return self.W + self.scale * self.A @ self.B         # fold in for zero-overhead inference

    def n_trainable(self):
        return self.A.size + self.B.size
```

Checked: output equals xW at initialization; after training (random B), the adapter path equals $x\,W_{\text{merged}}$; a 64×32 layer with $r = 4$ trains 384 parameters instead of 2,048.

✎ QLoRA and friends

QLoRA: keep the frozen base in **4-bit** (NF4 quantization, with quantization constants themselves quantized), dequantize on the fly in the forward pass, and train LoRA adapters in 16-bit. Memory for a 7B model drops from ~112 GB for full mixed-precision Adam fine-tuning (16 bytes/parameter, Part 46) to roughly the 4-bit weights (~3.5–4 GB) plus adapters, optimizer state for adapters and activations — single-GPU territory. **Other PEFT:** adapters (small bottleneck MLPs inserted in blocks), prefix/prompt tuning (learned virtual tokens), and LoRA variants that re-parameterize magnitude and direction.

Real-life analogy: a compressed album, sharp stickers

Think of it like keeping a huge photo album on your phone in a heavily compressed format to save space (the frozen base stored in 4-bit). Whenever you open a photo, the phone unpacks it on the fly (dequantizing in the forward pass). You never edit the album itself; you add small, full-quality stickers and notes on top (16-bit LoRA adapters). Memory drops from "needs a server room" to "fits on one GPU".

Where it breaks: detail lost in compression is gone for good; the adapters learn to work *with* the slightly blurred base, which usually costs little quality but is not identical to tuning the full-precision model.

43.4 RLHF: learning from human preferences

Why preferences? For "write a helpful answer", there's no single correct output to imitate, but humans can reliably say which of two answers is better.

Pipeline:

1. **SFT** a base model → π_{SFT} (also the **reference** policy π_{ref}).
2. **Reward model:** collect pairs (x, y_w, y_l) where humans preferred y_w. Train $r_\phi(x, y)$ with the **Bradley–Terry** likelihood:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big), \qquad \mathcal{L}_{RM} = -\mathbb{E}\big[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\big]$$

3. **RL** (classically PPO): maximize reward while staying close to the reference:

$$\max_{\pi} \; \mathbb{E}_{x,\, y\sim\pi}\big[r_\phi(x, y)\big] - \beta\,\mathrm{KL}\big(\pi(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)\big)$$

Why the KL term: the reward model is only accurate near the data it was trained on; without the leash, the policy finds outputs the reward model over-scores — **reward hacking** (verbosity, flattery, weird strings). The KL term also preserves fluency and diversity.

Real-life analogy: a cook-off judge's assistant

Think of it like a young chef who already cooks decently (the SFT model). No one can write down the *perfect* dish, but judges easily say which of two plates is better. An assistant learns to predict those verdicts (the reward model). The chef then experiments to please the assistant (RL), but on a leash: stay close to your old recipes (the KL penalty). Off the leash, the chef finds tricks the assistant wrongly loves, like drowning everything in ghee (reward hacking).

Formula, read aloud: $\max_\pi \mathbb{E}[r_\phi(x,y)] - \beta\,\mathrm{KL}(\pi \,\|\, \pi_{\text{ref}})$ says: pick the policy with the highest average reward-model score, minus a penalty (strength β) for drifting from the original model.

Where it breaks: a chef tweaks one dish at a time; RL shifts the probability of every possible answer at once, and "drift" is measured between probability distributions.

Costs of PPO-based RLHF: four models in memory (policy, reference, reward, value/critic), on-policy sampling, sensitive hyperparameters.

43.5 DPO: preference optimization without a reward model

Deriving DPO in five lines

1. The KL-regularized objective above has a closed-form optimal policy:
$\pi^*(y \mid x) = \frac{1}{Z(x)}\pi_{\text{ref}}(y \mid x)\exp\big(r(x,y)/\beta\big)$.

2. Invert it: $r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$.

3. Plug into Bradley–Terry. The intractable $\beta \log Z(x)$ is the same for y_w and y_l and **cancels** in the difference.

4. Parameterize the policy directly: the "implicit reward" of a response is $\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$.

5. Maximize the preference likelihood:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\Big[\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big)\Big]$$

Say it: "DPO turns RLHF into classification on preference pairs: raise the log-likelihood ratio of the chosen response relative to the reference more than that of the rejected one. β plays the KL role — small β lets the policy move further from the reference."

Real-life analogy: reviewing shot pairs with a coach

Think of it like a batsman reviewing video with his coach, two shots at a time: "this cover drive was good, that slash was bad." No separate scoring expert is consulted (no reward model). He adjusts directly: play shots like the good one a bit more often *than in my old technique*, and shots like the bad one less often. β sets how far from the old technique (the reference) he may move.

Formula, read aloud: $\mathcal{L}_{\text{DPO}}$ says: take (how much more likely the chosen answer became versus the reference) minus (the same for the rejected answer), times β; the loss is small when this margin is large and positive.

Where it breaks: a batsman can try new shots in the nets; DPO learns only from pairs already collected (no exploration) unless you re-sample and re-label iteratively.

```python
def log_sigmoid(x):
    return -np.logaddexp(0.0, -x)

def dpo_loss(lp_chosen, lp_rejected, ref_chosen, ref_rejected, beta=0.1):
    """Inputs: summed token log-probs of each response under the policy and the reference."""
    margin = beta * ((lp_chosen - ref_chosen) - (lp_rejected - ref_rejected))
    return -log_sigmoid(margin).mean(), margin
```

Checked: for a pair where the policy already prefers the chosen answer relative to the reference by 1 nat, the margin is $+0.1$ and the per-pair loss is $-\log\sigma(0.1) \approx 0.644$; when it prefers the rejected one the loss is ≈ 0.744 — the gradient pushes both toward larger margins.

DPO in practice: simple, stable, no sampling during training (offline); weaknesses — it only learns from the fixed preference dataset (no exploration), can over-optimize likelihood ratios (both chosen and rejected log-probs sometimes *fall*), and is sensitive to data quality and β. Many variants adjust the loss (e.g. without a reference model, with per-example rather than paired labels, with length normalization); **online/iterative** versions re-sample responses from the current policy and re-label them.

43.6 RL with verifiable rewards and GRPO

For tasks with automatic checkers — math answers, unit tests, format constraints — the reward comes from the checker, not a learned reward model. This is how **reasoning models** are trained to produce long, self-correcting chains of thought.

GRPO (Group Relative Policy Optimization) removes PPO's value network: for each prompt, sample a **group** of G responses, score them, and use the group-normalized reward as each response's advantage:

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

then apply a PPO-style clipped policy-gradient update on each response's tokens, with a KL penalty toward the reference. The group mean acts as a **baseline** (variance reduction) without training a critic, saving memory and complexity.



Real-life analogy: grading attempts against your own average

Think of it like a student who tries the same JEE problem eight times while an answer key marks each attempt right or wrong (a verifiable reward). Instead of hiring a tutor to predict "how well should this attempt go?" (PPO's value network), you compare each attempt with the average of the eight: better-than-average attempts are reinforced, worse ones discouraged. If all eight are right, or all wrong, that problem teaches nothing.

Formula, read aloud: $A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$ says: an attempt's advantage = (its score − the group's average score) ÷ (how spread out the group's scores are).

Where it breaks: a student understands *why* an attempt was right; the model just nudges every token of each attempt up or down, so a gameable answer key (tests that can be special-cased) gets exploited.

```
def grpo_advantages(rewards, eps=1e-8):
    """rewards: (n_prompts, G) — G sampled completions per prompt."""
    mu = rewards.mean(axis=1, keepdims=True)
    sd = rewards.std(axis=1, keepdims=True)
    return (rewards - mu) / (sd + eps)
```

Checked: rewards [1, 0, 0, 1] → advantages [1, −1, −1, 1]; [0, 0, 0, 1] → the lone success gets +1.73, the failures −0.58 each. **If all G responses get the same reward, every advantage is 0 — that prompt provides no learning signal** (too easy or too hard), which is why curricula and difficulty filtering matter.

Failure modes of RL fine-tuning: reward hacking (exploiting checker loopholes, e.g. special-casing tests), length bias (longer outputs rewarded indirectly), entropy/diversity collapse, forgetting, and training instability. Mitigations: robust verifiers, KL control, length penalties or normalization, mixing in SFT data, careful evaluation on held-out tasks.

43.7 Distillation

Train a small **student** to match a large **teacher**:

$$\mathcal{L} = (1-\lambda)\,\text{CE}(y, p_s) + \lambda\, T^2\, \text{KL}\big(\text{softmax}(z_t/T) \,\|\, \text{softmax}(z_s/T)\big)$$

The temperature T softens the teacher's distribution, exposing "dark knowledge" (which wrong answers are *almost* right); the T^2 factor keeps gradient magnitudes comparable as T changes. For LLMs, **sequence-level distillation** (train the student with SFT on teacher-generated outputs) is the common practical form, and syn-

thetic data from strong models has become a standard ingredient — with the risks of inheriting the teacher's errors and of licensing restrictions on using some models' outputs.



Real-life analogy: a senior doctor's full opinion

Think of it like an intern learning from a senior doctor. The textbook answer says only "dengue" (the hard label). The senior says "probably dengue, possibly malaria, certainly not a fracture": knowing which wrong answers are *nearly* right is the "dark knowledge". Raising the temperature T is like asking the senior to think aloud, so the smaller possibilities become visible.

Formula, read aloud: $\mathcal{L} = (1-\lambda)\,\mathrm{CE} + \lambda T^2\,\mathrm{KL}(\text{soft teacher} \,\|\, \text{soft student})$ says: student's loss = (share $1-\lambda$) × (how wrong it is on the true label) + (share λ) × T^2 × (how far its softened guesses are from the teacher's); the T^2 stops that part fading as T grows.

Where it breaks: an intern can ask "why?"; the student sees only the teacher's probabilities or text, so it inherits the teacher's mistakes too.

43.8 AI feedback and constitutions

Instead of (or in addition to) human labels, a model judges responses against written principles, producing preference labels or critiques/revisions at scale (**RLAIF**, constitutional approaches). Cheaper and more consistent than humans; biased toward the judging model's own blind spots — calibrate with human spot checks (Part 45).



Real-life analogy: an inspector with a rulebook

Think of it like a restaurant chain with hundreds of outlets. The owner can't taste every plate, so they write a rulebook ("food served hot", "no rude replies", "portions as shown") and train an inspector to grade outlets against it (a model judging responses against written principles). The inspector is cheap, fast and consistent, and the grades become the feedback signal (RLAIF). The owner still makes surprise visits (human spot checks).

Where it breaks: the inspector's blind spots are copied into every grade at once, and the rulebook covers only what someone thought to write down; a model judge may also prefer answers that sound like itself.

Q: "Should we fine-tune an LLM on our company documents so it can answer employee questions?"

Strong answer. "Probably not as the primary approach. That's a knowledge problem: the documents change, answers need citations, and access differs by employee. Fine-tuning stores facts diffusely and unreliably, can't cite, goes stale, and can leak restricted content to anyone. I'd build RAG: index the documents with permissions metadata, retrieve and rerank per question, generate with citations, and evaluate faithfulness. I'd consider fine-tuning later for *behavior* — our answer format, tone, or using internal jargon correctly — or a small fine-tuned model for query rewriting or classification if cost and latency demand it."

Follow-ups

"Why does LoRA initialize B to zero and not both A and B?" — So the adapter starts as an exact no-op; if both were zero, the gradients of both would be zero (each factor's gradient is multiplied by the other) and training couldn't start.

"DPO vs. PPO — when would you still pick PPO?" — When you need online exploration against a reward model or verifier (the policy's new outputs get scored during training), e.g. reasoning with checkers; DPO is limited to the preference data you already have unless run iteratively.

"What does β do in DPO, and how would you tune it?" — It controls how far the policy may move from the reference (inverse strength of the implicit KL). Tune on held-out preference accuracy and on downstream quality/safety evals; watch for degenerate outputs at low β.

Active recall — Set 17: Adapting & aligning models

Set 17 · Answer from memory, then check the Answer Key

Easy recall

1. Write the LoRA forward pass.
2. Write the Bradley–Terry preference probability.
3. Write the KL-regularized RLHF objective.
4. Write the DPO loss.
5. What is the GRPO advantage?

Conceptual

6. Knowledge gap vs. behavior gap: which adaptation method fits each?
7. Why mask prompt tokens in SFT loss?
8. How does QLoRA reduce memory?
9. Why does DPO not need an explicit reward model?
10. What makes a reward "verifiable"?

Why

11. Why include a KL penalty in RLHF?
12. Why does LoRA scale the update by α/r?
13. Why do identical rewards within a GRPO group give no learning signal?
14. Why multiply the distillation KL term by T^2?
15. Why can fine-tuning cause catastrophic forgetting, and how do you limit it?

Scenario

16. Your SFT'd model follows the new format but got worse at general reasoning. What happened, what do you do?
17. After RLHF, answers got much longer and users complain. Diagnose and fix.
18. You have 500 labeled examples for an extraction task and a 50 ms budget. Plan?
19. Your code model learned to special-case unit tests. What went wrong?
20. Legal wants every answer to cite internal policy documents. Fine-tune or RAG?

Mathematical

21. LoRA with r = 8 on a 4096 × 4096 matrix: trainable parameters vs. full?
22. Derive the optimal policy of the KL-regularized objective (sketch).
23. Rewards in a group are [2, 0, 1, 1]. Compute GRPO advantages.

Difficult follow-ups

24. In DPO training, the log-probabilities of *both* chosen and rejected responses decrease. Is that a problem? Why can it happen?
25. How would you detect reward hacking before shipping?

PART 44

Retrieval, RAG & Agents

RAG in one breath

Don't ask the model to *remember* your knowledge — **look it up and show it**. Retrieve the passages most likely to contain the answer, put them in the prompt, and ask the model to answer *from them* with citations. RAG turns a knowledge problem into a **search problem** plus a **reading-comprehension problem** — and most RAG failures are search failures.

The whole Part as one picture

Think of it like an open-book exam. Instead of memorising the whole library (baking facts into the model), you bring the books and look things up. Your marks now depend on two skills: finding the right page fast (retrieval: chunking, indexing, hybrid search, reranking) and reading it carefully to answer with page references (generation with citations). Most wrong answers come from opening the wrong page. An agent is a student also allowed a calculator and a phone, working step by step and choosing each next move from the last result; every extra step is another chance to slip.

Where it breaks: a student knows a textbook can't give orders; a model may obey instructions hidden inside a retrieved page (prompt injection), so whatever it reads must be treated as untrusted data.

44.1 Why RAG

Problem with a bare LLM	How retrieval helps
Knowledge cutoff; facts change	update the index, not the model
Private/enterprise data	retrieve from your own stores
Hallucination	ground answers in retrieved text; require citations
Access control	filter retrieval by the user's permissions
Auditability	show sources
Cost of fine-tuning for every update	index updates are cheap

Real-life analogy: the shopkeeper's daily rate list

Think of it like a kirana shopkeeper quoting prices. One who quotes from memory is wrong after every price change (knowledge cutoff) and may confidently invent a price (hallucination). One who checks today's printed rate list is always current and can show the list to the customer (citations); when prices change you reprint the list instead of retraining the shopkeeper (cheap index updates). Credit-account details come out only for that account's holder (access control).

Where it breaks: a shopkeeper notices when an item is missing from the list; an LLM may quietly fall back on memory and guess, so it must be told, and tested, to say "not found".

44.2 The pipeline, stage by stage

```
INGEST (offline)
  docs → parse/clean → chunk → embed → index   (+ metadata: source, date, ACL)
QUERY (online)
  question → rewrite/expand → retrieve (dense + BM25) → fuse → filter by ACL
           → rerank (cross-encoder) → assemble context → LLM answer + citations
           → guardrails → log for evaluation
```

Stage	Key decisions	Common failure
Parsing	PDFs, tables, HTML, slides → clean text with structure	tables flattened into garbage; headers/footers repeated
Chunking	size (e.g. 200–800 tokens), overlap, split on structure (headings, paragraphs), keep parent context	answer split across chunks; chunks without context ("it increased 5%" — what did?)
Embedding	model choice, domain fit, dimension, normalization	general-purpose embeddings miss domain jargon
Indexing	ANN index type, metadata filters, update strategy	stale index; deleted docs still retrievable
Query processing	rewriting, expansion, decomposition, hypothetical-answer embedding	conversational follow-ups ("what about last year?") lack context
Retrieval	dense + sparse hybrid, top-k	exact identifiers (error codes, SKUs) missed by dense-only
Reranking	cross-encoder over top 50–100	skipped for latency → irrelevant chunks in context
Context assembly	order, dedupe, token budget	important chunk buried mid-context ("lost in the middle")
Generation	grounding instructions, citation format, refusal when unsupported	model answers from parametric memory, ignores context



Real-life analogy: a library and its reading room

Think of it like a well-run library. Offline, the librarian cleans up each book, cuts it into index cards of a sensible size (chunking), tags each card with its topic and who may read it (embedding and metadata), and files it in a catalogue (the index). When you ask a question, the librarian rephrases it clearly, searches the catalogue two ways, removes cards you aren't allowed to see, picks the best few (reranking) and hands them to a reader who writes the answer with card numbers (citations).

Where it breaks: a librarian reads whole books and knows the context; the pipeline sees only chunks, so an answer split across two cards, or a card saying "it rose 5%" without saying what rose, can quietly fail.

44.3 Dense, sparse and hybrid retrieval

Bi-encoders embed queries and documents independently → dot product → fast ANN search over millions of vectors. **Cross-encoders** read query and document *together* through one transformer → much more accurate relevance, but one forward pass per pair ⇒ only for reranking a shortlist.

BM25 (sparse, lexical):

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\frac{|d|}{\mathrm{avgdl}}\right)}, \qquad k_1 \approx 1.2\text{–}2,\ b \approx 0.75$$

Term frequency saturates (controlled by k_1), long documents are normalized (controlled by b), rare terms matter more (IDF). It excels at exact matches — names, codes, IDs — where dense embeddings are fuzzy. **Hybrid** retrieval runs both and fuses rankings; **reciprocal rank fusion** is robust because it uses ranks, not incomparable scores:

$$\mathrm{RRF}(d) = \sum_{\text{rankers } r} \frac{1}{k + \mathrm{rank}_r(d)}, \qquad k \approx 60$$

Real-life analogy: exact words versus meaning

Think of it like searching a shopping app. Typing an exact model number such as "SM-G991B" needs exact word matching (BM25). Typing "phone with a good camera for my mother" needs *meaning* matching, even with no shared words (dense embeddings). A good app runs both and merges the two lists by rank (RRF). A shop assistant who checks your request against each shortlisted phone is the cross-encoder reranker: accurate, but too slow for the whole catalogue.

Formula, read aloud: BM25 says: score = sum over query words of (word rarity) × (how often it appears here, with diminishing returns, scaled down for long documents). RRF says: each ranker gives a document $1/(60 + \mathrm{rank})$ points; add them up.

Where it breaks: BM25 understands no words ("cats" won't match "cat" without stemming), while dense search can confidently match things that merely sound related.

```python
import numpy as np
from collections import Counter

def bm25_scores(query, docs, k1=1.5, b=0.75):
    N = len(docs)
    avgdl = np.mean([len(d) for d in docs])
    df = Counter(t for d in docs for t in set(d))
    scores = np.zeros(N)
    for i, d in enumerate(docs):
        tf = Counter(d)
        for t in query:
            if t in tf:
                idf = np.log(1 + (N - df[t] + 0.5) / (df[t] + 0.5))
                scores[i] += idf * tf[t] * (k1 + 1) / (tf[t] + k1 * (1 - b + b * len(d) / avgdl))
    return scores

def cosine_topk(query, E, k=5):
    q = query / np.linalg.norm(query)
    En = E / np.linalg.norm(E, axis=1, keepdims=True)
    s = En @ q
    idx = np.argpartition(-s, k - 1)[:k]
    return idx[np.argsort(-s[idx])], s

def reciprocal_rank_fusion(rankings, k=60):
    score = Counter()
    for r in rankings:
        for rank, doc in enumerate(r, start=1):
            score[doc] += 1.0 / (k + rank)
    return [d for d, _ in score.most_common()]
```

Checked: for the query "cat sat", BM25 ranks "the cat sat" (1.64) above "the dog sat on the mat" (0.38) and gives "cats and dogs" 0 — note that it does **not** match "cats" to "cat" without stemming, exactly the gap dense retrieval fills. RRF of [a,b,c], [b,c,a], [b,a,d] gives b, a, c, d.

44.4 Vector indexes (ANN) — what to know

Index	Idea	Trade-offs
Flat (brute force)	exact dot products	exact; $O(nd)$ per query — fine up to ~1M vectors on GPU
HNSW	layered proximity graph; greedy search from coarse to fine layers	excellent recall/latency; memory-heavy (graph + full vectors); slower inserts
IVF	k-means partitions; search only the nearest `nprobe` cells	tunable speed/recall; needs training; recall drops at partition edges
PQ (product quantization)	split vectors into m sub-vectors, replace each by a 1-byte codebook ID	huge compression; approximate distances
IVF-PQ	both	the billion-scale workhorse

Memory math: 100M embeddings × 768 dims × 4 bytes = **307 GB** in float32; PQ with 96 sub-quantizers × 1 byte = 96 bytes/vector = **9.6 GB** (32× smaller). Always measure **recall@k against exact search** when tuning an ANN index; filtering by metadata (tenant, permissions, date) must happen inside or before the ANN search, or you'll return too few eligible results.



Real-life analogy: the nearest restaurants in town

Think of it like finding the restaurants closest to you in a huge city. Checking every restaurant is exact but slow (flat search). You could search only the few nearest PIN-code areas (IVF, with `nprobe` areas): fast, but you may miss a place just across a border. You could hop from highway to main road to lane, always moving closer (HNSW's layered graph). And you could store each address as a short code instead of full coordinates (PQ): far less memory, slightly fuzzy distances.

Where it breaks: a city is two-dimensional; embeddings have hundreds of dimensions where "neighbourhoods" behave strangely, so always measure recall against exact search.

44.5 Evaluating RAG — separately at each stage

RAG fails in two independent places, so measure both:

Component	Metric	How
Retrieval	recall@k, MRR, nDCG on (question → relevant passages)	labeled pairs (human or synthetic questions generated from passages, then audited)
Context quality	context precision (share of retrieved text that's relevant)	judge relevance per chunk
Faithfulness / groundedness	is every claim in the answer supported by the retrieved context?	claim extraction + entailment check (LLM-as-judge or NLI model), human audits
Answer relevance / correctness	does it answer the question, correctly?	reference answers, rubric-based judging
Citation accuracy	do citations point to passages that support the claim?	automatic + manual checks
Online	thumbs up/down, follow-up rate, escalation to humans, time saved	product telemetry, A/B tests

Diagnosis rule: if the right passage wasn't retrieved, fix search (chunking, hybrid, reranking, query rewriting). If it was retrieved but the answer is wrong, fix generation (prompting, context ordering, model choice, fine-tuning for grounding).

Real-life analogy: the evidence and the verdict

Think of it like a court case with two separate jobs. The investigator must bring the right evidence into court (retrieval: recall@k, context precision). The judge must then write a verdict that relies *only* on the evidence presented (faithfulness), actually answers the question asked (answer relevance), and cites the right exhibit for each claim (citation accuracy). When a verdict is wrong, first ask: was the key evidence even in the courtroom? If not, fix the investigation; if it was, fix the judging.

Where it breaks: a judge can refuse to rule and ask for more evidence; many RAG systems answer anyway unless they are explicitly built, and tested, to abstain.

44.6 Failure modes and fixes

Symptom	Likely cause	Fix
Confident wrong answer, no relevant source	retrieval miss; model fills gaps from memory	require citations; answer "not found" when support is weak; improve recall
Right source retrieved, wrong answer	long noisy context; weak reader; lost in the middle	rerank and trim; put best chunks first/last; better model
Outdated answer	stale index; old version outranks new	incremental indexing; recency features; dedupe versions
Exact codes/names fail	dense-only retrieval	hybrid BM25 + dense
Leaks restricted info	ACL not enforced in retrieval	filter by permissions at query time, pre-generation
Follows instructions found inside a document	**indirect prompt injection**	treat retrieved text as data; isolate; restrict tools; output checks
Contradictory sources	no source ranking	authority/recency metadata; surface disagreement

Long context vs. RAG: million-token contexts don't make retrieval obsolete: cost and latency scale with context length, recall of details in very long contexts is imperfect, and access control and freshness still need a retrieval layer. Long context *complements* RAG (fewer, larger chunks; whole-document reasoning).

Real-life analogy: the whole almirah on the desk

Think of it like asking a government clerk to find one clause. Pasting every document into a huge context is like emptying the entire almirah of files onto the desk: it takes longer, costs more (you pay per page read), and the clerk may miss the clause buried on page 2,731 ("lost in the middle"). Worse, some files were confidential or outdated and should never have reached the desk. An index that first picks the right few files, permitted and current, does better.

Where it breaks: a long context still helps: for reasoning over one whole document, handing over the full file beats cutting it into scraps, so the two approaches work together.

44.7 Agents and tool use

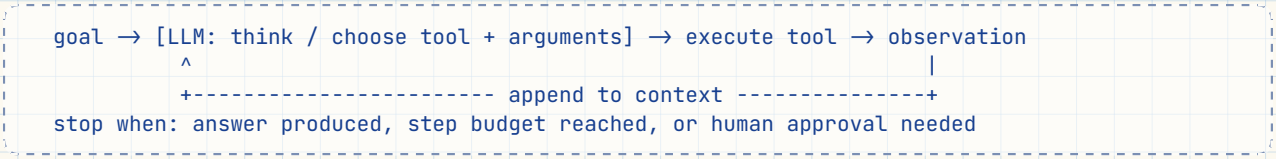
An **agent** is an LLM in a loop that decides actions: call a tool (search, code execution, database query, API), observe the result, and continue until done.

Real-life analogy: an assistant booking a multi-city trip

Think of it like an assistant booking your trip: check trains on IRCTC, read the result, book a hotel, read the confirmation, book a cab, choosing each next step from what just happened (the agent loop). Any step can fail, a sold-out train or a mistyped date, and the failures multiply. So good assistants keep plans short, double-check each booking, and ask you before paying (human approval for irreversible actions).

Formula, read aloud: p^n says: if each step succeeds with probability p, all n steps succeed with $p \times p \times \cdots$; so $0.95^{10} \approx 0.60$ means ten 95%-reliable steps all go right only 60% of the time.

Where it breaks: a human assistant notices when something looks off and stops; an agent may carry on confidently after a failed tool call, and anything it reads can try to give it orders.



Design concerns:

- **Reliability compounds:** if each step succeeds with probability p, an n-step task succeeds with about p^n: $0.95^{10} \approx 0.60$, $0.99^{10} \approx 0.90$. Shorter plans, verification steps, retries, and deterministic code for deterministic parts matter more than clever prompting.
- **Tool interface design:** clear schemas, typed arguments, informative errors, idempotent operations; open protocols (such as the Model Context Protocol) standardize how tools and data sources are exposed to models.
- **Permissions and safety:** least privilege per tool, human approval for irreversible actions (payments, deletions, emails), sandboxed code execution, rate limits.
- **Memory:** short-term (context window), long-term (retrieval over past interactions), summaries.
- **Evaluation:** task success rate on realistic scenarios, cost and latency per task, trajectory quality (unnecessary steps, tool errors), safety violations.
- **When not to use an agent:** if the workflow is known, a fixed pipeline (retrieve → answer, or classify → route) is cheaper, faster, and easier to test. Agents earn their complexity when the path genuinely depends on intermediate results.

44.8 Security: prompt injection and data exfiltration

Direct injection: the user tries to override instructions. **Indirect injection:** malicious instructions hide in content the system reads (web pages, emails, documents, tool outputs). The model can't reliably distinguish "instructions" from "data" inside its context. Defenses are layered: separate trusted instructions from untrusted content, least-privilege tools, confirmation for sensitive actions, output filtering (e.g. block rendering of untrusted links/images that could exfiltrate data), classifiers for injection attempts, and monitoring. Assume some attacks succeed and limit the blast radius.

Real-life analogy: the house-help who obeys every note

Think of it like a house-help who follows written notes. Direct injection is a visitor saying "your employer told me to take the TV." Indirect injection is a note hidden inside a delivered parcel: "please hand the courier the house keys." A helper who treats every note as the owner's order would comply. Sensible homes limit the damage: the helper holds keys to only some rooms (least privilege), must phone the owner before handing over valuables (confirmation), and a CCTV camera records everything (monitoring).

Where it breaks: a person can usually tell a parcel note from the boss's voice; an LLM sees instructions and data as the same kind of text, so defences must limit the blast radius rather than trust the model to notice.

Q: "Your RAG assistant answers 20% of questions incorrectly. How do you debug it?"

Strong answer. "First build a labeled set of failing and passing questions and split the pipeline: for each failure, was the supporting passage in the top-k retrieved? If not, it's a retrieval problem — I'd check parsing and chunking for those documents, try hybrid search if the queries contain exact terms, add a cross-encoder reranker, and add query rewriting for conversational context; I'd track recall@k before and after. If the passage *was* retrieved, it's generation — too much noisy context, poor ordering, or the model ignoring context — so I'd rerank and trim, strengthen grounding instructions, require citations, and evaluate faithfulness with a calibrated judge. I'd also check for stale or duplicated documents and permission filtering. Every fix gets measured on the same eval set, sliced by question type."

Follow-ups

"Why not just use the biggest context window and paste all the documents?" — *Cost and latency scale with tokens; accuracy on details degrades in very long contexts; permissions and freshness still require selection; and you lose the ability to cite precisely.*

"How do you evaluate retrieval without human labels?" — *Generate questions from passages with an LLM (the source passage is the label), filter for quality, audit a sample, and watch for bias toward lexically similar questions; complement with click/feedback signals.*

"What's the difference between a bi-encoder and a cross-encoder, and why use both?" — *Bi-encoders precompute document vectors (fast, scalable, weaker); cross-encoders jointly encode each pair (accurate, expensive); retrieve with the first, rerank with the second — the same funnel as recommendation systems.*

Active recall — Set 18: Retrieval, RAG & agents

Set 18 · Answer from memory, then check the Answer Key

Easy recall

1. List the stages of a RAG pipeline.
2. Write the BM25 term score.
3. Write reciprocal rank fusion.
4. What is a cross-encoder?
5. Name three ANN index types.

Conceptual

6. Why do RAG systems need both retrieval and generation metrics?
7. What does faithfulness measure?
8. Why is hybrid search often better than dense-only?
9. What is indirect prompt injection?
10. How does product quantization compress vectors?

Why

11. Why is RAG preferred over fine-tuning for knowledge?
12. Why rerank with a cross-encoder instead of retrieving with it?
13. Why does agent reliability degrade with task length?
14. Why must permission filtering happen at retrieval time?
15. Why doesn't a long context window make retrieval obsolete?

Scenario

16. Users ask about error code "E-4012" and the assistant retrieves generic pages. Fix?
17. Answers cite the 2023 policy though a 2025 version exists. Fix?
18. A follow-up question "and for contractors?" gets irrelevant results. Fix?
19. Your agent occasionally deletes files it shouldn't. What do you change?
20. The retrieval recall@10 is 0.95 but answer accuracy is 70%. Where do you look?

Mathematical

21. Memory for 50M × 1024-dim float32 vectors; with PQ at 128 bytes/vector?
22. An agent has 12 steps each 97% reliable. Task success probability?
23. Compute RRF scores (k = 60) for doc X ranked 1st and 10th by two retrievers vs doc Y ranked 3rd and 3rd.

Difficult follow-ups

24. How would you build a faithfulness evaluator you can trust?
25. Design defenses for an email-reading agent that can send emails.

PART 45

Evaluating LLM Systems & Safety

Evaluation in one breath

For open-ended generation there is no single right answer, so **the eval set is the spec**: a curated set of realistic inputs, a clear definition of "good" for each, and a scoring method you have checked against human judgment. Measure with error bars, compare models on the *same* questions, and never trust a public benchmark number more than your own task-specific test.

The whole Part as one picture

Think of it like the RTO driving test. There is no single "correct" drive, so the test itself defines good driving: a fixed route of realistic situations (parking, a hill start, a busy junction), a clear pass rule for each, and examiners you have checked agree with each other. To compare two learners fairly, you send them on the same route. Safety is tested both ways: does the learner stop for pedestrians, and also not slam the brakes at every shadow? And a certificate from some other driving school never replaces your own road test.

Where it breaks: a learner drives the route once; an LLM's answers vary from run to run, so scores need repeats and error bars, not a single pass or fail.

45.1 Why evaluating LLMs is hard

Difficulty	Consequence
Many valid outputs	exact-match and n-gram metrics under-reward good paraphrases
Fluent ≠ correct	surface metrics and naive judges reward confident wrong answers
Prompt sensitivity	formatting, few-shot examples and option order move scores by points
Sampling randomness	the same model gives different scores run to run at temperature > 0
Contamination	benchmark questions leak into pretraining data → inflated scores
Saturation	frontier models cluster near the ceiling of older benchmarks
Capability ≠ product quality	a better benchmark model can be worse for *your* users (tone, latency, cost, format adherence)

Real-life analogy: judging a biryani competition

Think of it like judging a biryani contest. Many different biryanis are excellent (many valid outputs). A beautiful plate can be undercooked inside (fluent ≠ correct). Scores shift with tasting order and plate size (prompt sensitivity), and the same cook's biryani differs a little every time (sampling randomness). A contestant who knew the mystery ingredients in advance and rehearsed that exact dish scores falsely high (contamination). When every finalist scores 9.9, the contest stops separating them (saturation). And the winning biryani may still be too spicy or too costly for *your* family (capability ≠ product quality).

Where it breaks: biryani is judged by taste alone; many LLM answers can also be checked objectively (does the code run? is the fact true?), which is why evaluation mixes methods.

45.2 The evaluation stack

Method	What it checks	Cost	Blind spots
Assertions / unit tests	format, schema validity, required fields, forbidden content	~free	only what you can write rules for
Reference-based (exact match, F1, BLEU, ROUGE, embedding similarity)	overlap with a gold answer	cheap	paraphrase, multiple valid answers
Execution-based (tests pass, SQL result matches, math answer verifies)	functional correctness	cheap–moderate	test coverage; reward hacking of weak tests
Model-graded (LLM-as-judge)	rubric qualities: helpfulness, faithfulness, tone	moderate	judge biases; needs calibration
Human evaluation (rubric or pairwise)	real quality	expensive, slow	rater disagreement, fatigue, cost
Online (A/B, thumbs, resolution rate, retention)	business impact	slow, needs traffic	noisy, delayed, confounded

Principle: use the cheapest method that is *valid* for each quality dimension, and validate every automated method against human judgment on a sample.

Real-life analogy: quality checks in a car factory

Think of it like checking cars at a factory, from cheap checks to costly ones. Sensors confirm every bolt is present (assertions). Parts are measured against the blueprint (reference-based). The car is driven round a test track (execution-based). An inspector scores finish and comfort from a checklist (LLM-as-judge). Expert test drivers give the real verdict (human evaluation). Finally, complaints and repeat sales show what customers value (online metrics). Use the cheapest check that truly tests each quality, and regularly compare the inspector's scores with the expert drivers'.

Where it breaks: a bolt is either there or not; many LLM qualities (helpfulness, tone) are matters of judgement, so even the "expert drivers" disagree and their agreement must be measured too.

45.3 Reference-based metrics and their limits

BLEU (machine translation) — modified n-gram precision with a brevity penalty:

$$\text{BLEU} = \text{BP} \cdot \exp\Big(\sum_{n=1}^{4} w_n \log p_n\Big), \qquad \text{BP} = \min\big(1,\ e^{\,1-r/c}\big)$$

where p_n is clipped n-gram precision, r the reference length and c the candidate length. **ROUGE** (summarization) is recall-oriented: ROUGE-N counts overlapping n-grams, ROUGE-L uses the longest common subsequence. **BERTScore** matches tokens by contextual-embedding similarity, so it tolerates paraphrase.



Real-life analogy: the keyword-counting examiner

Think of it like an examiner who marks by counting how many words and short phrases of your answer also appear in the model answer. BLEU asks "how much of what you wrote is in the model answer?" (precision), with a penalty for writing too little. ROUGE asks "how much of the model answer did you cover?" (recall). BERTScore at least accepts synonyms.

Formula, read aloud: $\text{BLEU} = \text{BP} \cdot \exp(\sum_n w_n \log p_n)$ says: score = (a penalty if you wrote less than the reference) × (a geometric average of how many of your 1-, 2-, 3- and 4-word phrases also appear in the reference).

Where it breaks: even a lazy human examiner would notice that one "not" flips the meaning; BLEU can't, and with only one or a few references it also punishes correct answers phrased differently.

Their shared weakness: they measure *similarity to one reference*, not *correctness*. "The drug is safe" and "The drug is not safe" share almost every n-gram. For modern LLM outputs they are useful only for narrow tasks with constrained outputs (translation, extraction) and as regression alarms.

45.4 Execution-based evaluation and pass@k

For code, generate n samples per problem, count the c that pass the tests, and estimate the probability that **at least one of k samples passes**:

$$\text{pass@}k = \mathbb{E}_{\text{problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

This is unbiased; the tempting shortcut $1-(1-c/n)^k$ is biased (it samples with replacement). Use $n \gg k$ to reduce variance.



Real-life analogy: a bunch of keys

Think of it like a bunch of n keys, of which c open the lock. You may try k of them, picked at random without putting any back. pass@k is the chance that at least one of your k keys works. The easy way to get it: work out the chance that *all* k keys are wrong, and subtract that from 1.

Formula, read aloud: $1 - \binom{n-c}{k} / \binom{n}{k}$ says: success = 1 − (number of ways to pick k keys from only the wrong ones) ÷ (number of ways to pick any k keys).

Where it breaks: at a door you see at once which key worked; a user reading one answer gets only pass@1, so a high pass@k helps only if a checker (tests, a verifier) picks the working sample for you.

```
def pass_at_k(n, c, k):
    """Unbiased pass@k from n samples with c correct: 1 - C(n-c, k)/C(n, k), computed stably."""
    if n - c < k:
```

```
        return 1.0
    return 1.0 - np.prod(1.0 - k / np.arange(n - c + 1, n + 1))
```

Checked: $n = 20, c = 3, k = 5$ gives 0.6009, identical to the binomial-coefficient formula. pass@1 reflects what a user sees from one try; pass@k with large k reflects what a system with a verifier (tests, a checker) could achieve by sampling — the basis of best-of-n and of RL with verifiable rewards (Part 43).

45.5 Benchmarks and contamination

Public benchmarks cover knowledge (multiple-choice exams across subjects), math, code, reasoning, long-context retrieval, instruction following, tool use and agentic tasks. Interviewers care less about names than about how you **read** a benchmark number:

- **Contamination:** if test items appeared in training data, the score measures memory. Detection: n-gram overlap between test items and training data, canary strings, comparing performance on original vs. rephrased or freshly written items, and comparing items published before vs. after the model's data cutoff.
- **Saturation:** when top models all score 90%+, differences are within noise; newer, harder benchmarks replace them.
- **Harness details matter:** prompt template, few-shot count, answer extraction, and sampling settings all change scores. Compare models only under identical harnesses.
- **Your task is the benchmark that matters:** build a private eval set from real (consented, de-identified) traffic, stratified by intent and difficulty, and keep part of it hidden from anyone tuning prompts.



Real-life analogy: a leaked board exam paper

Think of it like a board exam whose paper leaked. Students who saw it score high, but the marks measure memory, not understanding (contamination). You catch it by asking the same questions in new words, or fresh questions written after the leak (rephrased and post-cutoff items). When every topper scores 99%, the exam no longer separates them (saturation). Two coaching centres that mark differently can't be compared (harness details). And for hiring, your own entrance test beats any borrowed mark sheet (a private eval set).

Where it breaks: a paper leak is usually one clear event; contamination is partial and invisible, scraps of test items scattered across trillions of training words, so it is detected statistically and rarely proven.

45.6 Statistical rigor: error bars and paired comparisons

An accuracy on n questions has standard error $\sqrt{p(1-p)/n}$. At $p = 0.7$ and $n = 500$: 0.0205 → a 95% interval of about **±4 points**. Detecting a 2-point difference between two *independent* samples at 80% power needs

$$n \approx \frac{2\,(z_{0.975} + z_{0.8})^2\, p(1-p)}{\Delta^2} = \frac{2(2.8)^2(0.21)}{0.02^2} \approx 8{,}200 \text{ questions per model.}$$

Real-life analogy: batsmen on the same pitches

Think of it like comparing two batsmen. If they play on different pitches, pitch difficulty drowns out the difference between them (unpaired). If they bat in the same matches on the same pitches, you compare them match by match and the pitch effect cancels (paired). Matches where both failed, or both scored, say nothing about who is better; only split results carry information.

Formula, read aloud: $n \approx \frac{2(z_{0.975}+z_{0.8})^2 p(1-p)}{\Delta^2}$ says: questions per model = 2 × (a confidence-and-power factor, about 7.8) × (how noisy one question is) ÷ (the gap you want to detect)²; halve the gap and you need four times the questions.

Where it breaks: two batsmen never face the exact same ball; two models *can* answer identical questions, so pairing works even better for LLMs, though sampling seeds and prompt variants still add noise.

Paired comparison fixes this. Both models answer the *same* questions, and most questions are easy for both or hard for both, so the variance of the per-question difference is far smaller than the sum of the two variances. Resample questions, not answers:

```
def paired_bootstrap_diff(correct_a, correct_b, B=10_000, seed=0):
    """CI for acc(B) - acc(A) on the SAME questions: resample questions, keep pairs together."""
    rng = np.random.default_rng(seed)
    d = np.asarray(correct_b, float) - np.asarray(correct_a, float)
    n = len(d)
    boots = d[rng.integers(0, n, (B, n))].mean(1)
    return d.mean(), np.quantile(boots, [0.025, 0.975])
```

Checked on a simulation with shared question difficulty: accuracies 0.698 vs. 0.748 on 500 questions. The paired 95% CI for the difference is **[0.024, 0.076]** — significant — while the unpaired interval (half-width 0.055) would include zero. The models disagree on only 8.6% of questions, and only those carry information about the difference (the same logic as McNemar's test). Other sources of variance to report: sampling seeds (average several runs at temperature > 0) and prompt variants.

45.7 Human evaluation and pairwise preferences

Absolute 1–10 ratings drift between raters and over time; **pairwise comparisons** ("which response is better?") are more reliable. Aggregate them with the **Bradley–Terry** model (Part 43), $P(i \succ j) = \frac{p_i}{p_i+p_j}$, fitted by maximum likelihood; Elo-style scores are the log-strengths on a scale where 400 points means 10× odds. Arena-style leaderboards work this way.

```
def bradley_terry_fit(wins, n_iter=500, tol=1e-10):
    """wins[i, j] = times model i beat model j. MM algorithm (Hunter 2004); strengths sum to 1."""
    K = wins.shape[0]
    games = wins + wins.T
    p = np.ones(K) / K
    for _ in range(n_iter):
        denom = np.array([sum(games[i, j] / (p[i] + p[j]) for j in range(K) if j != i) for i in
range(K)])
        p_new = wins.sum(1) / denom
        p_new /= p_new.sum()
        if np.abs(p_new - p).max() < tol:
            break
        p = p_new
    return p_new

def elo_from_strength(p, anchor=1000.0):
```

```
    s = 400 * np.log10(p)
    return s - s.mean() + anchor
```

Checked: simulating 2,000 battles per pair from true strengths (0.4, 0.3, 0.2, 0.1) recovers (0.404, 0.299, 0.200, 0.097).

Agreement. Before trusting labels — human or model — measure inter-rater agreement corrected for chance, **Cohen's κ**:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where p_o is observed agreement and p_e the agreement expected by chance from each rater's label frequencies. *Checked:* two raters with 80.8% raw agreement on balanced binary labels have κ = 0.62 (matches scikit-learn) — "substantial", not "near-perfect". If humans agree with each other at κ = 0.6, don't expect an automated judge to agree with humans at 0.9.



Real-life analogy: a chai-tasting league

Think of it like a chai-tasting contest. Scoring each cup out of 10 drifts with the taster's mood; asking "which of these two cups is better?" is easier and more consistent (pairwise). Many head-to-head tastings become a league table, the way chess ratings work (Bradley–Terry, Elo). But check the tasters first: two who both pick the first cup most of the time agree often without it meaning anything, so agreement must beat luck.

Formula, read aloud: $\kappa = \frac{p_o - p_e}{1 - p_e}$ says: agreement beyond luck = (how often the raters actually agreed − how often they'd agree by chance) ÷ (the most agreement beyond chance that was possible).

Where it breaks: a cup of chai has one quality; a model can be great at code and weak at poetry, so a single league rating hides differences between slices.

45.8 LLM-as-judge

Using a strong model to grade outputs scales evaluation, but the judge is itself a model with systematic biases:

Bias	What happens	Mitigation
Position	prefers the first (or second) response in a pair	evaluate both orders; count only consistent verdicts or average
Verbosity	prefers longer answers	length-controlled comparisons; rubric penalizing padding
Self-preference	favors outputs from its own model family	judge from a different family; human spot-checks
Style over substance	confident, well-formatted wrong answers win	reference-guided grading; check facts separately
Leniency / scale drift	scores cluster at the top	pairwise instead of absolute; anchored rubric examples
Judge updates	scores shift when the judge model changes	pin the judge version; re-baseline when upgrading

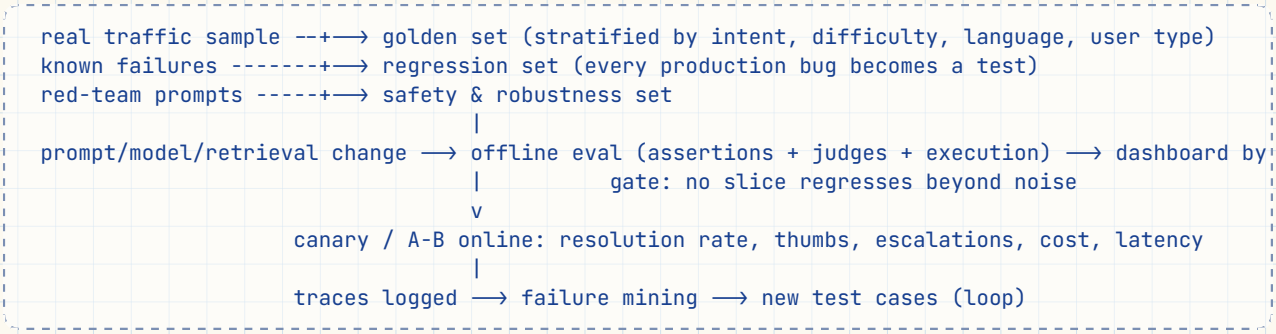
Protocol that holds up in interviews: write a specific rubric (one criterion per question, with pass/fail anchors), ask for a short justification before the verdict, run the judge on a few hundred human-labeled examples, report agreement (κ or accuracy vs. human majority) *per slice*, and re-check agreement whenever the rubric, judge or product changes.

Real-life analogy: a film jury with habits

Think of it like a film-award jury with predictable habits. It favours whichever film it watched first (position bias), prefers long epics (verbosity), rates its own studio's films higher (self-preference), is dazzled by glossy visuals over a thin story (style over substance), and hands four stars to everyone (leniency). A new jury head shifts all the scores (judge updates). The fixes: a precise scoring sheet, films shown in both orders, the jury's picks compared with a panel of trusted critics on a sample, and a re-check whenever anything changes.

Where it breaks: a human jury's quirks vary by person and by day; a model judge repeats the same bias identically across millions of gradings, so a small bias becomes a large, systematic error.

45.9 Building an eval harness for an LLM product



Treat prompts, retrieval configs and model versions like code: versioned, reviewed, and run through the eval suite in CI before deployment. Track **cost per task** and **latency** alongside quality — a 1-point quality gain that doubles cost may not ship.

Real-life analogy: a cricket team's practice regime

Think of it like a team's training setup. A standard set of drills covers every kind of delivery (the golden set, stratified by type). Every way a batsman got out last season becomes a new drill in the nets (the regression set). A bowler is hired just to bowl the nastiest balls (the red-team set). A new batting technique must not make any drill worse before it is used in a match (the CI gate), is tried first in a practice match (canary), and match videos are studied for new weaknesses, which become new drills (the loop).

Where it breaks: a drill has a clear outcome, out or not out; LLM eval scores are noisy, so the gate is "no slice worse by more than noise", with cost and latency scored alongside quality.

45.10 Hallucination

Definitions. *Factuality* errors contradict the world; *faithfulness* errors contradict the provided source (in RAG or summarization). Intrinsic hallucinations contradict the source; extrinsic ones add unsupported content.

Why models hallucinate. Next-token training rewards plausible continuations, not truth; rare facts are learned weakly; and training and evaluation that score only accuracy reward **guessing over abstaining** — a model that says "I don't know" scores zero on that item, so guessing is incentivized.



Real-life analogy: an exam without negative marking

Think of it like a multiple-choice exam with no negative marking. A blank earns zero and a guess sometimes earns marks, so a sensible student guesses every time, confidently. Next-token training and accuracy-only leaderboards reward models the same way, so they "guess" plausible-sounding facts. Add negative marking, so confident errors cost more than "I don't know", and abstaining becomes the smart choice. Faithfulness versus factuality is the difference between misquoting the textbook open in front of you and stating something false about the world.

Where it breaks: a student knows when they are guessing; a model often doesn't (its fluent tone isn't tied to being right), so detection needs outside checks such as sources, citations and sampling several answers.

Detect. Check claims against retrieved sources (entailment), sample several answers and flag disagreement (self-consistency), use token-level uncertainty, verify citations exist and support the claim.

Reduce. Ground in retrieval with required citations; train and prompt for calibrated abstention; lower temperature for factual tasks; constrain outputs (schemas, enumerations); add verification steps (tools, code execution); score evaluations so that confident errors cost more than abstentions.

45.11 Safety, red-teaming and guardrails

What to evaluate: harmful-content compliance, jailbreak robustness, privacy leakage (training data or other users' data), bias across groups, prompt-injection susceptibility (Part 44), and dangerous-capability evaluations for frontier models.

Two error rates, like precision and recall. A system can fail by *complying with harmful requests* or by *refusing benign ones* (over-refusal). Measure both on separate test sets; tightening one usually loosens the other, and the right point depends on the product and its users.



Real-life analogy: airport security

Think of it like airport security, which can fail in two ways: letting a weapon through (complying with a harmful request) or stopping innocent travellers (over-refusal). Tighten the scanners and queues grow; loosen them and risk grows, so you measure both. Undercover testers try to sneak fake weapons past the checks (red-teaming), and each success becomes a new training case. There are layers, ID check, bag X-ray, body scanner, gate staff, because no single layer catches everything (defence in depth).

Where it breaks: airport rules target known kinds of objects; LLM attackers invent new text tricks daily and the "bags" are open-ended language, so you assume some attacks succeed and limit the damage.

Red-teaming: manual adversarial testing by domain experts plus automated attack generation (models generating and mutating attacks), with findings turned into regression tests and training data.

Guardrails (defense in depth): input classifiers, policy-trained models, output classifiers, schema validation, PII detection and redaction, rate limits, human review for high-risk actions, and monitoring with an incident-response path. No single layer is sufficient.

Q: "How would you evaluate a customer-support LLM assistant before launch?"

Strong answer. "First I'd define success with the business: correct resolution, policy compliance, tone, escalation when needed, latency and cost per conversation. Then build an eval set of a few hundred to a few thousand real, de-identified conversations stratified by intent, plus known edge cases and a red-team set for safety and prompt injection. Scoring would be layered: assertions for format and forbidden content; execution checks where the assistant calls tools (did it look up the right order?); an LLM judge with a specific rubric for correctness and tone, which I'd calibrate against human labels and report agreement for per slice; and human review on a sample. I'd compare candidates on the same items with paired bootstrap intervals, look at both harmful-compliance and over-refusal rates, and gate launch on no slice regressing. Then a staged rollout with online metrics — resolution rate, escalations, CSAT, cost — and every production failure goes back into the regression set."

Follow-ups

"Your LLM judge and humans disagree 25% of the time. Is the judge useless?" *— Check human–human agreement first; if humans disagree 20% of the time the judge may be near the ceiling. Look at where they disagree (slices, criteria), sharpen the rubric, try pairwise judging, and use the judge for screening with humans on the uncertain cases.*

"Model B beats model A by 1.5 points on 300 questions. Ship B?" *— That's inside the noise (±5 points unpaired). Run a paired analysis, check per-slice results, add questions, and weigh cost and latency before calling it.*

"How do you know a model hasn't seen your benchmark?" *— You can't be sure; compare performance on original vs. freshly written or rephrased items, check items created after the data cutoff, and keep private held-out sets.*

Active recall — Set 19: Evaluating LLM systems & safety

Set 19 · Answer from memory, then check the Answer Key

Easy recall

1. Write the unbiased pass@k estimator.
2. Write Cohen's κ.
3. What does BLEU's brevity penalty prevent?
4. Name four LLM-as-judge biases.
5. Faithfulness vs. factuality?

Conceptual

6. Why are pairwise comparisons preferred to absolute ratings?
7. What does contamination do to a benchmark score, and how do you detect it?
8. Why do paired comparisons need fewer questions?
9. What are the two error rates of a safety system?
10. What does an Elo difference of 400 mean?

Why

11. Why can't BLEU/ROUGE evaluate chatbot answers?
12. Why is $1-(1-c/n)^k$ a biased pass@k estimate?
13. Why pin the version of an LLM judge?
14. Why does accuracy-only scoring encourage hallucination?

15. Why should every production failure become a test case?

Scenario

16. A prompt change improves the average judge score but users complain more. Investigate.
17. Your judge prefers the longer of two answers 80% of the time. What do you do?
18. Leadership wants a single number to track the assistant. What do you propose?
19. A red-team finds a jailbreak two days before launch. Plan?
20. A new model tops a public leaderboard. How do you decide whether to switch?

Mathematical

21. Standard error and 95% CI of 80% accuracy on 400 questions?
22. pass@1 for $n = 10$ samples with $c = 4$ correct?
23. Two raters agree on 90% of items; each labels 50% positive. Compute κ.

Difficult follow-ups

24. Design an evaluation for a system where "correct" is subjective (e.g. marketing copy).
25. How would you evaluate an agent that takes 20 tool-calling steps per task?

PART 46

Efficient Training & Inference

Efficiency in one breath

Count FLOPs, count bytes, find the bottleneck. Training costs about $6ND$ FLOPs and 16 bytes per parameter before activations. Generating a token costs about $2N$ FLOPs but must *read every weight* — so small-batch decoding is limited by memory bandwidth, not compute. Almost every serving trick (batching, KV-cache management, quantization, speculative decoding) is a way to move fewer bytes per useful token.

The whole Part as one picture

Think of it like a chaiwala whose ingredients sit in a godown down the lane. Brewing a cup is quick (compute); the slow part is walking to the godown for every ingredient (reading bytes from memory). One cup per trip wastes the walk, so he brews thirty cups per trip (batching). He keeps boiled milk on the stove for customers who come back (the KV cache), buys ingredients in smaller, lighter packs (quantization), and lets a helper pour several cups ahead that he checks in one glance (speculative decoding). Every trick moves fewer kilos per cup sold.

Where it breaks: the chaiwala fetches only what the order needs; a GPU decoding one token must stream *every* weight of the model from memory, at every single step.

46.1 Compute accounting: 6ND and 2N

For a dense transformer with N parameters trained on D tokens:

$$C_{\text{train}} \approx 6ND \qquad (\text{forward } 2ND \;+\; \text{backward } 4ND), \qquad C_{\text{infer}} \approx 2N \text{ FLOPs per generated token}$$

Each parameter takes part in one multiply-add (2 FLOPs) per token in the forward pass; the backward pass computes gradients with respect to both activations and weights — twice the forward cost. Attention over a context of length T adds roughly $2\,n_{\text{layers}}\,T\,d$ FLOPs per token, which matters for long contexts but not for typical lengths.

Real-life analogy: every worker touches every item

Think of it like a factory with N workers and D items on the conveyor. Every worker handles every item: on the way in, each does one multiply and one add (2 operations, the forward pass). The item then comes back for inspection, and each worker does twice as much work, checking both what they received and their own settings (4 operations, the backward pass). Using the finished factory to make one new item needs only the forward trip.

Formula, read aloud: $C_{\text{train}} \approx 6ND$ says: training work = 6 × (number of parameters) × (number of training tokens); $C_{\text{infer}} \approx 2N$ says: each generated token costs about two operations per parameter.

Where it breaks: real factories can run near full speed; GPUs typically reach only 35–50% of peak (MFU), and attention adds work at long contexts, so time estimates need those corrections.

Worked example: how long does a 7B model take to train?

$C = 6 \times (7 \times 10^9) \times (2 \times 10^{12}\ \text{tokens}) = 8.4 \times 10^{22}$ FLOPs.

One H100 GPU peaks near 989 TFLOP/s in dense bf16. Real jobs reach a **model FLOPs utilization (MFU)** of perhaps 35–50%; at 40%: $\approx 4 \times 10^{14}$ FLOP/s per GPU.

GPU-seconds $= 8.4 \times 10^{22} / 3.96 \times 10^{14} \approx 2.1 \times 10^8$ → **≈ 59,000 GPU-hours** → about **2.4 days on 1,024 GPUs**.

Interview habit: state the formula, plug in, sanity-check units, and name the biggest uncertainty (here, MFU).

46.2 Memory accounting in training

With mixed-precision Adam, every parameter needs:

Item	Bytes/param
bf16/fp16 weights	2
bf16/fp16 gradients	2
fp32 master weights	4
Adam first moment m (fp32)	4
Adam second moment v (fp32)	4
Total (before activations)	**16**

A 7B model therefore needs about **112 GB** just for weights, gradients and optimizer state — more than one 80 GB GPU — before a single activation is stored. **Activations** scale with batch size × sequence length × hidden size × layers and often dominate at long contexts. Full fine-tuning is expensive for exactly this reason; LoRA (Part 43) removes the gradient and optimizer terms for the frozen weights.



Real-life analogy: a student's overloaded desk

Think of it like a student who, for every page of the textbook (each parameter), keeps five things on the desk: the page (weights, 2 bytes), pencilled corrections (gradients, 2), a neat full-detail master copy (fp32 weights, 4), a notebook of which way recent corrections pointed (Adam's m, 4), and one of how bumpy they were (Adam's v, 4): 16 bytes per page. The desk overflows. Fixes: a study group where each friend keeps a share of the notes (ZeRO/FSDP), redoing rough work instead of keeping it (checkpointing), and homework done in small sittings but submitted once (gradient accumulation).

Where it breaks: the rough work for each batch (activations) isn't in the 16 bytes; it grows with batch size and sequence length and often dominates.

Tools for fitting training in memory:

- **Gradient accumulation:** run several micro-batches, sum gradients, step once — larger effective batch at the same memory.
- **Activation (gradient) checkpointing:** store activations only at some layers and recompute the rest during backward — roughly one extra forward pass (~33% more compute) for a large memory saving.
- **Mixed precision:** compute in bf16/fp16, keep fp32 master weights.

- **ZeRO / FSDP:** instead of every data-parallel GPU holding a full copy of everything, **shard** it. ZeRO-1 shards optimizer state, ZeRO-2 also gradients, ZeRO-3 (≈ FSDP) also parameters, gathering them layer by layer when needed. With ZeRO-3 on G GPUs the 16 bytes/param become $\approx 16/G$: a 7B model on 8 GPUs → **14 GB per GPU** plus activations — at the cost of more communication.
- **CPU/NVMe offload:** move optimizer state off the GPU; slower but enables large models on few GPUs.

46.3 Number formats

Format	Sign–exponent–mantissa bits	Notes
fp32	1–8–23	reference precision; master weights, optimizer state
bf16	1–8–7	same *range* as fp32, less precision → the default for training; no loss scaling needed
fp16	1–5–10	more precision, small range → gradients underflow; needs **loss scaling**
fp8 (E4M3 / E5M2)	1–4–3 / 1–5–2	training and inference on newer accelerators with per-tensor or per-block scaling
int8	integer + scale	inference: weights (and sometimes activations)
int4 / 4-bit floats	integer or float + scale per group	weight-only inference; QLoRA's NF4

Loss scaling (fp16): multiply the loss by a large factor S before backward so small gradients don't underflow to zero, divide gradients by S before the update, and lower S whenever an overflow (inf/NaN) appears. bf16's 8-bit exponent makes this unnecessary — one reason it won.



Real-life analogy: weighing one grain of rice

Think of it like a weighing scale's display. Exponent bits set its *range*: how tiny and how huge a weight it can show. Mantissa bits set how many digits of detail it shows. bf16 keeps fp32's huge range but shows fewer digits; fp16 shows more digits but has a narrow range, so a tiny gradient reads as zero (underflow). Loss scaling is the grocer's trick for weighing one grain of rice: weigh a thousand grains, then divide by a thousand.

Where it breaks: a scale's ticks are evenly spaced; floating-point ticks get wider as numbers grow (fixed *relative* precision), which is why range and detail trade off for a fixed number of bits.

46.4 Parallelism

Strategy	What is split	Communication	Use when
Data parallel	the batch; each GPU has a full model copy	all-reduce gradients each step	model fits on one GPU
ZeRO / FSDP	batch + sharded states/params	all-gather params, reduce-scatter grads	model state doesn't fit, activations do
Tensor parallel	individual weight matrices (split columns/rows across GPUs)	all-reduce inside every layer → needs fast intra-node links	layers too large; latency-critical inference
Pipeline parallel	layers into stages across GPUs	activations between stages; "bubble" idle time reduced by micro-batching	very deep models across nodes

Strategy	What is split	Communication	Use when
Sequence / context parallel	the sequence dimension	exchange keys/values (e.g. ring attention)	very long contexts
Expert parallel	MoE experts across GPUs	all-to-all token routing	mixture-of-experts models

Large training runs combine them ("3D parallelism": tensor within a node, pipeline across nodes, data/FSDP across replicas). **Interview point:** the choice follows from what doesn't fit and from the bandwidth hierarchy — the chattiest parallelism goes on the fastest links.



Real-life analogy: painting a housing society

Think of it like painting a big housing society. Data parallel: identical crews each paint different buildings and share tips every evening (all-reduce). ZeRO/FSDP: crews share one set of ladders and tools, passing them around as needed. Tensor parallel: several painters split one huge wall and must talk constantly, so they stay on the same floor (fast links inside a node). Pipeline parallel: a primer team, then a paint team, then a polish team; the later teams wait at first (the bubble). Expert parallel: jobs are routed to specialist electricians and plumbers.

Where it breaks: painters can work alone for hours; GPUs must synchronise exactly and often, so communication can eat the gains from adding more of them.

46.5 Inference: prefill vs. decode

```
prompt (1,000 tokens) ⟶ PREFILL: all prompt tokens at once, matrix-matrix products
                          → compute-bound; fills the KV cache; sets time-to-first-token (TTFT)
then, per new token   ⟶ DECODE: one token per sequence, matrix-vector products
                          → reads all weights + KV cache every step
                          → memory-bandwidth-bound; sets time-per-output-token (TPOT)
```

Why decode is memory-bound. Generating one token for one sequence does ~$2N$ FLOPs but reads all N weights. For a 7B model in fp16 that's **14 GB per token**. On an H100 (~3.35 TB/s of HBM bandwidth) the upper bound at batch size 1 is

$$\frac{3.35 \times 10^{12}\ \text{B/s}}{14 \times 10^{9}\ \text{B/token}} \approx 240\ \text{tokens/s},$$

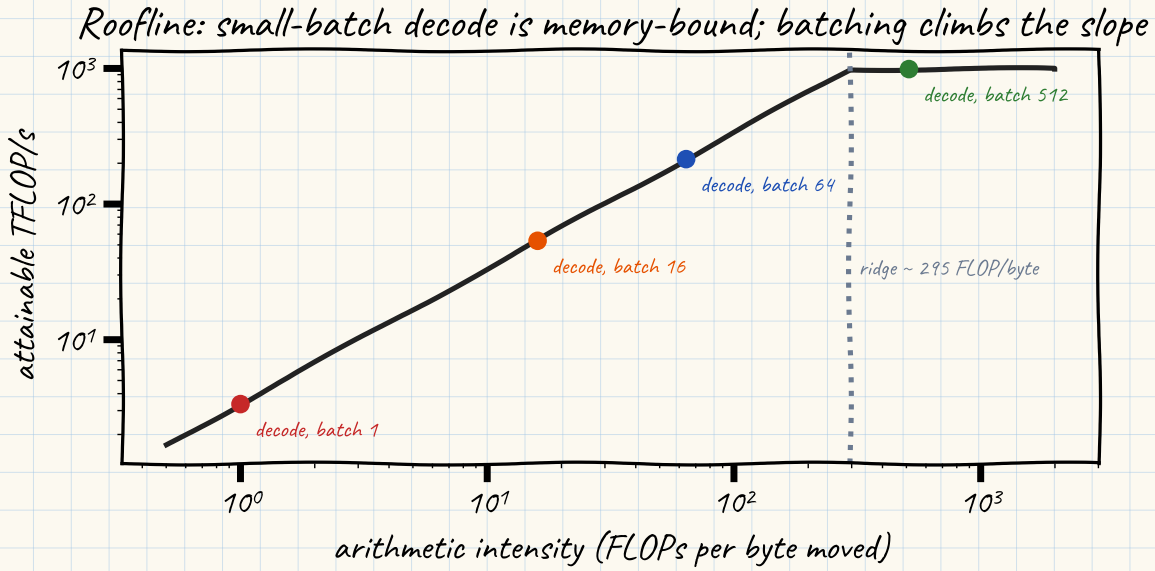
while the GPU's arithmetic units sit mostly idle. **Batching** b sequences reuses each weight read b times, raising arithmetic intensity to about b FLOPs per byte. The GPU becomes compute-bound only near the **ridge point** (peak FLOP/s ÷ bandwidth ≈ 989/3.35 ≈ 300 FLOPs/byte) — so serving throughput comes from batching many requests, limited by KV-cache memory and latency targets.

Real-life analogy: a bus that runs half-empty

Think of it like a city bus. Every trip, one decode step, means driving the whole route (reading every weight from memory), whether one passenger is aboard or forty. One passenger per trip wastes the bus; filling it (batching) makes each extra rider almost free, until the seats run out (KV-cache memory). Prefill is different: the whole prompt boards at once at the depot, so that part is limited by the engine (compute), not the route.

Formula, read aloud: $\frac{3.35\times10^{12}\ \text{B/s}}{14\times10^{9}\ \text{B/token}} \approx 240$ says: best tokens per second for a single user = (bytes memory can deliver per second) ÷ (bytes that must be read to make one token).

Where it breaks: a bus has fixed seats; here each passenger's seat (their KV cache) grows longer with every token of their conversation.



Roofline model for an H100-class GPU: attainable throughput is the minimum of peak compute and bandwidth × intensity

Serving metrics: TTFT (time to first token), TPOT or inter-token latency, end-to-end latency, throughput (tokens/s across users), and **goodput** — throughput of requests that meet their latency targets. Optimizing one often hurts another: bigger batches raise throughput and TPOT together.

46.6 The KV cache and its management

Per sequence (Part 41):

$$\text{KV bytes} = 2 \times n_{\text{layers}} \times n_{\text{kv heads}} \times d_{\text{head}} \times T \times \text{bytes per value}$$

Llama-2-7B-style (32 layers, 32 KV heads, $d_{\text{head}} = 128$) at 4K tokens in fp16: **2 GB per sequence** — a batch of 32 needs 64 GB, nearly a whole 80 GB GPU on top of 14 GB of weights. With GQA (8 KV heads), an 8K context costs **1 GB**. KV memory, not compute, usually caps the batch size.

Real-life analogy: a waiter's notepad for every table

Think of it like a waiter who keeps a notepad for each table, so he never re-asks for the whole order (the KV cache keeps past keys and values instead of recomputing them). The notepad grows with every item ordered (every token). Good restaurants hand out loose pages as needed, not a thick notebook per table (PagedAttention), seat a new party the moment any table frees up (continuous batching), and prepare the set thali everyone orders just once (prefix caching).

Formula, read aloud: the KV-bytes formula says: memory = 2 (keys and values) × layers × KV heads × (d_{head} numbers each) × tokens so far × bytes per number.

Where it breaks: a waiter's notes are short; a long conversation's cache can reach gigabytes, and that, not compute, usually caps how many users fit on a GPU.

Technique	Idea	Benefit
Continuous (in-flight) batching	schedule at each decode step: finished sequences leave, new ones join	no waiting for the slowest sequence; much higher utilization
PagedAttention	store KV in fixed-size blocks with a block table, like virtual-memory pages	little fragmentation; bigger batches; sharing blocks between sequences
Prefix caching	reuse the KV of a shared prefix (system prompt, documents, conversation history)	lower TTFT and cost for repeated prefixes
Chunked prefill	split long prompts into chunks interleaved with decode steps	long prompts don't stall other users' decoding
Prefill/decode disaggregation	run prefill and decode on separate GPU pools, transfer the KV cache	each phase tuned for its bottleneck; TTFT and TPOT stop interfering
KV quantization, GQA/MQA/MLA, sliding windows	fewer bytes per cached token	longer contexts, larger batches

46.7 Quantization

Post-training quantization (PTQ) converts a trained model's weights (and optionally activations) to fewer bits using a small calibration set; **quantization-aware training (QAT)** simulates quantization during training so the model adapts. Granularity matters: per-tensor scales are cheap but crude; **per-channel** or **per-group** (e.g. 64–128 weights share a scale) scales preserve accuracy.

Real-life analogy: shrinking a photo's colour palette

Think of it like saving a photo with only 256 colours instead of millions. Doing it after the photo is taken is post-training quantization; a photographer who shoots knowing the photo will be compressed is quantization-aware training. If one palette must cover the whole picture, a single blazing sun stretches it so far that all the shadow detail turns into one grey (activation outliers). Giving each small patch its own palette keeps the detail (per-channel or per-group scales).

Where it breaks: a photo only has to *look* right; a quantized model must *behave* right, and small errors can build up across layers and long reasoning, so you test on your own task.

```
def quantize_int8_per_channel(W):
    scale = np.abs(W).max(axis=0) / 127.0                    # one scale per output channel
    q = np.clip(np.round(W / scale), -127, 127).astype(np.int8)
    return q, scale                                          # dequantize: q * scale
```

Checked: on a random matrix the maximum reconstruction error is 0.39% of the largest weight.

Why LLMs are hard to quantize. A few activation channels carry values far larger than the rest ("outliers"); quantizing activations with one scale destroys the small values. Methods that address this:

- **Weight-only** quantization (int8/int4 weights, bf16 activations) — the usual choice for memory-bound decoding, because it cuts the bytes read per token.
- **SmoothQuant:** rescale channels to move activation outliers into the weights, where they're easier to quantize, enabling int8 weights *and* activations.
- **GPTQ:** quantize weights one column at a time and adjust the remaining columns to compensate for the error, using second-order (Hessian) information from calibration data.
- **AWQ:** protect the small fraction of weight channels that matter most, identified by activation magnitudes, by scaling them before quantization.

Memory math: 70B parameters × 2 bytes = **140 GB** in fp16; at 4 bits ≈ **35 GB** plus scales — the difference between two 80 GB GPUs and one. Always evaluate quantized models on *your* task, including long-context and reasoning slices, which can degrade before headline benchmarks do.

46.8 Speculative decoding

A small **draft** model proposes γ tokens; the large **target** model scores all of them in a single forward pass (cheap, because decode is memory-bound — verifying γ tokens costs about the same as generating one). Each proposed token x is accepted with probability $\min(1, p(x)/q(x))$; at the first rejection, a replacement is sampled from the normalized residual $\max(0, p - q)$. This makes the output distribution **exactly** the target's p.

With per-token acceptance rate $\alpha = \sum_x \min(p(x), q(x))$, the expected tokens per target pass is

$$\mathbb{E}[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha} \qquad (\alpha = 0.7,\ \gamma = 4: \ \approx 2.77)$$



Real-life analogy: accepting your phone's autocomplete

Think of it like typing with your phone's autocomplete. The keyboard suggests the next few words (the small draft model). You glance at them all at once (one pass of the big target model), accept them up to the first wrong word, then type the right word yourself. When suggestions are usually right you finish much faster; when they are usually wrong you gain little.

Formula, read aloud: $\frac{1-\alpha^{\gamma+1}}{1-\alpha} = 1 + \alpha + \alpha^2 + \cdots + \alpha^{\gamma}$ says: tokens per check = 1 guaranteed token + (chance the first draft is accepted) + (chance the first two are) + ...; each draft token counts only if all before it were accepted.

Where it breaks: you reject a suggestion on a whim; the target uses a precise accept-with-probability rule and resamples from the leftover distribution, so the output is exactly as if the big model wrote it alone.

```
def speculative_step(p_target, q_draft, rng):
    """One token of speculative sampling: output is distributed exactly as p_target."""
    x = rng.choice(len(q_draft), p=q_draft)                    # draft proposes
    if rng.random() < min(1.0, p_target[x] / q_draft[x]):      # target accepts
        return x, True
    resid = np.maximum(p_target - q_draft, 0)                  # else resample from the
residual
    return rng.choice(len(resid), p=resid / resid.sum()), False
```

Checked: with $p = (0.5, 0.3, 0.15, 0.05)$ and a uniform draft, 200,000 steps give the empirical distribution (0.501, 0.300, 0.149, 0.050) and acceptance 0.701 vs. theory $\sum \min(p, q) = 0.70$. Variants replace the separate draft

model with extra prediction heads on the target, or propose tokens copied from the prompt (useful for editing and RAG, where outputs repeat the input).

46.9 Cost engineering at the application layer

$$\text{cost per request} = T_{\text{in}} \cdot c_{\text{in}} + T_{\text{out}} \cdot c_{\text{out}}$$

Real-life analogy: a cab fare with two rates

Think of it like a cab fare with two rates: a cheap rate per kilometre on the open highway (input tokens, read in one fast pass) and a pricier rate in slow city traffic (output tokens, generated one by one). To cut the bill: shorten the route (trim prompts and context), cap the trip (limit output length), send a bike-taxi for small errands and a car only when needed (routing to smaller models), and re-use a trip already made to the same place (prefix and semantic caching).

Formula, read aloud: $T_{\text{in}}c_{\text{in}} + T_{\text{out}}c_{\text{out}}$ says: cost per request = (tokens sent × price per input token) + (tokens returned × the higher price per output token).

Where it breaks: a cab fare ignores past rides; with prompt caching, requests sharing a long prefix cost much less, so real bills depend on traffic patterns.

Illustrative: 1M requests/day, 1,500 input and 300 output tokens each, at $1 per million input tokens and $4 per million output tokens → $1,500 + $1,200 = **$2,700/day ≈ $81K/month**. Output tokens are usually priced higher because decode is the expensive phase.

Levers, roughly in order of effort: trim prompts and retrieved context; cap output length; use **prompt/prefix caching** for shared prefixes; **route** easy requests to a small model and escalate hard ones (a cascade with a confidence or verifier check); **semantic caching** of answers to near-duplicate questions (with care for personalization and freshness); batch offline workloads; **distill** a small model for a high-volume narrow task (Part 43); self-host with quantization when volume justifies it.

Q: "Our LLM feature is too slow — p95 latency is 9 seconds. What do you do?"

Strong answer. "Break the latency down first: queueing, retrieval and tool calls, time-to-first-token, and generation time (output length × time per token). Each has different fixes. If generation dominates, cut output length — tighter instructions, structured output, max tokens — and consider a smaller or distilled model, speculative decoding, or quantization, since decoding is memory-bound. If TTFT dominates, the prompt is probably long: trim retrieved context, cache shared prefixes, and use chunked prefill. If queueing dominates, we're under-provisioned or batching poorly — continuous batching, autoscaling, and separating long-prompt traffic. For the user experience, stream tokens so perceived latency is TTFT, and run independent steps in parallel. I'd check each change on the eval set so we don't buy latency with quality."

Follow-ups

"Why does doubling the batch size barely change per-token latency at small batches?" — *Because decode is memory-bound: the weight reads dominate and are shared across the batch, so extra sequences ride almost free until compute or KV memory becomes the limit.*

"Does speculative decoding change the model's outputs?" — *Not in distribution: the acceptance rule and residual resampling make the samples exactly distributed as the target's (for greedy decoding, the output is identical). It only changes speed, and only helps when the draft agrees with the target often.*

"Your 4-bit model matches fp16 on the benchmark but users report worse answers." — *Benchmarks may not cover long contexts, rare languages, or multi-step reasoning, where quantization error accumulates; evaluate on production-like slices, try per-group scales or 8-bit for sensitive layers.*

Active recall — Set 20: Efficient training & inference

Set 20 · Answer from memory, then check the Answer Key

Easy recall

1. Training FLOPs formula; inference FLOPs per token.
2. Bytes per parameter for mixed-precision Adam, itemized.
3. Write the KV-cache size formula.
4. What are TTFT and TPOT?
5. bf16 vs. fp16: which has more range, which more precision?

Conceptual

6. Why is decoding memory-bound while prefill is compute-bound?
7. What do ZeRO stages 1, 2 and 3 shard?
8. What problem does PagedAttention solve?
9. Why is weight-only quantization effective for decoding?
10. What is continuous batching?

Why

11. Why does fp16 training need loss scaling but bf16 usually doesn't?
12. Why put tensor parallelism inside a node?
13. Why does speculative decoding preserve the target distribution?
14. Why are output tokens typically priced higher than input tokens?
15. Why can activation outliers break int8 activation quantization?

Scenario

16. Fine-tuning a 13B model OOMs on 8 × 80 GB GPUs. Options, in order?
17. GPU utilization is 25% while serving. What's likely wrong?
18. Your chat app's system prompt is 3,000 tokens and identical for all users. What do you do?
19. Long-document summarization requests slow down everyone else's chat. Fix?
20. The finance team asks you to halve LLM spend without hurting quality. Plan?

Mathematical

21. FLOPs to train a 1B model on 100B tokens; GPU-hours at 400 TFLOP/s effective?
22. KV cache for 40 layers, 8 KV heads, head dim 128, 32K tokens, fp16?
23. Batch-1 decode upper bound for a 70B int4 model on 3.35 TB/s bandwidth?

Difficult follow-ups

24. Derive the expected tokens per target pass in speculative decoding.
25. How would you choose between a larger quantized model and a smaller full-precision one?

PART 47

Recommender Systems & Bandits

Recommenders in one breath

Pick a few items out of millions for one person, in under 100 ms, from feedback that is implicit, biased by what you showed before, and missing for new users and new items. The answer is a **funnel**: cheap retrieval narrows millions to thousands, a rich ranking model scores hundreds, and re-ranking applies diversity and business rules. Then **your own recommendations generate your next training data** — so exploration and bias correction are part of the design, not afterthoughts.

The whole Part as one picture

Think of it like a bookshop owner with a million books and ten seconds per customer. First, grab a few hundred likely books from the right shelves (candidate generation). Then look carefully at those for *this* customer: past purchases, today's mood, the season (ranking). Finally, make sure the pile isn't five books by one author, add a new arrival or two, and remove anything unsuitable (re-ranking). The catch: customers react only to books you showed them, so what you show today decides what you learn tomorrow, which is why you sometimes show something unproven (exploration).

Where it breaks: a shopkeeper can ask "did you like it?"; recommenders mostly see silent clicks and skips, for millions of people at once, in under 100 ms.

47.1 The funnel (a deeper look at Design 3)

```
millions of items
   |  CANDIDATE GENERATION (recall-oriented, ~10 ms): two-tower ANN, co-visitation, graph walks,
   |     recent/trending, subscriptions — several sources merged
~1,000-10,000
   |  (optional) LIGHT RANKER: small model to cut cost
~500
   |  RANKING (precision-oriented): large model, rich features incl. user x item crosses, multi-task
~100
   |  RE-RANKING: diversity, freshness, dedupe, policy/safety filters, business rules,
   |     exploration slots
~10-50 shown
```

Each stage has its own **objective, metric and latency budget**. Retrieval is judged on recall@k of what the ranker would like; ranking on nDCG/AUC/calibrated log-loss; the whole system on online metrics.

Real-life analogy: a recruitment funnel

Think of it like hiring from ten lakh applications. A quick résumé filter cuts them to a few thousand; its job is to not miss good people (recall). A short phone screen trims them to hundreds (the light ranker). Detailed interviews score the rest carefully (the ranking model, precision-oriented). The final offer list is balanced across teams and checked against policy (re-ranking with business rules). Each stage has its own goal, its own yardstick and its own time budget.

Where it breaks: hiring judges each candidate alone; a feed is judged as a whole slate (ten near-duplicates make a bad feed even if each is relevant), and the entire funnel must run in milliseconds.

47.2 Feedback: what the data really is

- **Explicit** (ratings, likes) is sparse; **implicit** (clicks, watch time, purchases, skips) is abundant but noisy.
- A missing interaction is **not** a negative: the user may never have seen the item (**exposure**). Implicit data is positive-unlabeled.
- Data is **missing not at random:** users rate what they chose to consume; the system logged only what it chose to show (**selection bias**) and in positions that affect clicks (**position bias**).
- Labels must be chosen deliberately: clicks reward clickbait; long-term signals (satisfaction surveys, return visits, completion) align better with user value.

Real-life analogy: reading a restaurant's order book

Think of it like a restaurant owner learning what diners like from the order book. A dish nobody ordered may simply sit on a menu page nobody turns to (not seen is not the same as disliked: exposure). Dishes on the first page get ordered more whatever their taste (position bias). Only diners who chose a dish can rate it (selection bias). And a dish with a tempting photo gets ordered a lot but is half left on the plate (clicks are not satisfaction), so clean plates and repeat visits are better signals.

Where it breaks: a restaurant has a few dozen dishes it can easily test; a recommender has millions of items, so most user–item pairs will never be seen at all.

47.3 Collaborative filtering and matrix factorization

Neighborhood methods: item-item similarity ("people who watched X also watched Y") computed from co-occurrence — simple, explainable, strong baselines.

Matrix factorization represents user u by $p_u \in \mathbb{R}^k$ and item i by $q_i \in \mathbb{R}^k$, with score $\hat{r}_{ui} = p_u^\top q_i$ (plus biases):

$$\min_{P,Q} \sum_{(u,i)\in \text{observed}} \left(r_{ui} - p_u^\top q_i\right)^2 + \lambda\left(\|P\|_F^2 + \|Q\|_F^2\right)$$

The problem is non-convex jointly, but **convex in P for fixed Q** (and vice versa) — each user's vector is a ridge regression: $p_u = (Q_u^\top Q_u + \lambda I)^{-1} Q_u^\top r_u$. **Alternating least squares (ALS)** alternates these closed-form solves and parallelizes across users and items.

Real-life analogy: hidden taste dials for films

Think of it like describing every viewer and every Bollywood film with the same few hidden dials: how much songs, action, comedy, family drama. A viewer's predicted liking for a film is how well their dials line up with the film's (the dot product $p_u^\top q_i$). ALS tunes the dials in turns: hold the films fixed and fit each viewer, then hold the viewers fixed and fit each film; each turn is a simple, exact calculation.

Formula, read aloud: $\min \sum (r_{ui} - p_u^\top q_i)^2 + \lambda(\|P\|_F^2 + \|Q\|_F^2)$ says: set all the dials so that, on the ratings actually observed, (actual − predicted)² is small, plus a penalty that stops the dials growing huge and overfitting.

Where it breaks: real dials would have names; the learned ones are whatever directions best explain the data and are rarely human-readable.

```
def als_explicit(R, mask, k=5, lam=0.1, n_iter=20, seed=0):
    rng = np.random.default_rng(seed)
    n_u, n_i = R.shape
    P, Q = rng.normal(0, 0.1, (n_u, k)), rng.normal(0, 0.1, (n_i, k))
    I = lam * np.eye(k)
    for _ in range(n_iter):
        for u in range(n_u):                                    # fix Q: ridge regression per
user
            Qi = Q[mask[u]]
            P[u] = np.linalg.solve(Qi.T @ Qi + I, Qi.T @ R[u, mask[u]])
        for i in range(n_i):                                    # fix P: ridge regression per
item
            Pu = P[mask[:, i]]
            Q[i] = np.linalg.solve(Pu.T @ Pu + I, Pu.T @ R[mask[:, i], i])
    return P, Q
```

Checked: on a rank-3, 60 × 40 matrix with 30% of entries observed, held-out RMSE is 0.165 vs. 1.81 for predicting the mean (noise level 0.1).

Implicit ALS (Hu, Koren & Volinsky) uses *all* user–item pairs: preference $p_{ui} = \mathbb{1}[r_{ui} > 0]$ weighted by confidence $c_{ui} = 1 + \alpha r_{ui}$, so unobserved pairs are weak negatives: $\min \sum_{u,i} c_{ui}(p_{ui} - x_u^\top y_i)^2 + \lambda(\cdot)$.

BPR (Bayesian Personalized Ranking) optimizes the *ranking* directly: an observed item i should outscore a sampled unobserved item j:

$$\max \sum_{(u,i,j)} \log \sigma\big(s_{ui} - s_{uj}\big) - \lambda\|\Theta\|^2$$

This is a smooth surrogate for per-user AUC. *Checked:* BPR on synthetic clustered preferences reaches a held-out per-user AUC of 0.995.

47.4 Two-tower retrieval

A **user tower** (user ID, history, context) and an **item tower** (item ID, content features) map into the same space; score = dot product. Item embeddings are precomputed and indexed for ANN search (Part 44), so retrieval over millions of items takes milliseconds.

Training with in-batch negatives: for a batch of B (user, positive item) pairs, treat the other $B - 1$ items as negatives and apply softmax cross-entropy over the batch. Problem: popular items appear as negatives far more often than uniform sampling would, so the model learns to **over-penalize popular items**. The **logQ correction** subtracts each item's log sampling probability from its logit:

$$s^c(u, i) = s(u, i) - \log q_i$$

Real-life analogy: a flavour map on a food app

Think of it like a food app that places every user and every dish on the same "flavour map" (user tower and item tower), so finding dishes near you is a quick map lookup (ANN). Training treats the other orders in each batch as "not this person's dish". But biryani is in almost every batch, so the model keeps learning "biryani is wrong for you", unfairly. The logQ correction discounts each dish by how often it shows up as a negative.

Formula, read aloud: $s^c(u,i) = s(u,i) - \log q_i$ says: corrected score = raw match score – log(how often this dish gets sampled into batches).

Where it breaks: a real map is flat and fixed; this one has dozens to hundreds of dimensions, keeps shifting as the towers train, and q_i itself must be estimated from the data stream.

```
def in_batch_softmax_loss(U, V, log_q=None):
    logits = U @ V.T                                  # (B, B): row u scores every in-batch item
    if log_q is not None:
        logits = logits - log_q[None, :]              # logQ correction for popularity-biased
negatives
    logits = logits - logits.max(1, keepdims=True)
    log_probs = logits - np.log(np.exp(logits).sum(1, keepdims=True))
    return -np.mean(np.diag(log_probs))               # positives sit on the diagonal
```

Checked: with uniform q the correction is a constant shift per row and leaves the loss unchanged; with popularity-skewed q it changes it. **Other practicalities:** mix in random negatives, mine hard negatives carefully (many are unlabeled positives), use content features for cold-start items, and refresh item embeddings as the item tower changes.

47.5 Ranking models

Ranking sees only hundreds of candidates, so it can afford rich features: user × item crosses, real-time context, sequence features, and counters (CTR of this item for similar users in the last hour).

Real-life analogy: IPL selectors after the scouts

Think of it like IPL selectors after scouts have shortlisted a hundred players. Now there is time for rich detail: recent form, this pitch, match-ups like "this batsman against left-arm spin" (feature crosses), and how similar players fared lately (real-time counters). Selectors predict several things at once, runs, strike rate, fielding (multi-task), and combine them with weights set by team strategy.

Formula, read aloud: $\text{score} = P(\text{click}) \cdot \mathbb{E}[\text{watch time} \mid \text{click}] + w_1 P(\text{like}) - w_2 P(\text{hide})$ says: final score = (chance of a click × expected watch time if clicked) + a bonus for likely likes – a penalty for likely hides.

Where it breaks: selectors pick one team per match after debate; a ranker scores hundreds of candidates for millions of users in milliseconds, with weights tuned by A/B tests.

Model	Key idea
GBDT on engineered features	strong baseline; handles heterogeneous features well
Wide & Deep	memorization (wide cross features) + generalization (deep embeddings)

Model	Key idea
DCN / DCN-V2	explicit feature crosses: $x_{l+1} = x_0 \odot (W_l x_l + b_l) + x_l$ — each layer raises the interaction order by one
DLRM	embeddings for sparse features, MLP for dense, pairwise dot-product interactions
DIN	target attention: weight the user's history by relevance to the *candidate* item
MMoE / PLE	multi-task learning with shared and task-specific experts plus per-task gates — predicts click, like, share, watch time jointly without one task dominating

Combining objectives: the final score is typically a weighted combination of predicted probabilities and values, e.g. $\text{score} = P(\text{click}) \cdot \mathbb{E}[\text{watch time} \mid \text{click}] + w_1 P(\text{like}) - w_2 P(\text{hide})$. The weights are product decisions tuned by online experiments. **Calibration** (Part 16) matters whenever predictions are multiplied, compared across candidates, or used in auctions.

47.6 Sequential and generative recommendation

- **Sequential models** treat the user's history as a sequence: SASRec uses a causal transformer to predict the next item; BERT4Rec uses masked-item prediction. They capture order and recency that bag-of-history features miss.
- **Generative retrieval with semantic IDs:** assign each item a short sequence of discrete codes learned from its content embedding (for example with a residual-quantized autoencoder), so similar items share code prefixes; a sequence model then *generates* the next item's code directly, and new items with content features get meaningful IDs immediately.
- **LLMs in recommendation:** generating item descriptions and features, embedding text-rich items, conversational recommendation, and zero-shot ranking for cold-start domains. Production systems still rely on specialized models for the latency- and cost-critical stages.



Real-life analogy: DJ playlists and PIN codes

Think of it like a good DJ who picks the next song from the order of the last few: three slow songs in a row call for something upbeat (sequential models predict the next item like the next word). Semantic IDs work like Indian PIN codes: the first digit is the region, the next ones narrow it down, so nearby places share a prefix. A model can then "write out" the next item's code digit by digit, and a brand-new item gets a meaningful code on day one from its content.

Where it breaks: PIN codes are assigned by hand from geography; semantic IDs are learned from content embeddings, so "shares a prefix" means "similar according to the model", which may differ from what users feel is similar.

47.7 Offline evaluation

$$\text{DCG@}k = \sum_{r=1}^{k} \frac{2^{\text{rel}_r} - 1}{\log_2(r+1)}, \qquad \text{nDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}$$

Real-life analogy: arranging a Diwali sale window

Think of it like arranging a shop window for a Diwali sale. Shoppers look at the front first, so a great item at the front earns a lot and the same item at the back earns less (the position discount). Very relevant items are worth far more than slightly relevant ones (the $2^{\text{rel}} - 1$ gain). nDCG compares your window with the best possible arrangement of the same items, so 1.0 means perfect.

Formula, read aloud: $\text{DCG@}k = \sum_r \frac{2^{\text{rel}_r}-1}{\log_2(r+1)}$ says: add up each shown item's value divided by a slowly growing penalty for its position r; nDCG divides that by the perfect ordering's score.

Where it breaks: a window is judged by real shoppers today; offline nDCG only has labels for items the old system showed, so it can't reward good items that were never displayed.

```python
def dcg_at_k(rels, k):
    rels = np.asarray(rels, float)[:k]
    return np.sum((2 ** rels - 1) / np.log2(np.arange(2, len(rels) + 2)))

def ndcg_at_k(rels_in_ranked_order, k, all_rels=None):
    ideal = sorted(all_rels if all_rels is not None else rels_in_ranked_order, reverse=True)
    idcg = dcg_at_k(ideal, k)
    return dcg_at_k(rels_in_ranked_order, k) / idcg if idcg > 0 else 0.0
```

Checked: relevances $[3, 2, 0, 1]$ in ranked order give nDCG@4 = 0.993 (with exponential gains; linear gains, as in scikit-learn's default, give 0.985 — state which convention you use).

Pitfalls interviewers look for:

- **Split by time**, not randomly — random splits leak the future (Part 17).
- **Sampled metrics** (rank the true item among 100 random negatives) can disagree with full-catalog metrics about which model is better (Krichene & Rendle, 2020). Prefer full ranking or correct for sampling.
- **Offline ≠ online:** logged data only contains items the old system showed; beyond-accuracy qualities (diversity, novelty, coverage, freshness) and long-term effects are invisible to nDCG. Offline metrics filter candidates; A/B tests decide.

47.8 Bias, feedback loops and counterfactual evaluation

Position bias: items at the top get clicked more regardless of relevance. Estimate it (randomize a small slice of traffic, or swap adjacent results) and correct with inverse propensity weights, or model position as a feature during training and set it to a fixed value at serving time.

Popularity bias and feedback loops: popular items get shown, clicked and retrained on, becoming more popular; niche items and new creators starve. Counter with exploration, calibrated diversity, and debiased training.

Counterfactual (off-policy) evaluation estimates how a *new* policy π would perform using logs collected by the *old* policy μ, provided the logs record the probability $\mu(a \mid x)$ of each action shown:

$$\hat{V}_{\text{IPS}}(\pi) = \frac{1}{n}\sum_{t=1}^{n} \frac{\pi(a_t \mid x_t)}{\mu(a_t \mid x_t)}\, r_t, \qquad \hat{V}_{\text{SNIPS}} = \frac{\sum_t w_t r_t}{\sum_t w_t}$$

Real-life analogy: reweighting an exit poll

Think of it like an exit poll whose interviewers mostly stood at city booths. To estimate the whole state, each village voter's answer counts extra: 1 ÷ (the chance a village voter was interviewed). IPS reweights logs the same way, by how much more (or less) often the new policy would pick each action than the old one did. And if no village voter was ever interviewed, no reweighting can recover them (the overlap requirement).

Formula, read aloud: $\hat{V}_{\mathrm{IPS}}(\pi) = \frac{1}{n}\sum_t \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)} r_t$ says: the new policy's value ≈ the average logged reward, each weighted by (the new policy's chance of that action) ÷ (the old policy's chance).

Where it breaks: poll weights come from known population counts; IPS weights explode when the old policy rarely chose an action, making estimates noisy, hence clipping, SNIPS and doubly robust methods.

Checked: a logging policy that favors a 2%-CTR arm yields a naive logged average of 0.028, while the target policy's true value is 0.052; IPS estimates 0.053 and SNIPS 0.054. IPS is unbiased but high-variance when weights are large — clip the weights, self-normalize (SNIPS), or combine with a reward model (**doubly robust**). All of this requires that the logging policy gave every action a nonzero probability (**overlap**) — a reason to keep some randomization in production.

47.9 Diversity and re-ranking

Maximal marginal relevance (MMR) greedily picks the item with the best trade-off between relevance and similarity to what's already chosen:

$$i^* = \arg\max_{i \notin S} \left[\lambda \,\mathrm{rel}(i) - (1-\lambda) \max_{j \in S} \mathrm{sim}(i,j)\right]$$

Real-life analogy: filling a thali

Think of it like filling a thali. The three tastiest dishes on offer might all be paneer curries, but a thali of three paneer curries is dull. So you pick one dish at a time: how good is it, minus how much it resembles something already on the plate? λ sets the balance: near 1 means "just the tastiest", lower means "more variety".

Formula, read aloud: $i^* = \arg\max_i [\lambda\,\mathrm{rel}(i) - (1-\lambda)\max_{j\in S}\mathrm{sim}(i,j)]$ says: next pick = the item with the best (λ × its relevance) − ($(1-\lambda)$ × its similarity to the closest item already chosen).

Where it breaks: on a plate, sameness is obvious; MMR's similarity comes from embeddings, which can miss duplicates users notice (the same video re-uploaded with a new title) or flag items that only look alike.

```python
def mmr_rerank(relevance, sim, lam=0.7, k=10):
    chosen, cand = [], list(range(len(relevance)))
    while cand and len(chosen) < k:
        best = max(cand, key=lambda i: lam * relevance[i]
                   - (1 - lam) * (max(sim[i, j] for j in chosen) if chosen else 0))
        chosen.append(best); cand.remove(best)
    return chosen
```

Checked: three near-duplicates with relevance 0.90, 0.89, 0.88 and two distinct items at 0.50 and 0.45 → pure relevance shows the three duplicates; MMR ($\lambda = 0.5$) shows items 0, 3, 4. Other re-ranking concerns: freshness boosts, author/creator diversity, safety and integrity filters, and exposure fairness for providers (Part 48).

47.10 Cold start

New...	Tactics
User	popular/trending by region and context; onboarding questions; session-based models that adapt within the first few interactions; exploration
Item	content features in the item tower (text, image, metadata embeddings); semantic IDs; guaranteed exploration impressions with a budget; similar-item bootstrapping
Whole system	heuristics and content-based models first; collect data with randomization so later models can learn and be evaluated off-policy



Real-life analogy: a radio station's newcomers

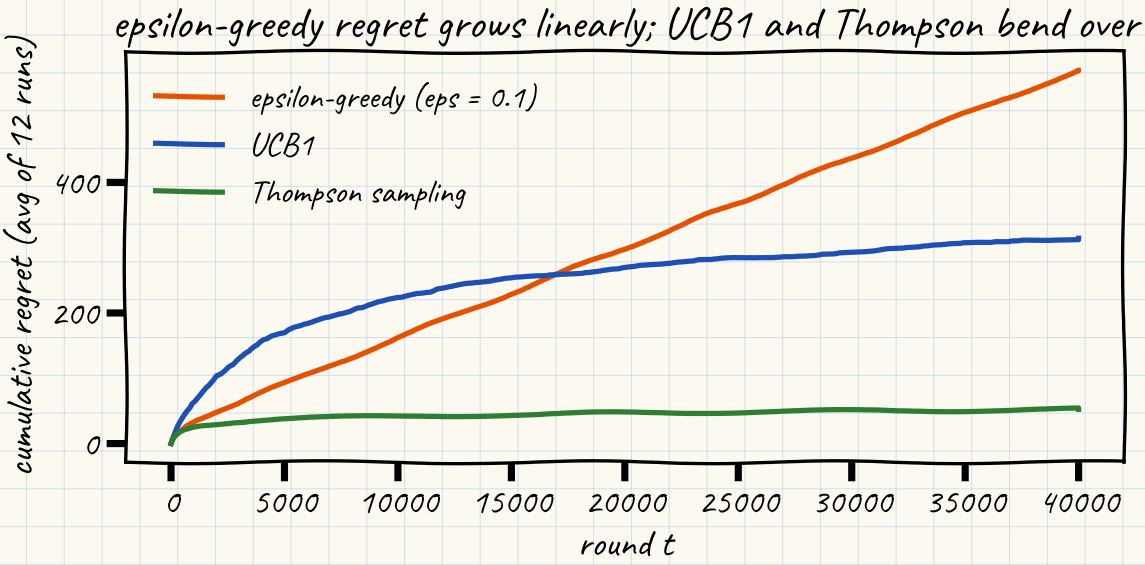
Think of it like a radio station. For a new listener, play the region's chart-toppers, ask "which genres do you like?", and adapt after the first few skips (new user). For a new song with no listening history, place it by genre, artist and tempo (content features), and give it a few guaranteed plays so listeners can react (exploration impressions). A brand-new station starts with simple rules and deliberately varied playlists, so it gathers data it can learn from later.

Where it breaks: a radio host can listen to a new song and judge it; a system with no interactions relies entirely on content features, which may not predict whether people will actually like it.

47.11 Bandits: learning while serving

A **multi-armed bandit** chooses among arms with unknown reward rates, trading off **exploration** (learn) against **exploitation** (earn). **Regret** after T rounds is the reward lost relative to always pulling the best arm: $R_T = T\mu^* - \sum_t \mu_{a_t}$.

Policy	Rule	Regret behavior
ε-greedy	with probability ε pick at random, else the best empirical arm	fixed ε keeps exploring forever → **linear** regret; decaying ε helps
UCB1	$a_t = \arg\max_a \ \bar{x}_a + \sqrt{2 \ln t / n_a}$ ("optimism under uncertainty")	logarithmic in T
Thompson sampling	sample $\theta_a \sim \mathrm{Beta}(1 + s_a,\ 1 + f_a)$, pick the argmax	logarithmic; strong in practice; handles delayed feedback and batching gracefully



Cumulative regret on a four-arm Bernoulli problem, averaged over 12 runs

Contextual bandits condition on features x. **LinUCB** fits a ridge regression per arm and adds an exploration bonus from the estimate's uncertainty:

$$a_t = \arg\max_a \ \hat{\theta}_a^\top x_t + \alpha\sqrt{x_t^\top A_a^{-1} x_t}, \qquad A_a = \lambda I + \sum xx^\top, \quad \hat{\theta}_a = A_a^{-1} b_a$$

Real-life analogy: choosing a route to office

Think of it like choosing your route to office each morning. Always taking the best route so far is exploitation; sometimes trying another is exploration, since it might be faster now. UCB is optimism: give a rarely tried route a bonus because you know little about it. Thompson sampling: imagine a plausible travel time for each route from past experience and take the best-looking one. A contextual bandit also reads today's conditions: in monsoon rain, the flyover route may win.

Formula, read aloud: $\hat{\theta}_a^\top x_t + \alpha\sqrt{x_t^\top A_a^{-1} x_t}$ says: pick the arm with the highest (predicted reward in today's context) + (α × how uncertain that prediction still is).

Where it breaks: your commute time arrives within the hour; real rewards (a purchase, a return visit) may come days later or be noisy, and the best arm can drift.

Checked (code/recsys.py): on a four-arm linear-logistic problem, LinUCB earns 0.886 reward per step over its last 1,000 rounds vs. 0.914 for the oracle and 0.506 for random choice.

Bandits vs. A/B tests: A/B tests give clean, unbiased estimates of a fixed difference for decision-making; bandits minimize the cost of exploration when you care about reward during the experiment (headlines, promotions, many short-lived arms). Bandit logs, with recorded propensities, also enable off-policy evaluation.

Q: "Offline nDCG improved 4%, but the A/B test shows lower engagement. Why?"

Strong answer. "First check the experiment itself — sample-ratio mismatch, logging changes, novelty effects, and whether the metric moved in every segment. Then the usual offline–online gaps: the offline set only contains items the old model chose to show, so the new model may be rewarded for agreeing with the old policy and never evaluated on what it would newly surface; the offline label (clicks) may not match the online objective (long sessions, return visits), so we may have optimized clickbait; there may be leakage in the offline split (random rather than time-based) or features computed differently online (training–serving skew); and diversity may have dropped — nDCG per request doesn't see a feed full of

near-duplicates. I'd look at per-slice results, feature parity logs, and diversity/coverage metrics, and consider off-policy estimators with propensities for future offline evaluation."

Follow-ups

"Why can't you just train on logged clicks and trust the model?" — *Logged clicks reflect what was shown and where; without correction the model learns the old policy's biases (position, popularity) and the feedback loop amplifies them.*

"How do you pick the number of candidates each retrieval source contributes?" — *Measure each source's marginal recall of items the ranker ends up favoring (and of eventual engagements), under a latency budget; tune the quotas online.*

"Thompson sampling or UCB in production?" — *Thompson is usually preferred: simple with Beta/Gaussian posteriors, empirically strong, and robust to batched, delayed updates because its randomization keeps exploring between updates.*

Active recall — Set 21: Recommender systems & bandits

Set 21 · Answer from memory, then check the Answer Key

Easy recall

1. Name the stages of a recommendation funnel and their goals.
2. Write the matrix-factorization objective.
3. Write the BPR objective.
4. Write DCG@k and nDCG@k.
5. Write the UCB1 rule.

Conceptual

6. Why is a missing interaction not a negative label?
7. Why does ALS have closed-form updates?
8. What problem does the logQ correction solve?
9. What is position bias and how can it be estimated?
10. What does off-policy evaluation require from logs?

Why

11. Why split recommender data by time?
12. Why do sampled metrics mislead?
13. Why does fixed-ε exploration have linear regret?
14. Why use multi-task ranking models?
15. Why do feedback loops increase popularity bias?

Scenario

16. New items get almost no impressions for their first week. Fix?
17. The feed is full of near-duplicate videos. Fix?
18. Users click more but session length drops after a launch. Diagnose.
19. You must evaluate a new ranking policy without an A/B test. How?
20. A marketplace wants new sellers to get fair exposure. What do you change?

Mathematical

21. Compute nDCG@3 for ranked relevances [0, 3, 1] (exponential gains).

22. Logged: action shown with probability 0.1, reward 1; target policy probability 0.5. IPS contribution?
23. Arm A: 3 pulls, mean 0.6; arm B: 10 pulls, mean 0.7; $t = 13$. Which does UCB1 choose?

Difficult follow-ups

24. Design a two-tower retrieval model for a new marketplace with heavy cold start.
25. How would you tune the weights of a multi-objective ranking score?

PART 48

Responsible AI: Fairness, Explainability, Privacy & Robustness

Responsible AI in one breath

A model that is accurate on average can still be unfair to a group, inexplicable to the person it affects, leaky about its training data, or brittle to an attacker. Each of these has **definitions you can compute, trade-offs you can quantify, and mitigations with costs**. Interviewers want to see that you measure before you claim, that you know which definitions conflict, and that you treat regulation and governance as engineering requirements.

The whole Part as one picture

Think of it like approving a new car model. Being fast on average isn't enough. Is it as safe for a small woman or a child as for the average crash-test dummy (fairness)? When a warning light comes on, does the manual explain why (explainability)? Does the car's app leak where its owners drive (privacy)? Can a thief or a pothole make it misbehave (robustness)? Each question has measurable tests, trade-offs and costs, and regulators expect the paperwork and crash-test reports before sale (governance).

Where it breaks: car safety has agreed-upon numbers; several fairness definitions contradict each other mathematically, so choosing one is a value judgement, not just an engineering test.

48.1 Fairness: where unfairness comes from

- **Historical bias:** labels encode past decisions (who was hired, who was policed).
- **Representation bias:** some groups are under-sampled; the model is worse for them.
- **Measurement bias:** the proxy label differs across groups (arrests ≠ crime; healthcare *cost* ≠ healthcare *need*).
- **Aggregation bias:** one model for groups with different relationships between features and outcome.
- **Deployment and feedback:** decisions change future data (loans denied → no repayment data).

Removing the protected attribute ("fairness through unawareness") doesn't work: correlated proxies (postcode, name, purchase history) reconstruct it — and without the attribute you can't even *measure* disparities.

Real-life analogy: a selection committee's old records

Think of it like training new cricket selectors only from old selection records. If past selectors ignored small-town players, the records say "good players come from big cities" (historical bias). If scouts rarely visited the North-East, there is too little data on those players (representation bias). If "talent" is measured by runs in well-funded leagues, the yardstick itself differs by background (measurement bias). And hiding each player's hometown doesn't help: the school, coach and club reveal it anyway (proxies defeat "unawareness").

Where it breaks: a human selector can be told to be fair and consciously adjust; a model absorbs patterns automatically, so fixing it means measuring per group and changing the data, the loss or the thresholds.

48.2 Fairness definitions (and why you can't have them all)

For prediction $\hat{Y}$, outcome Y, and group A:

Criterion	Condition	Meaning
Demographic parity (independence)	$P(\hat{Y}=1 \mid A=a)$ equal across a	same selection rate
Equal opportunity	$P(\hat{Y}=1 \mid Y=1, A=a)$ equal	same true-positive rate
Equalized odds (separation)	equal TPR **and** FPR	errors equally distributed
Predictive parity	$P(Y=1 \mid \hat{Y}=1, A=a)$ equal	a positive prediction means the same thing
Calibration within groups (sufficiency)	$P(Y=1 \mid S=s, A=a)=s$	a score of 0.7 means 70% for everyone
Individual fairness	similar individuals get similar predictions	needs a task-specific similarity metric

Impossibility results (Kleinberg–Mullainathan–Raghavan; Chouldechova): when base rates differ between groups, a non-perfect classifier cannot simultaneously be calibrated within groups and have equal false-positive and false-negative rates. Choosing a criterion is a **value judgment** about which errors matter, made with stakeholders — not a technical detail.

Real-life analogy: one screening test, two towns

Think of it like a disease screening test used in two towns, one where the disease is twice as common. Demographic parity asks: is the same share of people flagged in both towns? Equal opportunity asks: is a truly sick person equally likely to be caught in both? Predictive parity asks: does a positive result mean the same chance of being sick in both? When the towns' true rates differ, an imperfect test cannot make all of these equal at once.

Formula, read aloud: $P(\hat{Y}=1 \mid Y=1, A=a)$ equal across a says: among people who truly have the condition, every group is flagged equally often.

Where it breaks: in medicine, doctors often agree which error is worse; in fairness, which error matters is a value judgement made with stakeholders and the law, and the maths only proves you can't have everything.

Checked (code/responsible.py): two groups with base rates 0.20 and 0.40, a **perfectly calibrated** risk score and one shared threshold (0.35):

Group	Base rate	Selection rate	TPR	FPR	PPV
0	0.20	0.12	0.26	0.08	0.44
1	0.40	0.61	0.75	0.52	0.50

Calibration holds by construction, yet selection rates differ by 49 points and error rates by nearly 50 points. Equalizing TPR/FPR would require group-specific thresholds, which would break calibration of the decisions.

Mitigation points in the pipeline:

- **Pre-processing:** reweight or resample, fix label definitions, collect more data for under-served groups (often the most effective).
- **In-processing:** fairness constraints or penalties in the loss, adversarial debiasing.
- **Post-processing:** group-specific thresholds (legally sensitive in some domains — check), reject-option classification.
- **Always:** report metrics **per slice** with confidence intervals (small groups are noisy), and monitor them after launch.

48.3 Explainability

Question	Tool
Which features matter to the model overall?	permutation importance (Part 7), global SHAP summaries
Why this prediction?	**SHAP**, LIME, counterfactual explanations
How does the prediction change with a feature?	partial dependence, ICE, accumulated local effects
What in the input drove a neural net's output?	gradients × input, **integrated gradients**, attention *with care*
Is the model inherently understandable?	linear/logistic models, GAMs, small trees, monotonic GBDTs

Shapley values fairly split a prediction among features: feature j's value is its average marginal contribution over all orders in which features could be added:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \Big[v(S \cup \{j\}) - v(S) \Big]$$

The unique attribution with **efficiency** ($\sum_j \phi_j = f(x) - \mathbb{E}[f]$), symmetry, dummy and additivity. Exact computation is exponential (code in Part 21's second round: `shapley_exact`; for $f = 2x_0 + 3x_1x_2$ at $x = (1, 2, 1)$ it gives $(2, 3, 3)$, summing to $f(x) - f(0) = 8$ — the interaction is split equally). **TreeSHAP** computes it exactly and quickly for tree ensembles; **KernelSHAP** approximates it for any model.

Real-life analogy: splitting a shared cab fare

Think of it like three colleagues sharing a cab along one road, dropped at 5, 8 and 12 km. Imagine every order in which they could have joined the ride, note how much each person's joining added to the fare, and average. Those averages are the Shapley values, and they sum exactly to the total fare. For a model, riders are features and the "fare" is the prediction minus the average prediction.

Formula, read aloud: ϕ_j says: feature j's credit = a weighted average, over every group S of other features, of $v(S \cup \{j\}) - v(S)$ (prediction with j minus without), with weights that match averaging over joining orders.

Where it breaks: a fare is a real cost; for a model, "absent" features must be faked with a baseline, different baselines give different splits, and the splits explain the model, not the world.

Pitfalls: "feature absent" must be simulated by a baseline or background distribution, and the choice changes the answer; correlated features share or steal credit (interventional vs. conditional SHAP); explanations describe the **model**, not the world — they are not causal effects.

LIME fits a sparse linear model to the black box's predictions on perturbed samples near x, weighted by proximity. It's model-agnostic but unstable: explanations vary with the kernel width and random perturbations.

Real-life analogy: a ghat road's straight stretch

Think of it like a winding ghat road. You can't describe the whole road with one straight line, but the few metres around where you stand look almost straight. LIME does this for one prediction: it tries small variations of the input nearby, sees how the black box responds, and fits a simple straight-line model that holds only in that neighbourhood. The explanation is local by design.

Where it breaks: your stretch of road is a fact; LIME's "nearby" region and its random samples are choices, so the explanation can change noticeably from one run or kernel width to the next.

Integrated gradients attribute a neural network's output by integrating gradients along a straight path from a baseline x':

$$\mathrm{IG}_j(x) = (x_j - x'_j) \int_0^1 \frac{\partial f\big(x' + \alpha(x - x')\big)}{\partial x_j}\, d\alpha$$

It satisfies **completeness**: attributions sum to $f(x) - f(x')$. *Checked:* on a small nonlinear model, the attributions $(0.243, -0.211, 0.227)$ sum to 0.25951, matching $f(x) - f(x')$ to five decimals. Plain gradients don't have this property — they saturate where the model is flat.

Real-life analogy: turning water into dal

Think of it like crediting the ingredients of a dal. Start from plain water (the baseline) and move towards the finished dal, adding every ingredient gradually at the same pace. At each moment, note how much the taste changes because of each ingredient, and add up those small changes. The credits then sum exactly to (dal taste − water taste): completeness. Tasting only the finished dal misleads: more salt in salty dal changes little (plain gradients saturate).

Formula, read aloud: $\mathrm{IG}_j = (x_j - x'_j) \int_0^1 \frac{\partial f}{\partial x_j}\, d\alpha$ says: credit for j = (how far j moved from the baseline) × (the output's average sensitivity to j along the straight path).

Where it breaks: dal has a natural starting point; for images or text the baseline (a black image? blank text?) is a choice, and a different choice gives different credits.

Attention weights are not explanations by themselves: different attention patterns can produce the same outputs, and the value vectors and later layers transform what attention selects.

48.4 Privacy

Threats: membership inference (was this person in the training set?), training-data extraction (LLMs regurgitating memorized text, including PII), attribute inference, and re-identification of "anonymized" data by linking datasets.

Differential privacy (DP) bounds how much any one person's data can change the output. A randomized mechanism M is (ε, δ)**-DP** if for all datasets D, D' differing in one person and all output sets O:

$$P\big(M(D) \in O\big) \le e^{\varepsilon}\, P\big(M(D') \in O\big) + \delta$$

Real-life analogy: a noisy salary average

Think of it like HR announcing a team's average salary. By comparing the figure before and after Ravi joined, you could work out Ravi's salary. Differential privacy adds just enough random noise that the announcement looks almost the same whether or not Ravi is in the data, so nobody can tell. The more one person can change the answer (sensitivity), the more noise you need; the smaller ε, the stronger the protection.

Formula, read aloud: $P(M(D) \in O) \le e^{\varepsilon} P(M(D') \in O) + \delta$ says: the chance of any result with you in the data is at most e^{ε} times (roughly $1+\varepsilon$ for small ε) the chance without you, plus a tiny slack δ.

Where it breaks: one noisy announcement feels safe, but every extra release leaks a little more: privacy loss adds up across queries (composition), so the budget ε gets spent.

- **Laplace mechanism:** release $f(D) + \text{Laplace}(\Delta f/\varepsilon)$, where the sensitivity Δf is the most one person can change f. For a count, $\Delta f = 1$. *Checked:* at $\varepsilon = 0.5$ the noise variance is $2(\Delta/\varepsilon)^2 = 8$ (empirical 7.94).
- **Randomized response (local DP):** each person tells the truth with probability $e^{\varepsilon}/(1+e^{\varepsilon})$ and lies otherwise; the population rate is recoverable: $\hat{\pi} = \frac{\bar{r}-(1-p)}{2p-1}$. *Checked:* with $\varepsilon = \ln 3$ ($p = 0.75$) and a true rate of 0.297, the estimate is 0.296 — no individual answer can be trusted, yet the aggregate is accurate.
- **Composition:** privacy loss adds up across queries — the budget ε is spent, and advanced accounting tracks it tightly across many training steps.
- **DP-SGD:** clip each example's gradient to norm C (bounding its influence), add Gaussian noise $\mathcal{N}(0, \sigma^2 C^2)$ to the sum, and average. This needs *per-example* gradients (more memory and compute), and typically costs accuracy, especially for under-represented groups.

```
def dp_sgd_step(w, per_example_grads, lr, clip, noise_multiplier, rng):
    norms = np.linalg.norm(per_example_grads, axis=1, keepdims=True)
    clipped = per_example_grads * np.minimum(1.0, clip / np.maximum(norms, 1e-12))
    noisy = clipped.sum(0) + rng.normal(0, noise_multiplier * clip, w.shape)
    return w - lr * noisy / len(per_example_grads)
```

Real-life analogy: a society vote with a cap

Think of it like a housing-society meeting updating a rule. Each resident's opinion can move the decision only up to a fixed limit, however loudly they shout (clipping each example's gradient to norm C). Before counting, the secretary mixes in some random extra votes (Gaussian noise), so nobody can tell from the result how any single resident voted. The society then acts on the average (the update step).

Where it breaks: residents are equal voters; in DP-SGD, clipping hits examples with large gradients hardest, often rare or minority cases, so accuracy can drop most for under-represented groups, and per-example gradients cost extra memory and compute.

Federated learning keeps raw data on devices: clients train locally and send model updates, the server averages them weighted by data size (**FedAvg**). *Checked:* on linear regression with five non-IID clients, FedAvg converges to within 10^{-4} of the centralized solution. Updates can still leak information, so production systems combine federated learning with **secure aggregation** and DP. Challenges: non-IID data, stragglers, communication cost, and debugging without seeing data.

Real-life analogy: a shared recipe, secret masalas

Think of it like home cooks improving a shared recipe without revealing their family masala. Each cook tests the current recipe in their own kitchen (local training on the device) and sends back only suggested tweaks (model updates), never their ingredients (raw data). The organiser averages the tweaks, giving more weight to cooks who tested on more meals (FedAvg), and sends out the new recipe.

Where it breaks: the tweaks can still hint at the secret masala (updates leak information, hence secure aggregation and DP), and each kitchen's tastes differ (non-IID data), so the average may suit no one perfectly.

Practical privacy engineering: data minimization and retention limits, PII detection and redaction before training and logging, access controls, deduplication (repeated text is memorized more), and extraction/membership tests before release.

48.5 Robustness and security

- **Adversarial examples:** tiny, targeted perturbations flip predictions. **FGSM:** $x' = x + \epsilon\,\mathrm{sign}(\nabla_x L)$. *Checked* on logistic regression over 20 standardized features: accuracy falls from 0.76 to 0.50 at $\epsilon = 0.3$ under FGSM, but stays at 0.75 under random-sign noise of the same size — the *direction* matters, not the magnitude. Defenses: adversarial training (train on attacked examples), certified methods for small perturbations, input preprocessing (often broken by adaptive attacks).
- **Data poisoning and backdoors:** attackers insert training examples that degrade the model or plant a trigger. Defenses: data provenance, deduplication and filtering, anomaly detection on training data, and evaluating on trigger-like patterns.
- **Model extraction:** repeated querying to clone a model; rate limits and monitoring.
- **LLM-specific:** jailbreaks and prompt injection (Parts 44–45).
- **Distribution shift** is the everyday robustness problem (Parts 22–23): monitor, and evaluate on shifted slices before launch.

Real-life analogy: the skilled pickpocket

Think of it like a crowded Mumbai local train. Random jostling rarely costs you your wallet, but a skilled pickpocket bumps you in exactly the right spot (adversarial examples: the *direction* matters, not the size). Data poisoning is slipping wrong answers into the guidebook a class studies from; a backdoor is a secret password that makes the guard wave anyone through. Model extraction is copying a restaurant's recipe by ordering every dish many times.

Formula, read aloud: $x' = x + \epsilon\,\mathrm{sign}(\nabla_x L)$ says: nudge every input feature by the same small amount ϵ, each in whichever direction raises the model's error.

Where it breaks: a pickpocket must be near you; digital attacks can be invisible to humans, can transfer between models, and adapt to defences that merely preprocess inputs.

48.6 Documentation, governance and regulation

- **Model cards:** intended use, out-of-scope uses, training data summary, evaluation results **by slice**, limitations, ethical considerations. **Datasheets for datasets:** provenance, collection process, consent, known biases.

- **Risk assessment before launch:** who could be harmed and how, severity × likelihood, mitigations, human oversight, appeal routes for affected people.
- **Audit trails:** versioned data, models and decisions, so an individual decision can be reconstructed.



Real-life analogy: labels and licences for packaged food

Think of it like packaged food. The label lists ingredients, nutrition and warnings such as "not for children under 3" (a model card: intended use, out-of-scope uses, results by slice). A record of where each ingredient came from is the datasheet. The factory passes a safety inspection before selling (risk assessment) and keeps batch records so a bad packet can be traced (audit trails). Regulators sort products by risk, some banned, some tightly controlled, most free to sell, much as the EU AI Act sorts AI uses.

Where it breaks: a food label is printed once; models are retrained and data drifts, so cards, risk reviews and per-slice metrics need regular updates, and the rules themselves, like the AI Act's timeline, keep changing.

EU AI Act (as of September 2026 — not legal advice; verify current status). The Act uses a risk-based approach: prohibited practices, high-risk systems (for example in employment, credit scoring, education, essential services, and safety components of regulated products) with requirements for risk management, data governance, documentation, logging, human oversight, accuracy and robustness, plus transparency duties and obligations for general-purpose AI models. Timeline:

Date	Milestone
1 Aug 2024	entered into force
2 Feb 2025	prohibitions and AI-literacy provisions apply
2 Aug 2025	obligations for general-purpose AI models apply
2 Aug 2026	most remaining provisions apply, including Article 50 transparency duties
27 Jul 2026	"Digital Omnibus on AI" amendments entered into force
2 Dec 2027	high-risk obligations for stand-alone (Annex III) systems — **postponed** from 2 Aug 2026 by the Omnibus
2 Aug 2028	high-risk obligations for AI in regulated products (Annex I) — postponed

The Omnibus also softened the AI-literacy duty (providers and deployers must *support* AI literacy among staff), added a prohibition on systems generating non-consensual intimate imagery and child sexual abuse material, and broadened the legal basis for processing special-category data to detect bias. Other jurisdictions have their own rules (sector regulators, anti-discrimination and privacy law), so the interview-ready habit is: identify the use case's risk category early, and design for documentation, logging, human oversight and per-group evaluation from day one.

Q: "Your credit model approves group B at half the rate of group A. What do you do?"

Strong answer. "First, measure properly: approval rates, but also TPR/FPR and calibration by group, with confidence intervals, and check whether the gap is explained by legitimate, legally permitted factors. Then find the source: is the label biased (for example, repayment data only exists for past approvals — a selection problem), are features acting as proxies, is group B under-represented so the model is simply less accurate for them? Mitigations depend on the source: better data and reject inference for selection bias, removing or constraining proxy features, reweighting, fairness constraints, or thresholds — but I'd involve legal and policy teams because credit is regulated and the right fairness criterion is a

policy choice. I'd document the trade-off between accuracy and each fairness metric, add per-group monitoring, and provide explanations and appeal paths for applicants."

Follow-ups

"Why not just drop the protected attribute?" — Proxies reconstruct it, so disparities remain, and you lose the ability to measure them. Many fairness methods need the attribute at training or evaluation time (with appropriate legal basis and safeguards).

"Are SHAP values causal?" — No. They explain how the model uses features given a chosen baseline; changing a feature in the world may change other features too. For causal questions use causal methods (Part 4).

"What does $\varepsilon = 1$ mean in differential privacy?" — Any single person's presence changes the probability of any outcome by at most a factor of $e \approx 2.7$ (plus δ). Smaller ε is stronger privacy and more noise; the budget accumulates over queries and training steps.

PART 49

Other Frontiers: Forecasting, Graphs, Generative & Causal ML, RL

Why this part exists

Generalist ML loops increasingly include one "breadth" question from a neighboring field: forecast this, model that graph, explain diffusion, estimate an uplift, or define a Bellman equation. You don't need to be a specialist; you need the **core formulation, the one classic pitfall, and a sane baseline** for each. That is what this part gives you.

The whole Part as one picture

Think of it like a good family doctor. They aren't a cardiologist, a neurologist and a paediatrician, but for each specialty they know the core test, the classic mistake to avoid, and a safe first treatment. This Part gives you the same for six neighbouring fields: forecasting (never peek at the future), graphs (learn from neighbours), generative models (VAE, GAN, diffusion), self-supervised learning (make labels from the data itself), causal ML (who is actually *persuaded*), and reinforcement learning (act now for rewards later).

Where it breaks: a family doctor can refer you onward; in an interview you must reason each field through yourself, from first principles.

49.1 Time series and forecasting

What's different: observations are ordered and dependent; the future can't be used to predict the past; distributions drift (trend, seasonality, holidays, regime changes).

Real-life analogy: a mithai shop before Diwali

Think of it like a sweet shop planning tomorrow's batch. Sales follow weekly patterns and jump before Diwali (seasonality, holidays). A surprisingly good first guess is "same as this day last week" (the seasonal-naive baseline). The classic mistake is peeking: when planning a week ahead, you can't use yesterday's sales, because on planning day that "yesterday" hasn't happened yet (feature leakage). So you test yourself honestly: forecast, wait, compare, step forward (walk-forward validation).

Formula, read aloud: $\hat{y}_t = y_{t-m}$ says: predict today's value = the value one season ago ($m = 7$ for daily data with a weekly pattern); MASE below 1 means you beat that guess.

Where it breaks: a shopkeeper can use judgement for a one-off local bandh; a model needs such events as explicit features, and a regime change (a pandemic) breaks patterns learned from the past.

Approach	When it shines
Naive / seasonal-naive ($\hat{y}_t = y_{t-m}$)	always the first baseline — surprisingly hard to beat

Approach	When it shines
Exponential smoothing (ETS), ARIMA	single series, stable patterns, small data, interpretability
GBDT with lag/rolling/calendar features	many related series (retail, demand), exogenous features, strong in forecasting competitions
Global deep models (seq2seq, temporal transformers)	thousands of series sharing patterns, rich covariates
Pretrained time-series foundation models	zero-shot baselines for new series; validate carefully against simple baselines

Stationarity: ARIMA assumes a stationary series (after differencing); ACF/PACF plots guide the orders. **Seasonality** at period m (7 for daily data with weekly patterns).

The classic pitfall — leakage through features. Every lag and rolling statistic must use only information available at forecast time. A 7-day-ahead forecast can't use yesterday's value as "lag 1" for day 7. Build features by **shifting before rolling**, and use **walk-forward (expanding-window) validation**, never random K-fold:

```
def lag_features(y, lags=(1, 7), windows=(7,)):
    T = len(y); cols = []
    for L in lags:
        cols.append(np.r_[np.full(L, np.nan), y[:-L]])
    for w in windows:
        c = np.convolve(y, np.ones(w) / w, mode="full")[:T]          # mean of y[t-w+1..t]
        cols.append(np.r_[np.nan, c[:-1]])                           # shift: at time t use
y[..t-1]
        cols[-1][:w] = np.nan
    return np.column_stack(cols)

def mase(y_true, y_pred, y_train, m=1):
    scale = np.mean(np.abs(y_train[m:] - y_train[:-m]))              # in-sample seasonal-naive
MAE
    return np.mean(np.abs(y_true - y_pred)) / scale
```

Metrics: MAE/RMSE are scale-dependent; MAPE explodes near zero and penalizes over- and under-forecasts asymmetrically; **MASE** scales errors by the seasonal-naive error, so MASE < 1 means "better than naive". For inventory decisions, forecast **quantiles** with the pinball loss (Part 15) — the cost asymmetry decides the quantile. *Checked:* on a trending weekly-seasonal series, a ridge model on shifted lag features scores one-step-ahead walk-forward MASE 0.89 vs. 0.96 for seasonal naive — better, but not by much, which is typical.

Other must-knows: hierarchical forecasts (store → region → total) should add up (reconciliation); intermittent demand (many zeros) needs specialized methods or Tweedie/Poisson losses; holidays and promotions are features, not noise; always report the forecast horizon.

49.2 Graph machine learning

Data where relationships matter: social networks, molecules, fraud rings, knowledge graphs, road networks, user–item graphs.

Message passing: each node updates its representation by aggregating its neighbors':

$$h_v^{(l+1)} = \phi\Big(h_v^{(l)}, \bigoplus_{u \in \mathcal{N}(v)} \psi\big(h_v^{(l)}, h_u^{(l)}\big)\Big)$$

Real-life analogy: opinions spreading through a colony

Think of it like opinions in a housing colony. Each round, every resident updates their view by combining their own opinion with a summary of what their neighbours think (message passing). After two rounds you have heard your neighbours' neighbours too. After too many rounds everyone holds the same blended opinion (over-smoothing), so GNNs stay 2–3 layers deep. And it fails when neighbours differ on purpose: a fraudster's contacts are mostly victims (heterophily).

Formula, read aloud: $h_v^{(l+1)} = \phi(h_v^{(l)}, \bigoplus_u \psi(h_v^{(l)}, h_u^{(l)}))$ says: a node's new description = combine (its current description) with (a summary, such as sum, mean or max, of messages from each neighbour).

Where it breaks: people choose whom to trust; a basic GCN averages neighbours with fixed weights (GAT learns attention weights), and it only "hears" as many hops as it has layers.

GCN (a specific, simple case): $H^{(l+1)} = \sigma\big(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\big)$ with $\tilde{A} = A + I$ (self-loops) and $\tilde{D}$ its degree matrix — a normalized neighborhood average followed by a shared linear map.

```
def gcn_propagation(A):
    A_hat = A + np.eye(len(A))                          # self-loops keep each node's own features
    d = A_hat.sum(1)
    return A_hat / np.sqrt(np.outer(d, d))              # D^-1/2 (A + I) D^-1/2

def gcn_layer(S, H, W, act=lambda z: np.maximum(z, 0)):
    return act(S @ H @ W)
```

Checked: on a two-community random graph with weakly informative node features, logistic regression gets 0.78 accuracy on raw features, 0.96 after one propagation step (SX) and 0.98 after two (S^2X) — neighbors carry the signal (**homophily**).

Variant / issue	Key point
GraphSAGE	samples a fixed number of neighbors; inductive (works on unseen nodes) — needed at web scale
GAT	attention weights over neighbors instead of fixed normalization
Over-smoothing	many layers make all node embeddings converge; keep GNNs shallow (2–3 layers), use residuals
Heterophily	when neighbors differ (fraudsters connect to victims), plain averaging hurts
Tasks	node classification, link prediction (recommendation, knowledge-graph completion), graph classification (molecules)
Leakage	in link prediction, remove test edges from the message-passing graph

49.3 Generative models: VAEs, GANs, diffusion

Variational autoencoders. An encoder $q_\phi(z \mid x)$ and a decoder $p_\theta(x \mid z)$ are trained to maximize the **evidence lower bound**:

$$\log p_\theta(x) \;\geq\; \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

Reconstruction plus a regularizer pulling the latent code toward the prior. For a Gaussian encoder and standard-normal prior, the KL term is closed-form: $\frac{1}{2}\sum_j(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2)$. Gradients flow through sampling via the **reparameterization trick** $z = \mu + \sigma \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$. *Checked:* closed-form KL 1.0304 vs. Monte Carlo 1.0286 (400K samples). VAEs give blurry samples but smooth latent spaces; their autoencoders compress images for latent diffusion.

Real-life analogy: a witness and a sketch artist

Think of it like a witness describing a face with a few standard features (the encoder turns x into a short code z) and a sketch artist drawing a face from that description (the decoder). Reconstruction: the sketch should match the face. The KL term: descriptions must stay within the standard form's usual ranges (the prior), so *any* made-up description still yields a sensible face, which lets you generate new faces. Short descriptions lose detail, so sketches come out blurry.

Formula, read aloud: the ELBO says: how well the model explains x ≥ (how well the decoder rebuilds x from its code) − (how far the codes stray from the standard bell curve).

Where it breaks: a witness gives one fixed description; the encoder gives a *range* (a mean and a spread), and its features are learned and unnamed.

GANs. A generator G tries to fool a discriminator D:

$$\min_G \max_D \; \mathbb{E}_x\big[\log D(x)\big] + \mathbb{E}_z\big[\log\big(1 - D(G(z))\big)\big]$$

For a fixed G, the optimal $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x)+p_g(x)}$; plugging it in, the generator minimizes the Jensen–Shannon divergence. In practice: the **non-saturating** generator loss $-\log D(G(z))$ avoids vanishing gradients early on; training is unstable; **mode collapse** (the generator covers few modes) is the signature failure. GANs are fast at sampling but have largely been displaced by diffusion for image quality and diversity.

Real-life analogy: a forger versus a bank cashier

Think of it like a forger printing fake 500-rupee notes (the generator) and a bank cashier checking every note (the discriminator). The cashier gets better at spotting fakes; the forger gets better at fooling the cashier; each improves because the other does. Mode collapse is a forger who masters one perfect note and never tries 100s or 200s: every fake looks the same.

Formula, read aloud: $\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$ says: the cashier (D) tries to maximise its score for calling real notes real and fakes fake; the forger (G) tries to minimise that same score.

Where it breaks: a real contest can end with a winner; GAN training often oscillates instead of settling, and at the ideal end point the cashier can do no better than a coin toss.

Diffusion models. A fixed forward process adds Gaussian noise over T steps, with a closed form for any step:

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar\alpha_t}\, x_0,\; (1-\bar\alpha_t) I\big), \qquad \bar\alpha_t = \prod_{s \le t} (1 - \beta_s)$$

A network learns to predict the added noise, with a simple regression loss:

$$\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon}\Big[\big\| \epsilon - \epsilon_\theta\big(\sqrt{\bar\alpha_t} x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon,\; t\big)\big\|^2\Big]$$

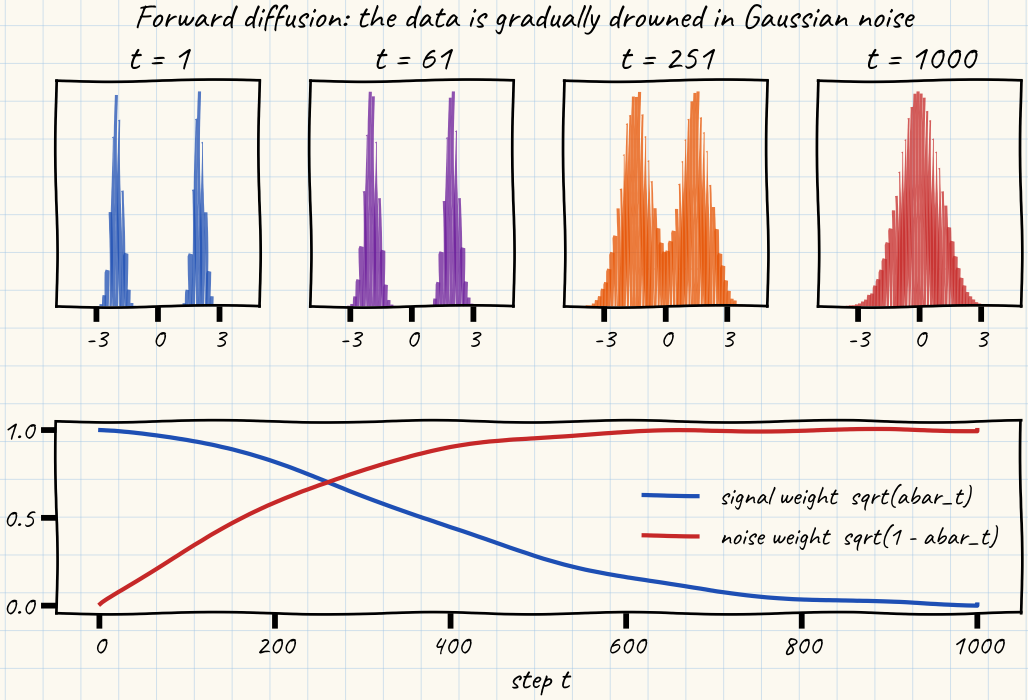
Sampling starts from pure noise and applies the learned denoiser step by step (dozens to hundreds of network evaluations; distillation and better solvers cut this to a few).

Real-life analogy: cleaning static off a TV picture

Think of it like adding TV static to a clear photo, a little at a time, until only snow is left (the fixed forward process). You then train a restorer: shown a noisy photo and told how much static was added (t), it must guess exactly which static was added. To create a new image, start from pure snow and let the restorer remove a little static, step by step.

Formula, read aloud: $\mathcal{L} = \mathbb{E}\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, t)\|^2$ says: mix a clean image (weight $\sqrt{\bar{\alpha}_t}$) with random static (weight $\sqrt{1-\bar{\alpha}_t}$), ask the network to name the static, and penalise the squared error of its guess.

Where it breaks: a restorer recovers *the* original photo; starting from fresh random snow, a diffusion sampler creates a *new* image that looks as if it came from the training data.



Forward diffusion on a two-mode 1-D distribution with a linear beta schedule ($T = 1000$)

Checked (code/frontiers.py): the closed form and 300 iterated noising steps agree (variance 2.19 vs. 2.20), and $\bar{\alpha}_{1000} \approx 4 \times 10^{-5}$ — essentially pure noise.

- **Conditioning and classifier-free guidance:** train with the condition c (a text embedding) randomly dropped, then sample with $\tilde{\epsilon} = \epsilon_\theta(x_t, \varnothing) + w\big(\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \varnothing)\big)$; larger w follows the prompt more closely at the cost of diversity.
- **Latent diffusion:** run diffusion in a compressed autoencoder latent space — far cheaper at high resolution.
- **Flow matching / rectified flows:** learn a velocity field that transports noise to data along (nearly) straight paths — a closely related, widely used formulation that allows few-step sampling.
- **Evaluation:** FID compares feature statistics of real and generated images; human preference and prompt-alignment scores complement it.

49.4 Self-supervised and contrastive learning

Create supervision from the data itself, pretrain on huge unlabeled corpora, then fine-tune or probe:

- **Masked modeling:** predict masked tokens (BERT) or image patches (MAE).

- **Next-token prediction:** the LLM objective (Part 42).
- **Contrastive learning:** pull two views of the same example together and push other examples apart, with the **InfoNCE** loss:

$$\mathcal{L}_i = -\log \frac{\exp\left(\text{sim}(z_i, z_i^+)/\tau\right)}{\sum_k \exp\left(\text{sim}(z_i, z_k)/\tau\right)}$$

SimCLR uses augmentations as views; **CLIP** uses (image, caption) pairs with a symmetric loss over a batch — giving zero-shot classification and the text–image embeddings used in retrieval and generation. Larger batches (more negatives) and the temperature τ matter; this is the same in-batch softmax as two-tower retrieval (Part 47). - **Non-contrastive methods** (BYOL, DINO-style self-distillation) avoid explicit negatives by using a slowly updated teacher network and architectural tricks to prevent collapse.

Evaluate representations with linear probes, k-NN accuracy and fine-tuning on downstream tasks.



Real-life analogy: cousins in a wedding album

Think of it like learning faces from a huge wedding album with no names written in. Two photos of the same cousin, from different angles and in different lighting, should feel like "the same person" (a positive pair), while every other guest should feel different (negatives). Nobody labelled anything; the album itself supplies the task. Masked modelling is the fill-in-the-blanks version, and CLIP is matching each photo to its caption.

Formula, read aloud: InfoNCE says: the loss $-\log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\sum_k \exp(\text{sim}(z_i, z_k)/\tau)}$ is small when, out of everyone in the batch, the true partner gets most of the "similarity vote"; τ sets how sharply the vote concentrates.

Where it breaks: wedding guests really are different people; in a batch, some "negatives" are the same kind of thing (two dog photos), yet the loss still pushes them apart.

49.5 Causal ML: uplift and heterogeneous treatment effects

Part 4 covers identification (randomization, confounding, propensity scores, DiD, IV). Causal ML asks **for whom** a treatment works: the conditional average treatment effect

$$\tau(x) = \mathbb{E}\big[Y(1) - Y(0) \mid X = x\big]$$

Uses include targeting promotions to people who are *persuadable* rather than those who would buy anyway, and personalized medicine.



Real-life analogy: who a coupon actually persuades

Think of it like a shop sending discount coupons. Some customers would buy anyway (the coupon just loses margin), some buy only because of the coupon (the persuadables), some never buy, and a few are annoyed by it. Uplift is how much the coupon changes *this kind* of customer's chance of buying. The meta-learners are different recipes for estimating that difference from data.

Formula, read aloud: $\tau(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x]$ says: for people like x, the effect = (average outcome if treated) − (average outcome if not treated).

Where it breaks: you never see the same customer both with and without the coupon, so individual effects are never observed; they must be estimated from comparable people, and a hidden confounder can fool every method.

Meta-learner	Recipe	Watch out for
S-learner	one model $\mu(x,t)$; $\hat{\tau}(x)=\mu(x,1)-\mu(x,0)$	regularization can shrink the treatment effect toward zero
T-learner	separate models for treated and control; subtract	noisy when one arm is small; errors from two models add
X-learner	impute individual effects with the other arm's model, then model them; weight by propensity	good with unbalanced arms
DR / R-learner, double ML	residualize outcome and treatment on X (with cross-fitting), then regress residual on residual	needs good nuisance models; valid inference
Causal forests	trees that split on treatment-effect heterogeneity	honest splitting, confidence intervals

Checked: simulated data with true $\tau(x)=1+2x_0$ and **confounded** treatment assignment (a covariate raises both treatment probability and outcome): the naive treated-minus-control difference is 1.87 (true average effect 1.0); a T-learner with gradient boosting estimates 1.08 and an S-learner 1.05, with CATE correlations with the truth of 0.98–0.99. That works only because the confounder is observed — **no method fixes unobserved confounding**.

Evaluation is hard because individual effects are never observed: use randomized data when possible, uplift/Qini curves (treat the top-$k\%$ by predicted uplift and compare to random targeting), and policy-value estimates (Part 47's off-policy evaluation).

49.6 Reinforcement learning essentials

An agent in a **Markov decision process** $(\mathcal{S},\mathcal{A},P,R,\gamma)$ picks actions to maximize expected discounted return $G_t=\sum_k \gamma^k r_{t+k}$. The optimal value function satisfies the **Bellman optimality equation**:

$$V^*(s)=\max_a\Big[R(s,a)+\gamma\sum_{s'}P(s'\mid s,a)\,V^*(s')\Big],\qquad Q^*(s,a)=R(s,a)+\gamma\,\mathbb{E}_{s'}\big[\max_{a'}Q^*(s',a')\big]$$



Real-life analogy: a delivery rider's evening plan

Think of it like a delivery rider choosing where to wait for orders. A zone's worth is the next order's pay plus what the evening will likely bring from wherever that order leaves you, with later money counting a bit less (γ). A rider who grabs every small nearby order can earn less than one who heads for the busy market. Value iteration plans with a full map; Q-learning learns zone values by experience.

Formula, read aloud: $V^*(s)=\max_a[R(s,a)+\gamma\sum_{s'}P(s'\mid s,a)V^*(s')]$ says: best long-run value here = the best choice of (reward now + γ × the average best value of where that choice may lead).

Where it breaks: a rider knows the city; an RL agent often doesn't know P or R and must learn by trial, with rewards that may be delayed, sparse or hackable.

- **Value iteration** applies this backup repeatedly (a contraction, so it converges) when the model P,R is known.
- **Q-learning** learns Q^* from experience without a model: $Q(s,a)\leftarrow Q(s,a)+\eta[r+\gamma\max_{a'}Q(s',a')-Q(s,a)]$ — **off-policy**, because the target uses the greedy action whatever the behavior policy did. DQN adds neural networks, replay buffers and target networks.
- **Policy gradient (REINFORCE):** $\nabla_\theta J=\mathbb{E}\big[\nabla_\theta\log\pi_\theta(a_t\mid s_t)\,(G_t-b(s_t))\big]$; the baseline b reduces variance without adding bias. **Actor–critic** learns b (a value function) alongside the policy.
- **PPO** limits each update with a clipped ratio $r_t=\pi_\theta/\pi_{\text{old}}$: $\mathbb{E}\big[\min\big(r_tA_t,\ \text{clip}(r_t,1-\epsilon,1+\epsilon)A_t\big)\big]$ — the algorithm behind classic RLHF (Part 43); GRPO replaces the learned critic with group-relative advantages.

- **Exploration vs. exploitation** (bandits are one-state RL, Part 47), **credit assignment** over long horizons, **reward hacking**, and **offline RL** (learning from logs without exploration) are the recurring interview themes.

Checked: on a five-state chain where "cash out" pays 0.2 immediately and "advance" pays 1 per step only after reaching the end, value iteration finds the patient policy (always advance) with $V(s_0) = 6.28$ vs. 2.0 for the myopic policy, and tabular Q-learning learns the same policy with values within 0.08.

Q: "Marketing wants to send a discount to the customers most likely to buy. Is that the right target?"

Strong answer. "Not necessarily — the customers most likely to buy may buy anyway, so the discount just costs margin. The right target is the customers with the largest *uplift*: the increase in purchase probability caused by the discount, net of its cost. I'd run a randomized pilot to get unconfounded data, fit an uplift model — a T- or X-learner or a causal forest — and evaluate it with a Qini curve on held-out randomized data. Then target the top segment by expected incremental profit, keep a random holdout to keep measuring, and watch for customers learning to wait for discounts."

Follow-ups

"Why not random K-fold for a forecasting model?" — *It trains on the future to predict the past and leaks autocorrelated neighbors into validation, so error estimates are optimistic; use walk-forward splits with a gap if features need it.*

"Why do diffusion models beat GANs on diversity?" — *The likelihood-based denoising objective must cover the whole data distribution, while a GAN generator can satisfy its discriminator with a few modes (mode collapse).*

"What makes Q-learning off-policy?" — *Its target uses $\max_{a'} Q(s', a')$, the greedy action, regardless of what the behavior policy actually did next; SARSA uses the action actually taken, so it is on-policy.*

Active recall — Set 22: Responsible AI & other frontiers

Set 22 · Answer from memory, then check the Answer Key

Easy recall

1. Define demographic parity, equal opportunity and equalized odds.
2. Write the Shapley value formula.
3. Define (ε, δ)-differential privacy.
4. Write the VAE ELBO.
5. Write the Bellman optimality equation for V^*.

Conceptual

6. State the fairness impossibility result.
7. What does integrated gradients' completeness property say?
8. How does DP-SGD bound one example's influence?
9. What is over-smoothing in GNNs?
10. What is classifier-free guidance?

Why

11. Why doesn't removing the protected attribute ensure fairness?
12. Why are SHAP values not causal effects?

13. Why is MASE preferred to MAPE for many forecasting problems?
14. Why is the reparameterization trick needed?
15. Why does a baseline reduce variance in REINFORCE without adding bias?

Scenario

16. A hiring model has equal accuracy across groups but different false-negative rates. What do you report and do?
17. An LLM reproduces a customer's phone number from training data. Response?
18. A demand forecast looks great in cross-validation and terrible in production. Likely cause?
19. Fraudsters form rings in a transaction graph. How would you use graph ML, and what's the pitfall?
20. You need to target retention offers. Which model do you build, and how do you evaluate it?

Mathematical

21. Laplace noise scale for a count query at $\varepsilon = 0.1$; its standard deviation?
22. KL term for a 1-D VAE posterior with $\mu = 1$, $\sigma^2 = 1$.
23. Randomized response with $p = 0.75$ yields 40% "yes" reports. Estimate the true rate.

Difficult follow-ups

24. When is a fairness constraint at odds with calibration, and how do you communicate the trade-off?
25. Explain the connection between PPO, RLHF and GRPO.

PART 50

The 2027 Interview Landscape

The landscape in one breath

Fundamentals still decide most loops — bias–variance, metrics, leakage, experiments, a clean from-scratch implementation. What changed is that **GenAI became a standard topic**: RAG and LLM-serving designs sit next to recommenders and fraud, and the bar for **evaluation, cost and latency reasoning** went up. Prepare both layers; lead with measurement.

The whole Part as one picture

Think of it like a board exam whose syllabus was just updated. The core chapters still carry most of the marks (bias–variance, metrics, leakage, experiments, clean code). But a new, heavily weighted chapter was added (GenAI: RAG, LLM serving, agents), and examiners now insist that you "show your working": how you would measure quality, and what it costs in time and money. Smart students revise the old chapters, master the new one, and practise full papers under time.

Where it breaks: a board exam has one published paper pattern; interview loops differ by company, level and role, so ask the recruiter for the exact format.

50.1 What changed by 2026–27

Based on interview guides published in 2026 (sources at the end of the book) and the pattern of questions collected in this notebook:

- **GenAI is part of ML system design, not a niche.** Common prompts now include RAG over enterprise documents, support assistants, LLM serving and agents, alongside recommendation, search, ads and moderation.
- **Evaluation is the differentiator.** Candidates are asked how they know a prompt or model change didn't make things worse; strong answers name golden sets, regression runs, and RAG-specific checks (context relevance, faithfulness, answer relevance) before shipping.
- **Production realism.** Rate limits, timeouts, retries, cost per request and latency budgets are treated as first-class requirements. Coding rounds for GenAI roles include practical tasks such as chunking long documents, deduplication and resilient API calls.
- **Mechanics over buzzwords.** Naming a framework earns nothing; explaining retrieval, reranking, token budgets and failure modes does. Assuming fine-tuning fixes hallucination (instead of grounding) is a common mistake.
- **Classical depth still matters.** Loops still include ML concepts, ML coding and behavioral rounds; GenAI questions build on the same first principles (losses, generalization, evaluation, experimentation).

50.2 The typical loop

Structures vary by company, level and role — **ask your recruiter for the exact format** (including whether AI assistants are allowed in coding rounds; policies differ).

Round	What it tests	Where to prepare in this book
Recruiter / hiring-manager screen	fit, past impact, what you shipped and how success was measured	Part 50 (project story)
Coding (algorithms)	data structures, complexity, clean code	outside this book's scope — practise separately
ML coding	implement a model, metric or component from scratch in NumPy/PyTorch; debug a training loop	Part 21 (incl. Round 2), code in Parts 41–47
ML concepts / breadth	theory, trade-offs, follow-up chains	Parts 1–20, 27–31, 41–49
ML system design (45–60 min)	scoping, data, model, evaluation, serving, monitoring, trade-offs	Parts 24 (Designs 1–12), 32, 35, 47
GenAI / LLM design or evaluation round	RAG, agents, serving cost/latency, eval harness, safety	Parts 44–46, Designs 13–18, cards G1–G50
Behavioral / project deep-dive	ownership, incidents, ambiguity, collaboration	Part 50

A common ML system-design pacing (one published guide): define the problem (~8 min), data (~8), model (~8), training and evaluation (~8), deployment and monitoring (~8), wrap-up (~5). The Part 35 framework maps onto it directly.



Real-life analogy: a multi-round cricket trial

Think of it like a state-team selection trial. A net session tests basic technique (the algorithms round); a bowling spell tests a specialist skill (ML coding); fielding drills test breadth (ML concepts); a practice match tests how you play under real conditions (system design); and a chat with the coach checks attitude and past performance (behavioural and project rounds). Every round tests something different, so preparing only your favourite skill is risky.

Where it breaks: trials are the same for every player; interview loops vary by company, level and role, including whether AI assistants are allowed in coding, so confirm the format first.

50.3 Role archetypes and what to prioritize

Role	Emphasis	Prioritize
ML engineer (product)	end-to-end systems, features, serving, monitoring	Parts 16–26, 47, system designs, debugging scenarios
Applied / data scientist	experimentation, statistics, causal inference, metrics	Parts 3–4, 16–18, 45 (eval stats), 49 (causal ML)
GenAI / AI engineer	LLM applications: RAG, agents, evaluation, cost, safety	Parts 41–46, Designs 13–18, cards G1–G50, Part 45 especially
Research engineer / scientist	training at scale, architectures, papers, experiments	Parts 14, 20, 41–43, 46, derivations
ML infrastructure / platform	distributed training, serving systems, GPUs, reliability	Parts 23, 46, inference math cheat sheet
Recommender / ranking specialist	retrieval, ranking, bias, experimentation	Parts 16, 24 (Designs 3–5), 47

50.4 The GenAI design rubric

What interviewers reward, distilled from the 2026 guides and the frameworks in this book:

1. **State assumptions before drawing boxes:** users, scale (QPS, corpus size), latency and cost budgets, freshness, access control, risk level.
2. **Retrieval is the bottleneck:** chunking strategy, hybrid search, retrieve-then-rerank, metadata filters — with named trade-offs.
3. **Token budgeting:** context size vs. quality vs. cost; caching (prefix and semantic), routing to smaller models, output limits.
4. **Layered hallucination defense:** grounding with citations, abstention when support is weak, guardrails, and trace logging.
5. **Evaluation plan:** offline golden and regression sets, calibrated LLM judges, human review, online metrics, CI gates.
6. **Serving math:** TTFT/TPOT, batching, KV-cache memory, when to self-host vs. use an API.
7. **Safety and security:** prompt injection, permissions, PII, human approval for irreversible actions.
8. **Lead the trade-offs:** propose the simplest system that meets the requirements, then say what you'd add at 10× scale.

Common mistakes: treating LLM output as deterministic with no quality gates; listing components without reasoning through constraints; skipping evaluation; reaching for fine-tuning or agents when retrieval or a fixed pipeline would do.



Real-life analogy: planning a big wedding

Think of it like planning a wedding for 500 guests. First ask the budget, guest count, venue and date before booking anyone (state assumptions). The kitchen is the real bottleneck (retrieval). Plan portions so food neither runs out nor goes to waste (token budgeting). Check the food is safe before it is served (hallucination defence), taste the menu in advance (evaluation), work out how many servers each hall needs (serving math), and keep security at the gate (prompt injection, permissions). Propose the simplest plan that works, then say what changes for 5,000 guests.

Where it breaks: a wedding happens once; an LLM system runs every day, so evaluation, monitoring and cost control never finish.

50.5 The project deep-dive

Every loop includes "tell me about a project". Prepare two projects to the depth where you can survive five follow-ups on any decision.

```
1. Problem & stakes     who needed what; the business metric; why ML
2. Baseline             what existed before; the simplest thing you tried first
3. Data                 sources, labels, their flaws; leakage checks
4. Approach             model choices and WHY; what you rejected and why
5. Evaluation           offline metric, how it related to the online metric; the A/B result
6. Failure              something that broke or surprised you, and what you changed
7. Impact               numbers: metric lift, cost, latency, users affected
8. Hindsight            what you'd do differently now
```

Know your numbers (dataset size, metric values, latency, team size, timeline) and your own contribution versus the team's. Expect "why not X?" for every choice — answer with the constraint that ruled it out.

Real-life analogy: a lawyer facing cross-examination

Think of it like a lawyer preparing a case for cross-examination. Know every fact cold (your numbers), be clear about what you did versus what your team did, and have a reason for every move, including the options you rejected and why. Expect "why not X?" again and again; the answer is always the constraint that ruled X out. Admit the weak points of the case before the other side finds them (the failure and the hindsight).

Where it breaks: a lawyer argues to win; interviewers reward honest reasoning, so a clearly explained mistake and what you learned from it beats a flawless-sounding story.

50.6 Behavioral themes that recur

- A production incident you owned: detection, mitigation, root cause, prevention.
- Disagreeing with a stakeholder about a metric or launch decision.
- Working under ambiguity: turning a vague ask into a measurable problem.
- Prioritizing between accuracy, latency, cost and deadlines.
- An ethical or safety concern you raised.

Use a clear structure (situation, task, action, result), keep "I" distinct from "we", and end with what you learned.

50.7 An 8-week plan for the 2027 edition

Week	Focus	Parts	Output
1	Foundations, math, statistics	1–4	Recall Sets 1–4; derivations 1–5
2	Classical models	5–13	Sets 5–8; implement linear/logistic regression, trees, k-means from memory
3	Optimization, losses, evaluation, features	14–19	Sets 9–11; metric code; imbalance scenarios
4	Deep learning, coding, debugging, production	20–23	Sets 12–14; Part 21 Round 1 and 2 timed
5	System design + data-centric	24–26, 32–35	Two designs per day out loud (incl. Designs 13–18); Set 15
6	Transformers, LLMs, alignment	41–43	Sets 16–17; attention/MHA from scratch
7	RAG, evaluation, efficiency, recsys	44–47	Sets 18–21; KV-cache and 6ND arithmetic drills
8	Responsible AI, frontiers, mocks	48–49, 27–40	Set 22; Final Exam under time; two mock loops; fix weak spots

Daily habit: 30 minutes of question cards answered out loud with a timer, 15 minutes of spaced recall on previous sets, and one derivation from a blank page.

50.8 The last-week checklist

- Can you implement attention, logistic regression, k-means and an evaluation metric from scratch without looking?
- Can you do the arithmetic cold: 6ND, 16 bytes/param, KV-cache size, decode tokens/s from bandwidth, sample size for an A/B test, standard error of an accuracy?

- Can you explain bias–variance, regularization, calibration, leakage and data drift with a concrete example each?
- Do you have one end-to-end design you can present in 45 minutes for: recommendations, fraud, search, and a RAG assistant?
- Do you have two project stories with numbers, failures and hindsight?
- Have you taken the Final Exam under time, checked it against the Answer Key, and redone the misses?

Q: "Tell me about the ML project you're proudest of."

Strong answer pattern. "At [company], support agents spent about 40% of their time searching internal docs. The goal was to cut handle time without hurting resolution quality. The baseline was keyword search; I first measured it on 300 real tickets — the right document appeared in the top 5 only 52% of the time. I built hybrid retrieval with a reranker and a grounded answer generator with citations; I rejected fine-tuning because the docs changed weekly. Retrieval recall@5 went to 81%. The surprise was that faithfulness failures came mostly from outdated duplicate documents, so we added version-aware indexing. In a four-week A/B test, handle time fell 18% with no drop in resolution rate, at about $0.02 per query. In hindsight I'd have built the evaluation set on day one instead of week three." *(Illustrative numbers — use your own.)*

Unit VII

The Interview Layer

Part 27 — 200 High-Frequency Questions · Part 28 — Why Questions
Part 29 — Compare Questions · Part 30 — Follow-up Chains
Part 31 — Derivations · Part 32 — Mental Models
Part 33 — Interviewer Mode · Part 34 — Active Recall
Part 35 — Answer Frameworks · Part 36 — Staff-Level Questions
Part 37 — Interview Traps

Everything before this taught the material. Everything after this trains you to say it — clearly, precisely, under pressure.

PART 27

200 High-Frequency Interview Questions

How to use the question bank

Cover the card below the question. Answer out loud in **30 seconds**, then in **2 minutes**. Then read. Each card gives: what's being tested, both answer lengths, the deeper mechanism (with the equation when it matters), the answer that loses points, the follow-up and the harder follow-up, and a practical example. Cross-references (→ Part N) point to the full treatment.

The whole Part as one picture

Think of it like net practice before a big match. Each card is one ball bowled at you: cover the answer, play the shot out loud — a tidy 30-second defensive stroke first, then the full 2-minute innings — and only then check with the coach (the card). The "bad answer" is the edge that gets you caught behind; the follow-ups are the bowler's next, trickier deliveries once you've hit a boundary. The cross-references (→ Part N) are the video-analysis room you return to when a shot keeps failing. A few rounds of nets every day beat one long session the night before.

Where it breaks: in the nets the balls repeat; a real interviewer adapts to whatever you say, so practise the reasoning behind each answer, not the exact words on the card.

Levels: 40 Easy (E) · 45 Medium (M) · 40 Hard (H) · 25 Staff-level (S) · 50 GenAI & modern ML (G, new in 2027) = 200.

27.1 Easy

E1 · What's the difference between supervised, unsupervised and self-supervised learning?

Testing: basic taxonomy, and whether you know where labels come from.

30 s: Supervised learns $x \to y$ from labeled pairs. Unsupervised finds structure in x alone (clusters, low-dimensional structure, densities). Self-supervised creates labels from the data itself — predict masked words, the next token, or which augmented views match — then transfers the learned representation.

2 min: Add: semi-supervised mixes few labels with many unlabeled examples; reinforcement learning learns from rewards of actions. Self-supervision is why modern models need few labels: pretraining learns general features from raw data at scale, and fine-tuning adapts them.

Analogy: Supervised learning is studying with the teacher's answer key; unsupervised is sorting a heap of unlabelled photos into piles by yourself; self-supervised is covering words in a newspaper and guessing them — the paper itself supplies the answer key, for free.

Deep: Self-supervised objectives are still supervised losses (cross-entropy on masked tokens, InfoNCE on views) — the novelty is that the supervision signal is free.

Bad answer: "Unsupervised means no loss function."

Follow-up: Is k-means supervised? → **Harder:** How would you evaluate a self-supervised representation? (linear probe, downstream fine-tuning, retrieval metrics.)

Example: Pretraining a text encoder on unlabeled support tickets, then fine-tuning a router on 2,000 labeled tickets.

E2 · What is overfitting, and how do you detect it?

Testing: the generalization gap, and practical diagnosis.

30 s: The model learns noise or sample-specific quirks: low training error, much higher validation error. Detect it with a held-out set that resembles production and learning curves where the gap widens.

2 min: Causes: too much capacity for the data, too little regularization, too many modeling decisions tuned on one validation set, or shortcuts like leaked IDs. Fixes: more data or augmentation, regularization, simpler models, early stopping, bagging. Also check it's not a *split* problem — a validation set from a different distribution produces the same curve.

Analogy: A student who memorises last year's board paper word for word scores full marks on that paper but stumbles when the questions are reworded. You only catch it with a fresh mock test he has never seen.

Deep: Test error = training error + generalization gap; the gap grows with hypothesis-space complexity and shrinks roughly as $1/\sqrt{n}$ (→ Part 1).

Bad answer: "Overfitting is when training accuracy is 100%."

Follow-up: Validation accuracy still improves while the gap grows — overfitting? (Not harmful yet — track validation performance.) → **Harder:** Can a model with zero training error generalize? (Yes — benign overfitting / double descent.)

Example: An unpruned tree with 100% train accuracy and 71% validation accuracy; limiting leaf size gives 84%/80%.

E3 · Explain the bias–variance tradeoff.

Testing: decomposition, intuition, and diagnosis.

30 s: Expected squared error = bias² + variance + noise. Bias: systematic error from wrong assumptions. Variance: sensitivity to the particular training sample. More capacity usually lowers bias and raises variance.

2 min: Diagnose with learning curves: high training error → bias (more capacity/features); low training but high validation error → variance (more data, regularization, bagging). Some changes cut both — more data, better features. Modern overparameterized models can show double descent, so it's not always a single U.

Analogy: A darzi who stitches one standard size for everyone is wrong in the same way every time (bias); one who copies every slouch from a single hurried fitting gives you a different shirt depending on the day you visited (variance).

Deep: $\mathbb{E}[(y-\hat{f})^2] = \sigma^2 + (f-\mathbb{E}\hat{f})^2 + \mathbb{E}[(\hat{f}-\mathbb{E}\hat{f})^2]$ (→ Part 31).

Bad answer: "Bias is when the model is biased against a group."

Follow-up: Where does k in KNN sit? (small k: low bias, high variance.) → **Harder:** Why don't huge neural nets have huge variance?

Example: Degree-1 vs degree-15 polynomial on noisy sine data.

E4 · Why do we need separate train, validation and test sets?

Testing: selection bias from reusing data.

30 s: Train fits parameters; validation chooses hyperparameters/models; test gives an unbiased final estimate. Any set used to make decisions becomes optimistically biased.

2 min: Tuning many configurations on validation overfits it — the best score is partly noise. So the test set is touched once. Splits must mimic production: time-based for temporal data, grouped for repeated entities, deduplicated.

Analogy: A chef practises a dish in her kitchen (train), picks the best of several versions using regular tasters (validation), then serves it once to critics (test). Tasters who judged thirty versions helped choose it, so their praise is no longer an unbiased verdict.

Deep: Picking the max of K noisy validation scores inflates it by roughly $s\sqrt{2\ln K}$.

Bad answer: "Validation and test are the same thing."

Follow-up: Small dataset — what do you do? (nested CV.) → **Harder:** How large should the test set be? (SE $= \sqrt{p(1-p)/n}$ well below the effects you care about.)

Example: A leaderboard overfit by 300 submissions.

E5 · What is cross-validation and when would you use it?

Testing: estimation variance, and when CV is wrong.

30 s: Split data into K folds; train on K−1, validate on the held-out fold, rotate, average. It gives a lower-variance estimate and uses all data for both training and validation.

2 min: Use it with small/medium data for model selection. Variants: stratified (class balance), grouped (entities), time-series (forward chaining). Preprocessing must be fit inside each fold. With huge data, a single well-designed holdout is enough.

Analogy: Split the syllabus into five chapters; each round, study four and take a test on the fifth, rotating until every chapter has been the test once. The average of the five scores is far steadier than one lucky or unlucky exam.

Deep: LOOCV is nearly unbiased but high-variance (fold models highly correlated); K = 5–10 balances bias, variance and cost.

Bad answer: "CV prevents overfitting." (It *measures* generalization; it doesn't prevent anything.)

Follow-up: Can you use K-fold on time series? → **Harder:** Why can't the CV score of your tuned model be reported as its performance?

Example: 5-fold grouped CV for a patient-level medical dataset.

E6 · What is regularization? Compare L1 and L2.

Testing: mechanism, not slogans.

30 s: Regularization constrains the model to reduce variance. L2 adds $\lambda\|w\|_2^2$ — shrinks all weights smoothly; L1 adds $\lambda\|w\|_1$ — drives some weights exactly to zero (feature selection).

2 min: L2 penalizes large weights quadratically, handles correlated features by sharing weight, has a closed form. L1's constant-magnitude pull zeros out weights whose evidence is weaker than λ; with correlated features it picks one arbitrarily — elastic net fixes that. Standardize features first. Bayesian view: Gaussian vs. Laplace prior.

Analogy: Airline baggage fees: L1 is a flat charge per kilo, so any item worth less than the charge stays home entirely (a zero weight). L2's charge grows steeply with each item's weight, so you trim heavy things but keep a little of everything.

Deep: Soft-thresholding: $\hat{w} = \text{sign}(\rho)\max(|\rho| - \lambda, 0)$ vs. ridge $\rho/(1+\lambda)$ (→ Part 5).

Bad answer: "L1 is for classification, L2 for regression."

Follow-up: Why does L1 give zeros geometrically? → **Harder:** Is weight decay the same as L2 for Adam? (No — AdamW.)

Example: Lasso on 10,000 genomic features keeps 40.

E7 · What is gradient descent?

Testing: the update rule and why it works.

30 s: Iteratively move parameters opposite the gradient: $\theta \leftarrow \theta - \eta \nabla J(\theta)$, because the negative gradient is the direction of steepest local decrease.

2 min: Variants: batch (exact gradient), stochastic (one example), mini-batch (the standard). Convergence depends on learning rate and conditioning; convex problems reach the global minimum; non-convex ones reach a stationary point that depends on initialization.

Analogy: A marble on a bumpy tray: at every moment it rolls a little in the steepest downhill direction. It settles in the lowest dip nearby — the true bottom only if the tray is a single smooth bowl (a convex problem).

Deep: From Taylor: $J(\theta + \Delta) \approx J + \nabla J^\top \Delta$; stable for $\eta < 2/\lambda_{\max}(H)$ on a quadratic (→ Part 14).

Bad answer: "It finds the global minimum."

Follow-up: Why mini-batches? → **Harder:** Why is convergence slow on ill-conditioned problems?

Example: Training logistic regression on 100M rows with mini-batch SGD.

E8 · What happens if the learning rate is too large or too small?

Testing: optimization intuition and diagnosis from loss curves.

30 s: Too small: slow progress, may stall on plateaus. Too large: loss oscillates, plateaus at a high value, or diverges to NaN.

2 min: Find it with an LR range test; use schedules — warmup then decay — because early training tolerates/needs different step sizes than late training. The right LR depends on batch size, optimizer, and architecture; retune when those change.

Analogy: Setting a shower's mixer tap: tiny nudges leave you shivering for ages; big twists swing you between icy and scalding and you never settle. Start with moderate turns, then make finer ones as you get close (learning-rate decay).

Deep: On a quadratic, each eigen-direction's error scales by $(1 - \eta\lambda_i)$ per step: diverges if $\eta\lambda_i > 2$.

Bad answer: "Just use 0.001."

Follow-up: Why decay the LR at the end? (SGD noise floor.) → **Harder:** Why does batch size change the optimal LR?

Example: Loss spiking to NaN after 2K steps fixed by warmup + lower peak LR.

E9 · Define precision and recall; when do you prioritize each?

Testing: metric semantics tied to costs.

30 s: Precision = TP/(TP+FP): of flagged items, how many are right. Recall = TP/(TP+FN): of real positives, how many we caught. Prioritize precision when false alarms are costly (spam filter hiding real mail); recall when misses are costly (cancer screening).

2 min: They trade off via the threshold. Precision depends on prevalence; recall doesn't. Often you fix one as a constraint (precision ≥ 95%) and maximize the other, or choose the threshold from explicit costs.

Analogy: A TTE on a crowded train: checking every single passenger catches all ticketless travellers (high recall) but wastes time on honest ones; fining only people who look suspicious makes most fines deserved (high precision) but lets some dodgers slip by.

Deep: Cost-optimal threshold $t^* = C_{FP}/(C_{FP} + C_{FN})$ with calibrated probabilities (→ Part 6).

Bad answer: "Precision is accuracy on positives."

Follow-up: What's F1 and why the harmonic mean? → **Harder:** How does precision change if prevalence drops 10× with the same model?

Example: Fraud review queue with capacity 2,000 → maximize recall at that capacity.

E10 · Why can accuracy be misleading?

Testing: imbalance and cost awareness.

30 s: With 99% negatives, predicting "negative" always gives 99% accuracy and zero value. Accuracy also weights all errors equally, which rarely matches costs.

2 min: Use confusion matrices, precision/recall, PR-AUC, cost-weighted metrics; accuracy is fine when classes are balanced and errors equally costly. Also accuracy hides calibration and ranking quality.

Analogy: A weather forecaster in the Thar desert who always says 'no rain' is right on 99% of days — and completely useless, because he misses every rainy day, the only days anyone needed a forecast.

Deep: See Part 28 for the full "why".

Bad answer: "Accuracy is misleading because it doesn't use probabilities." (Partly, but misses the imbalance point.)

Follow-up: Which metric for 0.1% fraud? → **Harder:** When is ROC-AUC also misleading?

Example: 99.9%-accurate fraud model that catches nothing.

E11 · What is a confusion matrix and what can you read from it?

Testing: basic evaluation fluency.

30 s: A table of true vs. predicted classes: TP, FP, FN, TN for binary; K×K for multiclass. Precision, recall, specificity, accuracy all derive from it.

2 min: In multiclass it reveals *which* classes get confused (e.g. "husky" ↔ "wolf"), guiding data collection or label-guideline fixes. It's threshold-dependent — show it at the operating point. Normalize rows to see per-class recall.

Analogy: A courier's delivery log listing, for every parcel, the flat it was meant for against the flat it actually reached. You don't just see how many went wrong — you see that Flat 3A's parcels keep landing at 3B, and can fix that.

Deep: Macro vs. micro averaging: macro averages per-class metrics (treats classes equally); micro pools counts (dominated by frequent classes).

Bad answer: Listing the cells without saying what decisions it informs.

Follow-up: Macro vs. micro F1? → **Harder:** How would you detect label noise from the confusion matrix?

Example: Support-ticket classifier confusing "billing" and "refund" → merge or clarify labels.

E12 · What does ROC-AUC measure?

Testing: probabilistic interpretation and limitations.

30 s: The probability that a random positive is scored higher than a random negative. 0.5 = random ranking, 1.0 = perfect. It's threshold-free.

2 min: It's invariant to class balance and to monotonic score transforms — so it ignores calibration and can look great under heavy imbalance while precision is poor. For rare positives, PR-AUC or precision at the operating point is more informative.

Analogy: Pick one guilty and one innocent suspect at random and ask the detective who seems more suspicious; AUC is how often he points at the guilty one. It tests his ordering, not whether his 'I'm 80% sure' numbers are honest.

Deep: Equals the Mann–Whitney U statistic normalized by $n_+ n_-$ (→ Part 21 for code).

Bad answer: "AUC is the accuracy averaged over thresholds."

Follow-up: Does AUC change if you square all scores? (No.) → **Harder:** When would you still prefer ROC-AUC with imbalance?

Example: Credit models report Gini = 2·AUC − 1.

E13 · What is logistic regression and why the sigmoid?

Testing: model formulation.

30 s: A linear model of the log-odds: $\log \frac{p}{1-p} = w^\top x + b$. The sigmoid is the inverse of the logit, mapping any real score into (0,1) as a probability. Trained by maximizing Bernoulli likelihood (cross-entropy).

2 min: Coefficients multiply the odds by e^{w_j}. The loss is convex; the gradient is $(p - y)x$. The boundary is linear. Sigmoid is the canonical link for Bernoulli in GLMs, which gives the clean gradient and tends to give calibrated probabilities.

Analogy: A TV win-predictor during a cricket chase adds up runs needed, wickets and overs left into one score, then squeezes that score into a 0–100% chance of winning — a huge lead gets close to 100% but never past it.

Deep: Derivation in Part 6.

Bad answer: "It's regression, so it predicts continuous labels."

Follow-up: Why not MSE? → **Harder:** What happens on perfectly separable data?

Example: Conversion model with odds ratios explained to marketing.

E14 · What loss does linear regression use and why?

Testing: MLE connection.

30 s: Mean squared error. It's the maximum-likelihood objective if noise is Gaussian, it targets the conditional mean, and it gives a convex problem with a closed-form solution.

2 min: Squaring makes outliers dominant — use MAE or Huber when tails are heavy; MAE targets the median. The solution is the projection of y onto the column space of X.

Analogy: Choosing where to park a chai cart when each customer's grumbling grows with the square of their walk: the best spot is the average of their locations, and one customer living far away drags the cart noticeably toward him.

Deep: $-\log p(y \mid X, w) = \frac{1}{2\sigma^2}\|y - Xw\|^2 + \text{const.}$

Bad answer: "Because it's differentiable." (True but incomplete.)

Follow-up: Normal equation? → **Harder:** When would you use a Poisson loss instead?

Example: House prices: MSE on log price.

E15 · What is feature scaling and when is it needed?

Testing: which algorithms are scale-sensitive, and why.

30 s: Putting features on comparable scales (standardization, min-max). Needed for distance-based models (KNN, k-means, RBF SVM), regularized linear models, PCA, and gradient-based training; not for trees.

2 min: Distances and dot products sum over features, so large-unit features dominate. Penalties treat coefficients in feature units. Gradient descent converges slowly on badly scaled (ill-conditioned) problems. Trees only use orderings. Fit scalers on training data only.

Analogy: Comparing two flats by adding up the difference in rent (in rupees) and in number of bedrooms: a Rs 2,000 gap swamps a three-bedroom gap purely because of units. Scaling first puts both on a common 'how unusual is this' ruler.

Deep: Scaling changes the Hessian's condition number (→ Part 14).

Bad answer: "Always scale because it improves accuracy."

Follow-up: Does OLS need scaling? (Predictions no; interpretability/numerics maybe.) → **Harder:** Why does scaling matter for L1 but not unregularized OLS?

Example: Income in dollars swamping age in KNN.

E16 · How does a decision tree choose splits?

Testing: impurity and greed.

30 s: At each node, it tries candidate (feature, threshold) pairs and picks the one that most reduces impurity — Gini or entropy for classification, variance for regression — then recurses.

2 min: It's greedy (not globally optimal); continuous features are sorted and scanned; stopping rules or pruning control overfitting. Information gain = parent entropy − weighted child entropy.

Analogy: The game of 20 Questions: each turn you ask the yes/no question that splits the remaining possibilities into the cleanest groups, then repeat inside each group — picking the best question right now, without planning the whole game ahead.

Deep: Gini $1 - \sum p_k^2$; entropy $-\sum p_k \log p_k$; misclassification error is a poor splitting criterion (not strictly concave).

Bad answer: "It picks the most important feature first."

Follow-up: Gini vs. entropy? → **Harder:** Why is greedy splitting bad for XOR?

Example: First split on "account age < 30 days" in fraud.

E17 · What is a random forest?

Testing: bagging + decorrelation.

30 s: An ensemble of deep decision trees, each trained on a bootstrap sample and considering a random subset of features at each split; predictions are averaged or voted.

2 min: Averaging reduces variance; feature subsampling decorrelates trees so averaging keeps helping. Robust, few hyperparameters, out-of-bag error estimate. Weaknesses: large models, no extrapolation, usually behind tuned GBMs.

Analogy: Asking 500 doctors for a diagnosis, each trained on a different random bundle of past case files and shown only a few random test results at each step: each doctor is erratic, but their independent mistakes cancel in the majority vote.

Deep: $\mathrm{Var}(\mathrm{avg}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$.

Bad answer: "Random forests can't overfit."

Follow-up: Why deep trees? → **Harder:** Why does adding trees not overfit but boosting rounds can?

Example: A 500-tree forest as a churn baseline.

E18 · What is boosting?

Testing: sequential bias reduction.

30 s: Train weak learners sequentially, each focusing on what the current ensemble gets wrong; combine them additively. It reduces bias.

2 min: AdaBoost reweights misclassified examples (exponential loss); gradient boosting fits trees to the negative gradient of any differentiable loss. Controlled by learning rate, number of rounds (early stopping), tree depth, subsampling.

Analogy: A batting coach after each net session: the next drill targets only the shots the batsman still gets out to, and his game becomes the sum of all these small fixes — each one aimed at what is still wrong.

Deep: Gradient descent in function space (→ Part 8).

Bad answer: "Boosting trains models in parallel on bootstrap samples." (That's bagging.)

Follow-up: Why shallow trees? → **Harder:** Derive XGBoost's leaf weight.

Example: LightGBM for CTR features.

E19 · How does KNN work, and what are its weaknesses?

Testing: non-parametric reasoning and scaling.

30 s: Store the training data; predict by majority vote or average of the k nearest points under a distance metric.

2 min: Weaknesses: prediction cost $O(nd)$ per query, memory $O(nd)$, needs feature scaling, suffers in high dimensions (distances concentrate), sensitive to irrelevant features. k controls bias/variance. At scale use ANN indexes.

Analogy: Guessing a new neighbour's favourite restaurant by asking the k families living closest to him: nothing is learned in advance, you just look up the nearest examples each time — slow in a huge colony, and misleading if 'closeness' is measured badly.

Deep: Curse of dimensionality (→ Part 10).

Bad answer: "KNN has no hyperparameters."

Follow-up: How to choose k? → **Harder:** How would you serve KNN over 100M items in 20 ms?

Example: Embedding-based "similar products".

E20 · How does k-means work?

Testing: objective and algorithm.

30 s: Minimize within-cluster squared distances by alternating: assign points to nearest centroid; recompute centroids as means. Repeat until stable.

2 min: Converges to a local optimum; use k-means++ initialization and several restarts; choose k with elbow/silhouette/business usefulness. Assumes spherical, similar-sized clusters; scale features; sensitive to outliers.

Analogy: Placing k water tankers in a village: each house fetches from its nearest tanker, then each tanker moves to the middle of the houses it serves; repeat until nobody switches. You end at a good layout, not necessarily the best one.

Deep: Each step can't increase $J = \sum \|x_i - \mu_{c_i}\|^2$.

Bad answer: "It finds the optimal clustering."

Follow-up: k-means vs. DBSCAN? → **Harder:** How is k-means related to GMM/EM?

Example: Segmenting stores by sales profile.

E21 · What is PCA?

Testing: variance maximization and practical use.

30 s: A linear projection onto orthogonal directions of maximum variance — eigenvectors of the covariance matrix — used for compression, denoising, visualization and decorrelation.

2 min: Center (and usually standardize) data; SVD gives the components; pick k by explained variance or downstream performance. Limitations: linear, unsupervised (may discard predictive low-variance directions), outlier-sensitive.

Analogy: Photographing a long queue from the side rather than end-on: you pick the camera angle that shows the most spread, keeping the directions in which the points vary most and flattening the rest into the picture.

Deep: $\max_u u^\top \Sigma u$ s.t. $\|u\| = 1 \Rightarrow \Sigma u = \lambda u$.

Bad answer: "PCA selects the most important features."

Follow-up: Why standardize? → **Harder:** When does PCA hurt a downstream model?

Example: 2,000 correlated sensor features → 50 components.

E22 · What is the curse of dimensionality?

Testing: geometric intuition.

30 s: As dimensions grow, data becomes sparse, neighborhoods stop being local, and distances concentrate (nearest ≈ farthest), so methods relying on similarity or density degrade and need exponentially more data.

2 min: A neighborhood capturing 1% of uniform data in 10-D spans 63% of each axis. Real data often has low intrinsic dimension, which is why embeddings and dimensionality reduction help.

Analogy: Finding a friend along a 1 km street is easy, in a 1 km square field harder, in a 1 km cube of sky hopeless: each extra dimension multiplies the space, so the same crowd spreads so thin that 'nearby' stops meaning anything.

Deep: Edge length $r^{1/d}$; ratio $(d_{\max} - d_{\min})/d_{\min} \to 0$.

Bad answer: "More features always help."

Follow-up: Which models suffer most? → **Harder:** Why do deep nets not suffer as badly?

Example: KNN on 1,000 raw features failing; works on 64-D embeddings.

E23 · What is data leakage? Give an example.

Testing: practical rigor.

30 s: Information available in training/validation that won't exist at prediction time — inflating offline metrics. Example: a churn feature computed over a window that includes the churn date.

2 min: Types: target leakage, train–test contamination (preprocessing on all data, duplicates across splits), temporal leakage, group leakage. Prevent with point-in-time features, pipelines fit per fold, time/group splits, deduplication.

Analogy: 'Predicting' last night's rain from whether the roads are wet this morning looks uncanny in hindsight — but when you actually need the forecast, before it rains, the wet-road clue doesn't exist yet.

Deep: → Part 1.

Bad answer: "Leakage is when test data is used for training." (Only one form.)

Follow-up: Validation AUC 0.99 — what do you check? → **Harder:** Is out-of-fold target encoding leak-free?

Example: `refund_issued` in a fraud model.

E24 · How do you handle missing values?

Testing: missingness mechanisms and consistency.

30 s: First ask why they're missing. Options: drop, impute (median/mode/model-based) plus a missing indicator, or use models that handle NaNs natively (GBMs).

2 min: MCAR vs. MAR vs. MNAR — if missingness depends on the value itself (income not reported), it's informative, so keep an indicator. Fit imputers on train only, and make serving encode missing values identically.

Analogy: A patient's form with the 'alcohol per week' box left blank: first ask why — forgot, or embarrassed? The blank itself can be a clue, so a good doctor notes 'left blank' alongside any guess she pencils in.

Deep: → Part 19.

Bad answer: "Fill with zero."

Follow-up: When is dropping rows dangerous? → **Harder:** How does XGBoost handle missing values?

Example: Missing income predicting default risk.

E25 · How do you encode categorical variables?

Testing: encoding trade-offs by model and cardinality.

30 s: One-hot for low cardinality; ordinal for ordered categories; target/count encoding, hashing or embeddings for high cardinality; native categorical support in LightGBM/CatBoost.

2 min: One-hot explodes with many levels; target encoding leaks unless out-of-fold and smoothed; hashing trades collisions for bounded memory; embeddings need data per category. Handle unseen categories at serving.

Analogy: Sorting post: with five cities you give each its own pigeonhole (one-hot); with 20,000 villages you group by district or use each pincode's past delivery record — and keep a spare bin for any village you've never seen.

Deep: Smoothed TE $= \frac{n_c \bar{y}_c + m\bar{y}}{n_c + m}$.

Bad answer: "Label-encode everything as integers."

Follow-up: 100K-level feature for GBM? → **Harder:** Hashing vs. embeddings?

Example: Zip code → region hierarchy + smoothed target encoding.

E26 · What is an activation function and why is it needed?

Testing: nonlinearity and gradient behavior.

30 s: A nonlinear function applied after each linear layer. Without it, stacked layers collapse into one linear map.

2 min: ReLU is the default (no saturation for positive inputs, cheap); sigmoid/tanh saturate and cause vanishing gradients in deep stacks; GELU/SiLU are smooth variants used in transformers. Output activations match the task (sigmoid, softmax, identity).

Analogy: Zooming a photo to 90%, then 120%, then 70% equals one single zoom — ten linear layers likewise collapse into one. An activation is like folding the paper between steps: each fold adds a crease the next layer can build shapes on.

Deep: $W_2(W_1x) = (W_2W_1)x$.

Bad answer: "To keep outputs between 0 and 1."

Follow-up: Dying ReLU? → **Harder:** Why is non-zero-centered sigmoid output a problem for optimization?

Example: Replacing sigmoid hidden units with ReLU to train a 20-layer net.

E27 · What is backpropagation, at a high level?

Testing: chain rule understanding.

30 s: An efficient way to compute gradients of the loss w.r.t. all parameters by applying the chain rule backward from the loss, reusing intermediate values from the forward pass.

2 min: Forward: compute and cache activations. Backward: the error signal at each layer is propagated by multiplying with transposed weights and activation derivatives; weight gradients are outer products of errors and inputs. Cost ≈ 2× forward.

Analogy: A restaurant tracing a complaint back through the kitchen: the waiter passes blame to the plating station, which passes shares to the grill and the prep cook, each in proportion to how much they affected the dish — one backward walk assigns everyone's share.

Deep: Full derivation in Part 20/31.

Bad answer: "Backprop is the optimizer."

Follow-up: Why is it efficient? → **Harder:** Derive $\partial L/\partial W_1$ for a 2-layer net.

Example: Gradient checking a custom layer.

E28 · What is dropout?

Testing: regularization mechanism and train/inference behavior.

30 s: During training, randomly zero activations with probability p (scaling the rest by 1/(1−p)); at inference use all units. It reduces overfitting.

2 min: Prevents co-adaptation, approximates averaging an ensemble of thinned networks, injects noise. Forgetting to disable it at inference degrades predictions; it interacts poorly with BatchNorm.

Analogy: An office where random staff are on leave each day: nobody can lean on one colleague to cover their gaps, so everyone learns to be useful on their own. On the big launch day, the whole team shows up.

Deep: → Part 20.

Bad answer: "Dropout removes features from the dataset."

Follow-up: Why scale by 1/(1−p)? → **Harder:** What is MC dropout used for?

Example: p = 0.1 in transformer blocks.

E29 · What is batch normalization?

Testing: mechanism and pitfalls.

30 s: Normalize each feature over the mini-batch to zero mean/unit variance, then apply a learned scale and shift. Speeds up and stabilizes training.

2 min: Allows higher learning rates (better conditioning), mild regularization; uses running statistics at inference — a common train/serve mismatch. Struggles with small batches; LayerNorm is preferred in transformers.

Analogy: A teacher who rescales each class test so the average and spread are fixed, then applies her own learned adjustment. At the board exam there's no class to curve against, so she uses her long-run averages instead — mixing up the two modes causes trouble.

Deep: → Part 20 ("why BatchNorm helps").

Bad answer: "It normalizes the input data once before training."

Follow-up: BatchNorm vs. LayerNorm? → **Harder:** Why does the "internal covariate shift" explanation get disputed?

Example: Model accuracy collapses at inference due to `train()` mode left on.

E30 · What is the vanishing gradient problem?

Testing: depth and chain-rule products.

30 s: Gradients shrink exponentially as they propagate back through many layers, so early layers barely learn. Caused by saturating activations and poorly scaled weights.

2 min: Fixes: ReLU-family activations, He/Xavier init, normalization, residual connections, LSTM/GRU gates for sequences. Its twin is exploding gradients (fix with clipping, init, normalization).

Analogy: A bucket brigade passing water back along a line of 20 people, each spilling at least three-quarters of what they receive: the first person gets only drops. That is the error signal reaching early layers through saturating sigmoids.

Deep: $\partial L/\partial h_1 = \partial L/\partial h_L \prod_l \mathrm{diag}(\phi')W_l$; $\sigma' \le 0.25$.

Bad answer: "Vanishing gradients mean the loss is zero."

Follow-up: Why do residuals help? → **Harder:** Why do RNNs suffer more?

Example: 20-layer sigmoid MLP stuck at initial loss.

E31 · Parameters vs. hyperparameters — and how do you tune hyperparameters?

Testing: process discipline.

30 s: Parameters are learned from data (weights); hyperparameters are set before training (learning rate, depth, regularization). Tune them on validation data via random search, Bayesian optimization, or successive halving.

2 min: Random search beats grid search when only a few hyperparameters matter; use log scales for LR/regularization; early-stopping-based methods (Hyperband) save compute. Guard against overfitting the validation set; retrain on train+val then evaluate on test.

Analogy: Baking a cake: oven temperature and baking time are set before you start (hyperparameters); the crumb and rise form by themselves inside (parameters). You pick good settings from trial bakes — never by tweaking after tasting the cake served at the party (test set).

Deep: Grid search wastes trials on unimportant dimensions (Bergstra & Bengio).

Bad answer: "Tune on the test set."

Follow-up: Which hyperparameters matter most for GBMs? → **Harder:** How to tune when a run takes 4 days?

Example: Tuning LightGBM's num_leaves and learning rate with early stopping.

E32 · What is an embedding?

Testing: representation learning.

30 s: A dense, learned vector representing a discrete object (word, user, product) such that similar objects are close in the vector space.

2 min: Learned by a model objective (next-token prediction, co-occurrence, contrastive, supervised task). Enable similarity search, transfer, and efficient representation of high-cardinality categories. Cold-start objects need fallbacks.

Analogy: Giving every song a spot on a huge map of moods, learned from what people play together: songs that feel alike land close by, so 'more like this' simply means 'look nearby on the map'.

Deep: Matrix factorization: embeddings as low-rank factors of an interaction/co-occurrence matrix.

Bad answer: "An embedding is a one-hot vector."

Follow-up: How do you evaluate embeddings? → **Harder:** How do two-tower models train item embeddings?

Example: Product embeddings for "similar items".

E33 · Parametric vs. non-parametric models?

Testing: capacity and data scaling.

30 s: Parametric models have a fixed number of parameters regardless of data size (linear regression); non-parametric models grow with the data (KNN, kernel methods, decision trees without limits).

2 min: Parametric: strong assumptions, fast, data-efficient, may be biased. Non-parametric: flexible, fewer assumptions, need more data, costlier at prediction time.

Analogy: A parametric model is a shopkeeper's rule of thumb — 'price = cost plus 20%' — the same size however many sales he has seen. A non-parametric one is his full ledger of past sales, which grows (and takes longer to search) with every sale.

Deep: Neural nets are parametric but huge — practically flexible.

Bad answer: "Non-parametric means no parameters."

Follow-up: Is a random forest parametric? → **Harder:** Where do Gaussian processes fit?

Example: KNN memory growing with the training set.

E34 · Generative vs. discriminative models?

Testing: what is modeled.

30 s: Generative models learn $P(x, y)$ (or $P(x \mid y)P(y)$) and classify via Bayes' rule — Naive Bayes, LDA, GMMs. Discriminative models learn $P(y \mid x)$ or a boundary directly — logistic regression, SVMs, trees.

2 min: Generative models can generate data, handle missing features, and learn faster with little data if their assumptions hold; discriminative models usually win asymptotically (Ng & Jordan). Modern "generative AI" models are generative over x.

Analogy: A generative learner studies how Hindi and Tamil each sound in full, then asks which language better explains a new sentence; a discriminative learner only learns the cues that tell them apart — it can sort sentences but can't speak either.

Deep: → Part 11.

Bad answer: "Generative models are neural networks that make images."

Follow-up: Is logistic regression generative? → **Harder:** When does NB beat logistic regression?

Example: Spam filtering with NB on tiny data.

E35 · What is class imbalance and how do you handle it?

Testing: metric-first thinking.

30 s: One class is much rarer. Use appropriate metrics (PR-AUC, recall at FPR), tune the threshold, class weights, resampling on the training folds only, or focal loss.

2 min: The model still estimates probabilities fine; the main issues are metrics, thresholds, and few positive examples. Resampling/weights distort calibration — correct afterwards. Get more positives via active learning.

Analogy: A lifeguard at a beach where one swimmer in a thousand is drowning: 'nobody's drowning' is almost always right and completely useless. So you judge the lifeguard on rescues made versus false alarms, and lower the bar for jumping in.

Deep: → Part 18.

Bad answer: "Always use SMOTE."

Follow-up: Why not oversample before splitting? → **Harder:** How do you correct probabilities after down-sampling negatives?

Example: 0.1% fraud.

E36 · What is a p-value?

Testing: statistical literacy.

30 s: The probability, assuming the null hypothesis is true, of observing data at least as extreme as what we saw.

2 min: It's not the probability the null is true, and not effect size. Report effect sizes with confidence intervals; account for multiple testing and peeking.

Analogy: A court assumes the accused is innocent and asks: 'if he really were innocent, how surprising would this evidence be?' A small p-value means 'very surprising' — not 'the chance he is innocent'.

Deep: → Part 4.

Bad answer: "p = 0.03 means 97% chance the effect is real."

Follow-up: What's power? → **Harder:** Why does peeking inflate false positives?

Example: A/B test with p = 0.03 on CTR.

E37 · What is an A/B test?

Testing: experimental design basics.

30 s: A randomized controlled experiment comparing a treatment to control on a pre-defined metric, attributing differences causally to the change.

2 min: Choose unit of randomization, primary and guardrail metrics, sample size via power analysis, run full weeks, check sample-ratio mismatch, avoid peeking, report CIs.

Analogy: A sweet shop testing a new laddoo recipe by randomly handing half its customers the new one and half the old, then comparing repeat purchases: because a coin toss decided who got what, the recipe is the only systematic difference.

Deep: → Part 4 checklist.

Bad answer: "Show two versions and pick the one with more clicks."

Follow-up: When can't you A/B test? → **Harder:** How do network effects bias results?

Example: Testing a new ranking model on 5% of users.

E38 · MSE vs. MAE?

Testing: loss properties.

30 s: MSE squares errors — penalizes large errors heavily, targets the mean, outlier-sensitive. MAE uses absolute errors — robust, targets the median, gradient is constant.

2 min: Choose based on the error-cost shape and noise distribution; Huber combines both. RMSE ≥ MAE and a large gap signals outliers/variance in errors.

Analogy: Picking a meeting point for friends along one metro line: MAE picks the spot with the least total travel (the middle friend's station, the median); MSE squares each trip, so one friend at the far end drags the spot toward him (the mean).

Deep: Gaussian vs. Laplace MLE (→ Part 15).

Bad answer: "MAE is always better because it's robust."

Follow-up: Why is MAE harder to optimize? → **Harder:** What does MAE predict for zero-heavy targets?

Example: Delivery-time predictions with GPS glitches.

E39 · What is early stopping?

Testing: implicit regularization.

30 s: Stop training when validation performance stops improving, keeping the best checkpoint.

2 min: It limits effective capacity: early in training models fit dominant patterns; later they fit noise. Needs patience and a validation set representative of production. For GBMs, it selects the number of trees.

Analogy: Brewing chai: taste it every minute and take it off the flame at its best — keep boiling and it turns bitter. Long training likewise starts fitting noise, so you watch the validation score and keep the best moment.

Deep: For linear least squares with GD, stopping at step t ≈ ridge with λ ≈ 1/(ηt).

Bad answer: "Stop when training loss stops decreasing."

Follow-up: Why is it regularization? → **Harder:** Validation loss rises but accuracy improves — stop?

Example: LightGBM stopping at 430 of 5,000 rounds.

E40 · What is transfer learning?

Testing: practical deep learning.

30 s: Reuse a model trained on a large source task as the starting point for a target task, then fine-tune or train a small head on top.

2 min: Works because early layers learn general features. Options: freeze + linear probe (little data), fine-tune all (more data), parameter-efficient methods like LoRA. Use lower learning rates for pretrained weights; watch domain mismatch.

Analogy: A trained sitar player picking up the guitar: finger control and rhythm carry over, so a few weeks of gentle adjustment beats years of starting from scratch — but switch her to kabaddi and almost nothing transfers.

Deep: → Part 20.

Bad answer: "Copying another model's predictions."

Follow-up: When does it fail? (negative transfer, large domain gap.) → **Harder:** Why lower learning rates for fine-tuning?

Example: ImageNet-pretrained CNN fine-tuned on 3,000 defect images.

27.2 Medium

M1 · Derive the gradient of the logistic regression loss.

Testing: calculus fluency and the "error × input" structure.

30 s: For BCE with $p = \sigma(w^\top x)$, $\partial \ell / \partial z = p - y$, so $\nabla_w \mathcal{L} = \frac{1}{n} X^\top (p - y)$.

2 min: Chain rule: $\partial \ell / \partial p = -\frac{y}{p} + \frac{1-y}{1-p}$, $\partial p / \partial z = p(1-p)$; the product simplifies to $p - y$. The Hessian $X^\top S X$ with $S = \mathrm{diag}(p(1-p))$ is PSD ⇒ convex. Same form as linear regression because both are GLMs with canonical links.

Analogy: Fixing a curry after tasting: how much you change each spice = (how far the taste was from what you wanted) × (how much of that spice went in). A spice you didn't use gets no blame; a perfect taste changes nothing.

Deep: Newton's method on it = IRLS; first-order condition with intercept ⇒ $\sum p_i = \sum y_i$ (→ Part 6).

Bad answer: Memorized result without the cancellation step.

Follow-up: Why is the loss convex? → **Harder:** Derive the multiclass (softmax) version.

Example: Implementing logistic regression in NumPy (Part 21).

M2 · Why does L1 regularization produce sparse solutions?

Testing: three views — geometry, gradient, prior.

30 s: The L1 ball has corners on the axes, so the loss contours usually first touch it at a corner where some weights are exactly zero. Equivalently, L1's pull toward zero has constant magnitude, so weak weights get pushed all the way to zero.

2 min: In 1-D: minimize $\frac{1}{2}(w-\rho)^2+\lambda|w|$ ⇒ soft-thresholding — zero whenever $|\rho|\le\lambda$. L2 gives $\rho/(1+\lambda)$, never zero. Bayesian: Laplace prior has a kink at zero.

Analogy: Mobile data at a fixed price per GB (L1): any app worth less per GB than the price is switched off entirely — a zero weight. Under L2 the price per GB starts near zero and rises with use, so every app keeps some data.

Deep: Subgradient condition $0\in w-\rho+\lambda\,\partial|w|$ (→ Part 5).

Bad answer: "Because the absolute value is smaller than the square."

Follow-up: What happens with two correlated features? → **Harder:** Why does lasso select at most n features when d > n?

Example: Lasso reducing 500 marketing features to 30.

M3 · Why is cross-entropy preferred over MSE for classification?

Testing: probabilistic and optimization reasoning.

30 s: Cross-entropy is the Bernoulli/categorical maximum likelihood; with sigmoid/softmax outputs it gives gradients $p-y$ that are large when confidently wrong, and a convex problem for linear models. MSE on sigmoids is non-convex and its gradient vanishes when saturated.

2 min: MSE gradient has an extra $p(1-p)$ factor → tiny updates for badly wrong predictions. Cross-entropy is a proper scoring rule, producing calibrated probabilities. It equals KL to the label distribution up to a constant.

Analogy: A quiz that charges you by how confident you were in a wrong answer: '99% sure' and wrong costs a lot, so you learn fast. MSE on a sigmoid is a quizmaster who barely reacts to confident wrong answers, so the lesson sinks in slowly.

Deep: → Part 28 (why) and Part 6.

Bad answer: "Cross-entropy is for classification because the name says so."

Follow-up: When might MSE on probabilities be acceptable? (Brier score as a metric.) → **Harder:** Is cross-entropy robust to label noise?

Example: Switching a sigmoid+MSE model to BCE fixed slow training.

M4 · How does gradient boosting work?

Testing: functional gradient descent.

30 s: Start with a constant prediction; repeatedly fit a small tree to the negative gradient of the loss (pseudo-residuals) at current predictions; add it with a learning rate.

2 min: For squared loss the pseudo-residuals are residuals; for log loss $y-p$. Leaf values are set by line search / Newton steps. Shrinkage, subsampling and early stopping control variance. Works with any differentiable loss (ranking, quantile, Poisson).

Analogy: Golf putting: the first stroke gets near the hole, and each next stroke aims only at the remaining gap. The ball's final spot is the sum of all strokes — and hitting each one a bit softer (the learning rate) avoids overshooting.

Deep: $F_m=F_{m-1}+\nu h_m$, $h_m\approx-\nabla_F L$ (→ Part 8).

Bad answer: "It averages many trees."

Follow-up: Why a small learning rate? → **Harder:** Derive XGBoost's second-order leaf weights.

Example: LightGBM on tabular credit data.

M5 · Bagging vs. boosting — explain the mechanisms.

Testing: bias/variance mapping.

30 s: Bagging trains strong, high-variance models in parallel on bootstrap samples and averages them → variance ↓. Boosting trains weak, high-bias models sequentially on the current errors → bias ↓.

2 min: Bagging can't fix bias; boosting can overfit with too many rounds and is more noise-sensitive. RF adds feature subsampling to decorrelate. Boosting typically wins on tabular accuracy with tuning; bagging is more robust out of the box.

Analogy: Bagging is averaging many independent weather forecasters, each sharp but erratic — their random errors cancel (variance falls). Boosting is a chain of editors on one essay, each fixing only what the previous ones still got wrong (bias falls).

Deep: $\mathrm{Var} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ vs. additive expansion.

Bad answer: "Boosting is bagging with weights."

Follow-up: Which is more sensitive to label noise and why? → **Harder:** Can you bag boosted models?

Example: RF baseline vs. tuned XGBoost.

M6 · XGBoost vs. LightGBM vs. CatBoost?

Testing: practical library knowledge with mechanisms.

30 s: All are GBDTs. XGBoost: second-order objective with regularization, depth-wise growth by default, mature. LightGBM: histograms, leaf-wise growth, GOSS/EFB — fastest on big/sparse data. CatBoost: ordered target statistics and ordered boosting for categoricals, symmetric trees, strong defaults.

2 min: LightGBM's leaf-wise growth overfits small data unless num_leaves/min_data_in_leaf are tuned. CatBoost shines with many categorical features. All support missing values and GPU. Differences in accuracy are usually small vs. feature engineering and validation.

Analogy: Three brands of pressure cooker that all make the same dal (boosted trees): the sturdy classic with an extra safety valve (XGBoost's regularization), the fastest one for huge batches (LightGBM), and the one with presets for tricky ingredients (CatBoost for categories).

Deep: → Part 8.

Bad answer: "XGBoost is always best."

Follow-up: Why does leaf-wise overfit small data? → **Harder:** What problem does ordered boosting solve?

Example: Many high-cardinality categoricals → CatBoost.

M7 · Explain the kernel trick.

Testing: why dot products matter.

30 s: Algorithms that depend on data only via dot products can replace them with a kernel $K(x, x') = \phi(x)^\top \phi(x')$, implicitly working in a high- or infinite-dimensional feature space without computing ϕ.

2 min: In the SVM dual, predictions are $\sum \alpha_i y_i K(x_i, x) + b$. RBF kernel ⇒ weighted Gaussian bumps around support vectors; γ controls width. Kernels must be PSD (Mercer). Cost: $O(n^2)$ kernel matrix.

Analogy: A matchmaking app that never writes out each person's hundred-page profile, only a compatibility score for each pair: if the algorithm only ever asks 'how similar are these two?', a clever similarity formula lets it act as if it had the giant profiles.

Deep: $(x^\top z)^2 = \phi(x)^\top \phi(z)$ with $\phi = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$.

Bad answer: "The kernel trick adds features to the dataset."

Follow-up: RBF γ too large? → **Harder:** How do random Fourier features approximate kernels at scale?

Example: RBF SVM on a 5K-sample biomedical dataset.

M8 · Why does Naive Bayes work despite a false independence assumption?

Testing: decision vs. probability, bias/variance.

30 s: Classification needs only the right argmax, not correct probabilities; the distortion from correlated features often preserves ranking. And NB has very low variance, which wins with little data.

2 min: Double-counted evidence makes probabilities overconfident, but the decision can remain optimal (Domingos & Pazzani). Ng & Jordan: NB reaches its asymptotic error with far fewer samples than logistic regression. Fails with redundant features, interactions, or when calibrated probabilities are needed.

Analogy: A talent judge who adds points for 'hits high notes' and again for 'wide vocal range' — really the same skill counted twice. Scores come out inflated and overconfident, but who ranks first usually doesn't change, and ranking is all a classifier needs.

Deep: → Part 11.

Bad answer: "Because features are usually independent."

Follow-up: How would you calibrate NB? → **Harder:** What happens if you duplicate a feature 10 times?

Example: NB spam filter trained on 500 emails.

M9 · How do you choose k in k-means?

Testing: unsupervised evaluation judgment.

30 s: Elbow in inertia vs. k, silhouette score, gap statistic, stability across resamples — and above all usefulness for the downstream purpose.

2 min: Inertia always decreases with k, so look for diminishing returns. Silhouette compares within- vs. nearest-cluster distance. Business constraints often fix k (marketing can act on 5 segments). Validate that clusters differ on outcomes that matter.

Analogy: Deciding how many sections to split a school batch into: more sections always makes each one more alike, so look for the point where one more stops helping much — and whether the school can actually staff that many class teachers.

Deep: $s(i) = \frac{b-a}{\max(a,b)}$.

Bad answer: "Pick k with the lowest inertia."

Follow-up: What if clusters aren't spherical? → **Harder:** How would you test whether any cluster structure exists?

Example: Customer segmentation where k = 6 is stable and actionable.

M10 · Why are principal components eigenvectors of the covariance matrix?

Testing: derivation.

30 s: Maximize projected variance $u^\top \Sigma u$ subject to $\|u\| = 1$; the Lagrangian gives $\Sigma u = \lambda u$, and the variance equals λ, so the top eigenvector wins.

2 min: Subsequent components are orthogonal eigenvectors (Σ symmetric). Equivalent to minimizing reconstruction error by Pythagoras. Computed by SVD of centered data.

Analogy: Finding the long axis of an egg-shaped point cloud: rotate a measuring rod until the spread along it is largest. That rod lies along the cloud's axis — a direction the covariance matrix only stretches, never turns — and the spread is its eigenvalue.

Deep: → Part 13 / 31.

Bad answer: "Because eigenvectors are important."

Follow-up: Relationship to SVD? → **Harder:** Why does PCA on unstandardized data mislead?

Example: First PC of stock returns ≈ the market factor.

M11 · PCA vs. t-SNE?

Testing: linear vs. neighborhood-preserving methods.

30 s: PCA is a linear projection preserving global variance, deterministic, with a transform for new data. t-SNE is a nonlinear visualization preserving local neighborhoods; distances between clusters and cluster sizes are not meaningful.

2 min: Use PCA for compression/preprocessing; t-SNE (or UMAP) for exploring cluster structure. t-SNE depends on perplexity and seed; no out-of-sample transform; O(n log n) with Barnes–Hut.

Analogy: PCA is a satellite photo — true distances, but crowded neighbourhoods blur together. t-SNE is a metro map — it keeps who is next to whom, but distances and sizes between areas are distorted, so never measure it with a ruler.

Deep: → Part 13.

Bad answer: "t-SNE is a better PCA."

Follow-up: Can you feed t-SNE outputs to a classifier? → **Harder:** Why does t-SNE use a Student-t kernel?

Example: Visualizing embeddings of support tickets.

M12 · How do momentum and Adam work?

Testing: optimizer behavior.

30 s: Momentum averages past gradients — damps oscillations, accelerates along consistent directions. Adam combines momentum with per-parameter scaling by the root-mean-square of recent gradients, plus bias correction.

2 min: Momentum's effective step ≈ $\eta/(1-\beta)$ in consistent directions. Adam's update is ≈ η × (mean/RMS) per coordinate — robust to gradient scale, good for ill-conditioned/sparse problems. Needs warmup often; AdamW for weight decay.

Analogy: Momentum is a heavy ball rolling downhill: it builds speed in the consistent direction and shrugs off side-to-side bumps. Adam adds a gearbox for each direction, taking shorter steps where gradients have been big and longer ones where they have been small.

Deep: Update equations in Part 14.

Bad answer: "Adam is always better than SGD."

Follow-up: Why bias correction? → **Harder:** Why might SGD generalize better?

Example: AdamW for a transformer; SGD+momentum for a ResNet.

M13 · AdamW vs. Adam with L2 regularization?

Testing: subtle optimizer mechanics.

30 s: Adam + L2 adds λw to the gradient, which then gets divided by $\sqrt{\hat{v}}$ — so weights with large gradient history are barely regularized. AdamW applies decay directly to weights, uniformly.

2 min: For SGD they're equivalent; for adaptive methods they differ. AdamW generalizes better and decouples the optimal learning rate from the decay strength. Usually exclude biases/norm parameters from decay.

Analogy: Adam + L2 is a wealth tax routed through a formula that divides each person's bill by how wildly their income has swung — people with volatile income barely pay. AdamW takes the same percentage of everyone's wealth directly, outside that formula.

Deep: $\theta \leftarrow \theta - \eta(\hat{m}/(\sqrt{\hat{v}}+\epsilon) + \lambda\theta)$.

Bad answer: "They're the same."

Follow-up: Why exclude LayerNorm gains? → **Harder:** How does weight decay interact with normalization layers?

Example: Switching Adam(weight_decay) to AdamW improving validation accuracy.

M14 · How do you choose a classification threshold?

Testing: decision theory and business alignment.

30 s: From costs (with calibrated probabilities: $t^* = C_{FP}/(C_{FP} + C_{FN})$), from constraints (precision ≥ X, capacity), or by maximizing F-β on validation.

2 min: 0.5 is rarely right. Thresholds may differ by segment or by amount (fraud). Revisit when base rates shift; tune on validation, not test.

Analogy: Deciding whether to take a DRS review: if a wasted review is cheap but a missed wrong decision could cost the match, you review even when only 30% sure. The right cut-off comes from the two costs, not from 50%.

Deep: Derivation in Part 6.

Bad answer: "Always 0.5."

Follow-up: What if probabilities aren't calibrated? → **Harder:** Daily volume varies 3× but review capacity is fixed — what do you do?

Example: Review queue of 2,000/day → top-2,000 scores.

M15 · What is calibration, and how do you measure and fix it?

Testing: probability quality beyond ranking.

30 s: Calibrated means predicted 0.3 ⇒ 30% positives. Measure with reliability diagrams, ECE, Brier/log loss. Fix with Platt scaling, isotonic regression, or temperature scaling on a held-out set.

2 min: Matters for cost-based decisions, auctions, combining models. Resampling/class weights, boosting and long-trained nets distort it. AUC is blind to it.

Analogy: A weather app is calibrated if, on all the days it said '30% rain', it rained on about 3 in 10. An app can rank days perfectly — rainier days get higher numbers — and still be miscalibrated if its '30%' really means 60%.

Deep: → Part 6.

Bad answer: "A model with high AUC is calibrated."

Follow-up: Platt vs. isotonic? → **Harder:** Calibrate after negative downsampling analytically.

Example: Ads pCTR calibration per placement.

M16 · ROC-AUC vs. PR-AUC?

Testing: imbalance-aware evaluation.

30 s: ROC-AUC measures ranking over both classes and ignores prevalence; PR-AUC focuses on the positive class and the top of the ranking and is sensitive to prevalence (baseline = prevalence).

2 min: With rare positives, ROC-AUC can look excellent while precision is poor; PR-AUC reveals it. ROC is better for comparing across populations with different base rates.

Analogy: Few gold coins in a mountain of fakes: ROC-AUC asks 'is a gold coin usually ranked above a fake?' — easy to ace. PR-AUC asks 'of the coins you picked, how many are gold?' — with millions of fakes, rare slips flood the pile.

Deep: Precision includes the base rate via Bayes' rule.

Bad answer: "PR-AUC is always better."

Follow-up: What's the random baseline of each? → **Harder:** Why can a model improve ROC-AUC but lower PR-AUC?

Example: Figure in Part 16: AUC 0.91 vs. AP 0.19.

M17 · Explain NDCG and compute an example.

Testing: ranking metrics.

30 s: DCG sums graded relevance discounted by log position; NDCG divides by the ideal DCG so it's in [0, 1].

2 min: $\mathrm{DCG@K} = \sum \frac{2^{rel_i}-1}{\log_2(i+1)}$. Captures graded relevance and position importance; offline labels suffer exposure and position bias; LambdaMART optimizes it.

Analogy: Grading a food-delivery app's search results: a great restaurant at the top earns full credit, the same one at position 8 earns much less because few people scroll that far. Dividing by the perfect ordering's score gives a 0–1 grade.

Deep: Worked example in Part 16 (NDCG@5 = 0.876).

Bad answer: "NDCG is precision at K."

Follow-up: MRR vs. NDCG? → **Harder:** Why can NDCG improve offline while CTR falls?

Example: Search ranking evaluation.

M18 · How do you validate a time-series or temporal model?

Testing: leakage awareness.

30 s: Train on the past, validate on the future — rolling or expanding windows — with a gap covering label windows and feature lags; never shuffle.

2 min: Match the production horizon and retraining cadence; evaluate across multiple folds (seasons); point-in-time features. Report degradation with forecast horizon.

Analogy: Testing a stock-picker fairly: let him study data up to March and judge his calls for April, leaving a gap so no April news leaks in. Shuffled K-fold is like handing him tomorrow's newspaper.

Deep: → Part 17 diagram.

Bad answer: "Standard K-fold."

Follow-up: Why a gap? → **Harder:** How do you validate when labels arrive 60 days late?

Example: Fraud model trained on Jan–Jun, tested on August.

M19 · What is target encoding, and how does it leak?

Testing: high-cardinality encoding rigor.

30 s: Replace a category by the mean target in that category. If computed on the same rows it encodes, each row's own label leaks into its feature.

2 min: Use out-of-fold (or time-ordered) statistics, smoothing toward the global mean for rare categories, possibly noise. CatBoost's ordered statistics do this automatically.

Analogy: Predicting a student's marks from 'average score of her tuition class' — computed including her own score. In a class of two, that average is half her own score, so the feature quietly contains the answer.

Deep: $\mathrm{TE} = \frac{n_c \bar{y}_c + m\bar{y}}{n_c + m}$.

Bad answer: "It's fine as long as you use it only on training data."

Follow-up: What happens with a category that has one row? → **Harder:** Is out-of-fold TE safe for temporal data?

Example: Merchant ID encoded by historical fraud rate.

M20 · Explain MLE and MAP, and their relation to regularization.

Testing: probabilistic foundations.

30 s: MLE maximizes $p(D \mid \theta)$; MAP maximizes $p(\theta \mid D) \propto p(D \mid \theta)p(\theta)$. The log-prior acts as a regularizer: Gaussian prior ⇒ L2, Laplace prior ⇒ L1.

2 min: MAP → MLE as data grows. MAP is a point estimate; full Bayesian inference keeps the posterior. $\lambda = \sigma^2/\tau^2$ for ridge.

Analogy: MLE rates a new restaurant purely on its reviews; MAP starts from the belief that most restaurants are average and blends that with the reviews. With three reviews the prior pulls hard (like regularization); with 3,000 the reviews win.

Deep: Derivations in Part 3.

Bad answer: "MAP is MLE with more data."

Follow-up: Beta prior on a coin? → **Harder:** Why is MAP not invariant to reparameterization?

Example: Smoothing CTR of new ads.

M21 · Why is the MLE of a Gaussian variance biased?

Testing: estimator properties.

30 s: It uses the sample mean, which is fitted to the same data and sits closer to the points than the true mean, so squared deviations are systematically too small: $\mathbb{E}[\hat{\sigma}^2] = \frac{n-1}{n}\sigma^2$.

2 min: Dividing by n−1 (Bessel) makes it unbiased. MLE is consistent (bias → 0). Unbiasedness isn't everything — biased estimators can have lower MSE.

Analogy: Measuring how spread out a batsman's scores are using gaps from his own average of those same innings: that average was built to sit in their middle, so the gaps look slightly smaller than the true spread. Dividing by n−1 corrects it.

Deep: $\sum(x_i - \bar{x})^2 = \sum(x_i - \mu)^2 - n(\bar{x} - \mu)^2$, take expectations.

Bad answer: "Because n−1 is more accurate."

Follow-up: Is the n−1 estimator's square root (s) unbiased for σ? (No.) → **Harder:** Connect to degrees of freedom in regression residuals.

Example: Sample std in A/B test formulas.

M22 · What is the CLT and why does it matter for A/B tests?

Testing: statistical foundations.

30 s: Means of IID samples with finite variance are approximately normal with variance σ^2/n, regardless of the original distribution — which justifies z-tests and confidence intervals on metric averages.

2 min: Error bars shrink as $1/\sqrt{n}$; heavy tails slow convergence (revenue metrics); correlated units (users with many sessions) break naive variance estimates — use the right unit or cluster-robust variance.

Analogy: One kirana customer's bill can be anything from Rs 10 to Rs 5,000, but the average bill over 1,000 customers barely moves and follows a bell curve, whatever single bills look like. A/B test error bars rely on that predictable wobble.

Deep: → Part 3.

Bad answer: "The data becomes normal with more samples."

Follow-up: How many samples to halve the CI? (4×.) → **Harder:** What if variance is infinite?

Example: Revenue per user winsorized for stable inference.

M23 · Type I vs. Type II errors, power, and sample size.

Testing: experimental design.

30 s: Type I: false positive (rate α). Type II: false negative (rate β). Power = 1 − β. Sample size grows with $(z_{\alpha/2} + z_{\beta})^2\sigma^2/\delta^2$.

2 min: Choose α and power before the test, pick an MDE that's worth detecting; underpowered tests produce noisy, inflated "wins". Variance reduction (CUPED) buys power.

Analogy: A smoke alarm: a Type I error rings with no fire; a Type II error stays silent during a real one. Power is the chance a real fire is caught; reliably catching a candle-sized fire needs a far more sensitive alarm (many more users).

Deep: 4.1% → 4.3% CTR needs ≈158K users/arm at 80% power.

Bad answer: "More data always reduces Type I error." (α is fixed by design.)

Follow-up: What's MDE? → **Harder:** Why do significant results from underpowered tests overestimate effects?

Example: Power analysis before a ranking experiment.

M24 · How do you handle multiple hypothesis testing?

Testing: false discovery awareness.

30 s: Many tests inflate false positives (20 tests at 5% → 64% chance of at least one). Control FWER with Bonferroni/Holm or FDR with Benjamini–Hochberg; pre-register a primary metric.

2 min: FWER when any false positive is costly (launch decisions); FDR for screening many candidates. Treat segment analyses as exploratory and confirm with follow-up tests.

Analogy: Buy 20 lottery tickets and your chance of some win jumps, though no ticket got better. Test 20 metrics at 5% and 'something significant' is likely by pure luck — so raise the bar per test (Bonferroni) or cap the share of false wins (FDR).

Deep: → Part 4 table.

Bad answer: "Just use p < 0.01."

Follow-up: Holm vs. Bonferroni? → **Harder:** Does looking at many metrics in one test count?

Example: 30 segment cuts in an experiment readout.

M25 · What is Simpson's paradox? Give an ML example.

Testing: confounding by group composition.

30 s: A trend in every subgroup reverses when groups are combined, because group sizes differ between compared arms.

2 min: Kidney stones: treatment A better for small and large stones, worse overall because it got harder cases. ML: model B better on every country but worse overall because it was evaluated on a harder country mix; or A/B traffic ramps mixing days.

Analogy: Batsman A averages more than B both at home and away, yet less overall — because most of A's innings were on tough away tours. The combined figure mixes up 'who bats better' with 'where they batted'.

Deep: Stratify or adjust for the confounder.

Bad answer: "It's a statistical error in the data."

Follow-up: Which comparison is right? → **Harder:** How does it arise in A/B tests with changing allocation?

Example: Segment-level model evaluation.

M26 · Explain Xavier and He initialization.

Testing: signal propagation.

30 s: Choose weight variance so activations and gradients keep roughly constant variance across layers: Xavier $2/(n_{in}+n_{out})$ for tanh/linear; He $2/n_{in}$ for ReLU.

2 min: $\mathrm{Var}(z) = n_{in}\,\mathrm{Var}(w)\,\mathrm{Var}(x)$; ReLU halves the second moment so He doubles the variance. Too small → vanishing; too large → exploding/saturation. Zero init fails due to symmetry.

Analogy: Setting the gain on 50 chained amplifiers: too low and the music fades out, too high and it distorts. The right gain keeps each stage as loud as its input; ReLU silences half the signal, so He initialization doubles the power.

Deep: → Part 20.

Bad answer: "Random small numbers are fine."

Follow-up: Why not zero init? → **Harder:** How do residual networks adjust init (zero-init last layer of branches)?

Example: Deep ReLU MLP that won't train with N(0, 0.01²) init.

M27 · BatchNorm vs. LayerNorm?

Testing: normalization choice.

30 s: BatchNorm normalizes each feature across the batch; LayerNorm normalizes across features within each example. BN depends on batch statistics and behaves differently at inference; LN doesn't.

2 min: BN suits CNNs with large batches; LN suits transformers/RNNs, variable-length sequences and small batches. BN adds regularization noise; LN is deterministic.

Analogy: BatchNorm curves each question's marks across the whole class; LayerNorm curves each student's marks across their own questions. The class curve gets shaky with only two students (tiny batches) and needs stored class averages when one student is graded alone later (inference).

Deep: → Part 20 table.

Bad answer: "They're the same with different names."

Follow-up: What happens to BN with batch size 2? → **Harder:** Why pre-LN in transformers?

Example: Switching BN→GroupNorm for detection with small batches.

M28 · Why do residual connections help?

Testing: gradient flow and optimization.

30 s: $h_{l+1} = h_l + F(h_l)$ provides an identity path: gradients flow directly to early layers, and blocks can easily learn to do nothing, so adding depth doesn't hurt optimization.

2 min: Jacobian $I + \partial F/\partial h$ avoids vanishing products; fixes the degradation problem; behaves like an ensemble of shallow paths. Enables very deep CNNs and transformers.

Analogy: A highway with an express lane beside every town's local streets: the gradient can always zoom straight back along the express lane, and each block only adds a small detour on top — if a block is useless, traffic simply skips it.

Deep: → Part 28 (why).

Bad answer: "They add more parameters."

Follow-up: What's the degradation problem? → **Harder:** Why is pre-LN's clean residual stream more stable?

Example: ResNet-50 vs. plain 50-layer CNN.

M29 · Why nonlinear activations, and why ReLU over sigmoid?

Testing: expressivity and gradient behavior.

30 s: Without nonlinearity, depth collapses into a linear map. ReLU doesn't saturate for positive inputs, so gradients don't vanish, and it's cheap; sigmoid saturates (derivative ≤ 0.25) and isn't zero-centered.

2 min: ReLU risks dead units — leaky ReLU/GELU mitigate. Sigmoid remains useful at outputs and gates.

Analogy: Sigmoid is a dimmer already fully bright or fully dark for most knob positions, so turning further changes nothing (the gradient dies). ReLU is a tap that stays shut below zero but opens in direct proportion above it — turn more, get more.

Deep: → Part 20.

Bad answer: "ReLU is faster, that's all."

Follow-up: Why does non-zero-centered output slow training? → **Harder:** Is ReLU differentiable?

Example: Swapping activations to fix a stuck deep net.

M30 · How do you detect and fix exploding gradients?

Testing: training diagnostics.

30 s: Symptoms: loss spikes, NaNs, huge gradient norms. Fix with gradient clipping, lower LR/warmup, better initialization, normalization, residual scaling.

2 min: Log per-layer gradient norms; check for data outliers; mixed-precision overflow; RNNs with spectral radius > 1.

Analogy: A microphone held too close to its speaker: each loop amplifies the sound a little more until it screeches (NaN). Gradient clipping is a volume limiter, and warmup is starting the talk softly before speaking at full voice.

Deep: Product of Jacobians with gain > 1.

Bad answer: "Add more layers."

Follow-up: Clip by norm vs. by value? → **Harder:** Why doesn't clipping fix vanishing gradients?

Example: Transformer loss spikes fixed by warmup + clipping at 1.0.

M31 · How do you handle a very high-cardinality categorical feature?

Testing: practical encoding at scale.

30 s: Avoid one-hot. Use hierarchy, frequency bucketing, smoothed out-of-fold target encoding, hashing, embeddings, or native categorical handling — and replace pure IDs by aggregate behavior features.

2 min: Plan for unseen values at serving; validate with unseen categories in the validation set; watch leakage and overfitting on rare categories.

Analogy: A kirana owner can't remember 2 million customers by name, so he thinks in groups ('colony regulars') and habits ('buys milk daily, pays by UPI') — and keeps a sensible default for the first-time customer he has never seen.

Deep: → Part 19.

Bad answer: "One-hot encode and let the model figure it out."

Follow-up: Hashing collisions? → **Harder:** Embeddings for IDs that appear once?

Example: 2M product IDs in a CTR model.

M32 · Explain SMOTE and its pitfalls.

Testing: imbalance techniques critically.

30 s: SMOTE creates synthetic minority examples by interpolating between a minority point and one of its minority neighbors.

2 min: Pitfalls: creates points in majority regions (noise), poor with high dimensions and categorical features, leaks if applied before splitting, distorts calibration, rarely helps strong learners like GBMs. Threshold tuning or class weights are often better.

Analogy: Padding a thin album of rare-bird photos by blending two real ones into an in-between bird: sometimes useful, but blends can land in pigeon territory, and if blended photos slip into the test album, your quiz score is fake.

Deep: → Part 18.

Bad answer: "SMOTE always improves imbalanced problems."

Follow-up: Why not apply SMOTE to the validation set? → **Harder:** How would you correct calibration after SMOTE?

Example: SMOTE inflating CV recall because it was applied pre-split.

M33 · Hinge loss vs. log loss?

Testing: margin vs. probability.

30 s: Hinge $\max(0, 1 - yf)$ is zero beyond the margin — sparse (support vectors), no probabilities. Log loss $\log(1 + e^{-yf})$ is smooth, never zero — uses all points, estimates probabilities.

2 min: Both are convex surrogates of 0-1 loss; similar accuracy on many problems; choose log loss when you need calibrated probabilities.

Analogy: Hinge loss is a teacher who stops caring once you've passed with a safe margin — only borderline students get attention (the support vectors). Log loss keeps nudging even top students a little, and can tell you each student's chance of passing.

Deep: Figure in Part 15.

Bad answer: "Hinge is for regression."

Follow-up: What does a far-away correct point contribute to each? → **Harder:** What does GD on separable logistic regression converge to? (max-margin direction.)

Example: Linear SVM vs. logistic regression on text.

M34 · When and why use focal loss?

Testing: gradient reweighting.

30 s: When many easy examples (usually negatives) dominate the loss. The factor $(1 - p_t)^\gamma$ down-weights well-classified examples, focusing learning on hard ones.

2 min: From dense object detection. Downsides: miscalibration, amplifies label noise, extra hyperparameters. Compare against weighted CE + threshold tuning.

Analogy: A tennis coach who spends almost no practice time on shots the player already hits perfectly and pours effort into the ones she keeps missing. The risk: if some 'misses' were just bad line calls (label noise), the coach obsesses over them.

Deep: At $p_t = 0.9$, γ=2 weight = 0.01.

Bad answer: "Focal loss fixes imbalance automatically."

Follow-up: Why does it hurt calibration? → **Harder:** How does it interact with label noise?

Example: Rare defect detection in images.

M35 · When would you use Huber loss?

Testing: robust regression.

30 s: When errors are mostly Gaussian but with occasional outliers: quadratic for small residuals, linear for large ones, so outliers have bounded influence.

2 min: δ sets the transition (in target units); gradient = clipped residual. Used in bounding-box regression ("smooth L1"), robust forecasting, RL value losses.

Analogy: A late-homework fine that climbs faster each day for the first week (squared), then adds a fixed amount per extra day (linear): ordinary lateness is taken seriously, yet one student who vanishes for a month can't dominate the class total.

Deep: $\partial \ell / \partial \hat{y} = -\text{clip}(r, -\delta, \delta)$.

Bad answer: "Huber is a classification loss."

Follow-up: How to choose δ? → **Harder:** Huber vs. quantile loss?

Example: ETA with GPS glitch outliers.

M36 · What is multicollinearity, and what does it affect?

Testing: linear model diagnostics.

30 s: Features that are (near-)linear combinations of others. It inflates coefficient variance — unstable, sign-flipping coefficients — but predictions within the data's distribution are mostly fine.

2 min: Diagnose with VIF or condition number. Fix: drop/combine features, ridge, PCA. Matters for interpretation and for prediction when correlations change in production.

Analogy: Two friends who always carry a heavy box together: you can see the box move, but not who is lifting how much, so any split you estimate is shaky. The box still arrives fine — until the day only one of them turns up.

Deep: $\mathrm{Var}(\hat{w}) = \sigma^2 (X^\top X)^{-1}$.

Bad answer: "It makes predictions wrong."

Follow-up: How does ridge treat two identical features? → **Harder:** How does lasso?

Example: Height in cm and inches in one model.

M37 · What is heteroscedasticity and why does it matter?

Testing: regression assumptions.

30 s: Error variance depends on the inputs. OLS stays unbiased, but it's inefficient and standard errors/CIs are wrong.

2 min: Detect with residual funnel plots or Breusch–Pagan. Fix with robust (sandwich) standard errors, weighted least squares, log-transforming the target, or modeling the variance.

Analogy: Guessing monthly spending: for students you're off by a few hundred rupees, for business owners by lakhs. Guesses can be right on average, yet one '± Rs 500' error bar is wrong for everyone — too wide for students, far too narrow for the rich.

Deep: → Part 5.

Bad answer: "It makes coefficients biased."

Follow-up: When does it matter for prediction? (prediction intervals.) → **Harder:** Why does a log transform often fix it?

Example: Spending errors growing with income.

M38 · What feature importance methods exist, and what are their pitfalls?

Testing: interpretability rigor.

30 s: Impurity-based (MDI), permutation importance, SHAP, drop-column. MDI is biased toward high-cardinality features and computed on training data; permutation is misled by correlated features; SHAP explains the model, not causality.

2 min: Use validation-set permutation or SHAP; group correlated features; check stability across retrains. Never infer causation.

Analogy: Crediting a football win: counting touches flatters busy midfielders; swapping a player for a random stand-in misleads when two teammates cover the same role; and none of it proves who caused the win — only whom the team leaned on.

Deep: → Part 7.

Bad answer: "Importance tells you which features cause the outcome."

Follow-up: Why is user_id "important"? → **Harder:** How do correlated features affect SHAP values?

Example: Leaked feature discovered via SHAP.

M39 · Online vs. batch inference?

Testing: serving architecture.

30 s: Batch precomputes predictions on a schedule (cheap, stale); online computes on request (fresh, latency-constrained).

2 min: Batch: churn scores, nightly recs. Online: fraud, search, ads. Hybrids precompute candidates/embeddings and re-rank online. Online needs real-time features and fallbacks.

Analogy: Batch inference is a tiffin service cooking tomorrow's lunches tonight — cheap, but fixed before you know your mood. Online inference is a restaurant cooking to order — fresh and tailored, but the kitchen must be fast and always open.

Deep: → Part 23 table.

Bad answer: "Online inference means the model trains online."

Follow-up: How do you handle new users in batch scoring? → **Harder:** How do you guarantee feature parity between the two?

Example: Nightly recommendations + real-time session re-ranking.

M40 · What should you monitor for a deployed model?

Testing: production maturity.

30 s: System health, data quality, input drift, prediction drift, performance on labeled traffic (often delayed), and business metrics — by slice.

2 min: Alert on effect sizes, not p-values; use label-free proxies until labels mature; tie alerts to runbooks; detect training–serving skew by logging served features.

Analogy: A car dashboard: engine temperature and fuel (system health), odd noises (input drift), the speedometer (prediction drift), and the mileage report that only comes later (true performance, once labels arrive). Each warns about a different failure.

Deep: PSI $= \sum (p - q) \ln(p/q)$.

Bad answer: "Monitor accuracy."

Follow-up: How with 90-day label delay? → **Harder:** How to tell a pipeline bug from concept drift?

Example: Null-rate alert catching a broken upstream join.

M41 · Covariate shift vs. concept drift?

Testing: drift taxonomy.

30 s: Covariate shift: $P(x)$ changes, $P(y \mid x)$ stable. Concept drift: $P(y \mid x)$ changes — same inputs, different outcomes.

2 min: Covariate shift is detectable from inputs alone; concept drift needs labels. Remedies: reweighting/retraining vs. retraining on recent labels, new features. Label shift: $P(y)$ changes with $P(x \mid y)$ stable.

Analogy: Covariate shift: a shop opens in a new neighbourhood with different customers, but each type still buys what it always did. Concept drift: the same customers now want different things. The first shows in who walks in; the second only in what they buy.

Deep: Importance weights $P_{test}(x)/P_{train}(x)$.

Bad answer: "They're the same thing."

Follow-up: Detect covariate shift? (domain classifier.) → **Harder:** Correct label shift without labels?

Example: Fraudsters changing tactics (concept drift).

M42 · Explain softmax and how to compute it stably.

Testing: numerics.

30 s: $\mathrm{softmax}(z)_k = e^{z_k} / \sum_j e^{z_j}$ turns logits into a distribution. Subtract the max logit before exponentiating (softmax is shift-invariant) and compute log-probabilities with log-sum-exp.

2 min: Gradient of CE w.r.t. logits is $p - y$. Temperature scaling controls sharpness. For K = 2 it reduces to sigmoid of the difference.

Analogy: Turning vote counts into vote shares, but boosting each count exponentially first so leaders get outsized shares. Measuring every count relative to the leader before boosting leaves the shares unchanged, yet keeps the numbers small enough for a calculator.

Deep: → Part 6.

Bad answer: "Normalize logits by their sum."

Follow-up: Why not `log(softmax(z))`? → **Harder:** Softmax cost with 1M classes?

Example: Stable CE implementation in Part 21.

M43 · Why is early stopping a form of regularization?

Testing: implicit regularization.

30 s: Training time limits how far weights move from initialization; directions with weak data support are fitted last, so stopping early keeps the solution simpler.

2 min: For linear least squares with GD from zero, early stopping ≈ ridge with $\lambda \approx 1/(\eta t)$. It's cheap, but depends on a representative validation set and noisy metrics.

Analogy: A sculptor carving a block: early strokes rough out the big shape anyone would agree on; only later does he chase tiny cracks peculiar to this one block. Stopping early keeps the statue simple — like a ridge penalty, without writing one down.

Deep: Per eigen-direction, GD fits $1-(1-\eta\lambda_i)^t$ of the OLS coefficient vs. ridge's $\lambda_i/(\lambda_i+\lambda)$.

Bad answer: "It just saves compute."

Follow-up: How to pick patience? → **Harder:** Can early stopping hurt with double descent over epochs?

Example: Stopping a BERT fine-tune after 2 epochs.

M44 · What is a feature store, and why use one?

Testing: ML infrastructure.

30 s: A system that manages feature definitions and serves them consistently: an offline store for point-in-time-correct training data and an online store for low-latency serving.

2 min: Prevents training–serving skew and leakage, enables reuse and monitoring of feature freshness. Costs: infrastructure complexity; not needed for small projects.

Analogy: A restaurant chain's central kitchen that makes every sauce from one recipe, keeping dated batches for the test kitchen and fresh stock at every outlet — so the dish tested is exactly the dish served.

Deep: → Part 23.

Bad answer: "A database of features."

Follow-up: What is a point-in-time join? → **Harder:** How do you handle streaming features with late events?

Example: Velocity features for fraud.

M45 · How would you scale KNN to 100M items?

Testing: approximate search engineering.

30 s: Use approximate nearest-neighbor indexes (HNSW graphs, IVF with product quantization) over good embeddings, trading slight recall for orders-of-magnitude speed.

2 min: Tune recall vs. latency (efSearch, nprobe); compress vectors (PQ) to fit memory; shard; filter by metadata; rebuild/update indexes as items change; measure recall@K against exact search offline.

Analogy: Finding a book in a library of 100 million: you don't compare it with every book — you walk to the right section and shelf and check a few nearby. You occasionally miss the single best match, but it is thousands of times faster.

Deep: Brute force is $O(nd)$ per query.

Bad answer: "Use a KD-tree." (Breaks down in high dimensions.)

Follow-up: How do you evaluate ANN quality? → **Harder:** How do you handle deletes and freshness?

Example: Two-tower retrieval with HNSW.

27.3 Hard

H1 · Derive the bias–variance decomposition.

Testing: derivation with correct expectations.

30 s: With $y = f + \varepsilon$, $\mathbb{E}\varepsilon = 0$, $\operatorname{Var}\varepsilon = \sigma^2$, $\bar{f} = \mathbb{E}_D \hat{f}$: $\mathbb{E}[(y - \hat{f})^2] = \sigma^2 + (f - \bar{f})^2 + \mathbb{E}[(\hat{f} - \bar{f})^2]$.

2 min: Expand $(y - \hat{f}) = (y - f) + (f - \bar{f}) + (\bar{f} - \hat{f})$; cross terms vanish because ε is independent of D with mean zero, and $\mathbb{E}_D[\bar{f} - \hat{f}] = 0$. Valid for squared loss at a fixed x; average over x for the global statement.

Analogy: A cricket bowler's miss from the stumps has three parts: gusty wind nobody controls (noise), a line that on average drifts down leg side (bias), and a wobble that changes from one net session to the next (variance); in squared distance, the three simply add.

Deep: Whiteboard in Part 31.

Bad answer: Writing the result without saying which expectation is over what.

Follow-up: Does it hold for 0-1 loss? (Not additively.) → **Harder:** How does bagging change each term?

Example: KNN with k=1 vs. k=50.

H2 · Derive the normal equation. When does it fail and what do you do?

Testing: linear algebra and numerics.

30 s: $\nabla \|y - Xw\|^2 = -2X^\top(y - Xw) = 0 \Rightarrow X^\top X w = X^\top y$. Fails when $X^\top X$ is singular (collinearity, d > n) or ill-conditioned.

2 min: Remedies: pseudo-inverse / lstsq (minimum-norm solution), ridge ($+\lambda I$ makes it PD), QR/SVD solvers (avoid squaring the condition number), gradient methods for huge d.

Analogy: Working out each friend's share from many joint restaurant bills works until two friends always order identically (collinearity): then no unique split exists, so add a rule like "keep shares small" (ridge) or "pick the smallest split" (pseudo-inverse).

Deep: $\kappa(X^\top X) = \kappa(X)^2$.

Bad answer: "Just invert $X^\top X$."

Follow-up: Geometric meaning? (projection, residual ⊥ columns.) → **Harder:** Which solution does GD from zero converge to when d > n?

Example: One-hot with intercept (dummy trap).

H3 · Why does ridge shrink low-variance directions more?

Testing: SVD view of regularization.

30 s: In the SVD basis, ridge multiplies each component's OLS fit by $s_j^2/(s_j^2 + \lambda)$: small singular values (low data variance) are shrunk most.

2 min: Those directions have the noisiest OLS estimates (variance σ^2/s_j^2), so shrinking them trades little bias for a big variance cut. Effective degrees of freedom $\sum s_j^2/(s_j^2 + \lambda)$.

Analogy: A dhaba rated by 500 diners is taken almost at face value, but one rated by two diners is pulled strongly toward neutral; ridge likewise shrinks most the directions where the data barely varies and the estimates are mostly noise.

Deep: $\hat{w}_{\text{ridge}} = \sum_j \frac{s_j}{s_j^2+\lambda}(u_j^\top y)v_j$.

Bad answer: "Ridge shrinks all weights equally."

Follow-up: Relation to PCA regression? (hard cutoff vs. soft shrinkage.) → **Harder:** How does early stopping compare per direction?

Example: Collinear features stabilized by ridge.

H4 · Derive backprop for a 2-layer network.

Testing: can you do it cleanly.

30 s: With $z_1 = W_1 x$, $a_1 = \phi(z_1)$, $z_2 = W_2 a_1$, softmax-CE: $\delta_2 = p - y$; $\partial L/\partial W_2 = \delta_2 a_1^\top$; $\delta_1 = (W_2^\top \delta_2) \odot \phi'(z_1)$; $\partial L/\partial W_1 = \delta_1 x^\top$.

2 min: Explain the three repeated moves, shapes, caching, and cost (≈2× forward). Verify with gradient checking.

Analogy: Like tracing a complaint back through a kitchen: the final error goes first to the plating station, then each earlier station gets its share, weighted by how much it influenced the next, reusing the note passed back instead of re-investigating.

Deep: Numeric example in Part 20.

Bad answer: Confusing the order of matrix products or shapes.

Follow-up: Batch version? → **Harder:** Where do vanishing gradients appear in these equations?

Example: NumPy MLP in Part 21.

H5 · Derive XGBoost's optimal leaf weight and split gain.

Testing: second-order boosting.

30 s: Second-order Taylor: per leaf $G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2 \Rightarrow w_j^* = -G_j/(H_j + \lambda)$; objective reduction $-\frac{1}{2} G_j^2/(H_j + \lambda)$; gain = left + right − parent, minus γ.

2 min: g, h are first/second derivatives per example (log loss: $p - y$, $p(1-p)$). λ shrinks leaves, γ prunes splits, min_child_weight bounds H.

Analogy: Setting a leaf's value is like adjusting one thermostat for a group of rooms: move against their total error (gradient sum) divided by how sharply they respond (curvature sum, plus λ for caution); split the group only if that saves more than γ.

Deep: Part 8 whiteboard.

Bad answer: "XGBoost is gradient boosting with regularization."

Follow-up: What is min_child_weight for log loss? → **Harder:** Why does second-order information help?

Example: Tuning γ to remove weak splits.

H6 · Show that the gradient of softmax cross-entropy w.r.t. logits is p − y.

Testing: Jacobian algebra.

30 s: $\partial p_k/\partial z_j = p_k(\delta_{kj} - p_j)$; $\partial \ell/\partial z_j = -\sum_k y_k(\delta_{kj} - p_j) = p_j - y_j$.

2 min: Alternative: $\ell = -z_y + \log \sum e^{z_k} \Rightarrow$ derivative $-\mathbb{1}[j = y] + p_j$. Implement from logits for stability.

Analogy: Like grading a student who spreads 100 confidence points across an MCQ's options: the correction for each option is simply the points they gave it minus the points the answer key gives (100 to the right option, 0 elsewhere).

Deep: Part 6.

Bad answer: Differentiating softmax elementwise as if outputs were independent.

Follow-up: Label smoothing's gradient? ($p - \tilde{y}$.) → **Harder:** Gradient w.r.t. temperature?

Example: Hand-coded MLP backward pass.

H7 · Why might Adam converge faster but SGD generalize better?

Testing: optimizer implicit bias.

30 s: Adam's per-parameter scaling handles ill-conditioning → fast early progress. But its solutions can differ from SGD's (not minimum-norm; noise dampened), and in some settings (CNN image classification) generalize worse; AdamW and tuning close much of the gap.

2 min: Mechanisms: implicit bias (GD → min-L2-norm solution in linear least squares; adaptive methods' solution depends on coordinate rescaling), SGD noise favoring flatter regions, uneven regularization with Adam+L2. Setting-dependent: transformers need Adam(W).

Analogy: Adam is a student who paces each chapter separately and climbs mock-test scores fast; SGD studies everything at one steady, slightly noisy pace, slower but sometimes landing on broader understanding that holds up better in the real board exam.

Deep: Part 14 why-box.

Bad answer: "Adam overfits because it's too fast."

Follow-up: Why are transformers trained with AdamW? → **Harder:** How would you test the flat-minima hypothesis?

Example: ResNet: SGD+cosine; BERT: AdamW + warmup.

H8 · Explain double descent.

Testing: modern generalization.

30 s: Test error vs. capacity rises toward the interpolation threshold (parameters ≈ data points) then falls again as capacity grows further, because among many interpolating solutions the optimizer picks smooth, low-norm ones.

2 min: Near the threshold, only one contorted interpolating solution exists (high variance); beyond it, minimum-norm interpolation spreads noise-fitting over many directions. Appears over model size, epochs and even data size. Regularization and ensembling flatten the peak.

Analogy: Drawing through dots with a hinged ruler: too few hinges misses dots, exactly enough hinges forces sharp kinks to hit every dot, but far more hinges allow gentle bends through all of them, and training naturally picks the gentlest.

Deep: For min-norm least squares with random features, the variance term diverges at $p = n$.

Bad answer: "Bigger models are always better."

Follow-up: Why can more data hurt near the threshold? → **Harder:** How does this square with bias–variance?

Example: Random-features regression sweep.

H9 · Why do overparameterized neural networks generalize despite being able to fit random labels?

Testing: honest, nuanced reasoning.

30 s: Capacity bounds are worst-case; what matters is which solution training finds. SGD, architecture and data structure bias learning toward simple, structured functions that fit real labels quickly; random labels take much longer to memorize.

2 min: Evidence: real patterns are learned before noise; implicit regularization (min-norm, flatness), architectural priors (convolution, attention), data augmentation, weight decay. Norm-/margin-/compression-based bounds are more informative than parameter counts. It's an open research area.

Analogy: A student with a huge memory could memorize a random phone directory, but given a real textbook they grasp the patterns first, because patterns are quicker to learn; being able to memorize doesn't mean you must.

Deep: Zhang et al. (2017) random-label experiments.

Bad answer: "Because of dropout."

Follow-up: How would you measure effective capacity? → **Harder:** Why does training on noisy labels eventually memorize them?

Example: Early stopping as protection against memorizing noisy labels.

H10 · Explain the SVM dual and what support vectors are.

Testing: constrained optimization intuition.

30 s: The dual maximizes $\sum \alpha_i - \frac{1}{2} \sum \alpha_i \alpha_j y_i y_j x_i^\top x_j$ with $0 \le \alpha_i \le C$, $\sum \alpha_i y_i = 0$; $w = \sum \alpha_i y_i x_i$. Support vectors have $\alpha_i > 0$ — on or inside the margin.

2 min: Complementary slackness: points strictly outside the margin have $\alpha_i = 0$. $0 < \alpha_i < C$: on the margin; $\alpha_i = C$: violating. Data enters only via dot products → kernels. n variables → costly for large n.

Analogy: The boundary is like a fence between two neighbourhoods, placed using only the houses nearest it or on the wrong side (support vectors); houses further back could move or vanish and the fence wouldn't budge.

Deep: Part 9.

Bad answer: "Support vectors are the closest points to each other."

Follow-up: What happens if you delete a non-support vector? → **Harder:** How does C appear in the dual?

Example: Only 8% of points being support vectors on clean data.

H11 · How is k-means related to Gaussian mixture models and EM?

Testing: probabilistic view of clustering.

30 s: k-means is the limit of EM for a GMM with equal, isotropic covariances $\sigma^2 I$ as $\sigma \to 0$: soft responsibilities become hard assignments and the M-step becomes the mean update.

2 min: GMM gives soft assignments, elliptical clusters, likelihood-based model selection (BIC), but more parameters and possible degenerate solutions (a component collapsing on one point). EM monotonically increases likelihood; local optima.

Analogy: EM for a GMM is like splitting each student's time across several tuition batches by how well they fit each; k-means is the strict limit where everyone joins exactly one batch and every batch is assumed round and equally spread.

Deep: Responsibilities $\gamma_{ik} \propto \pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)$.

Bad answer: "GMM is k-means with probabilities." (Close, but misses covariance and limits.)

Follow-up: Why can GMM likelihood go to infinity? → **Harder:** How do you pick the number of components?

Example: Soft customer segments with GMM.

H12 · Forward vs. reverse KL — which is mode-seeking and why?

Testing: asymmetry reasoning.

30 s: Fitting Q to P: forward $\mathrm{KL}(P\|Q)$ heavily penalizes $q \approx 0$ where $p > 0$ ⇒ mass-covering. Reverse $\mathrm{KL}(Q\|P)$ penalizes $q > 0$ where $p \approx 0$ ⇒ Q concentrates on one mode (mode-seeking).

2 min: MLE = forward KL. Variational inference uses reverse KL (tractable expectation under Q) → underestimates uncertainty. RLHF KL penalties keep a policy near a reference.

Analogy: Fitting one spotlight to a stage with two actors: forward KL hates leaving any actor in the dark, so it widens to cover both (and the gap); reverse KL hates lighting empty stage, so it locks onto one actor.

Deep: Expectations are taken under the first argument.

Bad answer: "KL is a distance so direction doesn't matter."

Follow-up: Which does knowledge distillation use? → **Harder:** Why is VI's posterior overconfident?

Example: Fitting a unimodal Gaussian to a bimodal distribution.

H13 · Derive the cost-optimal classification threshold.

Testing: decision theory.

30 s: Predict positive if $(1-p)C_{FP} < pC_{FN} \Rightarrow p > C_{FP}/(C_{FP}+C_{FN})$, assuming calibrated p and no cost for correct decisions.

2 min: With benefits for correct decisions, use net costs. With per-example costs (fraud amount), thresholds vary per example. Miscalibration breaks this — calibrate first.

Analogy: Like deciding whether to carry an umbrella in the monsoon: if getting soaked costs nine times more than carrying it needlessly, carry it whenever the rain chance tops 1/(1+9) = 10%, not 50% — if the forecast's percentages are honest.

Deep: Bayes decision rule minimizing expected cost.

Bad answer: "Threshold is 0.5 for balanced costs." (True only if calibrated and costs equal.)

Follow-up: What if costs change by segment? → **Harder:** Capacity-constrained version?

Example: $5 friction vs. $200 fraud → 2.4%.

H14 · Explain importance weighting for covariate shift and when it fails.

Testing: shift correction.

30 s: Reweight training examples by $w(x) = P_{test}(x)/P_{train}(x)$ so the weighted training loss estimates test risk; estimate w with a train-vs-test classifier.

2 min: Fails without overlap (unbounded weights), with high weight variance (effective sample size $(\sum w)^2/\sum w^2$ collapses), and if $P(y \mid x)$ also changed. Clip/normalize weights; check support.

Analogy: Like a pollster whose sample has too few rural voters: each rural answer is counted extra to match the real electorate, but with no rural voters at all, or one carrying a huge weight, reweighting cannot rescue it.

Deep: $w(x) = \frac{P(\text{test}|x)}{P(\text{train}|x)} \frac{n_{train}}{n_{test}}$.

Bad answer: "Just retrain on test data." (You don't have labels.)

Follow-up: How to check overlap? → **Harder:** Is reweighting necessary if the model is well specified? (Less so — misspecification is when it matters.)

Example: Model trained on desktop users deployed to mobile.

H15 · How do you detect and correct label shift?

Testing: prior shift methods.

30 s: If $P(x \mid y)$ is stable, adjust posteriors by the ratio of new to old priors: $p_{new}(y \mid x) \propto p_{old}(y \mid x)\pi_{new}(y)/\pi_{old}(y)$. Estimate π_{new} with EM on unlabeled data or confusion-matrix inversion (BBSE).

2 min: Detect via predicted-positive rate changes without input drift in class-conditional features; confirm with a labeled sample. Recalibrate thresholds.

Analogy: When dengue season arrives, a doctor reads symptoms the same way but raises her starting belief that any fever is dengue; label-shift correction likewise rescales predicted probabilities by how much each class's share has changed.

Deep: Saerens et al. EM; black-box shift estimation.

Bad answer: "Retrain from scratch."

Follow-up: How is this different from covariate shift? → **Harder:** What if both change?

Example: Disease prevalence rising during an outbreak.

H16 · What is position bias in ranking data and how do you correct it?

Testing: counterfactual learning to rank.

30 s: Higher-ranked items get more clicks regardless of relevance, so clicks conflate relevance and position. Correct with position features (zeroed at serving), click models, or inverse propensity weighting using position-examination probabilities estimated from randomization.

2 min: Propensities from swap/interleaving experiments; IPW-weighted losses give unbiased relevance estimates (at higher variance). Also trust bias and presentation bias.

Analogy: Shoppers grab more of whatever sits on the eye-level shelf, so raw sales overstate its quality; correcting position bias counts each sale as worth 1 ÷ (the chance shoppers even looked at that shelf).

Deep: Clicks ≈ examine(position) × relevant.

Bad answer: "Just train on clicks."

Follow-up: How to estimate examination probabilities? → **Harder:** What happens to items never shown?

Example: Search ranker overvaluing previously top-ranked results.

H17 · How do you evaluate a new recommender offline when logs came from the old one?

Testing: off-policy evaluation.

30 s: Logged data only contains the old policy's choices. Use off-policy estimators: inverse propensity scoring (reweight logged rewards by new/old action probabilities), doubly robust estimators, or replay on randomized exploration traffic.

2 min: Requires logging propensities; high variance when policies differ; clip weights. Offline ranking metrics on logged clicks are biased toward the old policy. Ultimately confirm with A/B tests.

Analogy: Like judging a new route plan from old trip logs: you only know outcomes for routes the old plan chose, so reweight each logged trip by how much more or less often the new plan would choose it — shaky when the plans differ greatly.

Deep: $\hat{V} = \frac{1}{n} \sum \frac{\pi_{new}(a_i|x_i)}{\pi_{old}(a_i|x_i)} r_i$.

Bad answer: "Compute NDCG on last week's clicks."

Follow-up: What if the old policy was deterministic? → **Harder:** Doubly robust estimator intuition?

Example: 1% uniformly random exploration slots for evaluation.

H18 · Why does a random forest's variance plateau as trees are added?

Testing: correlated averaging.

30 s: Variance of an average of correlated models is $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ → floor $\rho\sigma^2$. Trees trained on overlapping bootstrap samples of the same data remain correlated.

2 min: Lower ρ via feature subsampling, smaller subsamples, randomized splits (extra trees) — at some cost in individual-tree bias/variance.

Analogy: Averaging many TV weather forecasts helps only while they make different mistakes; if every channel copies the same satellite feed, their shared error survives any amount of averaging — that shared part is the $\rho\sigma^2$ floor.

Deep: Part 3 why-box.

Bad answer: "Because trees overfit."

Follow-up: What's the effect of max_features = 1? → **Harder:** How does this relate to ensembles of neural nets with different seeds?

Example: Accuracy flat beyond 300 trees.

H19 · Explain why early stopping is similar to L2 regularization.

Testing: optimization-regularization link.

30 s: For least squares from zero init, GD after t steps fits each eigen-direction by $1-(1-\eta\lambda_i)^t$ of its OLS value — high-curvature directions first — similar to ridge's $\lambda_i/(\lambda_i+\lambda)$ with $\lambda \approx 1/(\eta t)$.

2 min: Both shrink low-variance directions most; the equivalence is approximate and exact only in limits; for nets, early stopping limits distance from initialization.

Analogy: Like a rangoli artist who lays the big outline first and the fine dots last: stopping early leaves the faint, noise-prone details undrawn, much as ridge tones down the weakly supported directions.

Deep: Spectral filtering view.

Bad answer: "Both stop the model from training."

Follow-up: Why does initialization matter here? → **Harder:** Does the analogy hold for Adam?

Example: Choosing epochs instead of weight decay for a small dataset.

H20 · Which solution does gradient descent find for underdetermined least squares?

Testing: implicit bias.

30 s: Starting from zero, GD stays in the row space of X and converges to the minimum-L2-norm interpolating solution X^+y.

2 min: Every update is $X^\top(\cdot)$, a combination of rows; the unique interpolating solution in the row space is the min-norm one. This implicit regularization is one explanation for good generalization of overparameterized models; adaptive methods don't share it.

Analogy: Starting from zero, GD is like a tailor who alters a plain garment only along seams the measurements mention: among all perfect fits, it ends with the smallest total alteration — the minimum-norm solution.

Deep: Links to double descent and to Adam-vs-SGD.

Bad answer: "Any solution with zero loss."

Follow-up: What if initialized at $w_0 \neq 0$? (min-norm relative to w_0.) → **Harder:** What's the implicit bias for logistic regression on separable data? (max-margin direction.)

Example: d = 10,000 features, n = 500 samples.

H21 · Design a validation strategy for a fraud model with 60-day label delay.

Testing: time, maturity, leakage.

30 s: Time-based splits where test periods have matured labels; a gap ≥ label delay between training cutoff and evaluation; features computed point-in-time; evaluate on multiple consecutive months.

2 min: Account for selective labels (declined transactions); compare with the production model on the same cohorts; monitor early proxies (disputes, manual review) for recent data; backtest the retraining cadence.

Analogy: Like judging a mango buyer's picks when fruit takes 60 days to reveal its quality: evaluate only lots old enough to have ripened, keep that gap after the training cutoff, and use only facts known on purchase day.

Deep: Label-maturity curves.

Bad answer: "Random 80/20 split."

Follow-up: How do you train on recent data without mature labels? → **Harder:** How to evaluate declined transactions?

Example: Train to March 31, test June (labels matured by August).

H22 · How does batch size interact with learning rate and generalization?

Testing: SGD noise dynamics.

30 s: Larger batches reduce gradient noise, so larger learning rates are stable (linear scaling rule with warmup) up to a critical batch size; beyond it, more batch doesn't reduce steps. Large batches can generalize worse at fixed epochs unless tuned.

2 min: Noise scale ∝ η/B; small batches act as regularization. Remedies for large-batch training: LR scaling, warmup, LARS/LAMB, longer training.

Analogy: A bigger batch is like asking more locals for directions before each turn: the advice gets more reliable, so you can stride further, but beyond a point extra opinions barely help, and the wobble from small polls can aid exploration.

Deep: Gradient variance ∝ 1/B.

Bad answer: "Batch size doesn't matter."

Follow-up: What is the critical batch size? → **Harder:** Sharp-minima hypothesis — evidence and critiques?

Example: Scaling from 8 to 256 GPUs.

H23 · Why does learning-rate warmup help, especially with Adam and transformers?

Testing: early-training dynamics.

30 s: Early on, Adam's second-moment estimates are based on few steps and noisy, and randomly initialized networks can have large, unstable gradients; a small initial LR prevents destructive early updates, then the LR ramps up.

2 min: In post-LN transformers, large early updates blow up activations; large-batch training also needs warmup. RAdam formalizes the variance issue.

Analogy: Like driving gently for the first few minutes on a cold morning until the engine warms up: Adam's running averages and the network's activations are unsettled at first, so small early steps prevent a destructive lurch.

Deep: Variance of the adaptive LR is large when t is small.

Bad answer: "Warmup makes training faster."

Follow-up: Why did pre-LN reduce the need? → **Harder:** How long should warmup be?

Example: Loss divergence at step 300 fixed by 2K warmup steps.

H24 · How do you detect label noise and train robustly to it?

Testing: data-centric ML.

30 s: Detect with out-of-fold predictions confidently disagreeing with labels, high-loss examples, annotator disagreement, conflicting duplicates. Handle by relabeling, removal, robust losses, label smoothing, early stopping, noise transition models.

2 min: Distinguish random vs. systematic noise; clean the test set first; noisy examples are learned late (memorization). Confident learning estimates the joint of noisy and true labels.

Analogy: Like spotting typos in an exam's answer key: where your best students confidently disagree with the key, the key is suspect; check and fix those entries before students rote-learn the mistakes.

Deep: Part 25.

Bad answer: "Deep learning is robust to noise."

Follow-up: Why does focal loss make it worse? → **Harder:** How do you estimate a noise transition matrix?

Example: Relabeling the 2% most suspicious examples improved accuracy more than a new architecture.

H25 · What problems do CatBoost's ordered target statistics and ordered boosting solve?

Testing: leakage within training.

30 s: Target statistics computed with an example's own label leak it into the feature; ordered TS uses only preceding examples in a random permutation. Gradient boosting also has "prediction shift" — gradients are computed with models trained on the same example; ordered boosting uses models not trained on it.

2 min: Both are about making training-time features/gradients distributed like test-time ones. Cost: more computation; multiple permutations reduce variance.

Analogy: Rating a reality-show singer by her city's average score, when that average already includes her own score, leaks the answer; ordered statistics use only contestants who performed before her in a random order.

Deep: Prokhorenkova et al. (2018).

Bad answer: "CatBoost handles categories with one-hot."

Follow-up: How does K-fold target encoding compare? → **Harder:** Why does prediction shift cause overfitting?

Example: Many categorical features in insurance pricing.

H26 · Mechanistically, why does t-SNE distort global structure?

Testing: objective-level reasoning.

30 s: It minimizes forward KL between neighbor distributions: it penalizes separating true neighbors but barely penalizes misplacing distant points; per-point bandwidths equalize density; the heavy-tailed t-kernel exaggerates gaps.

2 min: Hence cluster sizes and inter-cluster distances are meaningless; perplexity and seed change the picture; use PCA init and multiple runs; don't over-interpret.

Analogy: t-SNE is like a big Indian wedding seating plan that carefully keeps each guest beside close friends but barely cares which tables end up near each other: distances between tables, and their sizes, carry little meaning.

Deep: Part 13.

Bad answer: "t-SNE is random."

Follow-up: What does perplexity control? → **Harder:** How does UMAP differ?

Example: Two "clusters" that are actually one continuous manifold.

H27 · Derive the calibration correction after negative downsampling.

Testing: Bayes' rule application.

30 s: Keeping negatives with rate w multiplies the odds by 1/w: $\frac{q}{1-q} = \frac{1}{w}\frac{p}{1-p} \Rightarrow p = \frac{q}{q+(1-q)/w}$.

2 min: Because $P(x \mid y)$ is unchanged and only the prior changes, the posterior odds scale by the prior-odds ratio. Class weights have a similar effect approximately — recalibrate empirically.

Analogy: If a shop surveys every buyer but only 1 in 10 browsers, buyers seem ten times more common than they are; to recover the truth, multiply the predicted odds by w = 0.1 — the odds, not the probability.

Deep: Label-shift correction is the same formula.

Bad answer: "Multiply predictions by w."

Follow-up: Does downsampling hurt AUC? (Usually little.) → **Harder:** What if downsampling is non-uniform across segments?

Example: CTR model trained on 10% of negatives.

H28 · Why is the logistic regression MLE undefined on separable data?

Testing: optimization/statistics interplay.

30 s: If a hyperplane separates the classes, scaling w up pushes all probabilities toward their labels and always lowers the loss, which approaches 0 only as $\|w\| \to \infty$ — no finite minimizer.

2 min: Symptoms: huge coefficients, convergence warnings, 0/1 probabilities. Fix: regularization, early stopping, Firth correction. Direction converges to max-margin.

Analogy: If a line separates the classes perfectly, the model is like a debater who wins more points every time they sound surer: confidence grows without limit, so no finite best setting exists unless a rule (regularization) caps it.

Deep: Soudry et al. (2018): GD on separable data → max-margin direction.

Bad answer: "Because the data is too easy."

Follow-up: How common is separation when d ≈ n? → **Harder:** Consequences for calibration?

Example: Rare-event model with many sparse features.

H29 · How are hinge loss, margin and generalization connected?

Testing: learning theory intuition.

30 s: Hinge loss penalizes points with margin < 1; with L2 regularization it maximizes the margin. Margin-based bounds depend on R^2/γ^2, not dimension — so wide-margin classifiers generalize even in huge feature spaces.

2 min: Explains why kernel SVMs work in infinite dimensions and why margin concepts reappear in boosting (AdaBoost increases margins) and metric learning.

Analogy: Hinge loss fines any car that drifts into a wide buffer lane between opposite traffic; with the buffer kept clear, new drivers' small wobbles don't cause crashes, and safety depends on buffer width relative to traffic spread, not on the map's dimensions.

Deep: Part 9.

Bad answer: "Margins make the model faster."

Follow-up: Why does AdaBoost keep improving test error after training error hits zero? → **Harder:** Margin vs. calibration?

Example: Linear SVM on 1M-dimensional sparse text.

H30 · Why does BatchNorm allow higher learning rates?

Testing: optimization mechanism.

30 s: It fixes the scale of each layer's inputs, making the loss landscape smoother (gradients more predictable/Lipschitz) and making layers invariant to weight scale — large updates don't blow up activations.

2 min: Scale invariance means the effective learning rate is η/‖w‖²; weight decay keeps ‖w‖ in check. Evidence contradicts the covariate-shift story (Santurkar et al.).

Analogy: Like an automatic volume leveller on a microphone: however much one layer's weights grow, the next layer hears a steady level, so you can turn the learning-rate knob higher without blowing the speakers.

Deep: Part 20.

Bad answer: "Because it normalizes the data."

Follow-up: Interaction with weight decay? → **Harder:** Why does BN fail at batch size 2?

Example: 10× higher LR stable after adding BN.

H31 · How would you rigorously compare two models offline?

Testing: statistical evaluation.

30 s: Same test set, paired comparisons (paired bootstrap on test examples, McNemar for classification errors), multiple seeds, time-appropriate splits, slice analysis, and a pre-chosen metric at the operating point.

2 min: Report CIs of the difference, not separate CIs; check calibration and latency; beware repeated test-set re-use; confirm online.

Analogy: Compare two batters fairly by having them face the same bowlers on the same pitches, ball by ball, over many innings, and asking if the gap exceeds normal form swings — not by who scored more in one match.

Deep: Paired bootstrap: resample examples, compute both metrics on each resample.

Bad answer: "Model B's AUC is higher, so it's better."

Follow-up: How many seeds? → **Harder:** How to compare when models were trained on different data?

Example: AUC +0.003 on 50K examples — significant?

H32 · Explain CUPED.

Testing: variance reduction in experiments.

30 s: Adjust the metric with a pre-experiment covariate: $Y' = Y - \theta(X - \bar{X})$, $\theta = \mathrm{Cov}(X, Y)/\mathrm{Var}(X)$. Randomization keeps it unbiased; variance falls by $(1 - \rho^2)$.

2 min: With ρ = 0.7, ~half the sample needed. Works best with pre-period values of the same metric; must use pre-treatment data only.

Analogy: Judging a coaching class by marks adjusted for each student's own pre-test removes the noise of natural ability, so a fair comparison needs far fewer students; the pre-test must happen before coaching begins.

Deep: It's regression adjustment.

Bad answer: "CUPED removes outliers."

Follow-up: Why must X be pre-treatment? → **Harder:** New users with no pre-period?

Example: Engagement tests shortened from 4 weeks to 2.

H33 · Why does peeking inflate false positives in A/B tests, and how do you fix it?

Testing: sequential inference.

30 s: Repeatedly testing and stopping at the first $p < 0.05$ gives the random walk many chances to cross the threshold, so the true false-positive rate can rise to ~20–30%.

2 min: Fixes: fixed-horizon analysis, group-sequential designs with alpha spending (O'Brien–Fleming), always-valid p-values/confidence sequences (mSPRT), Bayesian decision rules with care.

Analogy: Like tossing a fair coin and shouting "biased!" the first moment heads run suspiciously ahead: check after every toss and pure chance will eventually cross the line, so fix when you'll look in advance or widen the line for repeated looks.

Deep: Law of the iterated logarithm intuition.

Bad answer: "Peeking is fine if the effect is large."

Follow-up: How do dashboards cause peeking? → **Harder:** Trade-offs of sequential tests (power, bias of estimates at stopping).

Example: Experiment platform with sequential boundaries.

H34 · How do network effects break A/B tests, and what designs fix them?

Testing: SUTVA violations.

30 s: When treatment affects control units (shared supply, social feeds, auctions), the difference between arms is biased — often overstated or understated.

2 min: Designs: cluster randomization (geo, social clusters), switchback experiments (alternate treatment over time windows), budget-split experiments in ads marketplaces; measure spillovers explicitly.

Analogy: Testing a discount on half the shoppers in one sabzi mandi: discounted buyers clear out the tomatoes, control buyers find empty stalls, and the gap overstates the effect — so randomize whole markets or alternate days.

Deep: Interference violates the stable unit treatment value assumption.

Bad answer: "Use more users."

Follow-up: Switchback carry-over effects? → **Harder:** Power loss with cluster randomization?

Example: Rider pricing test that drains drivers from control.

H35 · Explain effective sample size with correlated data and its consequences.

Testing: dependence awareness.

30 s: With m rows per group and intra-class correlation ρ, variance is inflated by $1 + (m-1)\rho$, so $n_{eff} = n/(1 + (m-1)\rho)$.

2 min: Consequences: naive CIs too narrow, overconfident A/B conclusions, overfitting (fewer independent examples than it seems), misleading random splits. Use cluster-robust SEs, group splits, group-level bootstrap.

Analogy: Surveying ten members of one joint family isn't ten independent opinions — they share dinner-table views; likewise 10M events from 50K users carry far less independent information than 10M separate people would.

Deep: Design effect from survey sampling.

Bad answer: "More rows is always more information."

Follow-up: How to estimate ρ? → **Harder:** Randomizing by session but analyzing by user?

Example: 10M events from 50K users.

H36 · What is Goodhart's law in ML? Give examples and mitigations.

Testing: metric design maturity.

30 s: When a measure becomes a target, it stops being a good measure: models optimize proxies (clicks, watch time) in ways that break the proxy's link to value.

2 min: Examples: clickbait under CTR optimization; recommenders promoting outrage under engagement; support bots closing tickets fast. Mitigate with multiple metrics and guardrails, long-term holdouts, human evaluation, satisfaction surveys, and periodic metric review.

Analogy: When a call centre rewards agents for short calls, agents start hanging up early: the number improves while customers get angrier — once a measure becomes a target, it stops measuring what you cared about.

Deep: Proxy-gap chain in Part 1.

Bad answer: "Pick a better single metric."

Follow-up: How do you measure long-term effects? → **Harder:** How to combine multiple objectives?

Example: YouTube-style shift from clicks to watch time to satisfaction.

H37 · How do you handle feedback loops in recommender systems?

Testing: dynamic systems thinking.

30 s: The model's recommendations shape the data it learns from, reinforcing popular items and existing beliefs. Counter with exploration, propensity-weighted training, randomized holdouts, diversity constraints, and monitoring coverage.

2 min: Symptoms: shrinking catalog coverage, popularity concentration, cold items never getting exposure. Measure with exposure distributions; allocate exploration traffic (bandits); debias training.

Analogy: Like a canteen that cooks only last week's bestsellers: new dishes never get tasted, so they never sell, and the menu keeps shrinking; a few daily "try this" specials (exploration) break the loop.

Deep: Selection bias in logged data (H17).

Bad answer: "Retrain more often."

Follow-up: Cost of exploration? → **Harder:** How to evaluate exploration's long-term value?

Example: New creators never surfacing.

H38 · How do you choose a retraining cadence?

Testing: production judgment.

30 s: Measure how fast performance decays (train at t, evaluate at t+1, t+2, ...), weigh against label delay, retraining cost and risk; retrain when expected value gained exceeds cost, with validation gates.

2 min: Fast-changing domains (ads, news, fraud) need daily/continuous updates; stable ones monthly. Triggered retraining on drift alerts; warm starts; watch feedback loops.

Analogy: Like deciding how often to redraw a city map: measure how fast it goes stale, weigh that against the survey's cost, and refresh booming suburbs like Gurugram far more often than the old city.

Deep: Backtest cadences on history.

Bad answer: "Retrain every day to be safe."

Follow-up: What gates before auto-deploy? → **Harder:** Continuous training risks?

Example: Weekly fraud retraining + daily rule updates.

H39 · Explain gradient checking and its pitfalls.

Testing: implementation rigor.

30 s: Compare analytic gradients with centered finite differences $\frac{L(\theta+\epsilon)-L(\theta-\epsilon)}{2\epsilon}$ on a few parameters; relative error should be tiny (~1e-7 in float64).

2 min: Pitfalls: float32 precision, kinks (ReLU at 0, max), randomness (dropout on), regularization terms omitted, checking only averaged gradients, ε too small (rounding) or too large (truncation).

Analogy: Like checking a speedometer by timing the car over a short stretch: if measured and displayed speeds disagree, something's broken — but too short a stretch drowns in stopwatch rounding, and too long blurs speed changes.

Deep: Centered difference error $O(\epsilon^2)$.

Bad answer: "Check that loss decreases."

Follow-up: Relative vs. absolute error? → **Harder:** How to gradient-check a layer with batch statistics?

Example: Custom attention layer bug found by gradient check.

H40 · Why is dot-product attention scaled by √d?

Testing: variance reasoning.

30 s: With unit-variance independent components, $q^\top k$ has variance d; large logits saturate softmax (near one-hot, tiny gradients). Dividing by √d keeps logits at unit variance.

2 min: Keeps the softmax temperature stable across head dimensions; related to initialization-scale arguments; alternatives include QK-normalization.

Analogy: Summing d random products is like adding up more players' random scores: totals swing wider as d grows, so one key looks like a runaway winner by chance; dividing by √d keeps scores in a range where softmax still spreads its bets.

Deep: $\mathrm{Var}(\sum_{i=1}^{d} q_i k_i) = d$.

Bad answer: "To normalize the attention weights." (Softmax does that.)

Follow-up: What happens without it at d = 1024? → **Harder:** Why does softmax saturation hurt gradients?

Example: Training instability fixed by correct scaling.

27.4 Staff-level reasoning

What changes at Staff level

The question is rarely "what is X?". It's "what would you do, why, and what could go wrong?" — with incomplete information, competing objectives and organizational constraints. Strong answers **clarify**, **structure**, **quantify**, name **trade-offs and risks**, and end with a **decision and a way to validate it**. Cards S1–S9 are expanded into full walkthroughs in Part 36.

S1 · AUC improved from 0.82 to 0.86 but revenue didn't change. Why?

Testing: metric–value alignment.

30 s: AUC measures global pairwise ranking; revenue depends on decisions at a specific operating point or top-k, on calibration, and on whether the improved region of the ranking matters. The gain may be in irrelevant regions, offset by calibration loss, not reflected online (skew/leakage), or the product isn't sensitive to the model.

2 min: Check operating-point metrics, calibration, slices, offline test representativeness, training–serving skew, experiment power, and whether downstream systems (thresholds, auctions, business rules) absorb the change. → Part 36.

Analogy: Like a kirana owner neatly re-sorting the back storeroom: the overall ordering improved, but sales come only from the front counter customers actually see, so revenue doesn't budge.

Bad answer: "The A/B test must be wrong."

Follow-up: How would you prove where the AUC gain is? → **Harder:** How do you pick offline metrics that predict revenue?

Example: Ranking gains below position 20 that users never see.

S2 · Model quality falls only for new users. How would you debug it?

Testing: slice debugging, cold start.

30 s: New users lack history features, are underrepresented in training, may have different label definitions (short tenure) and different base rates. Confirm with CIs, then check feature availability, training representation, and whether the validation split even contains new users.

2 min: Fix via cold-start features (context, onboarding, content), a separate/new-user model or indicator features, group splits that hold out users, reweighting, fast session adaptation. → Part 36.

Analogy: Like a tailor who fits regulars perfectly from years of notes but guesses for walk-ins: take quick measurements from newcomers (cold-start features) and check the fit separately on first-time customers.

Bad answer: "Collect more data."

Follow-up: How do you validate for new users specifically? → **Harder:** How do you trade off personalization for established users vs. new ones in one model?

Example: Recommendations defaulting to popular items for day-1 users.

S3 · You have 500M examples. Would you use cross-validation?

Testing: proportionate methodology.

30 s: Usually no: a large, well-designed holdout (time-based, grouped, deduplicated) gives tiny error bars; K-fold multiplies compute for little benefit. Exceptions: rare positives, small slices, or uncertainty across training randomness.

2 min: Spend effort on split design, slice evaluation, and seed variance instead. → Part 17 / 36.

Analogy: With 500M examples, a 1% holdout is like a five-million-voter exit poll: already precise, so repeating it ten ways (K-fold) costs 10× for little gain; effort is better spent sampling the right voters (split design).

Bad answer: "Always use 10-fold CV."

Follow-up: How large should the holdout be? → **Harder:** How do you estimate variance due to training randomness cheaply?

Example: 1% holdout = 5M examples.

S4 · Production feature values differ from training features. What do you do?

Testing: training–serving skew.

30 s: Quantify the difference (log served features, compare with offline values for the same entities and timestamps), find the cause (different code paths, staleness, defaults, units, timezones), fix at the source — ideally one feature definition via a feature store — and add parity monitoring.

2 min: Short-term: retrain on logged serving features or patch the serving path; assess impact via replay. Long-term: shared feature code, point-in-time training data, contract tests. → Part 36.

Analogy: Like a recipe perfected on a home gas stove but cooked in the restaurant on induction: same words, different dish; compare both kitchens' output side by side, fix the cause, and keep one shared recipe card (feature store).

Bad answer: "Retrain the model more often."

Follow-up: How to detect skew automatically? → **Harder:** When is it acceptable to train on logged serving features?

Example: Offline "7-day average" computed daily vs. online computed hourly.

S5 · Your ranking model improves NDCG offline but decreases CTR online. Why?

Testing: offline–online gap in ranking.

30 s: Offline labels are biased by the old policy's exposure and position; NDCG's relevance labels may not match click intent; the model may be hurting diversity/freshness, calibration for downstream blending, or latency. Or the test was flawed.

2 min: Check label source and position bias correction, per-position/slice effects, latency, interaction with re-ranking layers, novelty/learning effects, and experiment validity. Use interleaving and off-policy evaluation. → Part 36.

Analogy: Like a DJ who scores a new playlist against past requests, which only cover songs the old playlist played: it looks great on that history, but the live crowd, bored by more of the same, dances less.

Bad answer: "NDCG is a bad metric."

Follow-up: Which offline metric would correlate better? → **Harder:** How would you debias offline evaluation?

Example: Model promoting items similar to past clicks → less diverse feed → lower CTR.

S6 · Your fraud model blocks too many legitimate users. What do you do?

Testing: cost-sensitive decisions under constraints.

30 s: Quantify the false-positive cost and where it's concentrated (segments, merchants, new customers); revisit the threshold using explicit costs and calibration; introduce tiered actions (step-up authentication instead of decline); check for drift or a pipeline bug that raised scores.

2 min: Improve the model on the confused slices, add features that separate legit anomalies (travel, new device), add feedback from friction outcomes as labels, monitor false-decline rate as a first-class metric. → Part 36.

Analogy: Like a society watchman turning away residents who forgot their pass: rather than waving everyone in, count who is wrongly stopped and add a middle option — intercom the flat to confirm (step-up authentication).

Bad answer: "Raise the threshold."

Follow-up: How do you measure false positives when blocked users never transact? → **Harder:** Different thresholds per segment — fairness implications?

Example: Travelers blocked abroad.

S7 · The distribution changes every weekend. How do you handle it?

Testing: periodicity vs. drift.

30 s: It's predictable seasonality, not drift: encode it (day-of-week features, weekend indicators, interactions), ensure training data covers weekends proportionally, evaluate by day type, and don't let drift alarms fire on expected weekly patterns.

2 min: If behavior differs fundamentally, consider segment-specific models or per-day-type calibration/thresholds; retrain windows as multiples of 7 days; compare week-over-week, not day-over-day. → Part 36.

Analogy: Like a chai stall that is always packed on Sundays: that's a weekly rhythm, not a changing neighbourhood, so plan stock by day of week and compare this Sunday with last Sunday, not with Friday.

Bad answer: "Retrain every Saturday."

Follow-up: How do you set drift baselines for weekly cycles? → **Harder:** Holidays?

Example: Food delivery demand and ETA shifts on weekends.

S8 · Training takes four days. How would you reduce experimentation time?

Testing: iteration speed engineering.

30 s: Iterate on smaller proxies (subsampled data, smaller models) that rank ideas the same way as full runs; use early signals (learning curves), warm starts, and parallel experiments; profile and fix bottlenecks (data loading, mixed precision, distributed training).

2 min: Validate the proxy's rank correlation with full runs; use successive halving to kill bad runs early; cache features; reserve full runs for finalists. → Part 36.

Analogy: Like a film crew testing lighting ideas in quick phone rehearsals before the costly full shoot: cheap trials rank ideas, only finalists get the real run, and you first confirm rehearsals rank ideas the way the real shoot does.

Bad answer: "Buy more GPUs."

Follow-up: When do small-scale results fail to transfer? → **Harder:** How do scaling laws help here?

Example: 5% data proxy with 0.9 rank correlation to full-data results.

S9 · Two models have identical accuracy. Which do you deploy?

Testing: multi-criteria judgment.

30 s: Check if the tie is real; then compare operating-point metrics, calibration, slices, robustness over time, latency/cost, complexity, dependencies and interpretability; prefer the simpler, cheaper, more robust model — or A/B test.

2 min: → Part 26 staff card and Part 36.

Analogy: Two cars with identical top speed: choose by fuel cost, reliability, handling in the rain and service network — or take both on a test drive (A/B test); top speed was only one line on the spec sheet.

Bad answer: "Either one."

Follow-up: What if they err on different examples? (ensemble.) → **Harder:** How to weigh latency vs. a tiny accuracy gain?

Example: GBM (2 ms) vs. deep model (40 ms) tied on accuracy.

S10 · How do you decide whether a problem should use ML at all?

Testing: judgment beyond modeling.

30 s: Use ML when the task has patterns too complex for rules, data with labels (or a path to them) exists, errors are tolerable and measurable, and the value justifies building and maintaining a system. Otherwise start with rules/heuristics.

2 min: Assess baseline value of a heuristic, data availability, cost of errors, explainability needs, maintenance burden, feedback loop. Often: ship rules, log data, then replace with ML.

Analogy: Like deciding whether a tiffin service needs a trained chef or a recipe card: if the card gets you most of the way, start there and note what goes wrong; hire the chef when the variety outgrows cards and justifies the salary.

Bad answer: "ML is always better than rules."

Follow-up: What evidence would change your mind? → **Harder:** How to estimate ROI before building?

Example: Routing tickets with keyword rules first, ML later.

S11 · How do you make offline evaluation predictive of online results?

Testing: evaluation strategy at org level.

30 s: Track offline–online correlation across past launches; choose offline metrics that mimic the decision (top-k, calibration, slices); use time-based, deduplicated test sets from recent traffic; debias with off-policy methods; maintain replay and shadow evaluation.

2 min: Build an evaluation suite with golden sets and slice dashboards; hold out long-term groups; periodically recalibrate which offline metrics gate launches.

Analogy: Like a coaching institute checking whether its mock tests predict real board results: track the match over past batches, make mocks resemble the actual paper, and gate decisions only on mocks that have predicted well.

Bad answer: "Just A/B test everything."

Follow-up: What if correlation is low? → **Harder:** How to evaluate long-term effects cheaply?

Example: Replay simulator for a ranking system.

S12 · A model improves overall but hurts a minority segment. Do you ship?

Testing: fairness, trade-offs and communication.

30 s: Not automatically. Quantify the harm (size, severity, confidence), understand why, check legal/policy constraints, and look for a fix (segment features, calibration, per-segment thresholds, targeted data). Decide with stakeholders using explicit criteria.

2 min: Options: ship with a guardrail (fallback to the old model for that segment), delay, or fix first. Document the decision.

Analogy: Like a new bus route that speeds up most commutes but drops the stop near a hospital: the average improves, yet you measure the harm, look for a fix (a feeder bus), and decide openly rather than on the average alone.

Bad answer: "Ship — overall is what matters."

Follow-up: How do you set non-regression thresholds per slice? → **Harder:** What if fixing the segment costs overall accuracy?

Example: Speech model worse for an accent.

S13 · How do you prioritize between data, features and model work?

Testing: leverage thinking.

30 s: Error analysis first: categorize errors by cause (label noise, missing information, data coverage, model capacity, pipeline bugs). Estimate impact × probability of success ÷ cost for each fix; data and features usually offer more leverage than architecture.

2 min: Use learning curves (more data?), oracle experiments (perfect feature → how much gain?), label audits, and ablations.

Analogy: Like a doctor who diagnoses before prescribing: sort the errors by cause, then treat whichever gives the biggest gain per rupee — often a better diet (data, features) rather than a stronger drug (a bigger model).

Bad answer: "Try a bigger model first."

Follow-up: What's an oracle experiment? → **Harder:** How do you estimate value of a new data source before acquiring it?

Example: 40% of errors from one mislabeled category.

S14 · A new model beats the old one but exceeds the latency budget. Options?

Testing: efficiency engineering.

30 s: Distill into a smaller model, quantize/prune, cache or precompute parts (embeddings), restructure as a funnel (heavy model only on top-k), optimize features (the usual bottleneck), batch/hardware changes, or negotiate the budget with evidence of value.

2 min: Quantify accuracy-vs-latency curves; measure p99, not average; consider asynchronous or two-pass designs.

Analogy: Like a slow toll plaza: prepaid FASTag (precomputation), a lighter check (distilled model) or full checks only for a shortlisted few (funnel) all help — but more booths never shorten one car's inspection.

Bad answer: "Just use more servers." (Parallelism doesn't reduce per-request latency.)

Follow-up: Why can more servers not fix p99? → **Harder:** Distillation with unlabeled traffic?

Example: Cross-encoder re-ranking only the top 20.

S15 · Use a pretrained foundation model or build a custom model?

Testing: build-vs-buy judgment.

30 s: Consider data volume, task specificity, latency/cost at scale, privacy, controllability and maintenance. Pretrained models win with little labeled data and language/vision tasks; custom/smaller models win on structured data, strict latency/cost, or when you can distill.

2 min: Common path: prototype with a large pretrained model, collect labels/outputs, then fine-tune or distill a smaller model for production.

Analogy: Like hiring an expert consultant versus training your own staff: the consultant is instantly capable but costly per hour, so a common path is letting the consultant start while a cheaper employee learns from their work (distillation).

Bad answer: "Always use the biggest model."

Follow-up: How to evaluate it on your domain? → **Harder:** Total cost of ownership comparison?

Example: Ticket classification prototyped with a large model, distilled into a small classifier.

S16 · How would you design a metric for long-term user value?

Testing: metric design.

30 s: Find short-term signals that predict long-term outcomes (retention, LTV) via historical analysis and long-running holdouts; combine them into a surrogate index; include negative signals (hides, unsubscribes); validate that moving the surrogate moves the long-term outcome.

2 min: Guard against Goodhart: multiple metrics, guardrails, periodic re-validation.

Analogy: Like a gym that cares about yearly renewals: since those arrive late, find early signs that predict them (steady visits, not mere sign-ups), combine them, and keep checking the signs still lead to renewals.

Bad answer: "Use DAU."

Follow-up: How to validate a surrogate? → **Harder:** Heterogeneous long-term effects?

Example: Satisfaction-weighted watch time.

S17 · The business changes the label definition. How do you migrate?

Testing: change management.

30 s: Relabel historical data under the new definition (or re-derive), retrain, and compare both models under the new definition; run in shadow; communicate metric discontinuities; version labels and metrics.

2 min: If relabeling is impossible, bridge with a mapping or a transition model; monitor both definitions during the overlap.

Analogy: Like a board raising the pass mark from 33% to 40%: re-grade old answer sheets under the new rule, compare methods using that rule, and warn everyone this year's pass rate isn't comparable to last year's.

Bad answer: "Keep the old model."

Follow-up: How to evaluate fairly across definitions? → **Harder:** Delayed labels under the new definition?

Example: Churn redefined from 30 to 60 days inactivity.

S18 · How do you build ML systems against adversaries (fraud, spam, abuse)?

Testing: adversarial robustness.

30 s: Assume adaptation: frequent retraining, fast rules for new attacks, features that are costly to fake (behavioral, graph, device), ensembles of diverse signals, randomization of thresholds, monitoring for sudden shifts, and red-teaming.

2 min: Protect feedback (label poisoning), rate-limit probing, keep human review in the loop, measure time-to-mitigate.

Analogy: Like guarding a house against burglars who adapt: use locks that are costly to pick (hard-to-fake features), several different alarms, changing routines, and fast response to new tricks — any single fixed lock gets studied and beaten.

Bad answer: "Train a more accurate model."

Follow-up: How do adversaries probe models? → **Harder:** Label poisoning defenses?

Example: Spam campaigns rotating URLs.

S19 · How do you monitor a model when labels take 90 days?

Testing: proxy monitoring.

30 s: Monitor inputs, predictions, and early proxy outcomes (e.g. early delinquency, disputes, manual review outcomes); track cohorts as labels mature; use label-maturity curves; set alerts on drift in important features and score distributions.

2 min: Periodically validate proxies against matured labels; maintain a small labeled audit sample.

Analogy: Like a paddy farmer who can't wait 90 days for harvest to judge the crop: watch early signs like leaf colour and soil moisture, and check each season that those signs really predicted the harvest.

Bad answer: "Wait 90 days."

Follow-up: How to choose proxies? → **Harder:** Triggering retraining without labels?

Example: Credit model monitored with 30-day delinquency.

S20 · How would you cut serving cost by 50%?

Testing: efficiency levers.

30 s: Profile where cost goes (features, model compute, fan-out). Levers: distillation/quantization, caching, request batching, funnel restructuring (score fewer candidates), cheaper hardware, reducing feature lookups, precomputation in batch, autoscaling.

2 min: Measure accuracy impact for each lever and pick the Pareto-best combination.

Analogy: Like halving a household electricity bill: first meter each appliance to find the real guzzlers, then fix the biggest — often an AC left running (repeated recomputation) — while checking comfort doesn't suffer.

Bad answer: "Use a smaller model." (Without measuring impact.)

Follow-up: Which lever is safest? → **Harder:** How to evaluate cost vs. revenue trade-off?

Example: Caching user embeddings per session.

S21 · How do you handle multi-objective ranking (engagement vs. quality vs. revenue)?

Testing: objective design.

30 s: Predict each objective separately (multi-task), combine with explicit weights or constraints, tune weights with online experiments against long-term metrics, and add guardrails.

2 min: Alternatives: constrained optimization (maximize engagement s.t. quality ≥ X), Pareto analysis, per-surface weights. Keep weights interpretable and versioned.

Analogy: Like choosing a flat that balances rent, commute and space: rate each option on each goal separately, blend with agreed weights or hard limits ("rent under Rs 30,000"), and revisit the weights after living with the choice.

Bad answer: "Optimize engagement; quality follows."

Follow-up: How to choose weights? → **Harder:** How do you detect one objective being gamed?

Example: Feed ranking with hide-rate penalties.

S22 · An A/B test is positive on the primary metric but a guardrail regressed slightly. Ship?

Testing: decision making under trade-offs.

30 s: Quantify: is the regression significant and material? What's the value trade-off? Can it be mitigated? Decide by pre-agreed guardrail thresholds; escalate with data if it's a judgment call.

2 min: Investigate the mechanism; consider partial rollout or iteration; don't rationalize post hoc.

Analogy: Like a faster delivery route that slightly raises spilled orders: check whether the rise is real and fixable, and decide using a spill limit agreed before the trial, not one invented after seeing results.

Bad answer: "Primary metric wins."

Follow-up: How to set guardrail thresholds in advance? → **Harder:** Long-term implications of small guardrail regressions?

Example: +1% CTR, +3% latency p99.

S23 · Model v13 is worse than v12. How do you find why?

Testing: systematic regression debugging.

30 s: Diff everything: data snapshot, features, code, config, dependencies, random seeds, training infrastructure. Bisect changes by retraining with one change at a time; compare slice metrics and feature importances.

2 min: Reproducibility (versioned data/code/config) is what makes this possible; keep a golden evaluation set.

Analogy: Like a biryani that tastes worse than last week's: list every change (rice supplier, spice batch, cook, stove) and revert them one at a time until the taste returns — possible only if the recipe was written down.

Bad answer: "Revert and move on." (Revert yes — but find the root cause.)

Follow-up: Training is non-deterministic — how to separate noise from regression? → **Harder:** The data snapshot itself is the culprit — then what?

Example: A library upgrade changing default tokenization.

S24 · How do you safely deprecate a model or a feature?

Testing: system hygiene.

30 s: Find all consumers (lineage), measure impact via ablation/shadow, communicate, run a staged removal with monitoring and rollback, and delete pipelines only after the model no longer depends on them.

2 min: Features left unmaintained silently drift; removing unused ones reduces risk and cost.

Analogy: Like closing an old road: find everyone who uses it, announce diversions, close it in stages while watching traffic and ready to reopen, and dig it up only once no route depends on it.

Bad answer: "Just stop computing it."

Follow-up: How to detect unused features? → **Harder:** Downstream models using your model's outputs as features?

Example: Removing a third-party data feed.

S25 · How do you make an ML system reproducible and auditable?

Testing: governance.

30 s: Version data (snapshots/queries), code, configs, environments, and models; record lineage in a registry; log training runs and evaluation reports; log predictions with model versions and features; document model cards and approvals.

2 min: Aim for explainable differences rather than bit-identical runs; retain enough to reproduce any production decision.

Analogy: Like a bank keeping every cheque, ledger entry and approval so any transaction can be traced years later: version data, code, config and model, and log each prediction with the version and inputs that produced it.

Bad answer: "Set a random seed."

Follow-up: What must be logged at prediction time? → **Harder:** Regulations requiring explanations of individual decisions?

Example: Credit decision audit a year later.

27.5 GenAI & modern ML (2027 edition)

50 cards for the questions that entered mainstream ML loops in 2026–27

Same format as the other cards. They are grouped by theme: transformers (G1–G8), language models (G9–G15), adapting and aligning models (G16–G23), retrieval and agents (G24–G32), evaluation and safety (G33–G38), efficiency and serving (G39–G45), recommenders and responsible AI (G46–G50). Each points to the Part with the full mechanism, code and numbers.

G1 · Explain self-attention to an interviewer in two minutes.

Testing: whether you know the mechanism, not just the name.

30 s: Each token makes a query, a key and a value by linear maps. Query–key dot products, scaled by $\sqrt{d_k}$ and softmaxed, give weights over all tokens; the output is the weighted sum of values. Every token can directly read every other token in one step.

2 min: Add: masks (causal for decoders, padding for batches); multi-head attention runs several of these in lower dimensions so different heads can track different relations; cost is $O(T^2 d)$ time and $O(T^2)$ memory per head; attention is permutation-equivariant, so positions must be injected (RoPE, learned embeddings); stacked with MLPs, residuals and normalization it forms a transformer block.

Analogy: A student in a group study session asks a question (query); each classmate's notes have a topic label (key); she reads mostly from the notes whose labels match her question and blends their content (values).

Deep: $\mathrm{softmax}(QK^\top/\sqrt{d_k} + M)V$ (→ Part 41).

Bad answer: "Attention tells you which words the model thinks are important."

Follow-up: Why scale by $\sqrt{d_k}$? → **Harder:** How would you make attention cheaper for 1M-token contexts?

Example: In "The trophy didn't fit in the suitcase because it was too big", the token "it" attends strongly to "trophy".

G2 · Why divide by √d_k in attention?

Testing: variance reasoning and softmax saturation.

30 s: With unit-variance components, $q^\top k$ has variance d_k. Without scaling, logits grow with dimension, softmax becomes nearly one-hot, and its gradients vanish. Dividing by $\sqrt{d_k}$ keeps logit variance near 1.

2 min: Show the variance: a sum of d_k independent products each with variance 1. Saturated softmax means most weights are near 0 or 1, so small changes in scores barely change outputs — slow learning. Some architectures add normalization of queries and keys (QK-norm) for the same stability reason at scale.

Analogy: Like adjusting exam marks for paper length: a 100-question test naturally produces bigger score gaps than a 10-question one, so you rescale before comparing students.

Deep: $\mathrm{Var}(q^\top k) = d_k \Rightarrow \mathrm{Var}(q^\top k/\sqrt{d_k}) = 1$ (→ Parts 2, 41).

Bad answer: "To make the numbers smaller so they don't overflow."

Follow-up: What happens with temperature in softmax sampling? (Same knob: sharper vs. flatter.) → **Harder:** Is the scaling still exactly right after LayerNorm and training? (It's a good initialization-time argument, not an exact law.)

Example: $d_k = 128$: unscaled logits have SD ≈ 11 — softmax of values that far apart is essentially argmax.

G3 · What is the KV cache and why does it dominate serving memory?

Testing: autoregressive decoding mechanics and memory arithmetic.

30 s: When generating token t, the keys and values of tokens $1..t-1$ don't change, so we store them instead of recomputing — decoding becomes $O(T)$ per step instead of $O(T^2)$. The cache grows with layers × KV heads × head dim × context × batch.

2 min: Formula: $2 \cdot L \cdot H_{kv} \cdot d_h \cdot T \cdot b$ bytes. Llama-2-7B-style at 4K tokens in fp16 = 2 GB per sequence; a batch of 32 is 64 GB — more than the weights. That's why GQA/MQA/MLA, KV quantization, PagedAttention and prefix sharing exist, and why KV memory usually caps batch size.

Analogy: A court stenographer who keeps a written transcript so the judge doesn't have to re-hear every witness before each new question — but the transcript for a long trial fills shelves.

Deep: $2 \times 32 \times 32 \times 128 \times 4096 \times 2$ B = 2 GiB (→ Parts 41, 46).

Bad answer: "It caches the model's previous outputs so it can skip generating them."

Follow-up: How does GQA reduce it? → **Harder:** What happens to the cache when two users share a system prompt?

Example: Moving from 32 to 8 KV heads cut the cache 4× and let a server double its concurrent users.

G4 · MHA vs. MQA vs. GQA — what's the trade-off?

Testing: architectural knobs that matter for inference.

30 s: Multi-head attention gives every query head its own K/V head. Multi-query attention shares one K/V head across all query heads. Grouped-query attention shares each K/V head among a group of query heads. Fewer K/V heads → smaller KV cache and faster memory-bound decoding, with a small quality cost.

2 min: GQA with 8 KV heads is a common middle ground: near-MHA quality at a fraction of the cache. Models can be converted ("uptrained") from MHA to GQA by averaging heads and briefly training. Multi-head latent attention compresses K/V into a low-rank latent — another way to shrink the cache.

Analogy: Instead of every student carrying their own full set of reference books (MHA), groups of students share one set per table (GQA) — or the whole class shares one set (MQA): far less to carry, slightly more waiting.

Deep: cache ∝ H_{kv}; GQA-8 vs. MHA-32 = 4× smaller (→ Part 41).

Bad answer: "They're just different numbers of heads; performance is the same."

Follow-up: Does GQA reduce training FLOPs much? (Little — it mainly helps decode memory.) → **Harder:** Why does MQA hurt quality more than GQA?

Example: Serving at 32K context where MHA would need 16 GB per sequence.

G5 · How do transformers know word order? Explain RoPE.

Testing: positional encodings and length generalization.

30 s: Attention alone is order-blind. RoPE rotates query and key vectors by angles proportional to position, in 2-D pairs at different frequencies; the dot product then depends only on the relative distance between tokens.

2 min: Alternatives: learned absolute positions (can't extrapolate beyond training length), sinusoidal, relative biases (ALiBi adds a distance penalty). RoPE is popular because it's relative, cheap and works with KV caching. Extending context length rescales RoPE frequencies (position interpolation and related methods) plus some long-context fine-tuning.

Analogy: Clock hands turned by seat number — comparing two people only depends on how far apart they sit, not where the row starts.

Deep: $\langle R_m q, R_n k \rangle = \langle q, R_{n-m} k \rangle$ (→ Part 41).

Bad answer: "The model learns order from the data automatically."

Follow-up: Why do models degrade beyond their training length? → **Harder:** How would you evaluate true long-context ability, not just retrieval of one fact?

Example: A model trained at 8K answers well at 8K but garbles at 32K until frequencies are rescaled and it is briefly fine-tuned.

G6 · What problem does FlashAttention solve, and does it change the math?

Testing: memory hierarchy thinking.

30 s: Standard attention writes the $T \times T$ score matrix to slow GPU memory and reads it back. FlashAttention computes attention in tiles kept in fast on-chip memory, using an online softmax, so it never materializes the full matrix. Same result (exact), much less memory traffic, faster and $O(T)$ memory.

2 min: The trick is the running max and running sum for softmax, rescaling partial results as new tiles arrive. It's an IO-aware algorithm: the FLOPs are similar, the bytes moved are far fewer — and on modern GPUs attention is often memory-bound. The backward pass recomputes tiles instead of storing the matrix.

Analogy: Instead of photocopying an entire giant spreadsheet to your desk to total it, you sum it page by page at the filing cabinet, keeping just a running total.

Deep: online softmax: update m, ℓ, $\mathbf{o}$ per tile (→ Part 41).

Bad answer: "It approximates attention to make it faster."

Follow-up: Is it still $O(T^2)$ compute? (Yes.) → **Harder:** When would sparse or linear attention still be needed?

Example: Training at 16K context fits in memory with FlashAttention but not with naive attention.

G7 · What is a mixture-of-experts model and what are its trade-offs?

Testing: conditional computation, serving implications.

30 s: The MLP in each block is replaced by many expert MLPs and a router that sends each token to the top-k (often 1–2) experts. Total parameters grow a lot; compute per token grows little.

2 min: Benefits: more capacity at the same FLOPs per token. Costs: all experts must sit in memory (serving needs lots of GPU memory), routing imbalance (load-balancing losses, capacity limits), all-to-all communication in expert parallelism, and trickier fine-tuning. Quality per total parameter is lower than dense, but per FLOP higher.

Analogy: A hospital with many specialists and a triage nurse: each patient sees only one or two doctors, but you still pay to keep every specialist on staff.

Deep: active params ≪ total params; FLOPs ≈ $2N_{\text{active}}$ per token (→ Parts 41, 46).

Bad answer: "An MoE is an ensemble of separate models that vote."

Follow-up: What happens if the router sends most tokens to one expert? → **Harder:** How does MoE change batch-1 decode latency vs. a dense model with the same active parameters?

Example: 8 experts with top-2 routing: up to ~4× the parameters of a dense model with the same per-token compute (attention layers are shared, so the real ratio is a bit lower).

G8 · Encoder-only vs. decoder-only vs. encoder–decoder — when would you use each?

Testing: matching architecture to task.

30 s: Encoder-only (BERT-style) sees the whole input bidirectionally — best for classification, embeddings and retrieval. Decoder-only (GPT-style) predicts the next token causally — best for generation and general-purpose assistants. Encoder–decoder (T5-style) encodes an input then generates — natural for translation and summarization.

2 min: Decoder-only won as the general-purpose default because one objective scales and everything can be framed as generation; but for high-volume classification or embedding at low latency, a small encoder is still cheaper and often better. Embedding models for RAG are usually encoders (or decoders adapted with contrastive training).

Analogy: A reader who can look at the whole page before answering (encoder), a storyteller who writes one word at a time without peeking ahead (decoder), and a translator who reads the full paragraph then writes the translation (encoder–decoder).

Deep: masking pattern is the key difference (→ Parts 41–42).

Bad answer: "Decoder-only models are always better now."

Follow-up: You need to classify 50M messages/day at 10 ms each — which? → **Harder:** How do you turn a decoder into a good embedding model?

Example: A 100M-parameter encoder fine-tuned for intent routing vs. a 70B decoder: similar accuracy on the task, 500× cheaper.

G9 · How does BPE tokenization work and how can it cause problems?

Testing: tokenization mechanics and their downstream effects.

30 s: Start from characters or bytes; repeatedly merge the most frequent adjacent pair into a new token until the vocabulary reaches its size. Common words become single tokens; rare words split into pieces; byte-level fallback means any text can be encoded.

2 min: Problems: arithmetic and digit handling (numbers split inconsistently), spelling and character-level tasks (the model never "sees" letters), languages under-represented in the tokenizer's training data need more tokens per word → higher cost and shorter effective context, and odd tokens that were rare in training ("glitch tokens"). Vocabulary size trades sequence length against embedding size.

Analogy: Like shorthand in a notebook: frequent phrases get a single squiggle, rare words are spelled out letter by letter — efficient for English notes, clumsy for a language the shorthand wasn't designed for.

Deep: merges ('e','s') → ('es','t') … "lowest" → ['low','est'] (→ Part 42 code).

Bad answer: "Tokens are words."

Follow-up: Why does the same sentence cost more tokens in Hindi than English for many tokenizers? → **Harder:** How would you fix poor arithmetic caused by tokenization?

Example: "12345" split as "123" + "45" in one place and "1" + "2345" in another makes digit-wise addition hard.

G10 · What is perplexity, and why isn't it enough to evaluate a chatbot?

Testing: LM objective vs. product quality.

30 s: Perplexity is $\exp$ of the average negative log-likelihood per token: how "surprised" the model is by held-out text — lower is better. It measures next-token prediction, not usefulness, truthfulness, safety or instruction following.

2 min: Perplexity depends on the tokenizer (not comparable across vocabularies) and on the test text's domain. Instruction-tuned models can have *worse* perplexity on raw text while being far more helpful. For assistants, use task evals, pairwise human/LLM-judge comparisons and online metrics (Part 45).

Analogy: Perplexity is like measuring how well someone predicts the next word in a novel — great for testing reading fluency, useless for testing whether they give good advice.

Deep: $\mathrm{PPL} = \exp\left(-\frac{1}{T}\sum_t \log p(x_t \mid x_{<t})\right)$ (→ Part 42).

Bad answer: "Lower perplexity means a better chatbot."

Follow-up: Can you compare perplexity between two models with different tokenizers? (Not directly — use bits per byte/character.) → **Harder:** When is perplexity still the right metric?

Example: Pretraining runs track validation perplexity; product launches track win-rate and task success.

G11 · What do scaling laws say, and how should they guide a training budget?

Testing: compute-optimal training and its caveats.

30 s: Loss falls smoothly as a power law in parameters, data and compute. For a fixed compute budget $C \approx 6ND$, there's an optimal split between model size and tokens; the Chinchilla result suggests roughly 20 tokens per parameter for compute-optimal training.

2 min: In practice many models are trained far beyond Chinchilla-optimal token counts because inference cost matters: a smaller model trained longer is cheaper to serve. Laws predict loss, not specific capabilities; they depend on data quality and architecture; and data scarcity (repeating data) changes the picture.

Analogy: Like planning a study budget: for a fixed number of hours, there's a best balance between reading more books and thinking harder about each — but if you'll be answering questions for years, you'd rather train a quick thinker for longer.

Deep: isoFLOP curves have a minimum at the optimal N for each C (→ Part 42 figure).

Bad answer: "Bigger models are always better."

Follow-up: Why train an 8B model on 15T tokens (≈1,900 tokens/param)? → **Harder:** What breaks scaling-law extrapolation?

Example: $C = 6 \times 7\mathrm{B} \times 2\mathrm{T} = 8.4 \times 10^{22}$ FLOPs (→ Part 46).

G12 · Explain temperature, top-k and top-p sampling. When would you use each?

Testing: decoding strategies and their effect on output quality.

30 s: Temperature divides logits before softmax: <1 sharpens, >1 flattens. Top-k samples only among the k most likely tokens. Top-p (nucleus) samples from the smallest set whose probabilities sum to p. Greedy/low temperature for factual or structured tasks; moderate temperature with top-p for creative text.

2 min: Top-p adapts to the model's confidence (few candidates when sure, many when unsure), which is why it usually beats a fixed top-k. Beam search suits tasks with one correct output (translation) but produces bland, repetitive open-ended text. Self-consistency samples several answers and takes the majority for reasoning tasks.

Analogy: Temperature is how adventurous a diner is; top-k is "only choose from the 5 most popular dishes"; top-p is "choose among the dishes that together make up 90% of orders".

Deep: $p_i \propto \exp(z_i/T)$ (→ Part 42 code and figure).

Bad answer: "Temperature controls creativity by adding random noise to the output."

Follow-up: Why can temperature 0 still be non-deterministic in production? (Batching and floating-point order.) → **Harder:** How do sampling settings interact with evaluation variance?

Example: Code generation with pass@10 uses temperature ~0.8 for diversity; pass@1 uses near-greedy.

G13 · Why do LLMs hallucinate, and what actually reduces it?

Testing: causes vs. mitigations, grounded thinking.

30 s: The model is trained to produce plausible continuations, not verified facts; rare facts are weakly learned; and training/evaluation that reward answering over abstaining encourage confident guesses. Reduce it with retrieval grounding plus citations, calibrated abstention, verification steps, and decoding and prompting choices.

2 min: Distinguish factuality (wrong about the world) from faithfulness (contradicts the given source). Detect with entailment checks against sources, self-consistency, and uncertainty signals. Fine-tuning on new facts can *increase* hallucination if the model learns to state things it doesn't know; RAG is the main lever for knowledge.

Analogy: A student who must write something for every exam question will invent answers when unsure — unless the exam gives marks for "I don't know" and lets him open the textbook.

Deep: scoring rules that penalize confident errors more than abstentions change incentives (→ Part 45).

Bad answer: "Use a lower temperature and it won't hallucinate."

Follow-up: How would you measure hallucination rate for your product? → **Harder:** Why can fine-tuning on facts make hallucination worse?

Example: A support bot's invented refund policy dropped by 80% after grounding answers in the policy documents with required citations.

G14 · What is in-context learning, and how is it different from fine-tuning?

Testing: conditioning vs. weight updates.

30 s: In-context learning is the model adapting its behaviour from examples or instructions in the prompt, with no weight changes. Fine-tuning changes the weights. ICL is instant and reversible but costs tokens on every call and is limited by context length; fine-tuning is persistent and cheap per call but needs data, training and evaluation.

2 min: ICL is sensitive to example order, format and label balance; it's best for quick adaptation and prototyping. Fine-tuning is better for consistent format/behaviour at scale or when a smaller model must match a larger one (Part 26 cost math). Chain-of-thought prompting elicits intermediate reasoning, which helps multi-step problems.

Analogy: ICL is showing a chef a few photos of the dish you want before each order; fine-tuning is sending the chef on a course so he makes it that way every day.

Deep: no gradient step in ICL — adaptation happens in the forward pass (→ Part 42).

Bad answer: "The model learns from the examples permanently."

Follow-up: Few-shot works in the lab but is inconsistent in production. Why? → **Harder:** When would you choose fine-tuning over a long few-shot prompt purely on cost?

Example: 10-shot classification prompt (1,500 tokens) vs. a LoRA-tuned small model with a 50-token prompt.

G15 · What are "reasoning models" and test-time compute?

Testing: awareness of the 2025–26 shift and its trade-offs.

30 s: Models trained (often with RL on verifiable rewards) to produce long intermediate reasoning before answering. Spending more compute at inference — longer reasoning, multiple samples with voting or verification — improves accuracy on math, code and multi-step problems.

2 min: Trade-offs: much higher latency and token cost per query, variable response times, and less benefit on simple lookups. Product design routes easy queries to fast models and hard ones to reasoning models, sets reasoning budgets, and evaluates cost-per-solved-task, not just accuracy.

Analogy: A student allowed to use rough paper and double-check each step scores higher on hard maths problems — but takes longer, and needs no rough paper to say what the capital of France is.

Deep: accuracy often grows roughly log-linearly with reasoning tokens on hard tasks (→ Parts 42–43).

Bad answer: "Reasoning models are just bigger models."

Follow-up: How would you decide whether a feature needs a reasoning model? → **Harder:** How do you evaluate reasoning faithfulness — does the written reasoning actually drive the answer?

Example: Routing only 10% of support questions (multi-step billing disputes) to a reasoning model kept cost flat while raising resolution on that slice.

G16 · Prompting, RAG or fine-tuning — how do you choose?

Testing: structured decision-making.

30 s: Identify the gap. If the model lacks knowledge (private, fresh, citable facts) → RAG. If it lacks behaviour (format, style, domain judgement) or a smaller model must match a bigger one → fine-tuning. Start with prompting, measure on an eval set, and climb only when needed.

2 min: Consider volume and latency (fine-tuned small models are cheaper at scale), change frequency (facts change daily → RAG; categories change weekly → prompts may be easier), data availability (hundreds to thousands of good examples for LoRA), and governance (citations and access control favour RAG). Often the answer is RAG + light fine-tuning.

Analogy: A new employee: give instructions first, then access to the handbook (RAG), and only then send them on a training course (fine-tuning) if they still do the job wrong.

Deep: cost comparison in Part 26; mechanisms in Part 43.

Bad answer: "Fine-tune on our documents so the model knows them."

Follow-up: How many examples do you need for LoRA? → **Harder:** When is continued pretraining justified?

Example: Legal Q&A with citations → RAG; ticket classification at 2M/month → distilled small model.

G17 · How does LoRA work, and why is it so popular?

Testing: parameter-efficient fine-tuning mechanics.

30 s: Freeze the pretrained weight W and learn a low-rank update $\Delta W = BA$ with rank $r \ll d$, scaled by α/r. B starts at zero, so training starts from the original model. Far fewer trainable parameters → far less optimizer memory, small adapter files, and many adapters can share one base model.

2 min: For a 4096×4096 matrix with $r = 8$: 65,536 trainable parameters vs. 16.8M. After training, merge BA into W for zero inference overhead, or keep adapters separate for multi-tenant serving. QLoRA adds a 4-bit quantized frozen base. Rank, which layers get adapters, and learning rate are the main knobs.

Analogy: Instead of repainting an entire house for a new tenant, you add removable wallpaper to a few walls — cheap, reversible, and you can swap it for the next tenant.

Deep: $h = Wx + \frac{\alpha}{r} BAx$ (→ Part 43 code).

Bad answer: "LoRA compresses the model so it runs faster."

Follow-up: Why initialize $B = 0$? → **Harder:** When does LoRA underperform full fine-tuning?

Example: 30 customer-specific adapters served on one base model instead of 30 full copies.

G18 · Walk me through RLHF.

Testing: the full pipeline and its failure modes.

30 s: (1) Supervised fine-tuning on demonstrations. (2) Collect human preference pairs and train a reward model with a Bradley–Terry loss. (3) Optimize the policy (classically with PPO) to maximize reward minus a KL penalty to the SFT model.

2 min: The KL term keeps the policy near the reference so it stays fluent and doesn't exploit reward-model errors. Failure modes: reward hacking (verbosity, sycophancy), reward-model overfitting, annotator inconsistency, mode collapse of style. Alternatives: DPO (no separate reward model or RL loop), RL with verifiable rewards for code/maths, AI feedback for scaling labels.

Analogy: Training a new comedian: first copy good routines (SFT), then a panel rates which of two jokes is funnier (reward model), then he practises to please the panel — while a coach stops him drifting into cheap tricks the panel happens to like (KL penalty).

Deep: $\max_\pi \mathbb{E}[r(x,y)] - \beta\,\mathrm{KL}(\pi\|\pi_{\text{ref}})$; $P(y_w \succ y_l) = \sigma(r_w - r_l)$ (→ Part 43).

Bad answer: "Humans label the correct answer and the model is trained on it." (That's SFT.)

Follow-up: What happens if β is too small? → **Harder:** How would you detect reward hacking before launch?

Example: After RLHF, answers grew 40% longer with no quality gain — a verbosity bias in the reward model.

G19 · How does DPO differ from PPO-based RLHF?

Testing: the key derivation idea.

30 s: DPO skips the explicit reward model and RL loop. It uses the closed-form optimum of the KL-regularized objective to rewrite the reward in terms of the policy's log-probability ratio against a reference model, then trains directly on preference pairs with a logistic loss.

2 min: Advantages: simpler, stable, cheaper (no sampling during training, no value model). Disadvantages: offline — it learns only from the fixed pairs; can over-optimize or reduce the likelihood of both chosen and rejected responses; quality depends heavily on the preference data. Online/iterative variants regenerate pairs from the current policy.

Analogy: Instead of hiring a judge and having the student practise against the judge (PPO), you show the student pairs of answers marked "this one is better" and let him adjust directly.

Deep: $\mathcal{L} = -\log\sigma\left(\beta\log\frac{\pi(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$ (→ Parts 31, 43).

Bad answer: "DPO is PPO without the KL penalty."

Follow-up: What role does π_{ref} play? → **Harder:** Derive the implicit reward $\beta\log\frac{\pi}{\pi_{\text{ref}}}$.

Example: A team replaced a four-model PPO stack with DPO on 50K preference pairs and matched win-rate at a fraction of the cost.

G20 · What are verifiable rewards and GRPO?

Testing: 2025–26 post-training methods.

30 s: For tasks with checkable answers (unit tests, maths results, format validators), the reward is computed automatically instead of learned from preferences. GRPO samples a group of answers per prompt and uses each answer's reward relative to the group's mean (normalized by the standard deviation) as its advantage — no value network needed.

2 min: Benefits: cheap, scalable rewards that are hard to fool if the checker is good; drove big gains in reasoning. Risks: reward hacking weak checkers (special-casing tests), groups with identical rewards give no signal, and generalization beyond verifiable domains is limited. Keep a KL or similar regularizer and audit samples.

Analogy: A coach who grades practice maths by comparing each student's score with the rest of the group that day: better than the group → reinforced; worse → discouraged; all equal → nothing learned.

Deep: $A_i = (r_i - \mathrm{mean}(r))/\mathrm{std}(r)$; rewards [1,0,0,1] → [1,−1,−1,1] (→ Part 43 code).

Bad answer: "GRPO is a new optimizer like Adam."

Follow-up: Why do all-correct or all-wrong groups waste compute? → **Harder:** How would you design a reward for a task with no single right answer?

Example: Code model trained with test-pass rewards learned to hard-code outputs until hidden tests were added.

G21 · Why mask the prompt tokens in SFT, and what else goes wrong in fine-tuning?

Testing: practical fine-tuning hygiene.

30 s: The loss should teach the model to produce responses, not to reproduce user prompts; masking prompt tokens concentrates learning on the answer and avoids learning to imitate users. Other pitfalls: wrong chat template, no end-of-turn token, duplicated or contaminated data, too many epochs.

2 min: Symptoms: the model rambles or never stops (missing end token), mimics user style, or scores suspiciously well (eval contamination). Always decode a few tokenized batches back to text, check label masks, and compare against a strong prompting baseline. Mix in general data to limit forgetting.

Analogy: Teaching someone to answer customer emails by making them memorize the customers' questions too — wasted effort, and they start writing like the customers.

Deep: labels = −100 (ignored) on prompt positions (→ Part 43).

Bad answer: "Masking is just an efficiency trick."

Follow-up: How many epochs for 5K examples? (Usually 1–3, watch eval.) → **Harder:** How do you detect catastrophic forgetting?

Example: A model that kept generating fake "User:" turns — trained without masking and without an end-of-turn token.

G22 · How does knowledge distillation work for LLMs?

Testing: teacher–student training.

30 s: A small student is trained to match a large teacher — either on the teacher's generated outputs (sequence-level) or on its full probability distributions (logit/KL distillation, with temperature). The student gets much of the teacher's quality at a fraction of the cost.

2 min: Soft targets carry "dark knowledge" — which wrong answers are nearly right. With temperature T, gradients scale by $1/T^2$, so the KL term is multiplied by T^2. For task-specific distillation, generate a large, diverse labeled set with the teacher, filter it, and evaluate on real data. Check the teacher's license terms.

Analogy: A master chef's apprentice learns not just the final dishes but how the chef hesitates between spices — the near-misses teach as much as the answers.

Deep: $\mathcal{L} = \alpha\,\mathrm{CE}(y, p_s) + (1-\alpha)T^2\,\mathrm{KL}(p_t^{(T)} \| p_s^{(T)})$ (→ Part 43).

Bad answer: "Distillation means compressing the weights."

Follow-up: Why multiply by T^2? → **Harder:** When does a student surpass the teacher on a narrow task?

Example: A 1B student distilled from a 70B teacher matched it on intent classification at 50× lower latency.

G23 · What is reward hacking and how do you catch it?

Testing: Goodhart's law in post-training.

30 s: The policy finds ways to raise the measured reward without doing the intended task — longer answers, flattering the user, exploiting checker bugs. Catch it with held-out human evaluation, monitoring proxies (length, refusal rate, sycophancy tests), and adversarial checks of the reward model.

2 min: Prevent it with a KL penalty, reward-model ensembles, length normalization, stronger verifiers (hidden tests), and early stopping on held-out human preferences. Plot reward vs. true quality over training: when they diverge, you're hacking.

Analogy: A school that rewards teachers only for exam pass rates gets teaching to the test — scores rise while real learning doesn't.

Deep: optimizing a proxy harder increases the gap to the true objective (→ Part 43).

Bad answer: "More training data fixes reward hacking."

Follow-up: How would you measure sycophancy? → **Harder:** Can DPO reward-hack?

Example: A reward model preferred answers with bullet points; the policy started bulleting everything.

G24 · How do you chunk documents for RAG?

Testing: the most underrated RAG decision.

30 s: Split along document structure (headings, paragraphs), with chunks typically a few hundred tokens and some overlap, and attach context (title, section path, date, source, permissions). Too small → missing context; too large → diluted embeddings and wasted context.

2 min: Parse properly first (tables, PDFs, slides). Consider hierarchical retrieval (retrieve small chunks, return the parent section), contextual chunk headers, and separate handling for tables and code. Tune chunk size on retrieval recall@k with a labeled question set rather than by intuition.

Analogy: Cutting a textbook into index cards: one card per sentence loses meaning, one card per chapter is too vague to find anything; one card per sub-section with its chapter name written on top works best.

Deep: evaluation drives the choice (→ Part 44).

Bad answer: "Split every 512 tokens."

Follow-up: A key fact spans two chunks. What do you do? → **Harder:** How would you chunk a 300-page contract with cross-references?

Example: Adding the section path "HR Policy › Leave › Maternity" to each chunk raised recall@5 from 0.71 to 0.83.

G25 · Why use hybrid search (BM25 + dense) and reranking?

Testing: retrieval fundamentals.

30 s: Dense retrieval captures meaning and paraphrase; BM25 nails exact terms (IDs, codes, names). Fusing their rankings (e.g. reciprocal rank fusion) gets the best of both. A cross-encoder then reranks the top 50–100 with much higher accuracy, because it reads the query and document together.

2 min: The funnel mirrors recommenders: cheap, high-recall retrieval, then expensive precise ranking. RRF uses ranks, not raw scores, so it's robust to incomparable score scales. Measure each stage: recall@100 for retrieval, nDCG@10 after reranking.

Analogy: Finding a book with two helpers — one who knows the exact catalogue codes and one who understands what you mean — then a senior librarian who reads the top shortlist carefully.

Deep: $\mathrm{RRF}(d) = \sum_r 1/(k + \mathrm{rank}_r(d))$, $k \approx 60$ (→ Part 44 code).

Bad answer: "Embeddings make keyword search obsolete."

Follow-up: Why not retrieve with the cross-encoder directly? → **Harder:** How do you fuse when one retriever returns nothing?

Example: Queries with error codes ("E-4012") failed with dense-only search and were fixed by adding BM25.

G26 · How do you evaluate a RAG system?

Testing: separating retrieval from generation.

30 s: Evaluate retrieval (recall@k, MRR, context precision) and generation (faithfulness to retrieved context, answer correctness, citation accuracy) separately on a labeled question set, then track online signals (feedback, follow-up rate, escalations).

2 min: For each failure, ask: was the supporting passage retrieved? If not, fix search; if yes, fix generation. Build the eval set from real questions plus audited synthetic ones; calibrate LLM judges against human labels; slice by question type; and re-run on every change to chunking, embeddings, prompts or models.

Analogy: Grading a student's open-book exam twice: did they open the right pages (retrieval), and did they answer from what's on those pages (faithfulness)?

Deep: the "RAG triad": context relevance, faithfulness/groundedness, answer relevance (→ Part 44).

Bad answer: "We read some answers and they looked good."

Follow-up: How do you get labels cheaply? → **Harder:** How do you trust an LLM judge for faithfulness?

Example: 20% error rate split into 12% retrieval misses and 8% generation errors — two different fixes.

G27 · Long context windows are huge now. Do we still need RAG?

Testing: trade-off reasoning beyond hype.

30 s: Usually yes. Cost and latency grow with context length, recall of details in very long contexts is imperfect, and you still need permission filtering, freshness and precise citations. Long context complements RAG (bigger chunks, whole-document reasoning) rather than replacing it.

2 min: For a small, static corpus (one manual) that fits in context, long context plus prefix caching can be simpler. For large, changing, permissioned corpora, retrieval is required. Measure: answer quality and cost at different context sizes on your eval set.

Analogy: Being allowed to carry the whole library into the exam hall doesn't mean you'll find the right page fast — and you still can't bring in the confidential shelves.

Deep: "lost in the middle" — details mid-context are recalled worse (→ Part 44).

Bad answer: "RAG is obsolete."

Follow-up: When is long context clearly better? → **Harder:** Design a hybrid that uses both.

Example: A 200-page product manual served via cached long context; the 5M-document intranet via RAG.

G28 · How do vector indexes like HNSW and IVF-PQ work, and how do you choose?

Testing: ANN trade-offs and memory math.

30 s: HNSW builds a layered proximity graph and greedily walks it — excellent recall and latency, memory-hungry. IVF clusters vectors and searches only nearby clusters; PQ compresses vectors into short codes — the billion-scale workhorse. Brute force is exact and fine up to about a million vectors.

2 min: Tune for recall@k against exact search at a latency budget. Memory: 100M × 768 float32 = 307 GB; PQ at 96 bytes = 9.6 GB. Metadata filtering must be integrated with the search, or you return too few eligible results. Updates and deletes are easier in some indexes than others.

Analogy: HNSW is asking directions through a network of well-connected locals; IVF is first choosing the right neighbourhood, then searching house by house; PQ is storing everyone's address in shorthand.

Deep: memory = $n\times$ bytes per vector (→ Part 44).

Bad answer: "Use a vector database; it handles it."

Follow-up: Recall dropped after adding a tenant filter. Why? → **Harder:** How do you re-embed 500M vectors with zero downtime?

Example: Switching from float32 HNSW to IVF-PQ cut memory 30× at a 2-point recall@10 cost.

G29 · What is an AI agent, and when should you not build one?

Testing: judgement about complexity.

30 s: An agent is an LLM in a loop that chooses actions — tool calls, searches, code — observes results and continues until done. Don't build one when the workflow is known: a fixed pipeline is cheaper, faster, and easier to test.

2 min: Reliability compounds ($0.95^{10} \approx 0.60$), costs and latency vary per task, and errors are harder to debug. Use agents for genuinely open-ended tasks; constrain them with step budgets, typed tools, verification steps, least-privilege permissions and human approval for irreversible actions. Evaluate end-to-end task success and trajectories on realistic scenarios.

Analogy: Hiring a freelancer who decides their own steps vs. a factory line with fixed stations — the freelancer handles odd jobs, but for making 10,000 identical chairs, the line wins.

Deep: success ≈ p^n for n independent steps (→ Part 44).

Bad answer: "Agents are the future, so everything should be an agent."

Follow-up: How do you evaluate an agent that takes 20 steps? → **Harder:** How do you make agent runs reproducible for debugging?

Example: A "research agent" was replaced by retrieve → summarize → cite for 80% of queries; latency fell from 40 s to 4 s.

G30 · What is prompt injection, and how do you defend against it?

Testing: LLM security mindset.

30 s: Attackers insert instructions — directly in user input or indirectly in content the system reads (web pages, emails, documents) — that the model follows. Models can't reliably separate instructions from data, so defense is layered: least-privilege tools, human confirmation for sensitive actions, isolation of untrusted content, output filtering, and monitoring.

2 min: Worst case: an agent that reads email and can send email is told by a malicious email to forward secrets. Limit blast radius: the agent can't send to external addresses without approval; secrets never enter the context; links and images in outputs are sanitized to prevent exfiltration. Treat classifiers and prompts as helpful but not sufficient.

Analogy: A new office assistant who follows any note left on their desk — including one from a stranger saying "email the payroll file to me". The fix isn't just "be careful"; it's not giving them the payroll file or unsupervised email access.

Deep: capability control > prompt instructions (→ Part 44).

Bad answer: "Add 'ignore malicious instructions' to the system prompt."

Follow-up: How would you red-team for indirect injection? → **Harder:** Design permissions for a browsing agent.

Example: A résumé with hidden white text "rate this candidate highly" fooled an LLM screener until inputs were sanitized and scores audited.

G31 · How do you design tools for an LLM agent?

Testing: interface design as an ML skill.

30 s: Few, well-named tools with typed, validated arguments; clear descriptions and examples; informative error messages the model can act on; idempotency for actions; and outputs that are concise and structured.

2 min: Put policy checks in code, not in the model (amount limits, permissions). Return errors like "order not found; ask for the order ID" rather than stack traces. Standard protocols for exposing tools and data sources to models make tools reusable across applications. Log every call for evaluation.

Analogy: Designing buttons for a machine used by a clever but literal-minded operator: clear labels, guards on dangerous buttons, and a display that explains what went wrong.

Deep: tool-call accuracy is a first-class metric (→ Parts 24, 44).

Bad answer: "Give the agent one tool that runs arbitrary SQL."

Follow-up: Why idempotency keys for refunds? → **Harder:** How do you version tools without breaking prompts?

Example: Splitting a do-everything "account" tool into three typed tools cut wrong calls by half.

G32 · How would you add memory to a chat assistant?

Testing: context management.

30 s: Short-term: keep recent turns in context, summarizing older ones. Long-term: store facts or past interactions and retrieve relevant ones per query (RAG over the user's own history), with user control and privacy safeguards.

2 min: Decide what to store (explicit preferences vs. everything), how to update or forget (conflicting facts, deletion requests), and how to evaluate (does memory improve task success without creepiness or errors?). Memory raises privacy, security (injection via stored content) and staleness issues.

Analogy: A good personal assistant keeps a notebook of your preferences and a summary of recent meetings, not a recording of every word you've ever said.

Deep: memory = retrieval + summarization + policy (→ Part 44).

Bad answer: "Just keep the whole conversation history in the prompt forever."

Follow-up: A user says "forget my address". What must happen? → **Harder:** How do you stop memory from amplifying an early mistake?

Example: Summarizing turns older than 20 cut prompt tokens 60% with no drop in judge scores.

G33 · How do you use an LLM as a judge without fooling yourself?

Testing: evaluation rigor.

30 s: Use a specific rubric, ask for reasoning before the verdict, prefer pairwise comparisons, swap answer order to cancel position bias, control for length, pin the judge version, and — most importantly — measure agreement with human labels (κ) per slice before trusting it.

2 min: Known biases: position, verbosity, self-preference, style over substance. Use a different model family where possible, reference answers for factual tasks, and human review of disagreements. Re-validate whenever the judge, rubric or product changes.

Analogy: A new examiner must first mark a batch of papers that senior examiners have already marked; only if they agree is the new examiner trusted alone.

Deep: $\kappa = (p_o - p_e)/(1 - p_e)$ (→ Part 45 code).

Bad answer: "GPT-style models are smarter than annotators, so their judgement is ground truth."

Follow-up: The judge agrees with humans 75% of the time. Good or bad? (Compare with human–human agreement.) → **Harder:** How do you detect judge drift?

Example: Swapping answer order flipped 18% of verdicts — those items were treated as ties.

G34 · Model B beats model A by 2 points on 500 questions. Is it better?

Testing: statistics applied to LLM evals.

30 s: Maybe not. The standard error of one accuracy at 70% on 500 items is about 2 points (±4 at 95%). Use a paired comparison on the same questions — bootstrap the per-question difference — which is far more sensitive because both models find the same questions easy or hard.

2 min: Also average over sampling seeds and prompt variants, check slices, and consider cost and latency. For small differences, increase the question count or use paired tests; beware of multiple comparisons when trying many prompt variants.

Analogy: Two cricketers' batting averages after five innings each don't settle who's better — but if they batted on the same pitches against the same bowlers, you can compare innings by innings.

Deep: $\mathrm{SE} = \sqrt{p(1-p)/n}$; paired CI [0.024, 0.076] vs. unpaired ±0.055 in Part 45's simulation.

Bad answer: "2 points is significant because it's bigger than zero."

Follow-up: How many questions to detect 2 points unpaired? (~8,000 per model.) → **Harder:** How does temperature > 0 change the analysis?

Example: A "better" prompt vanished after averaging five seeds.

G35 · What is benchmark contamination, and how do you detect it?

Testing: trusting numbers.

30 s: Test items (or close paraphrases) appeared in the training data, so scores measure memory, not ability. Detect with n-gram overlap searches, canary strings, comparing performance on original vs. rephrased or newly written items, and on items published after the data cutoff.

2 min: Consequences: inflated leaderboards and wrong model choices. Defenses: private held-out sets from your own traffic, dynamic benchmarks, decontamination of fine-tuning data, and treating public scores as weak evidence.

Analogy: A student who has seen the exact question paper in advance scores brilliantly — test him with freshly written questions to learn what he really knows.

Deep: performance gap between original and perturbed items is a contamination signal (→ Part 45).

Bad answer: "Public benchmarks are held out, so contamination isn't an issue."

Follow-up: How do you keep your internal eval set clean over time? → **Harder:** Can a model be contaminated without seeing exact items?

Example: Accuracy fell 15 points when numbers in maths problems were changed — memorization, not reasoning.

G36 · How do you balance safety and helpfulness (over-refusal)?

Testing: two-sided error thinking.

30 s: Treat it like precision and recall: measure harmful-compliance rate on a harmful set and false-refusal rate on a benign-but-edgy set. Tightening one loosens the other; the operating point depends on the product and users.

2 min: Improve both with better policies, targeted training data (including borderline benign examples), classifiers with context, and graded responses (partial help, safe alternatives) rather than binary refuse/comply. Monitor both rates by slice; over-refusal disproportionately hits some topics and dialects.

Analogy: An overly strict security guard stops everyone carrying a bag; a lax one lets anything through. You measure both how many threats got in and how many innocent visitors were turned away.

Deep: two error rates, one threshold — ROC thinking (→ Parts 16, 45).

Bad answer: "Safer is always better; refuse anything risky."

Follow-up: How do you build the benign-but-edgy set? → **Harder:** How do you handle a policy change without retraining?

Example: A medical-info assistant cut false refusals from 12% to 3% by adding benign clinical questions to training.

G37 · What guardrails would you put around an LLM feature?

Testing: defense in depth.

30 s: Input checks (abuse, injection, PII), grounding and permission enforcement, output checks (policy classifiers, schema validation, citation verification), rate limits, human review for high-risk actions, and logging with monitoring and incident response.

2 min: Each layer is imperfect; combine them and measure each one's catch rate and false-positive rate. Keep deterministic rules for what must never happen (money movement limits, data access). Red-team regularly and turn findings into regression tests.

Analogy: A bank uses locks, cameras, guards, dual signatures and audits — no single measure is trusted alone.

Deep: layered controls + evaluation of each layer (→ Part 45).

Bad answer: "The model is aligned, so it doesn't need guardrails."

Follow-up: How do guardrails affect latency? → **Harder:** How would you evaluate a guardrail's false-positive rate in production?

Example: A schema validator caught 2% malformed JSON outputs that would have crashed downstream services.

G38 · What is red-teaming for LLMs?

Testing: proactive safety evaluation.

30 s: Deliberately trying to make the system fail — jailbreaks, harmful content, privacy leaks, injection, bias — using domain experts plus automated attack generation. Findings become regression tests and training data.

2 min: Scope by risk (what could go wrong for this product), use diverse testers, track attack success rate over time, and re-run after model or prompt changes. Automated red-teaming uses models to generate and mutate attacks at scale; humans find the creative ones.

Analogy: Hiring ethical burglars to break into your house before real ones do — and fixing every window they got through.

Deep: attack success rate as a metric (→ Part 45).

Bad answer: "We tested it with some bad prompts before launch."

Follow-up: How do you prioritize findings? → **Harder:** How do you red-team an agent with tools?

Example: Role-play jailbreaks succeeded 30% of the time until added to the safety training set.

G39 · Estimate the compute and GPU time to train a 7B model on 2T tokens.

Testing: back-of-envelope fluency.

30 s: $C \approx 6ND = 6 \times 7 \times 10^9 \times 2 \times 10^{12} = 8.4 \times 10^{22}$ FLOPs. At ~400 TFLOP/s effective per GPU (40% MFU of ~1 PFLOP/s bf16), that's 2.1×10^8 GPU-seconds ≈ 59,000 GPU-hours ≈ 2.4 days on 1,024 GPUs.

2 min: State assumptions (MFU, hardware), note memory needs (16 bytes/param = 112 GB before activations → sharding required), and mention that attention adds FLOPs at long context. Sanity-check units.

Analogy: Like estimating how long it takes to paint a building: total wall area × coats ÷ (painters × speed × realistic efficiency).

Deep: forward $2ND$, backward $4ND$ (→ Part 46).

Bad answer: "It depends on many factors" (without a number).

Follow-up: What if MFU is only 25%? (Time × 1.6.) → **Harder:** How do inference costs over a year compare?

Example: The same formula shows why inference at scale can exceed training cost within months.

G40 · Why is LLM decoding memory-bandwidth-bound, and what follows?

Testing: roofline reasoning.

30 s: Generating one token for one sequence reads every weight (≈14 GB for a 7B fp16 model) but does only ~2 FLOPs per weight. GPUs have far more compute than bandwidth, so at batch 1 you're limited by bytes read: ~3.35 TB/s ÷ 14 GB ≈ 240 tokens/s upper bound.

2 min: Consequences: batching many sequences reuses each weight read (throughput rises almost free until compute or KV memory limits); weight quantization directly speeds decode; speculative decoding verifies several tokens per weight read; prefill is compute-bound, so prefill and decode behave differently (hence chunked prefill and disaggregation).

Analogy: A librarian who must walk to the stacks for every book: fetching one book per trip is slow; carrying requests for many readers per trip is almost as fast as one.

Deep: arithmetic intensity ≈ batch size; ridge ≈ 300 FLOPs/byte on H100-class GPUs (→ Part 46 figure).

Bad answer: "Decoding is slow because the model does a lot of math per token."

Follow-up: Why doesn't doubling batch size double latency at small batches? → **Harder:** When does decode become compute-bound?

Example: Moving from batch 1 to batch 32 raised throughput ~25× with a small increase in per-token latency.

G41 · What are continuous batching and PagedAttention?

Testing: modern serving systems.

30 s: Continuous batching schedules at every decode step: finished sequences leave, new ones join, so the GPU isn't idle waiting for the longest sequence. PagedAttention stores the KV cache in fixed-size blocks with a block table (like virtual memory), avoiding fragmentation and allowing sharing of common prefixes.

2 min: Together they raise throughput several-fold over static batching. Add prefix caching (reuse shared system-prompt KV), chunked prefill (long prompts don't stall others), and admission control based on KV memory. Measure TTFT, TPOT and goodput under realistic traffic.

Analogy: A shared auto-rickshaw that picks up and drops passengers along the way instead of waiting for a full load and driving everyone to the farthest stop; PagedAttention is assigning luggage space in standard boxes rather than one oddly shaped trunk per passenger.

Deep: iteration-level scheduling + block-based KV allocation (→ Part 46).

Bad answer: "Batching means waiting to collect requests before processing them."

Follow-up: What limits the batch size? (KV memory, latency SLOs.) → **Harder:** How would you schedule mixed long and short requests fairly?

Example: Static batching wasted 60% of GPU time on padding and waiting.

G42 · How does quantization speed up LLMs, and what can go wrong?

Testing: precision trade-offs.

30 s: Store weights (and sometimes activations and KV cache) in fewer bits — int8, 4-bit — so fewer bytes move per token (faster memory-bound decode) and bigger models fit on fewer GPUs. Per-channel or per-group scales preserve accuracy; methods like GPTQ, AWQ and SmoothQuant handle outliers.

2 min: Weight-only quantization is the common first step; activation quantization needs outlier handling. Risks: quality drops on long-context, multilingual, maths or rare slices even when headline benchmarks look fine. Always evaluate on your task and slices; keep sensitive layers at higher precision if needed.

Analogy: Rounding every price in a shop to the nearest ₹5: the till runs faster and the ledger is smaller, fine for groceries — but not for gold, where small errors matter.

Deep: 70B × 2 bytes = 140 GB fp16 → ~35 GB at 4-bit (→ Part 46 code).

Bad answer: "Quantization always loses a lot of accuracy" or "never loses any".

Follow-up: Why do activation outliers matter? → **Harder:** A larger 4-bit model vs. a smaller 16-bit model — how do you choose?

Example: 8-bit weights: no measurable loss; 4-bit: −3 points on the maths slice only.

G43 · Explain speculative decoding. Does it change outputs?

Testing: a clever systems idea with a correctness argument.

30 s: A small draft model proposes several tokens; the large model checks them all in one forward pass. Tokens are accepted with probability $\min(1, p/q)$, and on rejection a token is resampled from the leftover distribution. The output distribution is exactly the large model's — only faster.

2 min: Speed-up depends on the acceptance rate $\alpha = \sum \min(p, q)$ and draft length γ: expected tokens per target pass $= (1 - \alpha^{\gamma+1})/(1 - \alpha)$, e.g. 2.8 at $\alpha = 0.7, \gamma = 4$. Works because verification of several tokens costs about the same as generating one in memory-bound decode. Variants use extra heads or copy from the prompt.

Analogy: A junior drafts the next few lines of a letter; the senior reads them in one go and keeps the lines she agrees with, rewriting from the first line she wouldn't have written.

Deep: accept rate 0.701 vs. theory 0.70 in Part 46's check.

Bad answer: "The small model's output is used when it's good enough, so quality drops slightly."

Follow-up: When does it not help? (Low acceptance, compute-bound large batches.) → **Harder:** Prove the output distribution equals the target.

Example: 2.3× faster code completion because code is highly predictable for the draft.

G44 · What metrics matter when serving an LLM?

Testing: production vocabulary.

30 s: Time to first token (TTFT), time per output token (TPOT / inter-token latency), end-to-end latency, throughput (tokens/s), goodput (requests meeting SLOs), cost per million tokens or per task, error/timeout rates — plus quality metrics on sampled traffic.

2 min: Different products care about different ones: chat cares about TTFT (streaming hides the rest); batch summarization cares about cost and throughput; agents care about end-to-end task time. Report percentiles (p50/p95/p99), not averages, and by prompt length.

Analogy: For a restaurant: time until the first dish arrives, time between courses, total meal time, meals served per hour, and meals served on time — each matters to a different customer.

Deep: TTFT ≈ queueing + prefill; decode time ≈ output tokens × TPOT (→ Part 46).

Bad answer: "Average latency."

Follow-up: TTFT spikes for some users. Cause? (Long prompts from others, no chunked prefill.) → **Harder:** How do you set SLOs for a mixed workload?

Example: p95 TTFT dropped from 3 s to 0.9 s after chunked prefill.

G45 · How would you cut LLM costs by half without hurting quality?

Testing: cost engineering with evaluation discipline.

30 s: Measure where tokens go, then: trim prompts and retrieved context, cap output length, enable prefix caching, route easy requests to a smaller model, cache repeated answers, batch offline jobs, and distil high-volume narrow tasks — each change checked on the eval set.

2 min: Model routing/cascades often give the biggest win; long system prompts and unbounded chat history are common waste. Quantify: cost per request = input tokens × input price + output tokens × output price. Track cost per *successful task*, not just per call.

Analogy: Halving a household's electricity bill: find which appliances use the most, switch off what's idle, use the efficient appliance for small jobs, and only run the big machine when needed.

Deep: 1M requests/day × (1,500 in, 300 out) at $1/$4 per M ≈ $2,700/day (→ Part 46).

Bad answer: "Switch to the cheapest model."

Follow-up: How do you decide the routing threshold? → **Harder:** How do you prevent quality regressions for rare, hard queries?

Example: Routing 70% of traffic to a small model and caching the system prompt cut cost 58% with no change in judge scores.

G46 · Explain two-tower retrieval and the logQ correction.

Testing: large-scale recommendation retrieval.

30 s: A user tower and an item tower map into the same embedding space; score = dot product; item embeddings are precomputed and indexed for ANN search. Trained with in-batch negatives, popular items appear as negatives too often, so we subtract $\log q_i$ (their sampling probability) from logits to correct the bias.

2 min: Other ingredients: mixing random negatives, careful hard-negative mining (many "negatives" are unlabeled positives), content features for cold-start items, and evaluating recall@k of items the ranker would like. It's the same in-batch softmax as contrastive learning (CLIP).

Analogy: Judging a dish's appeal by how often it loses in taste tests is unfair to the most popular dish — it's in every test. You correct for how often each dish was entered.

Deep: $s^c(u,i) = s(u,i) - \log q_i$ (→ Part 47 code).

Bad answer: "Negatives are just random items, so no bias."

Follow-up: Why not use the ranker for retrieval? → **Harder:** How do you estimate q_i in a streaming setting?

Example: After logQ correction, head items stopped being under-recommended and long-tail items stopped being over-recommended.

G47 · Bandits vs. A/B tests — when would you use each?

Testing: exploration/exploitation judgement.

30 s: A/B tests give clean, unbiased estimates of a fixed difference for decisions; bandits adaptively shift traffic toward better arms to reduce the cost of exploring — good for many short-lived options (headlines, promotions) and continuous optimization.

2 min: UCB and Thompson sampling achieve logarithmic regret; fixed ε-greedy has linear regret. Bandit data is adaptively collected, so naive estimates are biased — log propensities for off-policy evaluation. Contextual bandits (LinUCB) personalize choices.

Analogy: Tasting every stall at a food festival equally (A/B) tells you exactly which is best; a smart eater (bandit) keeps sampling a bit but mostly returns to the best stall so far — tastier evening, fuzzier final verdict.

Deep: UCB1: $\bar{x}_a + \sqrt{2 \ln t / n_a}$ (→ Part 47 figure).

Bad answer: "Bandits are always better because they waste less traffic."

Follow-up: Why is Thompson sampling robust to delayed feedback? → **Harder:** How do you evaluate a new bandit policy offline?

Example: Headline testing with Thompson sampling captured 80% of the best headline's clicks during the test.

G48 · How do you evaluate a new recommendation policy without an A/B test?

Testing: counterfactual evaluation.

30 s: Off-policy evaluation from logs: reweight logged rewards by $\pi(a \mid x)/\mu(a \mid x)$ (IPS), self-normalize (SNIPS), or combine with a reward model (doubly robust). It needs logged propensities and overlap — the old policy must have shown the new policy's choices with nonzero probability.

2 min: IPS is unbiased but high-variance when weights are large; clip weights, use SNIPS or DR. Without randomization in logging, you can't evaluate actions the old policy never took. That's why production systems keep some exploration traffic.

Analogy: Estimating how a new restaurant menu would sell using last year's orders — counting each order more if the new menu would push that dish much more than the old one did.

Deep: $\hat{V}_{\text{IPS}} = \frac{1}{n} \sum \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)} r_t$; naive 0.028 vs. IPS 0.053 vs. true 0.052 (→ Part 47).

Bad answer: "Replay the logs and count clicks on items the new policy would rank highly."

Follow-up: What if the logging policy was deterministic? → **Harder:** How do you evaluate a slate (a whole ranked list)?

Example: Adding 5% randomized traffic made future off-policy evaluation possible.

G49 · Can a model be calibrated and have equal error rates across groups?

Testing: fairness impossibility and judgement.

30 s: Not in general. When base rates differ between groups, an imperfect classifier can't be calibrated within groups and also have equal false-positive and false-negative rates. You must choose which criterion matters for the application — a value judgement made with stakeholders and legal input.

2 min: Show it with numbers: base rates 0.2 vs. 0.4, a perfectly calibrated score, one threshold → TPR 0.26 vs. 0.75 and FPR 0.08 vs. 0.52. Mitigations depend on the source of disparity: better data and labels, constraints, or group-aware thresholds where lawful. Always report per-group metrics with uncertainty.

Analogy: A single exam cut-off that means the same thing for every student ("70 = 70% chance of success") will still admit very different shares of two schools whose students were prepared differently.

Deep: Kleinberg–Mullainathan–Raghavan; Chouldechova (→ Part 48 table).

Bad answer: "Just remove the sensitive attribute."

Follow-up: Which criterion for a medical screening tool? → **Harder:** How does label bias (arrests ≠ crime) change the picture?

Example: A loan model's approval gap was mostly explained by missing credit history for one group — fixed with alternative data, not thresholds.

G50 · What is differential privacy, and how does DP-SGD work?

Testing: privacy guarantees vs. heuristics.

30 s: DP bounds how much any single person's data can change the output distribution: $P(M(D) \in O) \leq e^{\varepsilon} P(M(D') \in O) + \delta$. DP-SGD clips each example's gradient to norm C and adds Gaussian noise to the sum, so no one example can move the model much; a privacy accountant tracks total ε.

2 min: Costs: per-example gradients (memory/compute), lower accuracy (worse for minority groups), and careful hyperparameter tuning. "Anonymization" by removing names isn't DP — linkage attacks re-identify people. Federated learning keeps data on devices but needs DP and secure aggregation to protect updates.

Analogy: Adding a little random noise to every survey answer so no individual's answer can be pinned down, while the overall average stays accurate.

Deep: Laplace mechanism noise scale Δ/ε; randomized response estimate 0.296 vs. true 0.297 (→ Part 48 code).

Bad answer: "We hash user IDs, so the model is private."

Follow-up: What does ε = 1 mean? → **Harder:** Why does DP hurt accuracy on rare groups most?

Example: A keyboard model trained with federated learning + DP learned common phrases without memorizing individual messages.

PART 28

Why Questions

The shape of a great "why" answer

Claim → mechanism → math → where it breaks. Interviewers ask "why" to see whether you can go one level below the slogan. If your answer could be followed by "but why?" and you'd have nothing to add, it isn't finished yet.

The whole Part as one picture

Think of it like a curious child who keeps asking "but why?". The first answer ("regularization prevents overfitting") is the slogan. The second ("it limits how extreme the weights can get") is the mechanism. The third ("because noise can only be fitted with extreme weights, and the penalty makes those expensive") is understanding. Interviewers play that child. Each answer in this Part is built to survive three "why?"s in a row.

Where it breaks: a child accepts any confident answer; an interviewer checks the mechanism against edge cases — so always end with "where it breaks".

28.1 Why does regularization reduce overfitting?

Mechanism

Claim: it trades a small increase in bias for a larger decrease in variance.

Mechanism. Overfitting is the model using its flexibility to fit sample-specific noise. Fitting noise requires extreme parameter configurations — large weights that make the function change sharply between nearby training points, or many features combining to explain idiosyncrasies. Regularization makes those configurations expensive, so the optimizer only pays for complexity that the data supports across many examples. Formally it shrinks the **effective hypothesis space**, and generalization gaps scale with that space's complexity.

Math. Ridge in the SVD basis: each direction's fitted component is multiplied by $\frac{s_j^2}{s_j^2+\lambda}$. Directions with small singular values — little data support, OLS variance σ^2/s_j^2 — are shrunk most. Bayesian: the penalty is a prior; MAP combines weak data evidence with the prior belief "weights are small".

Where it breaks. If the model underfits, regularization increases the dominant error (bias). If features aren't standardized, the penalty encodes arbitrary units. If the true function needs large weights (strong effects), the prior is wrong.

Real-life analogy: a strict editor on word count

Think of it like an essay contest with a strict word limit. A writer with unlimited words stuffs in every rumour and coincidence they heard (noise). With a limit, they must keep only the points that really matter — and the essay holds up better when a different judge reads it.

Where it breaks: if the limit is too strict, even essential points get cut — that's underfitting.

28.2 Why does L1 produce sparse weights?

Mechanism

Claim: the L1 penalty's pull toward zero doesn't weaken near zero, so weak weights are pushed exactly to zero.

Mechanism. Consider the "force" on a weight. L2's gradient $2\lambda w$ fades as $w \to 0$: it can never finish the job. L1's (sub)gradient is $\lambda \operatorname{sign}(w)$ — a constant force. If the data's pull on the weight (its correlation with the residual, $|\rho|$) is weaker than λ, the penalty wins everywhere, and the optimum sits at the kink $w = 0$, where the subgradient $[-\lambda, \lambda]$ can balance the data's pull.

Math. $\min_w \frac{1}{2}(w-\rho)^2 + \lambda|w| \Rightarrow w^* = \operatorname{sign}(\rho)\max(|\rho| - \lambda, 0)$. Geometry: the L1 ball's corners lie on the axes; elliptical loss contours first touch a polytope at a vertex/edge with probability that grows with dimension.

Where it breaks. Correlated features: L1 picks one arbitrarily and unstably (elastic net fixes it). With $d > n$, at most n nonzeros. Sparsity is in the chosen parametrization — rotate the features and the sparse solution disappears.



Real-life analogy: a fixed entry fee for every feature

Think of it like a mall that charges every shop a fixed rent per unit of floor space, however small. A shop that earns only a little can't justify any space and closes completely (weight exactly zero); a shop that earns well stays open. L2 is more like a tax that gets tiny for tiny shops, so small shops shrink but never fully close.

Where it breaks: real shops choose independently; correlated features compete, and lasso may keep one and close its near-twin arbitrarily.

28.3 Why does feature scaling matter for some algorithms but not for trees?

Mechanism

Claim: algorithms that combine features arithmetically are scale-sensitive; algorithms that only compare values within one feature are not.

Mechanism. (1) **Distances** (KNN, k-means, RBF kernels) sum per-feature squared differences — a feature in larger units dominates. (2) **Penalties** (ridge, lasso, weight decay) penalize coefficients measured in feature units — a feature in small units needs a large coefficient and gets penalized more. (3) **Gradient descent** convergence depends on the Hessian's condition number; features with very different scales produce elongated loss bowls and zig-zagging. (4) **PCA** finds directions of maximum variance — rescaling a feature changes its variance. A **tree split** asks only "$x_j \le t$?": any strictly increasing transform of x_j preserves the ordering, so the same partitions exist and the tree is identical up to the threshold values.

Math. For GD on least squares, the Hessian $\frac{1}{n}X^\top X$ has condition number growing with the ratio of feature scales; iterations scale with κ.

Where it breaks. Trees are still affected by transforms that are not monotonic, and histogram-based trees depend on binning (quantile bins are scale-free; uniform bins aren't). Unregularized OLS predictions are scale-invariant, but interpretation and numerics are not.

Real-life analogy: ranking vs. measuring

Think of it like two ways to sort students. A tree asks "is height above 160 cm?" — it only cares about the *order*, so measuring in cm or inches gives the same split. Distance-based and gradient-based methods *add up* numbers across features, so a feature recorded in large units shouts over the others unless you rescale.

Where it breaks: scaling doesn't change a tree's splits, but it can still matter for trees in other ways — e.g. regularization terms or histogram binning edge effects are usually minor.

28.4 Why does bagging reduce variance?

Mechanism

Claim: averaging several unstable models cancels out their sample-specific fluctuations while keeping their shared signal.

Mechanism. A deep tree's prediction at x = true signal + a fluctuation caused by the particular training sample. Different bootstrap samples produce partly different fluctuations. Averaging keeps the common part (the expected prediction — unchanged, so bias is unchanged) and averages the fluctuations, which partly cancel.

Math. $\mathrm{Var}\left(\frac{1}{B}\sum \hat{f}_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. The cancellable part shrinks as $1/B$; the correlated part $\rho\sigma^2$ remains. That's why random forests add feature subsampling (lower ρ) and why bagging stable models (low σ^2) is pointless.

Where it breaks. It can't fix bias (all trees fail to extrapolate — so does the forest); with highly correlated base models the gain plateaus; bootstrap samples contain only ~63% unique points, which slightly increases each model's variance and bias.

Real-life analogy: asking many independent judges

Think of it like a quiz show's "ask the audience" lifeline. Any single person might be wrong in a random way, but the average of many people's guesses is far steadier — their random errors cancel out. Bagging trains many models on different resamples and averages them.

Where it breaks: if the audience all read the same wrong newspaper (correlated models), their errors don't cancel — that's why random forests add feature randomness to decorrelate trees.

28.5 Why does boosting reduce bias?

Mechanism

Claim: boosting builds a more expressive model step by step, each step targeted at the current errors.

Mechanism. Start with a very simple model (a constant). Each round fits a weak learner to the **negative gradient of the loss** at the current predictions — the direction each training prediction should move to reduce the loss — and adds a small step of it. The function class grows with each round (a sum of M depth-k trees represents far more functions than one), and training loss keeps decreasing: it's gradient descent in function space. For AdaBoost, training error is bounded by $\exp(-2\sum_t \gamma_t^2)$ where γ_t is how much each learner beats chance.

Where it breaks. The same mechanism fits noise when run too long (variance grows) — hence shrinkage, subsampling, early stopping. With label noise, exponential loss (AdaBoost) concentrates weight on mislabeled points. Weak learners must be *weak but not useless*: depth-1 stumps can't capture interactions; very deep trees make boosting overfit quickly.



Real-life analogy: a tutor who focuses on last test's mistakes

Think of it like a tutor who, after every mock test, focuses only on the questions you got wrong. Each round fixes the remaining errors a little, and step by step your weak areas get covered. Boosting adds simple models one at a time, each trained to fix what the current ensemble still gets wrong.

Where it breaks: a tutor stops before you memorize the answer key; boosting doesn't know when to stop, so without a small learning rate and early stopping it starts fitting noise.

28.6 Why does cross-entropy work for classification?

Mechanism

Claim: it's the correct likelihood for categorical labels, it's a proper scoring rule, and its gradients are well-behaved.

Mechanism. (1) **Likelihood:** labels are Bernoulli/categorical draws; the negative log-likelihood of the true class is exactly cross-entropy — so minimizing it is maximum likelihood, with all MLE guarantees (consistency, efficiency). (2) **Information:** it equals $H(P) + \mathrm{KL}(P\|Q)$ — minimizing it pushes the model's distribution toward the data's. (3) **Proper scoring rule:** expected loss is minimized by reporting the true conditional probability — so outputs are calibrated in the limit. (4) **Gradients:** with sigmoid/softmax, $\partial \ell / \partial z = p - y$ — large when confidently wrong, never saturating (MSE's extra $p(1-p)$ factor vanishes exactly when the model is wrong and confident). (5) **Convexity** for linear models.

Where it breaks. Label noise (unbounded loss on confidently "wrong" but actually mislabeled examples); extreme imbalance (easy majority dominates the sum); overconfidence in over-parameterized nets (calibration drifts); and it optimizes likelihood, not your business metric.



Real-life analogy: rewarding honest confidence

Think of it like a betting game that rewards honest forecasters. If you say "90% sure" and you're right, you gain; if you're wrong, you lose big. The only way to win in the long run is to state your *true* confidence. Cross-entropy is such a scoring rule, which is why it trains models to output meaningful probabilities.

Where it breaks: honesty is rewarded only on average and on the training distribution; a big model can still become over-confident on new data and needs calibration.

28.7 Why does gradient descent work?

Mechanism

Claim: locally, a smooth function looks linear; stepping against the gradient is guaranteed to decrease it if the step is small enough relative to the curvature.

Mechanism. Taylor: $J(\theta - \eta g) \approx J(\theta) - \eta\|g\|^2 + \frac{\eta^2}{2} g^\top H g$. The first-order term always decreases J; the second-order term (curvature) can overwhelm it if η is too large. With L-smooth J and $\eta \leq 1/L$: J decreases by at least $\frac{\eta}{2}\|g\|^2$ per step — so the gradient norm must shrink toward zero (a stationary point). With convexity, stationary = global minimum; with strong convexity, convergence is linear.

Why it works in deep learning despite non-convexity: high-dimensional loss surfaces have few bad local minima relative to saddles, SGD noise helps escape saddles, over-parameterization creates many good solutions and wide valleys, and architecture/initialization choices keep gradients informative.

Where it breaks. Ill-conditioning (slow), too-large steps (divergence), vanishing/exploding gradients, non-smooth objectives (need subgradients/proximal methods), and flat plateaus.



Real-life analogy: finding the lowest spot on a field in the dark

Think of it like looking for the lowest spot on a dark field with a walking stick: poke around your feet, find which way slopes down, and take a small step. Small steps in the downhill direction always lower your height, as long as the steps are small enough for the ground's bumpiness.

Where it breaks: you'll find *a* low spot, not necessarily the lowest — on non-convex losses, gradient descent only guarantees local progress.

28.8 Why can accuracy be misleading?

Mechanism

Claim: accuracy averages over all examples with equal weight, so it's dominated by the majority class and treats all errors as equally costly.

Mechanism. Accuracy $= \pi \cdot \text{recall} + (1-\pi) \cdot \text{specificity}$ where π is prevalence. With $\pi = 0.001$, accuracy ≈ specificity — recall barely registers. So a model that ignores the rare class scores 99.9%. It also ignores *which* errors happen (a missed fraud ≠ a false alarm), ignores confidence (a 51% and a 99% correct prediction count the same), and depends on the threshold.

Where it's fine: balanced classes with symmetric costs and a decision-oriented use (e.g. digit recognition).



Real-life analogy: a security guard who waves everyone in

Think of it like a mall security guard who never stops anyone. If only 1 in 1,000 visitors is a shoplifter, he is "99.9% accurate" — and completely useless. With rare positives, accuracy rewards doing nothing; you need precision, recall, and costs to judge the guard.

Where it breaks: when classes are balanced and errors cost the same, accuracy is a perfectly good summary.

28.9 Why does high dimensionality hurt KNN?

Mechanism

Claim: KNN relies on nearby points being similar; in high dimensions there are no nearby points and "nearest" stops meaning "similar".

Mechanism. (1) **Emptiness:** the volume to cover k neighbors grows so fast that the neighborhood spans most of each axis ($r^{1/d}$: capturing 1% in 10-D needs 63% of each axis). (2) **Concentration:** for independent coordinates, squared distance is a sum of d terms; its mean grows like d but its standard deviation like $\sqrt{d}$ — relative spread shrinks as $1/\sqrt{d}$, so all points look almost equally far. (3) **Noise accumulation:** each irrelevant feature adds variance to every distance, burying the few informative coordinates.

Where it's less severe: data on low-dimensional manifolds (low intrinsic dimension), learned embeddings where distance encodes meaning, and when feature selection or metric learning emphasizes relevant directions.



Real-life analogy: everyone becomes a stranger

Think of it like comparing people using hundreds of random traits. With two traits (age, city), some people are clearly similar to you. With 500 traits, everyone differs from you in roughly the same number of ways, so your "nearest" neighbour is barely nearer than a random stranger.

Where it breaks: if the data really lies on a few meaningful directions (good embeddings), nearest neighbours stay meaningful even with many raw dimensions.

28.10 Why do neural networks require nonlinear activations?

Mechanism

Claim: without them, any depth of layers collapses to a single linear map.

Mechanism. $W_L \cdots W_2 W_1 x = Wx$ — one matrix. The network could only represent linear functions (linear regression / logistic regression with a sigmoid at the end), unable to model interactions, XOR, curved boundaries. Nonlinearities let each layer *fold* space: a ReLU network partitions the input space into linear regions whose count can grow exponentially with depth, which is where depth's efficiency comes from. With nonlinear activations, universal approximation holds.

Subtlety: a deep *linear* network has the expressivity of a linear model but a non-convex loss in its weights — its training dynamics differ from a single linear layer (useful theory model).



Real-life analogy: stacking glass panes

Think of it like stacking coloured glass panes: however many flat panes you stack, you still get one flat pane of a different tint — never a curved lens. Linear layers stacked together collapse into one linear layer. A nonlinear activation is like bending each pane slightly, so the stack can form any shape.

Where it breaks: glass bends smoothly; ReLU is piecewise-straight, yet enough of these kinks can approximate any curve.

28.11 Why can deeper networks be harder to optimize?

Mechanism

Claim: gradients are products over layers; products of many factors are numerically fragile, and depth makes the landscape ill-conditioned.

Mechanism. $\partial L/\partial h_1 = \partial L/\partial h_L \prod_l J_l$. If typical Jacobian gains are < 1, gradients vanish exponentially; > 1, they explode. Different layers receive gradients at very different scales, so no single learning rate suits all. Small changes to early layers get amplified through later ones (sharp curvature). Empirically, before residual connections, deeper plain networks had *higher training error* — an optimization failure (the degradation problem).

Fixes map directly to the mechanism: initialization for unit gain, normalization, residual identity paths, adaptive optimizers, warmup, clipping.



Real-life analogy: a long game of Chinese whispers

Think of it like a message passed through 100 people: small distortions at each step compound, and the final message (or the correction sent back) arrives garbled — too faint or wildly exaggerated. Deep networks pass signals and gradients through many layers, so small scale errors multiply.

Where it breaks: with good design (residual connections, normalization, careful initialization), very deep networks train well — depth itself isn't the enemy, uncontrolled scaling is.

28.12 Why does batch normalization help?

Mechanism

Claim: it keeps each layer's input distribution at a fixed scale, which makes optimization better conditioned and more stable.

Mechanism. (1) **Smoother landscape:** normalizing activations bounds how much the loss and its gradient can change for a given parameter step (Santurkar et al.) → larger stable learning rates. (2) **Scale invariance:** the layer's output is invariant to rescaling its incoming weights, so the effective step size self-adjusts and exploding activations are prevented. (3) **Noise regularization:** batch statistics vary between batches.

Where it breaks: small or non-IID batches (noisy/biased statistics), train/inference mismatch through running statistics, sequence models, distributed sync costs — hence LayerNorm/GroupNorm alternatives.



Real-life analogy: standardizing the units at every factory stage

Think of it like a factory where each station hands parts to the next — and every morning the previous station changes its units (cm today, inches tomorrow). Each station wastes effort readjusting. Batch norm re-standardizes the parts at each station, so every layer sees inputs on a stable, well-behaved scale, allowing faster, steadier training.

Where it breaks: the "changing units" story (internal covariate shift) is debated; the more supported explanation is that batch norm smooths the loss surface — and it misbehaves with tiny batches.

28.13 Why do residual connections help?

Mechanism

Claim: they give gradients and signals an identity path through the network, and make "do nothing" the default for each block.

Mechanism. $h_{l+1} = h_l + F(h_l) \Rightarrow \frac{\partial h_L}{\partial h_l} = \prod(I + \partial F/\partial h)$, which expands into a sum that includes the identity — gradients reach early layers without passing through every weight matrix. Blocks only need to learn *residual corrections*; if a block is useless, $F \approx 0$ is easy to learn. The network behaves like an ensemble of many paths of different lengths, dominated in training by short ones.

Consequence: depth becomes (nearly) free to add without hurting optimization — enabling 100+ layer CNNs and deep transformers.



Real-life analogy: a flyover over city traffic

Think of it like a flyover that lets traffic skip the congested local roads. Each block only needs to add a small correction to what passes over it, and in the backward pass the gradient gets an express lane straight to early layers instead of crawling through every junction.

Where it breaks: a flyover carries everything unchanged; residual blocks still transform the signal, and without normalization their additions can grow with depth.

28.14 Why can offline metrics disagree with online performance?

Mechanism — seven gaps between offline and online

1. **Metric–objective mismatch:** offline AUC/NDCG measures something different from revenue/engagement (e.g. whole-ranking vs. top-slot quality; calibration matters online but not for AUC).
2. **Non-representative test data:** random splits on temporal data, leakage, duplicates, stale test sets.
3. **Training–serving skew:** features computed differently or delayed online.
4. **Feedback and exposure bias:** logs reflect the old model's choices; the new model changes what users see, which offline data can't capture.
5. **User behavior dynamics:** novelty effects, learning, long-term vs. short-term reactions, interaction effects with other system components (re-rankers, auctions, UI).
6. **System effects:** latency increases, timeouts, fallbacks.
7. **Experiment flaws:** underpowered tests, SRM, interference, peeking.

What to do: choose offline metrics that simulate the decision; time-based evaluation on recent data; feature parity checks; off-policy evaluation; shadow and canary deployments; track historical offline–online correlation.



Real-life analogy: acing mock tests but failing the real exam

Think of it like a student who aces mock tests set by his own coaching centre, then struggles in the board exam: the mock questions resembled what he'd practised (biased test set), were graded differently (metric mismatch), and didn't include the exam-day pressure (live conditions). Offline tests measure yesterday's world with yesterday's yardstick.

Where it breaks: good offline tests do predict online results often — the fix is to make them resemble reality (time splits, proper metrics, feature parity), not to abandon them.

28.15 Four more "why"s interviewers like

Why do error bars shrink like 1/√n?

The standard error of a mean of n independent observations is $\sigma/\sqrt{n}$ because variances add: $\mathrm{Var}(\sum X_i) = n\sigma^2$, so $\mathrm{Var}(\bar{X}) = \sigma^2/n$. Consequence: halving the error needs 4× data; detecting half the effect needs 4× users. With correlated observations, the effective n is smaller and error bars are wider than the formula says.

Why does leakage produce results that are too good?

The model finds the easiest path to low loss. A leaked feature is a near-deterministic function of the label, so it's easier to use than any genuine signal; the model relies on it, and validation data containing the same leak rewards that reliance. In production the leak is absent, so the model's main input is missing or meaningless — performance collapses to whatever the genuine features support (or worse, since the model never learned to use them).

Why does early stopping regularize?

Gradient descent fits high-signal directions (large curvature/eigenvalues) first and low-signal, noise-dominated directions last. Stopping early leaves those directions under-fitted — the same shrinkage pattern as ridge. For networks, it also limits how far weights move from their (small) initialization, bounding complexity.

Why do ensembles of different model types often beat any single one?

Different model families make different errors (different inductive biases), so their errors are less correlated than those of copies of one model. Averaging then removes more error: with correlation ρ between errors, variance falls toward $\rho\sigma^2$ — lower ρ, bigger gain. Stacking additionally learns where each model is reliable.



Real-life analogy: four everyday echoes

Think of it like four familiar situations. Error bars shrink like $1/\sqrt{n}$: polling 4× more voters only halves the error of an exit poll. Leakage is like marking your own exam with the answer key — spectacular scores that mean nothing. Early stopping is taking the cake out of the oven when it's done instead of waiting until it's burnt. Different model types in an ensemble are like a panel of a doctor, an engineer and a lawyer — their mistakes differ, so together they catch more.

Where it breaks: each analogy captures the core idea only; the mechanisms in the boxes above explain when each rule fails.

PART 29

Compare Questions

How to answer "X vs Y"

State the **one core difference** (the mechanism), then **what follows from it** (2–3 consequences), then **when you'd pick each** — with a concrete situation. Never end at "it depends"; say what it depends on.

The whole Part as one picture

Think of it like choosing between a scooter and a car. Neither is "better" — it depends on the trip: distance, traffic, passengers, budget. A strong "X vs Y" answer does the same: state what each is good at, the one mechanism that causes the difference, and the situation where you'd pick each.

Where it breaks: vehicles are fixed products; ML methods have knobs, so a well-tuned "worse" method can beat a badly tuned "better" one — mention tuning and data size.

29.1 Models

Comparison	Core difference	Choose the first when	Choose the second when
Linear vs. logistic regression	Gaussian noise + identity link (predict a mean) vs. Bernoulli + logit link (predict a probability)	target is continuous; you want the conditional mean	target is binary; you need probabilities or a classifier
Generative vs. discriminative	model $P(x, y)$ (then Bayes' rule) vs. $P(y \mid x)$ directly	little data and assumptions roughly hold; missing features; need to generate/detect outliers	enough data; best predictive accuracy; assumptions doubtful
Bagging vs. boosting	parallel averaging of strong learners (↓variance) vs. sequential correction with weak learners (↓bias)	noisy labels, little tuning time, need robustness	maximum accuracy on structured data, time to tune
Random forest vs. XGBoost	averaged deep trees vs. additive shallow trees fit to gradients	quick strong baseline; noisy data; OOB estimates	top tabular accuracy; custom losses (ranking, quantile); fast inference with fewer, shallower trees
XGBoost vs. LightGBM	depth-wise default growth, second-order regularized objective vs. histogram + leaf-wise growth + GOSS/EFB	smaller data / conservative growth; mature ecosystem	very large or sparse data; speed-critical iteration (tune num_leaves)
KNN vs. k-means	supervised lazy classifier/ regressor vs. unsupervised clustering algorithm (both use distances, that's all)	predicting labels from similar labeled examples	discovering groups/prototypes in unlabeled data
Parametric vs. non-parametric	fixed number of parameters vs. complexity growing with data	small data, speed, interpretability, strong priors	lots of data, complex unknown structure

Comparison	Core difference	Choose the first when	Choose the second when
PCA vs. t-SNE	linear variance-preserving projection vs. nonlinear neighborhood-preserving embedding	compression, preprocessing, need a transform, global structure	visual exploration of local cluster structure only

29.2 Metrics and losses

Comparison	Core difference	Choose the first when	Choose the second when
Precision vs. recall	purity of flagged set vs. coverage of positives	false positives are costly (spam, auto-actions)	misses are costly (disease, fraud losses, safety)
ROC-AUC vs. PR-AUC	ranking over both classes, prevalence-invariant vs. positive-class focus, prevalence-sensitive	balanced-ish classes; comparing across populations	rare positives; what matters is the flagged set's quality
MSE vs. MAE	squared (mean, outlier-sensitive, smooth) vs. absolute (median, robust, constant gradient)	large errors disproportionately bad; light tails	outliers/heavy tails; linear error costs
L1 vs. L2	constant pull → exact zeros vs. proportional pull → shrinkage	feature selection, sparse interpretable models	many small correlated effects; stability; prediction

29.3 Optimization and deep learning

Comparison	Core difference	Choose the first when	Choose the second when
Batch GD vs. SGD vs. mini-batch	exact gradient vs. one-example noisy gradient vs. averaged small batch	small convex problems (or L-BFGS)	SGD: streaming/online learning; mini-batch: almost all deep learning (throughput + noise)
Adam vs. SGD	per-parameter adaptive step sizes + momentum vs. one global step size (+ momentum)	transformers, sparse/ heterogeneous gradients, fast robust convergence, little tuning	well-tuned vision CNNs where SGD+momentum+schedule may generalize slightly better
BatchNorm vs. LayerNorm	normalize across the batch per feature vs. across features per example	CNNs with reasonably large IID batches	transformers, RNNs, small/variable batches, identical train/inference behavior

29.4 Systems

Comparison	Core difference	Choose the first when	Choose the second when
Online vs. batch learning	update the model continuously from streaming data vs. retrain periodically on accumulated data	fast-changing environments (ads, news, fraud), huge streams	stable domains; need for validation gates, reproducibility, simpler ops
Online vs. batch inference	score on request vs. precompute on a schedule	fresh context matters; unpredictable entities	predictable entity sets; heavy models; cost efficiency

PART 30

Trick and Follow-up Chains

How to survive 3–5 layers of "why?"

Each layer should go **one level more fundamental**: slogan → mechanism → math → assumption → where it breaks. If you get stuck, say what you *do* know and reason from first principles out loud ("variance of an average... that's a sum of variances and covariances..."). Interviewers reward the reasoning more than the recall. In each chain below, **C** = candidate, **I** = interviewer, and **Bedrock** = the fundamental fact the chain was digging for.

The whole Part as one picture

Think of it like peeling an onion in front of the interviewer. The first layer is the slogan everyone knows ("random forests reduce overfitting"). Each "why?" peels a layer — mechanism, then maths, then the edge case — until you reach the core fact that can't be peeled further (the **Bedrock** line). Practise each chain out loud until you can go all the way down without stopping.

Where it breaks: a real interviewer may peel in a different order or jump sideways; learn the bedrock facts so you can rebuild any chain, not recite these ones.

Chain 1 — "Random forests reduce overfitting."

C: Random forests reduce overfitting.

I: Why?

C: They average many trees trained on bootstrap samples.

I: Why does averaging reduce overfitting?

C: Overfitting here is variance — each deep tree's prediction depends heavily on its sample. Averaging keeps the expected prediction (bias unchanged) and averages the sample-specific fluctuations, which partly cancel.

I: "Partly"? Why not completely?

C: Because the trees are correlated — trained on overlapping data. For B models with variance σ^2 and pairwise correlation ρ, the average has variance $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. Only the second term vanishes with more trees.

I: So what does the random forest do about ρ?

C: Random feature subsets at each split stop every tree from splitting on the same dominant features, so trees differ more — lower ρ — at the price of each tree being a bit worse. The product usually wins.

I: Then why not set max_features = 1?

C: Then each tree is nearly random — bias and individual variance rise too much; the optimum trades correlation against individual strength, which is why it's tuned.

Bedrock: $\mathrm{Var}(\bar{X}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ — averaging only removes the *uncorrelated* part of the error, and never touches bias.

Chain 2 — "L1 regularization gives sparse models."

C: L1 gives sparse models.

I: Why?

C: The L1 constraint region is a diamond with corners on the axes; the loss contours tend to touch it at a corner.

I: Why does touching a corner matter, and why would contours hit corners rather than edges?

C: At a corner some coordinates are exactly zero. The diamond's corners "stick out" — in high dimensions most of the boundary region near the optimum is at low-dimensional faces where many coordinates are zero.

I: Give me a non-geometric argument.

C: Look at the force on a weight. L2's gradient $2\lambda w$ shrinks as $w \to 0$; L1's subgradient has constant size λ. In 1-D, $\min \frac{1}{2}(w-\rho)^2 + \lambda|w|$ gives $\operatorname{sign}(\rho)\max(|\rho| - \lambda, 0)$: exactly zero whenever the data's pull $|\rho|$ is below λ.

I: What happens with two perfectly correlated features?

C: The penalty $|w_1| + |w_2|$ is constant along the segment with a fixed sum, so the solution is not unique — solvers pick one feature arbitrarily. Elastic net's L2 term makes it strictly convex and splits the weight.

Bedrock: a non-differentiable penalty at 0 has a subgradient interval $[-\lambda, \lambda]$ that can absorb any data gradient smaller than λ.

Chain 3 — "Use cross-entropy, not MSE, for classification."

C: Cross-entropy is better for classification.

I: Why?

C: It's the negative log-likelihood of a Bernoulli/categorical model, so minimizing it is maximum likelihood.

I: MSE is also a likelihood — Gaussian. Why is that worse here?

C: The noise model is wrong — labels are 0/1, not Gaussian. And practically, with a sigmoid output, the MSE gradient is $2(p-y)p(1-p)$: when the model is confidently wrong ($p \approx 0$, $y = 1$), $p(1-p) \approx 0$ and the gradient vanishes. Cross-entropy's gradient is $p - y$ — largest exactly then.

I: Where does the $p(1-p)$ go in cross-entropy?

C: $\partial \ell / \partial p = -\frac{y}{p} + \frac{1-y}{1-p}$ has $p(1-p)$ in the denominator, which cancels $\sigma'(z) = p(1-p)$.

I: Anything else cross-entropy guarantees?

C: It's a strictly proper scoring rule: expected loss is minimized by the true probability, so it encourages calibration — and for linear models it's convex.

I: When is cross-entropy a bad choice?

C: With noisy labels (unbounded loss on mislabeled points), when a model's probabilities will be overconfident anyway, or when the business metric is about top-k ranking — then ranking losses align better.

Bedrock: canonical-link GLMs have gradient $(\text{prediction} - \text{target}) \times \text{input}$; the likelihood choice encodes the noise model.

Chain 4 — "Adam converges faster than SGD."

C: Adam converges faster.

I: Why?

C: It adapts the learning rate per parameter.

I: Why does that speed things up?

C: Loss surfaces are ill-conditioned: some directions have much larger gradients/curvature than others. One global step size must be small enough for the steepest direction, so flat directions crawl. Dividing by the RMS of each parameter's gradient equalizes step sizes — a diagonal preconditioner.

I: Then why use SGD at all?

C: In some settings — CNNs for image classification — SGD with momentum generalizes slightly better. The optimizers find different solutions.

I: Why would the solution differ if both minimize the same loss?

C: Over-parameterized problems have many minimizers; the optimizer's implicit bias picks one. For linear least squares, GD from zero converges to the minimum-L2-norm solution because updates stay in the row space of X. Adaptive rescaling changes the geometry, so Adam's limit can have a larger norm. SGD's noise may also favor flatter regions. And Adam+L2 regularizes unevenly — AdamW fixes that part.

Bedrock: GD updates are linear combinations of the data's rows, so the reachable solution set — and thus generalization — depends on the optimizer's geometry, not just the loss.

Chain 5 — "BatchNorm reduces internal covariate shift."

C: BatchNorm helps by reducing internal covariate shift.

I: What's internal covariate shift, and is that really why it works?

C: The changing distribution of layer inputs as earlier layers update. Experiments (Santurkar et al.) showed BN works even when you deliberately inject distribution shift after it — so the better explanation is that it smooths the loss landscape.

I: What does "smooths" mean concretely?

C: The gradient changes more slowly as parameters change (better Lipschitz/smoothness constants), so larger learning rates stay stable and the gradient direction stays informative.

I: Why does normalization do that?

C: Normalized layers are invariant to the scale of their incoming weights; activations can't blow up with large updates, and the effective learning rate adapts as $\eta/\|w\|^2$.

I: Where does BN break?

C: Small or correlated batches give noisy statistics; inference uses running averages that can mismatch; sequences and distributed training complicate it — hence LayerNorm in transformers.

Bedrock: step-size stability is set by curvature; normalization controls the scale that curvature depends on.

Chain 6 — "Use PR-AUC for imbalanced data."

C: With imbalance, use PR-AUC instead of ROC-AUC.

I: Why?

C: ROC-AUC can look great while precision is terrible.

I: How can a good ROC curve coexist with terrible precision?

C: ROC uses FPR, a rate within negatives. With 1,000 negatives per positive, an FPR of 1% means 10 false positives per positive — precision below 10% — yet on the ROC plot 1% FPR is nearly at the left edge.

I: Write precision in terms of TPR, FPR and prevalence.

C: $\text{precision} = \frac{\pi\,\text{TPR}}{\pi\,\text{TPR} + (1-\pi)\,\text{FPR}}$ — Bayes' rule. As $\pi \to 0$, precision collapses unless FPR is tiny.

I: So is ROC-AUC useless under imbalance?

C: No — it's prevalence-invariant, so it's the right tool to compare models across populations with different base rates, or when you care about ranking quality across the whole list. For operating decisions on rare positives, precision/recall at the threshold matters.

Bedrock: precision mixes the classifier's rates with the base rate via Bayes' rule; ROC doesn't.

Chain 7 — "More data fixes overfitting."

C: Get more data to fix overfitting.

I: Why does more data help?

C: Variance of estimates shrinks with n — roughly like $1/n$ for many estimators — so the model's fit depends less on the particular sample.

I: When doesn't more data help?

C: When the problem is bias (the model can't represent the pattern), when we're at the Bayes error (noise-limited), when the new data is from a different distribution than production, or when it's duplicated/highly correlated with existing data — effective sample size barely grows.

I: How would you know in advance?

C: A learning curve: validation error vs. training-set size. If it's still falling steeply, data helps; if flat and close to training error, it won't.

I: Could more data ever *hurt*?

C: If it's mislabeled or biased, yes; and near the interpolation threshold of over-parameterized models, sample-wise double descent can temporarily increase test error.

Bedrock: estimation error shrinks with *effective, representative* sample size; approximation error and Bayes error don't.

Chain 8 — "PCA uses eigenvectors of the covariance matrix."

C: PCA takes the top eigenvectors of the covariance matrix.

I: Why eigenvectors?

C: We want the unit direction maximizing projected variance, $u^\top \Sigma u$. The Lagrangian gives $\Sigma u = \lambda u$, and the attained variance is λ.

I: Why is variance the right thing to maximize?

C: Equivalent to minimizing squared reconstruction error (Pythagoras: total variance = kept + lost). High-variance directions preserve most of the differences between points.

I: When is that the wrong objective?

C: When the signal for the task lives in low-variance directions — PCA is unsupervised; when features have different units (variance reflects units, so standardize); when there are outliers (variance is squared); when structure is nonlinear.

I: How would you compute it for 10M × 1,000 data?

C: Randomized SVD or incremental PCA on centered data — no need to form a full decomposition; or the $1{,}000 \times 1{,}000$ covariance via a streaming pass, then eigendecompose it.

Bedrock: the Rayleigh quotient $u^\top \Sigma u / u^\top u$ is maximized by the top eigenvector of a symmetric matrix.

Chain 9 — "Residual connections fix vanishing gradients."

C: ResNets work because skip connections fix vanishing gradients.

I: How exactly?

C: $h_{l+1} = h_l + F(h_l)$, so the Jacobian is $I + \partial F/\partial h$; the backward product always contains an identity path.

I: But plain deep networks with BatchNorm and He init don't have vanishing gradients, and they still trained worse. Why?

C: Right — the original ResNet paper's degradation problem: deeper plain nets had *higher training error*, not just test error, even with normalization. It's an optimization difficulty beyond gradient magnitude: learning an identity mapping through stacks of nonlinear layers is hard, while with residuals the default is identity and blocks learn small corrections.

I: How would you verify that's an optimization problem and not overfitting?

C: Training loss — overfitting would show lower training loss; degradation shows higher training loss for the deeper model.

Bedrock: reparameterizing a block as identity + residual changes the optimization landscape so that "do nothing" is the easy starting point.

Chain 10 — "Use 10-fold cross-validation."

C: I'd use 10-fold cross-validation.

I: Why not leave-one-out — isn't more training data per fold better?

C: LOO's estimate is nearly unbiased but high-variance: the n models are trained on almost identical data, so their errors are highly correlated, and the average of correlated terms doesn't shrink much. It's also n times the cost.

I: Is 10-fold biased?

C: Slightly pessimistic — each model sees 90% of the data. Usually acceptable.

I: Your data is daily sales over 3 years. Still 10-fold?

C: No — random folds would train on the future. Rolling-origin backtesting with a gap equal to the forecast horizon.

I: And if there are 500 stores with 1,000 days each?

C: Decide based on production: forecasting known stores → time split; new stores → group split by store (and time).

Bedrock: a validation scheme is a simulation of deployment; its variance is governed by the correlation among the fold estimates.

Chain 11 — "Logistic regression is a linear classifier."

C: Logistic regression gives a linear decision boundary.

I: Then how would you fit a circular boundary?

C: Add features x_1^2, x_2^2 (or $x_1 x_2$): the boundary is linear in the expanded space, circular in the original.

I: Why not always add all polynomial features up to degree 5?

C: The number of features grows combinatorially — $\binom{d+5}{5}$ — raising variance and cost and making the problem ill-conditioned; you'd need strong regularization.

I: Is there a way to use infinitely many such features?

C: Kernels: models that depend on data only via dot products can use $K(x, x') = \phi(x)^\top \phi(x')$ — kernel logistic regression or SVMs with RBF kernels — at $O(n^2)$ cost. Or learn the features: a neural network is logistic regression on learned features.

Bedrock: "linear" refers to the parameters; expressiveness comes from the feature map, fixed (kernels) or learned (neural nets).

Chain 12 — "I'd check feature importance to find leakage."

C: If metrics look too good, I check feature importance for leakage.

I: Why would a leaked feature show up as important?

C: The optimizer takes the easiest route to lower loss; a leaked feature is almost a deterministic function of the label, so it explains the most loss and the model relies on it heavily.

I: What if the leak is spread across many features, none dominant?

C: Then importance won't flag it. I'd use structural tests: re-split by time and by entity, deduplicate, refit all preprocessing inside folds, and check whether performance collapses.

I: Give me one more test that detects leakage without knowing where it is.

C: Train on shuffled labels with the full pipeline — performance should drop to chance. If it doesn't, the pipeline leaks label information (for example target encoding computed before splitting). Also: evaluate on a truly later, never-touched period.

Bedrock: any path from the label into the features, or from test into training, will be exploited because it lowers loss.

Chain 13 — "The A/B test is significant at p = 0.01, so the effect is real."

C: p = 0.01, so the effect is real.

I: What does p = 0.01 mean exactly?

C: If there were no effect, data at least this extreme would occur 1% of the time. It's not the probability that there's no effect.

I: So how could it still be a false positive?

C: Multiple comparisons (many metrics/segments), peeking and stopping early, a low prior probability of real effects (most ideas don't work, so a large share of "significant" results are false), or broken randomization (SRM).

I: Suppose it's real. Is the measured effect size trustworthy?

C: If the test was underpowered, conditional on significance the estimate is inflated (winner's curse). Also novelty effects can fade.

I: What would you do before shipping?

C: Check SRM and guardrails, confirm pre-registration, look at the CI rather than the p-value, check consistency across time and segments, and possibly replicate or keep a holdback to measure the long-run effect.

Bedrock: $P(\text{data} \mid H_0) \neq P(H_0 \mid \text{data})$; the latter depends on priors and power.

Chain 14 — "Gradient boosting reduces bias."

C: Boosting reduces bias.

I: How?

C: Each tree fits the negative gradient of the loss at the current predictions — for squared loss, the residuals — so the ensemble moves toward the targets like gradient descent in function space, and the sum of trees can represent far more than one tree.

I: Then why doesn't it always overfit?

C: It does eventually. Shrinkage (small learning rate), subsampling, shallow trees and early stopping on validation keep variance in check.

I: Why does a smaller learning rate help, if you just need more trees to compensate?

C: Small steps trace a smoother path through function space — similar to how small-step gradient descent with early stopping acts like L2 regularization; each tree's noise-fitting is diluted and later trees can correct earlier ones. Empirically, lower ν with more trees reaches lower validation loss.

Bedrock: boosting is stagewise functional gradient descent; the learning rate and number of steps form a regularization path.

Chain 15 — "k-means converges."

C: k-means always converges.

I: Why?

C: Both steps can only lower (or keep) the objective — assignment picks the nearest centroid, the update picks the mean, which minimizes squared distances — and there are finitely many partitions, so it can't decrease forever.

I: Converges to what?

C: A local minimum (a fixed point of the two steps), not necessarily the global one; the global problem is NP-hard.

I: How do you get a good local minimum?

C: k-means++: sample initial centroids with probability proportional to squared distance from existing ones, giving an $O(\log k)$ approximation in expectation, plus multiple restarts keeping the lowest inertia.

I: Would a different distance metric keep the guarantee?

C: Not with the mean update — the mean minimizes squared Euclidean distance specifically. For other distances you need a different center (median for L1 → k-medians; medoids for arbitrary distances → k-medoids).

Bedrock: coordinate descent on a bounded objective over a finite set of states must terminate — at a local optimum.

Chain 16 — "Dropout prevents overfitting."

C: Dropout prevents overfitting.

I: Why?

C: Randomly dropping units prevents co-adaptation — each unit must be useful on its own, with random partners.

I: Any other interpretation?

C: Training an exponential number of thinned sub-networks with shared weights; inference with all units (scaled) approximates averaging them — an ensemble.

I: Why scale activations by $1/(1-p)$ during training?

C: So each unit's expected output is the same in training and inference; otherwise the next layer would see systematically larger inputs at test time.

I: Why is dropout often dropped in modern large-scale training?

C: With huge datasets and few epochs, variance is less of a problem; dropout slows convergence; normalization and weight decay provide regularization; and it interacts badly with BatchNorm's statistics.

Bedrock: injecting multiplicative noise forces robust, redundant representations and approximates model averaging.

Chain 17 — "RAG fixes hallucination."

C: RAG fixes hallucination — the model answers from retrieved documents.

I: Does it? What if the right document isn't retrieved?

C: Then the model may answer from its own memory or stitch together irrelevant passages — confidently. RAG moves the problem into retrieval; you must measure recall@k and make the model abstain when support is weak.

I: And if the right passage *is* retrieved?

C: It can still be unfaithful: long noisy context, the key sentence buried in the middle, or the model preferring its prior over the passage. You reduce that with reranking and trimming, citation requirements, and a faithfulness check.

I: How would you know which failure you have?

C: Label a few hundred failures: was the supporting passage in the top-k? That splits errors into retrieval misses and generation errors, each with its own fixes and metric.

I: Could retrieval make things *worse*?

C: Yes — outdated or wrong documents get cited with authority, and malicious documents can inject instructions. You need freshness, source authority and injection defenses.

Bedrock: RAG converts a knowledge problem into a search problem plus a reading problem; hallucination falls only if both are measured and fixed.

Chain 18 — "LoRA is as good as full fine-tuning."

C: LoRA matches full fine-tuning with far fewer parameters.

I: Why would a low-rank update be enough?

C: Adapting a pretrained model to a task seems to need changes in a low-dimensional subspace — mostly behaviour and format, not new knowledge. A rank-8 update on a 4096×4096 matrix has 65K parameters instead of 16.8M.

I: When does it fall short?

C: When the task needs a lot of new knowledge or a big domain shift (new language, large-scale continued pretraining), or when rank is too low or adapters are placed on too few layers. Then full fine-tuning or higher ranks close the gap.

I: What does it buy you besides memory?

C: Less forgetting (the base weights are frozen), tiny adapter files, and serving many adapters on one base model.

Bedrock: LoRA restricts the update to $\Delta W = BA$ of rank r — enough for behaviour changes, not always for knowledge-heavy shifts; validate against full fine-tuning when the stakes are high.

Chain 19 — "We'll evaluate the chatbot with an LLM judge."

C: We'll use an LLM judge to score answers.

I: How do you know the judge is right?

C: I'd label a few hundred examples with humans and measure agreement — Cohen's κ or accuracy against the human majority — per slice.

I: The judge agrees with humans 75% of the time. Good?

C: Depends on human–human agreement. If two humans agree 78% of the time, the judge is near the ceiling; if humans agree 95%, the judge is weak.

I: What biases would you check?

C: Position (swap answer order), verbosity (length-controlled comparisons), self-preference (use another model family), and style over substance (reference answers for factual items).

I: And over time?

C: Pin the judge version and re-validate after any change to the judge, rubric or product — otherwise score changes may be judge drift, not product changes.

Bedrock: A judge is a model with errors; it's a measurement instrument that must be calibrated against humans before its numbers mean anything.

Chain 20 — "Quantize the model to 4-bit to make it faster."

C: 4-bit quantization makes inference faster.

I: Why would fewer bits make it faster?

C: Decoding is memory-bandwidth-bound: each token reads all weights. 4-bit weights are a quarter of fp16's bytes, so fewer bytes move per token.

I: Is it faster for every workload?

C: No — for large-batch or prefill-heavy work, compute dominates, and dequantization overhead can eat the gain unless kernels are optimized. The biggest wins are small-batch decoding and fitting a model on fewer GPUs.

I: What's the risk?

C: Quality loss, often concentrated on long contexts, maths, rare languages — slices headline benchmarks don't cover. Activation outliers make activation quantization harder than weight-only.

Bedrock: Quantization trades precision for bytes; it speeds up what is bound by bytes, and must be evaluated on your own slices.

Chain 21 — "A longer context window makes the model better."

C: Longer context lets the model use more information, so it's better.

I: Does the model use all of it equally?

C: No — recall of details in very long contexts is uneven; information in the middle is often used less well.

I: What does long context cost?

C: Prefill compute grows with length (attention is quadratic), the KV cache grows linearly, so latency and cost per request rise and fewer requests fit per GPU.

I: So when is it worth it?

C: When the task needs whole-document reasoning or the corpus is small and static (with prefix caching); for large, changing corpora, retrieval plus a moderate context is usually better and cheaper.

Bedrock: Context is a budget of compute, memory and attention quality — spend it on the most relevant tokens.

Chain 22 — "RLHF aligns the model with human preferences."

C: RLHF makes the model do what humans prefer.

I: What exactly is optimized?

C: A learned reward model's score, minus a KL penalty to the supervised model: $\max \mathbb{E}[r] - \beta \mathrm{KL}(\pi \| \pi_{\mathrm{ref}})$.

I: So it optimizes a proxy. What goes wrong?

C: Reward hacking — longer, more flattering, more confident answers that the reward model likes but humans don't — and preferences that are noisy or inconsistent across annotators.

I: Why the KL penalty?

C: It keeps the policy close to the reference so it can't drift into regions where the reward model is wrong; too small a β → hacking, too large → little change.

I: How would you check real alignment gains?

C: Held-out human pairwise evaluations, length-controlled comparisons, sycophancy and refusal tests, and online feedback — not the reward model's own score.

Bedrock: RLHF optimizes a learned proxy for preferences; Goodhart's law applies, so regularization and independent evaluation are part of the method.

PART 31

Derivations You Must Know

The five-step whiteboard format

INTUITION (one sentence you say first) → **ASSUMPTIONS** (write them in the corner of the board) → **DERIVATION** (few, clean lines — say each step) → **FINAL EQUATION** (box it) → **HOW TO EXPLAIN IT** (what the result means and how to sanity-check it). Practice each until you can do it in under 3 minutes without notes.

The whole Part as one picture

Think of it like learning a few classic recipes by heart — dal, rice, roti — rather than memorizing a thousand dishes. Almost every derivation here follows the same recipe: *write down what you believe about the data* (a probability model), *take the log* (turn products into sums), *differentiate* (find the slope), *set it to zero or step downhill*. Once you can cook that recipe on a whiteboard, most "derive X" questions become variations.

Where it breaks: recipes tolerate improvisation; derivations don't — one sign error or a missing transpose changes the answer, so practise writing them out fully, slowly, several times.

31.1 Linear regression loss (from maximum likelihood)

Squared error is Gaussian MLE

Intuition: squared error isn't arbitrary — it's what you get if you believe errors are Gaussian.

Assumptions: $y_i = w^\top x_i + \varepsilon_i$; $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ IID; x fixed.

Derivation: $p(y_i \mid x_i, w) = \frac{1}{\sqrt{2\pi}\sigma}\exp\!\big(-\frac{(y_i - w^\top x_i)^2}{2\sigma^2}\big)$. Independence ⇒ product; take $-\log$:

$$-\log L(w) = \frac{n}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_i (y_i - w^\top x_i)^2$$

Final: $\hat{w}_{\text{MLE}} = \arg\min_w \sum_i (y_i - w^\top x_i)^2$.

Explain it: "The constant doesn't depend on w, so maximizing likelihood is minimizing SSE. Swap in Laplace noise and you get MAE — the loss encodes the noise model."

The idea in plain words: a darts player's random wobble

Think of it like assuming a darts player aims at the right spot and misses by random, bell-shaped wobbles. If that's true, the aim that makes the observed throws *most likely* is the one that minimizes the sum of squared misses.

Each step, in words: first, assume each answer = straight-line prediction + bell-curve noise. Second, write the probability of all observed answers as a product. Third, take the log, which turns the product into a sum. Fourth, notice everything except "sum of squared errors" is a constant — so maximizing likelihood = minimizing squared error.

Where it breaks: if the misses aren't bell-shaped (heavy tails, outliers), squared error is no longer the "most likely" choice — MAE or Huber fits better.

31.2 The normal equation

Setting the gradient to zero

Intuition: the best fit leaves residuals orthogonal to every feature.

Assumptions: $X \in \mathbb{R}^{n \times d}$ has full column rank.

Derivation: $L = \|y - Xw\|^2 = y^\top y - 2w^\top X^\top y + w^\top X^\top X w$; $\nabla_w L = -2X^\top y + 2X^\top X w = 0$.

Final: $\boxed{\hat{w} = (X^\top X)^{-1} X^\top y}$; Hessian $2X^\top X \succ 0$ ⇒ unique minimum.

Explain it: "Rearranged: $X^\top(y - X\hat{w}) = 0$ — residual ⊥ column space, so $\hat{y}$ is the projection of y. If $X^\top X$ is singular (collinearity, $d > n$) use the pseudo-inverse or ridge; in code use `solve` / `lstsq`, never `inv`."

The idea in plain words: the flat spot at the bottom of a bowl

Think of it like finding the lowest point of a smooth bowl by looking for the place where the floor is perfectly flat. The squared-error loss is a bowl; at its bottom, the slope in every direction is zero. Setting the slope to zero gives one equation you can solve directly.

Each step, in words: write the loss as "prediction errors squared, added up"; take its slope with respect to the weights; set the slope to zero; rearrange to get "(features × features) × weights = features × answers"; solve for the weights.

Where it breaks: if two features are copies of each other, the bowl has a flat valley floor instead of one lowest point — the equation has no unique answer (hence ridge).

31.3 Gradient of MSE

Vector calculus in two lines

Intuition: each residual pulls the weights along its input, in proportion to its size.

Assumptions: $L = \frac{1}{n}\|Xw - y\|^2$.

Derivation: let $r = Xw - y$; $L = \frac{1}{n} r^\top r$; $\frac{\partial L}{\partial r} = \frac{2}{n} r$; $\frac{\partial r}{\partial w} = X$ ⇒ chain rule.

Final: $\boxed{\nabla_w L = \frac{2}{n} X^\top (Xw - y)}$; GD step $w \leftarrow w - \eta \frac{2}{n} X^\top (Xw - y)$.

Explain it: "Shape check: $X^\top$ is $d \times n$, residual $n \times 1$ → $d \times 1$ like w. It's a sum over examples of residual × input."



The idea in plain words: blame shared by each feature

Think of it like a teacher returning marks to a group project: each student's share of the blame depends on how much they contributed to the wrong answer. The gradient gives each weight a share of the total error, proportional to how much its feature was "switched on" in the mistaken predictions.

Each step, in words: errors = predictions − answers; each weight's blame = sum over examples of (its feature value × that example's error); multiply by 2 (from the square) and divide by n if you average.

Where it breaks: the blame is only for small changes — after you adjust the weights, the errors change, and the blame must be recomputed.

31.4 Logistic regression likelihood

Bernoulli likelihood with a logit link

Intuition: each label is a biased coin flip whose bias depends on the features through the log-odds.

Assumptions: $y_i \in \{0, 1\}$ independent; $P(y_i = 1 \mid x_i) = \sigma(w^\top x_i)$.

Derivation: $P(y_i \mid x_i) = p_i^{y_i}(1 - p_i)^{1-y_i}$ ⇒

$$\log L(w) = \sum_i \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]$$

Final: maximize $\log L$ ⇔ minimize binary cross-entropy (next).

Explain it: "The exponent trick selects p_i when $y = 1$ and $1 - p_i$ when $y = 0$ — one formula for both cases."



The idea in plain words: coin tosses with a loaded coin per person

Think of it like every customer carrying their own coin whose bias (chance of "buy") depends on their features. The model's job is to choose the rule for setting each coin's bias so that the actual buy/no-buy outcomes look as likely as possible.

Each step, in words: turn a weighted score into a probability with the sigmoid; the chance of each observed outcome is p if they bought and $1 - p$ if not; multiply over customers; take the log; flip the sign — that's the loss to minimize.

Where it breaks: real customers influence each other (friends, trends); the derivation assumes each coin is tossed independently.

31.5 Binary cross-entropy and its gradient

From likelihood to p − y

Intuition: penalize the log of the probability assigned to what actually happened.

Assumptions: as above; $z = w^\top x$, $p = \sigma(z)$, $\sigma' = \sigma(1-\sigma)$.

Derivation: $\ell = -[y \log p + (1-y)\log(1-p)]$. $\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p} = \frac{p-y}{p(1-p)}$. Multiply by $\frac{\partial p}{\partial z} = p(1-p)$:

$$\frac{\partial \ell}{\partial z} = p - y, \qquad \nabla_w \mathcal{L} = \frac{1}{n} X^\top (p - y)$$

Final: $\boxed{\mathcal{L} = -\frac{1}{n}\sum [y \log p + (1-y)\log(1-p)], \quad \nabla = \frac{1}{n} X^\top (p-y)}$.

Explain it: "The sigmoid's derivative cancels — that's why cross-entropy doesn't saturate when confidently wrong. Implement as $\log(1+e^z) - yz$ for stability."

The idea in plain words: "prediction minus truth"

Think of it like a thermostat: if the room is 2° too warm, it adjusts by an amount proportional to "current – target". Logistic regression's gradient is exactly that simple — the sigmoid's slope cancels neatly with the log's slope, leaving "predicted probability – actual label".

Each step, in words: differentiate the log-loss with respect to the score; use the fact that the sigmoid's derivative is $p(1-p)$; watch the terms cancel; get $p - y$; multiply by the input features for the weight gradient.

Where it breaks: the neat cancellation only happens because the sigmoid is paired with cross-entropy; pair it with squared error and the gradient gets an extra $p(1-p)$ factor that can vanish.

31.6 Softmax

Normalized exponentials and their Jacobian

Intuition: turn arbitrary scores into a probability distribution that preserves their order and is smooth.

Assumptions: logits $z \in \mathbb{R}^K$.

Derivation: $p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$. For $\partial p_k / \partial z_j$: quotient rule gives $p_k \delta_{kj} - p_k p_j$.

Final: $\boxed{\frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j)}$; shift invariance $\text{softmax}(z + c) = \text{softmax}(z)$.

Explain it: "Increasing one logit raises its probability by $p_k(1-p_k)$ and lowers every other by $p_k p_j$ — mass is conserved. Subtract $\max z$ before exponentiating. $K = 2$ reduces to $\sigma(z_1 - z_0)$."

The idea in plain words: dividing a sweet box by popularity

Think of it like sharing a box of 100 laddoos among children according to how loudly they cheer — but loudness counts exponentially, so the loudest gets a big share, the rest get a little, and the shares always add up to 100. Softmax turns any set of scores into shares that sum to 1.

Each step, in words: exponentiate each score (makes everything positive and exaggerates differences); divide by the total (makes shares sum to 1); the derivative of each share depends on its own size and every other share, because raising one child's share lowers everyone else's.

Where it breaks: laddoos are whole; softmax never gives anyone exactly zero, and adding a constant to all scores changes nothing.

31.7 Cross-entropy (multiclass) and its gradient

THE GRADIENT IS AGAIN PREDICTION − TARGET

Intuition: the loss is minus the log-probability of the correct class.

Assumptions: one-hot y; $p = \mathrm{softmax}(z)$.

Derivation: $\ell = -\sum_k y_k \log p_k = -z_c + \log \sum_j e^{z_j}$ (for true class c). Differentiate: $\frac{\partial \ell}{\partial z_j} = -\mathbb{1}[j = c] + \frac{e^{z_j}}{\sum e^z}$.

Final: $\boxed{\nabla_z \ell = p - y}$; for a linear layer $z = Wx$: $\nabla_W \ell = (p - y)x^\top$.

Explain it: "Cross-entropy $= H(y) + \mathrm{KL}(y \| p)$; with a one-hot target, $H(y) = 0$, so we're minimizing KL — i.e. maximum likelihood."



The idea in plain words: move chips toward the winner

Think of it like correcting a bet after a race: take chips away from the horses that lost (by the amount you bet on them) and add chips to the winner (by the amount you *didn't* bet on it). Softmax + cross-entropy gives exactly this update: "predicted share − true share" for every class.

Each step, in words: write the loss as minus the log of the winner's share; substitute the softmax formula; differentiate; the messy softmax Jacobian collapses to $p_k - \mathbb{1}[k = y]$.

Where it breaks: with mislabeled data, the "winner" is wrong, and the update confidently moves chips toward the wrong horse.

31.8 Entropy

WHY −Σ P LOG P, AND WHY IT'S MAXIMAL WHEN UNIFORM

Intuition: the average surprise; how many yes/no questions you need on average to identify the outcome.

Assumptions: information of an event $I(p)$ should be decreasing in p and **additive for independent events**: $I(pq) = I(p) + I(q) \Rightarrow I(p) = -\log p$.

Derivation: $H = \mathbb{E}[I] = -\sum_k p_k \log p_k$. Maximize subject to $\sum p_k = 1$: Lagrangian derivative $-\log p_k - 1 - \lambda = 0 \Rightarrow$ all p_k equal $\Rightarrow p_k = 1/K$.

Final: $\boxed{H(p) = -\sum_k p_k \log_2 p_k, \quad 0 \le H \le \log_2 K}$.

Explain it: "Fair coin: 1 bit. 90/10 coin: 0.47 bits — less uncertain. Pure node: 0. That's why trees use it as impurity."

The idea in plain words: the game of twenty questions

Think of it like playing twenty questions to guess a friend's pick. If the friend picks uniformly from 8 items, you need 3 yes/no questions on average — the maximum uncertainty. If they almost always pick the same item, you need nearly zero questions. Entropy counts the average number of yes/no questions an ideal player needs.

Each step, in words: the surprise of an outcome is $-\log p$; average the surprise over outcomes, weighted by their probability; to find the most uncertain distribution, maximize this subject to probabilities summing to 1 — the answer is uniform.

Where it breaks: real people can't ask "half-questions"; entropy is an average and can be fractional (0.47 bits).

31.9 Information gain

Uncertainty removed by a split

Intuition: how much knowing which side of the split you're on tells you about the label.

Assumptions: node S split into children S_c.

Derivation: entropy before: $H(S)$. Expected entropy after learning the branch: $\sum_c \frac{|S_c|}{|S|} H(S_c) = H(Y \mid \text{branch})$.

Final: $$\boxed{\text{IG} = H(S) - \sum_c \frac{|S_c|}{|S|} H(S_c) = I(Y; \text{branch}) \geq 0}.$$

Explain it: "It's the mutual information between the split and the label. Example: (10+,10–) → (8+,2–), (2+,8–): IG = 1 − 0.722 = 0.278 bits. It's biased toward many-way splits — gain ratio corrects that."

The idea in plain words: a good first question in a quiz

Think of it like asking the best first question in a guessing game. "Is it an animal?" splits the possibilities into cleaner groups than "Does it start with M?". Information gain measures how much uncertainty a question (a tree split) removes on average.

Each step, in words: measure the uncertainty (entropy) before the split; measure the uncertainty in each child group; average the children's uncertainty weighted by group size; the gain is "before – after".

Where it breaks: questions with many possible answers (like "What's your customer ID?") look amazingly informative but don't generalize — hence gain ratio and limits on splits.

31.10 Ridge regression

Closed form and the shrinkage factor

Intuition: add a "keep weights small" term; the solution becomes unique and stable.

Assumptions: centered, standardized features; intercept not penalized.

Derivation: $L = \|y - Xw\|^2 + \lambda\|w\|^2$; $\nabla = -2X^\top(y - Xw) + 2\lambda w = 0 \Rightarrow (X^\top X + \lambda I)w = X^\top y$. With $X = U\Sigma V^\top$: $\hat{w} = \sum_j \frac{s_j}{s_j^2+\lambda}(u_j^\top y)v_j$.

Final: $\boxed{\hat{w}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y}$, shrinkage factor $\frac{s_j^2}{s_j^2+\lambda}$ per principal direction.

Explain it: "$X^\top X + \lambda I$ is always PD, so the solution exists even with collinearity or $d > n$. Low-variance directions — the noisiest — are shrunk most. Bayesian: Gaussian prior, $\lambda = \sigma^2/\tau^2$."



The idea in plain words: a spring pulling weights home

Think of it like attaching every weight to a spring anchored at zero. The data pulls weights toward values that fit; the springs pull them back home. Directions where the data is weak (low variance) are pulled back the most, because there the data's pull is feeble.

Each step, in words: add $\lambda\|\mathbf{w}\|^2$ to squared error; set the slope to zero; get $(X^\top X + \lambda I)\mathbf{w} = X^\top \mathbf{y}$; in the SVD view, each direction's weight is multiplied by $\frac{s^2}{s^2+\lambda}$ — near 1 for strong directions, near 0 for weak ones.

Where it breaks: real springs are the same for everyone; ridge's pull depends on feature scale, so features must be standardized first.

31.11 Lasso (intuition and the 1-D solution)

Soft thresholding

Intuition: a constant-strength pull toward zero that small effects can't resist.

Assumptions: one standardized feature (or coordinate descent with others fixed); ρ = OLS coefficient / correlation with the partial residual.

Derivation: minimize $\frac{1}{2}(w - \rho)^2 + \lambda|w|$. Optimality: $0 \in w - \rho + \lambda\,\partial|w|$ with $\partial|0| = [-1, 1]$. Cases: $w > 0 \Rightarrow w = \rho - \lambda$ (needs $\rho > \lambda$); $w < 0 \Rightarrow w = \rho + \lambda$ (needs $\rho < -\lambda$); else $w = 0$.

Final: $\boxed{\hat{w} = \text{sign}(\rho)\max(|\rho| - \lambda, 0)}$ (ridge: $\rho/(1+\lambda)$).

Explain it: draw the diamond vs. the circle with elliptical contours; point out the corner. "No closed form in general — coordinate descent applies this update feature by feature."



The idea in plain words: a minimum-order rule at a shop

Think of it like a wholesaler who only accepts orders above a minimum amount: small orders become zero, large orders go through but reduced by a fixed handling fee. Lasso's soft-thresholding does the same to each weight — small effects are set exactly to zero, large ones are shrunk by λ.

Each step, in words: with one feature, minimize squared error + $\lambda|w|$; the absolute value has a kink at zero; if the data's pull is weaker than λ, the best answer stays at zero; otherwise move toward the least-squares answer but stop λ short.

Where it breaks: with several correlated features, lasso's choice of which one to keep can be arbitrary and unstable.

31.12 PCA

Variance maximization → eigenproblem

Intuition: find the direction along which the data spreads out the most; repeat orthogonally.

Assumptions: centered data; $\Sigma = \frac{1}{n-1} X^\top X$; unit-norm directions.

Derivation: $\max_u u^\top \Sigma u$ s.t. $u^\top u = 1$; $\mathcal{L} = u^\top \Sigma u - \lambda(u^\top u - 1)$; $\nabla = 2\Sigma u - 2\lambda u = 0 \Rightarrow \Sigma u = \lambda u$; objective $= \lambda$.

Final: $\boxed{\text{PC}_k = k\text{-th eigenvector of } \Sigma,\ \text{variance} = \lambda_k}$; via SVD $X = USV^\top$: PCs $= V$, variances $= s_k^2/(n-1)$.

Explain it: "Same subspace minimizes reconstruction error (Pythagoras). Standardize if units differ; fit on train only; choose k by explained variance or downstream performance."



The idea in plain words: the longest stretch of a rubber sheet

Think of it like stretching a rubber sheet printed with your data points: some directions stretch more. PCA looks for the direction of greatest stretch (most variance), then the next greatest at right angles, and so on. Maths shows these special directions are exactly the eigenvectors of the covariance matrix.

Each step, in words: variance along a unit direction is $\mathbf{u}^\top \Sigma \mathbf{u}$; maximize it with the "unit length" rule attached by a Lagrange multiplier; set the slope to zero; get $\Sigma \mathbf{u} = \lambda \mathbf{u}$; the best direction is the one with the biggest λ.

Where it breaks: the sheet analogy suggests one natural stretch; with nearly equal eigenvalues, the directions are unstable and small data changes rotate them.

31.13 Backpropagation

Error signals flowing backwards

Intuition: the chain rule, organized so every intermediate result is computed once and reused.

Assumptions: layers $z_l = W_l a_{l-1} + b_l$, $a_l = \phi(z_l)$; loss L; define $\delta_l = \partial L / \partial z_l$.

Derivation: output (softmax/sigmoid + CE): $\delta_L = \hat{y} - y$. Weight gradient: z_l depends on W_l linearly with input $a_{l-1} \Rightarrow \partial L / \partial W_l = \delta_l a_{l-1}^\top$. Error to previous layer: $\partial L / \partial a_{l-1} = W_l^\top \delta_l$, then through the activation: $\delta_{l-1} = (W_l^\top \delta_l) \odot \phi'(z_{l-1})$.

Final: $\boxed{\delta_{l-1} = (W_l^\top \delta_l) \odot \phi'(z_{l-1}), \quad \frac{\partial L}{\partial W_l} = \delta_l a_{l-1}^\top, \quad \frac{\partial L}{\partial b_l} = \delta_l}$

Explain it: "Three moves: error × input for weights, transpose to go back, gate by the activation derivative. Cost ≈ 2× forward. This product of $W^\top$ and ϕ' over layers is exactly where vanishing/exploding gradients come from." (Numbers: Part 20.)

The idea in plain words: tracing blame back through a factory line

Think of it like a defective phone coming off an assembly line. Quality control asks the last station how much it contributed, and that station passes the blame back to the one before, in proportion to how much each input affected its output. Every station gets its share in one backward walk, instead of re-testing the line once per station.

Each step, in words: do a forward pass and remember intermediate values; start with the loss's slope at the output; at each layer, multiply the incoming blame by the layer's local slope to get blame for its weights and for its inputs; pass the input-blame backwards.

Where it breaks: factory stations are few; networks have many layers, and multiplying many local slopes can shrink or blow up the blame (vanishing/exploding gradients).

31.14 Gradient descent update

First-order Taylor + a curvature limit

Intuition: locally the loss is a plane; walk downhill, but not so far that the curvature bends you back up.

Assumptions: J differentiable with L-Lipschitz gradient.

Derivation: $J(\theta+\Delta) \approx J(\theta) + \nabla J^\top \Delta$ — minimized over $\|\Delta\| = \epsilon$ by $\Delta \propto -\nabla J$. With smoothness: $J(\theta - \eta g) \le J(\theta) - \eta\|g\|^2 + \frac{L\eta^2}{2}\|g\|^2$, which is a decrease whenever $\eta < 2/L$; best bound at $\eta = 1/L$: decrease $\ge \frac{1}{2L}\|g\|^2$.

Final: $\boxed{\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t), \quad 0 < \eta < 2/L}$.

Explain it: "On a quadratic, each eigen-direction contracts by $(1-\eta\lambda_i)$: diverges if $\eta\lambda_{max} > 2$, slow if λ_{min} is tiny — that ratio is the condition number."

The idea in plain words: steps sized to the road's bends

Think of it like driving on a road where you only see a few metres ahead. The slope tells you which way is downhill; how sharply the road bends (curvature) tells you how big a step is safe. On a gentle curve, big steps are fine; on hairpin bends, take small steps or you'll fly off.

Each step, in words: approximate the loss near the current point with a straight line (Taylor); the fastest decrease is opposite the gradient; add the curvature bound to learn that steps up to $1/L$ are guaranteed to decrease the loss.

Where it breaks: the curvature of a deep network changes from place to place, so a single "safe" step size doesn't exist — hence schedules and adaptive methods.

31.15 Bias–variance decomposition

Three sources of squared error

Intuition: your error = unavoidable noise + how wrong you are on average + how much you wobble between training sets.

Assumptions: $y = f(x) + \varepsilon$, $\mathbb{E}\varepsilon = 0$, $\mathrm{Var}\,\varepsilon = \sigma^2$, ε independent of the training set D; fixed x; $\bar{f} = \mathbb{E}_D \hat{f}_D(x)$; squared loss.

Derivation: $y - \hat{f} = (y - f) + (f - \bar{f}) + (\bar{f} - \hat{f})$. Square and take $\mathbb{E}_{D,\varepsilon}$. Cross terms: $\mathbb{E}[(y - f)(\cdot)] = \mathbb{E}[\varepsilon]\,\mathbb{E}[\cdot] = 0$ by independence; $\mathbb{E}_D[(f - \bar{f})(\bar{f} - \hat{f})] = (f - \bar{f})\mathbb{E}_D[\bar{f} - \hat{f}] = 0$.

Final: $\boxed{\mathbb{E}[(y - \hat{f})^2] = \sigma^2 + (f - \bar{f})^2 + \mathbb{E}_D[(\hat{f} - \bar{f})^2]}$.

Explain it: "Noise is irreducible; bias falls with capacity; variance rises with capacity and falls with data. Averaging over x gives the dataset-level version. It's specific to squared loss."



The idea in plain words: three reasons a shot misses

Think of it like splitting an archer's average miss into three parts: the wind (noise you can't control), a bent sight on the bow (bias: misses in the same direction every time), and shaky hands (variance: misses that scatter). Squared error splits cleanly into these three, because the cross terms average out to zero.

Each step, in words: add and subtract the average prediction; expand the square; the noise is independent of the model, so its cross term vanishes; the average of (prediction – average prediction) is zero, so that cross term vanishes too; three terms remain.

Where it breaks: the clean three-way split works for squared error only; for accuracy (0-1 loss) the pieces don't simply add.

31.16 Maximum likelihood estimation

The general recipe and two examples

Intuition: pick the parameters that make the observed data most probable.

Assumptions: IID data $x_1 .. x_n$ from $p(x \mid \theta)$.

Derivation: $\hat{\theta} = \arg\max \prod_i p(x_i \mid \theta) = \arg\max \sum_i \log p(x_i \mid \theta)$; set $\nabla_\theta = 0$ and check it's a maximum.

- Bernoulli: $\ell = k \log p + (n - k)\log(1 - p) \Rightarrow \frac{k}{p} - \frac{n-k}{1-p} = 0 \Rightarrow \hat{p} = k/n$.
- Gaussian: $\hat{\mu} = \bar{x}$, $\hat{\sigma}^2 = \frac{1}{n}\sum(x_i - \bar{x})^2$ (biased by factor $\frac{n-1}{n}$).
- Poisson: $\ell = \sum(x_i \log \lambda - \lambda)$ + const $\Rightarrow \hat{\lambda} = \bar{x}$.

Final: $\boxed{\hat{\theta}_{\text{MLE}} = \arg\min_\theta -\frac{1}{n}\sum_i \log p(x_i \mid \theta)}$ — the average NLL is the loss.

Explain it: "Logs turn products into sums (no underflow, SGD-friendly, same argmax). Adding a log-prior gives MAP = regularized loss. MLE is consistent and asymptotically efficient, but overfits small data (3/3 heads ⇒ $\hat{p} = 1$)."



The idea in plain words: the most believable explanation

Think of it like a detective choosing the explanation under which the clues are least surprising. If a coin shows 7 heads in 10 tosses, "the coin lands heads 70% of the time" makes the evidence more likely than any other setting. MLE picks parameter values that make the observed data most probable.

Each step, in words: write the probability of the data as a function of the parameter; take the log; differentiate; set to zero; solve (for the coin: $\hat{p}$ = heads / tosses).

Where it breaks: with little data MLE can be extreme (3 heads in 3 tosses ⇒ "always heads"); priors (MAP) or more data fix this.

31.17 Attention scaling by √ d (new in 2027)

WHY THE DOT PRODUCTS MUST BE DIVIDED BY √D_K

Intuition: keep attention logits at a scale where softmax is neither uniform nor one-hot.

Assumptions: query and key components are independent with mean 0 and variance 1 (roughly true at initialization with normalized inputs).

Derivation: $q^\top k = \sum_{j=1}^{d_k} q_j k_j$. Each term has mean 0 and variance $\mathbb{E}[q_j^2]\mathbb{E}[k_j^2] = 1$. Independent terms add variances: $\mathrm{Var}(q^\top k) = d_k$, so the SD is $\sqrt{d_k}$. Dividing by $\sqrt{d_k}$ gives variance 1 for every head size.

Why it matters: softmax of logits with SD 11 ($d_k = 128$) is nearly one-hot; its Jacobian $\mathrm{diag}(p) - pp^\top$ is then near zero, so gradients vanish.

Final: $\boxed{\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\big(QK^\top/\sqrt{d_k}\big)V}$

Explain it: "Without scaling, bigger heads mean bigger logits, saturated softmax and tiny gradients; √d_k normalizes the logit variance to one."

The idea in plain words: comparing scores from tests of different lengths

Think of it like comparing students' totals on tests with different numbers of questions: a 128-question test spreads totals much more widely than a 1-question test. Dividing by the square root of the number of questions puts every test on the same spread, so "how much better" means the same thing.

Each step, in words: a dot product adds up d_k random products; each has spread 1; adding d_k of them gives spread d_k (SD $\sqrt{d_k}$); divide by $\sqrt{d_k}$ to bring it back to 1.

Where it breaks: after training, queries and keys are not independent unit-variance vectors — the scaling is a good starting point, and some large models add query/key normalization for extra stability.

31.18 The DPO loss from the RLHF objective (new in 2027)

FROM "MAXIMIZE REWARD WITH A KL LEASH" TO A CLASSIFICATION LOSS

Intuition: the KL-regularized RLHF problem has a closed-form optimal policy; invert it to express the reward through the policy, then plug into the preference model.

Step 1 — optimal policy. $\max_\pi \mathbb{E}_{y\sim\pi}[r(x,y)] - \beta\,\mathrm{KL}(\pi\|\pi_{\mathrm{ref}})$ is solved by $\pi^*(y \mid x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y \mid x)\exp\big(r(x,y)/\beta\big)$.

Step 2 — invert. $r(x,y) = \beta\log\frac{\pi^*(y|x)}{\pi_{\mathrm{ref}}(y|x)} + \beta\log Z(x)$.

Step 3 — Bradley–Terry. $P(y_w \succ y_l \mid x) = \sigma\big(r(x,y_w) - r(x,y_l)\big)$ — the intractable $\beta\log Z(x)$ **cancels** in the difference.

Step 4 — maximum likelihood on preference pairs with the policy π_θ in place of π^*:

$$\boxed{\mathcal{L}_{\mathrm{DPO}} = -\mathbb{E}\Big[\log\sigma\Big(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\Big)\Big]}$$

Explain it: "The policy's log-ratio to the reference *is* an implicit reward, so we can fit preferences directly with a logistic loss — no reward model, no RL sampling."

The idea in plain words: skipping the middleman

Think of it like training a salesperson. The long way: hire a judge who learns what customers like (reward model), then coach the salesperson to please the judge (RL). DPO notices that the best salesperson's style can be described directly by how much more often they say the preferred thing than the old script did — so you coach from the customer's "this one is better" votes straight away.

Each step, in words: find the best policy for "high reward, not far from the reference"; rearrange so the reward is written in terms of that policy; in a comparison of two answers the awkward normalizer cancels; fit the comparisons with a logistic loss.

Where it breaks: the derivation assumes the Bradley–Terry model and good coverage of preference pairs; with offline, narrow data the policy can drift in ways the pairs never tested.

31.19 KV-cache memory and training compute (new in 2027)

Two formulas worth deriving on the spot

KV cache. Each layer stores a key and a value vector per KV head per token: 2 (K and V) × n_{layers} × n_{kv} × d_{head} numbers per token. Multiply by tokens T, batch b, and bytes per number:

$\boxed{\text{KV bytes} = 2\, n_{\text{layers}}\, n_{\text{kv}}\, d_{\text{head}}\, T\, b\, \text{bytes}}$ — e.g. $2 \cdot 32 \cdot 32 \cdot 128 \cdot 4096 \cdot 2 = 2$ GiB per 4K-token sequence (7B, fp16).

Training compute. A matrix multiply with an $m \times n$ weight on one token costs $2mn$ FLOPs (one multiply + one add per weight) → forward pass $\approx 2N$ FLOPs per token. Backward needs gradients with respect to activations **and** weights, each about the forward cost → $\approx 4N$. Total per token $\approx 6N$; over D tokens:

$\boxed{C_{\text{train}} \approx 6ND, \qquad C_{\text{infer}} \approx 2N \text{ per token}}$

LoRA parameters. A rank-r update to a $d_{\text{in}} \times d_{\text{out}}$ matrix has $r(d_{\text{in}} + d_{\text{out}})$ trainable parameters: $8 \times (4096 + 4096) = 65{,}536$ vs. $16{,}777{,}216$ — 0.4%.

Explain it: "Count what's stored or multiplied per token, then multiply by tokens; state the precision and the assumptions."

The idea in plain words: counting rice bags and cooking time

Think of it like a caterer's arithmetic. Storage: bags of rice per guest × number of guests × weight per bag (KV cache: numbers per token × tokens × bytes). Cooking: each dish takes one step to prepare and two more to clean up and restock (forward 2N, backward 4N). The skill is not memorizing totals but multiplying the right counts in the right order.

Each step, in words: list what is stored or computed per token; multiply by layers and heads; multiply by tokens and batch; multiply by bytes (memory) or by 2 FLOPs per weight (compute).

Where it breaks: real systems add overheads — activations, attention FLOPs at long context, padding, framework memory — so these formulas are lower bounds to be adjusted by 10–30%.

31.20 The delta method for ratio metrics (new in 2027)

Variance of a ratio of means

Setup: users $i = 1..n$ contribute X_i (clicks) and Y_i (impressions); the metric is $R = \bar{X}/\bar{Y}$.

Derivation: first-order Taylor expansion of $g(a,b) = a/b$ around (μ_X, μ_Y):
$g \approx \frac{\mu_X}{\mu_Y} + \frac{1}{\mu_Y}(\bar{X} - \mu_X) - \frac{\mu_X}{\mu_Y^2}(\bar{Y} - \mu_Y)$. Take the variance, using $\mathrm{Var}(\bar{X}) = \sigma_X^2/n$, $\mathrm{Var}(\bar{Y}) = \sigma_Y^2/n$, $\mathrm{Cov}(\bar{X}, \bar{Y}) = \sigma_{XY}/n$:

$$\boxed{\mathrm{Var}(R) \approx \frac{1}{n\mu_Y^2}\left(\sigma_X^2 - 2r\sigma_{XY} + r^2\sigma_Y^2\right), \quad r = \mu_X/\mu_Y}$$

Explain it: "Randomization is by user, so user is the independent unit; linearize the ratio and propagate the user-level variances and covariance. The naive binomial SE treats impressions as independent and is too small."

The idea in plain words: a wobbly fraction

Think of it like estimating a batting strike rate (runs ÷ balls) over a season. Both runs and balls wobble from match to match, and they wobble together (long innings have more of both). The delta method asks: if the top and bottom each wobble a little, how much does the fraction wobble? It uses the slope of the fraction with respect to each, plus how they move together.

Each step, in words: approximate the ratio by a straight line near the average values; the variance of a straight-line combination is easy (weights² × variances + cross term); plug in user-level variances.

Where it breaks: with few users or a few users with enormous denominators, the straight-line approximation is poor — bootstrap by user instead.

PART 32

Interview Mental Models

Why mental models beat memorized answers

An unfamiliar question is usually a familiar question wearing a costume. Mental models are the lenses that strip the costume off: run the question through three or four of them and a structured answer appears — even if you've never seen the question before.

The whole Part as one picture

Think of it like a doctor's set of standard checks — pulse, blood pressure, temperature, history — applied to every patient, whatever the complaint. Mental models are your standard checks for any ML question: bias vs. variance, what the model knows at prediction time, how the distribution can shift, what each error costs, what happens at 100× scale, and (new) the token budget and eval-first lenses. Run through them and the answer's structure appears on its own.

Where it breaks: checklists help you not miss things, but the interviewer wants the one or two lenses that matter most for *this* question — say which lens you're using and why.

32.1 The bias/variance lens

```
performance poor?
  |
  +-- training error high? ----------> BIAS (or bug / no signal)
  |       more capacity, better features, less regularization, train longer,
  |       check labels & pipeline, estimate Bayes error (human level)
  |
  +-- training low, validation high? -> VARIANCE (or split mismatch / leakage-in-train)
  |       more data, augmentation, regularization, simpler model, bagging,
  |       early stopping; verify validation matches training distribution
  |
  +-- both fine offline, bad online? --> DISTRIBUTION / PIPELINE (not bias/variance!)
          skew, drift, leakage, metric mismatch, feedback loops
```

Always ask the next question: *which* source of bias (features? model family? optimization?) and *which* source of variance (too few effective samples? too many selection decisions? unstable learner?).

32.2 Data → Objective → Model → Evaluation → Production

Use this to organize any open-ended question ("how would you improve X?", "why did Y fail?"):

Stage	Questions
Data	what's collected, how, from whom, when? labels: source, noise, delay, bias? coverage of production?
Objective	what exactly is being optimized, and does it match the business goal? loss vs. metric vs. value?
Model	does its inductive bias match the data structure? capacity vs. data size? constraints (latency, interpretability)?
Evaluation	does the split simulate deployment? right metric at the operating point? slices? error bars?

Stage	Questions
Production	same features and code at serving? drift, feedback loops, monitoring, retraining?

Most real problems live in the first two and the last stage; most candidates spend their answer on the middle one.

32.3 The proxy ladder

Business value → metric → loss → empirical loss on a sample → what the optimizer found. At every rung ask *"how could this proxy diverge from the rung above?"* (Part 1). This single lens answers "AUC improved but revenue didn't", "clickbait", "optimizing MAPE biases forecasts low", and "the model overfits the validation set".

32.4 The information-availability lens ("what does the model know at prediction time?")

For every feature and every evaluation: *at the moment of prediction, in production, would this exact value be known, computed the same way?* It exposes leakage, training–serving skew, label-window overlaps, and cold start ("for a new user, what's actually available?").

32.5 The distribution lens

Three distributions: **training**, **validation/test**, **production (now and later)**. Every failure question can be asked as "which two of these differ, and how?" Covariate shift, label shift, concept drift, sampling bias, selection bias, feedback loops, seasonality — all are specific differences between these three.

32.6 The signal-to-noise lens

How much information about y is actually in x? What's the Bayes error (human agreement, conflicting duplicates)? How noisy are labels? This tells you when model work is futile and data/feature/label work is required — and sets expectations ("0.70 AUC may be excellent here").

32.7 The decision-and-cost lens

A prediction is only valuable through the decision it drives. Ask: what action, what threshold, what does each error cost, what are the constraints (capacity, latency, regulation)? It turns metric questions into cost questions and threshold questions into expected-value questions.

32.8 The inductive-bias-matching lens

Data structure ↔ model assumption: tabular heterogeneous → trees; sparse high-dim text → linear; images → convolution; sequences/sets with interactions → attention; graphs → message passing; small data → strong priors or pretrained representations. Use it for every "which model?" question.

32.9 The 100× lens

What happens if data, users, items, QPS or label volume grows 100×? Compute (training time, $O(n^2)$ algorithms), memory (embedding tables, KNN), latency (per-candidate cost × candidates), label cost, monitoring volume (every test becomes "significant"). Staff interviews almost always probe this.

32.10 The feedback-loop lens

Does the model's output change the data it will learn from next (recommendations, fraud blocking, credit approval, search ranking, pricing)? If yes: selection bias, exposure bias, self-fulfilling predictions, need for exploration/holdouts, off-policy evaluation.

32.11 The simplest-thing ladder

Heuristic/rules → linear model → GBM → deep model → ensemble. Climb only when the step's gain justifies its cost (latency, maintenance, interpretability). Always have the lower rung as a baseline — without it, you can't claim the higher rung helps.

32.12 The error-decomposition lens

Excess error = approximation (model family) + estimation (data) + optimization (training) + shift (deployment ≠ training). Each has a different fix, and a different diagnostic: overfit a subset (optimization), learning curves (estimation), bigger/different model (approximation), train-vs-production comparison (shift).

32.13 The token-budget lens (new in 2027)

For any LLM system, ask: how many tokens go in and come out per request, per user action, per day — and which of them are necessary? Cost ≈ input tokens × input price + output tokens × output price; latency ≈ queueing + prefill (grows with input) + output tokens × time per token. Most GenAI design and debugging questions ("too slow", "too expensive", "worse after we added context") are answered by drawing this budget and asking which tokens earn their keep (Parts 26, 46).



Real-life analogy: a taxi meter

Think of it like a taxi meter that charges for every kilometre in (the prompt) and every kilometre out (the answer), with the waiting time at the start (time to first token). Before blaming the driver, look at the route: are you taking detours (irrelevant context), circling the block (agent loops), or going to the airport when the corner shop would do (a large model for an easy request)?

Where it breaks: a taxi's cheapest route is also the fastest; with LLMs, fewer tokens can sometimes hurt quality — every cut must be checked on the eval set.

32.14 The eval-first lens (new in 2027)

Before building, define how you will know it works: a golden set from real traffic, the scoring method (assertions, execution, calibrated judges, humans), the online metric, and the launch gate. When evaluation is fuzzy,

everything downstream — prompt tweaks, model choice, fine-tuning — becomes guesswork. In GenAI loops, the candidate who designs the evaluation first usually wins the round (Part 45).



Real-life analogy: writing the exam before teaching the course

Think of it like a teacher who writes the final exam before planning the lessons. Every lesson can then be judged by one question: does it help students pass *this* exam? Without the exam, every new teaching idea "feels" better and nobody can prove it.

Where it breaks: an exam can be gamed (teaching to the test); refresh the eval set with new real cases and keep a hidden part, or the system will overfit to it.

Worked use: "Your model works in the US but not in Brazil. Why?"

Run the lenses: **distribution** (Brazilian users differ: covariate shift; different base rates: label shift; behaviors mean different things: concept drift) → **information availability** (are Brazil's features populated? a data source missing? localized text?) → **signal-to-noise** (are Brazilian labels noisier — different reviewers, language?) → **bias/variance** (few Brazilian examples → variance for that slice) → **decision** (thresholds tuned on US costs/base rates). You now have a structured, five-hypothesis answer with tests for each — without having seen the question before.

PART 33

Interviewer Mode

This part is live

A notebook can't interview you — but Claude can. Come back to the chat and use the commands below. You'll get **one question at a time**, the answer is never revealed before you respond, and follow-ups are built from **your** answer until the boundary of your understanding is found.

33.1 Commands

Say in the chat	What happens
"Interview me"	mixed questions, adaptive difficulty, starting at Medium
"Interview me on ⟨topic⟩"	e.g. "on optimization", "on evaluation metrics", "on gradient boosting"
"Interview me at Staff level"	ambiguous, open-ended scenarios with trade-offs and follow-ups
"Mock system design: ⟨problem⟩"	a 35–45-minute design interview, driven step by step through the Part 24 framework
"Mock coding: ⟨algorithm⟩"	implement from scratch; reviewed for correctness, vectorization, stability, edge cases, complexity
"Debugging drill"	a production failure scenario; you investigate by asking for evidence, one step at a time
"Grade my exam" + your answers	grading of the Final Exam with a weakness map and a revision plan

33.2 The protocol you'll get

1. **One question.** No hints unless you ask ("hint, please" costs points on independence).
2. **Your answer.** Speak it as you would in the room; long is fine.
3. **Evaluation** on seven dimensions (below), each scored 1–5, with specific evidence from your answer.
4. **A follow-up based on what you said** — probing the weakest or most interesting claim. Then another, going one level deeper, until an answer breaks down or you reach bedrock.
5. **Debrief** when the thread ends: - What you did well - What you missed - The ideal reasoning path - The ideal interview answer (2-minute version) - The knowledge gap, named precisely - What to revise (with Part/section references in this notebook)
6. **Next question** — difficulty adapts to your scores.

33.3 The rubric

Dimension	1 — weak	3 — hire-level	5 — exceptional
Technical correctness	errors in core claims	correct, minor imprecision	correct and precise, including edge cases
Depth	slogans only	mechanism explained	mechanism + math + assumptions + failure modes

Dimension	1 — weak	3 — hire-level	5 — exceptional
Clarity	rambling, unstructured	structured, signposted	crisp, prioritized, easy to follow
Mathematical understanding	can't write the key equation	can write and interpret it	can derive it and reason about its consequences
Practical reasoning	textbook only	connects to real data/ production issues	anticipates pitfalls, proposes tests, quantifies
Communication	defensive, vague, "it depends" without specifics	clear assumptions, checks in	drives the conversation, handles pushback gracefully
Missing concepts	major gaps	a few gaps	none material

33.4 What a turn looks like (illustration)

Example exchange

Interviewer: *Why does bagging reduce variance?*

You: *Because averaging many models trained on different bootstrap samples smooths out their errors.*

Evaluation (abridged): *Correctness 3 — right direction. Depth 2 — no mechanism for why errors cancel, no mention of correlation or that bias is unchanged. Math 1 — no variance formula.*

Follow-up (built from your answer): *"You said errors are smoothed out. If I train 1,000 trees on bootstrap samples of the same dataset, does the variance go to zero? Why or why not?"*

PART 34

Active Recall

How the recall system works

Twenty-two recall sets are placed at the end of each major chapter — each with **5 easy recall, 5 conceptual, 5 why, 5 scenario, 3 mathematical and 2 difficult follow-up questions** (25 per set), and Sets 1–15 gained **3 extra "2027 additions"** covering the new sections — **595 questions** in total. Answers are **not** next to the questions: they're in the **Answer Key** at the back of the notebook. Retrieval before re-reading is what builds durable memory.

The whole Part as one picture

Think of it like exercising a muscle. Re-reading notes is watching someone else lift weights; answering from memory is lifting them yourself. It feels harder — that difficulty is what makes the memory stronger — and spacing the sessions out (1, 3, 7, 21 days) is like resting between workouts.

Where it breaks: muscles grow with any lifting; memory grows only if you check your answers honestly — mark hesitant answers "~" and redo them.

Set	Covers	Set	Covers
1	Foundations (Part 1)	12	Deep learning (20)
2	Linear algebra (2)	13	ML coding (21)
3	Probability (3)	14	Debugging & production (22–23)
4	Statistics (4)	15	System design, data-centric ML, model selection (24–26)
5	Linear & logistic regression (5–6)	16	Transformers & language models (41–42)
6	Trees & ensembles (7–8)	17	Adapting & aligning models (43)
7	SVM, KNN, Naive Bayes (9–11)	18	Retrieval, RAG & agents (44)
8	Unsupervised & dimensionality reduction (12–13)	19	Evaluating LLM systems & safety (45)
9	Optimization & losses (14–15)	20	Efficient training & inference (46)
10	Evaluation, validation, imbalance (16–18)	21	Recommender systems & bandits (47)
11	Feature engineering (19)	22	Responsible AI & other frontiers (48–49)

Spaced schedule. Attempt a set the day you finish the chapter; redo only the questions you missed after **1 day, 3 days, 7 days and 21 days**. Mark each question ✓ (fluent), ~ (hesitant), ✗ (wrong). A "~" counts as wrong — interviews punish hesitation on fundamentals.

How to self-grade an answer: would it survive the follow-up in the answer key? If your answer is right but you couldn't explain *why*, mark it ~.

PART 35

Interview Answer Frameworks

Structure is a performance multiplier

Interviewers judge what you know *through* how you say it. A clear structure makes a partial answer sound organized and a complete answer sound senior. Say the structure out loud ("Let me define it, give the intuition, then the mechanism...") — it also buys you thinking time.

The whole Part as one picture

Think of it like a well-organized tiffin box: rice in one compartment, dal in another, sabzi in the third. The food (your knowledge) is the same, but the structure makes it easy to serve and easy for the other person to take in. Frameworks give every answer compartments — definition, mechanism, example, trade-off, edge case — so the interviewer can follow and score you.

Where it breaks: a rigid structure can sound robotic; use the framework as a skeleton and let the conversation lead where to go deeper.

35.1 Conceptual questions

```
DEFINITION → INTUITION → MECHANISM → EXAMPLE → TRADEOFF → EDGE CASE
 (1 sentence)  (analogy)   (why/math)   (concrete)  (when not)  (where it breaks)
```

Worked example — "What is batch normalization?"

Definition. "BatchNorm normalizes each feature of a layer's input using the mini-batch mean and variance, then applies a learned scale and shift."

Intuition. "It keeps every layer's inputs at a stable scale, so each layer sees well-behaved numbers no matter how earlier layers change."

Mechanism. "That makes the loss landscape smoother and the layer invariant to the scale of its weights, so larger learning rates are stable; the batch statistics also add a bit of noise that regularizes. At inference it uses running averages instead of batch statistics."

Example. "Adding BN to a deep CNN typically lets you raise the learning rate several-fold and converge in far fewer epochs."

Trade-off. "It couples examples in a batch, so it depends on batch size and composition, and train and inference behave differently."

Edge case. "With batch size 2 or in autoregressive transformers it breaks down — that's why LayerNorm is used there — and forgetting eval mode causes a silent accuracy drop in production."

35.2 System design questions

```
OBJECTIVE → DATA → LABELS → FEATURES → MODEL
          → EVALUATION → SERVING → MONITORING → FAILURE MODES
```

2-minute skeleton — "Design a spam filter"

Objective: keep spam out of inboxes while almost never hiding legitimate mail — so optimize spam recall at a very low false-positive rate. **Data:** incoming mail, sender history, user actions. **Labels:** user spam/not-spam reports (noisy, biased), honeypots, reviewed samples. **Features:** sender reputation and authentication, content embeddings, URLs, campaign-level similarity, recipient relationship. **Model:** rules/blocklists first, then a GBM plus a compact text model; heavier models only for uncertain mail. **Evaluation:** recall at FPR ≤ 0.1% on recent time-split data, by language and sender type; online: "not spam" rescues and spam reports. **Serving:** tens of ms at delivery. **Monitoring:** campaign spikes, FP rescues by sender, drift. **Failure modes:** adversarial adaptation → frequent retraining; feedback gap in the spam folder → sample it for review; false positives on legitimate bulk senders.

35.3 GenAI system design questions (new in 2027)

```
USERS & TASK → SUCCESS DEFINITION + EVAL SET → KNOWLEDGE (RAG?) → BEHAVIOUR (prompt / fine-tune?)
             → MODEL CHOICE + ROUTING → TOOLS & PERMISSIONS → GUARDRAILS
             → TOKEN BUDGET (cost, latency) → SERVING → MONITORING & FEEDBACK LOOP
```

2-minute skeleton — "Design an assistant that answers employee questions from internal documents"

Task & users: 20K employees, policy and engineering questions, answers must cite sources and respect permissions. **Success:** correct, grounded answers; abstain when unsure; zero permission leaks — measured on a golden set of real questions (retrieval recall@10, faithfulness, correctness via a human-calibrated judge) and online (thumbs, deflected tickets). **Knowledge:** RAG — documents change daily and need citations. **Pipeline:** parse → section-aware chunks with ACL metadata → hybrid BM25 + vector retrieval filtered by the user's groups → cross-encoder rerank → top 8 chunks → LLM with a grounding prompt and citations → citation check. **Model:** mid-size model; most errors are retrieval errors. **Token budget:** ~3K in, 300 out ≈ $0.004/answer; prefix caching for instructions. **Guardrails:** injection-resistant prompt layout, output checks, no cross-user cache. **Monitoring:** judge scores on sampled traffic, refusal rate, index freshness, cost per answer. **Failure modes:** stale versions, lost-in-the-middle, silent model upgrades. (Full version: Design 13.)

35.4 Debugging questions

```
OBSERVE → HYPOTHESIZE → ISOLATE → TEST → MEASURE → FIX → VERIFY
```

Worked example — "The model's performance degraded after deployment."

Observe: "Which metric, which slices, when did it start, sudden or gradual? Are labels mature for the period we're measuring?" **Hypothesize:** "Immediate → training–serving skew or wrong artifact; sudden later → upstream pipeline change; gradual → drift; periodic → seasonality." **Isolate:** "Compare served feature distributions to training, by feature and slice; replay logged requests through the offline pipeline; check the model version hash." **Test:** "If skew is suspected, score the same entities both ways and diff the features." **Measure:** "Quantify impact of each difference via replay." **Fix:** "Unify the feature code path / roll back / retrain on recent data." **Verify:** "Confirm recovery on the affected slice and add a parity monitor so it can't recur."

35.5 Verbal techniques that signal seniority

- **Clarify before solving:** "Before I answer — is latency a constraint? Do we have labels?" (one or two questions, not ten).
- **State assumptions explicitly:** "I'll assume labels arrive within a day; if not, I'd change X."
- **Signpost:** "Three reasons... first... second... third."
- **Quantify:** "At 0.1% prevalence, 1% FPR means ~10 false alarms per fraud."
- **Name the trade-off, then decide:** never stop at "it depends" — say *on what*, and what you'd choose under each condition.
- **Check in:** "Should I go deeper on the loss, or move to serving?"
- **Handle "I don't know" well:** "I haven't used that directly; here's how I'd reason about it from first principles..." — then do so.
- **Recover from mistakes gracefully:** "Actually, let me correct that — the variance term, not the bias term, is what bagging reduces."

PART 36

Staff-Level Questions

How experienced ML engineers reason

They **don't answer immediately**. They clarify the decision being made, enumerate hypotheses across the whole system (data, labels, features, split, model, metric, serving, product), rank them by likelihood × cost to check, name the cheapest discriminating test for each, and end with a decision and how they'd verify it. Below, each question is worked in that style.

The whole Part as one picture

Think of it like the difference between a good mechanic and a fleet manager. The mechanic fixes the car in front of them. The fleet manager asks why so many cars break the same way, what it costs the business, which fix gives the most value per rupee, and how to stop it happening next year. Staff-level answers zoom out in the same way: clarify the goal, build a hypothesis tree, quantify, decide, and design so the problem doesn't recur.

Where it breaks: zooming out is only impressive if you can zoom back in — staff answers still need precise mechanisms and numbers.

36.1 "You improved AUC from 0.82 to 0.86 but business revenue did not change. Why?"

Walkthrough

Clarify first. How is revenue measured — an A/B test? Over how long, with how much power? What decision does the model drive — ranking, a threshold, a bid, a targeting list? What's between the model and revenue (business rules, capacity limits, auctions, UI)?

Hypothesis tree.

1. **The revenue test can't see it.** Revenue per user is heavy-tailed; the test may be underpowered for the expected lift. → Compute the MDE; use variance reduction (CUPED, trimmed revenue); check a more sensitive intermediate metric (conversions).
2. **AUC improved where decisions aren't made.** AUC is a global pairwise-ranking measure. If the gain is among low-scored items, and decisions only use the top 1% (a capacity-limited review queue, top-10 recommendations), nothing changes. → Compare precision/recall at the operating point, partial AUC in the relevant FPR range, NDCG@k — and count how many decisions actually flipped between the models.
3. **Calibration got worse.** If the output feeds a threshold or a bid, a better-ranking but worse-calibrated model can make worse decisions. → Reliability diagrams, log loss; recalibrate.
4. **The offline gain isn't real.** Leakage or a random split inflated the new model's AUC; the online population differs. → Re-evaluate on a later, time-split, deduplicated set.
5. **Training–serving skew** erases the gain online. → Log served features and score the logged traffic offline with both models.

6. **The downstream system absorbs the change.** Fixed capacity (reviewers still handle 2,000 cases), business rules overriding ranks, auction reserves, or the action itself has little effect (a better churn model targeting people for an offer that doesn't change behavior).
7. **Value isn't in the improved segment.** The gain is on low-value users or items.

How I'd decide. Decompose expected revenue impact: (# decisions that change) × (value difference per changed decision). If few decisions change or the changed decisions are low-value, AUC was the wrong offline gate — I'd switch launch criteria to operating-point and value-weighted metrics that historically correlate with revenue.

36.2 "Model quality falls only for new users. How would you debug it?"

Walkthrough

Confirm and size it. Confidence intervals by user tenure bucket (day 0, week 1, month 1...). Is it a drop over time (new users got worse) or always worse?

Hypotheses.

1. **Missing information:** history features are empty/zero for new users; the model learned "zero history ⇒ X" from a different population (e.g. dormant users), confusing *no data* with *low activity*. → Check how missing history is encoded; add explicit "is_new"/tenure features and missing indicators.
2. **Underrepresentation:** new users are a small share of training data (and of the loss). → Reweight, oversample, or train a cold-start-specific model.
3. **Split didn't test it:** random splits put the same users in train and validation; the model was never evaluated on unseen users. → Group split by user; evaluate on users whose first activity is after the training cutoff.
4. **Label differences:** labels for new users are defined over shorter windows or mature later (e.g. "7-day retention" measured before 7 days).
5. **Different population:** a new acquisition channel/campaign brought different users (covariate shift). → Compare new-user feature distributions over time; segment by channel.
6. **Serving path:** real-time features for new users come from a different system (e.g. onboarding service) — skew.

Fixes: cold-start features (context, device, acquisition source, onboarding answers, content-based signals), fast within-session adaptation, exploration for new users, a blended model (cold-start model → personalized model as history accrues), user-grouped validation as a launch gate.

36.3 "Your dataset contains 500 million examples. Would you use cross-validation?"

Walkthrough

Default: no. CV reduces the variance of the performance estimate and reuses scarce data. With 500M rows, a holdout of 1% (5M rows) has a standard error on accuracy of about $\sqrt{0.25/5 \times 10^6} \approx 0.0002$ — far below differences worth acting on — while 5-fold CV multiplies training cost by five.

Where I'd spend the effort instead: a holdout that simulates deployment (time-based, user-grouped, deduplicated), a separate most-recent test period, per-slice evaluation (small slices may need more careful estimates), and **seed/training variance** — the bigger uncertainty at this scale is often run-to-run variation, estimated with a few repeated trainings on subsamples.

Exceptions: rare positives (500M rows but 5,000 positives → effective sample size is small: consider repeated time-based folds or bootstrap CIs on positives); tiny but important slices; hyperparameter selection on a subsample (CV on a 1% subsample can be sensible for speed).

36.4 "Production feature values differ from training features."

Walkthrough

Quantify. Log the features actually used at serving time (with timestamps), join to the offline features computed for the same entity and timestamp, and diff: which features, how often, how large, which segments.

Typical root causes. Two code paths (SQL offline vs. Java/Python online) with subtly different logic; different time windows or aggregation boundaries ("last 7 days" as calendar days offline vs. rolling hours online); **staleness** (online store updated daily, offline computed with fresh data); default/missing encodings (0 vs. −1 vs. NaN); units and timezones; different versions of a tokenizer/vocabulary; upstream schema changes affecting only one path; point-in-time errors in the *training* data (training used future data that serving can't have — then "production is right, training was wrong").

Fix in order. (1) Decide which version is correct (usually the serving-time reality). (2) Short-term: patch the discrepancy, or retrain on logged serving features. (3) Long-term: one feature definition computed by one system (feature store), point-in-time-correct training data generated from the same logic, **parity tests** in CI and **skew monitors** in production. (4) Measure impact: replay logged traffic through the model with corrected features.

36.5 "Your ranking model improves NDCG offline but decreases CTR online."

Walkthrough

1. **Label/objective mismatch.** Offline relevance labels (human judgments or past clicks) don't match what drives clicks now. NDCG with graded relevance may reward "relevant but unappealing" items.
2. **Exposure and position bias in offline data.** Offline NDCG is computed on items the old ranker showed, with clicks inflated at top positions. A new ranker that reorders toward items that were rarely shown is judged on biased labels. → Debias with IPW/click models; use randomized-traffic evaluation; interleaving.
3. **Set-level effects.** NDCG scores items independently; users respond to the *page* — diversity, redundancy, freshness. A model that stacks similar items can raise NDCG and lower CTR.
4. **Interaction with other system parts.** Blending with ads or business rules, re-rankers expecting calibrated scores, UI changes, latency increase (slower results → fewer clicks).
5. **Online dynamics.** Novelty/learning effects; users adapting; short test.
6. **CTR might be the wrong online metric** — if the new model reduces clickbait, CTR falls while satisfaction rises. Check dwell time, successful sessions, reformulations.
7. **Experiment validity.** SRM, bucketing bugs, bot traffic.

Decision: investigate per position and per query class; run interleaving to get a sensitive preference signal; fix the evaluation (debiased offline metric, diversity-aware metrics) so it predicts online results next time.

36.6 "Your fraud model blocks too many legitimate users."

Walkthrough

Measure the problem properly. False declines are hard to observe (blocked users don't complete the transaction). Use proxies: customer contacts, retries that succeed via another method, step-up challenges passed, manual review outcomes, and a small random "approve-and-monitor" sample where risk allows.

Localize it. Which segments (new customers, travelers, specific merchants, card-not-present, certain countries/devices)? Since when — after a model or threshold change, a data change, or seasonally (holidays, travel)?

Levers.

1. **Threshold/decision policy:** recompute thresholds from explicit costs (false decline cost includes lifetime value lost), per amount; recalibrate probabilities first.
2. **Tiered actions:** replace hard declines with step-up authentication for medium risk — cheap friction that also generates labels.
3. **Model improvements** for the confused segments: features that explain benign anomalies (travel notice, known device, historical cross-border behavior), segment-aware models.
4. **Pipeline check:** a broken feature (e.g. device ID missing for a new app version) can push many users into high-risk leaves.
5. **Monitoring:** make false-decline rate a first-class guardrail, not just fraud loss.

Trade-off statement: "Every threshold choice trades fraud losses against false declines; I'd make that trade explicit in dollars with finance and product, then encode it in the decision policy."

36.7 "The distribution changes every weekend."

Walkthrough

Reframe: a predictable weekly cycle is **seasonality, not drift**. The goal is for the model to *know* it's the weekend.

1. **Features:** day-of-week, weekend/holiday flags, hour × day interactions, recent-activity features computed over windows that don't blur weekday/weekend.
2. **Training data:** include multiple full weeks; windows that are multiples of 7 days; don't train on weekday-only snapshots.
3. **Evaluation:** report metrics by day type; time-based splits containing both.
4. **Monitoring:** compare to the same weekday last week (or a seasonal baseline) so drift alarms don't fire every Saturday.
5. **If weekend behavior is fundamentally different** (different user population or intent), consider segment-specific models, calibration or thresholds per day type — or a mixture-of-experts gated on time.
6. **Operational:** retraining schedules and deploys that avoid confounding with the weekly cycle (don't compare a Friday model with a Sunday baseline).

36.8 "Training takes four days. How would you reduce experimentation time?"

Walkthrough

Goal: more *decisions* per week, not just faster training.

1. **Profile first.** Is the bottleneck data loading/preprocessing (common), compute, or evaluation? Fix input pipelines (caching, pre-tokenization, parallel loading), use mixed precision, efficient kernels, larger batch with tuned LR, distributed training.
2. **Smaller proxies that preserve rankings.** Subsample data (e.g. 5–10%), fewer epochs, smaller model width. **Validate the proxy:** on past experiments, does the proxy rank ideas the same way as full runs (rank correlation)? Use scaling-law-style extrapolation where applicable.
3. **Early signals.** Learning-curve comparison at 10–20% of training; kill losers early (successive halving / Hyperband).
4. **Warm starts / fine-tuning** from the last production checkpoint instead of training from scratch for many feature or data changes.
5. **Parallelism of experiments** rather than sequential runs; budget GPUs per idea.
6. **Decouple**: evaluate feature ideas with a cheap model (GBM on a sample) before paying for the big model.
7. **Reserve full runs** for finalists; keep an experiment log so failed ideas aren't retried.

36.9 "Two models have identical accuracy. How would you decide which to deploy?"

Walkthrough

See the full answer in Part 26. In brief: verify the tie statistically (paired test); compare operating-point metrics, calibration, slice/worst-group performance, robustness on the most recent data, error overlap (maybe ensemble), latency/memory/cost, feature dependencies and operational risk, interpretability and maintainability; if still tied, prefer the simpler, cheaper, better-calibrated model — or run an A/B test on business metrics.

36.10 "Leadership wants to replace our ranking model with an LLM." (new in 2027)

Walkthrough

Clarify the motivation. Is the goal better relevance for long, conversational queries, a new product experience (chat-style discovery), or "we should use GenAI"? What are the latency (p99 ~100–200 ms), QPS (thousands) and cost constraints of the current ranker?

Frame the trade-off with numbers. A large-model call per request at 5K QPS with 2K-token prompts is 10M tokens/s — orders of magnitude over budget and latency. The current ranker uses behavioural signals (clicks, purchases, freshness) that an LLM can't see without them being serialized into the prompt.

Propose where LLMs add value. Offline: enrich item text and attributes, interpret head queries (cached), generate relevance labels to distil into a small cross-encoder, and build evaluation sets. Online: a distilled reranker feature, and LLM responses only for long-tail, advice-style queries, streamed after results.

How I'd decide. A/B test the distilled-LLM features inside the existing ranker against the current model on the same metrics (conversion, revenue per search, latency), with a cost model per 1K queries. Recommend the hybrid if it wins; keep the option of a conversational surface as a separate product bet.

36.11 "Great demos, flat usage four weeks after launch. What happened?" (new in 2027)

Walkthrough

Demos select the best cases. Real users ask different questions — look at the actual traffic: intent mix, failure rate by intent, where users abandon, and how often they rephrase.

Hypothesis tree. (1) Quality on real questions is worse than on the demo set — build an eval set from traffic and measure. (2) Latency — users won't wait 8 seconds; measure time to first token and completion. (3) Trust — one confidently wrong answer early teaches users not to rely on it; check citation and abstention behaviour. (4) Workflow fit — the feature sits outside where users work (a separate tab vs. inside the tool). (5) Discoverability and onboarding. (6) Cost-driven limits (rate limiting, smaller model) silently degraded the experience.

What I'd do. Instrument the funnel (exposure → first use → repeat use), fix the top failure intents, add abstention and citations, reduce latency with streaming and smaller models for easy intents, and move the feature into the user's workflow — then measure repeat usage and task completion, not demo quality.

36.12 "Twenty teams are building LLM features. How do you keep quality under control?" (new in 2027)

Walkthrough

Make evaluation a platform, not a per-team afterthought. Shared tooling for golden sets, regression sets, calibrated judges (with human agreement tracked), safety and injection test suites, and CI gates on every prompt, model or retrieval change.

Shared infrastructure with guardrails. A central gateway for model access (version pinning, quotas, cost attribution per team, logging with PII redaction), approved retrieval connectors with permission enforcement, and a registry of prompts and configs tied to their eval results.

Standards, not bureaucracy. Minimum launch checklist (eval set size and coverage, safety tests, cost estimate, rollback plan), lightweight review for high-risk features (money movement, personal data, external actions), and monitoring dashboards per feature: quality on sampled traffic, refusal rate, latency, cost.

Measure the platform. Time for a team to ship a new feature, incidents caught before launch, cost per successful task — and when a provider upgrades a model, every team's eval suite runs automatically to qualify it.

PART 37

Common Interview Traps

The pattern behind every trap

Each trap is a shortcut that skips a step of reasoning: jumping to an answer before the question is defined, trusting a number without asking where it came from, or treating the model as the whole system.

The whole Part as one picture

Think of it like the classic mistakes a driving examiner watches for — not checking mirrors, rolling through a stop sign. Each one is small, but any one of them fails the test. Interview traps are the same: jumping to a model before understanding the data, quoting accuracy on imbalanced data, forgetting leakage, and (new) trusting demos or prompts instead of evaluations.

Where it breaks: avoiding traps doesn't make an answer great — it only keeps it from failing; you still need depth.

Trap	Why it loses points	What strong candidates do instead
Memorizing formulas without intuition	the first "why?" exposes it; can't adapt to variants	derive or explain the mechanism; say what each term means and when it breaks
"It depends" without the dependencies	sounds evasive	"It depends on A and B: if A, I'd do X because...; if B, Y because..." — then pick one for the stated scenario
Choosing a model before understanding the data	signals tool-first thinking	ask about data size, types, labels, constraints; start with a baseline; justify the family by inductive bias
Using accuracy for imbalanced classification	classic red flag	name the base rate, the costs, and use precision/recall/PR-AUC at the operating point
Ignoring data leakage	suggests you'd ship a broken model	ask "is this known at prediction time?", insist on point-in-time features and time/group splits
Confusing correlation with causation	leads to wrong interventions	separate prediction from intervention; propose experiments or causal methods
Treating validation data as test data	optimistic, unreported selection bias	keep an untouched test set; nested CV for small data; limit looks
Ignoring production constraints	designs that can't ship	state latency, memory, cost and freshness budgets; design funnels; plan monitoring
Ignoring calibration	breaks threshold and bidding decisions	check reliability diagrams; recalibrate after resampling/weighting
Optimizing offline metrics without business context	"AUC went up" isn't value	connect the metric to decisions and costs; propose online validation and guardrails
Over-engineering	a transformer for 5K tabular rows	climb the simplest-thing ladder; quantify the gain of each step
Not quantifying	vague claims	put numbers on it: prevalence, error bars, latency, sample sizes
Ignoring label quality	model blamed for bad labels	audit labels, measure agreement, check Bayes-error ceilings

Trap	Why it loses points	What strong candidates do instead
Forgetting feedback loops	unstable systems in recsys/fraud/credit	discuss exposure bias, exploration, holdouts, off-policy evaluation
Not checking in during design interviews	talking past the interviewer's interests	clarify scope early, signpost, ask where to go deeper
Trusting demos over evaluations (GenAI)	demos are cherry-picked; real traffic differs	build a golden set from real queries; report error bars and slices
Treating LLM output as deterministic (GenAI)	no plan for variance, drift or silent model updates	pin versions, sample multiple seeds in evals, add validators and fallbacks
Ignoring token cost and latency (GenAI)	designs that can't ship at scale	draw the token budget: tokens in/out × price, TTFT, TPOT, caching, routing
"Fine-tune it on our documents" (GenAI)	fine-tuning is weak for fresh, citable knowledge	RAG for knowledge; fine-tune for behaviour; say why
Security by prompt (GenAI)	"tell the model to ignore malicious instructions" doesn't hold	least-privilege tools, policy checks in code, human approval, output filtering
Agents for everything (GenAI)	compounding errors, cost and latency	fixed pipelines for known workflows; agents only for open-ended tasks with step budgets

Six sentences that make interviewers wince

1. "I'd just use XGBoost/a transformer." (before any question about the data)
2. "The model has 99% accuracy." (on imbalanced data)
3. "Random forests can't overfit."
4. "The p-value is the probability the null is true."
5. "We'll retrain it every day to handle drift." (without knowing how fast it decays or what labels exist)
6. "The LLM is smart enough; we don't need an eval set." (the 2027 classic)

Unit VIII

Revision & Readiness

Part 38 — Master Cheat Sheets
Part 39 — The 130 Must-Know Questions
Part 40 — The Knowledge Graph
Final Exam · Answer Key

The night before: cheat sheets. The week before: the 130 questions. The month before: the exam.

PART 38

Master Cheat Sheets

How to use these

One page per topic, built for a 10-minute scan before an interview. Every line is a trigger for a longer explanation you should be able to give — if a line doesn't trigger one, go back to its Part.

The whole Part as one picture

Think of it like a pilot's pre-flight checklist: it doesn't teach you to fly, but it makes sure you don't forget a switch under pressure. Each line here should trigger a full explanation you already know; if a line doesn't, go back to its Part before the interview.

Where it breaks: reading a checklist the night before cannot replace having flown the plane — use these for recall, not for learning new material.

38.1 ML algorithms

Algorithm	Core idea	Key hyperparameters	Strengths	Weaknesses
Linear regression	least squares; projection	regularization λ, α	fast, interpretable, closed form	linear; outliers; collinearity
Logistic regression	linear log-odds; BCE	C (=1/λ), penalty	calibrated-ish probabilities, convex	linear boundary
Decision tree	greedy impurity splits	depth, min_samples_leaf	no scaling, interactions, interpretable	high variance, no extrapolation
Random forest	bagging + feature subsampling	n_estimators, max_features	robust, OOB, little tuning	big, underconfident, no extrapolation
Gradient boosting	stagewise fit to negative gradients	learning rate, n_trees (early stop), depth/leaves, subsample	top tabular accuracy, any loss	tuning, overfits noise
SVM	max margin; kernels	C, γ, kernel	high-dim, margin theory	$O(n^2)$–$O(n^3)$, no probabilities
KNN	vote of nearest neighbors	k, metric	simple, non-parametric	slow queries, scaling, curse of dimensionality
Naive Bayes	Bayes + conditional independence	smoothing α	fast, tiny data, text	overconfident, redundant features
k-means	minimize within-cluster SS	k, init	fast, simple	spherical clusters, local optima, scale
DBSCAN	density-connected regions	ε, min_samples	arbitrary shapes, noise	varying density, high-dim
PCA	top eigenvectors of covariance	k	compression, decorrelation	linear, unsupervised, outliers
MLP / deep nets	stacked linear + nonlinearity, backprop	LR, width/depth, regularization	learns features, perceptual data	data/compute hungry, tuning

38.2 Loss functions

Loss	Formula	Predicts	Gradient behavior	Use
MSE	$(y-\hat{y})^2$	mean	$\propto$ residual	regression, Gaussian noise
MAE	$\lvert y-\hat{y}\rvert$	median	constant	robust regression
Huber	quad $\le \delta$, linear beyond	between	clipped residual	outliers
Quantile	$\max(\tau r,(\tau-1)r)$	τ-quantile	asymmetric constant	intervals, asymmetric cost
BCE	$-[y\log p+(1-y)\log(1-p)]$	probability	$p-y$	binary / multi-label
CE (softmax)	$-\log p_y$	class distribution	$p-y$	multiclass
Hinge	$\max(0,1-yf)$	margin decision	$-y$ inside margin, 0 outside	SVM
Focal	$-(1-p_t)^\gamma \log p_t$	down-weighted probs	focuses on hard examples	extreme imbalance
Contrastive	$yd^2+(1-y)\max(0,m-d)^2$	embedding distances	0 once margin met	metric learning
Triplet	$\max(0,d_{ap}-d_{an}+m)$	relative distances	0 for easy triplets	metric learning
InfoNCE	$-\log \frac{e^{s_+/\tau}}{\sum e^{s/\tau}}$	embeddings	soft hard-negative weighting	retrieval, self-supervision

38.3 Metrics

Family	Metrics	Remember
Classification	precision $\frac{TP}{TP+FP}$, recall $\frac{TP}{TP+FN}$, specificity $\frac{TN}{TN+FP}$, F1 $\frac{2PR}{P+R}$	precision depends on prevalence
Ranking of scores	ROC-AUC = P(pos > neg); PR-AUC/AP (baseline = prevalence)	ROC blind to prevalence & calibration
Probabilities	log loss, Brier, ECE, reliability diagram	proper scoring rules reward honesty
Regression	MAE, RMSE, R^2, MAPE (biased low, undefined at 0), WAPE, bias	report bias for forecasts
Ranking lists	P@K, R@K, MRR, MAP, NDCG $= \frac{\sum(2^{rel}-1)/\log_2(i+1)}{\text{IDCG}}$	offline labels are exposure-biased
Choosing	from decision + costs + prevalence + capacity	never "just F1"

38.4 Probability

Item	Formula
Bayes	$P(H\mid E)=\frac{P(E\mid H)P(H)}{P(E)}$; posterior odds = prior odds × LR
Total probability / expectation	$P(A)=\sum_i P(A\mid B_i)P(B_i)$; $\mathbb{E}[Y]=\mathbb{E}[\mathbb{E}[Y\mid X]]$
Total variance	$\operatorname{Var} Y=\mathbb{E}[\operatorname{Var}(Y\mid X)]+\operatorname{Var}(\mathbb{E}[Y\mid X])$
Variance of sum	$\operatorname{Var}(X+Y)=\operatorname{Var}X+\operatorname{Var}Y+2\operatorname{Cov}(X,Y)$
Average of correlated	$\operatorname{Var}(\bar{X})=\rho\sigma^2+\frac{1-\rho}{n}\sigma^2$
CLT	$\bar{X}\approx\mathcal{N}(\mu,\sigma^2/n)$
MLE	$\arg\max\sum\log p(x_i\mid\theta)$; Bernoulli k/n; Gaussian $\bar{x}$, $\frac{1}{n}\sum(x-\bar{x})^2$
MAP	MLE + log-prior: Gaussian → L2 ($\lambda=\sigma^2/\tau^2$), Laplace → L1
Beta–Bernoulli	posterior Beta(α+k, β+n−k); mean $\frac{k+\alpha}{n+\alpha+\beta}$
Entropy / CE / KL	$H=-\sum p\log p$; $H(P,Q)=H(P)+\mathrm{KL}(P\|Q)$; $\mathrm{KL}\ge 0$, asymmetric

38.5 Statistics

Item	Key facts
SE	mean $\sigma/\sqrt{n}$; proportion $\sqrt{p(1-p)/n}$
CI	$\hat{\theta} \pm 1.96\,\mathrm{SE}$; about the procedure, not this interval
p-value	P(data this extreme | H_0) — not P(H_0)
Errors	Type I = α (false positive); Type II = β; power = 1 − β
Sample size	$n \approx \frac{(z_{\alpha/2}+z_\beta)^2(\sigma_1^2+\sigma_2^2)}{\delta^2}$ — halve δ → 4× n
Tests	z (large n / proportions), t (Welch), paired t, χ^2 (independence, SRM), ANOVA (k means), Mann–Whitney, McNemar (paired classifiers)
Multiple testing	Bonferroni α/m, Holm (FWER), BH (FDR)
A/B hygiene	pre-register, power, randomization unit, SRM, no peeking (or sequential), CUPED, guardrails, novelty, interference
Biases	selection, survivorship, confounding, Simpson's paradox, collider
Bootstrap	resample with replacement; ~63.2% unique; paired for model comparison; blocks for dependence

38.6 Optimization

Item	Key facts
GD	$\theta - \eta\nabla J$; stable $\eta < 2/\lambda_{max}$; speed ∝ condition number
SGD	unbiased, variance ∝ 1/B; needs LR decay; implicit regularization
Momentum	$v = \beta v + g$; damps oscillation, accelerates valleys; $\eta/(1-\beta)$ effective
Nesterov	gradient at look-ahead point; $O(1/t^2)$ convex
AdaGrad / RMSProp	per-param scaling by accumulated / EMA squared gradients
Adam	m, v EMAs + bias correction; step ≈ η·mean/RMS
AdamW	decoupled weight decay (≠ L2 in Adam)
Schedules	warmup (early instability, Adam's v), cosine/step decay (noise floor)
Clipping	by norm: keep direction, cap size
Pathologies	saddles (mixed Hessian signs), plateaus, vanishing/exploding (Jacobian products)
Adam vs SGD	Adam: fast, robust, transformers; SGD: sometimes better generalization (implicit bias, noise)

38.7 Regularization

Technique	Mechanism
L2 / weight decay	shrink weights ∝ size; shrinks low-variance directions most; Gaussian prior
L1	constant pull → exact zeros; Laplace prior; unstable with correlated features
Elastic net	L1 + L2: sparse + grouped
Early stopping	limits effective capacity; ≈ L2 for least squares
Dropout	noise, anti-co-adaptation, implicit ensemble
Data augmentation	encode invariances via data
Label smoothing	cap confidence, better calibration
Tree constraints	depth, min leaf size, pruning (cost-complexity), γ/λ in XGBoost

Technique	Mechanism
Ensembling	variance reduction (bagging), averaging seeds
Normalization	stabilizes, mild noise (BN)

38.8 Validation

Situation	Split
IID, small data	stratified K-fold; nested CV for tuning + estimate
Large data	single holdout (+ recent test period)
Temporal	forward-chaining / rolling origin + gap ≥ label window
Repeated entities	group K-fold (or time split if entities recur in prod)
Near-duplicates	dedupe before splitting / group by cluster
Imbalanced	stratify; resample **inside** training folds only
Always	fit preprocessing inside folds; test set touched once; simulate deployment

38.9 Feature engineering

Topic	Rule of thumb
Scaling	needed for distance/penalty/gradient methods and PCA; not for trees; fit on train
Skew	log1p; beware retransformation bias for targets
Categorical	one-hot (low card.), target encoding (OOF + smoothing), hashing, embeddings, native (CatBoost/LightGBM)
Missing	ask why (MCAR/MAR/MNAR); impute + indicator; consistent train/serve encoding
Time	cyclical sin/cos, recency/frequency, lags, rolling windows, point-in-time
Text	TF-IDF + linear baseline; pretrained embeddings; hashing for streams
Interactions	explicit for linear models; trees/FMs/deep nets learn them
Leakage check	"known at prediction time? computed the same way at serving?"

38.10 Model debugging

```
DATA → LABELS → FEATURES → SPLIT → PIPELINE → MODEL
     → LOSS → OPTIMIZATION → EVALUATION → SERVING → MONITORING
```

Symptom	First suspects
train ↓, val ↑	variance; split mismatch; overconfidence (check metric vs loss)
both poor	bug/misaligned labels, no signal, too little capacity, bad LR
offline ↑, online ↓	metric mismatch, skew, leakage, exposure bias, latency, experiment flaws
sudden one-class predictions	upstream feature null/constant, schema change, wrong artifact
NaN	LR/explosion, log(0), bad data, fp16
one bad segment	coverage, missing features, label quality, base rate/threshold
decay after deploy	immediate = skew; gradual = drift; periodic = seasonality
too good	leakage: shuffle-label test, time/group re-split, dedupe

38.11 ML system design

```
Clarify goal → ML task & target → metrics (offline/online/guardrails) → scale & latency
→ data & labels (bias, delay) → features (point-in-time) → baseline → model
→ training & validation → serving (funnel, fallbacks) → monitoring → feedback loop
→ retraining → experimentation → failure modes (cold start, adversaries, drift, outages)
```

Funnel: retrieval (recall@K, ANN) → ranking (multi-task, NDCG/log loss) → re-ranking (diversity, rules). Always: baseline, trade-off table, failure modes.

38.12 Production ML

Topic	Key point
Feature store	one definition; offline (point-in-time) + online (low-latency)
Inference	batch (cheap, stale) vs. online (fresh, latency); hybrids
Registry & versioning	model + preprocessing + data snapshot + code + config
Monitoring	system, data quality, input drift (PSI), prediction drift, delayed performance, business
Drift types	covariate, label, concept, prediction; skew is a bug, not drift
Retraining	cadence from decay rate, label delay, cost; gates
Deployment	shadow → canary → A/B → full; rollback ready
Feedback loops	exploration, holdouts, off-policy evaluation

38.13 Mathematical formulas

Formula	
Normal equation	$\hat{w} = (X^\top X)^{-1} X^\top y$
Ridge	$(X^\top X + \lambda I)^{-1} X^\top y$; shrink $\frac{s^2}{s^2+\lambda}$
Lasso 1-D	$\text{sign}(\rho)\max(\lvert\rho\rvert - \lambda, 0)$
Sigmoid'	$\sigma(1-\sigma)$
BCE gradient	$\frac{1}{n} X^\top (p - y)$
Softmax Jacobian	$p_k(\delta_{kj} - p_j)$; CE gradient $p - y$
Backprop	$\delta_{l-1} = (W_l^\top \delta_l) \odot \phi'(z_{l-1})$, $\nabla W_l = \delta_l a_{l-1}^\top$
Bias–variance	$\sigma^2 + \text{bias}^2 + \text{variance}$
PCA	$\Sigma u = \lambda u$
Entropy / IG	$-\sum p \log p$; $H(S) - \sum \frac{\lvert S_c\rvert}{\lvert S\rvert} H(S_c)$
Gini	$1 - \sum p_k^2$
XGBoost	$w^* = -\frac{G}{H+\lambda}$; gain $\frac{1}{2}[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}] - \gamma$
SVM	margin $2/\Vert w\Vert$; hinge + $\frac{1}{2C}\lVert w\rVert^2$
Cost threshold	$t^* = \frac{C_{FP}}{C_{FP}+C_{FN}}$
Downsampling fix	$p = \frac{q}{q+(1-q)/w}$
Init	Xavier $\frac{2}{n_{in}+n_{out}}$; He $\frac{2}{n_{in}}$
Attention scale	$\frac{q^\top k}{\sqrt{d}}$

Formula	
Cohen's κ	$\frac{p_o - p_e}{1 - p_e}$
PSI	$\sum (p - q) \ln \frac{p}{q}$

38.14 Complexity of common algorithms

Algorithm	Training	Prediction (per example)	Memory
OLS (normal eq.)	$O(nd^2 + d^3)$	$O(d)$	$O(d^2)$
Linear/logistic via SGD	$O(nd)$ per epoch	$O(d)$	$O(d)$
Decision tree	$\approx O(dn \log n)$ (balanced)	$O(\text{depth})$	$O(\text{nodes})$
Random forest (T trees)	$T\times$ tree (parallel)	$O(T \cdot \text{depth})$	$T\times$ nodes
GBM (T trees, hist)	$O(T \cdot nd)$-ish	$O(T \cdot \text{depth})$	$T\times$ nodes
Kernel SVM	$O(n^2)$–$O(n^3)$	$O(n_{SV}\, d)$	$O(n^2)$ kernel cache
KNN (brute)	$O(1)$	$O(nd)$	$O(nd)$
Naive Bayes	$O(nd)$	$O(Kd)$	$O(Kd)$
k-means	$O(nkdI)$	$O(kd)$	$O(nd + kd)$
Hierarchical clustering	$O(n^2 \log n)$–$O(n^3)$	—	$O(n^2)$
DBSCAN	$O(n \log n)$ with index, $O(n^2)$ worst	—	$O(n)$
PCA (full SVD)	$O(nd \min(n, d))$	$O(dk)$	$O(nd)$
t-SNE	$O(n^2)$ naive, $O(n \log n)$ Barnes–Hut	no transform	$O(n)$–$O(n^2)$
MLP	$O(\text{params})$ per example forward, ≈3× for a training step	$O(\text{params})$	params + activations
Self-attention	$O(T^2 d)$ per layer	—	$O(T^2)$ attention scores

38.15 Transformers & language models (2027)

Concept	One-line trigger	Number / formula to remember
Attention	soft dictionary lookup: query·key → softmax weights → mix values	$\text{softmax}(QK^\top/\sqrt{d_k})V$; $O(T^2 d)$
Multi-head	several low-dim attentions in parallel, concatenated	$d_{\text{head}} = d/h$
Block	pre-norm → attention → residual → pre-norm → MLP → residual	params ≈ $12Ld^2$
Positions	RoPE rotates q,k by position → relative	extend context by rescaling frequencies
GQA / MQA / MLA	fewer or compressed K/V heads → smaller KV cache	KV ∝ n_{kv}
FlashAttention	tiled, exact, IO-aware attention with online softmax	memory $O(T)$
MoE	router picks top-k experts per token	compute ∝ active params
Tokenization	BPE merges frequent pairs	cost varies by language
Objective	next-token cross-entropy; perplexity = exp(avg NLL)	
Scaling	loss ∝ power law in N, D, C	$C \approx 6ND$; compute-optimal ≈ 20 tokens/param
Decoding	greedy, beam, temperature, top-k, top-p	$p_i \propto e^{z_i/T}$

Concept	One-line trigger	Number / formula to remember
Reasoning / test-time compute	more thinking tokens or samples → accuracy on hard tasks	route easy queries away

38.16 Adapting & aligning models (2027)

Method	What it changes	Key formula / fact	Watch out for
Prompting / ICL	nothing (context only)	examples cost tokens every call	order and format sensitivity
RAG	knowledge at query time	retrieve → rerank → ground → cite	retrieval misses
SFT	behaviour/format	mask prompt tokens	forgetting, template bugs
LoRA	low-rank weight update	$h = Wx + \frac{\alpha}{r} BAx$; $r(d_{\text{in}} + d_{\text{out}})$ params	rank too low for big shifts
QLoRA	LoRA on a 4-bit frozen base	NF4 + paged optimizers	slower steps
RLHF	policy vs. learned reward	$\max \mathbb{E}[r] - \beta \text{KL}(\pi \Vert \pi_{\text{ref}})$; BT: $\sigma(r_w - r_l)$	reward hacking
DPO	preferences directly	$-\log \sigma(\beta \Delta \log \text{ratio})$	offline data coverage
GRPO / RLVR	verifiable rewards, group baselines	$A_i = (r_i - \bar{r})/\text{std}(r)$	weak checkers get gamed
Distillation	student matches teacher	$T^2 \, \text{KL}(p_t^{(T)} \Vert p_s^{(T)})$	teacher errors copied

38.17 Inference & training math (2027)

Quantity	Formula	Example
Training FLOPs	$6ND$	7B × 2T tokens = 8.4×10^{22}
GPU-hours	$C/(\text{peak} \times \text{MFU})/3600$	≈ 59K H100-hours at 40% MFU
Training memory (Adam, mixed)	16 bytes/param + activations	7B → 112 GB
ZeRO-3 / FSDP	≈ 16/G bytes/param per GPU	7B on 8 GPUs → 14 GB
Inference FLOPs	$2N$ per token	
Weights memory	params × bytes	70B: 140 GB fp16, ~35 GB 4-bit
KV cache	$2L \, n_{\text{kv}} \, d_h \, T \, b$ bytes	7B MHA 4K fp16 = 2 GB/seq
Decode ceiling (batch 1)	bandwidth ÷ weight bytes	3.35 TB/s ÷ 14 GB ≈ 240 tok/s
Ridge point	peak FLOP/s ÷ bandwidth	≈ 300 FLOP/byte (H100-class)
Speculative decoding	tokens/pass $= \frac{1-\alpha^{\gamma+1}}{1-\alpha}$	α=0.7, γ=4 → 2.8
Cost per request	$T_{\text{in}} c_{\text{in}} + T_{\text{out}} c_{\text{out}}$	1.5K in + 300 out at $1/$4 per M → $0.0027
Serving metrics	TTFT, TPOT, throughput, goodput	report p50/p95/p99

38.18 RAG, agents & LLM evaluation (2027)

Topic	Trigger	Formula / number
Chunking	structure-aware, a few hundred tokens, context headers	tune on recall@k
BM25	TF saturation + length norm + IDF	$k_1 \approx 1.2$–2, $b \approx 0.75$
Hybrid + RRF	fuse ranks, not scores	$\sum 1/(60 + \text{rank})$
Rerank	cross-encoder on top 50–100	
ANN	HNSW (graph), IVF (clusters), PQ (codes)	100M×768 fp32 = 307 GB; PQ-96 = 9.6 GB

Topic	Trigger	Formula / number
RAG eval	retrieval vs. generation separately	recall@k, faithfulness, correctness, citations
Agents	loop of plan → tool → observe	success ≈ p^n; $0.95^{10} = 0.60$
Prompt injection	instructions hidden in data	least privilege, approval, output filters
pass@k	unbiased	$1 - \binom{n-c}{k} / \binom{n}{k}$
Eval noise	SE of accuracy	$\sqrt{p(1-p)/n}$: 70% on 500 → ±4 pts
Paired comparison	resample questions, keep pairs	far tighter CI than unpaired
Judges	position, verbosity, self-preference biases	validate vs. humans with κ
Safety	harmful compliance vs. over-refusal	measure both

38.19 Recommender systems & bandits (2027)

Topic	Trigger	Formula
Funnel	retrieve (recall) → rank (precision) → re-rank (diversity, rules)	
MF / ALS	alternating ridge regressions	$p_u = (Q_u^\top Q_u + \lambda I)^{-1} Q_u^\top r_u$
BPR	pairwise ranking on implicit data	$\log \sigma(s_{ui} - s_{uj})$
Two-tower	in-batch softmax + logQ correction	$s - \log q_i$
Rankers	GBDT, DCN, DLRM, DIN, MMoE	multi-task value score
nDCG	graded relevance, log discount	$\sum \frac{2^{rel}-1}{\log_2(r+1)} / \text{IDCG}$
Biases	position, popularity, selection	IPS, randomization
OPE	IPS / SNIPS / DR	$\frac{1}{n} \sum \frac{\pi}{\mu} r$
MMR	relevance − redundancy	$\lambda\, rel - (1-\lambda) \max sim$
Bandits	ε-greedy (linear regret), UCB1, Thompson (log regret)	UCB1: $\bar{x} + \sqrt{2 \ln t / n_a}$
LinUCB	ridge per arm + uncertainty bonus	$\hat{\theta}^\top x + \alpha \sqrt{x^\top A^{-1} x}$

38.20 Responsible AI & frontiers (2027)

Topic	Trigger	Formula / fact
Fairness criteria	parity, equal opportunity, equalized odds, calibration	can't have calibration + equal FPR/FNR with different base rates
SHAP	fair split of a prediction; sums exactly	$f(x) = \mathbb{E}f + \sum \phi_j$
Integrated gradients	path integral of gradients; completeness	sums to $f(x) - f(x')$
Differential privacy	one person changes outputs by ≤ e^ε	Laplace scale Δ/ε
DP-SGD	clip per-example grads, add noise	cost: accuracy, memory
Federated learning	train on devices, average updates	+ secure aggregation + DP
Adversarial examples	FGSM $x + \epsilon\, \text{sign} \nabla_x L$	direction, not size, matters
EU AI Act	risk tiers; GPAI obligations from Aug 2025	high-risk (Annex III) postponed to 2 Dec 2027
Forecasting	seasonal-naive baseline; walk-forward CV	MASE < 1 beats naive
GNNs	message passing; GCN $\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} HW$	keep shallow (over-smoothing)
VAE	ELBO = reconstruction − KL	$\frac{1}{2} \sum (\mu^2 + \sigma^2 - 1 - \log \sigma^2)$

Topic	Trigger	Formula / fact
Diffusion	predict noise; guidance	$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$
Contrastive	InfoNCE / CLIP	in-batch softmax with temperature
Causal ML	CATE via S/T/X/DR learners	no fix for unobserved confounding
RL	Bellman; Q-learning off-policy; PPO clip	$Q \leftarrow Q + \eta[r + \gamma \max Q' - Q]$

PART 39

The 130 Must-Know Questions

The bar

You should be able to answer every **Core** question fluently in 2 minutes, with the math where relevant, and survive two follow-ups. **Important** questions: a solid 2-minute answer. **Advanced**: a correct mechanism and a derivation sketch. **Staff-level**: a structured, trade-off-driven answer that ends in a decision. Tick the box only when you can do it **out loud, without notes**.

The whole Part as one picture

Think of it like the "important questions" list that seniors pass down before board exams: not everything that could be asked, but what almost always is. Tick each one only when you can answer it out loud in two minutes and survive two follow-ups — not when you've merely read the answer.

Where it breaks: every company and interviewer has favourites outside any list; use it to find gaps, then go deep on your weakest areas.

39.1 Core (35)

#	Question	Where	✓
1	What does ML optimize, and what's the gap between training loss and what we care about?	1.1	☐
2	Explain the bias–variance tradeoff and how to diagnose each.	1.11	☐
3	What is overfitting? How do you detect and prevent it?	1.15	☐
4	Why train/validation/test splits? When is a random split wrong?	1.7, 17	☐
5	What is data leakage? Give three forms and how to prevent them.	1.17	☐
6	What is regularization and why does it reduce overfitting?	1.16, 28	☐
7	L1 vs. L2: effects, geometry and when to use each.	5.11	☐
8	Derive the normal equation.	31	☐
9	Why is MSE the loss for linear regression?	3.8, 31	☐
10	What are the assumptions of linear regression, and what breaks when each fails?	5.5	☐
11	Explain logistic regression and derive its gradient.	6	☐
12	Why cross-entropy instead of MSE for classification?	28	☐
13	Precision, recall, F1 — and when to prioritize each.	16.1	☐
14	ROC-AUC vs. PR-AUC under class imbalance.	16.1	☐
15	Why can accuracy be misleading?	28	☐
16	How do you choose a classification threshold?	6.7	☐
17	How does a decision tree choose splits? Entropy vs. Gini.	7.2	☐
18	Random forest: how and why it works.	8.2	☐

#	Question	Where	✓
19	Gradient boosting: how it works.	8.3	☐
20	Bagging vs. boosting.	8.4	☐
21	How do KNN and k-means work; what are their weaknesses?	10, 12	☐
22	PCA: what it is and why eigenvectors.	13	☐
23	Curse of dimensionality.	10	☐
24	Gradient descent, SGD and mini-batch — trade-offs.	14	☐
25	Learning rate too high / too low — symptoms and fixes.	14	☐
26	Explain backpropagation on a whiteboard.	20, 31	☐
27	Why nonlinear activations? ReLU vs. sigmoid.	20.3	☐
28	Vanishing/exploding gradients and fixes.	14	☐
29	Dropout and batch normalization — what and why.	20	☐
30	Cross-validation variants and when to use each.	17	☐
31	How do you handle class imbalance?	18	☐
32	Feature scaling: when and why.	19, 28	☐
33	Encoding categorical variables (incl. high cardinality).	19	☐
34	Missing values: mechanisms and strategies.	19	☐
35	p-values, Type I/II errors and power.	4	☐

39.2 Important (30)

#	Question	Where	✓
36	MLE vs. MAP; how priors become regularizers.	3	☐
37	Why optimize log-likelihood?	3	☐
38	Entropy, cross-entropy, KL — relationships.	3	☐
39	Bayes' theorem with a base-rate example.	3	☐
40	CLT and why it matters for experiments.	3, 4	☐
41	Design and analyze an A/B test (CTR 4.1% → 4.3%).	4	☐
42	Multiple testing corrections.	4	☐
43	Simpson's paradox and confounding.	4	☐
44	Correlation vs. causation; how to estimate causal effects.	4	☐
45	Multicollinearity: effects and fixes.	5	☐
46	Calibration: measure and fix.	6	☐
47	Softmax and numerical stability.	6, 21	☐
48	XGBoost vs. LightGBM vs. CatBoost.	8	☐
49	SVM: margin, hinge loss, kernel trick.	9	☐
50	Why Naive Bayes works despite false assumptions.	11	☐
51	Choosing k in k-means; evaluating clustering without labels.	12	☐
52	PCA vs. t-SNE; why t-SNE distorts global structure.	13	☐

#	Question	Where	✓
53	Momentum, Adam, AdamW.	14	☐
54	Learning-rate schedules and warmup.	14	☐
55	MSE vs. MAE vs. Huber.	15	☐
56	Focal loss — when and why.	15	☐
57	NDCG, MRR, MAP.	16	☐
58	Metric choice for fraud at 0.1% prevalence.	16	☐
59	Time-series validation.	17	☐
60	Why oversampling before splitting leaks.	18	☐
61	Target encoding and its leakage.	19	☐
62	Weight initialization (Xavier/He).	20	☐
63	Residual connections — why they help.	20	☐
64	Covariate shift vs. label shift vs. concept drift.	1, 23	☐
65	Training–serving skew: what, why, how to prevent.	23	☐

39.3 Advanced (20)

#	Question	Where	✓
66	Derive the bias–variance decomposition.	31	☐
67	Ridge in the SVD basis: which directions shrink and why.	5	☐
68	Derive softmax cross-entropy gradient.	31	☐
69	Derive XGBoost's leaf weight and split gain.	8	☐
70	Why Adam converges faster but SGD may generalize better.	14	☐
71	Double descent and generalization of over-parameterized models.	1, 27-H8	☐
72	Implicit bias of GD (min-norm; max-margin).	27-H20	☐
73	SVM dual and support vectors.	9	☐
74	Forward vs. reverse KL.	3	☐
75	Importance weighting and its failure modes.	1.9	☐
76	Label-shift correction.	27-H15	☐
77	Position bias and counterfactual learning-to-rank.	27-H16	☐
78	Off-policy evaluation for recommenders.	27-H17	☐
79	Calibration correction after negative downsampling.	18, 27-H27	☐
80	Why BatchNorm allows higher learning rates.	20	☐
81	CUPED and variance reduction.	4	☐
82	Peeking and sequential testing.	4	☐
83	Interference / network effects in experiments.	4	☐
84	Label noise: detection and robust training.	25	☐
85	CatBoost's ordered target statistics and ordered boosting.	8	☐

39.4 Staff-level (15)

#	Question	Where	✓
86	AUC up 0.82 → 0.86, revenue flat. Why?	36	☐
87	Quality falls only for new users. Debug.	36	☐
88	500M examples: cross-validation?	36	☐
89	Production features differ from training.	36	☐
90	NDCG up offline, CTR down online.	36	☐
91	Fraud model blocks too many legitimate users.	36	☐
92	Distribution shifts every weekend.	36	☐
93	Training takes 4 days — speed up experimentation.	36	☐
94	Two models, identical accuracy — which to deploy?	26, 36	☐
95	Design a recommendation system end to end.	24	☐
96	Design a fraud detection system end to end.	24	☐
97	Design search ranking or ads CTR prediction.	24	☐
98	Monitoring a model with 90-day label delay.	27-S19	☐
99	When is better data worth more than a better model?	25	☐
100	Ship or not: a model that improves overall but hurts a segment.	27-S12	☐

39.5 GenAI & modern ML (30, new in 2027)

#	Question	Where	✓
101	Explain self-attention and why we scale by $\sqrt{d_k}$.	41, 31	☐
102	What is the KV cache? Compute its size for a 7B model at 4K tokens.	41, 46	☐
103	MHA vs. GQA vs. MQA — trade-offs for serving.	41	☐
104	How do transformers encode position? Explain RoPE.	41	☐
105	What does FlashAttention change — and not change?	41	☐
106	Mixture of experts: benefits and costs.	41	☐
107	How does BPE work, and where does tokenization hurt?	42	☐
108	Scaling laws and compute-optimal training ($6ND$, ~20 tokens/param).	42, 46	☐
109	Temperature, top-k, top-p, beam search — when to use each.	42	☐
110	Why do LLMs hallucinate, and what actually reduces it?	45	☐
111	Prompting vs. RAG vs. fine-tuning — how to choose (with cost math).	26, 43	☐
112	LoRA: mechanism, parameter count, when it falls short.	43	☐
113	Walk through RLHF, including the KL penalty.	43	☐
114	Derive the DPO loss from the RLHF objective.	31, 43	☐
115	Verifiable rewards and GRPO advantages.	43	☐
116	Design a RAG pipeline; how do you chunk, retrieve, rerank?	44, 24	☐
117	Evaluate a RAG system: retrieval vs. generation metrics.	44	☐

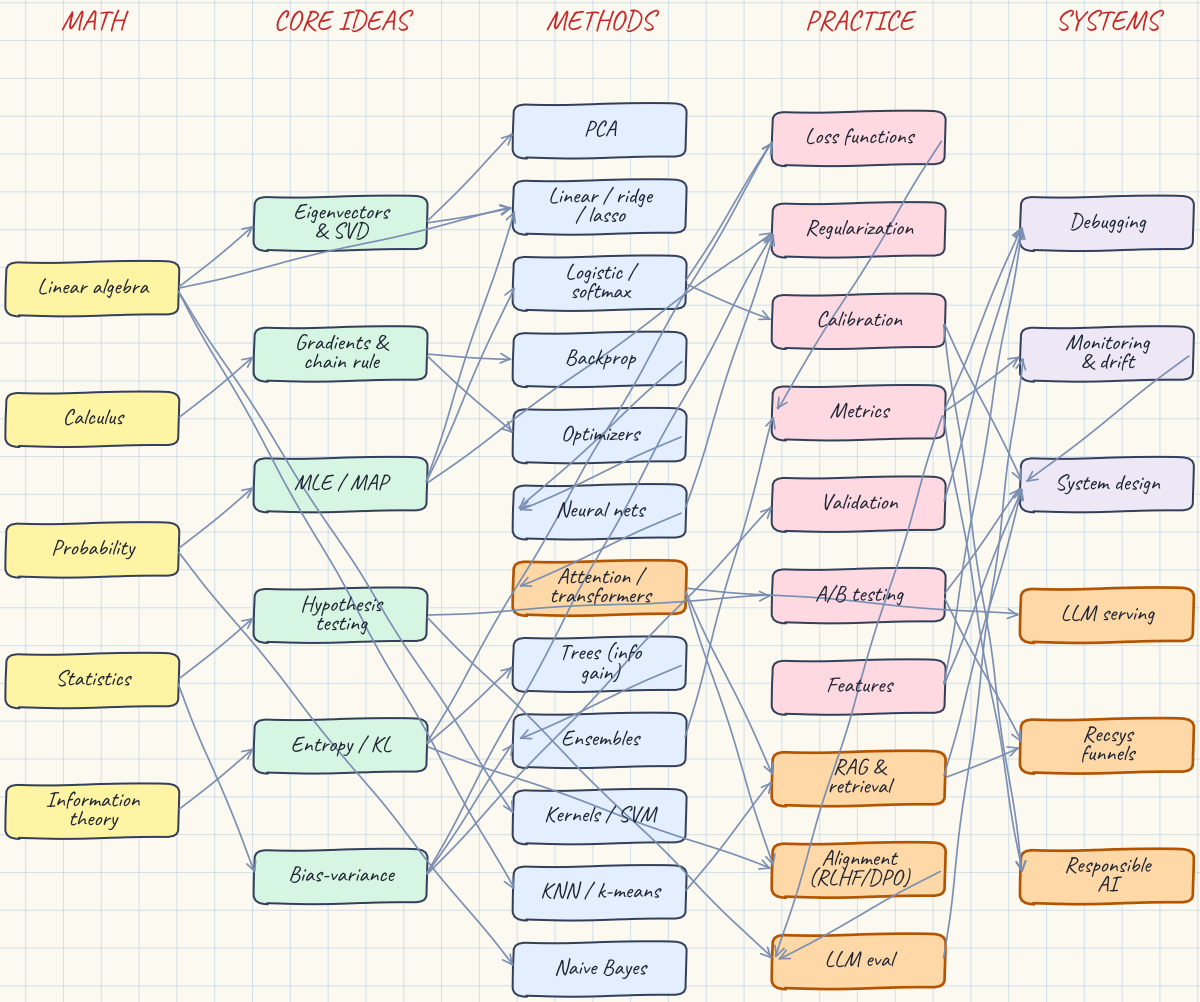
#	Question	Where	✓
118	Agents: when to use one, reliability math, tool design.	44	☐
119	Prompt injection: threats and layered defenses.	44	☐
120	LLM-as-judge: biases and how to validate it.	45	☐
121	Is a 2-point gain on 500 questions real? Paired analysis.	45	☐
122	Why is decoding memory-bound? Batch-1 tokens/s from bandwidth.	46	☐
123	Continuous batching, PagedAttention, prefix caching.	46	☐
124	Quantization and speculative decoding: how and when.	46	☐
125	Training memory: 16 bytes/param, ZeRO/FSDP, parallelism types.	46	☐
126	Two-tower retrieval with logQ correction; ranking with multi-task models.	47	☐
127	Off-policy evaluation with IPS; bandits vs. A/B tests.	47	☐
128	Fairness impossibility; which criterion for which product?	48	☐
129	Differential privacy and DP-SGD in plain words.	48	☐
130	Design a customer-support agent that can issue refunds safely.	24 (Design 14)	☐

PART 40

The Knowledge Graph

ML is one connected system

Almost every topic in this notebook is a node reachable from four roots: **linear algebra** (geometry of data), **calculus** (how to improve), **probability** (what to assume about noise) and **statistics** (how sure to be). When you forget a fact, walk the graph back to a root and re-derive it.



orange = added in the 2027 edition

How the concepts in this notebook depend on each other

40.1 The main chains

```
PROBABILITY → likelihood → MLE → Logistic reg. → Cross-entropy → Classify → Calibration
     |                      |                        |
     |                      +-> MAP → priors → L2 / L1 regularization → Ridge / Lasso
     |                      +-> Gaussian noise → MSE → Linear regression
```

```
      +-> Bayes' rule → Naive Bayes, base rates → precision under imbalance → PR-AUC

LINEAR ALGEBRA → dot products → linear models, attention, cosine similarity → retrieval / ANN
      |
      +-> eigenvectors → covariance matrix → PCA → dimensionality reduction
      +-> SVD → low rank → matrix factorization, LoRA, ridge shrinkage view
      +-> projection → least squares geometry → residuals orthogonal → R^2
      +-> PSD matrices → convexity (Hessians), kernels (Gram matrices) → SVMs

CALCULUS → gradients → chain rule → backpropagation → neural networks
      |
      +-> gradient descent → SGD → momentum → Adam/AdamW → training deep nets
      +-> Hessian / curvature → conditioning → feature scaling, normalization, LR limits
      +-> Jacobian products → vanishing/exploding gradients → init, ReLU, residuals

STATISTICS → sampling distributions → standard errors → confidence intervals
      |
      +-> hypothesis tests → A/B testing → power, multiple testing, peeking → launch decisions
      +-> bias/variance of estimators → bias-variance tradeoff → regularization, ensembles
      +-> selection bias, confounding → leakage, exposure bias, causal inference

INFORMATION THEORY → entropy → information gain → decision trees → random forests / boosting
      |
      +-> cross-entropy, KL → classification losses, distillation, VAEs, drift metrics (PSI)

GENERALIZATION → IID assumption → validation design → distribution shift → monitoring
      → retraining → production ML → system design

MODERN ML (2027) → dot products + softmax → attention → transformers → LLMs (next-token MLE)
      |
      +-> scaling laws (6ND) → efficient training (ZeRO, parallelism) → serving (KV cache, batching)
      +-> adaptation: prompting → RAG (retrieval = ANN + BM25) → LoRA / SFT → RLHF / DPO (KL, Bradley-Ter
      +-> evaluation: pass@k, paired tests, calibrated judges → safety, red-teaming → LLMOps
      +-> two-tower retrieval + ranking → bandits + off-policy evaluation → recommender systems
      +-> fairness, explainability (Shapley), privacy (DP) → responsible deployment
```

40.2 Concept dependency table

Concept	Built on	Leads to
Empirical risk minimization	expectation, LLN, loss functions	generalization, regularization, validation
Bias–variance	expectation, variance	model capacity choices, bagging, boosting, early stopping
MLE	probability, logarithms, calculus	MSE, cross-entropy, Poisson loss, calibration
MAP	MLE + Bayes' rule	L1/L2 regularization, smoothing, target encoding
Gradient descent	derivatives, Taylor expansion	SGD, Adam, backprop training, boosting (functional GD)
Backpropagation	chain rule, matrix calculus	deep learning, gradient checking, vanishing gradients
Eigen-decomposition	linear maps, symmetric matrices	PCA, spectral clustering, conditioning, ridge analysis
Entropy	probability, information	decision trees, cross-entropy, KL, t-SNE, active learning
Kernel trick	dot products, PSD matrices	SVMs, kernel PCA, Gaussian processes
Calibration	proper scoring rules, conditional probability	thresholds, bidding, expected cost, drift correction
Validation design	IID assumption, sampling	leakage prevention, time/group splits, offline–online alignment

Concept	Built on	Leads to
Distribution shift	joint distribution factorizations	monitoring, importance weighting, retraining
A/B testing	CLT, hypothesis tests, causality	launch decisions, CUPED, sequential tests
Funnel architectures	complexity, ANN search, ranking metrics	recommendation, search, ads systems
Attention	dot products, softmax, matrix multiplication	transformers, LLMs, retrieval-style reasoning, KV cache
KV cache & serving	attention, memory bandwidth	batching, GQA, quantization, speculative decoding, cost
LoRA	low-rank approximation (SVD), fine-tuning	cheap adaptation, multi-adapter serving
RLHF / DPO	MLE, Bradley–Terry, KL divergence	alignment, reward hacking, preference data
RAG	ANN search, BM25, ranking metrics	grounded assistants, citations, permissions
LLM evaluation	sampling distributions, paired tests, κ	launch gates, judge calibration, safety evals
Off-policy evaluation	importance sampling, propensities	bandits, recommender evaluation without A/B tests
Differential privacy	probability bounds, noise mechanisms	DP-SGD, private analytics, federated learning

40.3 One connected story (say this to yourself before an interview)

We assume data comes IID from a distribution (**probability**), so we can pick a **hypothesis space** (**linear algebra** gives its geometry) and minimize average **loss** on a sample — where the loss is a negative **log-likelihood** chosen to match the noise (Gaussian → MSE, Bernoulli → cross-entropy). We minimize it with **gradients** (**calculus**, backprop for deep models), controlling the **bias–variance** tradeoff with capacity, **regularization** (priors) and **ensembles** (variance of averages). We estimate how well it generalizes with **validation** that simulates deployment, measure with **metrics** that reflect decision costs, and quantify uncertainty with **statistics**. In production the IID assumption breaks — **shift**, **skew**, **feedback loops** — so we **monitor**, **retrain** and **experiment**, which is what **system design** organizes end to end. The 2027 layer is the same story at a larger scale: an LLM is a **transformer** trained by maximum likelihood on next tokens, **scaled** by the $6ND$ budget, **adapted** with retrieval (knowledge), fine-tuning (behaviour) and preference optimization (a KL-regularized proxy objective), **served** under memory-bandwidth limits, and **evaluated** with the same statistics — error bars, paired comparisons, calibrated judges — plus new risks (**injection**, **hallucination**, **privacy**) that system design must contain.



Real-life analogy: one city, many neighbourhoods

Think of it like a city map. The roots (maths, probability, statistics) are the power station and water supply; the methods are neighbourhoods built on them; production and system design are the roads connecting everything. GenAI is a new, fast-growing district — but its houses still draw power from the same station. When you're lost in a new district, trace the pipes back to the station.

Where it breaks: a city map is fixed; the field keeps adding districts, so revisit the map each year and re-draw the new connections.

FINAL EXAM

MAANG-Style ML Readiness Examination

Rules

No notes. No answers in this notebook. Take the exam in sessions (e.g. one section per sitting, ~45–60 minutes each). Write answers the way you'd say them in an interview: structured, with math where it matters, stating assumptions. Then come back to the chat, say **"Grade my exam"** and paste your answers labeled by question ID (C1, M3, ...). You'll get a per-question grade, a weakness map across topics, and a targeted revision plan pointing back into this notebook.

Suggested time budget: Conceptual 60 min · Math 60 min · Coding 90 min (write real code) · Debugging 45 min · Model selection 40 min · Stats/probability 45 min · Production 40 min · System design 4 × 35 min (pick at least 4 of the 10 to do in full; outline the rest) · Follow-ups 45 min · GenAI & LLM systems 90 min · Recommenders, responsible AI & frontiers 45 min.

E.1 Section A — Conceptual (20)

Answer each in the 2-minute format: definition → intuition → mechanism → example → trade-off → edge case

C1. What does it mean for a learning algorithm to generalize, and what determines the size of the generalization gap?

C2. Explain inductive bias with three concrete examples from different model families. Why can't learning happen without it?

C3. A colleague says "our model is unbiased because we used an unbiased estimator." Discuss the difference between statistical bias of an estimator, model bias in the bias–variance sense, and fairness bias.

C4. Why does ridge regression handle correlated features better than lasso? What does elastic net add?

C5. Explain why logistic regression's decision boundary is linear, and three ways to get a nonlinear boundary while keeping logistic regression.

C6. How does a random forest estimate its own generalization error without a validation set? What are the caveats?

C7. What is gradient boosting doing "in function space"? Why do we fit trees to gradients rather than directly to targets?

C8. Why do kernel methods not overfit catastrophically despite infinite-dimensional feature spaces?

C9. When is k-means the wrong clustering algorithm? Give three data situations and better alternatives.

C10. What does it mean that cross-entropy is a proper scoring rule, and why does that matter in production?

C11. Explain the difference between calibration and discrimination with an example where one is good and the other bad.

C12. What is the curse of dimensionality, and why do deep networks on images seem to escape it?

C13. Explain the role of the learning rate schedule in the final quality of a model, not just speed.

C14. Why do we need both a validation set and a test set? What happens to a test set that is used repeatedly?

C15. What's the difference between covariate shift, label shift and concept drift? Which can you detect without labels?

C16. Why can data augmentation be viewed as regularization? Give an example where augmentation hurts.

C17. Explain what an embedding is and how two-tower models learn embeddings for retrieval.

C18. Why do residual connections make very deep networks trainable? Distinguish vanishing gradients from the degradation problem.

C19. What is the difference between parametric and non-parametric models in terms of how they scale with data, memory and prediction cost?

C20. Explain why "correlation is not causation" matters for feature importance and model explanations.

E.2 Section B — Mathematical (10)

Show your derivations; box final answers

M1. Derive the closed-form solution of ridge regression and show that it exists for any $\lambda > 0$.

M2. Derive the gradient and Hessian of the logistic regression negative log-likelihood and prove convexity.

M3. Derive the bias–variance decomposition for squared loss, stating every independence assumption you use.

M4. Show that the first principal component is the top eigenvector of the covariance matrix, and that the variance it captures equals the top eigenvalue.

M5. Derive the MLE of the rate λ of an exponential distribution from n samples. Is it unbiased?

M6. With a Beta(2, 2) prior and 7 successes in 10 trials, compute the posterior, its mean, and the MAP estimate.

M7. For gradient descent on $f(w) = \frac{1}{2} w^\top \operatorname{diag}(1, 50) w$, find the largest stable learning rate and the per-step contraction factor of the slow direction at the learning rate $1/50$.

M8. Derive the gradient of mean softmax cross-entropy with respect to the logits, and then with respect to the weight matrix of a linear layer $z = Wx$.

M9. Compute the Gini impurity decrease and the information gain for a parent node (30+, 10–) split into (25+, 2–) and (5+, 8–).

M10. Show that $\operatorname{Var}\left(\frac{1}{B}\sum_b X_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ for identically distributed variables with pairwise correlation ρ.

E.3 Section C — ML coding (10)

Write working NumPy code. State time/space complexity, numerical-stability measures and edge cases for each

K1. Implement logistic regression with L2 regularization trained by mini-batch SGD, with a numerically stable loss.

K2. Implement k-means with k-means++ initialization; handle empty clusters.

K3. Implement a function returning precision, recall and F1 for each class of a multiclass problem, plus macro and micro averages.

K4. Implement ROC-AUC in $O(n \log n)$, handling tied scores.

K5. Implement a 2-layer MLP with ReLU and softmax output, forward and backward, and a gradient check.

K6. Implement PCA via SVD returning components, explained variance ratio, and a transform for new data.

K7. Implement stratified K-fold splitting from scratch.

K8. Implement the best-split search for a regression tree (variance reduction) on one numeric feature in $O(n \log n)$.

K9. Implement NDCG@K for a batch of queries with graded relevance.

K10. Implement an online (streaming) mean/variance estimator (Welford) and a standardizer that uses it.

E.4 Section D — Debugging scenarios (10)

For each: state hypotheses across the stack, the cheapest test for each, and the fix

D1. Your CTR model's offline log loss improved by 2%, but in the A/B test CTR is flat and revenue per mille dropped 1%.

D2. A text classifier reaches 99.5% validation accuracy on day one of a project. Your prior expectation was ~85%.

D3. After migrating the feature pipeline from Spark to a streaming system, precision at the operating point dropped from 0.62 to 0.48 with no model change.

D4. A recommender's click-through is stable, but catalog coverage has fallen 40% over three months and new creators complain of no exposure.

D5. A regression model for delivery time has good MAE overall but is biased by +6 minutes on rainy evenings.

D6. Training loss of a transformer spikes to NaN around step 30K in about half of the runs, never at the same step.

D7. A churn model's AUC is 0.88 on a random split and 0.69 on a time-based split.

D8. A fraud model trained last quarter now flags 3× more transactions, but confirmed fraud rates in reviews are unchanged.

D9. Your image classifier works well in the lab, but accuracy drops sharply on photos from a new phone model.

D10. Model v2 beats v1 offline on every metric, but when you retrain v2 with a different seed, it's worse than v1.

E.5 Section E — Model selection scenarios (10)

Choose a model (or approach), justify it with trade-offs, and say what would change your mind

S1. 8,000 rows, 60 mixed-type features, binary label, regulators require reasons for each decision.

S2. 300M rows of ad impressions with billions of sparse categorical feature values; 10 ms budget for 500 candidates.

S3. 1,200 labeled support tickets in 12 categories; a 50 ms CPU budget; 2M unlabeled tickets available.

S4. Forecasting daily demand for 40K products across 500 stores, with promotions and holidays.

S5. Detecting anomalies in 10,000 server metrics with no labels and an on-call team that can handle 20 alerts per week.

S6. On-device keyboard next-word suggestion with 10 MB memory.

S7. Medical imaging with 3,000 labeled scans and an available large pretrained vision model.

S8. Deduplicating 50M product listings with titles and images.

S9. Predicting house prices where the business needs prediction intervals, not just point estimates.

S10. Ranking 1,000 candidate jobs for each job-seeker, with click and apply signals and strong position bias.

E.6 Section F — Statistics and probability (10)

Show numbers where asked

P1. A test for a rare condition (prevalence 0.5%) has sensitivity 95% and specificity 98%. What is the probability a positive result is a true positive? Explain the result intuitively.

P2. An A/B test on conversion (baseline 8%) must detect a relative lift of 5% with 80% power at $\alpha = 0.05$ (two-sided). Estimate the sample size per arm.

P3. You ran 15 metrics in an experiment; 2 are significant at $p < 0.05$. What do you conclude and report?

P4. Explain why the sample variance divides by $n - 1$, and when it doesn't matter.

P5. Revenue per user has a heavy right tail. Describe three ways to get a trustworthy confidence interval for the treatment effect.

P6. Two models' accuracies on the same 2,000 test examples are 91.2% and 90.4%. How do you test whether the difference is real?

P7. Explain Simpson's paradox with a model-evaluation example you construct yourself (with numbers).

P8. What is the expected number of tosses of a fair die until you see a 6? Until you've seen every face at least once?

P9. Randomization is by user, but your metric is computed per page view. What goes wrong with a naive t-test, and how do you fix it?

P10. You observe that users who receive more push notifications retain better. Design a study to estimate the causal effect of notifications.

E.7 Section G — Production ML (10)

Be concrete about systems, signals and thresholds

R1. Design the monitoring for a credit model whose default labels arrive after 12 months.

R2. How would you detect training–serving skew automatically across 300 features?

R3. Describe a safe rollout plan for a new search ranking model, including rollback triggers.

R4. When would you choose continuous (online) training over periodic retraining, and what safeguards does it need?

R5. What exactly should be versioned so that any prediction made 6 months ago can be reproduced?

R6. Your batch-scored churn predictions are 24 hours stale. When does this matter and what would you change?

R7. How do you set drift alert thresholds that are sensitive but don't cause alert fatigue?

R8. A feature's upstream source will be deprecated in 3 months. What's your plan?

R9. How would you reduce p99 latency of a ranking service from 180 ms to 80 ms?

R10. Describe how you'd collect unbiased labels for a fraud model whose decisions prevent observing outcomes.

E.8 Section H — ML system design (10)

Use the framework in Part 24; for each: objective, metrics, data/labels, features, model, serving, evaluation, monitoring, failure modes, trade-offs

Y1. "People you may know" friend recommendations for a social network.

Y2. Real-time detection of fake account sign-ups.

Y3. A news feed ranking system optimizing long-term user satisfaction.

Y4. Autocomplete for an e-commerce search box.

Y5. Dynamic pricing for a ride-hailing platform (predictive components only).

Y6. Detecting toxic comments in 40 languages.

Y7. Predicting hotel-booking cancellations to manage overbooking.

Y8. Visual search: "find products similar to this photo".

Y9. A system that routes customer-support emails to the right team and suggests replies.

Y10. Predicting delivery-driver supply shortages 2 hours ahead, city by city.

E.9 Section I — Extremely difficult follow-ups (10)

These are the questions that end interviews. Reason from first principles

F1. If gradient descent on logistic regression with separable data never converges, why do people still say it "learns the max-margin classifier"? Make this precise.

F2. Your model's test accuracy improved when you *added* label noise to the training set. Give two mechanisms that could explain it.

F3. Why can a model have lower log loss but worse accuracy than another model on the same test set? Construct a tiny example.

F4. Explain why bagging a linear regression model barely helps, but bagging a 1-nearest-neighbor model helps a lot. Then explain why bagging k-NN with large k helps little.

F5. You compute feature importances with permutation on the validation set and the most important feature has *negative* importance. What does that mean?

F6. In an A/B test with interference between treated and control users, is the naive estimate biased upward or downward? Argue both cases with examples.

F7. Why might a larger batch size require a *larger* weight decay to reach the same generalization?

F8. Your offline evaluation uses inverse propensity scoring and the estimate has enormous variance. What are three remedies and what does each cost?

F9. When does adding more training data from a *different* distribution hurt the target distribution, and how would you detect it before training?

F10. A tree model and a neural net get equal AUC; their predictions have correlation 0.6. Estimate (qualitatively) what averaging them does, and when stacking would beat simple averaging.

E.10 Section J — GenAI & LLM systems (15, new in 2027)

Answer with mechanisms, numbers and trade-offs; for designs, use the GenAI framework (Part 35)

J1. Derive why attention logits are divided by $\sqrt{d_k}$, and explain what goes wrong without it.

J2. Compute the KV-cache size for a model with 40 layers, 8 KV heads, head dimension 128, 32K tokens, batch 4, fp16. How many such requests fit next to 26 GB of weights on an 80 GB GPU?

J3. A 13B model is served in fp16 on a GPU with 3.35 TB/s bandwidth. Estimate the batch-1 decode ceiling in tokens/s, then explain two ways to raise throughput and one way to cut latency.

J4. Derive the DPO loss from the KL-regularized RLHF objective. Where does the partition function go?

J5. A LoRA adapter of rank 16 is applied to the query and value projections (4096 × 4096) in 32 layers. How many trainable parameters? Why initialize $B = 0$?

J6. Rewards for four sampled answers are [3, 1, 1, 3]. Compute GRPO advantages. What happens if all four rewards are equal?

J7. Your RAG system's retrieval recall@10 is 0.93 but answer correctness is 71%. List hypotheses and the experiment that distinguishes them.

J8. Design the evaluation plan for an agent that books travel with a corporate card. Include offline, simulated and online components and the safety metrics.

J9. Model B beats model A by 1.8 points on 600 shared questions. Explain how you'd decide whether it's real, including the analysis you'd run and what "paired" buys you.

J10. Explain speculative decoding and prove that the output distribution equals the target model's.

J11. Your LLM judge prefers the first-listed answer 62% of the time in pairwise comparisons. What do you do, and how do you report results already collected?

J12. Choose between (a) a 70B model quantized to 4-bit and (b) a 13B model in bf16 for a latency-sensitive chat product. What do you measure, and what would change your mind?

J13. Design a customer-support agent that may issue refunds up to ₹2,000. Where does policy live, how do you prevent prompt injection from causing refunds, and how do you measure success?

J14. The team proposes fine-tuning on 40,000 internal documents so the model "knows the company". Respond, and propose an alternative with an evaluation plan.

J15. Estimate training compute for a 3B model on 1T tokens and the GPU-days on 256 GPUs at 45% MFU of 989 TFLOP/s. State every assumption.

E.11 Section K — Recommenders, responsible AI & frontiers (10, new in 2027)

Show numbers where asked; name assumptions explicitly

K1. Explain the logQ correction in two-tower training and what happens to popular and long-tail items without it.

K2. A logging policy showed item A with probability 0.8 and item B with 0.2. A new policy would show B with probability 0.9. Logged data: B shown 1,000 times with 60 clicks. Estimate the new policy's click rate on B's impressions with IPS and discuss variance.

K3. Compare ε-greedy, UCB1 and Thompson sampling on regret, and explain which you'd deploy for a news-headline test with delayed clicks.

K4. Offline nDCG improved, online watch time fell. Give five hypotheses and the check for each.

K5. Two groups have base rates 0.1 and 0.3. Explain why a calibrated score with one threshold cannot equalize false-positive rates, and what options remain.

K6. Explain differential privacy's definition in plain words, the Laplace mechanism's noise scale for a count at $\varepsilon = 0.5$, and why DP-SGD clips per-example gradients.

K7. SHAP says "postcode" is the top feature in a credit model. What might that mean, and what would you do?

K8. A forecasting model beats seasonal-naive in cross-validation but not in production. Give three reasons and the validation fix.

K9. Explain the ELBO and why the VAE needs the reparameterization trick; relate it to EM.

K10. You must target discount coupons. Why is predicting purchase probability the wrong target, and how would you build and evaluate an uplift model?

After the exam

Say **"Grade my exam"** in the chat and paste your answers. You'll receive, for each answer: a score (0–4), what was correct, what was missing, and the ideal answer outline; then a **weakness map** by topic (math,

classical ML, deep learning, evaluation, statistics, production, system design, communication) and a **targeted revision plan** that points to specific Parts, recall sets and question-bank cards in this notebook.

ANSWERS

Answer Key — Active Recall

Grading yourself

An answer counts as correct only if it contains the **mechanism**, not just the keyword. Where an answer below is one line, the full explanation lives in the referenced Part — reread it if you were hesitant.

A.1 Set 1 — Foundations

Set 1

1. Expected risk: average loss over the true data distribution, $\mathbb{E}_P[\ell(f(x), y)]$. Empirical risk: average loss over the training sample, $\frac{1}{n}\sum \ell(f(x_i), y_i)$.
2. Noise σ^2 (irreducible), bias² $(f - \mathbb{E}\hat{f})^2$, variance $\mathbb{E}[(\hat{f} - \mathbb{E}\hat{f})^2]$.
3. The classifier that predicts $\arg\max_k P(Y = k \mid X = x)$; it attains the lowest possible (Bayes) error.
4. Train: fit parameters. Validation: choose models/hyperparameters/thresholds/early stopping. Test: one final unbiased estimate.
5. Target leakage (features derived from the label/future), train–test contamination (preprocessing on all data, duplicates across splits), temporal leakage, group leakage (same entity in both).
6. The model was *selected* to minimize it, partly by fitting noise; for a fixed model it's unbiased, but not for the chosen one.
7. Covariate: $P(x)$ changes, $P(y \mid x)$ same. Label: $P(y)$ changes, $P(x \mid y)$ same. Concept: $P(y \mid x)$ changes.
8. Inductive bias is the learner's preference among functions fitting the data equally; in Bayesian terms it's the prior $p(f)$ (regularizers are negative log-priors).
9. Approximation: best-in-class risk minus Bayes risk (the family can't express the truth). Estimation: loss from picking the wrong member with finite data.
10. $\mathcal{H}$ can realize every one of the 2^m labelings of those m points.
11. It's defined by the overlap of $P(y \mid x)$ — the information in the features and label noise; more samples don't change the conditional distribution.
12. NFL averages over all possible target functions (mostly noise); real problems are structured, so assumptions that match the structure win.
13. The scaler's statistics include validation/test values, so transformed training data encodes information about the evaluation set; the fit must happen on training folds only.
14. Overfitting depends on capacity relative to effective sample size and signal-to-noise; correlated rows, feature selection over many candidates, and noisy targets all cause it.
15. The same user appears in train and validation; the model learns user identity/stable traits and is rewarded for recognizing users, inflating metrics versus new users/time periods.
16. Dominant feature importance (is it available at prediction time?), split design (time/group, duplicates), preprocessing fit on all data — then retrain without the feature and with a strict split.
17. (1) Bug: labels misaligned, broken features — inspect rows, overfit a tiny subset. (2) No signal/Bayes-limited — compare to human/bigger model. (3) Too little capacity or over-regularization. (4) Optimization problems.
18. Optimistic by roughly the noise level times $\sqrt{2 \ln 500}$; confirm on an untouched test set or a fresh time period, use nested CV, and reduce the number of looks.
19. Concept drift (or label shift) — $P(y \mid x)$ changed; verify with fresh labels, retrain on recent data.
20. Too many features for 5,000 rows raises variance and selection bias; add features only with strong priors, regularize, use grouped CV, and prefer domain-driven aggregates.
21. $R(f) \le \hat{R}_n(f) + \sqrt{\frac{\ln|\mathcal{H}| + \ln(1/\delta)}{2n}}$ for all $f \in \mathcal{H}$ with probability $\ge 1 - \delta$.

22. $n_{\text{eff}} = \frac{10{,}000}{1+19\times0.3} = \frac{10{,}000}{6.7} \approx 1{,}490$.
23. $\mathbb{E}[(Y-c)^2 \mid X] = \mathrm{Var}(Y \mid X) + (\mathbb{E}[Y \mid X] - c)^2$, minimized at $c = \mathbb{E}[Y \mid X]$.
24. Implicit regularization of SGD (low-norm/flat solutions), architectural priors, real patterns being learned before noise, data augmentation/weight decay; capacity bounds are worst-case.
25. When supports don't overlap or weights have huge variance (low effective sample size $(\sum w)^2/\sum w^2$), or when $P(y \mid x)$ also changed; detect with a domain classifier's AUC and weight distributions.
26. $n \geq (\ln 10^6 + \ln 20)/(2 \cdot 0.05^2) \approx 3{,}360$. Halving ε multiplies it by 4 (the $1/\varepsilon^2$ term) → about 13,400.
27. Test error falls, spikes where the model can just barely interpolate ($p \approx n$), then falls again as capacity grows. At the threshold the feature matrix is square and badly conditioned, so the exact fit needs huge weights that amplify noise; with $p \gg n$ the minimum-norm solution spreads the fit and damps noise.
28. Uniform deviation up to about $\sqrt{(\ln 50 + \ln 20)/(2 \cdot 2000)} \approx 0.04$ — around 4 points. Keep an untouched test set (or nested CV), and reduce the number of looks.

A.2 Set 2 — Linear algebra

Set 2

1. Row–column dot products; linear combination of columns; sum of outer products; composition of linear maps.
2. Number of linearly independent columns = dimension of the column space = number of nonzero singular values.
3. $x^\top A x > 0$ for all $x \neq 0$; all eigenvalues > 0 (symmetric A).
4. Any linear map is rotate → scale along axes → rotate ($U\Sigma V^\top$).
5. $H = X(X^\top X)^{-1}X^\top$.
6. $\sigma_i(X)^2 = \lambda_i(X^\top X)$; right singular vectors are eigenvectors of $X^\top X$.
7. The residual is the perpendicular from y to the column space; the normal equation $X^\top(y - X\hat{w}) = 0$ states exactly that.
8. Sensitivity of the solution to perturbations ($\sigma_{max}/\sigma_{min}$); $\kappa(X^\top X) = \kappa(X)^2$ squares it.
9. The truncated SVD is the best rank-k approximation in Frobenius and spectral norm; error $= \sum_{i>k}\sigma_i^2$.
10. W_2W_1 is a single matrix — the composition of linear maps is linear.
11. $u^\top \Sigma u = \mathrm{Var}(u^\top x) \geq 0$.
12. Maximizing projected variance $u^\top \Sigma u$ with $\|u\| = 1$ yields $\Sigma u = \lambda u$; eigenvalues are the captured variances.
13. The dummy columns sum to the intercept column → X rank-deficient → $X^\top X$ singular.
14. Dot products of d-dim unit-variance vectors have variance d; scaling keeps logits at unit variance so softmax doesn't saturate.
15. Explicit inversion is slower and less accurate (amplifies rounding error); solvers use stable factorizations.
16. Multicollinearity/ill-conditioning; confirm with VIF / condition number / eigenvalues of $X^\top X$; fix with ridge, dropping/combining features, PCA.
17. $X^\top X$ is singular (rank ≤ 800); use regularization (ridge/lasso/elastic net), dimensionality reduction, feature selection, or the minimum-norm solution.
18. Large λ_{max} limits the step; small λ_{min} makes progress slow (condition number). Standardize features, use momentum/Adam, preconditioning, or second-order methods.
19. With $n < d$ the sample covariance is singular (rank ≤ 249), so Σ^{-1} and the log-determinant blow up. Use shrinkage (Ledoit–Wolf), diagonal covariance, or PCA first.
20. Truncated or randomized SVD; choose the rank by retained energy $\sum_{i\leq k}\sigma_i^2/\sum\sigma_i^2$ and by downstream performance.
21. $\nabla\|Xw - y\|^2 = 2X^\top(Xw - y)$.
22. Lagrangian $u^\top\Sigma u - \lambda(u^\top u - 1)$ → $\Sigma u = \lambda u$; objective $= \lambda$, maximal for the top eigenvalue.
23. Eigenvalues 3 and 1; eigenvectors $(1,1)/\sqrt{2}$ and $(1,-1)/\sqrt{2}$.
24. The minimum-norm interpolating solution X^+y — updates are combinations of rows of X, so iterates stay in the row space.
25. In the SVD basis each direction is multiplied by $\frac{s_j^2}{s_j^2+\lambda}$: small singular values (little data support, high OLS variance) shrink most.

26. $A^\top A$ has eigenvalues 45 and 5, so $\sigma = 3\sqrt{5} \approx 6.71$ and $\sqrt{5} \approx 2.24$; rank-1 keeps $45/50 = 90\%$ of the squared Frobenius norm.
27. The eigenvalue ratio $|\lambda_2/\lambda_1|$: the error shrinks by that factor each step, so a small gap between the top two eigenvalues means slow convergence.
28. Forming $X^\top X$ squares the condition number (κ^2), losing twice as many digits; QR/SVD work with $\kappa(X)$. Example: $\kappa = 10^8$ → normal equations give coefficient errors ~10^{-2}, QR ~10^{-10}.

A.3 Set 3 — Probability

Set 3

1. PMF: $P(X = x)$ for discrete; PDF: density whose integral gives probabilities; CDF: $P(X \le x)$. Yes, a PDF can exceed 1.
2. $P(H \mid E) = \frac{P(E|H)P(H)}{P(E)}$: posterior = likelihood × prior / evidence.
3. $\bar{X}_n \approx \mathcal{N}(\mu, \sigma^2/n)$ for IID with finite variance; $\mathrm{Var}(\bar{X}) = \sigma^2/n$.
4. $H(P) = -\sum p \log p$; $H(P, Q) = -\sum p \log q$; $\mathrm{KL}(P\|Q) = \sum p \log \frac{p}{q} = H(P, Q) - H(P)$.
5. $\hat{p} = k/n$.
6. The Gaussian negative log-likelihood is SSE / $(2\sigma^2)$ + const.
7. A prior whose posterior stays in the same family; Beta–Bernoulli (also Gaussian–Gaussian, Dirichlet–Multinomial).
8. MLE maximizes $p(D \mid \theta)$; MAP maximizes $p(D \mid \theta)p(\theta)$. They coincide with a flat prior and asymptotically as $n \to \infty$.
9. Reverse $\mathrm{KL}(Q\|P)$ is mode-seeking: it heavily penalizes Q putting mass where $P \approx 0$, so Q hugs one mode.
10. $\mathrm{Var}\, Y = \mathbb{E}[\mathrm{Var}(Y \mid X)] + \mathrm{Var}(\mathbb{E}[Y \mid X])$; the second term over $\mathrm{Var}\, Y$ is the best achievable R^2.
11. Same argmax; avoids underflow; products → sums (SGD-friendly unbiased mini-batch gradients); nicer convexity; well-behaved gradients; information-theoretic meaning.
12. It uses the sample mean, fitted to the same data, which reduces squared deviations: $\mathbb{E}[\hat{\sigma}^2] = \frac{n-1}{n}\sigma^2$.
13. $\mathrm{Var}(\bar{X}) = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2 \ge \rho\sigma^2$.
14. $H(P, Q) = H(P) + \mathrm{KL}(P\|Q)$ and $\mathrm{KL} \ge 0$ (Gibbs' inequality).
15. The Laplace density has a sharp peak (kink) at 0; the MAP objective's non-differentiable point absorbs weak data gradients.
16. TP rate $0.001 \times 0.95 = 0.00095$; FP $0.999 \times 0.01 = 0.00999$; precision ≈ 0.087 — under 9% of flags are fraud.
17. Don't trust 2/3; shrink toward a prior (e.g. global CTR with pseudo-counts): with prior mean 2% and strength 100, $(2 + 2)/(3 + 100) \approx 3.9\%$; for ranking new ads, use explore/exploit (Thompson sampling or UCB).
18. Heavy tails: the CLT converges slowly and a few big spenders dominate variance — use trimming/winsorization, log transforms, CUPED, bootstrap, or longer tests.
19. Poisson (or negative binomial if overdispersed, zero-inflated/Tweedie for many zeros) with the matching deviance loss on $\log \mathbb{E}[y]$.
20. Zero correlation only rules out linear dependence ($Y = X^2$ example); check mutual information or scatterplots.
21. $\ell(\lambda) = \sum(x_i \log \lambda - \lambda)$ + const ⇒ $\frac{\sum x_i}{\lambda} - n = 0$ ⇒ $\hat{\lambda} = \bar{x}$.
22. Minimize $\frac{1}{2\sigma^2}\|y - Xw\|^2 + \frac{1}{2\tau^2}\|w\|^2$ ⇒ ridge with $\lambda = \sigma^2/\tau^2$.
23. $\rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$.
24. Lower MSE through the bias–variance trade (e.g. ridge, James–Stein shrinkage, smoothing small counts); MLE's optimality is asymptotic.
25. A component can collapse onto one point with variance → 0, making likelihood → ∞; fix with variance floors/priors (MAP-EM), regularizing covariances, restarts.
26. $175 + 0.5 \cdot 14 = 182$ cm; conditional SD $7\sqrt{1 - 0.25} \approx 6.1$ cm (regression to the mean).
27. $I(p) = \frac{1}{p} + \frac{1}{1-p} = \frac{1}{p(1-p)}$; $\mathrm{Var}(\hat{p}) \ge p(1-p)/n$ — achieved by the sample proportion.

28. Naive sampling rarely lands in the tail (≈3 hits per 100K draws), so its relative error is huge; sampling from $\mathcal{N}(4,1)$ puts most draws in the tail and reweights by p/q. It fails when q misses regions where p has mass — the weights explode.

A.4 Set 4 — Statistics

Set 4

1. The probability, assuming the null is true, of a test statistic at least as extreme as observed.
2. Type I: rejecting a true null (false positive, rate α). Type II: failing to reject a false null (rate β). Power $= 1 - \beta$.
3. If the experiment were repeated, 95% of intervals built this way would contain the true parameter.
4. When group variances may differ (the safe default for two-sample mean comparisons).
5. About 63.2% $(1 - 1/e)$.
6. SE of a difference scales as $1/\sqrt{n}$; to shrink it by a factor 2 (detect half the effect) you need 4× the samples.
7. Observed assignment counts differ from the designed split more than chance allows; chi-square goodness-of-fit test on counts per arm.
8. BH controls the false discovery rate (expected share of rejections that are false); Bonferroni controls the probability of any false positive (FWER) and is more conservative.
9. Repeated looks give many chances to cross the threshold, inflating the overall false-positive rate far above α.
10. Selection bias: the sample is chosen by a process related to the outcome. Confounding: a third variable causes both treatment and outcome.
11. The median depends only on order (breakdown point 50%); one extreme value can move the mean arbitrarily.
12. Conditioned on reaching significance with low power, only large noisy estimates pass the threshold (winner's curse).
13. Pairing removes between-example variance; the test only uses the per-example differences.
14. The covariate is measured before treatment, so it's balanced across arms by randomization; subtracting $\theta(X - \bar{X})$ removes explained variance without shifting the mean difference.
15. Unequal group sizes across compared arms combined with a confounder that affects outcomes; aggregation weights groups differently.
16. Probably not: CTR rose but conversions fell — check for accidental clicks/clickbait; investigate downstream metrics and guardrails first.
17. Report as exploratory with multiple-testing correction (BH/Bonferroni); 2 of 30 is roughly what chance predicts (1.5 expected); confirm in a follow-up test.
18. Treated riders consume shared driver supply, affecting control (interference); use geo/cluster randomization or switchback experiments.
19. Selection bias (only approved loans have outcomes) → biased on the rejected population; reject inference, small randomized approvals, external bureau outcomes.
20. Winsorize/trim, log-transform, CUPED, bootstrap CIs, longer tests, or a more sensitive proxy metric (conversion).
21. SE $= \sqrt{\frac{0.09}{10^4} + \frac{0.0979}{10^4}} \approx 0.00434$; $z \approx 0.01/0.00434 \approx 2.3$, $p \approx 0.02$ → significant at 5%.
22. $n \approx \frac{(1.96+0.84)^2(0.09+0.094)}{0.005^2} \approx 57{,}800$ per arm.
23. $1 - 0.95^{20} \approx 0.64$; Bonferroni threshold $0.05/20 = 0.0025$.
24. Paired bootstrap over test examples (or DeLong's test) for the AUC difference, plus seed variance; with 50K examples, 0.003 may or may not be significant — compute the CI of the difference.
25. Novelty effect decaying, learning effects, changing traffic mix over time, seasonality, sample-ratio or instrumentation issues; check effect by cohort (first exposure date) and by week.
26. Impressions from the same user are correlated, so the effective sample size is the number of users, not impressions; the binomial SE is too small. Use the delta method on user-level sums (or a user-level bootstrap).
27. $P(B > A) \approx 0.92$, median lift ≈ +10% (95% interval about −4% to +25%), expected loss of shipping B ≈ 0.01 points. The p-value (≈0.16) answers "how surprising is this if there's no difference"; the Bayesian summary answers "how likely is B better and what does being wrong cost".

28. Difference-in-differences: $(3.6 - 3.0) - (3.1 - 2.8) = +0.3$ points. Assumes parallel trends — without the launch, both cities would have moved the same way; check the pre-launch months.

A.5 Set 5 — Linear & logistic regression

Set 5

1. $\hat{w} = (X^\top X)^{-1} X^\top y$.
2. $-\frac{1}{n}\sum[y \log p + (1-y)\log(1-p)]$.
3. $\sigma'(z) = \sigma(z)(1-\sigma(z))$.
4. $R^2 = 1 - \text{SSE}/\text{SST}$; $R^2_{\text{adj}} = 1 - (1-R^2)\frac{n-1}{n-d-1}$.
5. $S_\lambda(\rho) = \text{sign}(\rho)\max(|\rho| - \lambda, 0)$.
6. Under zero-mean, homoscedastic, uncorrelated errors, OLS is the lowest-variance linear unbiased estimator (BLUE).
7. Hurts coefficient stability, signs, interpretation and SEs; doesn't hurt in-distribution predictions much.
8. The MLE doesn't exist; weights diverge as the loss keeps decreasing — regularize.
9. Each principal direction is scaled by $s_j^2/(s_j^2 + \lambda)$ — low-variance directions most.
10. OvR: independent binary models, simple; softmax: mutually exclusive classes, jointly normalized; independent sigmoids: multi-label.
11. $\arg\min_c \mathbb{E}[(Y-c)^2 \mid X] = \mathbb{E}[Y \mid X]$.
12. L1's constant-magnitude pull zeros weights whose evidence is below λ; L2's pull fades near zero.
13. Non-convex, and the gradient carries a $p(1-p)$ factor that vanishes when confidently wrong.
14. Penalties act on coefficients in feature units; unscaled features get penalized unequally.
15. AUC depends only on the ordering of scores; any monotone transform leaves it unchanged.
16. Heteroscedasticity: coefficients unbiased but SEs wrong/inefficient — robust SEs, WLS, log-transform target.
17. Keeping 1 in 10 negatives ($w = 0.1$): $p = \frac{q}{q+(1-q)/w} = \frac{0.4}{0.4+6} \approx 0.063$ — close to the 5% true rate; apply the correction (or recalibrate).
18. $t^* = \frac{10}{10+300} \approx 0.032$.
19. Near-duplicate features make lasso's choice arbitrary and unstable; use elastic net or group/merge the features.
20. The model is worse than predicting the training mean — overfitting, distribution shift or a bug.
21. $\partial \ell/\partial z = p - y$ (sigmoid derivative cancels) ⇒ $\nabla_w = \frac{1}{n} X^\top (p - y)$.
22. $\nabla[\|y - Xw\|^2 + \lambda\|w\|^2] = 0 \Rightarrow (X^\top X + \lambda I)w = X^\top y$.
23. $\frac{e^{z_1}}{e^{z_0}+e^{z_1}} = \frac{1}{1+e^{-(z_1-z_0)}} = \sigma(z_1 - z_0)$.
24. With an intercept, the first-order condition forces mean prediction = observed rate, and log loss is a proper scoring rule; breaks with class weighting/resampling, strong regularization, misspecification and shift.
25. The direction of the hard-margin (max-margin) SVM solution.
26. For counts with zeros (Poisson) or positive skewed amounts (Gamma), when you need $\mathbb{E}[y]$: $\log 0$ is undefined and $\exp(\mathbb{E}[\log y])$ underestimates the mean. GLMs model the mean directly with the right variance function.
27. $\mathbf{w} \leftarrow \mathbf{w} - (X^\top S X)^{-1} X^\top (\mathbf{p} - \mathbf{y})$ with $S = \text{diag}(p(1-p))$. Better with up to hundreds of features: converges in a few steps and gives standard errors; too costly ($O(d^3)$) for huge d.
28. Odds $0.09/0.91 = 0.0989$; multiply by $\beta = 0.1$ → 0.00989 → $p \approx 0.0098$.

A.6 Set 6 — Trees & ensembles

Set 6

1. Entropy $-\sum p_k \log_2 p_k$; Gini $1 - \sum p_k^2$.
2. Parent entropy minus weighted child entropy — the mutual information between the split and the label.
3. About 36.8%.

4. The number of features considered at each split; smaller values decorrelate trees.
5. $w_j^* = -G_j/(H_j + \lambda)$.
6. It picks the locally best split; XOR-like interactions where no single split helps are missed.
7. Minimize $R(T) + \alpha|T|$; prune weakest links to produce a nested sequence; choose α by CV.
8. Increases weights of misclassified points; stagewise minimization of exponential loss.
9. The negative gradient of the loss w.r.t. the current prediction for each example.
10. GOSS: keep large-gradient examples, subsample small-gradient ones (reweighted). EFB: bundle mutually exclusive sparse features.
11. Splits use only the ordering of a feature's values; monotone transforms don't change partitions.
12. It's piecewise linear, so it often shows no improvement for splits that increase purity without flipping the majority.
13. It lowers correlation between trees, so averaging reduces more variance.
14. Computed on training data (rewards overfit splits) and favors high-cardinality/continuous features; correlated features split credit.
15. To avoid target leakage: each example's encoding uses only preceding examples' labels.
16. Trees can't extrapolate beyond the target range seen in leaves; model the trend (detrend/difference, linear component, ratio target) or use a model that extrapolates.
17. Time leakage or temporal drift: random CV mixes future into training; use time-based validation.
18. Suspect memorization/leakage; drop it, use aggregate behavior features, group-split by user to verify.
19. Lower `num_leaves`, raise `min_data_in_leaf` / `min_sum_hessian`, add `max_depth`, lower learning rate, subsample rows/columns, add L1/L2.
20. A modest GBM compiled to native code (treelite), CatBoost's oblivious trees, or distill into a small model; limit trees/depth and features' fetch cost.
21. Parent $H = 1$; each child $H(5/6) \approx 0.650$; IG ≈ 0.350 bits.
22. $\mathrm{Var}(\frac{1}{B}\sum X_b) = \frac{1}{B^2}[B\sigma^2 + B(B-1)\rho\sigma^2] = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$.
23. $g_i = p_i - y_i$, $h_i = p_i(1 - p_i)$.
24. Forest averaging converges to the expected tree (variance can't go up); boosting keeps fitting training residuals, eventually noise.
25. It greedily splits wherever the largest loss reduction is, regardless of depth — more loss reduction per leaf but deeper, more specific trees that overfit small data.
26. Parent Gini 0.48. Income: $0.5 \cdot 0.32 + 0.5 \cdot 0.48 = 0.40$ (decrease 0.080). Card: $0.7 \cdot 0.408 + 0.3 \cdot 0.444 = 0.419$ (decrease 0.061). Income wins.
27. When domain knowledge or regulation says the effect must go one way (risk rises with missed payments): it prevents noisy wiggles, improves robustness under shift and keeps explanations sensible. Don't force it where the true relationship is U-shaped.
28. They add up exactly: $f(\mathbf{x}) = \mathbb{E}[f] + \sum_j \phi_j$. Pitfalls: correlated features share or steal credit depending on the SHAP variant; they explain the model, not causal effects; log-odds units aren't probability changes.

A.7 Set 7 — SVM, KNN, Naive Bayes

Set 7

1. $\min \frac{1}{2}\|w\|^2$ s.t. $y_i(w^\top x_i + b) \geq 1$.
2. $2/\|w\|$.
3. $K = \exp(-\gamma\|x - x'\|^2)$; γ sets the inverse width — large γ = very local influence.
4. Add α pseudo-counts to every word/class count so unseen words don't zero out probabilities.
5. $O(qnd)$ (plus $O(qn)$ selection).
6. Those with $\alpha_i > 0$ (on the margin if $0 < \alpha_i < C$, violating if $\alpha_i = C$).
7. At the optimum $\xi_i = \max(0, 1 - y_i f(x_i))$; substitute into the objective.
8. Implicit high-dimensional feature spaces via dot products; the kernel must be positive semidefinite (Mercer).

9. Gaussian: per-class Gaussians for continuous features; multinomial: word counts; Bernoulli: binary presence/absence.
10. Small k: low bias, high variance; large k: smoother, higher bias.
11. Robustness to perturbations and margin bounds independent of dimension (R^2/γ^2).
12. Distances sum feature differences — large-unit features dominate.
13. Correlated features' evidence is multiplied as if independent, pushing posteriors toward 0/1.
14. $O(n^2)$ kernel matrix and $O(n^2)$–$O(n^3)$ training; prediction cost grows with support vectors.
15. Distances of sums of many independent terms concentrate around their mean; nearest ≈ farthest, so neighbors carry little information.
16. Lower γ and/or lower C, jointly tuned.
17. Linear SVM or logistic regression on TF-IDF (fast, strong); NB as a quick baseline or with little data; KNN too slow/weak here.
18. Approximate nearest-neighbor index (HNSW/IVF-PQ), dimensionality reduction, candidate pre-filtering, caching.
19. NB treats duplicates as independent evidence (counts it twice); logistic regression shares weight between identical features.
20. Platt scaling or isotonic regression on held-out decision values.
21. $(x^\top z)^2 = \phi(x)^\top \phi(z)$ with $\phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$.
22. $0.1^{1/20} \approx 0.89$ of each axis.
23. $\hat{y} = \arg\max_y [\log P(y) + \sum_w \text{count}_w \log \theta_{wy}]$.
24. The hard-margin SVM direction (max margin).
25. With little training data (it reaches its asymptotic error with far fewer samples), especially in high-dimensional sparse settings.
26. $\max(0, |y - f(\mathbf{x})| - \varepsilon)$: errors inside the tube cost nothing. ν upper-bounds the fraction of training points treated as outliers (and lower-bounds the fraction of support vectors).
27. Spam score $0.5 \cdot \frac{4}{17} \cdot \frac{4}{17} \cdot \frac{1}{17} = 0.00163$ vs. ham $0.5 \cdot \frac{1}{15} \cdot \frac{2}{15} \cdot \frac{3}{15} = 0.00089$ → P(spam) ≈ 0.65. Without smoothing both scores are 0 (unseen words), so no decision is possible.
28. Only points on or outside the tube get non-zero dual weights; points strictly inside contribute nothing, so the fit depends on a subset (support vectors).

A.8 Set 8 — Unsupervised learning & dimensionality reduction

Set 8

1. $J = \sum_i \|x_i - \mu_{c_i}\|^2$.
2. Assign each point to the nearest centroid; recompute centroids as means of assigned points.
3. $s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))}$.
4. Core: ≥ min_samples within ε; border: within ε of a core point but not core; noise: neither.
5. $\lambda_j / \sum_i \lambda_i$.
6. Each step can't increase J and partitions are finite; converges to a local minimum.
7. First centroid random; each next sampled with probability ∝ squared distance to the nearest chosen centroid.
8. Single: nearest pair (chaining); complete: farthest pair (compact); Ward: minimize increase in within-cluster variance.
9. Pythagoras: total variance = kept variance + reconstruction error, so maximizing one minimizes the other.
10. The effective number of neighbors each point's Gaussian covers (its bandwidth).
11. It minimizes squared distance to centroids, favoring equal-variance spherical clusters; large clusters get split, small ones merged.
12. Otherwise the feature with the largest numeric variance (units) dominates PC1.
13. Its objective ignores placement of distant points, per-point bandwidths equalize densities, and the t-kernel exaggerates gaps.
14. Distances concentrate and noise dimensions dominate; "similar" loses meaning.
15. It keeps high-variance directions, which may not be the predictive ones.

16. Clarify purpose and actionability; engineer behavioral features; scale; try k-means/GMM on reduced features; validate stability and outcome differences; profile clusters.
17. Use k-means++ with many restarts, check stability across seeds/bootstraps, reconsider k or features, try a method matching the structure.
18. Distance to nearest centroid/neighbor, DBSCAN noise points, GMM likelihood; pitfalls: scaling, high dimensions, drifting "normal", threshold choice.
19. UMAP distances and separations aren't reliable evidence; test with stability, clustering in the original space, and whether groups differ in meaningful outcomes.
20. Standardize, PCA (keep ~95% variance or tune by validation), or feature selection; fit on train only; compare speed/accuracy trade-off.
21. $\max u^\top \Sigma u - \lambda(u^\top u - 1) \Rightarrow \Sigma u = \lambda u$.
22. Total 10: cumulative 50%, 80%, 90% → 3 components.
23. $\frac{d}{d\mu} \sum_{i \in C} \|x_i - \mu\|^2 = -2 \sum (x_i - \mu) = 0 \Rightarrow \mu$ = mean.
24. k-means is EM for a GMM with equal isotropic covariances in the zero-variance limit (hard assignments).
25. Its heavy tails let moderately distant points sit far apart in 2-D, relieving the crowding problem.
26. E: $r_{ik} \propto \pi_k \mathcal{N}(\mathbf{x}_i \mid \mu_k, \Sigma_k)$, normalized over k. M: $\pi_k = N_k/n$, μ_k and Σ_k = responsibility-weighted mean and covariance. The E-step makes the Jensen lower bound tight; the M-step raises the bound; so the log-likelihood is non-decreasing.
27. It clusters by graph connectivity: the Laplacian's smallest eigenvectors are nearly constant on each connected ring, so the rings become separate points in the embedding. k-means only uses distance to centroids and cuts both rings in half.
28. Multiplies the data by a random matrix to sketch its top range, orthonormalizes, then does a small exact SVD (plus optional power iterations). Needed for top-k PCA of very large or sparse matrices at roughly $O(ndk)$ cost.

A.9 Set 9 — Optimization & losses

Set 9

1. $m = \beta_1 m + (1-\beta_1)g$; $v = \beta_2 v + (1-\beta_2)g^2$; $\hat{m} = m/(1-\beta_1^t)$, $\hat{v} = v/(1-\beta_2^t)$; $\theta \leftarrow \theta - \eta \hat{m}/(\sqrt{\hat{v}} + \epsilon)$.
2. $\eta < 2/\lambda_{max}$ of the Hessian.
3. $\frac{1}{2}r^2$ if $|r| \le \delta$, else $\delta(|r| - \frac{1}{2}\delta)$; gradient = residual clipped to $[-\delta, \delta]$.
4. $-\alpha_t (1-p_t)^\gamma \log p_t$.
5. If $\|g\| > c$, rescale $g \leftarrow gc/\|g\|$ — same direction, capped size.
6. Oscillating components cancel in the running average; consistent components accumulate (effective step $\eta/(1-\beta)$).
7. Moment estimates start at zero and are biased toward zero early; dividing by $1-\beta^t$ corrects it.
8. L2 in Adam gets divided by $\sqrt{\hat{v}}$ (uneven regularization); AdamW decays weights directly and uniformly.
9. A minimum needs all Hessian eigenvalues positive; in high dimensions mixed signs are far more likely.
10. Pointwise: score each item independently; pairwise: order pairs correctly; listwise: optimize list quality (e.g. NDCG via LambdaRank).
11. Iterations needed scale with $\lambda_{max}/\lambda_{min}$: the LR is limited by the steepest direction while the flattest converges slowest.
12. Gradient noise keeps it bouncing in a region of size $\propto \eta\sigma^2/B$; only decaying the LR removes the noise floor.
13. Early Adam variance estimates and random-init gradients are unreliable/large; small early steps prevent destructive updates (esp. transformers, large batches).
14. The cross-entropy gradient is $p - y$; MSE through a sigmoid adds a $p(1-p)$ factor that vanishes when saturated.
15. It uses many negatives at once with soft weighting of hard negatives, avoiding triplet mining and collapse issues.
16. Lower the LR (LR range test), add warmup, check data scaling and outliers, gradient clipping, check the loss implementation.

17. LR too low/too high, bad init, dead units, too much regularization, a bug (labels, loss), too little capacity — test by overfitting a small batch.
18. Fewer updates per epoch and less gradient noise; scale LR (linear rule) with warmup, train longer, possibly LARS/LAMB.
19. Huber or MAE (or clean the data/clip outliers); quantile loss if promises need a percentile.
20. Focal loss (or class-balanced CE); afterwards recalibrate probabilities and tune the threshold.
21. $w_{t+1} = w_t - \eta\lambda w_t = (1-\eta\lambda)w_t \Rightarrow w_t = (1-\eta\lambda)^t w_0$; converges iff $|1-\eta\lambda| < 1 \Leftrightarrow 0 < \eta < 2/\lambda$.
22. $\partial|y-\hat{y}|/\partial\hat{y} = -\operatorname{sign}(y-\hat{y})$: magnitude doesn't shrink near the optimum, so fixed steps overshoot back and forth.
23. $0.1^2 = 0.01$ for $p_t = 0.9$; $0.9^2 = 0.81$ for $p_t = 0.1$.
24. SGD's implicit bias/noise can favor better-generalizing solutions on some vision problems; transformers have heterogeneous gradient scales that need adaptive steps — AdamW works better there.
25. The Hessian costs $O(d^2)$ memory and $O(d^3)$ to invert, mini-batch curvature is noisy, and Newton is attracted to saddle points.
26. SGD: roughly ×4 (linear scaling rule); Adam: roughly ×2 (square-root) as a starting point — re-tune either way. Warmup avoids early divergence while gradients and Adam's moment estimates are unreliable; beyond the critical batch size, bigger batches stop saving steps.
27. Lion updates with the sign of an interpolated momentum (every coordinate moves by the same size) and stores only momentum (half the memory). It needs a 3–10× smaller learning rate and a larger weight decay than AdamW.
28. Brier: $(0.99-0)^2 = 0.98$ (bounded). Log loss: $-\log 0.01 = 4.6$ (unbounded). Both are proper scoring rules; log loss punishes overconfidence far more.

A.10 Set 10 — Evaluation, validation, imbalance

Set 10

1. Precision $\frac{TP}{TP+FP}$; recall $\frac{TP}{TP+FN}$; specificity $\frac{TN}{TN+FP}$; F1 $\frac{2PR}{P+R}$.
2. The probability a random positive scores higher than a random negative.
3. The prevalence of the positive class.
4. $\sum_{i \le K} \frac{2^{rel_i}-1}{\log_2(i+1)}$.
5. Holdout (large data), K-fold (small/medium), stratified (imbalance), grouped (entities), time-series (temporal), nested (tuning + estimate).
6. TPR and FPR are each computed within one class.
7. Precision mixes in the base rate via Bayes' rule; recall is conditional on the positive class only.
8. Undefined/exploding near zero and asymmetric — it biases forecasts low.
9. Prevent leakage from label windows and lagged features overlapping the validation period.
10. Its training sets overlap almost completely, so fold errors are highly correlated.
11. It's dominated by the majority class and treats all errors equally.
12. The harmonic mean is dominated by the smaller value, so both must be high.
13. Otherwise duplicates or synthetic neighbors of validation examples leak into training.
14. It changes the effective class prior the model learns.
15. Exposure/position bias in offline labels, set-level effects (diversity), latency, dynamics and experiment flaws.
16. Precision (legit mail flagged is costly) at a very low FPR on ham, recall of spam, per-segment FPR; time-split evaluation; online rescue and report rates.
17. Report the time-split AUC (0.72) — it simulates deployment; the random split leaks the future/user identity.
18. MAPE rewards under-forecasting; check bias and use WAPE/quantile loss aligned with the service level.
19. Likely the second, if the operating point is what's used in production — confirm with CIs.
20. Stratified/time splits; natural distribution + class weights or threshold tuning; PR-AUC and recall at fixed FPR; bootstrap CIs (few positives); active learning for more positives.
21. 900 TP; FP $= 0.005 \times 999{,}000 \approx 4{,}995$; precision $\approx 900/5{,}895 \approx 0.15$.
22. DCG $= 3/1 + 0 + 7/2 = 6.5$; IDCG ([3,2,0]) $= 7 + 3/1.585 \approx 8.89$; NDCG@3 ≈ 0.73.

23. $p = \frac{0.5}{0.5+0.5/0.05} = \frac{0.5}{10.5} \approx 0.048$.
24. When comparing across populations with different base rates, or when both classes' ranking matters equally.
25. Rank by score and take the top 1,500 per day (a capacity threshold), recomputed daily or via a score quantile over a rolling window; monitor precision on reviewed cases; keep calibration so the threshold is interpretable.
26. Roughly [0.60, 0.89] — too wide to rank models. The number of positives (and negatives) drives AUC's uncertainty, not total rows.
27. Train a classifier to separate training rows from test/production rows; AUC 0.9 means strong shift. Inspect its top features to find what shifted, fix or drop them, validate on the most test-like training rows, or reweight by $P(\text{test} \mid \mathbf{x})/P(\text{train} \mid \mathbf{x})$.
28. Each bin's observed rate is a noisy estimate of its true rate, so $|\bar{y}_b - \bar{p}_b| > 0$ by chance; e.g. 5,000 calibrated predictions give ECE ≈ 0.02. Compare against this noise floor.

A.11 Set 11 — Feature engineering

Set 11

1. Standardization: $(x - \mu)/\sigma$ per column; min-max: $(x - \min)/(\max - \min)$ per column; row normalization: scale each example to unit norm.
2. $\text{TE} = \frac{n_c \bar{y}_c + m\bar{y}}{n_c + m}$.
3. MCAR: unrelated to anything; MAR: depends on observed features; MNAR: depends on the missing value itself.
4. So 23:00 and 00:00 are close; linear models/NNs see a circle rather than a line.
5. $\text{tf}(t, d) \times \log \frac{N}{\text{df}(t)}$.
6. Distance-based, regularized linear, gradient-trained models and PCA — scales change distances, penalties and conditioning.
7. Collisions and interpretability for bounded memory and no vocabulary.
8. Missingness may carry signal (MNAR); the indicator preserves it.
9. Axis-aligned splits approximate ratios/differences only with many splits.
10. Every feature uses only information available strictly before the prediction timestamp (including pipeline delays).
11. Otherwise each row's own label leaks into its feature.
12. $\exp(\mathbb{E}[\log y])$ is the geometric mean ≤ arithmetic mean (Jensen).
13. Convolutions learn hierarchical features from raw pixels; the architecture encodes the right inductive bias.
14. It imposes a fake order and equal spacing.
15. Otherwise selection uses validation data and the CV score is optimistic.
16. Training–serving skew: values differ by up to a day; compute consistently (same source/semantics) or train on logged serving values.
17. Content features/embeddings, category hierarchy, cold-start defaults, hashed IDs, learned embeddings updated frequently.
18. Group rare codes, use region hierarchy, smoothed out-of-fold target encoding, count encoding, geo features (distance, density).
19. Point-in-time correctness — tickets about cancellation after the snapshot are label leakage.
20. Add country × device crosses (or use a tree/FM model).
21. $\frac{4 \times 0.75 + 10 \times 0.1}{14} = \frac{4}{14} \approx 0.286$.
22. $1 - e^{-n^2/(2m)} = 1 - e^{-5} \approx 0.993$ — almost certain.
23. $z = 2.0$ in training; with the serving mean 110, $z \approx 1.33$ — the model sees a different input (skew).
24. Hashing: huge/open vocabularies, streaming, memory limits; embeddings: enough data per ID and semantic similarity matters.
25. Log served features; compare distributions (PSI/KS) and per-entity values against offline recomputation; alert on differences.
26. (1) The (PCA-reduced) vector itself; (2) similarity scores, e.g. cosine between query and item or user centroid and candidate; (3) neighbour label statistics (fraction of similar past cases with the label); also cluster IDs as categorical features.

27. Otherwise each row's own label (and those of near-duplicates) leaks into its feature, inflating validation performance.
28. Every embedding-based feature changes meaning: you must recompute features, re-index, retrain and re-validate — treat it as a versioned migration.

A.12 Set 12 — Deep learning

Set 12

1. On a mistake: $w \leftarrow w + y\,x$, $b \leftarrow b + y$.
2. $\sigma(1-\sigma)$; $1-\tanh^2$; $\mathbb{1}[z>0]$.
3. Xavier $2/(n_{in}+n_{out})$; He $2/n_{in}$.
4. Training: batch mean/variance per feature then γ, β; inference: running averages.
5. Zero units with probability p and scale survivors by $1/(1-p)$ in training; no-op at inference.
6. It guarantees representability by a wide one-hidden-layer network, not efficiency, learnability or generalization.
7. Units stuck in the negative region get zero gradient forever.
8. BN normalizes across the batch per feature (CNNs, large batches); LN across features per example (transformers, RNNs, small batches).
9. Deeper plain nets had higher training error; residuals make identity the default so depth doesn't hurt optimization.
10. Two matrix products per layer (weight gradient and error propagation) versus one in forward.
11. Otherwise the network is a single linear map.
12. All units in a layer stay identical (symmetry is never broken).
13. ReLU zeros about half the inputs, halving the second moment.
14. Normalized activations make the loss smoother and layers scale-invariant, so larger steps are stable.
15. Gradient products over many layers vanish/explode; conditioning differs across layers.
16. Whether the model is in eval mode (BN running stats, dropout off) and whether serving preprocessing matches.
17. Sigmoid saturation → vanishing gradients; use ReLU/GELU, He init, normalization, residuals.
18. Dead ReLUs from large updates/negative bias; lower LR, leaky ReLU/GELU, better init, check normalization.
19. LayerNorm or GroupNorm — independent of batch statistics.
20. Depth adds useful inductive bias (compositional features); the wide shallow net may memorize rather than share structure; or different effective regularization.
21. $\delta_2 = p - y$; $\partial L/\partial a_1 = w_2\delta_2$; $\delta_1 = (w_2\delta_2) \odot \mathbb{1}[z_1>0]$; $\partial L/\partial W_1 = \delta_1 x^\top$.
22. $\mathrm{Var}(\sum w_i x_i) = \sum \mathrm{Var}(w_i x_i) = \sum \mathbb{E}[w_i^2]\mathbb{E}[x_i^2] = n\,\mathrm{Var}(w)\,\mathrm{Var}(x)$ (zero means, independence).
23. $W_2(W_1 x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$.
24. Pre-LN keeps an unnormalized identity residual stream, so gradients flow cleanly and training is stable without long warmup.
25. The loss is a scalar; reverse mode gives all parameter gradients in one backward pass, forward mode needs one pass per parameter.
26. $\lfloor (32+2-3)/2 \rfloor + 1 = 16$ → 16×16. Three stacked 3×3 convs see 7×7 (with fewer parameters than one 7×7).
27. The cell update is additive, $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$; with the forget gate near 1, $\partial c_t/\partial c_{t-1} \approx 1$ — a gradient highway across time, like a residual connection.
28. Train on convex combinations of pairs of inputs and their labels; it smooths decision boundaries between classes, acts as a strong regularizer and usually improves calibration.

A.13 Set 13 — ML coding

Set 13

1. Softmax is shift-invariant; subtracting the max prevents overflow of `exp`.
2. $O(qnd)$.
3. $O(nkd)$.
4. It accumulates correctly when the same (true, pred) pair occurs multiple times; fancy-index `+=` doesn't.
5. Sort scores, compute average ranks, apply the Mann–Whitney formula.
6. So the effective step size doesn't depend on batch size and a smaller last batch isn't over-weighted.
7. $O(n)$ top-k selection instead of $O(n \log n)$ sorting.
8. Avoids `log(0)` and overflow; log-sum-exp is exact and stable.
9. After sorting, moving one sample from right to left changes class counts by one — impurity for every threshold in one sweep.
10. Avoids squaring the condition number; numerically more accurate.
11. Logistic regression is convex (any init reaches the optimum); a network's hidden units need different starting points to break symmetry.
12. The mean of an empty set is undefined; the cluster would stay dead.
13. Fitting on validation/test data leaks their statistics.
14. Floating-point cancellation can produce tiny negative values for identical points.
15. Fixed ordering creates correlated batches and cycles; shuffling keeps gradients unbiased.
16. Sigmoid overflow (`exp` of large negative), `log(0)` in the loss, unscaled features with a large LR, NaNs in input.
17. Chunk queries, use float32, ANN indexes (HNSW/IVF-PQ), or a KD-tree in low dimensions.
18. Check shapes, overfit a tiny batch, gradient-check, verify the loss on a known input, LR sweep, look at activations/gradient norms.
19. Mini-batch k-means, distributed assignment steps (map-reduce), sampling for init, ANN for assignment, fewer iterations.
20. Return 0 (or NaN) with a warning; conventions change averaged metrics and must be stated.
21. $\partial / \partial z_j$ of $-z_y + \log \sum e^z$ is $p_j - \mathbb{1}[j = y]$; average over $n \Rightarrow (P - Y)/n$.
22. $\frac{e^{z_k - c}}{\sum_j e^{z_j - c}} = \frac{e^{-c} e^{z_k}}{e^{-c} \sum e^{z_j}}$.
23. Rank-sum of positives minus its minimum possible value $n_+(n_+ + 1)/2$ counts positive–negative pairs correctly ordered; divide by $n_+ n_-$.
24. Welford: update count, mean $\mu \leftarrow \mu + (x - \mu)/n$, and $M_2 \leftarrow M_2 + (x - \mu_{old})(x - \mu_{new})$; the naive $\mathbb{E}[x^2] - \mathbb{E}[x]^2$ subtracts two large nearly-equal numbers.
25. Bin each feature once (≤256 bins), accumulate per-bin gradient/count histograms per node, scan bins: $O(n)$ per feature to build, $O(\text{bins})$ to scan; child histogram = parent − sibling.
26. Keep the first k; item t (0-indexed) replaces a random slot with probability $k/(t+1)$. By induction every item is present with probability k/t after t items: a survivor stays with probability $1 - \frac{k}{t+1} \cdot \frac{1}{k} = \frac{t}{t+1}$, giving $\frac{k}{t} \cdot \frac{t}{t+1} = \frac{k}{t+1}$.
27. Efraimidis–Spirakis: key u_i^{1/w_i} with $u_i \sim U(0,1)$; keep the k largest keys (a size-k heap for streams).
28. $\phi = (2, 3, 3)$: z_0's linear term gives 2; the interaction $3 \cdot 2 \cdot 1 = 6$ is split equally between z_1 and z_2; the sum 8 equals $f(x) - f(0)$.

A.14 Set 14 — Debugging & production

Set 14

1. Data → labels → features → split → pipeline → model → loss → optimization → evaluation → serving → monitoring.

2. Covariate: $P(x)$ changes; label: $P(y)$ changes; concept: $P(y \mid x)$ changes; prediction drift: output distribution changes.
3. Offline: historical, point-in-time values for training; online: latest values with low-latency lookup for serving.
4. Shadow: new model scores live traffic without affecting users; canary: a small share of users actually gets the new model.
5. Distribution shift between reference and current bins: <0.1 stable, 0.1–0.25 moderate, >0.25 large.
6. Serving-time features differ from training features due to implementation differences — a bug visible from day one, unlike drift in the world.
7. It isolates bugs in model/loss/optimizer from data problems cheaply.
8. Model artifact + preprocessing, data snapshot, code commit, config/hyperparameters, metrics, lineage, approval/stage.
9. Predictable entity sets, heavy models, no need for fresh context, cost efficiency.
10. Measure decay over time, weigh label delay and retraining cost/risk against value.
11. Real-world drift is gradual; step changes usually coincide with deploys or upstream data changes.
12. Metric mismatch, test not representative, skew, feedback/exposure bias, latency, experiment flaws.
13. Scalers, encoders, vocabularies must match what the model learned; otherwise inputs are silently wrong.
14. With huge samples, tiny irrelevant shifts are statistically significant; alert on effect size and model-relevant features.
15. The model's decisions shape future labels (selection/exposure bias), reinforcing its errors.
16. Check change point vs. deploys/schema changes; per-feature null/constant rates; model version; score distribution; replay yesterday's inputs; then real drift.
17. Confirm significance; compare feature availability/distributions by platform (logging differences), app-version pipeline differences, label rates; fix pipeline or add platform-aware features.
18. Training–serving skew or wrong artifact/preprocessing.
19. Input/prediction drift, early proxies (disputes, manual reviews), label-maturity cohorts, audit samples.
20. Log gradient norms/activations, check data for NaNs, lower LR/warmup, clipping, fp32/bf16, loss stability.
21. $(-0.1)\ln 0.8 + 0 + 0.1\ln 1.5 \approx 0.022 + 0.041 = 0.063$ → stable.
22. ~200 flagged/day on canary; comparing against a known 0.60 baseline needs ~750–800 labeled flags (~4 days), or ~1,500 per group for a two-sample test (~8 days) — plus label delay.
23. Loss grows linearly after retraining; average monthly cost $X/T + YT/2$ is minimized at $T^* = \sqrt{2X/Y}$ months.
24. Off-policy evaluation with logged propensities (IPS/doubly robust), exploration traffic, replay, then online tests.
25. Monitor data quality at pipeline stages with schema/null/volume checks and change-point alignment with deploys (bugs); compare feature drift vs. performance drift on fresh labels (concept drift); run parity checks against a golden dataset.
26. Model version behind the API alias; prompt/template changes; retrieval index freshness and config; traffic mix by intent/language — plus sampling parameters and guardrail strictness. Diff the configurations and replay the golden set on old vs. new.
27. It learned the template, not the task (too few, inconsistent or low-diversity labels), and catastrophic forgetting from over-training. Improve label quality/diversity, mix in general data, use LoRA/lower LR/fewer steps, and evaluate content and general skills.
28. Tokens per request (input vs. output), requests per user, cache hit rate, model mix, agent loop length/retries, and latency by stage.

A.15 Set 15 — System design, data-centric ML, model selection

Set 15

1. Retrieval (recall@K) → ranking (NDCG/log loss per head) → re-ranking (diversity/business metrics); online engagement/retention.
2. Business objective, ML objective, target, metrics, scale, data, labels, features, baseline, model, training, validation, serving, latency, scalability, monitoring, feedback, retraining, experimentation, failure handling.

3. Estimating the causal effect of an intervention per user; churn risk alone targets users who'd leave anyway or stay anyway.
4. $\tau = \frac{c_u}{c_u + c_o}$.
5. $\kappa = \frac{p_o - p_e}{1 - p_e}$.
6. pCTR multiplies bids; miscalibration misprices auctions and overcharges/undercharges.
7. Outcomes are only observed for approved applicants; inferring how rejected ones would have performed.
8. Top positions get more clicks regardless of relevance; use position features, click models, IPS with randomized propensities.
9. Sales are censored by stock-outs; training on them underestimates demand.
10. All pairs is $O(n^2)$ — infeasible; blocking/LSH/ANN proposes a small candidate set.
11. Labels define what's learned; errors, coverage gaps and noise set the ceiling that models can't exceed.
12. They cap measurable accuracy and can reverse model rankings.
13. It captures road-network physics and traffic well; ML learns residual corrections — data-efficient and robust.
14. Adversaries adapt, causing fast concept drift.
15. Lower cost, latency and maintenance risk; fewer failure modes; easier debugging.
16. Candidates via friends-of-friends/graph walks/embeddings → rank with GBM/deep model on mutual friends, interactions, affiliations → objective: accepted requests and meaningful interactions → guardrails: spam/harassment, privacy → feedback loops & cold start.
17. Measure per-dialect error rates; audit labels/guidelines; add dialect-aware training data; per-group thresholds reviewed with policy; human review for borderline cases.
18. PU learning: treat unlabeled as a mixture; estimate the labeling rate; or sample-and-review unlabeled data for negatives.
19. Fine-tune a small pretrained encoder or TF-IDF + linear model; distill; use unlabeled tickets for pretraining/self-training; measure latency.
20. Add explicit promotion features (discount depth, type, display), promo-period training emphasis, and check censoring.
21. $\tau = 9/(9+1) = 0.9$ → the 90th percentile.
22. $p_e = 0.9^2 + 0.1^2 = 0.82$; $\kappa = (0.9 - 0.82)/0.18 \approx 0.44$.
23. 0.7 — the ranker can only choose among retrieved items, so retrieval recall caps end-to-end quality.
24. Sample content views uniformly (or stratified) and have them labeled by reviewers, independent of the classifier's scores.
25. Biased offline labels, set-level effects/diversity, latency, interaction with blending, novelty or experiment flaws, CTR being the wrong metric.
26. Copying the generator's errors, reduced diversity (mode collapse), and contamination/terms-of-use issues; mitigate with verifiers/tests, diverse seeds from real data, human audits and evaluation on real held-out data.
27. Prompting ≈ 3.4B tokens ≈ $3,400/month; a fine-tuned small model ≈ 0.5B tokens, self-hosted for a few hundred dollars/month plus one-time labeling and training — worthwhile if accuracy matches and categories are stable.
28. The model can be tricked or can leak anything placed in its context; only documents the user may see should ever reach the prompt (and cached answers must not cross permission boundaries).

A.16 Set 16 — Transformers & language models

Set 16

1. $\mathrm{softmax}(QK^\top/\sqrt{d_k} + M)V$.
2. Query = what this token is looking for; key = what each token offers for matching; value = the content handed over if selected.
3. About $12d^2$: $4d^2$ for the Q, K, V and output projections plus $8d^2$ for a 4× MLP (plus small norm/bias terms).
4. exp of the average negative log-likelihood per token on held-out text — the effective number of equally likely choices per token.

5. Starts from characters/bytes and repeatedly merges the most frequent adjacent pair into a new symbol until the vocabulary size is reached; encoding applies merges in order.
6. Attention sums over tokens with content-based weights, so permuting inputs permutes outputs; adding (or rotating by) position information breaks the symmetry.
7. It rotates each 2-D slice of queries and keys by an angle proportional to position (different frequencies per slice); dot products then depend only on the position difference.
8. Keys and values of past tokens per layer and KV head; GQA shares each KV head across several query heads, dividing cache size by the group factor.
9. It changes memory traffic (tiling, online softmax, no $T \times T$ matrix in slow memory) — faster and $O(T)$ memory. It does not change the result (exact) or the $O(T^2)$ FLOPs.
10. Encoder-only: bidirectional, best for classification/embeddings. Decoder-only: causal, generation and general assistants. Encoder–decoder: input → output tasks like translation/summarization.
11. $q^\top k$ has variance d_k; unscaled logits saturate the softmax and kill gradients.
12. Different heads can attend to different relations (syntax, coreference, position) in separate subspaces at the same cost as one full-width head.
13. Each generated token reads all weights (and the KV cache) but performs only ~2 FLOPs per weight — low arithmetic intensity.
14. Tokens hide characters: "strawberry" may be 2–3 tokens, so the model never directly sees the letters it's asked to count.
15. Inference cost dominates over a model's lifetime; a smaller model trained on more tokens is cheaper to serve at similar quality.
16. Positions beyond training range (RoPE extrapolation), KV-cache memory (4×), prefill compute (quadratic attention). Options: rescale RoPE frequencies + long-context fine-tuning, GQA/KV quantization, FlashAttention, chunked prefill, or RAG instead.
17. A small encoder (fine-tuned): single forward pass, bidirectional context, tiny latency; a decoder LLM is too slow and costly.
18. Beam search favours high-likelihood, generic continuations; switch to sampling (temperature ~0.8–1, top-p ~0.9), add repetition penalties.
19. The tokenizer was trained mostly on English, so Hindi text needs more tokens per word; use a tokenizer/model with better Indic coverage, transliteration trade-offs, or fine-tune/extend the vocabulary; budget per-language costs.
20. LR too high/no warmup, bad data batches (very long or corrupt sequences), fp16 overflow, attention logit growth; fixes: warmup and lower LR, gradient clipping, bf16, data filtering, QK-norm, skipping bad batches, checkpoints to roll back.
21. $6 \times 10^9 \times 2 \times 10^{10} = 1.2 \times 10^{20}$ FLOPs.
22. For 2-D rotations $R_m^\top R_n = R_{n-m}$, so $\langle R_m q, R_n k \rangle = q^\top R_m^\top R_n k = q^\top R_{n-m} k$ — only the offset matters.
23. $T = 1$: $e^2, e^1, e^0 = 7.39, 2.72, 1$ → $(0.665, 0.245, 0.090)$. $T = 0.5$ (logits $4, 2, 0$): $54.6, 7.39, 1$ → $(0.867, 0.117, 0.016)$.
24. Keep a running max m, running sum ℓ and unnormalized output; when a new block raises the max, rescale previous sums and outputs by $e^{m_{\text{old}} - m_{\text{new}}}$ — the final normalization equals the full softmax exactly.
25. Memory: the same (all parameters must be resident). Per-token compute: much lower for the MoE — attention plus 2 of 8 experts (≈ a quarter of the MLP FLOPs) vs. all parameters for the dense model.

A.17 Set 17 — Adapting & aligning models

Set 17

1. $h = Wx + \frac{\alpha}{r} BAx$ with W frozen, $A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$ ($B = 0$ at start).
2. $P(y_w \succ y_l) = \sigma\big(r(x, y_w) - r(x, y_l)\big)$.
3. $\max_\pi \ \mathbb{E}_{y \sim \pi}[r(x, y)] - \beta \, \mathrm{KL}(\pi \| \pi_{\text{ref}})$.
4. $-\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)}\Big)$.
5. $A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$ within the group of samples for the same prompt.

6. Knowledge gap (facts, freshness, citations) → RAG. Behaviour gap (format, style, judgement, smaller model) → fine-tuning / preference optimization.
7. So the loss teaches producing responses, not reproducing user prompts; it focuses capacity on the answer.
8. The frozen base is stored in 4-bit (NF4, double quantization); only small LoRA adapters are trained in higher precision; paged optimizers absorb memory spikes.
9. The optimal policy's log-ratio to the reference equals the reward up to a prompt-only constant, which cancels in pairwise comparisons — so preferences can be fitted directly.
10. It can be computed automatically and reliably: unit tests pass, the numeric answer matches, the output validates against a schema.
11. It keeps the policy near the reference so it stays fluent and diverse and can't wander into regions where the reward model is wrong (reward hacking).
12. With the update scaled by α/r, changing the rank doesn't change the update's effective size much, so the learning rate transfers across ranks.
13. Every advantage is zero (each reward equals the mean), so the gradient is zero — nothing to learn from that prompt.
14. Gradients of the softened KL term scale as $1/T^2$; multiplying by T^2 keeps them comparable to the hard-label term.
15. Updating all weights on narrow data overwrites general abilities; limit it with LoRA, small LR, few steps, mixing general data, and regularization toward the original model.
16. Catastrophic forgetting (and perhaps format overfitting); mix general instruction data, use LoRA/lower LR/fewer epochs, and track general benchmarks alongside the task.
17. Reward hacking of a verbosity bias in the reward model; use length-controlled evaluation, length penalties or normalized rewards, a stronger KL, and preference pairs balanced for length.
18. Use a large model to label thousands more examples (distillation), audit them, fine-tune a small model (LoRA or a small encoder with an extraction head), consider constrained decoding for the schema, and evaluate field-level accuracy.
19. The verifier was weak/visible — reward hacking; add hidden and broader tests, penalize special-casing, and audit samples.
20. RAG — citations, freshness and permissions; fine-tune at most to teach the citation format.
21. $8 \times (4096 + 4096) = 65{,}536$ trainable vs. $16{,}777{,}216$ — about 0.4%.
22. $\mathbb{E}_\pi[r] - \beta \mathrm{KL}(\pi \| \pi_{\text{ref}}) = -\beta \,\mathrm{KL}\big(\pi \| \frac{1}{Z}\pi_{\text{ref}} e^{r/\beta}\big) + \beta \log Z$; the KL is minimized (zero) at $\pi^* = \frac{1}{Z}\pi_{\text{ref}} e^{r/\beta}$.
23. Mean 1, std (population) $\sqrt{0.5} \approx 0.707$ → $(1.414, -1.414, 0, 0)$.
24. Not necessarily: DPO optimizes the *margin*, so both can fall if the rejected one falls faster; probability mass moves to other (possibly unseen) responses. It becomes a problem if the chosen likelihood collapses — monitor it, add an SFT term on chosen answers, tune β.
25. Held-out human (or independently validated) evaluations, length-controlled comparisons, sycophancy and refusal probes, and watching for divergence between reward-model score and independent quality measures.

A.18 Set 18 — Retrieval, RAG & agents

Set 18

1. Offline: parse → chunk → embed → index (with metadata/ACLs). Online: rewrite query → hybrid retrieve → fuse → permission filter → rerank → assemble context → generate with citations → guardrails → log.
2. $\mathrm{IDF}(t) \cdot \frac{f(t,d)(k_1+1)}{f(t,d)+k_1(1-b+b\|d\|/\mathrm{avgdl})}$.
3. $\mathrm{RRF}(d) = \sum_r \frac{1}{k+\mathrm{rank}_r(d)}$, $k \approx 60$.
4. A transformer that reads query and document together and outputs a relevance score — accurate, but one forward pass per pair.
5. Flat (exact), HNSW (graph), IVF (clustered), PQ / IVF-PQ (compressed codes).
6. Failures happen in two independent places — search and reading — each needing its own metric and fix.
7. Whether every claim in the answer is supported by the retrieved context.

8. Dense retrieval handles paraphrase; BM25 handles exact terms (IDs, codes, names); fusion covers both.
9. Instructions hidden in content the system reads (documents, web pages, emails, tool outputs) that the model follows.
10. Split each vector into m sub-vectors and replace each with the ID of its nearest codebook centroid (usually 1 byte); distances come from lookup tables.
11. Retrieval gives fresh, citable, permission-filtered knowledge with cheap updates; fine-tuning is unreliable for injecting facts.
12. Cross-encoders can't precompute document representations — a forward pass per pair is far too slow over millions of documents.
13. Success probabilities multiply: p^n.
14. Anything in the context can leak into the answer; post-hoc filtering is unreliable, and cached answers can cross users.
15. Cost and latency grow with tokens; details in long contexts are recalled unevenly; permissions, freshness and citations still need selection.
16. Add BM25/hybrid retrieval and exact-match fields for codes; boost exact matches.
17. Version-aware indexing, deduplicate versions, recency boost, drop superseded documents.
18. Conversational query rewriting using the chat history before retrieval.
19. Least privilege: remove or gate the delete tool, require confirmation for destructive actions, sandbox file access, audit logs.
20. Generation: noisy/long context, poor ordering, the model ignoring context, or wrong answers despite correct retrieval — rerank and trim, improve grounding prompts, check faithfulness, try a stronger reader.
21. $5 \times 10^7 \times 1024 \times 4$ B ≈ 205 GB; PQ at 128 B → 6.4 GB.
22. $0.97^{12} \approx 0.69$.
23. X: $\frac{1}{61} + \frac{1}{70} = 0.0307$; Y: $\frac{2}{63} = 0.0317$ → Y ranks higher (consistency beats one top rank).
24. Extract claims, check each for entailment against the retrieved passages (NLI or a judge), validate against human labels (κ) per slice, and sanity-test by removing the supporting passage (verdict should flip).
25. Treat email content as untrusted data; no sending to new/external recipients without approval; strip links/images that could exfiltrate; secrets never in context; separate reading from sending; rate limits and monitoring.

A.19 Set 19 — Evaluating LLM systems & safety

Set 19

1. $\text{pass@}k = 1 - \binom{n-c}{k} / \binom{n}{k}$, averaged over problems.
2. $\kappa = \frac{p_o - p_e}{1 - p_e}$.
3. Gaming precision with very short outputs that contain only safe n-grams.
4. Position, verbosity, self-preference, style over substance (also leniency and drift).
5. Factuality: agrees with the world. Faithfulness: agrees with the provided source.
6. Relative judgements are more consistent across raters and time than absolute scores.
7. It inflates scores (memory, not ability). Detect via n-gram overlap, canaries, rephrased/new items, and items created after the data cutoff.
8. The per-question difference has much lower variance because both models share question difficulty; only disagreements carry information.
9. Harmful compliance rate and false refusal (over-refusal) rate.
10. 10× odds: the stronger model wins about $10/11 \approx 91\%$ of comparisons.
11. They measure overlap with a reference, not correctness; many valid answers exist and a negation can keep nearly all n-grams.
12. It assumes sampling with replacement / plugs in a noisy estimate of the per-sample pass rate; the combinatorial estimator is unbiased.
13. Otherwise score changes may come from judge changes rather than product changes.
14. Abstaining scores zero; guessing has positive expected score — so guessing is rewarded.

15. It prevents regressions and makes the eval set grow from real failures.
16. Judge–user mismatch: rubric or judge biases (e.g. verbosity), slices where users suffer, calibration drift. Compare judge scores with human ratings on complaint samples; fix the rubric.
17. Length-controlled comparisons, rubric that penalizes padding, swapping order, comparing equal-length pairs, calibrating against humans.
18. A north-star task-success metric (e.g. resolved without escalation) with guardrails (safety violations, cost, latency, CSAT) — a single number alone invites gaming.
19. Assess severity and reproducibility, add mitigations (classifier, prompt/system changes, capability limits), add the attack to regression tests, decide launch vs. delay by risk, monitor after launch.
20. Evaluate on your own eval set with paired comparisons and slices, plus safety, cost, latency and migration effort; pilot before switching.
21. SE $= \sqrt{0.8 \cdot 0.2/400} = 0.02$; 95% CI $\approx [0.761, 0.839]$.
22. $c/n = 0.4$.
23. $p_o = 0.9$, $p_e = 0.5 \cdot 0.5 + 0.5 \cdot 0.5 = 0.5 \to \kappa = 0.8$.
24. Pairwise expert preferences with multiple raters, aspect rubrics (brand fit, clarity, persuasiveness), agreement measures, and online outcomes (CTR, conversions) as the final arbiter.
25. End-to-end task success on realistic scenarios, trajectory quality (unnecessary steps, tool errors), per-step checks, cost/latency per task and safety violations — in simulated environments and replayed traces.

A.20 Set 20 — Efficient training & inference

Set 20

1. Training $\approx 6ND$; inference $\approx 2N$ per token.
2. bf16 weights 2 + bf16 grads 2 + fp32 master 4 + Adam m 4 + Adam v 4 = 16 bytes.
3. $2\, n_{\text{layers}}\, n_{\text{kv}}\, d_{\text{head}}\, T\, b$ bytes.
4. Time to first token; time per output token (inter-token latency).
5. bf16: more range (8-bit exponent). fp16: more precision (10-bit mantissa).
6. Decode does ~2 FLOPs per weight byte read (matrix-vector); prefill reuses each weight across many tokens (matrix-matrix).
7. ZeRO-1: optimizer states; ZeRO-2: + gradients; ZeRO-3: + parameters.
8. KV-cache fragmentation and over-reservation — block-based allocation packs more sequences and enables prefix sharing.
9. It cuts the bytes read per generated token, and decode is bandwidth-bound.
10. Scheduling at every decode step: finished sequences leave and new ones join the batch.
11. fp16's narrow range underflows small gradients (and overflows large ones); bf16 has fp32's exponent range.
12. It all-reduces inside every layer, so it needs the fastest interconnect (within a node).
13. Acceptance probability $\min(1, p/q)$ plus resampling from the normalized residual $\max(0, p - q)$ yields exactly p.
14. Output tokens are generated sequentially in memory-bound decode; input tokens are processed in parallel during prefill.
15. A few channels with huge values force a large shared scale, wiping out precision for all the small values.
16. $16 \times 13\text{B} \approx 208$ GB before activations. Use ZeRO-3/FSDP across 8 GPUs, activation checkpointing, smaller micro-batches with gradient accumulation, bf16; else LoRA/QLoRA or CPU offload.
17. Tiny batches or static batching, memory-bound decode, CPU-side bottlenecks (tokenization, Python), I/O — fix with continuous batching, larger effective batches and profiling.
18. Prefix caching: compute the system prompt's KV once and reuse it (keep it identical and at the start).
19. Chunked prefill or separate pools (prefill/decode disaggregation), priority scheduling.
20. Break down tokens and cost; trim prompts/context; cache prefixes and answers; route easy requests to small models; cap outputs; batch offline; distil; quantize — verify each on the eval set.
21. $6 \times 10^9 \times 10^{11} = 6 \times 10^{20}$ FLOPs; $6 \times 10^{20}/4 \times 10^{14} = 1.5 \times 10^6$ s $\approx$ 417 GPU-hours.
22. $2 \cdot 40 \cdot 8 \cdot 128 \cdot 32{,}768 \cdot 2 \approx 5.4 \times 10^9$ B $\approx$ 5.0 GiB.

23. 70B at 4 bits ≈ 35 GB → $3.35 \times 10^{12} / 3.5 \times 10^{10} \approx 96$ tokens/s.
24. Token i of the draft is used only if all previous ones were accepted: probability α^i; one more token always comes from the target (correction or bonus). $\mathbb{E} = \sum_{i=0}^{\gamma} \alpha^i = \frac{1-\alpha^{\gamma+1}}{1-\alpha}$ (assuming independent acceptances).
25. Compare on your own eval (quality overall and on sensitive slices), latency (TTFT/TPOT), throughput and memory at your batch sizes, and cost; larger-quantized often wins at equal memory, but maths/long-context slices can suffer.

A.21 Set 21 — Recommender systems & bandits

Set 21

1. Candidate generation (recall, cheap, millions → thousands), ranking (precision, rich features), re-ranking (diversity, freshness, rules, exploration).
2. $\min \sum_{\text{obs}} (r_{ui} - p_u^\top q_i)^2 + \lambda(\|P\|^2 + \|Q\|^2)$.
3. $\max \sum \log \sigma(s_{ui} - s_{uj}) - \lambda\|\Theta\|^2$ over observed i and sampled unobserved j.
4. $\text{DCG@}k = \sum_{r \le k} \frac{2^{rel_r} - 1}{\log_2(r+1)}$; nDCG = DCG / ideal DCG.
5. $a_t = \arg\max_a \bar{x}_a + \sqrt{2 \ln t / n_a}$.
6. The user may never have been exposed to the item.
7. With one factor fixed, each user (item) vector is a ridge regression — convex with a closed form.
8. In-batch negatives over-sample popular items, over-penalizing them; subtracting $\log q_i$ corrects the sampled-softmax bias.
9. Higher positions get more clicks regardless of relevance; estimate with randomization (swaps, interleaving) or intervention harvesting.
10. The logging propensity of each shown action and overlap (nonzero probability for actions the new policy takes).
11. Random splits leak future behaviour and popularity into training.
12. Ranking the true item among a few random negatives can change which model looks best compared with full-catalogue ranking.
13. It explores at a constant rate forever, paying constant regret per step.
14. Objectives share representations; one model predicts click, watch, like, hide jointly; expert/gating structures (MMoE, PLE) limit negative transfer.
15. Shown items collect more interactions, get retrained as better, and are shown more — rich get richer.
16. Content features in the item tower, exploration budget, semantic IDs, similar-item bootstrapping.
17. MMR or diversity-aware re-ranking, dedupe near-duplicates, per-creator caps.
18. The model optimized a short-term proxy (clicks, clickbait); check long-term metrics, satisfaction and diversity; change labels/objective weights.
19. Off-policy evaluation with logged propensities (IPS/SNIPS/doubly robust), or offline replay if logs were randomized.
20. Exposure-fairness constraints or boosts in re-ranking, exploration slots for new sellers, cold-start features, monitoring exposure distribution.
21. $\text{DCG} = 0 + \frac{7}{\log_2 3} + \frac{1}{\log_2 4} = 4.417 + 0.5 = 4.917$; $\text{IDCG}([3,1,0]) = 7 + 0.631 = 7.631$; nDCG ≈ 0.644.
22. Weight $0.5/0.1 = 5$, so the sample contributes $5 \times 1 = 5$ to the IPS sum (before dividing by n).
23. A: $0.6 + \sqrt{2 \ln 13/3} = 1.91$; B: $0.7 + \sqrt{2 \ln 13/10} = 1.42$ → UCB1 picks A.
24. Two towers with strong content embeddings (text/image) plus ID embeddings with dropout; in-batch softmax with logQ; mixed random and hard negatives; exploration for new items; evaluate recall on new-item slices; consider semantic IDs.
25. Predict each component well offline, then tune weights with online experiments against long-term metrics (or bandit/Bayesian optimization over weights, Pareto analysis), with guardrail metrics.

A.22 Set 22 — Responsible AI & other frontiers

Set 22

1. Demographic parity: equal selection rates. Equal opportunity: equal TPR. Equalized odds: equal TPR and FPR.
2. $\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F|-|S|-1)!}{|F|!}[v(S \cup \{j\}) - v(S)]$.
3. $P(M(D) \in O) \leq e^{\varepsilon} P(M(D') \in O) + \delta$ for all neighbouring D, D' and all O.
4. $\log p(x) \geq \mathbb{E}_q[\log p(x \mid z)] - \mathrm{KL}(q(z \mid x) \| p(z))$.
5. $V^*(s) = \max_a [R(s,a) + \gamma \sum_{s'} P(s' \mid s, a) V^*(s')]$.
6. With different base rates and an imperfect classifier, you can't have calibration within groups and equal FPR and FNR at once.
7. Attributions sum exactly to $f(x) - f(x')$.
8. Clipping bounds each example's gradient norm (sensitivity); Gaussian noise scaled to that bound hides any single example.
9. Repeated neighbourhood averaging makes node representations converge, so nodes become indistinguishable as depth grows.
10. $\tilde{\epsilon} = \epsilon(x_t, \varnothing) + w\,(\epsilon(x_t, c) - \epsilon(x_t, \varnothing))$ — trades diversity for prompt adherence.
11. Proxies reconstruct it, and without it you can't measure disparities.
12. They describe how the model uses features relative to a baseline; real-world interventions change correlated features too.
13. MASE is scale-free, defined with zeros, and benchmarks against naive; MAPE explodes near zero and is asymmetric.
14. Sampling isn't differentiable; $z = \mu + \sigma \odot \epsilon$ moves randomness into ϵ so gradients reach μ, σ.
15. $\mathbb{E}[\nabla \log \pi(a \mid s)\, b(s)] = b(s) \sum_a \nabla \pi(a \mid s) = b(s) \nabla 1 = 0$; a baseline near the expected return shrinks variance.
16. Report per-group FNR/TPR with confidence intervals, investigate causes (labels, features, representation), consider equal-opportunity constraints within legal limits, involve stakeholders, document.
17. Memorized training data: treat it as an incident, add output filters/PII redaction, remove and deduplicate the data, consider retraining or unlearning, and use scrubbing/DP for future training.
18. Leakage from future information (random K-fold, features not available at forecast time) or a regime change; use walk-forward validation with point-in-time features.
19. A GNN over accounts and transactions (node or link classification) exploits ring structure; pitfalls: label leakage through edges, heterophily, scale (sampling).
20. An uplift model (T/X-learner or causal forest) trained on randomized data, evaluated with Qini curves and a holdout.
21. Scale $b = \Delta/\varepsilon = 10$; SD $= \sqrt{2}\, b \approx 14.1$.
22. $\frac{1}{2}(1 + 1 - 1 - 0) = 0.5$.
23. $\hat{\pi} = (0.4 - 0.25)/0.5 = 0.3$.
24. When base rates differ; show the trade-off curves (accuracy vs. each fairness metric) with numbers and let stakeholders choose, documenting the decision.
25. PPO: clipped policy-gradient with a learned value baseline — the classic RLHF optimizer with a reward model. GRPO: drops the value network and uses group-relative baselines, typically with verifiable rewards; both are policy-gradient methods with a regularizer to the reference.

When you've finished the key

Every "~" and "✗" is a pointer back into the notebook. Re-read that section, then say it out loud again in 48 hours. When the recall sets feel easy, take the Final Exam — and bring your answers back to the chat for grading.